

Display 10.12 Some Methods in the Class `File` (part 2 of 3)

```
public boolean canRead()
```

Tests whether the program can read from the file. Returns `true` if the file named by the abstract path name exists and is readable by the program; otherwise returns `false`.

```
public boolean setReadOnly()
```

Sets the file represented by the abstract path name to be read only. Returns `true` if successful; otherwise returns `false`.

```
public boolean canWrite()
```

Tests whether the program can write to the file. Returns `true` if the file named by the abstract path name exists and is writable by the program; otherwise returns `false`.

```
public boolean delete()
```

Tries to delete the file or directory named by the abstract path name. A directory must be empty to be removed. Returns `true` if it was able to delete the file or directory. Returns `false` if it was unable to delete the file or directory.

```
public boolean createNewFile() throws IOException
```

Creates a new empty file named by the abstract path name, provided that a file of that name does not already exist. Returns `true` if successful, and returns `false` otherwise.

```
public String getName()
```

Returns the last name in the abstract path name (that is, the simple file name). Returns the empty string if the abstract path name is the empty string.

```
public String getPath()
```

Returns the abstract path name as a `String` value.

```
public boolean renameTo(File New_Name)
```

Renames the file represented by the abstract path name to *New_Name*. Returns `true` if successful; otherwise returns `false`. *New_Name* can be a relative or absolute path name. This may require moving the file. Whether or not the file can be moved is system dependent.

```
public boolean isFile()
```

Returns `true` if a file exists that is named by the abstract path name and the file is a normal file; otherwise returns `false`. The meaning of *normal* is system dependent. Any file created by a Java program is guaranteed to be normal.

```
public boolean isDirectory()
```

Returns `true` if a directory (folder) exists that is named by the abstract path name; otherwise returns `false`.

```
public boolean mkdir()
```

Makes a directory named by the abstract path name. Will not create parent directories. See `mkdirs`, which follows. Returns `true` if successful; otherwise returns `false`.

Display 10.12 Some Methods in the Class `File` (part 3 of 3)

```
public boolean mkdirs()
```

Makes a directory named by the abstract path name. Will create any necessary but nonexistent parent directories. Returns `true` if successful; otherwise returns `false`. Note that if it fails, then some of the parent directories may have been created.

```
public long length()
```

Returns the length in bytes of the file named by the abstract path name. If the file does not exist or the abstract path name designates a directory, then the value returned is not specified and may be anything.

Self-Test Exercises

26. Write a complete (although simple) Java program that tests whether or not the directory (folder) containing the program also contains a file named `Sally.txt`. The program has no input, and the only output tells whether or not there is a file named `Sally.txt`.
27. Write a complete Java program that asks the user for a file name, tests whether the file exists, and, if the file exists, asks the user whether or not it should be deleted. It then either deletes or does not delete the file as the user requests.

10.4 Binary Files ★

A little more than kin, and less than kind.

WILLIAM SHAKESPEARE, *Hamlet*, 1603.

Binary files store data in the same format that is used in the computer's memory to store the values of variables. For example, a value of type `int` is stored as the same sequence of bytes (same sequence of zeros and ones) whether it is stored in an `int` variable in memory or in a binary file. So, no conversion of any kind needs to be performed when you store or retrieve a value in a binary file. This is why binary files can be handled more efficiently than text files.

Java binary files are unlike binary files in other programming languages in that they are portable. A binary file created by a Java program can be moved from one computer to another and still be read by a Java program—but only by a Java program. They cannot normally be read with a text editor or with a program written in any programming language other than Java.

The preferred stream classes for processing binary files are `ObjectInputStream` and `ObjectOutputStream`. Each has methods to read or write data one byte at a time. These streams can also automatically convert numbers and characters to bytes that can be stored in a binary file. They allow your program to be written as if the data placed in

the file, or read from the file, is not just bytes but also strings or items of any of Java's primitive data types, such as `int`, `char`, and `double`, or even objects of classes you define. If you do not need to access your files using an editor, then the easiest and most efficient way to read data from and write data to files is to use binary files in the way we describe here.

We conclude this section with a discussion of how you can use `ObjectOutputStream` and `ObjectInputStream` to write and later read objects of any class you define. This will let you store objects of the classes you define in binary files and later read them back, all with the same convenience and efficiency that you get when storing strings and primitive type data in binary files.

Writing Simple Data to a Binary File

The class `ObjectOutputStream` is the preferred stream class for writing to a binary file.² An object of the class `ObjectOutputStream` has methods to write strings and values of any of the primitive types to a binary file. Display 10.13 shows a sample program that writes values of type `int` to a binary file. Display 10.14 describes the methods used for writing data of other types to a binary file.

Display 10.13 Writing to a Binary File (part 1 of 2)

```
1  import java.io.ObjectOutputStream;
2  import java.io.FileOutputStream;
3  import java.io.IOException;

4  public class BinaryOutputDemo
5  {
6      public static void main(String[] args)
7      {
8          try
9          {
10             ObjectOutputStream outputStream =
11                 new ObjectOutputStream(
12                     new FileOutputStream("numbers.dat"));

13             int i;
14             for (i = 0; i < 5; i++)
15                 outputStream.writeInt(i);

16             System.out.println("Numbers written to the file numbers.dat.");
17             outputStream.close();
18         }
19     }
20 }
```

²`DataOutputStream` is also widely used and behaves exactly as we describe for `ObjectOutputStream` in this section. However, the techniques given in the subsections “Binary I/O of Objects” and “Array Objects in Binary Files” work only for `ObjectOutputStream`; they do not work for `DataOutputStream`.

Display 10.13 Writing to a Binary File (part 2 of 2)

```

18     catch (IOException e)
19     {
20         System.out.println("Problem with file output.");
21     }
22 }
23 }

```

FILE REPRESENTATION (after program is run)

0	<i>This is a binary file. It really contains representations of each number as bytes—that is, zeros and ones—and is read as bytes. You cannot read this file with your text editor.</i>
1	
2	
3	
4	

Display 10.14 Some Methods in the Class `ObjectOutputStream` (part 1 of 2)

`ObjectOutputStream` and `FileOutputStream` are in the `java.io` package.

```
public ObjectOutputStream(OutputStream streamObject)
```

There is no constructor that takes a file name as an argument. If you want to create a stream using a file name, use

```
new ObjectOutputStream(new FileOutputStream(File_Name))
```

This creates a blank file. If there already is a file named *File_Name*, then the old contents of the file are lost.

If you want to create a stream using an object of the class `File`, use

```
new ObjectOutputStream(new FileOutputStream(File_Object))
```

The constructor for `FileOutputStream` may throw a `FileNotFoundException`, which is a kind of `IOException`. If the `FileOutputStream` constructor succeeds, then the constructor for `ObjectOutputStream` may throw a different `IOException`.

```
public void writeInt(int n) throws IOException
```

Writes the `int` value `n` to the output stream.

```
public void writeShort(short n) throws IOException
```

Writes the `short` value `n` to the output stream.

```
public void writeLong(long n) throws IOException
```

Writes the `long` value `n` to the output stream.

```
public void writeDouble(double x) throws IOException
```

Writes the `double` value `x` to the output stream.

(continued)

Display 10.14 Some Methods in the Class `ObjectOutputStream` (part 2 of 2)

```
public void writeFloat(float x) throws IOException
```

Writes the float value `x` to the output stream.

```
public void writeChar(int n) throws IOException
```

Writes the char value `n` to the output stream. Note that it expects its argument to be an `int` value. However, if you simply use the `char` value, then Java will automatically type cast it to an `int` value. The following are equivalent:

```
outputStream.writeChar((int) 'A');
```

and

```
outputStream.writeChar('A');
```

```
public void writeUTF(String aString) throws IOException
```

Writes the `String` value `aString` to the output stream. UTF refers to a particular method of encoding the string. To read the string back from the file, you should use the method `readUTF` of the class `ObjectInputStream`.

```
public void writeObject(Object anObject) throws IOException
```

Writes its argument to the output stream. The object argument should be an object of a serializable class, a concept discussed later in the section titled “The `Serializable` Interface.” Throws various `IOExceptions`.

```
public void close() throws IOException
```

Closes the stream’s connection to a file. This method calls `flush` before closing the file.

```
public void flush() throws IOException
```

Flushes the output stream. This forces an actual physical write to the file of any data that has been buffered and not yet physically written to the file. Normally, you should not need to invoke `flush`.

Notice that almost all the code in the sample program in Display 10.13 is in a `try` block. Any part of the code that does binary file I/O in the ways we are describing can throw an `IOException`.

The output stream for writing to the binary file `numbers.dat` is created and named with the following:

```
ObjectOutputStream outputStream =  
    new ObjectOutputStream(new  
        FileOutputStream("numbers.dat"));
```

opening a file

As with text files, this is called **opening the file**. If the file `numbers.dat` does not already exist, this statement will create an empty file named `numbers.dat`. If the file `numbers.dat` already exists, this statement will erase the contents of the file so that the file starts out empty. The situation is basically the same as what you learned for text files, except that we are using a different class.

As is typical of Java I/O classes, the constructor for the class `ObjectOutputStream` takes another I/O class object as an argument—in this case, an anonymous argument of the class `FileOutputStream`.

Opening a Binary File for Output

You create a stream of the class `ObjectOutputStream` and connect it to a binary file as follows:

SYNTAX

```
ObjectOutputStream Output_Stream_Name =  
    new ObjectOutputStream(new FileOutputStream(File_Name));
```

The constructor for `FileOutputStream` may throw a `FileNotFoundException`, which is a kind of `IOException`. If the `FileOutputStream` constructor succeeds, then the constructor for `ObjectOutputStream` may throw a different `IOException`. A single catch block for `IOException` would cover all cases.

EXAMPLES

```
ObjectOutputStream myOutputStream =  
    new ObjectOutputStream(new  
        FileOutputStream("mydata.dat"));
```

After opening the file, you can use the methods of the class `ObjectOutputStream` (Display 10.14) to write to the file.

`writeInt`

The class `ObjectOutputStream` does not have a method named `println`, as we had with text file output and screen output. However, as shown in Display 10.13, an object of the class `ObjectOutputStream` does have a method named `writeInt` that can write a single `int` value to a file, and it also has the other output methods described in Display 10.14.

In Display 10.13, we made it look as though the numbers in the file `numbers.dat` were written one per line in a human-readable form. That is not what happens, however. There are no lines or other separators between the numbers. Instead, the numbers are written in the file one immediately after the other, and they are encoded as a sequence of bytes in the same way that the numbers would be encoded in the computer's main memory. These coded `int` values cannot be read using your editor. Realistically, they can be read only by another Java program.

You can use a stream from the class `ObjectOutputStream` to output values of any primitive type and also to write data of the type `String`. Each primitive data type has a corresponding write method in the class `ObjectOutputStream`. We have already mentioned the write methods for outputting `int` values. The methods for the other primitive types are completely analogous to `writeInt`. For example, the following would write a double value, a boolean value, and a char value to the binary file connected to the `ObjectOutputStream` object `outputStream`:

```
outputStream.writeDouble(9.99);  
outputStream.writeBoolean(false);  
outputStream.writeChar((int)'A');
```

writeChar

The method `writeChar` has one possibly surprising property: It expects its argument to be of type `int`. So if you start with a value of type `char`, the `char` value can be type cast to an `int` before it is given to the method `writeChar`. For example, to output the contents of a `char` variable named `symbol`, you can use

```
outputStream.writeChar((int) symbol);
```

In actual fact, you do not need to write in the type cast to an `int`, because Java automatically performs a type cast from a `char` value to an `int` value for you. So, the following is equivalent to the previous invocation of `writeChar`:

```
outputStream.writeChar(symbol);
```

**writeUTF
for strings**

To output a value of type `String`, use the method `writeUTF`. For example, if `outputStream` is a stream of type `ObjectOutputStream`, the following will write the string "Hello friend." to the file connected to that stream:

```
outputStream.writeUTF("Hello friend.");
```

You may write output of different types to the same file. So, you may write a combination of, for example, `int`, `double`, and `String` values. However, mixing types in a file does require special care to make it possible to read them back out of the file. To read them back, you need to know the order in which the various types appear in the file, because, as you will see, a program that reads from the file will use a different method to read data of each different type.

**closing a
binary file**

Note that, as illustrated in Display 10.13 and as you will see shortly, you close a binary output or input stream in the same way that you close a stream connected to a text file.

UTF and writeUTF

Recall that Java uses the Unicode character set, which is a set of characters that includes many characters used in languages whose character sets are different from English. Readers of this book are undoubtedly using editors and operating systems that use the ASCII character set, which is the character set normally used for English and for our Java programs. The ASCII character set is a subset of the Unicode character set, so the Unicode character set has a lot of characters you probably do not need. There is a standard way of encoding all the Unicode characters, but for English-speaking countries, it is not a very efficient coding scheme. The UTF coding scheme is an alternative scheme that still codes all Unicode characters, but that favors the ASCII character set. The UTF coding method gives short, efficient codes for the ASCII characters. The price is that it gives long, inefficient codes to the other Unicode characters. However, because you probably do not use the other Unicode characters, this is a very favorable trade-off. The method `writeUTF` uses the UTF coding method to write strings to a binary file.

The method `writeInt` writes integers into a file using the same number of bytes—that is, the same number of zeros and ones—to store any integer. Similarly, the method `writeLong` uses the same number of bytes to store each value of type `long`. (But the methods `writeInt` and `writeLong` use a different number of bytes

from each other.) The situation is the same for all the other write methods that write primitive types to binary files. However, the method `writeUTF` uses differing numbers of bytes to store different strings in a file. Longer strings require more bytes than shorter strings. This can present a problem to Java, because there are no separators between data items in a binary file. The way that Java manages to make this work is by writing some extra information at the start of each string. This extra information tells how many bytes are used to write the string, so `readUTF` knows how many bytes to read and convert. (The method `readUTF` will be discussed a little later in this chapter, but, as you may have already guessed, it reads a `String` value that was written using the UTF coding method.)

The situation with `writeUTF` is even a little more complicated than what we discussed in the previous paragraph. Notice that we said that the information at the start of the string code in the file tells how many *bytes* to read, *not how many characters are in the string*. These two figures are not the same. With the UTF way of encoding, different characters are encoded in different numbers of bytes. However, all the ASCII characters are stored in just one byte, and you are undoubtedly using only ASCII characters, so this difference is more theoretical than real to you now.

Reading Simple Data from a Binary File

The stream class `ObjectInputStream` is used to read from a file that has been written to using `ObjectOutputStream`. Display 10.15 gives some of the most commonly used methods for this class. If you compare that table with the methods for `ObjectOutputStream` given in Display 10.14, you will see that each output method in `ObjectOutputStream` has a corresponding input method in `ObjectInputStream`. For example, if you write an integer to a file using the method `writeInt` of `ObjectOutputStream`, then you can read that integer back with the method `readInt` of `ObjectInputStream`. If you write a number to a file using the method `writeDouble` of `ObjectOutputStream`, then you can read that number back with the method `readDouble` of `ObjectInputStream`, and so forth. Display 10.16 gives an example of using `readInt` in this way.

Self-Test Exercises

28. How do you open the binary file `bindata.dat` so that it is connected to an output stream of type `ObjectOutputStream` that is named `outputThisWay`?
29. Give two statements that will write the values of the two double variables `v1` and `v2` to the file `bindata.dat`. Use the stream `outputThisWay` that you created as the answer to Self-Test Exercise 28.
30. Give a statement that will write the string value `"Hello"` to the file `bindata.dat`. Use the stream `outputThisWay` that you created as the answer to Self-Test Exercise 28.
31. Give a statement that will close the stream `outputThisWay` created as the answer to Self-Test Exercise 28.

The input stream for reading from the binary file `numbers.dat` is opened as follows:

```
ObjectInputStream inputStream =  
    new ObjectInputStream(new  
        FileInputStream("numbers.dat"));
```

Note that this is identical to how we opened a file using `ObjectOutputStream` in Display 10.13, except that here we have used the classes `ObjectInputStream` and `FileInputStream` instead of `ObjectOutputStream` and `FileOutputStream`.

Opening a Binary File for Reading

Create a stream of the class `ObjectInputStream` and connect it to a binary file as follows:

SYNTAX

```
ObjectInputStream Input_Stream_Name =  
    new ObjectInputStream(new FileInputStream(File_Name));
```

The constructor for `FileInputStream` may throw a `FileNotFoundException`, which is a kind of `IOException`. If the `FileInputStream` constructor succeeds, then the constructor for `ObjectInputStream` may throw a different `IOException`.

EXAMPLES

```
ObjectInputStream inputFile =  
    new ObjectInputStream(new  
        FileInputStream("somefile.dat"));
```

After this, you can use the methods in Display 10.15 to read from the file.

Display 10.15 Some Methods in the Class `ObjectInputStream` (part 1 of 3)

The classes `ObjectInputStream` and `FileInputStream` are in the `java.io` package.

```
public ObjectInputStream(InputStream streamObject)
```

There is no constructor that takes a file name as an argument. If you want to create a stream using a file name, use

```
new ObjectInputStream(new FileInputStream(File_Name))
```

Alternatively, you can use an object of the class `File` in place of the `File_Name`, as follows:

```
new ObjectInputStream(new FileInputStream(File_Object))
```

The constructor for `FileInputStream` may throw a `FileNotFoundException`, which is a kind of `IOException`. If the `FileInputStream` constructor succeeds, then the constructor for `ObjectInputStream` may throw a different `IOException`.

Display 10.15 Some Methods in the Class `ObjectInputStream` (part 2 of 3)

```
public int readInt() throws IOException
```

Reads an `int` value from the input stream and returns that `int` value. If `readInt` tries to read a value from the file and that value was not written using the method `writeInt` of the class `ObjectOutputStream` (or written in some equivalent way), then problems will occur. If an attempt is made to read beyond the end of the file, an `EOFException` is thrown.

```
public int readShort() throws IOException
```

Reads a `short` value from the input stream and returns that `short` value. If `readShort` tries to read a value from the file and that value was not written using the method `writeShort` of the class `ObjectOutputStream` (or written in some equivalent way), then problems will occur. If an attempt is made to read beyond the end of the file, an `EOFException` is thrown.

```
public long readLong() throws IOException
```

Reads a `long` value from the input stream and returns that `long` value. If `readLong` tries to read a value from the file and that value was not written using the method `writeLong` of the class `ObjectOutputStream` (or written in some equivalent way), then problems will occur. If an attempt is made to read beyond the end of the file, an `EOFException` is thrown.

```
public double readDouble() throws IOException
```

Reads a `double` value from the input stream and returns that `double` value. If `readDouble` tries to read a value from the file and that value was not written using the method `writeDouble` of the class `ObjectOutputStream` (or written in some equivalent way), then problems will occur. If an attempt is made to read beyond the end of the file, an `EOFException` is thrown.

```
public float readFloat() throws IOException
```

Reads a `float` value from the input stream and returns that `float` value. If `readFloat` tries to read a value from the file and that value was not written using the method `writeFloat` of the class `ObjectOutputStream` (or written in some equivalent way), then problems will occur. If an attempt is made to read beyond the end of the file, an `EOFException` is thrown.

```
public char readChar() throws IOException
```

Reads a `char` value from the input stream and returns that `char` value. If `readChar` tries to read a value from the file and that value was not written using the method `writeChar` of the class `ObjectOutputStream` (or written in some equivalent way), then problems will occur. If an attempt is made to read beyond the end of the file, an `EOFException` is thrown.

```
public boolean readBoolean() throws IOException
```

Reads a `boolean` value from the input stream and returns that `boolean` value. If `readBoolean` tries to read a value from the file and that value was not written using the method `writeBoolean` of the class `ObjectOutputStream` (or written in some equivalent way), then problems will occur. If an attempt is made to read beyond the end of the file, an `EOFException` is thrown.

(continued)

Display 10.15 Some Methods in the Class `ObjectInputStream` (part 3 of 3)

```
public String readUTF() throws IOException
```

Reads a `String` value from the input stream and returns that `String` value. If `readUTF` tries to read a value from the file and that value was not written using the method `writeUTF` of the class `ObjectOutputStream` (or written in some equivalent way), then problems will occur. If an attempt is made to read beyond the end of the file, an `EOFException` is thrown.

```
Object readObject() throws ClassNotFoundException, IOException
```

Reads an object from the input stream. The object read should have been written using `writeObject` of the class `ObjectOutputStream`. Throws a `ClassNotFoundException` if a serialized object cannot be found. If an attempt is made to read beyond the end of the file, an `EOFException` is thrown. May throw various other `IOExceptions`.

```
public int skipBytes(int n) throws IOException
```

Skips `n` bytes.

```
public void close() throws IOException
```

Closes the stream's connection to a file.

reading
multiple
types

closing a
binary file

`ObjectInputStream` allows you to read input values of different types from the same file. So, you may read a combination of, for example, `int` values, `double` values, and `String` values. However, if the next data item in the file is not of the type expected by the reading method, the result is likely to be a mess. For example, if your program writes an integer using `writeInt`, then any program that reads that integer should read it using `readInt`. If you instead use `readLong` or `readDouble`, your program will misbehave.

Note that, as illustrated in Display 10.16, you close a binary input stream in the same way that you close all the other I/O streams we have seen.

Self-Test Exercises

32. Write code to open the binary file named `someStuff` and connect it to an `ObjectInputStream` object named `inputThing` so it is ready for reading.
33. Give a statement that will read a number of type `double` from the file `someStuff` and place the value in a variable named `number`. Use the stream `inputThing` that you created in Self-Test Exercise 32. (Assume the first thing written to the file was written using the method `writeDouble` of the class `ObjectOutputStream` and assume `number` is of type `double`.)
34. Give a statement that will close the stream `inputThing` created in Self-Test Exercise 32.
35. Can one program write a number to a file using `writeInt` and then have another program read that number using `readLong`? Can a program read that number using `readDouble`?
36. Can you use `readUTF` to read a string from a text file?

Display 10.16 Reading from a Binary File

```
1  import java.io.ObjectInputStream;
2  import java.io.FileInputStream;
3  import java.io.IOException;
4  import java.io.FileNotFoundException;

5  public class BinaryInputDemo
6  {
7      public static void main(String[] args)
8      {
9          try
10         {
11             ObjectInputStream inputStream =
12                 new ObjectInputStream(new FileInputStream("numbers.dat"));

13             System.out.println("Reading the file numbers.dat.");
14             int n1 = inputStream.readInt();
15             int n2 = inputStream.readInt();

16             System.out.println("Numbers read from file:");
17             System.out.println(n1);
18             System.out.println(n2);
19             inputStream.close();
20         }
21         catch (FileNotFoundException e)
22         {
23             System.out.println("Cannot find file numbers.dat.");
24         }
25         catch (IOException e)
26         {
27             System.out.println("Problems with input from numbers.dat.");
28         }
29         System.out.println("End of program.");
30     }
31 }
```

Sample Dialogue

Reading the file numbers.dat.

Numbers read from file:

0

1

End of program.

*Assumes the program in
Display 10.13 was run to
create the file numbers.dat.*

**EOF-
Exception**

Checking for the End of a Binary File

All of the `ObjectInputStream` methods that read from a binary file will throw an `EOFException` when they try to read beyond the end of a file. So, your code can test for the end of a binary file by catching an `EOFException` as illustrated in Display 10.17.

In Display 10.17, the reading is placed in an “infinite loop” through the use of `true` as the Boolean expression in the `while` loop. The loop is not really infinite, because when the end of the file is reached, an exception is thrown, and that ends the entire `try` block and passes control to the `catch` block.

EOFException

If your program is reading from a binary file using any of the methods listed in Display 10.15 for the class `ObjectInputStream`, and your program attempts to read beyond the end of the file, then an `EOFException` is thrown. This can be used to end a loop that reads all the data in a file.

The class `EOFException` is a derived class of the class `IOException`. So, every exception of type `EOFException` is also of type `IOException`.

Display 10.17 Using `EOFException` (part 1 of 2)

```
1  import java.io.ObjectInputStream;
2  import java.io.FileInputStream;
3  import java.io.EOFException;
4  import java.io.IOException;
5  import java.io.FileNotFoundException;

6  public class EOFDemo
7  {
8      public static void main(String[] args)
9      {
10         try
11         {
12             ObjectInputStream inputStream =
13                 new ObjectInputStream(new FileInputStream("numbers.dat"));
14             int number;
15             System.out.println("Reading numbers in numbers.dat");
16             try
17             {
18                 while (true)
19                 {
20                     number = inputStream.readInt();
21                     System.out.println(number);
22                 }
23             }
```

Display 10.17 Using `EOFException` (part 2 of 2)

```
24         catch (EOFException e)
25         {
26             System.out.println("No more numbers in the file.");
27         }
28         inputStream.close();
29     }
30     catch (FileNotFoundException e)
31     {
32         System.out.println("Cannot find file numbers.dat.");
33     }
34     catch (IOException e)
35     {
36         System.out.println("Problem with input from file
37             numbers.dat.");
38     }
39 }
```

Sample Dialogue

Reading numbers in numbers.dat

0

1

2

3

4

No more numbers in the file.

*Assumes the program in
Display 10.13 was run to
create the file numbers.dat.*

**PITFALL: Checking for the End of a File in the Wrong Way**

Different file-reading methods check for the end of a file in different ways. If you test for the end of a file in the wrong way, one of two things will probably happen: Your program will either go into an unintended infinite loop, or it will terminate abnormally.

For the classes discussed in this book, the following rules apply: If your program is reading from a binary file, then an `EOFException` will be thrown when the reading goes beyond the end of the file. If your program is reading from a text file, then no `EOFException` will be thrown when reading goes beyond the end of the file. ■

Self-Test Exercises

37. When opening a binary file for output in the ways discussed in this chapter, might an exception be thrown? What kind of exception? When opening a binary file for input in the ways discussed in this chapter, might an exception be thrown? What kind of exception?
38. Suppose a binary file contains three numbers written to the file with the method `writeDouble` of the class `ObjectOutputStream`. Suppose further that your program reads all three numbers with three invocations of the method `readDouble` of the class `ObjectInputStream`. When will an `EOFException` be thrown? Right after reading the third number? When your program tries to read a fourth number? Some other time?
39. The following appears in the program in Display 10.17:

```
try
{
    while (true)
    {
        number = inputStream.readInt();
        System.out.println(number);
    }
}
catch (EOFException e)
{
    System.out.println("No more numbers in the file.");
}
```

Why isn't this an infinite loop?

Binary I/O of Objects

You can output objects of classes you define as easily as you output `int` values using `writeInt`, and you can later read the objects back into your program as easily as you read `int` values with the method `readInt`. For you to be able to do this, the class of objects that your code is writing and reading must implement the `Serializable` interface.

We will discuss interfaces in general in Chapter 13. However, the `Serializable` interface is particularly easy to use and requires no knowledge of interfaces. All you need to do to make a class implement the `Serializable` interface is add the two words `implements Serializable` to the heading of the class definition, as in the following example:

```
public class Person implements Serializable
{
```

The `Serializable` interface is in the same `java.io` package that contains all the I/O classes we have discussed in this chapter. For example, in Display 10.18, we define a toy class named `SomeClass` that implements the `Serializable` interface. We will

**Serializable
interface**

explain the effect of the `Serializable` interface a bit later in this chapter, but first let's see how you do binary file I/O with a serializable class, such as this class `SomeClass` in Display 10.18.

`writeObject`

Display 10.19 illustrates how class objects can be written to and read from a binary file. To write an object of a class such as `SomeClass` to a binary file, simply use the method `writeObject` of the class `ObjectOutputStream`. You use `writeObject` in the same way that you use the other methods of the class `ObjectOutputStream`, such as `writeInt`, but you use an object as the argument.

`readObject`

If an object is written to a file with `writeObject`, then it can be read back out of the file with `readObject` of the stream class `ObjectInputStream`, as also illustrated in Display 10.19. The method `readObject` returns its value as an object of type `Object`. If you want to use the values returned by `readObject` as an object of a class such as `SomeClass`, you must do a type cast, as shown in Display 10.19.

The Serializable Interface

`serializable`

A class that implements the `Serializable` interface is said to be a **serializable** class. To use objects of a class with `writeObject` and `readObject`, that class must be serializable. But to make the class serializable, we change nothing in the class. All we do is add the phrase `implements Serializable`. This phrase tells the run-time system that it is OK to treat objects of the class in a particular way when doing file I/O. If a class is

Display 10.18 A Serializable Class

```

1  import java.io.Serializable;

2  public class SomeClass implements Serializable
3  {
4      private int number;
5      private char letter;

6      public SomeClass()
7      {
8          number = 0;
9          letter = 'A';
10     }

11     public SomeClass(int theNumber, char theLetter)
12     {
13         number = theNumber;
14         letter = theLetter;
15     }
16     public String toString()
17     {
18         return "Number = " + number
19                + " Letter = " + letter;
20     }
21 }
```

Display 10.19 Binary File I/O of Objects (part 1 of 2)

```

1  import java.io.ObjectOutputStream;
2  import java.io.FileOutputStream;
3  import java.io.ObjectInputStream;
4  import java.io.FileInputStream;
5  import java.io.IOException;
6  import java.io.FileNotFoundException;

7  /**
8   Demonstrates binary file I/O of serializable class objects.
9   */
10 public class ObjectIODemo
11 {
12     public static void main(String[] args)
13     {
14         try
15         {
16             ObjectOutputStream outputStream =
17                 new ObjectOutputStream(new FileOutputStream("datafile"));

18             SomeClass oneObject = new SomeClass(1, 'A');
19             SomeClass anotherObject = new SomeClass(42, 'Z');

20             outputStream.writeObject(oneObject);
21             outputStream.writeObject(anotherObject);

22             outputStream.close();

23             System.out.println("Data sent to file.");
24         }
25         catch(IOException e)
26         {
27             System.out.println("Problem with file output.");
28         }

29         System.out.println(
30             "Now let's reopen the file and display the data.");
31         try
32         {
33             ObjectInputStream inputStream =
34                 new ObjectInputStream(new FileInputStream("datafile"));
35             SomeClass readOne = (SomeClass)inputStream.readObject();
36             SomeClass readTwo = (SomeClass)inputStream.readObject();

37             System.out.println("The following were read from the file:");
38             System.out.println(readOne);
39             System.out.println(readTwo);
40         }
41         catch(FileNotFoundException e)
42         {
43             System.out.println("Cannot find datafile.");
44         }

```

Notice the type casts.

Display 10.19 Binary File I/O of Objects (part 2 of 2)

```
45         catch(ClassNotFoundException e)
46         {
47             System.out.println("Problems with file input.");
48         }
49         catch(IOException e)
50         {
51             System.out.println("Problems with file input.");
52         }
53         System.out.println("End of program.");
54     }
55 }
```

Sample Dialogue

```
Data sent to file.
Now let's reopen the file and display the data.
The following were read from the file:
Number = 1 Letter = A
Number = 42 Letter = Z
End of program.
```

serializable, Java assigns a serial number to each object of the class that it writes to a stream of type `ObjectOutputStream`. If the same object is written to the stream more than once, then after the first time, Java writes only the serial number for the object and not a full description of the object's data. This makes file I/O more efficient and makes the files smaller. When read back out with a stream of type `ObjectInputStream`, duplicate serial numbers are returned as references to the same object. Note that this means that if two variables contain references to the same object and you write the objects to the file and later read them from the file, then the two objects that are read will again be references to the same object. So nothing in the structure of your object data is lost when you write the objects to the file and later read them back.

**class instance
variables**

When a serializable class has instance variables of a class type, then the classes for the instance variables must also be serializable, and so on for all levels of **class instance variables** within classes. So, a class is not serializable unless the classes for all instance variables are also serializable.

Why aren't all classes made serializable? For security reasons. The serial number system makes it easier for programmers to get access to the object data written to secondary storage. Also, for some classes, it may not make sense to write objects to secondary storage, because they would be meaningless when read out again later. For example, if the object contains system-dependent data, the data may be meaningless when later read out.



PITFALL: Mixing Class Types in the Same File

The best way to write and read objects using `ObjectOutputStream` and `ObjectInputStream` is to store only data of one class type in any one file. If you store objects of multiple class types or even objects of only one class type mixed in with primitive type data, it has been our experience that the system can get confused and you could lose data. ■

Array Objects in Binary Files

An array is an object and hence a suitable argument for `writeObject`. An entire array can be saved to a binary file using `writeObject` and later read using `readObject`. When doing so, if the array has a base type that is a class, then the class must be serializable. This means that if you store all your data for one serializable class in a single array, then you can output all your data to a binary file with one invocation of `writeObject`.

This way of storing an array in a binary file is illustrated in Display 10.20. Note that the base class type, `SomeClass`, is serializable. Also, notice the type cast that uses the array type `SomeClass[]`. Because `readObject` returns its value as an object of type `Object`, it must be type cast to the correct array type.

Self-Test Exercises

40. How do you make a class implement the `Serializable` interface?
41. What import statement do you need to be able to use the `Serializable` interface?
42. What is the return type for the method `readObject` of the class `ObjectInputStream`?
43. Is an array of type `Object`?

Display 10.20 File I/O of an Array Object (part 1 of 3)

```
1 import java.io.ObjectOutputStream;
2 import java.io.FileOutputStream;
3 import java.io.ObjectInputStream;
4 import java.io.FileInputStream;
5 import java.io.IOException;
6 import java.io.FileNotFoundException;

7 public class ArrayIODemo
8 {

9     public static void main(String[] args)
```

Display 10.20 File I/O of an Array Object (part 2 of 3)

```

10      {
11          SomeClass[] a = new SomeClass[2];
12          a[0] = new SomeClass(1, 'A');
13          a[1] = new SomeClass(2, 'B');

14          try
15          {
16              ObjectOutputStream outputStream =
17                  new ObjectOutputStream(new FileOutputStream("arrayfile"));
18              outputStream.writeObject(a);
19              outputStream.close();
20          }
21          catch(IOException e)
22          {
23              System.out.println("Error writing to file.");
24              System.exit(0);
25          }

26          System.out.println(
27              "Array written to file arrayfile.");

28          System.out.println(
29              "Now let's reopen the file and display the array.");

30          SomeClass[] b = null ;

31          try
32          {
33              ObjectInputStream inputStream =
34                  new ObjectInputStream(new FileInputStream("arrayfile"));
35              b = (SomeClass[])inputStream.readObject();
36              inputStream.close();
37          }
38          catch(FileNotFoundException e)
39          {
40              System.out.println("Cannot find file arrayfile.");
41              System.exit(0);
42          }
43          catch(ClassNotFoundException e)
44          {
45              System.out.println("Problems with file input.");
46              System.exit(0);
47          }
48          catch(IOException e)
49          {
50              System.out.println("Problems with file input.");
51              System.exit(0);
52          }
53          System.out.println(
54              "The following array elements were read from the file.");
55          int i;

```

Notice the type cast.

(continued)

Display 10.20 File I/O of an Array Object (part 3 of 3)

```

56         for (i = 0; i < b.length; i++)
57             System.out.println(b[i]);
58     System.out.println("End of program.");
59 }
60 }

```

Sample Dialogue

```

Array written to file arrayfile.
Now let's reopen the file and display the array.
The following array elements were read from the file:
Number = 1 Letter = A
Number = 2 Letter = B
End of program.

```

10.5 Random Access to Binary Files ★

Anytime, anywhere.

Common response to a challenge for a confrontation.

The streams for sequential access to files, which we discussed in the previous sections of this chapter, are the ones most often used for file access in Java. However, some applications that require very rapid access to records in very large databases require some sort of random access to particular parts of a file. Such applications might best be done with specialized database software. But perhaps you are given the job of writing such a package in Java, or perhaps you are just curious about how such things are done in Java. Java does provide for random access to files so that your program can both read from and write to random locations in a file. In this section, we will describe simple uses of random access to files.

Reading and Writing to the Same File

If you want random access to both read and write to a file in Java, use the stream class `RandomAccessFile`, which is in the `java.io` package like all other file I/O classes.

file pointer

A random access file consists of a sequence of numbered bytes. There is a kind of marker called the **file pointer** that is always positioned at one of these bytes. All reads and writes take place starting at the location of the file pointer. You can move the file pointer to a new location with the method `seek`.

Although a random access file is byte oriented, there are methods to allow for reading or writing values of the primitive types and of string values to a random access file. In fact, these are the same methods as those we already used for sequential access files, as previously discussed. A `RandomAccessFile` stream has methods `writeInt`, `writeDouble`, `writeUTF`, and so forth, as well as methods `readInt`, `readDouble`,

Display 10.21 Some Methods of the Class `RandomAccessFile` (part 1 of 3)

The class `RandomAccessFile` is in the `java.io` package.

```
public RandomAccessFile(String fileName, String mode)
```

```
public RandomAccessFile(File fileObject, String mode)
```

Opens the file, does not delete data already in the file, but does position the file pointer at the first (zeroth) location.

The mode must be one of the following:

"r" Open for reading only.

"rw" Open for reading and writing.

"rws" Same as "rw", and also requires that every update to the file's content or metadata be written synchronously to the underlying storage device.

"rwd" Same as "rw", and also requires that every update to the file's content be written synchronously to the underlying storage device.

"rws" and "rwd" are not covered in this book text.

```
public long getFilePointer() throws IOException
```

Returns the current location of the file pointer. Locations are numbered starting with 0.

```
public void seek(long location) throws IOException
```

Moves the file pointer to the specified location.

```
public long length() throws IOException
```

Returns the length of the file.

```
public void setLength(long newLength) throws IOException
```

Sets the length of this file.

If the present length of the file as returned by the `length` method is greater than the `newLength` argument, then the file will be truncated. In this case, if the file pointer location as returned by the `getFilePointer` method is greater than `newLength`, then after this method returns, the file pointer location will be equal to `newLength`.

If the present length of the file as returned by the `length` method is smaller than `newLength`, then the file will be extended. In this case, the contents of the extended portion of the file are not defined.

```
public void close() throws IOException
```

Closes the stream's connection to a file.

```
public void write(int b) throws IOException
```

Writes the specified byte to the file.

```
public void write(byte [] a) throws IOException
```

Writes `a.length` bytes from the specified byte array to the file.

(continued)

Display 10.21 Some Methods of the Class `RandomAccessFile` (part 2 of 3)

```
public final void writeByte(byte b) throws IOException
```

Writes the byte `b` to the file.

```
public final void writeShort(short n) throws IOException
```

Writes the short `n` to the file.

```
public final void writeInt(int n) throws IOException
```

Writes the int `n` to the file.

```
public final void writeLong(long n) throws IOException
```

Writes the long `n` to the file.

```
public final void writeDouble(double d) throws IOException
```

Writes the double `d` to the file.

```
public final void writeFloat(float f) throws IOException
```

Writes the float `f` to the file.

```
public final void writeChar(char c) throws IOException
```

Writes the char `c` to the file.

```
public final void writeBoolean(boolean b) throws IOException
```

Writes the boolean `b` to the file.

```
public final void writeUTF(String s) throws IOException
```

Writes the String `s` to the file.

```
public int read() throws IOException
```

Reads a byte of data from the file and returns it as an integer in the range 0 to 255.

```
public int read(byte [] a) throws IOException
```

Reads `a.length` bytes of data from the file into the array of bytes `a`. Returns the number of bytes read or -1 if the end of the file is encountered.

```
public final byte readByte() throws IOException
```

Reads a byte value from the file and returns that value. If an attempt is made to read beyond the end of the file, an `EOFException` is thrown.

```
public final short readShort() throws IOException
```

Reads a short value from the file and returns that value. If an attempt is made to read beyond the end of the file, an `EOFException` is thrown.

Display 10.21 Some Methods of the Class `RandomAccessFile` (part 3 of 3)

```
public final int readInt() throws IOException
```

Reads an `int` value from the file and returns that value. If an attempt is made to read beyond the end of the file, an `EOFException` is thrown.

```
public final long readLong() throws IOException
```

Reads a `long` value from the file and returns that value. If an attempt is made to read beyond the end of the file, an `EOFException` is thrown.

```
public final double readDouble() throws IOException
```

Reads a `double` value from the file and returns that value. If an attempt is made to read beyond the end of the file, an `EOFException` is thrown.

```
public final float readFloat() throws IOException
```

Reads a `float` value from the file and returns that value. If an attempt is made to read beyond the end of the file, an `EOFException` is thrown.

```
public final char readChar() throws IOException
```

Reads a `char` value from the file and returns that value. If an attempt is made to read beyond the end of the file, an `EOFException` is thrown.

```
public final boolean readBoolean() throws IOException
```

Reads a `boolean` value from the file and returns that value. If an attempt is made to read beyond the end of the file, an `EOFException` is thrown.

```
public final String readUTF() throws IOException
```

Reads a `String` value from the file and returns that value. If an attempt is made to read beyond the end of the file, an `EOFException` is thrown.

`readUTF`, and so on. However, the class `RandomAccessFile` does not have the methods `writeObject` or `readObject`. The most important methods of the class `RandomAccessFile` are given in Display 10.21. A demonstration program for random access files is given in Display 10.22.

opening a file

The constructor for `RandomAccessFile` takes either a string name for the file or an object of the class `File` as its first argument. The second argument must be one of the four strings `"rw"`, `"r"`, and two modes we do not discuss in this book, `"rws"` and `"rwd"`. The string `"rw"` means your code can both read and write to the file after it is open. The string `"r"` means your code can read from the file but cannot write to the file.

If the file already exists, then when it is opened, the length is not reset to 0, but the file pointer will be positioned at the start of the file, which is what you would expect at least for `"r"`. If the length of the file is not what you want, you can change it with the method `setLength`. In particular, you can use `setLength` to empty the file.

Display 10.22 Random Access to a File (part 1 of 2)

```
1  import java.io.RandomAccessFile;
2  import java.io.IOException;
3  import java.io.FileNotFoundException;

4  public class RandomAccessDemo
5  {
6      public static void main(String[] args)
7      {
8          try
9          {
10             RandomAccessFile ioStream =
11                 new RandomAccessFile("bytedata", "rw");

12             System.out.println("Writing 3 bytes to the file bytedata.");
13             ioStream.writeByte(1);
14             ioStream.writeByte(2);
15             ioStream.writeByte(3);
16             System.out.println("The length of the file is now = "
17                                 + ioStream.length());
18             System.out.println("The file pointer location is "
19                                 + ioStream.getFilePointer());

20             System.out.println("Moving the file pointer to location 1.");
21             ioStream.seek(1);
22             byte oneByte = ioStream.readByte();
23             System.out.println("The value at location 1 is " + oneByte);
24             oneByte = ioStream.readByte();
25             System.out.println("The value at the next location is "
26                                 + oneByte);

27             System.out.println("Now we move the file pointer back to");
28             System.out.println("location 1, and change the byte.");
29             ioStream.seek(1);
30             ioStream.writeByte(9);
31             ioStream.seek(1);
32             oneByte = ioStream.readByte();
33             System.out.println("The value at location 1 is now "
34                                 + oneByte);

34             System.out.println("Now we go to the end of the file");
35             System.out.println("and write a double.");
36             ioStream.seek(ioStream.length());
37             ioStream.writeDouble(41.99);
38             System.out.println("The length of the file is now = "
39                                 + ioStream.length());
```

Display 10.22 Random Access to a File (part 2 of 2)

```

40         System.out.println("Returning to location 3,");
41         System.out.println("where we wrote the double.");
42         inputStream.seek(3);
43         double oneDouble = inputStream.readDouble();
44         System.out.println("The double value at location 3 is "
45                             + oneDouble);
46
47         inputStream.close();
48     catch (FileNotFoundException e)
49     {
50         System.out.println("Problem opening file.");
51     }
52     catch (IOException e)
53     {
54         System.out.println("Problems with file I/O.");
55     }
56     System.out.println("End of program.");
57 }
58 }

```

The location of readDouble must be a location where writeDouble wrote to the file.

The dialogue assumes the file bytedata did not exist before the program was run.

Sample Dialogue

```

Writing 3 bytes to the file bytedata.
The length of the file is now = 3
The file pointer location is 3
Moving the file pointer to location 1.
The value at location 1 is 2
The value at the next location is 3
Now we move the file pointer back to
location 1, and change the byte.
The value at location 1 is now 9
Now we go to the end of the file
and write a double.
The length of the file is now = 11
Returning to location 3,
where we wrote the double.
The double value at location 3 is 41.99
End of program.

```

Byte locations are numbered starting with zero.

Three 1-byte values and 1 double value that uses 8 bytes = 11 bytes total.



PITFALL: `RandomAccessFile` Need Not Start Empty

If a file already exists, then when it is opened with `RandomAccessFile`, the length is not reset to 0, but the file pointer will be positioned at the start of the file. So, old data in the file is not lost and the file pointer is set for the most likely position for reading, not the most likely position for writing.

Self-Test Exercises

44. If you run the program in Display 10.22 a second time, will the output be the same?
45. How can you modify the program in Display 10.22 so the file always starts out empty?

Chapter Summary

- Files that are considered to be strings of characters and that look like characters to your program and your editor are called *text files*. Files whose contents must be handled as strings of binary digits are called *binary files*.
- You can use the class `PrintWriter` to write to a text file and can use the class `Scanner` or `BufferedReader` to read from a text file.
- The class `File` can be used to check whether there is a file with a given name. It can also check whether your program is allowed to read the file and/or allowed to write to the file.
- Your program can use the class `ObjectOutputStream` to write to a binary file and can use the class `ObjectInputStream` to read from a binary file.
- Your program can use the method `writeObject` of the class `ObjectOutputStream` to write class objects to a binary file. The objects can be read back with the method `readObject` of the class `ObjectInputStream`.
- To use the method `writeObject` of the class `ObjectOutputStream` or the method `readObject` of the class `ObjectInputStream`, the class whose objects are written to a file must implement the `Serializable` interface.
- The way that you test for the end of a file depends on whether your program is reading from a text file or a binary file.
- You can use the class `RandomAccessFile` to create a stream that gives random access to the bytes in a file.

Answers to Self-Test Exercises

1. With an input stream, data flows from a file or input device to your program. With an output stream, data flows from your program to a file or output device.
2. A binary file contains data that is processed as binary data. A text file allows your program and editor to view the file as if it contained a sequence of characters. A text file can be viewed with an editor, whereas a binary file cannot.
3. A `FileNotFoundException` would be thrown if the file cannot be opened because, for example, there is already a directory (folder) named `stuff.txt`. Note that if the file does not exist but can be created, then no exception is thrown. If you answered `IOException`, you are not wrong, because a `FileNotFoundException` is an `IOException`. However, the better answer is the more specific exception class—namely, `FileNotFoundException`.

4. Yes. This functionality was added to the constructor in Java 1.5. Prior to Java 1.5 the correct way to express the code displayed in the question is as follows:

```
PrintWriter outputStream =  
    new PrintWriter(new FileOutputStream("stuff.txt"));
```

5.

```
PrintWriter outputStream =  
    new PrintWriter(new FileOutputStream("sam"));
```

6.

```
PrintWriter outputStream =  
    new PrintWriter(new FileOutputStream("sam", true));
```

7. Yes, it will send suitable output to the text file because the class `Person` has a well-defined `toString()` method.

8.

```
Scanner fileIn =  
    new Scanner(new FileInputStream("sally"));
```

9. It throws a `NoSuchElementException` if there are no more tokens. It throws an `InputMismatchException` if the next token is not a well-formed string representation of an `int`. It throws an `IllegalStateException` if the `Scanner` stream is closed.

10. No. Reading may have reached the end of the file, but another possibility is that the next token may not be a well-formed string representation of an `int` value.

11. The `FileInputStream` constructor, and thus the `Scanner` constructor invocation, can throw a `FileNotFoundException`. This exception needs to be caught or declared in a `throws` clause.

12.

```
BufferedReader fileIn =  
    new BufferedReader(new FileReader("joe"));
```

13. The method `readLine` returns a value of type `String`. The method `read` reads a single *character*, but it returns it as a value of type `int`. To get the value to be of type `char`, you need to do a type cast.

14. Both `read` and `readLine` in the class `BufferedReader` might throw an `IOException`.
15. The `try` block in Display 10.7 is larger so that it includes the invocations of the method `readLine`, which might throw an `IOException`. The method `println` in Display 10.1 does not throw any exceptions that must be caught.
16. Yes.
17. No, you must read the number as a string and then convert the string to a number with `Integer.parseInt` (or in some other way).
18. When the method `readLine` tries to read beyond the end of a file, it returns the value `null`. Thus, you can test for the end of a file by testing for `null`.
19. The method `read` reads a single *character*, but it returns it as a value of type `int`. To get the value to be of type `char`, you need to do a type cast.
20. When the method `read` tries to read beyond the end of a file, it returns the value `-1`. Thus, you can test for the end of a file by testing for the value `-1`. This works because all “real” characters return a positive `int` value.
21. Yes, if `original.txt` is an empty file, then the file `numbered.txt` produced by the program will also be empty.
22. Only the classes `FileReader` and `FileOutputStream` have a constructor that accepts a file name as an argument. (Although we have not discussed it, the class `Scanner` has a constructor that takes a `String` argument, but the argument is not a file name.)
23. Yes, it is legal.
24. Replace

```
System.setErr(errStream);  
  
with  
  
System.setOut(errStream);
```

25. Add

```
System.setOut(errStream);  
  
to get  
  
System.setErr(errStream);  
System.setOut(errStream);
```

26. `import java.io.File;`
`public class FileExercise`
`{`

```
    public static void main(String[] args)  
    {  
        File fileObject = new File("Sally.txt");
```

```

        if (fileObject.exists())
            System.out.println(
                "There is a file named Sally.txt.");
        else
            System.out.println(
                "There is no file named Sally.txt.");
    }
}

27. import java.io.IOException;
import java.io.File;
import java.util.Scanner;

public class FileExercise2
{
    public static void main(String[] args)
    {
        Scanner keyboard = new Scanner(System.in);
        String fileName = null;
        File fileObject = null;

        try
        {
            System.out.print("Enter a file name and I will");
            System.out.println(" tell you if it exists.");
            fileName = keyboard.next();
            fileObject = new File(fileName);

            if (fileObject.exists())
            {
                System.out.println("There is a file named"
                                    + fileName);
                System.out.println("Delete the file? (y/n)");
                char answer = (char)System.in.read();

                if ((answer == 'y') || (answer == 'Y'))
                {
                    if (fileObject.delete())
                        System.out.println("File deleted.");
                    else
                        System.out.println(
                            "Cannot delete file.");
                }
            }
            else
                System.out.println(
                    "No file named " + fileName);
        }
    }
}

```

```

        catch(IOException e)
        {
            System.out.println(
                "Error reading from keyboard.");
        }
    }
}

```

28. `ObjectOutputStream outputThisWay =`
 `new ObjectOutputStream(`
 `new FileOutputStream("bindata.dat"));`
29. `outputThisWay.writeDouble(v1);`
 `outputThisWay.writeDouble(v2);`
30. `outputThisWay.writeUTF("Hello");`
31. `outputThisWay.close();`
32. `ObjectInputStream inputThing =`
 `new ObjectInputStream(`
 `new FileInputStream("someStuff"));`
33. `number = inputThing.readDouble();`
34. `inputThing.close();`
35. If a number is written to a file with `writeInt`, it should be read only with `readInt`. If you use `readLong` or `readDouble` to read the number, something will go wrong.
36. You should not use `readUTF` to read a string from a text file. You should use `readUTF` only to read a string from a binary file. Moreover, the string should have been written to that file using `writeUTF`.
37. When opening a binary file for either output or input in the ways discussed in this chapter, a `FileNotFoundException` might be thrown and other `IOExceptions` may be thrown.
38. An `EOFException` is thrown when your program tries to read the (nonexisting) fourth number.
39. It is not an infinite loop because when the end of the file is reached, an exception will be thrown, and that will end the entire `try` block.
40. You add the two words `implements Serializable` to the beginning of the class definition. You also must do this for the classes for the instance variables and so on for all levels of class instance variables within classes.
41. `import java.io.Serializable;` or `import java.io.*;`
42. The return type is `Object`, which means the returned value usually needs to be type cast.

43. Yes. That is why it is a legitimate argument for `writeObject`.
44. No. Each time the program is run, the file will get longer.
45. Add the following near the start of `main`:

```
ioStream.setLength(0);
```

Programming Projects

PROJECTS INVOLVING ONLY TEXT FILES



VideoNote
Solution to
Programming
Project 10.1

1. The text files `boynames.txt` and `girlnames.txt`, which are included in the source code for this book text, contain a list of the 1,000 most popular boy and girl names in the United States for the year 2003 as compiled by the Social Security Administration.

These are blank-delimited files, where the most popular name is listed first, the second most popular name is listed second, and so on, to the 1,000th most popular name, which is listed last. Each line consists of the first name followed by a blank space and then the number of registered births using that name in the year. For example, the `girlnames.txt` file begins with

```
Emily 25494
```

```
Emma 22532
```

```
Madison 19986
```

This indicates that Emily was the most popular name with 25,494 registered namings, Emma was the second most popular with 22,532, and Madison was the third most popular with 19,986.

Write a program that reads both the girl and boy files into memory using arrays. Then, allow the user to input a name. The program should search through both arrays. If there is a match, then it should output the popularity ranking and the number of namings. The program should also indicate if there is no match.

For example, if the user enters the name “Justice,” then the program should output
Justice is ranked 456 in popularity among girls with 655 namings.
Justice is ranked 401 in popularity among boys with 653 namings.

If the user enters the name “Walter,” then the program should output

```
Walter is not ranked among the top 1000 girl names.
```

```
Walter is ranked 356 in popularity among boys with 775 namings.
```

2. Write a program that will count the total occurrences of the number ‘10’ in a text file of strings representing numbers of type `int` and will show the value of the count on the screen once the whole file is read. The file contains the following numbers separated by space.

```
10 4 7 8 10 34 11 10 15 6 10
```

3. Write a program that takes its input from a text file of strings representing numbers of type `double` and outputs the average of the numbers in the file to the screen. The file contains nothing but strings representing numbers of type `double`, one per line.

4. Write a program that takes its input from a text file of strings representing numbers of type `double`. The program outputs to the screen the average and standard deviation of the numbers in the file. The file contains nothing but strings representing numbers of type `double`, one per line. The standard deviation of a list of numbers n_1, n_2, n_3 , and so forth is defined as the square root of the average of the following numbers:

$(n_1 - a)^2, (n_2 - a)^2, (n_3 - a)^2$, and so forth.

The number a is the average of the numbers n_1, n_2, n_3 , and so forth. *Hint:* Write your program so that it first reads the entire file and computes the average of all the numbers, then closes the file, and then reopens the file and computes the standard deviation. You will find it helpful to first do Programming Project 10.3 and then modify that program in order to obtain the program for this project.

5. Write a program that edits a text file to display each complete sentence with a period at the end in a separate line. Your program should work as follows: Create a temporary file, copy from the source file to a temporary file and perform the required operation. Copy the contents of the temporary file back into the source file. Use a method (or methods) in the class `File` to remove the temporary file. You will also want to use the class `File` for other things in your program. The temporary file should have a name that is different from all existing files so that the existing files are not affected (except for the file being edited). Your program will ask the user for the name of the file to be edited. However, it will not ask the user for the name of the temporary file, but will instead generate the name within the program. You can generate the name any way that is clear and efficient. One possible way to generate the temporary file is to start with an unlikely name, such as "Temp1", and to append a digit, such as '1', until a name is found that does not name an existing file.
6. Write a program that gives and takes advice on program writing. The program starts by writing a piece of advice to the screen and asking the user to type in a different piece of advice. The program then ends. The next person to run the program receives the advice given by the person who last ran the program. The advice is kept in a text file and the content of the file changes after each run of the program. You can use your editor to enter the initial piece of advice in the file so that the first person who runs the program receives some advice. Allow the user to type in advice of any length so that it can be any number of lines long. The user is told to end his or her advice by pressing the Return key two times. Your program can then test to see that it has reached the end of the input by checking to see when it reads two consecutive occurrences of the character '\n'.
7. Write a class that keeps track of the top five high scores that could be used for a video game. Internally, the class should store the top scores in a data structure of your choice (the most straightforward way is to use arrays). Each entry consists of a name and a score. The data stored in memory should be synchronized with a text file for persistent storage. For example, here are the contents of a sample file where Ronaldo has the highest score and Pele has the third highest score:

```

Ronaldo
10400
Didier
9800
Pele
9750
Kaka
8400
Cristiano
8000

```

The constructor should test if the file exists. If it does not exist, then the file should be created with blank names for each of the players and a score of 0. If the file does exist, then the data from the file should be read into the class's instance variables. Along with appropriate constructors, accessors, and mutators, add the following methods:

- `void playerScore(String name, int score)`: Whenever a game is over, this method is called with the player's name and final score. If the name is one of the top five, then it should be added to the list and the lowest score should be dropped out. If the score is not in the top five, then nothing happens.
- `String[] getTopNames()`: Returns an array of the names of the top players, with the top player first, the second best player second, etc.
- `int[] getTopScores()`: Returns an array of the scores of the top players, with the highest score first, the second highest score second, etc.

Test your program with several calls to `playerScore` and print out the list of top names and scores to ensure that the correct values are stored. When the program is restarted, it should remember the top scores from the last session.

8. Create a file `WordBuff.txt` that contains the following list of words: MADAM, DAD, RISK, JAVA, MALAYALAM, RACECAR, RADAR, ROTOR, REFER, SEDES, SOLOS, COURSE, STATS, TOROT, TENET, MACHINE, VIRTUAL, STUDENT, PULLUP, PROGRAMME, and CORE. Write a program that reads each word from the file and outputs the number of palindromes in the file.

PROJECTS INVOLVING BINARY FILES



VideoNote
Solution to
Programming
Project 10.9

9. Write a program that will search a binary file of numbers of type `int` and write the largest and the smallest numbers to the screen. The file contains nothing but numbers of type `int` written to the file with `writeInt`.
10. Write a program that reads grades of type `double` of eight students that the user provides. The grades lie between 0 and 10. These grades should be written to a binary file and read from it. The program outputs the highest and lowest grades achieved by students on the screen. The file contains nothing but numbers of type `double` written to the file with `writeDouble`.
11. Write a program that takes its input from a binary file of numbers of type `double`. The file contains nothing but numbers of type `double` written to the file with

`writeDouble`. The program outputs to the screen the average and standard deviation of the numbers in the file. The standard deviation of a list of numbers n_1 , n_2 , n_3 , and so forth is defined as the square root of the average of the following numbers:

$(n_1 - a)^2$, $(n_2 - a)^2$, $(n_3 - a)^2$, and so forth.

The number a is the average of the numbers n_1 , n_2 , n_3 , and so forth. *Hint:* Write your program so that it first reads the entire file and computes the average of all the numbers, then closes the file, and then reopens the file and computes the standard deviation. You will find it helpful to first do Programming Project 10.8 and then modify that program in order to obtain the program for this project.

12. Change the definition of the class `Person` in Display 5.19 to be serializable. Note that this requires that you also change the class `Date`. Then write a program to maintain a binary file of records of people (records of type `Person`). Allow commands to delete a record specified by the person's name, to add a record, to retrieve and display a record, and to obtain all records of people within a specified age range. To obtain the age of a person, you need the current date. Your program will ask the user for the current date when the program begins. You can do this with random access files, but do not use random access files for this exercise. Use a file or files that record records with the method `writeObject` of the class `ObjectOutputStream`.
13. Programming Projects 6.12 and 6.13 asked you to write a program to play a simple trivia game consisting of five questions. The questions, answers, and point values were hardcoded into array(s). This programming project involves moving the trivia questions into one or more binary files instead, and then loading the trivia questions into memory when the program starts.

First, write a program that allows an administrator to manage the questions for the trivia game. When the program is run, it should check to see if a data file exists. If the data file exists, then the trivia questions should be loaded from the data file into array(s) in memory. If the data file does not exist, start the program with no trivia questions in memory. The program should then present a menu that allows the administrator to list all trivia items (question, answer, and value) in the database, add a new trivia item, or delete an existing trivia item. Upon exiting the program, the trivia data in memory should be stored to one or more binary files using the `writeObject` method.

Second, modify either solution to Programming Project 6.12 or 6.13 to read in the trivia data from the binary file created by the administrator's program. Note that the game is no longer limited to five questions, since an arbitrary number of trivia items may be created by the administrator's program and stored in the binary file(s).



Recursion 11

11.1 RECURSIVE `void` METHODS 685

Example: Vertical Numbers 685

Tracing a Recursive Call 688

A Closer Look at Recursion 691

Stacks for Recursion ★ 694

Recursion versus Iteration 696

11.2 RECURSIVE METHODS THAT RETURN A VALUE 697

General Form for a Recursive Method That
Returns a Value 698

Example: Another Powers Method 698

11.3 THINKING RECURSIVELY 703

Recursive Design Techniques 703

Binary Search ★ 704

Efficiency of Binary Search ★ 710

Example: Finding a File 712

After a lecture on cosmology and the structure of the solar system, William James was accosted by a little old lady.

“Your theory that the sun is the center of the solar system, and the earth is a ball which rotates around it has a very convincing ring to it, Mr. James, but it's wrong. I've got a better theory,” said the little old lady.

“And what is that, madam?” inquired James politely.

“That we live on a crust of earth which is on the back of a giant turtle.”

Not wishing to demolish this absurd little theory by bringing to bear the masses of scientific evidence he had at his command, James decided to gently dissuade his opponent by making her see some of the inadequacies of her position.

“If your theory is correct, madam,” he asked, “what does this turtle stand on?”

“You're a very clever man, Mr. James, and that's a very good question” replied the little old lady, “but I have an answer to it. And it is this: the first turtle stands on the back of a second, far larger, turtle, who stands directly under him.”

“But what does this second turtle stand on?” persisted James patiently.

To this the little old lady crowed triumphantly. “It's no use, Mr. James— it's turtles all the way down.”

J. R. ROSS, *Constraints on Variables in Syntax*, Massachusetts Institute of Technology, 1967.

Introduction

recursive method

A method definition that includes a call to itself is said to be **recursive**. Like most modern programming languages, Java allows methods to be recursive; if used with a little care, this can be a useful programming technique. In this chapter, we introduce the basic techniques needed for defining successful recursive methods. There is nothing in this chapter that is truly unique to Java. If you are already familiar with recursion, you can safely skip this chapter. No new Java elements are introduced here.

Prerequisites

Except for the subsections on binary search and searching for a file, this chapter uses material only from Chapters 1–5. The subsection entitled “Binary Search” also uses the basic material on one-dimensional arrays from Chapter 6 and the Example entitled “Finding a File” uses material from the `File` class in Chapter 10.

You may postpone all or part of this chapter if you wish. Nothing in the rest of this book requires any of this chapter.

11.1 Recursive void Methods

I remembered too that night which is at the middle of the Thousand and One Nights when Scheherazade (through a magical oversight of the copyist) begins to relate word for word the story of the Thousand and One Nights, establishing the risk of coming once again to the night when she must repeat it, and thus to infinity.

JORGE LUIS BORGES, *The Garden of Forking Paths*, Editorial Sur, 1948.

When you are writing a method to solve a task, one basic design technique is to break the task into subtasks. Sometimes it turns out that at least one of the subtasks is a smaller example of the same task. For example, if the task is to search a list for a particular value, you might divide this into the subtask of searching the first half of the list and the subtask of searching the second half of the list. The subtasks of searching the halves of the list are “smaller” versions of the original task. Whenever one subtask is a smaller version of the original task to be accomplished, you can solve the original task by using a recursive method. We begin with a simple example to illustrate this technique. (For simplicity, our examples are static methods; however, recursive methods need not be static.)

Recursion

In Java, a method definition may contain an invocation of the method being defined. In such cases, the method is said to be **recursive**.

EXAMPLE: Vertical Numbers

Display 11.1 contains a demonstration program for a recursive method named `writeVertical`, which takes one (nonnegative) `int` argument and writes that `int` to the screen with the digits going down the screen one per line. For example, the invocation

```
writeVertical(1234);
```

would produce the output

```
1
2
3
4
```

(continued)

Display 11.1 A Recursive void Method

```
1  public class RecursionDemo1
2  {
3      public static void main(String[] args)
4      {
5          System.out.println("writeVertical(3):");
6          writeVertical(3);

7          System.out.println("writeVertical(12):");
8          writeVertical(12);

9          System.out.println("writeVertical(123):");
10         writeVertical(123);
11     }

12     public static void writeVertical(int n)
13     {
14         if (n < 10)
15         {
16             System.out.println(n);
17         }
18         else //n is two or more digits long:
19         {
20             writeVertical(n / 10);
21             System.out.println(n % 10);
22         }
23     }
24 }
```

Sample Dialogue

```
writeVertical(3):
3
writeVertical(12):
1
2
writeVertical(123):
1
2
3
```

EXAMPLE: (continued)

The task to be performed by `writeVertical` may be broken down into the following two subtasks:

Simple Case: If $n < 10$, then write the number n to the screen.

After all, if the number is only one digit long, the task is trivial.

Recursive Case: If $n \geq 10$, then do two subtasks:

1. Output all the digits except the last digit.
2. Output the last digit.

For example, if the argument were 1234, the first part would output

1
2
3

and the second part would output 4. This decomposition of tasks into subtasks can be used to derive the method definition.

Subtask 1 is a smaller version of the original task, so we can implement this subtask with a recursive call. Subtask 2 is just the simple case we listed previously. Thus, an outline of our algorithm for the method `writeVertical` with parameter n is given by the following pseudocode:

```
if (n < 10)
{
    System.out.println(n);
}
else //n is two or more digits long:
{
    writeVertical (the number n with the last digit removed);
    System.out.println(the last digit of n);
}
```

Recursive subtask 

If you observe the following identities, it is easy to convert this pseudocode to a complete Java method definition:

$n / 10$ is the number n with the last digit removed.
 $n \% 10$ is the last digit of n .

For example, $1234 / 10$ evaluates to 123 and $1234 \% 10$ evaluates to 4.

(continued)

EXAMPLE: (continued)

The following is the complete code for the method:

```
public static void writeVertical(int n)
{
    if (n < 10)
    {
        System.out.println(n);
    }
    else //n is two or more digits long:
    {
        writeVertical(n / 10);
        System.out.println(n % 10);
    }
}
```

Tracing a Recursive Call

Let's see exactly what happens when the following method call is made (as in Display 11.1):

```
writeVertical(123);
```

When this method call is executed, the computer proceeds just as it would with any method call. The argument 123 is substituted for the parameter *n*, and the body of the method is executed. After the substitution of 123 for *n*, the code to be executed is equivalent to

```
if (123 < 10)
{
    System.out.println(123);
}
else //n is two or more digits long:
{
    writeVertical(123 / 10);
    System.out.println(123 % 10);
}
```

Computation will stop here
until the recursive call
returns.

Because 123 is not less than 10, the *else* part is executed. However, the *else* part begins with the method call

```
writeVertical(n / 10);
```

which (because n is equal to 123) is the call

```
writeVertical(123 / 10);
```

which is equivalent to

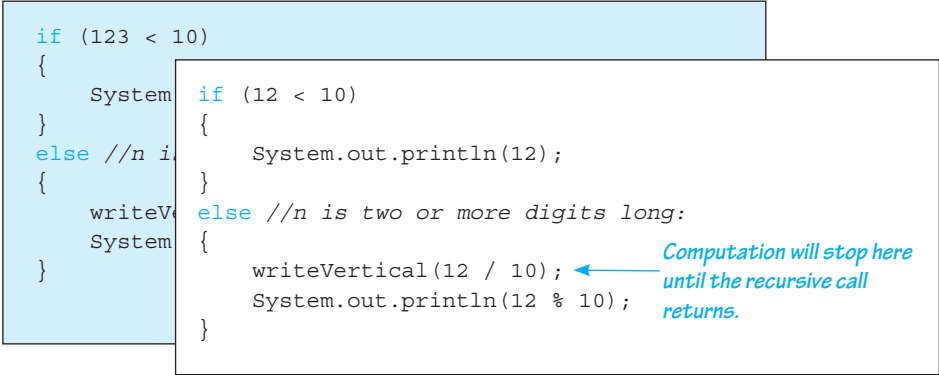
```
writeVertical(12);
```

When execution reaches this recursive call, the current method computation is placed in suspended animation, and this recursive call is executed. When this recursive call is finished, the execution of the suspended computation will return to this point, and the suspended computation will continue from this point.

The recursive call

```
writeVertical(12);
```

is handled just like any other method call. The argument 12 is substituted for the parameter n , and the body of the method is executed. After substituting 12 for n , there are two computations, one suspended and one active, as follows:



```

if (123 < 10)
{
    System.out.println(123);
}
else //n is two or more digits long:
{
    writeVertical(123 / 10);
    System.out.println(123 % 10);
}

if (12 < 10)
{
    System.out.println(12);
}
else //n is two or more digits long:
{
    writeVertical(12 / 10);
    System.out.println(12 % 10);
}

```

Computation will stop here until the recursive call returns.

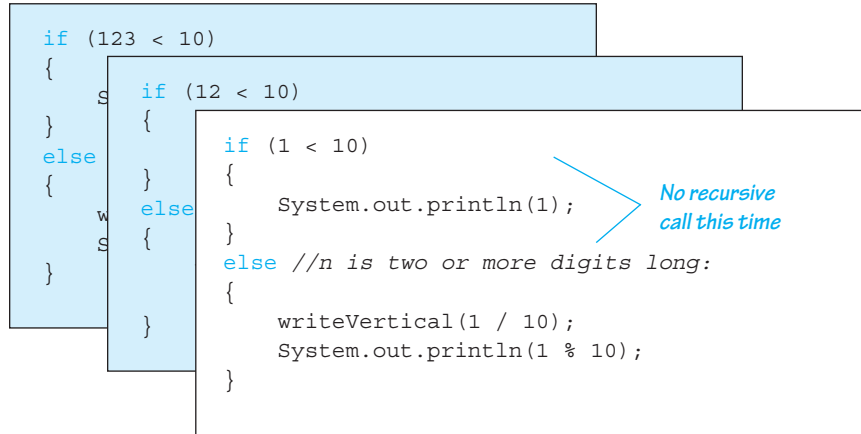
Because 12 is not less than 10, the `else` part is executed. However, as you already saw, the `else` part begins with a recursive call. The argument for the recursive call is $n/10$, which in this case is equivalent to $12/10$. So, this second computation of the method `writeVertical` is suspended, and the following recursive call is executed:

```
writeVertical(12 / 10);
```

which is equivalent to

```
writeVertical(1);
```

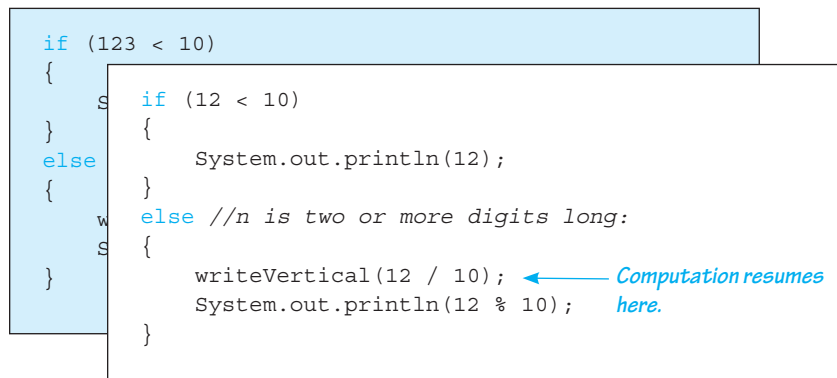
At this point, there are two suspended computations waiting to resume, and the computer begins to execute this new recursive call, which is handled just like all the previous recursive calls. The argument 1 is substituted for the parameter *n*, and the body of the method is executed. At this point, the computation looks like the following:



output the
digit 1

When the body of the method is executed this time, something different happens. Because 1 is less than 10, the Boolean expression in the if-else statement is true, so the statement before the else is executed. That statement is simply an output statement that writes the argument 1 to the screen, so the call `writeVertical(1)` writes 1 to the screen and ends without any recursive call.

When the call `writeVertical(1)` ends, the suspended computation that is waiting for it to end resumes where that suspended computation left off, as shown by the following:



output the
digit 2

When this suspended computation resumes, it executes an output statement that outputs the value $12 \% 10$, which is 2. That ends that computation, but there is yet another suspended computation waiting to resume. When this last suspended computation resumes, the situation is

```

if (123 < 10)
{
    System.out.println(123);
}
else //n is two or more digits long:
{
    writeVertical(123 / 10); ← Computation resumes
    System.out.println(123 % 10); here.
}

```

output the
digit 3

When this last suspended computation resumes, it outputs the value $123 \% 10$, which is 3, and the execution of the original method call ends. And, sure enough, the digits 1, 2, and 3 have been written to the screen one per line, in that order.

A Closer Look at Recursion

The definition of the method `writeVertical` uses recursion. Yet, we did nothing new or different in evaluating the method call `writeVertical(123)`. We treated it just like any of the method calls we saw in previous chapters. We simply substituted the argument 123 for the parameter `n` and then executed the code in the body of the method definition. When we reached the recursive call

```
writeVertical(123 / 10)
```

we simply repeated this process one more time.

how
recursion
works

The computer keeps track of recursive calls in the following way. When a method is called, the computer plugs in the arguments for the parameter(s) and begins to execute the code. If it should encounter a recursive call, then it temporarily stops its computation, because it must know the result of the recursive call before it can proceed. It saves all the information it needs to continue the computation later on, and proceeds to evaluate the recursive call. When the recursive call is completed, the computer returns to finish the outer computation.

how
recursion
ends

The Java language places no restrictions on how recursive calls are used in method definitions. However, in order for a recursive method definition to be useful, it must be designed so that any call of the method must ultimately terminate with some piece of code that does not depend on recursion. The method may call itself, and that recursive call may call the method again. The process may be repeated any number of

times. However, the process will not terminate unless eventually one of the recursive calls does not depend on recursion to return a value. The general outline of a successful recursive method definition is as follows:

base case
stopping
case

- One or more cases in which the method accomplishes its task by using recursive call(s) to accomplish one or more smaller versions of the task.
- One or more cases in which the method accomplishes its task without the use of any recursive calls. These cases without any recursive calls are called **base cases** or **stopping cases**.

Often an if-else statement determines which of the cases will be executed. A typical scenario is for the original method call to execute a case that includes a recursive call. That recursive call may in turn execute a case that requires another recursive call. For some number of times, each recursive call produces another recursive call, but eventually one of the stopping cases should apply. *Every call of the method must eventually lead to a stopping case, or else the method call will never end because of an infinite chain of recursive calls.* (In practice, a call that includes an infinite chain of recursive calls will usually terminate abnormally rather than actually running forever.)

The most common way to ensure that a stopping case is eventually reached is to write the method so that some (positive) numeric quantity is decreased on each recursive call and to provide a stopping case for some “small” value. This is how we designed the method `writeVertical` in Display 11.1. When the method `writeVertical` is called, that call produces a recursive call with a smaller argument. This continues with each recursive call producing another recursive call until the argument is less than 10. When the argument is less than 10, the method call ends without producing any more recursive calls, the process works its way back to the original call, and the process ends.

General Form of a Recursive Method Definition

The general outline of a successful recursive method definition is as follows:

- One or more cases that include one or more recursive calls to the method being defined. These recursive calls should solve “smaller” versions of the task performed by the method being defined.
- One or more cases that include no recursive calls. These cases without any recursive calls are called **base cases** or **stopping cases**.



PITFALL: Infinite Recursion

In the example of the method `writeVertical` discussed in the previous subsections, the series of recursive calls eventually reached a call of the method that did not involve recursion (that is, a stopping case was reached). If, on the other hand, every recursive call produces another recursive call, then a call to the method will, in theory, run forever. This is called **infinite recursion**. In practice, such a method will typically run until the computer runs out of resources, and the program terminates abnormally.

Examples of infinite recursion are not hard to come by. The following is a syntactically correct Java method definition, which might result from an attempt to define an alternative version of the method `writeVertical`:

```
public static void newWriteVertical(int n)
{
    newWriteVertical(n / 10);
    System.out.println(n % 10);
}
```

If you embed this definition in a program that calls this method, the program will compile with no error messages, and you can run the program. Moreover, the definition even has a certain reasonableness to it. It says that to output the argument to `newWriteVertical`, first output all but the last digit and then output the last digit. However, when called, this method will produce an infinite sequence of recursive calls. If you call `newWriteVertical(12)`, that execution will stop to execute the recursive call `newWriteVertical(12/10)`, which is equivalent to `newWriteVertical(1)`. The execution of that recursive call will, in turn, stop to execute the recursive call

```
newWriteVertical(1 / 10);
```

which is equivalent to

```
newWriteVertical(0);
```

This, in turn, will stop to execute the recursive call `newWriteVertical(0 / 10)`; which is also equivalent to

```
newWriteVertical(0);
```

This will produce another recursive call to again execute the same recursive method call `newWriteVertical(0)`; and so on, forever. Because the definition of `newWriteVertical` has no stopping case, the process will proceed forever (or until the computer runs out of resources). ■

Self-Test Exercises

1. What is the output of the following program?

```
public class Exercisel
{
    public static void main(String[] args)
    {
        cheers(3);
    }
    public static void cheers(int n)
    {
        if (n == 1)
        {
            System.out.println("Hurray");
        }
        else
        {
            System.out.println("Hip");
            cheers(n - 1);
        }
    }
}
```

2. Write a recursive void method that has one parameter that is an integer and that writes to the screen the number of asterisks '*' given by the argument. The output should be all on one line. You can assume the argument is positive.
3. Write a recursive void method that has one parameter, which is a positive integer. When called, the method writes its argument to the screen backward. That is, if the argument is 1234, it outputs the following to the screen:
4321
4. Write a recursive void method that takes a single (positive) int argument n and writes the integers 1, 2, ..., n to the screen.
5. Write a recursive void method that takes a single (positive) int argument n and writes integers n, n-1, ..., 3, 2, 1 to the screen. *Hint:* The solution for Self-Test Exercise 4 and this exercise vary by an exchange of as little as two lines.

Stacks for Recursion ★

stack

To keep track of recursion, and a number of other things, most computer systems use a structure called a *stack*. A **stack** is a very specialized kind of memory structure that is analogous to a stack of paper. In this analogy, there is an inexhaustible supply of extra blank sheets of paper. To place some information in the stack, it is written on one of these sheets of paper and placed on top of the stack of papers. To place more

information in the stack, a clean sheet of paper is taken, the information is written on it, and this new sheet of paper is placed on top of the stack. In this straightforward way, more and more information may be placed on the stack.

Getting information out of the stack is also accomplished by a very simple procedure. The top sheet of paper can be read, and when it is no longer needed, it is thrown away. There is one complication: Only the top sheet of paper is accessible. In order to read, say, the third sheet from the top, the top two sheets must be thrown away. Because the last sheet that is put on the stack is the first sheet taken off the stack, a stack is often called a **last-in/first-out** memory structure, abbreviated as **LIFO**.

Using a stack, the computer can easily keep track of recursion. Whenever a method is called, a new sheet of paper is taken. The method definition is copied onto this sheet of paper, and the arguments are plugged for the method parameters. Then the computer starts to execute the body of the method definition. When it encounters a recursive call, it stops the computation it is doing on that sheet in order to compute the value returned by the recursive call. But, before computing the recursive call, it saves enough information so that, when it does finally determine the value returned by the recursive call, it can continue the stopped computation. This saved information is written on a sheet of paper and placed on the stack. A new sheet of paper is used for the recursive call. The computer writes a second copy of the method definition on this new sheet of paper, plugs in the arguments for the method parameters, and starts to execute the recursive call. When it gets to a recursive call within the recursively called copy, it repeats the process of saving information on the stack and using a new sheet of paper for the new recursive call. This process is illustrated in the earlier subsection entitled “Tracing a Recursive Call.” Even though we did not call it a stack at the time, the illustrations of computations placed one on top of the other illustrate the actions of the stack.

This process continues until some recursive call to the method completes its computation without producing any more recursive calls. When this happens, the computer turns its attention to the top sheet of paper on the stack. This sheet contains the partially completed computation that is waiting for the recursive computation that just ended. So, it is possible to proceed with that suspended computation. When that suspended computation ends, the computer discards that sheet of paper, and the suspended computation that is below it on the stack becomes the computation on top of the stack. The computer turns its attention to the suspended computation that is now on the top of the stack, and so forth. The process continues until the computation on the bottom sheet is completed. Depending on how many recursive calls are made and how the method definition is written, the stack may grow and shrink in any fashion. Notice that the sheets in the stack can only be accessed in a last-in/first-out fashion—but, this is exactly what is needed to keep track of recursive calls. Each suspended version is waiting for the completion of the version directly above it on the stack.

Of course, computers do not have stacks of paper. This is just an analogy. The computer uses portions of memory rather than pieces of paper. The contents of one of these portions of memory (“sheets of paper”) is called a **stack frame** or **activation record**. These stack frames are handled in the last-in/first-out manner we just discussed.

last-in/
first-out

stack frame

activation
record

(These stack frames do not contain a complete copy of the method definition, but merely reference a single copy of the method definition. However, a stack frame contains enough information to allow the computer to act as if the stack frame contains a complete copy of the method definition.)



VideoNote
Recursion and
the Stack

Stack ★

A **stack** is a last-in/first-out memory structure. The first item referenced or removed from a stack is always the last item entered into the stack. Stacks are used by computers to keep track of recursion (and for other purposes).



PITFALL: Stack Overflow ★

There is always some limit to the size of the stack. If there is a long chain in which a method makes a recursive call to itself, and that call results in another recursive call, and that call produces yet another recursive call, and so forth, then each recursive call in this chain will cause another suspended computation to be placed on the stack. If this chain is too long, then the stack will attempt to grow beyond its limit. This is an error condition known as a **stack overflow**. If you receive an error message that says *stack overflow*, it is likely that some method call has produced an excessively long chain of recursive calls. One common cause of stack overflow is infinite recursion. If a method is recursing infinitely, then it will eventually try to make the stack exceed any stack size limit. ■

Recursion versus Iteration

Recursion is not absolutely necessary. In fact, some programming languages do not allow it. Any task that can be accomplished using recursion can also be done in some other way without using recursion. For example, Display 11.2 contains a nonrecursive version of the method given in Display 11.1. The nonrecursive version of a method typically uses a loop (or loops) of some sort in place of recursion. For this reason, the nonrecursive version is usually referred to as an **iterative version**. If the definition of the method `writeVertical` given in Display 11.1 is replaced by the version given in Display 11.2, then the output will be the same. As is true in this case, a recursive version of a method can sometimes be much simpler than an iterative version. The full program with the iterative version of the method is given in the file `IterativeDemo1` on the accompanying website.

A recursively written method will usually run slower and use more storage than an equivalent iterative version. The computer must do extra work manipulating the stack to keep track of the recursion. However, because the system does all this for you automatically, using recursion can sometimes make your job as a programmer easier and can sometimes produce code that is easier to understand. Additionally, there are some cases in which the compiler or JVM can convert a recursive algorithm into an iterative version for you.

iterative
version

extra code on
website

Display 11.2 Iterative Version of the Method in Display 11.1

```
1  public static void writeVertical(int n)
2  {
3      int nsTens = 1;
4      int leftEndPiece = n;
5      while (leftEndPiece > 9)
6      {
7          leftEndPiece = leftEndPiece / 10;
8          nsTens = nsTens * 10;
9      }
10     //nsTens is a power of 10 that has the same number
11     //of digits as n. For example, if n is 2345, then
12     //nsTens is 1000.
13
14     for (int powerOf10 = nsTens;
15          powerOf10 > 0; powerOf10 = powerOf10 / 10)
16     {
17         System.out.println(n / powerOf10);
18         n = n % powerOf10;
19     }
```

Self-Test Exercises

6. If your program produces an error message that says *stack overflow*, what is a likely source of the error?
7. Write an iterative version of the method `cheers` defined in Self-Test Exercise 1.
8. Write an iterative version of the method defined in Self-Test Exercise 2.
9. Write an iterative version of the method defined in Self-Test Exercise 3.
10. Trace the recursive solution you made to Self-Test Exercise 4.
11. Trace the recursive solution you made to Self-Test Exercise 5.

11.2 Recursive Methods That Return a Value

To iterate is human, to recurse divine.

L. PETER DEUTSCH

General Form for a Recursive Method That Returns a Value

The recursive methods you have seen thus far are all `void` methods, but recursion is not limited to these methods. A recursive method can return a value of any type. The technique for designing recursive methods that return a value is basically the same as what you learned for `void` methods. An outline for a successful recursive method definition that returns a value is as follows:

- One or more cases in which the value returned is computed in terms of calls to the same method (that is, using recursive calls). As is the case with `void` methods, the arguments for the recursive calls should intuitively be “smaller.”
- One or more cases in which the value returned is computed without the use of any recursive calls. These cases without any recursive calls are called **base cases** or **stopping cases** (just as they were with `void` methods).

This technique is illustrated in the next Programming Example.

EXAMPLE: Another Powers Method

In Chapter 5, we introduced the static method `pow` of the class `Math`, that computes powers. For example, `Math.pow(2.0, 3.0)` returns $2.0^{3.0}$, so the following sets the variable `result` equal to 8.0:

```
double result = Math.pow(2.0, 3.0);
```

The method `pow` takes two arguments of type `double` and returns a value of type `double`. Display 11.3 contains a recursive definition for a static method that is similar to `pow`, but that works with the type `int` rather than `double`. This new method is called `power`. For example, the following will set the value of `result2` equal to 8, because 2^3 is 8:

```
int result2 = power(2, 3);
```

Outside the defining class, this would be written as

```
int result2 = RecursionDemo2.power(2, 3);
```

Our main reason for defining the method `power` is to have a simple example of a recursive method, but there are situations in which the method `power` would be preferable to the method `pow`. The method `pow` returns a value of type `double`, which is only an approximate quantity. The method `power` returns a value of type `int`, which is an exact quantity. In some situations, you might need the additional accuracy provided by the method `power`.

The definition of the method `power` is based on the following formula:

x^n is equal to $x^{n-1} * x$

EXAMPLE: (continued)

Translating this formula into Java says that the value returned by `power(x, n)` should be the same as the value of the expression

```
power(x, n - 1)*x
```

The definition of the method `power` given in Display 11.3 does return this value for `power(x, n)`, provided $n > 0$.

The case where n is equal to 0 is the stopping case. If n is 0, then `power(x, n)` simply returns 1 (because x^0 is 1).

Let's see what happens when the method `power` is called with some sample values. First, consider the simple expression:

```
power(2, 0)
```

When the method is called, the value of x is set equal to 2, the value of n is set equal to 0, and the code in the body of the method definition is executed. Because the value of n is a legal value, the `if-else` statement is executed. Because this value of n is not greater than 0, the `return` statement after the `else` is used, so the method call returns 1. Thus, the following would set the value of `result3` equal to 1:

```
int result3 = power(2, 0);
```

Now let's look at an example that involves a recursive call. Consider the expression

```
power(2, 1)
```

When the method is called, the value of x is set equal to 2, the value of n is set equal to 1, and the code in the body of the method definition is executed. Because this value of n is greater than 0, the following `return` statement is used to determine the value returned:

```
return ( power(x, n - 1)*x );
```

which in this case is equivalent to

```
return ( power(2, 0)*2 );
```

At this point, the computation of `power(2, 1)` is suspended, a copy of this suspended computation is placed on the stack, and the computer then starts a new method call to compute the value of `power(2, 0)`. As you have already seen, the value of `power(2, 0)` is 1. After determining the value of `power(2, 0)`, the computer replaces the expression

(continued)

EXAMPLE: (continued)

`power(2, 0)` with its value of 1 and resumes the suspended computation. The resumed computation determines the final value for `power(2, 1)` from the above return statement as

```
Power(2, 0)*2 is 1*2 which is 2
```

so the final value returned for `power(2, 1)` is 2. So, the following would set the value of `result4` equal to 2:

```
int result4 = power(2, 1);
```

Larger numbers for the second argument will produce longer chains of recursive calls. For example, consider the statement

```
System.out.println(power(2, 3));
```

The value of `power(2, 3)` is calculated as follows:

```
power(2, 3) is power(2, 2)*2
power(2, 2) is power(2, 1)*2
power(2, 1) is power(2, 0)*2
power(2, 0) is 1 (stopping case)
```

When the computer reaches the stopping case, `power(2, 0)`, there are three suspended computations. After calculating the value returned for the stopping case, it resumes the most recently suspended computations to determine the value of `power(2, 1)`. After that, the computer completes each of the other suspended computations, using each value computed as a value to plug into another suspended computation, until it reaches and completes the computation for the original call `power(2, 3)`. The details of the entire computation are illustrated in Display 11.4.

Display 11.3 The Recursive Method `power` (part 1 of 2)

```
1 public class RecursionDemo2
2 {
3     public static void main(String[] args)
4     {
5         for (int n = 0; n < 4; n++)
6             System.out.println("3 to the power " + n
7                               + " is " + power(3, n));
8     }
9
10    public static int power(int x, int n)
11    {
```

Display 11.3 The Recursive Method `power` (part 2 of 2)

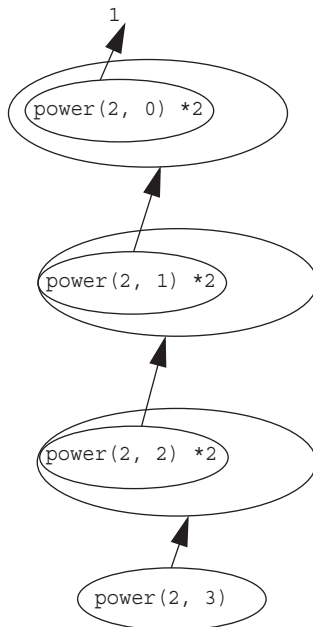
```
11         if (n < 0)
12         {
13             System.out.println("Illegal argument to power.");
14             System.exit(0);
15         }
16         if (n > 0)
17             return ( power(x, n - 1)*x );
18         else // n == 0
19             return (1);
20     }
21 }
```

Sample Dialogue

```
3 to the power 0 is 1
3 to the power 1 is 3
3 to the power 2 is 9
3 to the power 3 is 27
```

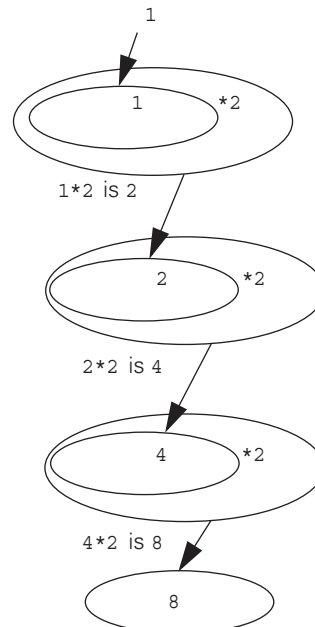
Display 11.4 Evaluating the Recursive Method Call `power(2, 3)`

SEQUENCE OF RECURSIVE CALLS:



Start here

HOW THE FINAL VALUE IS COMPUTED:



power(2, 3) is 8

Self-Test Exercises

12. What is the output of the following program?

```
public class Exercise12
{
    public static void main(String[] args)
    {
        System.out.println(mystery(3));
    }

    public static int mystery(int n)
    {
        if (n <= 1)
            return 1;
        else
            return ( mystery(n - 1) + n );
    }
}
```

13. What is the output of the following program? What well-known mathematical method is rose?

```
public class Exercise13
{
    public static void main(String[] args)
    {
        System.out.println(rose(4));
    }

    public static int rose(int n)
    {
        if (n <= 0)
            return 1;
        else
            return ( rose(n - 1) * n );
    }
}
```

14. Redefine the method `power` (Display 11.3) so that it also works for negative exponents. To do this, you also have to change the type of the value returned to `double`. The method heading for the redefined version of `power` is as follows:

```
/**
 * Precondition: If  $n < 0$ , then  $x$  is not 0.
 * Returns  $x$  to the power  $n$ .
 */
public static double power(int x, int n)
```

Hint: x^{-n} is equal to $1/(x^n)$.

11.3 Thinking Recursively

There are two kinds of people in the world, those who divide the world into two kinds of people and those who do not.

ROBERT BENCHLEY

Recursive Design Techniques

When defining and using recursive methods, you do not want to be continually aware of the stack and the suspended computations. The power of recursion comes from the fact that you can ignore that detail and let the computer do the bookkeeping for you. Consider the example of the method `power` in Display 11.3. The way to think of the definition of `power` is as follows:

`power(x, n)` returns `power(x, n - 1) * x`

Because x^n is equal to $x^{n-1} * x$, this is the correct value to return, provided that the computation will always reach a stopping case and will correctly compute the stopping case. So, after checking that the recursive part of the definition is correct, all you need to check is that the chain of recursive calls will always reach a stopping case and that the stopping case will always return the correct value. In other words, all that you need to do is check that the following three properties are satisfied:

criteria for
methods that
return a value

1. There is no infinite recursion. (A recursive call may lead to another recursive call, which may lead to another, and so forth, but every such chain of recursive calls eventually reaches a stopping case.)
2. Each stopping case returns the correct value for that case.
3. For the cases that involve recursion: *if* all recursive calls return the correct value, *then* the final value returned by the method is the correct value.

For example, consider the method `power` in Display 11.3:

1. **There is no infinite recursion:** The second argument to `power(x, n)` is decreased by one in each recursive call, so any chain of recursive calls must eventually reach the case `power(x, 0)`, which is the stopping case. Thus, there is no infinite recursion.
2. **Each stopping case returns the correct value for that case:** The only stopping case is `power(x, 0)`. A call of the form `power(x, 0)` always returns 1, and the correct value for x^0 is 1. So, the stopping case returns the correct value.
3. **For the cases that involve recursion, if all recursive calls return the correct value, then the final value returned by the method is the correct value:** The only case that involves recursion is when $n > 1$. When $n > 1$, `power(x, n)` returns

`power(x, n - 1) * x`.

To see that this is the correct value to return, note that, *if* `power(x, n - 1)` returns the correct value, *then* `power(x, n - 1)` returns x^{n-1} and so `power(x, n)` returns

$$x^{n-1} * x, \text{ which is } x^n$$

This is the correct value for `power(x, n)`.

That is all you need to check to be sure that the definition of `power` is correct. (The previous technique is known as *mathematical induction*, a concept that you may have heard about in a mathematics class. However, you do not need to be familiar with the term *mathematical induction* to use this technique.)

We gave you three criteria to use in checking the correctness of a recursive method that returns a value. Basically, the same rules can be applied to a recursive `void` method. If you show that your recursive `void` method definition satisfies the following three criteria, then you will know that your `void` method performs correctly:

criteria for void methods

1. There is no infinite recursion.
2. Each stopping case performs the correct action for that case.
3. For each of the cases that involve recursion, *if* all recursive calls perform their actions correctly, *then* the entire case performs correctly.

Binary Search ★

In this subsection, we will develop a recursive method that searches an array to find out whether it contains a specified value. For example, the array may contain a list of the numbers for credit cards that are no longer valid. A store clerk needs to search the list to see if a customer's card is valid or invalid.

The indices of the array `a` are the integers 0 through `finalIndex`. To make the task of searching the array easier, we will assume that the array is sorted. Hence, we know the following:

$$a[0] \leq a[1] \leq a[2] \leq \dots \leq a[\text{finalIndex}]$$

In fact, the binary search algorithm we will use requires that the array be sorted like this.

When searching an array, you are likely to want to know both whether the value is in the array and, if it is, where it is in the array. For example, if you are searching for a credit card number, then the array index may serve as a record number. Another array indexed by these same indices may hold a phone number or other information to use for reporting the suspicious card. Hence, if the sought-after value is in the array, we will have our method return an index of where the sought-after value is located. If the value is not in the array, our method will return `-1`. (The array may contain repeats, which is why we say “an index” and not “the index.”)

Now let us proceed to produce an algorithm to solve this task. It will help to visualize the problem in very concrete terms. Suppose the list of numbers is so long that it takes a book to list them all. This is in fact how invalid credit card numbers are distributed to stores that do not have access to computers. If you are a clerk and are

handed a credit card, you must check to see if it is on the list and hence invalid. How would you proceed? Open the book to the middle and see if the number is there. If it is not and it is smaller than the middle number, then work backward toward the beginning of the book. If the number is larger than the middle number, you work your way toward the end of the book. This idea produces our first draft of an algorithm:

algorithm first version

```
mid = approximate midpoint between 0 and finalIndex;
if (key == a[mid])
    return mid;
else if (key < a[mid])
    search a[0] through a[mid - 1];
else if (key > a[mid])
    search a[mid + 1] through a[finalIndex];
```

Because the searchings of the shorter lists are smaller versions of the very task we are designing the algorithm to perform, this algorithm naturally lends itself to the use of recursion. The smaller lists can be searched with recursive calls to the algorithm itself.

Our pseudocode is a bit too imprecise to be easily translated into Java code. The problem has to do with the recursive calls. There are two recursive calls shown:

```
search a[0] through a[mid - 1];
    and
search a[mid + 1] through a[finalIndex];
```

more parameters

To implement these recursive calls, we need two more parameters. A recursive call specifies that a subrange of the array is to be searched. In one case, it is the elements indexed by 0 through `mid - 1`. In the other case, it is the elements indexed by `mid + 1` through `finalIndex`. The two extra parameters will specify the first and last indices of the search, so we will call them *first* and *last*. Using these parameters for the lowest and highest indices, instead of 0 and `finalIndex`, we can express the pseudocode more precisely as follows:

algorithm first refinement

```
To search a[first] through a[last] do the following:
mid = approximate midpoint between first and last;
if (key == a[mid])
    return mid;
else if (key < a[mid])
    return the result of searching a[first] through a[mid - 1];
else if (key > a[mid])
    return the result of searching a[mid + 1] through a[last];
```

To search the entire array, the algorithm would be executed with `first` set equal to 0 and `last` set equal to `finalIndex`. The recursive calls will use other values for `first` and `last`. For example, the first recursive call would set `first` equal to 0 and `last` equal to the calculated value `mid - 1`.

stopping case

As with any recursive algorithm, we must ensure that our algorithm ends rather than producing infinite recursion. If the sought-after number is found on the list, then there

**algorithm
final version**

is no recursive call, and the process terminates, but we need some way to detect when the number is not on the list. On each recursive call, the value of `first` is increased or the value of `last` is decreased. If they ever pass each other and `first` actually becomes larger than `last`, then we will know that there are no more indices left to check and that the number `key` is not in the array. If we add this test to our pseudocode, we obtain a complete solution, as shown in Display 11.5.

Now we can routinely translate the pseudocode into Java code. The result is shown in Display 11.6. The method `search` is an implementation of the recursive algorithm given in Display 11.5. A diagram of how the method performs on a sample array is given in Display 11.7. Display 11.8 illustrates how the method `search` is used.

Notice that the method `search` solves a more general problem than the original task. Our goal was to design a method to search an entire array. Yet the method will let us search any interval of the array by specifying the indices `first` and `last`. This is common when designing recursive methods. Frequently, it is necessary to solve a more general problem in order to be able to express the recursive algorithm. In this case, we want only the answer in the case where `first` and `last` are set equal to 0 and `finalIndex`. However, the recursive calls will set them to values other than 0 and `finalIndex`.

Display 11.5 Pseudocode for Binary Search ★

ALGORITHM TO SEARCH `a[first]` THROUGH `a[last]`

```
/**
  Precondition:
  a[first] <= a[first + 1] <= a[first + 2] <= ... <= a[last]
  */
```

TO LOCATE THE VALUE `KEY`

```
if (first > last) //A stopping case
    return -1;
else
{
    mid = approximate midpoint between first and last;
    if (key == a[mid]) //A stopping case
        return mid;
    else if key < a[mid] //A case with recursion
        return the result of searching a[first] through a[mid - 1];
    else if key > a[mid] //A case with recursion
        return the result of searching a[mid + 1] through a[last];
}
```

Display 11.6 Recursive Method for Binary Search ★

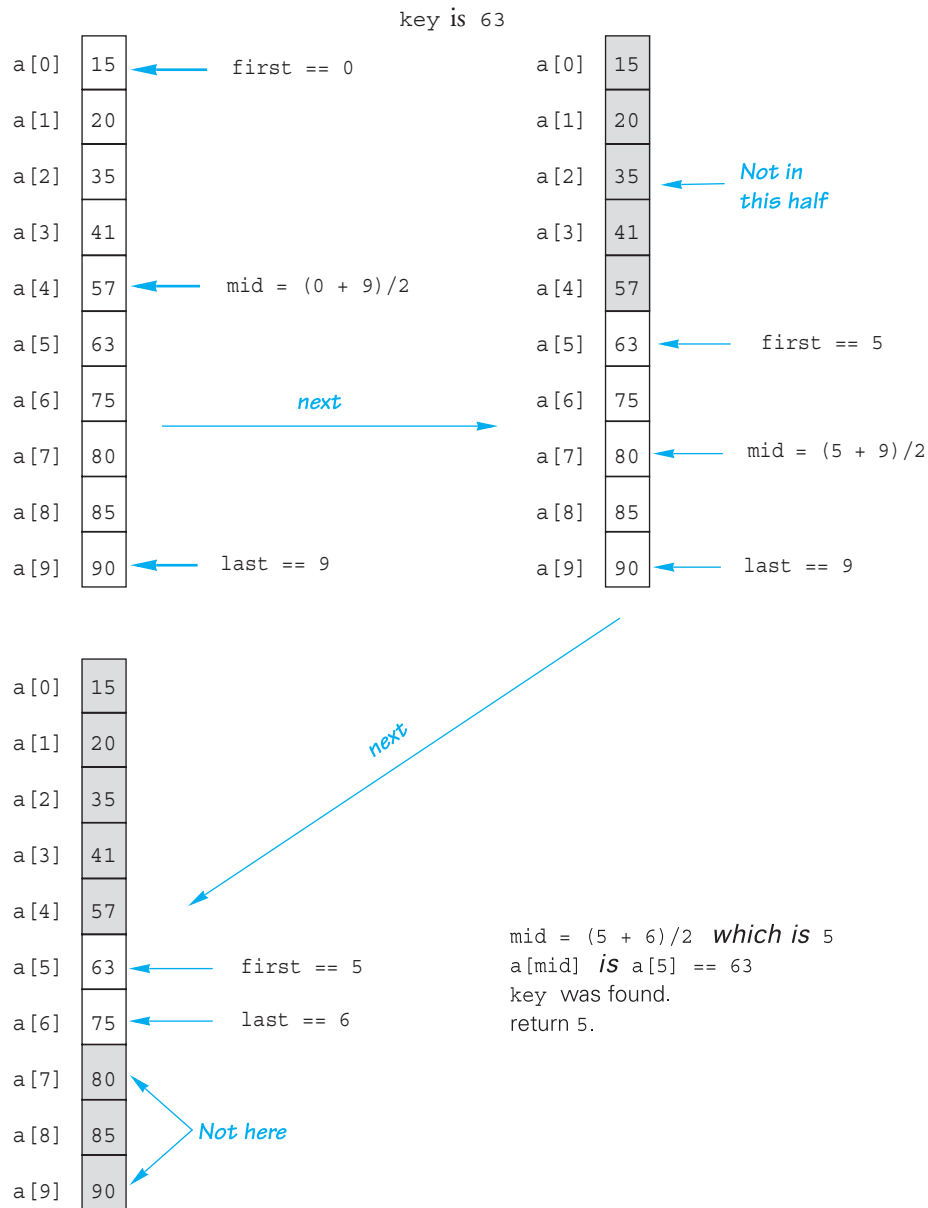
```

1  public class BinarySearch
2  {
3      /**
4       * Searches the array a for key. If key is not in the array segment,
5       * then -1 is returned. Otherwise returns an index in the segment such
6       * that key == a[index].
7       * Precondition: a[first] <= a[first + 1] <= ... <= a[last]
8       */
9      public static int search(int[] a, int first, int last, int key)
10     {
11         int result = 0; //to keep the compiler happy.
12
13         if (first > last)
14             result = -1;
15         else
16         {
17             int mid = (first + last)/2;
18
19             if (key == a[mid])
20                 result = mid;
21             else if (key < a[mid])
22                 result = search(a, first, mid - 1, key);
23             else if (key > a[mid])
24                 result = search(a, mid + 1, last, key);
25         }
26         return result;
27     }
28 }

```

In the earlier subsection entitled “Tracing a Recursive Call,” we gave three criteria that you should check to ensure that a recursive void method definition is correct. Let’s check these three things for the method `search` given in Display 11.6:

1. **There is no infinite recursion:** On each recursive call, the value of `first` is increased or the value of `last` is decreased. If the chain of recursive calls does not end in some other way, then eventually the method will be called with `first` larger than `last`, which is a stopping case.
2. **Each stopping case performs the correct action for that case:** There are two stopping cases, when `first > last` and when `key == a[mid]`. Let’s consider each case.
 If `first > last`, there are no array elements between `a[first]` and `a[last]`, so `key` is not in this segment of the array. (Nothing is in this segment of the array!) So, if `first > last`, the method `search` correctly returns `-1`, indicating that `key` is not in the specified range of the array.
 If `key == a[mid]`, the algorithm correctly sets `location` equal to `mid`. Thus, both stopping cases are correct.

Display 11.7 Execution of the Method `search` ★

Display 11.8 Using the `search` Method ★

```
1  public class BinarySearchDemo
2  {
3      public static void main(String[] args)
4      {
5          int[] a = {-2, 0, 2, 4, 6, 8, 10, 12, 14, 16};
6          int finalIndex = 9;

7          System.out.println("Array contains:");
8          for (int i = 0; i < a.length; i++)
9              System.out.print(a[i] + " ");
10         System.out.println();
11         System.out.println();

12         int result;
13         for (int key = -3; key < 5; key++)
14         {
15             result = BinarySearch.search(a, 0, finalIndex, key);
16             if (result >= 0)
17                 System.out.println(key + " is at index " + result);
18             else
19                 System.out.println(key + " is not in the array.");
20         }
21     }
22 }
```

Sample Dialogue

```
Array contains:
-2 0 2 4 6 8 10 12 14 16

-3 is not in the array.
-2 is at index 0
-1 is not in the array.
0 is at index 1
1 is not in the array.
2 is at index 2
3 is not in the array.
4 is at index 3
```

3. **For each of the cases that involve recursion, *if* all recursive calls perform their actions correctly, *then* the entire case performs correctly:** There are two cases in which there are recursive calls, when `key < a[mid]` and when `key > a[mid]`. We need to check each of these two cases.

First, suppose `key < a[mid]`. In this case, because the array is sorted, we know that if `key` is anywhere in the array, then `key` is one of the elements `a[first]` through `a[mid - 1]`. Thus, the method need only search these elements, which is exactly what the recursive call

```
search(a, first, mid - 1, key)
```

does. So if the recursive call is correct, then the entire action is correct.

Next, suppose `key > a[mid]`. In this case, because the array is sorted, we know that if `key` is anywhere in the array, then `key` is one of the elements `a[mid + 1]` through `a[last]`. Thus, the method need only search these elements, which is exactly what the recursive call

```
search(a, mid + 1, last, key)
```

does. So if the recursive call is correct, then the entire action is correct. Thus, in both cases, the method performs the correct action (assuming that the recursive calls perform the correct action).

The method `search` passes all three of our tests, so it is a good recursive method definition.

Efficiency of Binary Search ★

The binary search algorithm is extremely fast compared to an algorithm that simply tries all array elements in order. In the binary search, you eliminate about half the array from consideration right at the start. You then eliminate a quarter, then an eighth of the array, and so forth. These savings add up to a dramatically fast algorithm. For an array of 100 elements, the binary search will never need to compare more than 7 array elements to the key. A serial search could compare as many as 100 array elements to the key, and on the average will compare about 50 array elements to the key. Moreover, the larger the array is, the more dramatic the savings will be. On an array with 1,000 elements, the binary search will only need to compare about 10 array elements to the key value, as compared to an average of 500 for the serial search algorithm.¹

An iterative version of the method `search` is given in Display 11.9. On some systems, the iterative version will run more efficiently than the recursive version. The algorithm for the iterative version was derived by mirroring the recursive version. In the iterative version, the local variables `first` and `last` mirror the roles of the parameters in the recursive version, which are also named `first` and `last`. As this example

iterative
version

¹The binary search algorithm has worst-case running time that is logarithmic—that is, $O(\log n)$. The serial search algorithm is linear—that is, $O(n)$. If the terms used in this footnote are not familiar to you, you can safely ignore it.

Display 11.9 Iterative Version of Binary Search ★

```
1  /**
2   Searches the array a for key. If key is not in the array segment, then
3   -1 is returned. Otherwise returns an index in the segment such that key
4   == a[index].
5   Precondition: a [lowEnd] <= a[lowEnd + 1] <= ... <= a[highEnd]
6   */
7  public static int search(int[] a, int lowEnd, int highEnd, int key)
8  {
9      int first = lowEnd;
10     int last = highEnd;
11     int mid;
12
13     boolean found = false; //so far
14     int result = 0; //to keep compiler happy
15
16     while ( (first <= last) && !(found) )
17     {
18         mid = (first + last)/2;
19
20         if (key == a[mid])
21         {
22             found = true;
23             result = mid;
24         }
25         else if (key < a[mid])
26         {
27             last = mid - 1;
28         }
29         else if (key > a[mid])
30         {
31             first = mid + 1;
32         }
33     }
34
35     if (first > last)
36         result = -1;
37     return result;
38 }
```

[extra code
on website](#)

illustrates, it often makes sense to derive a recursive algorithm even if you expect to later convert it to an iterative algorithm. You can see the iterative method from Display 11.9 embedded in a full demonstration in the files `IterativeBinarySearch.java` and `IterativeBinarySearchDemo.java` on the accompanying website.

**tail
recursion**

Most modern compilers will convert certain simple recursive method definitions to iterative ones before running the program. A method that uses **tail recursion** has the property that it does nothing after the recursive call except return the method's value. In this case, a tail recursive method can be easily converted to an equivalent iterative solution. This operation may be performed by the compiler or by the JVM.

Self-Test Exercise

15. Write a recursive method definition for the following method:

```
/**
 * Precondition: n >= 1
 * Returns the sum of the squares of the numbers 1 through n.
 */
public static int squares(int n)
```

For example, `squares(3)` returns 14 because $1^2 + 2^2 + 3^2$ is 14.

EXAMPLE: Finding a File

The next program is an example where a recursive solution is much easier to write and understand than an iterative solution. Consider the problem of finding a file buried somewhere in your file system. For example, using the Windows file system, let's say that you have the following file and directory structure on your hard drive:

Display 11.10 Sample File System Structure

```
C:\
  JavaPrograms\
    Recursion\
      FindFile.java
      BinarySearch.java
    Homework\
      Homework1.java
      Homework2.java
    Test.java
  Papers\
    TermPaper.odt
    Workfile.docx
  Letter.txt
```

We would like to find the location of a file given its name. For example, given `TermPaper.odt`, we would like to know that it is located in `C:\Papers`. Given `FindFile.java`, we would like to know that it is located in `C:\JavaPrograms\Recursion`. The general solution is to start at some root directory, such as `C:\`, and to go through all the items in that directory. If an item is a file, check if it matches the target. If an item is a directory, then make a recursive call to restart the search using that directory as the new root directory.

Pseudocode of our recursive solution follows:

```
searchForFile(currentPath, targetFile)
    if currentPath is not a directory
        return error
    for every item i in currentPath
        if i is a directory
            r = searchForFile(i, targetFile)
            if r is a successful match
                return r
        if i is a file
            if i matches targetFile
                return path to file i
    return target not found
```

To implement our algorithm, we need a way to determine if a path is a directory or a file, and we need to find a way to get all of the items within a directory. Java's `File` object will do all of this for us. To use it, we must import `java.io.File`. Here are the relevant constructor and methods:

CONSTRUCTORS AND METHODS	DESCRIPTION
<code>File(String pathname)</code>	The constructor takes a pathname and creates a <code>File</code> object corresponding to the file or directory with that pathname.
<code>String getAbsolutePath()</code>	Returns the pathname of the <code>File</code> object, e.g., <code>C:\Papers\TermPaper.odt</code> .
<code>String getName()</code>	Returns the name of the <code>File</code> object, e.g., <code>TermPaper.odt</code> .
<code>boolean isDirectory()</code>	Returns <code>true</code> if the <code>File</code> object is a directory.
<code>File[] listFiles()</code>	If the <code>File</code> object is a directory then this returns an array of <code>File</code> objects corresponding to all the items within the directory.

Using the `File` object, we can implement the recursive algorithm to find a file. The implementation in Display 11.11 returns an empty string if the target file is not found. If the initial root folder supplied is not a directory, then an error message is returned. The sample results assume the program runs using the directory structure given in Display 11.10.

Display 11.11 Program to Recursively Find a File

```
1  import java.io.File;
2  public class FindFile
3  {
4      public static String searchForFile(File dir, String target)
5      {
6          String result = "";
7          // If dir is not a directory, return
8          if (!dir.isDirectory())
9              return "Path is not a directory.";
10
11         // Check each item in the directory
12         for (File folderItem : dir.listFiles())
13         {
14             // Recurse if it's a directory
15             if (folderItem.isDirectory())
16             {
17                 result = searchForFile(folderItem,target);
18                 // Return the result if it is not empty
19                 if (!result.equals(""))
20                     return result;
21             }
22             // If it's a file, check for a match
23             else
24             {
25                 if (folderItem.getName().equals(target))
26                     return folderItem.getAbsolutePath();
27             }
28         }
29         // If we got here, nothing was found
30         return "";
31     }
32
33     public static void main(String[] args)
34     {
35         // The root folder to search
36         File rootFolder = new File("C:\\");
37         String result = searchForFile(rootFolder, "FindFile.java");
38         if (!result.equals(""))
39             System.out.println(result);
40         else
41             System.out.println("File not found.");
42     }
43 }
```

Sample Dialogue Using Directory Structure of Display 11.10

```
C:\JavaPrograms\Recursion\FindFile.java
```

Self-Test Exercises

16. How might you write a nonrecursive version of the program in Display 11.11? You do not have to write actual code, just think of what approach you might use.
17. The program in Display 11.11 could make thousands of recursive calls if you have a lot of subdirectories on your hard drive. Why is it unlikely that you will encounter a stack overflow error?
18. What is the stopping case in Display 11.11?

Chapter Summary

- If a problem can be reduced to smaller instances of the same problem, then a recursive solution is likely to be easy to find and implement.
- A recursive algorithm for a method definition normally contains two kinds of cases: one or more cases that include at least one recursive call and one or more stopping cases in which the problem is solved without any recursive calls.
- When writing a recursive method definition, always check to see that the method will not produce infinite recursion.
- When you define a recursive method, use the three criteria given in the subsection “Recursive Design Techniques” to check that the method is correct.
- When you design a recursive method to solve a task, it is often necessary to solve a more general problem than the given task. This may be required to allow for the proper recursive calls, because the smaller problems may not be exactly the same problem as the given task. For example, in the binary search problem, the task was to search an entire array, but the recursive solution is an algorithm to search any portion of the array (either all of it or a part of it).

Answers to Self-Test Exercises

1. Hip Hip Hurray
2.

```
public static void stars(int n)
{
    System.out.print('*');
    if (n > 1)
        stars(n - 1);
}
```

The following answer to Self-Test Exercise 3 is also correct, but is more complicated.


```
3. public static void stars(int n)
{
    if (n <= 1)
    {
        System.out.print('*');
    }
    else
    {
        stars(n - 1);
        System.out.print('*');
    }
}

public static void backward(int n)
{
    if (n < 10)
    {
        System.out.print(n);
    }
    else
    {
        System.out.print(n%10); //write last digit
        backward(n/10); //write the other digits backward
    }
}

4. public static void writeUp(int n)
{
    if (n >= 1)
    {
        writeUp(n - 1);
        System.out.print(n + " "); //write while the
                                   //recursion unwinds
    }
}

5. public static void writeDown(int n)
{
    if (n >= 1)
    {
        System.out.print(n + " "); //write while the
                                   //recursion winds
        writeDown(n - 1);
    }
}
```

6. An error message that says *stack overflow* is telling you that the computer has attempted to place more stack frames on the stack than are allowed on your system. A likely cause of this error message is infinite recursion.

7.

```
public static void cheers(int n)
{
    while (n > 1)
    {
        System.out.print("Hip ");
        n--;
    }
    System.out.println("Hurray");
}
```
8.

```
public static void stars(int n)
{
    for (int count = 1; count <= n; count++)
        System.out.print('*');
}
```
9.

```
public static void backward(int n)
{
    while (n >= 10)
    {
        System.out.print(n%10); //write last digit
        n = n/10; //discard the last digit
    }
    System.out.print(n);
}
```

10. The trace for Self-Test Exercise 4: If $n = 3$, the code to be executed is

```
if (3 >= 1)
{
    writeUp(2);
    System.out.print(3 + " ");
}
```

The execution is suspended before the `System.out.println`. On the next recursion, $n = 2$; the code to be executed is

```
if (2 >= 1)
{
    writeUp(1);
    System.out.print(2 + " ");
}
```

The execution is suspended before the `System.out.println`. On the next recursion, $n = 1$ and the code to be executed is

```
if (1 >= 1)
{
    writeUp(0);
    System.out.print(1 + " ");
}
```

The execution is suspended before the `System.out.println`. On the final recursion, $n = 0$ and the code to be executed is

```
if (0 >= 1) // condition false, body skipped
{
    // skipped
}
```

The suspended computations are completed from the most recent to the least recent. The output is 1 2 3.

11. The trace for Self-Test Exercise 5: If $n = 3$, the code to be executed is

```
if (3 >= 1)
{
    System.out.print(3 + " ");
    writeDown(2);
}
```

Next recursion, $n = 2$, the code to be executed is

```
if (2 >= 1)
{
    System.out.print(2 + " ");
    writeDown(1);
}
```

Next recursion, $n = 1$, the code to be executed is

```
if (1 >= 1)
{
    System.out.print(1 + " ");
    writeDown(0);
}
```

Final recursion, $n = 0$, and the `if` statement does nothing, ending the recursive calls:

```
if (0 >= 1) // condition false
{
    // this clause is skipped
}
```

The output is 3 2 1.

12. 6

13. The output is 24. The method `rose` is the factorial method, usually written as $n!$ and defined as follows:

$n!$ is equal to $n*(n-1)*(n-2)*\dots*1$

```

14. public static double power(int x, int n)
    {
        if (n < 0 && x == 0)
        {
            System.out.println(
                "Illegal argument to power.");
            System.exit(0);
        }

        if (n < 0)
            return ( 1/power(x, - n));
        else if (n > 0)
            return ( power(x, n - 1)*x );
        else // n == 0
            return (1.0);
    }

```

```

15. public static int squares(int n)
    {
        if (n <= 1)
            return 1;
        else
            return ( squares(n - 1) + n*n );
    }

```

16. One approach is to keep a list of all directories that have been encountered. The list could be implemented with an array. Initially, this list would be set to the root directory. While there is at least one directory on the list, repeat the following:

- Remove a directory from the list.
 - Find all files within the directory. If one of these files matches the target, return the pathname to the file.
 - Find all subdirectories within the directory. Add each subdirectory to the list.

This approach is a little more complicated than the recursive version because we have to manage the list. In the recursive version, the list is essentially managed for us through the series of recursive calls.

17. A stack overflow in the context of recursion occurs when there is a long chain of recursive calls. In the FindFile program, one link of this chain is created when a subdirectory is in another directory. To create a stack overflow, we would need to have a folder in a folder, which is in a folder, which is in a folder, etc. How many folders must be linked in this way to cause a stack overflow will vary from one system to another, but you can expect hundreds would be necessary, an unlikely scenario in a typical file system. Although there could be thousands of recursive calls when searching for a file, there will likely be many short chains instead of single long chains. This allows recursive calls to exit and prevent a stack overflow.
18. Recursion stops if the file passed in is not a directory, if there are no subdirectories in the directory, or after all subdirectories have been recursively called. If a match is found, then recursion also stops.

Programming Projects

1. A savings account typically accrues savings using compound interest. If you deposit \$1,000 with a 10% interest rate per year, then after one year you have a total of \$1,100. If you leave this money in the account for another year at 10% interest, then after two years the total will be \$1,210. After three years, you would have \$1,331, and so on.

Write a program that inputs the amount of money to deposit, an interest rate per year, and the number of years the money will accrue compound interest. Write a recursive function that calculates the amount of money that will be in the savings account using the input information.

To verify your function, the amount should be equal to $P(1 + i)^n$, where P is the amount initially saved, i is the interest rate per year, and n is the number of years.

2. There are n people in a room, where n is an integer greater than or equal to 1. Each person shakes hands once with every other person. What is the total number, $h(n)$, of handshakes? Write a recursive function to solve this problem. To get you started, if there are only one or two people in the room, then

`handshake(1) = 0`

`handshake(2) = 1`

If a third person enters the room, he or she must shake hands with each of the two people already there. This is two handshakes in addition to the number of handshakes that would be made in a room of two people, or a total of three handshakes.

If a fourth person enters the room, he or she must shake hands with each of the three people already present. This is three handshakes in addition to the number of handshakes that would be made in a room of three people, or six handshakes.

If you can generalize this to n handshakes, then it should help you write the recursive solution.

3. Consider a frame of bowling pins shown below, where each * represents a pin:

```

      *
    * *
  * * *
* * * *
* * * * *
```

There are 5 rows and a total of 15 pins.

If we had only the top 4 rows, then there would be a total of 10 pins.

If we had only the top three rows, then there would be a total of six pins.

If we had only the top two rows, then there would be a total of three pins.

If we had only the top row, then there would be a total of one pin.



VideoNote

Solution to
Programming
Project 11.3

Write a recursive function that takes as input the number of rows n and outputs the total number of pins that would exist in a pyramid with n rows. Your program should allow for values of n that are larger than 5.

4. The game of “Jump It” consists of a board with n positive integers in a row except for the first column, which always contains zero. These numbers represent the cost to enter each column. Here is a sample game board where n is 6:

0	3	80	6	57	10
---	---	----	---	----	----

The object of the game is to move from the first column to the last column in the lowest total cost. The number in each column represents the cost to enter that column. Always start the game in the first column and have two types of moves. You can either move to the adjacent column or jump over the adjacent column to land two columns over. The cost of a game is the sum of the costs of the visited columns.

In the board shown above, there are several ways to get to the end. Starting in the first column, our cost so far is 0. We could jump to 80, then jump to 57, then move to 10 for a total cost of $80 + 57 + 10 = 147$. However, a cheaper path would be to move to 3, jump to 6, then jump to 10, for a total cost of $3 + 6 + 10 = 19$.

Write a recursive solution to this problem that computes the cheapest cost of the game and outputs this value for an arbitrarily large game board represented as an array. Your program does not have to output the actual sequence of jumps, only the cheapest cost of this sequence. After making sure that your solution works on small arrays, test your solution on boards of larger and larger values of n to get a feel for how efficient and scalable your solution is.

5. Write a recursive method definition to implement a method named `findNumber` that has two parameters: the first is an array of integers, and the second is the number to search. This parameter should receive from the user a list of n integer values. This method should search the number using a binary search. For example, if the user enters the values (23 34 45 65 78 90 98) the output should be `Number found in array` or `Number not found in array`. Also, write the main method to implement the program.
6. The formula for calculating the exponentiation of a number x with exponent m is:

$$\text{ExpResult} = x^m$$

Write a recursive method named `expEvaluate` with the following header:

```
public long expEvaluate(int x, int m)
```

The method should accept the value of x and m with the following conditions:

- a. Value of x should be in the range $0 < x \leq 10$
- b. Value of m should be in the range $0 \leq m \leq 10$

The method should return the exponent. Also, write the main method to implement the program.

7. *Towers of Hanoi.* There is a story about Buddhist monks who are playing this puzzle with 64 stone disks. The story claims that when the monks finish moving the disks from one post to a second via the third post, time will end.

A stack of n disks of decreasing size (from bottom to top) is placed on one of three posts. The task is to move the disks one at a time from the first post to the second. To do this, any disk can be moved from any post to any other post, subject to the rule that you can never place a larger disk over a smaller disk. The (spare) third post is provided to make the solution possible. Your task is to write a recursive static method that gives instructions for a solution to this problem. We do not want to bother with graphics, so you should output a sequence of instructions that will solve the problem. The number of disks is a parameter to the method.

Hint: If you could move up $n-1$ of the disks from the first post to the third post using the second post as a spare, the last disk could be moved from the first post to the second post. Then, by using the same technique (whatever that may be), you can move the $n-1$ disks from the third post to the second post, using the first disk as a spare. There! You have the puzzle solved. You have only to decide what the nonrecursive case is, what the recursive case is, and when to output instructions to move the disks.

8. Write a recursive method named `searchList` with the following header:

```
boolean searchList ( int[] a, int size, int num)
```

The method should accept as parameter the following values with the specification mentioned below:

- a. A list of integer values
- b. Size of the array, $\text{size} \geq 5$
- c. Number to be searched in the parameter `num`, $\text{num} \geq 0$

The method should return `true` if the number is contained within the array list and `false` if the number is not in the array list.

9. The program to recursively find a file in Display 11.11 stops searching when the first match is found. Modify the program so that if there are multiple files with the same name in different directories, then all matching files are found and output. The simplest way to do this is to output all matches in the recursive method with a print statement. For a more challenging version, modify the method to return an array of Strings containing the pathnames of all matching files. It can return `null` or an empty array if there are no matches. Feel free to create additional helper classes if needed (e.g., to manage the number of items in the array of Strings). In Chapter 14, we will introduce ArrayLists, which make it easier to create an array-like structure with an arbitrary number of entries.

10. Given the definition of a 2-D array such as the following:

```
String[][] data = {
    { "A", "B" },
    { "1", "2" },
    { "XX", "YY", "ZZ" }
};
```

write a recursive program that outputs all combinations of each subarray in order. In the previous example, the desired output (although it does not have to be in this order) is

```
A 1 XX
A 1 YY
A 1 ZZ
A 2 XX
A 2 YY
A 2 ZZ
B 1 XX
B 1 YY
B 1 ZZ
B 2 XX
B 2 YY
B 2 ZZ
```

Your program should work with arbitrarily sized arrays in either dimension. For example, the following data

```
String[] [] data = {
    { "A" },
    { "1" },
    { "2" },
    { "XX", "YY" }
};
```

should output

```
A 1 2 YY
A 1 2 YY
```

11. Simulate a simple word game. Begin with a start word and then rearrange its letters to note down all the possible word combinations that can be generated from the word start following the criteria mentioned below.
 - a. Each word entered by the user must be a three-letter word.
 - b. Each word generated must be a three-letter word.
 - c. No letter should be repeated in the word entered by the use. For example, “bee” is an invalid word, but “cat” is a valid word.
 - d. The program should not only give valid English words but also all combinations. For example, if the given word is eat, then the output of the program should list all the possible permutations:

```
tea, tae, ate, aet, eta
```

Write a recursive program to find the list of words and also letter combinations by keeping in mind all the criteria mentioned above.

This page intentionally left blank



UML and Patterns **12**

12.1 UML 726

History of UML 727

UML Class Diagrams 727

Class Interactions 728

Inheritance Diagrams 728

More UML 730

12.2 PATTERNS 731

Adaptor Pattern ★ 731

The Model-View-Controller Pattern ★ 732

Example: A Sorting Pattern 733

Restrictions on the Sorting Pattern 739

Efficiency of the Sorting Pattern ★ 739

Pattern Formalism 740

12 UML and Patterns

Einstein argued that there must be simplified explanations of nature, because God is not capricious or arbitrary. No such faith comforts the software engineer. Much of the complexity that he must master is arbitrary complexity.

F. BROOKS, “No Silver Bullet: Essence and Accidents of Software Engineering,” *IEEE Computer*, April 1987.

Introduction

UML and patterns are two software design tools that apply no matter what programming language you are using, as long as the language provides for classes and related facilities for object-oriented programming (OOP). This chapter presents a very brief introduction to these two topics. It contains no new details about the Java language.

UML is a graphical language that is used for designing and documenting software created within the OOP framework.

A pattern in programming is very similar to a pattern in any other context. It is a kind of template or outline of a software task that can be realized as different code in different, but similar, applications.

Prerequisites

Section 12.1 on UML and Section 12.2 on patterns can be read in either order. Nothing in the rest of this book requires any of this chapter. Section 12.1 on UML uses material from Chapters 1–5 and Chapter 7 on inheritance. Section 12.2 on patterns uses material from Chapters 1–7 and Chapter 11.

12.1 UML

One picture is worth a thousand words.

FREDERICK BARNARD, *ascribed to Chinese origin*, 1911.

Most people do not think in Java or in any other programming language. As a result, computer scientists have always sought to produce more human-oriented ways of representing programs. One widely used representation is pseudocode, which is a mixture of a programming language, such as Java, and a natural language, such as English. To think about a programming problem without needing to worry about the syntax details of a language such as Java, you can simply relax the syntax rules and write in pseudocode. Pseudocode has become a standard tool used by programmers,

UML

but it is a linear and algebraic representation of programming. Computer scientists have long sought to give software design a graphical representation. To this end, a number of graphical representation systems for program design have been proposed, used, and ultimately found to be wanting. Terms such as *flowchart*, *structure diagram*, and many more names of graphical program representations are today recognized only by those of the older generation. Today's candidate for a graphical representation formalism is the **Unified Modeling Language (UML)**. UML was designed to reflect and be used with the OOP philosophy. It is too early to say whether or not UML will stand the test of time, but it is off to a good start. A number of companies have adopted the UML formalism to use in their software design projects.

History of UML

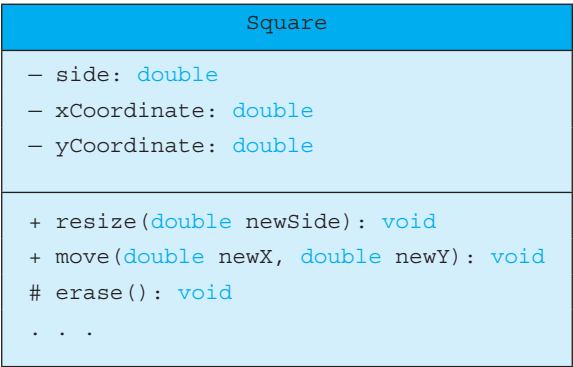
UML developed along with OOP. As the OOP philosophy became more and more commonly used, different groups worked out their own graphical or other representations for OOP design. In 1996, Grady Booch, Ivar Jacobson, and James Rumbaugh released an early version of UML. UML was intended to bring together the various different graphical representation methods to produce a standardized graphical representation language for object-oriented design and documentation. Since that time, UML has been developed and revised in response to feedback from the OOP community. Today the UML standard is maintained and certified by the Object Management Group (OMG), a nonprofit organization that promotes the use of object-oriented techniques.

UML Class Diagrams

class diagram

Classes are central to OOP, and the **class diagram** is the easiest of the UML graphical representations to understand and use. Display 12.1 shows the class diagram for a class to represent a square. The diagram consists of a box divided into three sections. (The colors are optional and not standardized.) The top section has the class name, `Square`. The next section has the data specification for the class. In this example, there are three pieces of data (three instance variables), a value of type `double` giving the length of a side, and two more values of type `double` giving the x and y coordinates of the center of the square. The third section gives the actions (class methods). The notation for method entries is not identical to that of a Java method heading, but it contains the same information. A minus sign indicates a private member. So, for the class `Square`, all data is private. A plus sign indicates a public member. A sharp (`#`) indicates a protected member. A tilde (`~`) indicates package access. So, for the class `Square`, the class diagram shows two public methods and one protected method. A class diagram need not give a complete description of the class. When you do not need all the members in a class for the analysis at hand, you do not list all the members in the class diagram. Missing members are indicated with an ellipsis (three dots).

Display 12.1 A UML Class Diagram



Class Interactions

Class diagrams by themselves are of little value, because they simply repeat the class interface, possibly with ellipses. To understand a design, you need to indicate how objects of the various classes interact. UML has various ways to indicate class interactions; for example, various sorts of annotated arrows indicate the information flow from one class object to another. UML also has annotations for class groupings into packages, annotations for inheritance, and annotations for other interactions. Moreover, UML is extensible. If what you want and need is not in UML, you can add it. Of course, this all takes place inside a prescribed framework so that different software developers can understand each other’s UML. One of the most fundamental of class interactions is inheritance, which is discussed in the next subsection.

Inheritance Diagrams

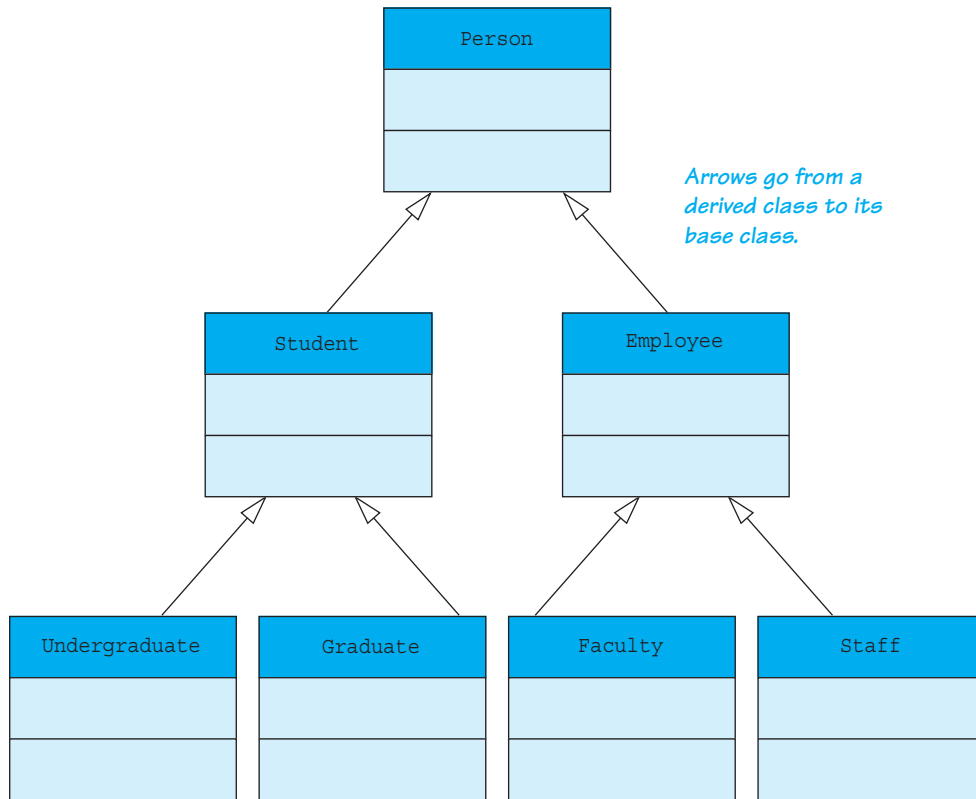
inheritance
diagram

arrows

Display 12.2 shows a possible **inheritance diagram** used in a university’s record-keeping software for some of its classes. Note that the class diagrams are incomplete. You normally show only as much of the class diagram as you need for the design task at hand. Note that the arrow heads point up from a derived class to its base class. In UML an unfilled arrowhead is used to indicate an inheritance relationship between two classes.

The arrows also help in locating method definitions. If you are looking for a method definition for some class, the arrows show the path you (or the computer) should follow. If you are looking for the definition of a method used by an object of the class `Undergraduate`, first look in the definition of the class `Undergraduate`; if it is not there, look in the definition of `Student`; if it is not there, look in the definition of the class `Person`.

Display 12.2 A Class Hierarchy in UML Notation



Display 12.3 shows some possible additional details of the inheritance hierarchy for the two classes `Person` and one of its derived classes, `Student`. Suppose `s` is an object of the class `Student`. The diagram in Display 12.3 tells you that you can find the definition of

```
s.toString();
```

and

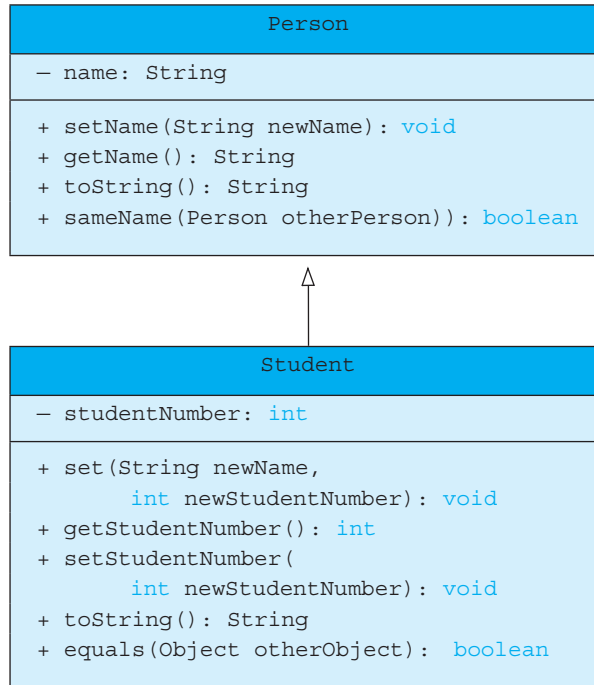
```
s.set("Joe Student", 4242);
```

in the class `Student`, but the definition of

```
s.setName("Josephine Student");
```

is found in the definition of the class `Person`.

Display 12.3 Some Details of a UML Class Hierarchy



More UML

This is just a hint of what UML is all about. If you are interested in learning more, consult one of the many available references on UML.

Self-Test Exercises

1. Draw a class diagram for a class whose objects represent circles. Use Display 12.1 as a model.
2. Suppose `aStudent` is an object of the class `Student`. Based on the inheritance diagram in Display 12.3, where will you find the definition of the method `sameName` used in the following invocation, which compares `aStudent` and another object named `someStudent`? Explain your answer.

```

Student someStudent =
    new Student("Joe Student", 7777);
if (aStudent.sameName(someStudent))
    System.out.println("wow");
  
```

Self-Test Exercises (continued)

3. Suppose `aStudent` is an object of the class `Student`. Based on the inheritance diagram in Display 12.3, where will you find the definition of the method used in the following invocation? Explain your answer.

```
aStudent.setNumber(4242);
```

12.2 Patterns ★

I bid him look into the lives of men as though into a mirror, and from others to take an example for himself.

TERENCE (Publius Terentius Afer) 190–159 B.C., *Adelphoe*, 190–159 B.C.

pattern

Container- Iterator

container

iterator

Patterns are design outlines that apply across a variety of software applications. To be useful, the pattern must apply across a variety of situations. To be substantive, the pattern must make some assumptions about the domain of applications to which it applies. For example, one well-known pattern is the **Container-Iterator** pattern. A **container** is a class (or other construct) whose objects hold multiple pieces of data. One example of a container is an array. Other examples, which will be discussed later in this book, are vectors and linked lists. Any class or other construct designed to hold multiple values can be viewed as a container. For example, a `String` value can be viewed as a container that contains the characters in the string. Any construct that allows you to cycle through all the items in a container is an **iterator**. For example, an array index is an iterator for an array. It can cycle through the array as follows:

```
for (int i; i < a.length; i++)
    Do something with a[i]
```

The index variable `i` is the iterator. The Container-Iterator pattern describes how an iterator is used on a container.

In this brief chapter, we can give you only a taste of what patterns are all about. In this section, we will discuss a few sample patterns to let you see what patterns look like. There are many more known and used patterns and many more yet to be explicated. This is a new and still developing field of software engineering.

Adaptor Pattern ★

Adaptor

The **Adaptor** or **Adapter** pattern transforms one class into a different class without changing the underlying class but merely by adding a new interface. (The new interface replaces the old interface of the underlying class.) For example, in Chapter 11, we mentioned the `stack` data structure, which is used to, among other things, keep track of recursion. One way to create a stack data structure is to start with an array and add

the stack interface. The Adaptor pattern says to start with a container, such as an array, and add an interface, such as the stack interface.

The Model-View-Controller Pattern ★

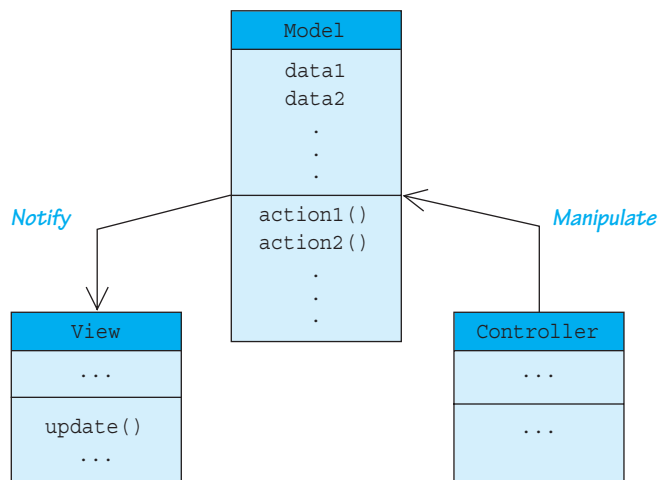
Model-View-Controller

The **Model-View-Controller** pattern is a way of separating the I/O task of an application from the rest of the application. The Model part of the pattern performs the heart of the application. The View part is the output part; it displays a picture of the Model's state. The Controller is the input part; it relays commands from the user to the Model. Normally, each of the three interacting parts is realized as an object with responsibilities for its own tasks. The Model-View-Controller pattern is an example of a divide-and-conquer strategy. One big task is divided into three smaller tasks with well-defined responsibilities. Display 12.4 gives a diagram of the Model-View-Controller pattern. Note that the arrowheads are open. In UML, an open arrowhead indicates an association where one object has a reference to another object of the type connected by the arrow.

As a very simple example, the Model might be a container class, such as an array. The View might display one element of the array. The Controller gives commands to display the element at a specified index. The Model (the array) notifies the View to display a new element whenever the array contents change or a different index location is given.

Any application can be made to fit the Model-View-Controller pattern, but it is particularly well suited to GUI (Graphical User Interface) design projects where the View can indeed be a visualization of the state of the Model. (A GUI interface is simply a windowing interface of the form you find in most modern software applications, as opposed to the simple text I/O we have used so far in this book.) For example, the Model might be an object to represent your list of computer desktop object names. The View could then be a GUI object that produces a screen display of your desktop icons. The Controller relays commands to the Model (which is a desktop object) to add or delete names. The Model object notifies the View object when the screen needs to be updated.

Display 12.4 Model-View-Controller Pattern



We have presented the Model-View-Controller pattern as if the user is the Controller, primarily to simplify the examples. The Controller need not be under the direct control of the user but could be some other kind of software or hardware component.

EXAMPLE: A Sorting Pattern

The most efficient sorting algorithms all seem to follow a similar pattern. Expressed recursively, they divide the list of elements to be sorted into two smaller lists, recursively sort the two smaller lists, and then recombine the two sorted lists to obtain the final sorted list. In Display 12.5, this pattern is expressed as pseudocode (in fact, almost correct Java code) for a method to sort an array into increasing order using the < operator.

Our sorting pattern uses a divide-and-conquer strategy. It divides the entire collection of elements to be sorted into two smaller collections, sorts the smaller collections by recursive calls, and then combines the two sorted collections to obtain the final sorted array. The following is the heart of our sorting pattern:

```
int splitPoint = split(a, begin, end);
sort(a, begin, splitPoint);
sort(a, splitPoint + 1, end);
join(a, begin, splitPoint, end);
```

Although the pattern does impose some minimum requirements on the methods `split` and `join`, it does not say exactly how the methods `split` and `join` are defined. Different definitions of `split` and `join` will yield different sorting algorithms.

The method `split` rearranges the elements in the interval `a[begin]` through `a[end]` and divides the rearranged interval at a split point, `splitPoint`. The two smaller intervals `a[begin]` through `a[splitPoint]` and `[splitPoint + 1]` through `a[end]` are then sorted by a recursive call to the method `sort`. Note that the `split` method both rearranges the elements in the array interval `a[begin]` through `a[end]` and returns the index `splitPoint` that divides the interval. After the two smaller intervals are sorted, the method `join` then combines the two sorted intervals to obtain the final sorted version of the entire larger interval.

The pattern says nothing about how the method `split` rearranges and divides the interval `a[begin]` through `a[end]`. In a simple case, `split` might simply choose a value `splitPoint` between `begin` and `end` and divide the interval into the points before `splitPoint` and the points after `splitPoint`, with no rearranging. Display 12.6 realizes the sorting pattern by defining `split` this way. On the other hand, the method `split` could do something more elaborate such as move all the “small” elements to the front of the array and all the “large” elements toward the end of the array. This would be a step on the way to fully sorting the values. We will also see an example in Display 12.8 that realizes the sorting pattern in this second way.

(continued)

Display 12.5 Divide-and-Conquer Sorting Pattern

```

1  /**
2   Precondition: Interval a[begin] through a[end] of a have elements.
3   Postcondition: The values in the interval have
4   been rearranged so that a[begin] <=a[begin+1] <= . . . <= a[end].
5  */
6  public static void sort(Type [] a, int begin, int end)
7  {
8      if ((end - begin) >= 1)
9      {
10         int splitPoint = split(a, begin, end);
11         sort(a, begin, splitPoint);
12         sort(a, splitPoint + 1, end);
13         join(a, begin, splitPoint, end);
14     } //else sorting one (or fewer) elements so do nothing.
15 }

```

To get a correct Java method definition, Type must be replaced with a suitable type name.

Different definitions for the methods split and join will give different realizations of this pattern.

EXAMPLE: (continued)

The simplest realization of this sorting pattern is the **merge sort** realization given in Display 12.6. In this realization, the array base type, *Type*, is specialized to the type `double`. The merge sort is an example where the definition of `split` is very simple. It just divides the array into two intervals with no rearranging of elements. The `join` method is more complicated. After the two subintervals are sorted, the method `join` merges the two sorted subintervals, copying elements from the array to a temporary array. The merging starts by comparing the smallest elements in each smaller sorted interval. The smaller of these two elements is the smallest of all the elements in either subinterval, so it is moved to the first position in the temporary array. The process is then repeated with the remaining elements in the two smaller sorted intervals to find the next smallest element, and so forth. A demonstration of using the merge sort version of `sort` is given in Display 12.7.

There is a trade-off between the complexity of the methods `split` and `join`. You can make either of them simple at the expense of making the other more complicated. For merge sort, `split` was simple and `join` was complicated. We next give a realization where `split` is complicated and `join` is simple.

Display 12.8 gives the **quick sort** realization of our sorting pattern for the type `double`.

In the quick sort realization, the definition of `split` is quite sophisticated. An arbitrary value in the array is chosen; this value is called the **splitting value**. In our realization, we chose `a[begin]` as the splitting value, but any value will do equally well. The elements in the array are rearranged so that all those elements that are less than or equal to the splitting value are at the front of the array, all the values that are greater

Display 12.6 Merge Sort Realization of Sorting Pattern (part 1 of 2)

```

1  /**
2   Class that realizes the divide-and-conquer sorting pattern and
3   uses the merge sort algorithm.
4   */
5   public class MergeSort
6   {
7       /**
8       Precondition: Interval a[begin] through a[end] of a have elements.
9       Postcondition: The values in the interval have
10      been rearranged so that a[begin] <= a[begin+1] <= . . . <=
11      a[end].
12      */
13      public static void sort(double [] a, int begin, int end)
14      {
15          The method sort is identical to the version in
16          the pattern (Display 12.5) except that Type is
17          replaced with double.
18          if ((end - begin) >= 1)
19          {
20              int splitPoint = split(a, begin, end);
21              sort(a, begin, splitPoint);
22              sort(a, splitPoint + 1, end);
23              join(a, begin, splitPoint, end);
24          } //else sorting one (or fewer) elements so do nothing.
25      }
26
27      private static int split(double [] a, int begin, int end)
28      {
29          return ((begin + end)/2);
30      }
31
32      private static void join(double [] a, int begin, int splitPoint,
33      int end)
34      {
35          double[] temp;
36          int intervalSize = (end - begin + 1);
37          temp = new double [intervalSize];
38          int nextLeft = begin; //index for first chunk
39          int nextRight = splitPoint + 1; //index for second chunk
40          int i = 0; //index for temp
41
42          //Merge til one side is exhausted:
43          while ((nextLeft <= splitPoint) && (nextRight <= end))
44          {
45              if (a[nextLeft] < a[nextRight])
46              {
47                  temp[i] = a[nextLeft];
48                  i++; nextLeft++;
49              }
50          }
51      }
52  }

```

(continued)

Display 12.6 Merge Sort Realization of Sorting Pattern (part 2 of 2)

```

42         else
43         {
44             temp[i] = a[nextRight];
45             i++; nextRight++;
46         }
47     }

48     while (nextLeft <= splitPoint)
49         //Copy rest of left chunk, if any.
50     {
51         temp[i] = a[nextLeft];
52         i++; nextLeft++;
53     }
54     while (nextRight <= end) //Copy rest of right chunk, if any.
55     {
56         temp[i] = a[nextRight];
57         i++; nextRight++;
58     }
59     for (i = 0; i < intervalSize; i++)
60         a[begin + i] = temp[i];
61 }

```

EXAMPLE: (continued)

than the splitting value are at the other end of the array, and the splitting value is placed so that it divides the entire array into these smaller and larger elements. Note that the smaller elements are not sorted and the larger elements are not sorted, but all the elements before the splitting value are smaller than any of the elements after the splitting value. The smaller elements are sorted by a recursive call, the larger elements are sorted by another recursive call, and then these two sorted segments are combined with the `join` method. In this case, the `join` method is as simple as it could be. It does nothing. Because the sorted smaller elements all precede the sorted larger elements, the entire array is sorted.

A demonstration program for the quick sort method `sort` in Display 12.8 is given in the file `QuickSortDemo.java` on the accompanying website.

(Both the merge sort and the quick sort realizations can be done without the use of a second temporary array, `temp`. However, that detail would only distract from the message of this example. In a real application, you may or may not, depending on details, want to consider the possibility of doing a sort realization without the use of the temporary array.)

Display 12.7 Using the MergeSort Class

```

1  public class MergeSortDemo
2  {
3      public static void main(String[] args)
4      {
5          double[] b = {7.7, 5.5, 11, 3, 16, 4.4, 20, 14, 13, 42};

6          System.out.println("Array contents before sorting:");
7          int i;
8          for (i = 0; i < b.length; i++)
9              System.out.print(b[i] + " ");
10         System.out.println();

11         MergeSort.sort(b, 0, b.length-1);
12         System.out.println("Sorted array values:");
13         for (i = 0; i < b.length; i++)
14             System.out.print(b[i] + " ");
15         System.out.println();
16     }
17 }

```

Sample Dialogue

```

Array contents before sorting:
7.7 5.5 11.0 3.0 16.0 4.4 20.0 14.0 13.0 42.0
Sorted array values:
3.0 4.4 5.5 7.7 11.0 13.0 14.0 16.0 20.0 42.0

```

Display 12.8 Quick Sort Realization of Sorting Pattern (part 1 of 3)

```

1  /**
2   Class that realizes the divide-and-conquer sorting pattern and
3   uses the quick sort algorithm.
4  */
5  public class QuickSort
6  {
7      /**
8       Precondition: Interval a[begin] through a[end] of a have elements.
9       Postcondition: The values in the interval have
10      been rearranged so that a[begin] <= a[begin+1] <= . . . <=
       a[end].

```

(continued)

Display 12.8 Quick Sort Realization of Sorting Pattern (part 2 of 3)

```

11     */
12     public static void sort(double[] a, int begin, int end)
13     {
14         if ((end - begin) >= 1) The method sort is identical to the version in the
15         { pattern (Display 12.5) except that Type is replaced
16             with double.
17             int splitPoint = split(a, begin, end);
18             sort(a, begin, splitPoint);
19             sort(a, splitPoint + 1, end);
20             join(a, begin, splitPoint, end);
21         } //else sorting one (or fewer) elements so do nothing.
22     }
23
24     private static int split(double [] a, int begin, int end)
25     {
26         double[] temp;
27         int size = (end - begin + 1);
28         temp = new double [size];
29
30         double splitValue = a[begin];
31         int up = 0;
32         int down = size - 1;
33
34         //Note that a[begin] = splitValue is skipped.
35         for (int i = begin + 1; i <= end; i++)
36         {
37             if (a[i] <= splitValue)
38             {
39                 temp[up] = a[i];
40                 up++;
41             }
42             else
43             {
44                 temp[down] = a[i];
45                 down--;
46             }
47         }
48
49         //0 <= up = down < size
50
51         temp[up] = a[begin]; //Positions the split value
52
53         //temp[i] <= splitValue for i < up
54         // temp[up] = splitValue
55         // temp[i] > splitValue for i > up
56         for (int i = 0; i < size; i++)

```

Display 12.8 Quick Sort Realization of Sorting Pattern (part 3 of 3)

```
50         a[begin + i] = temp[i];
51         return (begin + up);
52     }

53     private static void join(double [] a, int begin,
54                             int splitPoint, int end)
55     {
56         //Nothing to do.
57     }

58 }
```

Restrictions on the Sorting Pattern

The sorting pattern, like all patterns, has some restrictions on where it applies. As we formulated the sorting pattern, it applies only to types for which the `<` operator is defined. Also, it applies only to sorting into increasing order; it does not apply to sorting into decreasing order. However, this is a result of our simplifying details to make the presentation clearer. You can make the pattern more general by replacing the `<` operator with a `boolean` valued method called `compare` that has two arguments of the base type of the array, which returns `true` or `false` depending on whether the first “comes before” the second. Then, the only restriction is that the `compare` method must have a reasonable definition.¹ This sort of generalization is discussed in Chapter 13 in the subsection entitled “The Comparable Interface.”

Efficiency of the Sorting Pattern ★

Essentially any sorting algorithm can be realized using this sorting pattern. However, the most efficient implementations are those for which the `split` method divides the array into two substantial size chunks, such as half and half, or one-fourth and three-fourths. A realization of `split` that divides the array into one or a very few elements and the rest of the array will not be very efficient.

For example, the merge sort realization of `split` divides the array into two roughly equal parts, and merge sort is indeed very efficient. It can be shown (although we will not do so here) that merge sort has a worst-case running time that is the best possible “up to an order of magnitude.”

¹The technical requirement is that the `compare` method be a *total ordering*, a concept discussed in Chapter 13. Essentially, all common orderings that you might want to sort by are total orderings.

The `Comparable` interface has a method `compareTo`, which is slightly different from `compare`. However, the method we described as `compare` can easily be defined using the method `compareTo` as a helping method.

The quick sort realization of `split` divides the array into two portions that might be almost equal or might be very different in size depending on the choice of a splitting value. Since in extremely unfortunate cases the `split` might be very uneven, the worst-case running time for quick sort is not as fast as that of merge sort. However, in practice, quick sort turns out to be a very good sorting algorithm and usually preferable to merge sort.

Selection sort, which we discussed in Chapter 6, divides the array into two pieces, one with a single element and one with the rest of the array interval. (See Self-Test Exercise 4.) Because of this uneven division, selection sort has a poor running time, although it does have the virtue of simplicity.



TIP: Pragmatics and Patterns

You should not feel compelled to follow all the fine details of a pattern. Patterns are guides, not requirements. For example, we did the quick sort implementation by exactly following the pattern. We did this to have a clean example. In practice, we would have taken some liberties. Notice that, with quick sort, the `join` method does nothing. In practice, we would simply eliminate the calls to `join`. These calls incur overhead and accomplish nothing. Other optimizations can also be done once the general pattern of the algorithm is clear. ■

Pattern Formalism

There is a well-developed body of techniques for using patterns. We will not go into the details here. The UML discussed in Section 12.1 is one formalism used to express patterns. The place within the software design process of patterns and any specific formalisms for patterns is not yet clear. However, it is evident that the basic idea of patterns, as well as certain pattern names, such as *Model-View-Controller*, have become standard and useful tools for software design.

Self-Test Exercises

4. Give an implementation of the divide-and-conquer sorting pattern (Display 12.5) that will realize the selection sort algorithm (Display 6.11) for an array with base type `double`.
5. Which of the following would give the fastest run time when an array is sorted using the quick sort algorithm: a fully sorted array, an array of random values, or an array sorted from largest to smallest (that is, sorted backward)? Assume all arrays are of the same size and have the same base type.

Chapter Summary

- The *Unified Modeling Language (UML)* is a graphical representation language for object-oriented software design.
- *Patterns* are design principles that apply across a variety of software applications.
- The patterns discussed in this chapter are the *Container-Iterator*, *Adaptor*, *Model-View-Controller*, and *Divide-and-Conquer Sorting* patterns.
- UML is one formalism that can and is used to express patterns.

Answers to Self-Test Exercises

1. There are many correct answers. The following is one:

Circle
– radius: double – centerX: double – centerY: double
+ resize(double newRadius): void + move(double newX, double newY): void # erase(): void . . .

2. The method `sameName` is not listed in the class diagram for `Student`. So, you follow the arrow to the class diagram for `Person`. The method `sameName` with a single parameter of type `Person` is in the class diagram for `Person`. Because you know a `Student` is a `Person`, you know that this definition works for the method `sameName` with a single parameter of type `Student`. So, the definition used for the method `sameName` is in the class definition of `Person`.
3. You start at the class diagram for `Student`. The method `setStudentNumber` with a single parameter of type `int` is in the class diagram for `Student`, so you need look no further. The definition used for the method `setStudentNumber` is in the class definition of `Student`.
4. The code for this is also on the website that comes with this book. This code is in the file `SelectionSort.java`. A demonstration program is in the file `SelectionSortDemo.java`.

```
public class SelectionSort
{
    public static void sort(double [] a,
                           int begin, int end)
    {
        if ((end - begin) >= 1)
        {
            int splitPoint = split(a, begin, end);
            sort(a, begin, splitPoint);
            sort(a, splitPoint + 1, end);
            join(a, begin, splitPoint, end);
        }//else sorting one (or fewer) elements
        //so do nothing.
    }

    private static int split(double [] a,
                           int begin , int end)
    {
        int index = indexOfSmallest(begin, a, end);
        interchange(begin, index, a);

        return begin;
    }

    private static void join(double [] a, int begin,
                           int splitPoint, int end)
    {
        //Nothing to do.
    }

    private static int indexOfSmallest(int startIndex,
                                       double [] a, int endIndex)
    {
        double min = a[startIndex];
        int indexOfMin = startIndex;
        int index;
        for (index = startIndex + 1;
             index < endIndex; index++)
            if (a[index] < min)
            {
                min = a[index];
                indexOfMin = index;
                //min is smallest of a[startIndex]
                //through a[index]
            }
        return indexOfMin;
    }
}
```

```

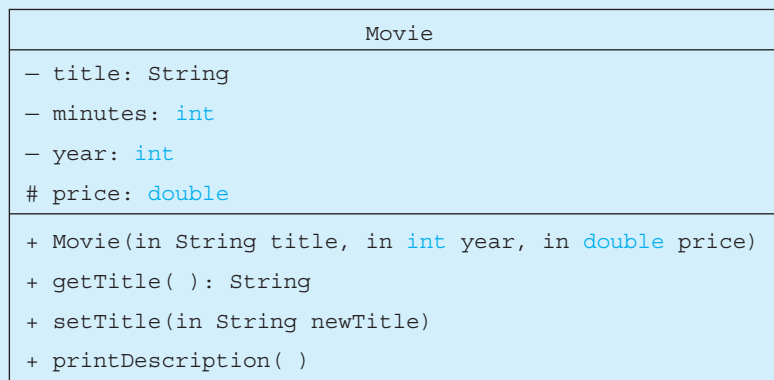
private static void interchange(int i,int j, double [] a)
{
    double temp;
    temp = a[i];
    a[i] = a[j];
    a[j] = temp; //original value of a[i]
}
}

```

5. An array of random values would have the fastest run time, because it would divide the array segments into approximately equal subarrays most of the time. The other two cases would give approximately the same running time and would be significantly slower, because the algorithms would always divide an array segment into very unequal size pieces, one piece with only one element and one piece with the rest of the elements. It is ironic but true that our version of the quick sort algorithms has its worst behavior on an already sorted array. There are variations on the quick sort algorithms that perform well on a sorted array. For example, choosing the middle element as the splitting value will give good performance on an already sorted array. But, whatever splitting value you choose, there will always be a few cases with slow running time.

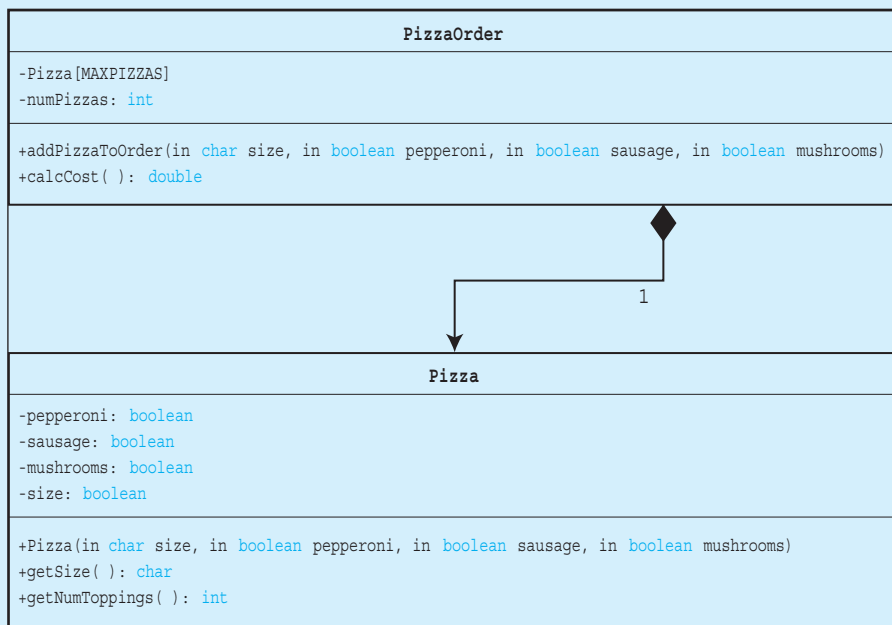
Programming Projects

1. The UML diagram below describes a class named `Movie`. Implement this class in Java and test it from a `main` method that creates several `Movie` objects. The `printDescription()` method should output all member variables for the class.



The word “in” means the parameter is used to deliver data to the method.

2. The following UML diagram shows the relationship between a class called `PizzaOrder` and a class called `Pizza`:



The word “in” means the parameter is used to deliver data to the method.

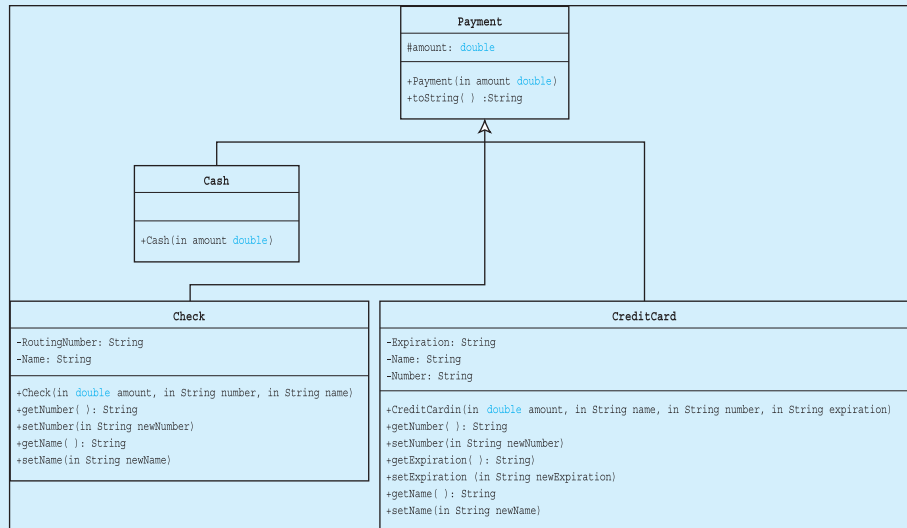
The `Pizza` class contains information about a specific pizza. The variables of `pepperoni`, `sausage`, and `mushrooms` are booleans that indicate whether or not these toppings are present on the pizza. The `size` variable is a character of value 's', 'm', or 'l' to indicate small, medium, or large. There is also a `Pizza` constructor that initializes all of these values. The `getSize()` method returns the size of the pizza and the `getNumToppings()` method returns a number from 0–3 depending on what toppings are present (e.g., if the pizza has pepperoni and mushrooms, it would be 2).

The `PizzaOrder` class contains an array of `Pizza`'s. There is a method to add a new pizza to the array (which increments `numPizzas`) and also a method to calculate the cost of the entire order. A small pizza costs \$8, a medium pizza is \$10, and a large pizza costs \$12. Each topping adds \$1 to the pizza.

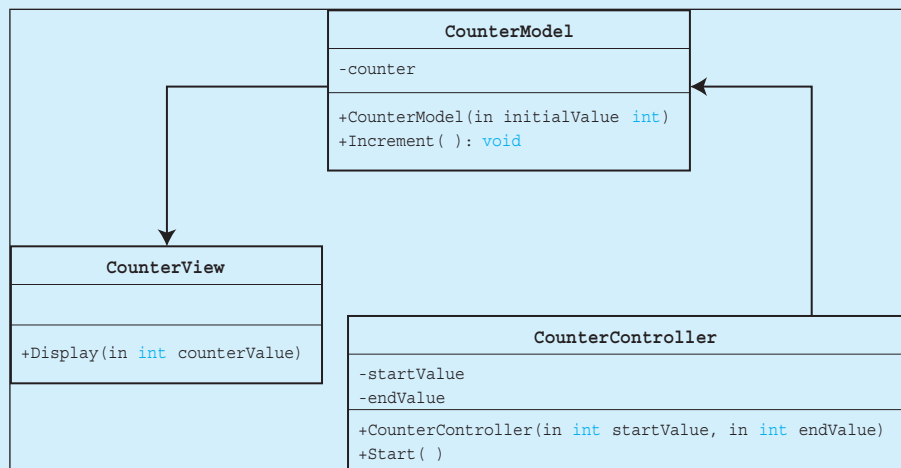
The arrow connecting `PizzaOrder` to `Pizza` indicates that the `PizzaOrder` class has a reference to the `Pizza` class, but not vice versa. The solid diamond on the `PizzaOrder` class is a UML construct that indicates that the `PizzaOrder` class has a collection of the `Pizza` class. There may be many (*) `Pizza`'s for a single (one) `PizzaOrder`.

Given this information, write Java code that implements the `Pizza` and `PizzaOrder` classes. Also, write a test `main` function that creates a pizza order, adds several pizzas to it, and outputs the cost of the order.

3. The UML diagram below shows the relationship between four classes. Implement the classes in a test program that creates instances of `CreditCard`, `Cash`, and `Check`. Output the string description of each. Note that the italicized *Payment* class indicates that this is an abstract class. The word “in” means the parameter is used to deliver data to the method.



4. Use the Model-View-Controller pattern to implement a simple timer-based counter. The counter should start at a user-specified initial value and increment by one every second until it reaches a user-specified final value. A UML diagram depicting the three classes for the pattern is shown below. The word “in” means the parameter is used to deliver data to the method.



The `CounterView` class should simply take an input counter value and print it on the console screen.

The `CounterModel` class should have a variable that represents the counter's value. The `Increment` method increments the counter by one and calls `CounterView`'s `Display` method.

The `CounterController` class takes a start and end value that is specified by the user. `CounterModel` is then initialized with the start value. When the `Start` method is invoked, it calls `CounterModel`'s `increment` method once per second until `endValue - startValue` seconds have elapsed.

You will need to implement additional variables or methods in addition to those shown above in order to create the linkages between classes.

You can use the method call `Thread.sleep(1000)` to make the `CounterController` wait for one second. The call must be placed inside a `try/catch` block.

Test your program with a `main` method that counts several different ranges of values.

5. It is possible to purchase “geek” watches that output the time of day in binary. To illustrate the flexibility of the Model-View-Controller pattern, modify the view class (`CounterView`) of the previous problem so that the display outputs the counter's value in binary.

Test your new program by counting values in binary. You should not have to change the model or controller classes.

6. Recode the `QuickSort` class implementation by adding two efficiency improvements to the method `sort`: (1) Eliminate the calls to `join`, because it accomplishes nothing. (2) Add code for the special case of an array of exactly two elements and make the general case apply to arrays of three or more elements.
7. Redo the `QuickSort` class so that it chooses the splitting point as follows: The splitting point is the middle (in size) of the first element, the last element, and an element at approximately the middle of the array. This will make a very uneven split less likely.
8. Redo the `QuickSort` class to have the modifications given for Programming Projects 12.6 and 12.7.
9. Use the sorting pattern to implement insertion sort. In insertion sort, the `split` method always returns the value `(end - 1)`. This results in splitting the array into two pieces, one with a single value at the end of the array and the other with everything else. The `join` method does more work. A precondition for entry into `join` is that the elements from `a[begin]` to `a[end-1]` will be in sorted order. The method should insert `a[end]` into the correct spot from `a[begin]` to `a[end]` such that sorted order is maintained. For example, if array `a` contains `{2, 4, 6, 8, 5}` where `begin = 0` and `end = 4`, then `a[end] = 5` and the method should insert the value 5 between the 4 and 6, resulting in `{2, 4, 5, 6, 8}`. This entails copying the 6 and 8 one element to the right and then copying the value 5 to index 2.





Interfaces and Inner Classes **13**

13.1 INTERFACES 749

Interfaces 749

Abstract Classes Implementing Interfaces 751

Derived Interfaces 751

The Comparable Interface 755

Example: Using the Comparable Interface 756

Defined Constants in Interfaces 761

The Serializable Interface ★ 765

The Cloneable Interface 765

13.2 SIMPLE USES OF INNER CLASSES 770

Helping Classes 770

Example: A Bank Account Class 771

The .class File for an Inner Class 775

13.3 MORE ABOUT INNER CLASSES 776

Static Inner Classes 776

Public Inner Classes 777

Nesting Inner Classes 781

Inner Classes and Inheritance 781

Anonymous Classes 782

Art, it seems to me, should simplify. That, indeed, is very nearly the whole of the higher artistic process; finding what conventions of form and what details one can do without and yet preserve the spirit of the whole....

WILLA SIBERT CATHER, *On the Art of Fiction*, The Borzoi, 1920.

Introduction

A Java *interface* specifies a set of methods that any class that implements the interface must have. An interface is itself a type, which allows you to define methods with parameters of an interface type and then have the code apply to all classes that implement the interface. One way to view an interface is as an extreme form of an abstract class. However, as you will see, an interface allows you to do more than an abstract class allows you to do. Interfaces are Java's way of approximating multiple inheritance. You cannot have multiple base classes in Java, but interfaces allow you to approximate the power of multiple base classes.

The second major topic of this chapter is *inner classes*. An inner class is simply a class defined within another class. Because inner classes are local to the class that contains them, they can help make a class self-contained by allowing you to make helping classes inner classes.

Prerequisites

Section 13.1 on interfaces and Section 13.2 on simple uses of inner classes are independent of each other and can be covered in any order. Section 13.3 on more subtle details of inner classes requires both Sections 13.1 and 13.2.

Section 13.1 on interfaces requires Chapters 1 through 9. No material from Chapters 10 through 12 is used anywhere in this chapter.

Section 13.2 on simple uses of inner classes requires Chapters 1 through 5. It does not use any material from Chapters 6 through 12.

Section 13.3 on more advanced inner class material requires both Sections 13.1 and 13.2 (and of course their prerequisites). The material in Section 13.3 is not used elsewhere in this book.

13.1 Interfaces

Autonomy of Syntax

A linguistic concept attributed to Noam Chomsky

In this section, we describe *interfaces*. An interface is a type that groups together a number of different classes that all include method definitions for a common set of method headings.

Interfaces

interface

An **interface** is something like the extreme case of an abstract class. *An interface is not a class. It is, however, a type that can be satisfied by any class that implements the interface.* An interface is a property of a class that says what methods it must have.

An interface specifies the headings for methods that must be defined in any class that implements the interface. For example, Display 13.1 shows an interface named `Ordered`. Note that an interface contains only method headings. It contains no instance variables nor any complete method definitions. (Although, as we will see, it can contain defined constants.)

implementing an interface

To **implement an interface**, a concrete class (that is, a class other than an abstract class) must do two things:

1. It must include the phrase

```
implements Interface_Name
```

at the start of the class definition. To implement more than one interface, you list all the interface names, separated by commas, as in

```
implements SomeInterface, AnotherInterface
```

Display 13.1 The `Ordered` Interface

```
1 public interface Ordered
2 {
3     public boolean precedes(Object other);
4
5     /**
6      * For objects of the class o1 and o2,
7      * o1.follows(o2) == o2.precedes(o1).
8     */
9     public boolean follows(Object other);
10 }
```

Do not forget the semicolons at the end of the method headings.

Neither the compiler nor the run-time system will do anything to ensure that this comment is satisfied. It is only advisory to the programmer implementing the interface.

2. The class must implement *all* the method headings listed in the definitions of the interfaces.

For example, to implement the `Ordered` interface, a class definition must contain the phrase `implements Ordered` at the start of the class definition, as shown in the following:

```
public class OrderedHourlyEmployee
    extends HourlyEmployee implements Ordered
{
```

The class must also implement the two methods `precedes` and `follows`. The full definition of `OrderedHourlyEmployee` is given in Display 13.2.

Display 13.2 Implementation of an Interface

```
1  public class OrderedHourlyEmployee
2      extends HourlyEmployee implements Ordered
3  {
4      public boolean precedes(Object other)
5      {
6          if (other == null)
7              return false;
8          else if (!(other instanceof OrderedHourlyEmployee))
9              return false;
10         else
11         {
12             OrderedHourlyEmployee otherOrderedHourlyEmployee =
13                 (OrderedHourlyEmployee) other;
14             return (getPay() < otherOrderedHourlyEmployee.getPay());
15         }
16     }

17     public boolean follows(Object other)
18     {
19         if (other == null)
20             return false;
21         else if (!(other instanceof OrderedHourlyEmployee))
22             return false;
23         else
24         {
25             OrderedHourlyEmployee otherOrderedHourlyEmployee =
26                 (OrderedHourlyEmployee) other;
27             return (otherOrderedHourlyEmployee.precedes(this));
28         }
29     }
30 }
```

Although getClass works better than instanceof for defining equals, instanceof works better in this case. However, either will do for the points being made here.

An interface and all of its method headings are normally declared to be public. They cannot be given private, protected, or package access. (The modifier `public` may be omitted, but all the methods will still be treated as if they are public.) When a class implements an interface, it must make all the methods in the interface public.

An interface is a type. This allows you to write a method with a parameter of an interface type, such as a parameter of type `Ordered`, and that parameter will accept as an argument any class you later define that implements the interface.

An interface serves a function similar to a base class, but it is important to note that it is not a base class. (In fact, it is not a class of any kind.) Some programming languages (such as C++) allow one class to be a derived class of two or more different base classes. This is not allowed in Java. In Java, a derived class can have only one base class. However, in addition to any base class that a Java class may have, it can also implement any number of interfaces. This allows Java programs to approximate the power of multiple base classes without the complications that can arise with multiple base classes.

You might want to say the argument to `precedes` in the `Ordered` interface (Display 13.2) is the same as the class doing the implementation (for example, `OrderedHourlyEmployee`). There is no way to say this in Java, so we normally make such parameters of type `Object`. It would be legal to make the argument to `precedes` of type `Ordered`, but that is not normally preferable to using `Object` as the parameter type. If you make the argument of type `Ordered`, you would still have to handle the case of `null` and the case of an argument that (while `Ordered`) is not of type `OrderedHourlyEmployee`.

An interface definition is stored in a `.java` file and compiled just as a class definition is compiled.

Abstract Classes Implementing Interfaces

As you saw in the previous subsection, a concrete class (that is, a regular class) must give definitions for all the method headings given in an interface in order to implement the interface. However, you can define an abstract class that implements an interface but gives only definitions for some of the method headings given in the interface. The method headings given in the interface that are not given definitions are made into abstract methods. A simple example is given in Display 13.3.

Derived Interfaces

extending an interface

You can derive an interface from a base interface. This is often called **extending** the interface. The details are similar to deriving a class. An example is given in Display 13.4.

Display 13.3 An Abstract Class Implementing an Interface ★

```
1 public abstract class MyAbstractClass implements Ordered
2 {
3     private int number;
4     private char grade;
5
6     public boolean precedes(Object other)
7     {
8         if (other == null)
9             return false;
10        else if (!(other instanceof HourlyEmployee))
11            return false;
12        else
13        {
14            MyAbstractClass otherOfMyAbstractClass =
15                (MyAbstractClass)other;
16            return (this.number < otherOfMyAbstractClass.number);
17        }
18    }
19
20    public abstract boolean follows(Object other);
21 }
```

Display 13.4 Extending an Interface

```
1 public interface ShowablyOrdered extends Ordered
2 {
3     /**
4      * Outputs an object of the class that precedes the calling object.
5      */
6     public void showOneWhoPrecedes();
7 }
```

*Neither the compiler nor the run-time system
will do anything to ensure that this comment is
satisfied.*

*A (concrete) class that implements the ShowablyOrdered interface must have a
definition for the method showOneWhoPrecedes and also have definitions for the
methods precedes and follows given in the Ordered interface.*

Interfaces

An interface is a type that specifies method headings (and, as we will see, possibly defined constants as well). The syntax for defining an interface is similar to the syntax of defining a class, except that the word `interface` is used in place of `class` and the method headings without any method body (but followed by a semicolon) are given only.

Note that an interface has no instance variables and no method definitions.

A class can implement any number of interfaces. To implement an interface, the class must include

```
implements Interface_Name
```

at the end of the class heading and must supply definitions for the method headings given in the interface. If the class does not supply definitions for all the method headings given in the interface, then the class must be an abstract class and the method headings without definitions must be abstract methods.

EXAMPLE

See Displays 13.1, 13.2, and 13.3.

Self-Test Exercises

1. Can you have a variable of an interface type? Can you have a parameter of an interface type?
2. Can an abstract class ever implement an interface?
3. Can a derived class have two base classes? Can it implement two interfaces?
4. Can an interface implement another interface?



PITFALL: Interface Semantics Are Not Enforced

As far as the Java compiler is concerned, an interface has syntax but no semantics. For example, the definition of the `Ordered` interface (Display 13.1) says the following in a comment:

```
/**
 * For objects of the class o1 and o2,
 * o1.follows(o2) == o2.precedes(o1).
 */
```

You might have assumed that this is true even if there were no comment in the interface. After all, in the real world, if I precede you, then you follow me. However, that is giving your intuitive interpretation to the word “precedes.”

(continued)

**PITFALL: (continued)**

As far as the compiler and run-time systems are concerned, the `Ordered` interface merely says that the methods `precedes` and `follows` each take one argument of type `Object` and return a `boolean` value. The interface does not really require that the `boolean` value be computed in any particular way. For example, the compiler would be satisfied if both `precedes` and `follows` always return `true` or if they always return `false`. It would even allow the methods to use a random number generator to generate a random choice between `true` and `false`.

It would be nice if we could safely give an interface some simple semantics, such as saying that `o1.follows(o2)` means the same as `o2.precedes(o1)`. However, if Java did allow that, there would be problems with having a class implement two interfaces or even with having a class derived from one base class and implementing an interface. Either of these situations could produce two semantic conditions, both of which must be implemented for the same method, and the two semantics may not be consistent. For example, suppose that (contrary to fact) you could require that `o1.follows(o2)` means the same as `o2.precedes(o1)`. You could also define another interface with an inconsistent semantics, such as saying that `precedes` always returns `true` and that `follows` always returns `false`. As long as a class can have two objects, there is no way a class could implement both of these semantics. Interfaces in Java are very well behaved, the price of which is that you cannot count on Java to enforce any semantics in an interface.

If you want to require semantics for an interface, you can add it to the documentation, as illustrated by the comments in Displays 13.1 and 13.4, but always remember that these are just comments; they are not enforced by either the compiler or the run-time system, so you cannot necessarily rely on such semantics being followed. However, we live in an imperfect world, and sometimes you will find that you must specify a semantics for an interface; do so in the interface's documentation. It then becomes the responsibility of the programmers implementing the interface to follow the semantics.

Having made our point about interface semantics not being enforced by the compiler or run-time system, we want to nevertheless urge you to follow the specified semantics for an interface. Software written for classes that implement an interface will assume that any class that implements the interface does satisfy the specified semantics. So, if you define a class that implements an interface but does not satisfy the semantics for the interface, then software written for classes that implement that interface will probably not work correctly for your class. ■

Interface Semantics Are Not Enforced

When you define a class that implements an interface, the compiler and run-time system will let you define the body of an interface method any way you want, provided you keep the method heading as it is given in the interface. However, you should follow the specified semantics for an interface whenever you define a class that implements that interface; otherwise, software written for that interface may not work for your class.

The Comparable Interface

This subsection requires material on arrays from Chapter 6. If you have not yet covered Chapter 6, you can skip this section and the following Programming Example without any loss of continuity. But if you have read Chapter 6, you should not consider this section to be optional.

In Chapter 6 (Display 6.11), we introduced a method for sorting a partially filled array of base type `double` into increasing order. It is very easy to transform the code into a method to sort into decreasing order instead of increasing order. (See Self-Test Exercise 20 of Chapter 6 and its answer if this is not clear to you.) It is also easy to modify the code to obtain methods for sorting integers instead of `doubles` or sorting strings into alphabetical order. Although these changes are easy, they seem to be—and in fact are—a useless nuisance. All these sorting methods are essentially the same. The only differences are the types of the values being sorted and the definition of the ordering. It would seem that we should be able to give a single sorting method that covers all these cases. The `Comparable` interface lets us do this.

The `Comparable` interface is in the `java.lang` package and so is automatically available to your program. The `Comparable` interface has only the following method heading that must be implemented for a class to implement the `Comparable` interface:

```
public int compareTo(Object other);
```

The `Comparable` interface has semantics, and it is the programmer's responsibility to follow the semantics when implementing the `Comparable` interface. The semantics says that `compareTo` returns

`compareTo`

- a negative number if the calling object “comes before” the parameter `other`,
- a zero if the calling object “equals” the parameter `other`,
- and a positive number if the calling object “comes after” the parameter `other`.¹

Almost any reasonable notions of “comes before” should be acceptable. In particular, all of the standard less-than relations on numbers and lexicographic ordering on strings are suitable ordering for `compareTo`. (The relationship “comes after” is just the reverse of “comes before.”) If you need to consider other ordering, the precise rule is that the ordering must be a total ordering, which means the following rules must be satisfied:

(Irreflexive) For no object `o` does `o` come before `o`.

(Trichotomy) For any two objects `o1` and `o2`, one, and only one, of the following holds true: `o1` comes before `o2`, `o1` comes after `o2`, or `o1` equals `o2`.

(Transitivity) If `o1` comes before `o2` and `o2` comes before `o3`, then `o1` comes before `o3`.

The “equals” of the `compareTo` method semantics should coincide with the `equals` methods if possible, but this is not absolutely required by the semantics.

¹Because the parameter to `compareTo` is of type `Object`, an argument to `compareTo` might not be an object of the class being defined. If the parameter `other` is not of the same type as the class being defined, then the semantics specifies that a `ClassCastException` should be thrown.

The Comparable Interface

The `Comparable` interface is in the `java.lang` package and so is automatically available to your program. The `Comparable` interface has only the following method heading that must be given a definition for a class to implement the `Comparable` interface:

```
public int compareTo(Object other);
```

The method `compareTo` should return

- a negative number if the calling object “comes before” the parameter `other`,
- a zero if the calling object “equals” the parameter `other`,
- and a positive number if the calling object “comes after” the parameter `other`.

The “comes before” ordering that underlies `compareTo` should be a total ordering. Most normal ordering, such as less-than ordering on numbers and lexicographic ordering on strings, is total ordering.

If you define a class that implements the `Comparable` interface but that does not satisfy these conditions, then code written for `Comparable` objects will not work properly. It is the responsibility of you, the programmer, to ensure that the semantics is satisfied. Neither the compiler nor the run-time system enforces any semantics on the `Comparable` interface.

If you have read this subsection, you should also read the following Programming Example.

EXAMPLE: Using the `Comparable` Interface

Display 13.5 shows a class with a method that can sort any partially filled array whose base type implements the `Comparable` interface (including implementing the semantics we discussed in the previous subsection). To obtain the code in Display 13.5, we started with selection sort of an array of doubles and mechanically replaced all occurrences of the array type `double[]` with the type `Comparable[]`. We replaced all Boolean expressions of the form

Expression_1 < *Expression_2*

with

Expression_1.compareTo(Expression_2) < 0

The changes are highlighted in Display 13.5. Only four small changes to the code were needed.

(continued on page 726)

Display 13.5 Sorting Method for Array of Comparable (part 1 of 2)

```

1  public class GeneralizedSelectionSort
2  {
3      /**
4       * Precondition: numberUsed <= a.length;
5       *           The first numberUsed indexed variables have values.
6       * Action: Sorts a so that a[0], a[1], ... , a[numberUsed - 1] are in
7       * increasing order by the compareTo method.
8       */
9      public static void sort(Comparable[] a, int numberUsed)
10     {
11         int index, indexOfNextSmallest;
12         for (index = 0; index < numberUsed - 1; index++)
13         { //Place the correct value in a[index]:
14             indexOfNextSmallest = indexOfSmallest(index, a,
15                 numberUsed);
16             interchange(index,indexOfNextSmallest, a);
17             //a[0], a[1],..., a[index] are correctly ordered and
18             //these are
19             //the smallest of the original array elements. The remaining
20             //positions contain the rest of the original array elements.
21         }
22     }
23
24     /**
25     * Returns the index of the smallest value among
26     * a[startIndex], a[startIndex+1], ... a[numberUsed - 1]
27     */
28     private static int indexOfSmallest(int startIndex,
29         Comparable [] a, int numberUsed)
30     {
31         Comparable min = a[startIndex];
32         int indexOfMin = startIndex;
33         int index;
34         for (index = startIndex + 1; index < numberUsed; index++)
35         {
36             if (a[index].compareTo(min) < 0) //if a[index] is less than min
37             {
38                 min = a[index];
39                 indexOfMin = index;
40             }
41         }
42         return indexOfMin;
43     }
44 }

```

(continued)

Display 13.5 Sorting Method for Array of Comparable (part 2 of 2)

```

40  /**
41   * Precondition: i and j are legal indices for the array a.
42   * Postcondition: Values of a[i] and a[j] have been interchanged.
43   */
44  private static void interchange(int i, int j, Comparable[] a)
45  {
46      Comparable temp;
47      temp = a[i];
48      a[i] = a[j];
49      a[j] = temp; //original value of a[i]
50  }
51  }

```

EXAMPLE: (continued)

Display 13.6 shows a demonstration of using the sorting method given in Display 13.5. To understand why the demonstration works, you need to be aware of the fact that both of the classes `Double` and `String` implement the `Comparable` interface.

If you were to check the full documentation for the class `Double`, you would see that `Double` implements the `Comparable` interface and so has a `compareTo` method. Moreover, for objects `o1` and `o2` of `Double`,

```
o1.compareTo(o2) < 0 //o1 "comes before" o2
```

means the same thing as

```
o1.doubleValue() < o2.doubleValue()
```

So, the implementation of the `Comparable` interface for the class `Double` is really just the ordinary less-than relationship on the `double` values corresponding to the `Double` objects.

Similarly, if you were to check the full documentation for the class `String`, you would see that `String` implements the `Comparable` interface and so has a `compareTo` method. Moreover, the implementation of the `compareTo` method for the class `String` is really just the ordinary lexicographic relationship on the strings.

This Programming Example uses the standard library classes `Double` and `String` for the base type of the array. You can do the same thing with arrays whose base class is a class you defined, so long as the class implements the `Comparable` interface (including the standard semantics, which we discussed earlier in the Pitfall “Interface Semantics Are Not Enforced”).

This Programming Example does point out one restriction on interfaces. They can apply only to classes. A primitive type cannot implement an interface. So, in Display 13.6, we could not sort an array with base type `double` using the sorting method for an array of `Comparable`. We had to settle for sorting an array with base type `Double`. This is a good example of using a wrapper class with its “wrapper class personality.”

Display 13.6 Sorting Arrays of Comparable (part 1 of 2)

```

1  /**
2   Demonstrates sorting arrays for classes that
3   implement the Comparable interface.
4   */
5  public class ComparableDemo           The classes Double and String do
6  {                                     implement the Comparable interface.
7      public static void main(String[] args)
8      {
9          Double[] d = new Double[10];
10         for (int i = 0; i < d.length; i++)
11             d[i] = new Double(d.length - i);

12         System.out.println("Before sorting:");
13         for (int i = 0; i < d.length; i++)
14             System.out.print(d[i].doubleValue() + ", ");
15         System.out.println();

16         GeneralizedSelectionSort.sort(d, d.length);

17         System.out.println("After sorting:");
18         for (int i = 0; i < d.length; i++)
19             system.out.print(d[i].doubleValue() + ", ");
20         System.out.println();

21         String[] a = new String[10];
22         a[0] = "dog";
23         a[1] = "cat";
24         a[2] = "cornish game hen";
25         int numberUsed = 3;

26         System.out.println("Before sorting:");
27         for (int i = 0; i < numberUsed; i++)
28             System.out.print(a[i] + ", ");
29         System.out.println();
30
31         GeneralizedSelectionSort.sort(a, numberUsed);

32         System.out.println("After sorting:");
33         for (int i = 0; i < numberUsed; i++)
34             System.out.print(a[i] + ", ");
35         System.out.println();
36     }
37 }

```

(continued)

Display 13.6 Sorting Arrays of Comparable (part 2 of 2)

Sample Dialogue

```
Before Sorting
10.0, 9.0, 8.0, 7.0, 6.0, 5.0, 4.0, 3.0, 2.0, 1.0,
After sorting:
1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0,
Before sorting;
dog, cat, cornish game hen,
After sorting:
cat, cornish game hen, dog,
```

Self-Test Exercises

These exercises are for the material on the Comparable interface.

5. The method `interchange` in Display 13.5 makes no use of the fact that its second argument is an array with base type `Comparable`. Suppose we change the parameter type `Comparable[]` to `Object[]` and change the type of the variable `temp` to `Object`. Would the program in Display 13.6 produce the same dialogue?
6. Is the following a suitable implementation of the `Comparable` interface?

```
public class Double2 implements Comparable
{
    private double value;
    public Double2(double theValue)
    {
        value = theValue;
    }
    public int compareTo(Object other)
    {
        return -1;
    }
    public double doubleValue()
    {
        return value;
    }
}
```

You can think of the underlying “comes before” relationship as saying that for any objects `d1` and `d2`, `d1` comes before `d2`.

Self-Test Exercises (continued)

7. Suppose you have a class `Circle` that represents circles which all have centers at the same point. (To make it concrete, you can take the circles to be in the usual x,y plane and to all have their centers at the origin.) Suppose there is a `boolean` valued method `inside` of the class `Circle` such that, for circles `c1` and `c2`,

```
c1.inside(c2)
```

returns `true` if `c1` is completely inside of `c2` (and `c2` is not the same as `c1`). Is the following a total ordering?

`c1` comes before `c2` if `c1` is inside of `c2`
(that is, if `c1.inside(c2)` returns `true`).

You could represent objects of the class `Circle` by a single value of type `double` that gives the radius of the circle, but the answer does not depend on such details.

Defined Constants in Interfaces

The designers of Java often used the interface mechanism to take care of a number of miscellaneous details that do not really fit the spirit of what an interface is supposed to be. One example of this is the use of an interface as a way to name a group of defined constants.

An interface can contain defined constants as well as method headings, or instead of method headings. When a method implements the interface, it automatically gets the defined constants. For example, the following interface defines constants for months:

```
public interface MonthNumbers
{
    public static final int JANUARY = 1,
        FEBRUARY = 2, MARCH = 3, APRIL = 4, MAY = 5,
        JUNE = 6, JULY = 7, AUGUST = 8, SEPTEMBER = 9,
        OCTOBER = 10, NOVEMBER = 11, DECEMBER = 12;
}
```

Any class that implements the `MonthNumbers` interface will automatically have the 12 constants defined in the `MonthNumbers` interface. For example, consider the following toy class:

```
public class DemoMonthNumbers implements MonthNumbers
{
    public static void main(String[] args)
    {
        System.out.println(
            "The number for January is " + JANUARY);
    }
}
```

**no instance
variables**

Note that the constant `JANUARY` is used in the class `DemoMonthNumbers` but is not defined there. The class `DemoMonthNumbers` automatically gets the month constants because it implements the `MonthNumbers` interface.

An interface cannot have instance variables, although it can use the syntax for instance variables as a way to define constants. Any variables defined in an interface must be `public`, `static`, and `final`, so Java allows you to omit those modifiers. The following is an equivalent definition of the interface `MonthNumbers`:

```
public interface MonthNumbers
{
    int JANUARY = 1,
        FEBRUARY = 2, MARCH = 3, APRIL = 4, MAY = 5,
        JUNE = 6, JULY = 7, AUGUST = 8, SEPTEMBER = 9,
        OCTOBER = 10, NOVEMBER = 11, DECEMBER = 12;
}
```

Thus, an interface can be used to give a name for a group of defined constants, so that you can easily add the needed constants to any class by implementing the interface. This is really a different use for interfaces than what we have seen before, which was to use interfaces to specify method headings. It is legal to mix these two uses by including both defined constants and method headings in a single interface.

**PITFALL: Inconsistent Interfaces**

Java allows a class to have only one base class but also allows the class to implement any number of interfaces. The reason that a class can have only one base class is that if Java allowed two base classes, the two base classes could provide different and inconsistent definitions of a single method heading. Because interfaces have no method bodies at all, this problem cannot arise when a class implements two interfaces. The ideal that the designers of Java apparently hoped to realize was that any two interfaces will always be consistent. However, this ideal was not fully realized. Although it is a rare phenomenon, two interfaces can be inconsistent. In fact, there is more than one kind of inconsistency that can be exhibited. If you write a class definition that implements two inconsistent interfaces, that is an error, and the class definition is illegal. Let's see how two interfaces can be inconsistent.

**inconsistent
constants**

The most obvious way that two interfaces can be inconsistent is by defining two constants with the same name but with different values. For example,

```
public interface Interface1
{
    int ANSWER = 42;
}

public interface Interface2
{
    int ANSWER = 0;
}
```



PITFALL: (continued)

Suppose a class definition begins with

```
public class MyClass
    implements Interface1, Interface2
{ ...
```

Clearly this has to be, and is, illegal. The defined constant `ANSWER` cannot be simultaneously 42 and 0.²

Even two method headings can be inconsistent. For example, consider the following two interfaces:

```
public interface InterfaceA
{
    public int getStuff();
}
public interface InterfaceB
{
    public String getStuff();
}
```

Suppose a class definition begins with

```
public class YourClass
    implements InterfaceA, InterfaceB
{ ...
```

Clearly this has to be, and is, illegal. The method `getStuff` in the class `YourClass` cannot be simultaneously a method that returns an `int` and a method that returns a value of type `String`. (Remember that you cannot overload a method based on the type returned; so, overloading cannot be used to get around this problem.) ■

inconsistent
method
headings

Self-Test Exercises

8. Will the following program compile? If it does compile, will it run? `Interface1` and `Interface2` were defined in the previous subsection.

```
public class MyClass
    implements Interface1, Interface2
{
    public static void main(String[] args)
    {
        System.out.println(ANSWER);
    }
}
```

(continued)

²If the class never uses the constant `ANSWER`, then there is no inconsistency, and the class will compile and run with no error messages.

Self-Test Exercises (continued)

9. Will the following program compile? If it does compile, will it run?
Interface1 and Interface2 were defined in the previous subsection.

```
public class MyClass
    implements Interface1, Interface2
{
    public static void main(String[] args)
    {
        System.out.println("Hello");
    }
}
```

10. Will the following program compile? If it does compile, will it run?
InterfaceA and InterfaceB were defined in the previous subsection.

```
public class YourClass
    implements InterfaceA, InterfaceB
{
    public String getStuff()
    {
        return "one";
    }
}
```

11. Will the following two interfaces and the following program class compile? If they compile, will the program run with no error messages?

```
public interface InterfaceA
{
    public int getStuff();
}
public interface InterfaceOtherB
{
    public String getStuff(String someStuff);
}
public class OurClass
    implements InterfaceA, InterfaceOtherB
{
    private int intStuff = 42;

    public static void main(String[] args)
    {
        OurClass object = new OurClass();
        System.out.println(object.getStuff()
            + object.getStuff("Hello"));
    }
}
```

Self-Test Exercises (continued)

```
public int getStuff()
{
    return intStuff;
}
public String getStuff(String someStuff)
{
    return someStuff;
}
}
```

The Serializable Interface ★

Serializable

As we have already noted, the designers of Java often used the interface mechanism to take care of miscellaneous details that do not really fit the spirit of what an interface is supposed to be. An extreme example of this is the `Serializable` interface. The `Serializable` interface has no method headings and no defined constants. As a traditional interface, it is pointless. However, Java uses it as a type tag that means the programmer gives permission to the system to implement file I/O in a particular way. If you want to know what that way of implementing file I/O is, see Chapter 10, in which the `Serializable` interface is discussed in detail.

The Cloneable Interface

Cloneable

The `Cloneable` interface is another example where Java uses the interface mechanism for something other than its traditional role. The `Cloneable` interface has no method headings that must be implemented (and has no defined constants). However, it is used to say something about how the method `clone`, which is inherited from the class `Object`, should be used and how it should be redefined.

So, what is the purpose of the `Cloneable` interface? When you define a class to implement the `Cloneable` interface, you are agreeing to redefine the `clone` method (inherited from `Object`) in a particular way. The primary motivation for this appears to be security issues. Cloning can potentially copy private data if not done correctly. Also, some software may depend on your redefining the `clone` method in a certain way. Programmers have strong and differing views on how to handle cloning and the `Cloneable` interface. What follows is the official Java line on how to do it.

The method `Object.clone()` does a bit-by-bit copy of the object's data in storage. If the data is all primitive type data or data of immutable class types (such as `String`), then this works fine and has no unintended side effects. However, if the data in the object includes instance variables whose type is a mutable class, then this would cause what we refer to as *privacy leaks*. (See the Pitfall section entitled "Privacy Leaks" in Chapter 5.) To avoid these privacy leaks when you define the `clone` method in a derived class, you should invoke the `clone` method of the base class `Object` (or

whatever the base class is) and then reset the values of any new instance variables whose types are mutable class types. Reset these instance variables to copies of the instance variables in the calling object. There are also issues of exception handling to deal with. An example may be clearer than an abstract discussion.

Let's start with the simple case. Suppose your class has no instance variables of a mutable class type, or to phrase it differently, suppose your class has instance variables all of whose types are either a primitive type or an immutable class type, like `String`. And to make it even simpler, suppose your class has no specified base class, so the base class is `Object`. If you want to implement the `Cloneable` interface, you should define the `clone` method as in Display 13.7.

The try-catch blocks are required because the inherited method `clone` can throw the exception `CloneNotSupportedException` if the class does not correctly implement the `Cloneable` interface. Of course, in this case the exception will never be thrown, but the compiler will still insist on the try-catch blocks.

Now let's suppose your class has one instance variable of a mutable class type named `DataClass`. Then, the definition of the `clone` method should be as in Display 13.8. First, a bit-by-bit copy of the object is made by the invocation of `super.clone()`. The dangerous part of copy is the reference to the mutable object in the instance variable

Display 13.7 Implementation of the Method `clone` (Simple case)

```

1  public class YourCloneableClass implements Cloneable
2  {
3      .
4      .
5      .
6      public Object clone()
7      {
8          try
9          {
10             return super.clone(); //Invocation of clone
11                                     //in the base class Object
12          }
13          catch (CloneNotSupportedException e)
14          { //This should not happen.
15              return null; //To keep the compiler happy.
16          }
17      }
18      .
19      .
20      .
21  }
```

Display 13.8 Implementation of the Method `clone` (Harder Case)

```

1  public class YourCloneableClass2 implements Cloneable
2  {
3      private DataClass someVariable;
4      .
5      .
6      .
7      public Object clone()
8      {
9          try
10         {
11             YourCloneableClass2 copy =
12                 (YourCloneableClass2) super.clone();
13             copy.someVariable = (DataClass) someVariable.clone();
14             return copy;
15         }
16         catch (CloneNotSupportedException e)
17         { // This should not happen.
18             return null; // To keep the compiler happy.
19         }
20     }
21     .
22     .
23     .
24 }

```

DataClass is a mutable class. Any other instance variables are each of a primitive type or of an immutable type like String.

If the clone method return type is DataClass rather than Object, then this type cast is not needed.

The class DataClass must also properly implement the Cloneable interface including defining the clone method as we are describing.

`someVariable`. So, the reference is replaced by a reference to a copy of the object named by `someVariable`. This is done with the following line:³

```
copy.someVariable = (DataClass) someVariable.clone();
```

The object named by `copy` is now safe and so can be returned by the `clone` method.

If there are more instance variables that have a mutable class type, then you repeat what we did for `someVariable` for each of the mutable instance variables.

This requires that the class type `DataClass` has a correctly working `clone` method that is marked `public`, but that will be true if the class type `DataClass` implements the `Cloneable` interface in the way we are now describing. That is, `DataClass` should implement the `Cloneable` interface following the model of Displays 13.7 or 13.8,

³Depending on how the `clone` method was defined in the class `DataClass`, the type cast may or may not be needed, but causes no harm in any case.

whichever is appropriate; similarly, all classes for instance variables in `DataClass` should follow the model of Display 13.7 or 13.8, and so forth for classes for instance variables inside of classes all the way down. You want every class in sight and every class used in every class in sight to follow the model of Display 13.7 or 13.8.

The same basic technique applies if your class is derived from some class other than `Object`, except that, in this case, there normally is no required exception handling. To implement the `Cloneable` interface in a derived class with a base class other than `Object`, the details are as follows: The base class must properly implement the `Cloneable` interface, and the derived class must take care of any mutable class instance variable added in the definition of the derived class. These new mutable class instance variables are handled by the technique shown in Display 13.8 for the instance variable `someVariable`. As long as the base class properly implements the `Cloneable` interface, including defining the `clone` method as we are describing, then the derived class's `clone` method need not worry about any inherited instance variables. Also, usually, you do not need to have `try` and `catch` blocks for `CloneNotSupportedException` because the base class `clone` method, `super.clone()`, normally catches all its `CloneNotSupportedException`s, so `super.clone()` will never throw a `CloneNotSupportedException`. (See Self-Test Exercise 15 for an example.)

Self-Test Exercises

12. Modify the following class definition so it correctly implements the `Cloneable` interface (all the instance variables are shown):

```
public class StockItem
{
    private int number;
    private String name;
    public void setNumber(int newNumber)
    {
        number = newNumber;
    }
    ...
}
```

13. Modify the following class definition so it correctly implements the `Cloneable` interface (all the new instance variables are shown):

```
public class PricedItem extends StockItem
{
    private double price;
    ...
}
```

Self-Test Exercises (continued)

14. Modify the following class definition so it correctly implements the `Cloneable` interface (all the new instance variables are shown):

```
public class PricedItem extends StockItem
{
    private double price;

    ...
}
```

15. Modify the following class definition so it correctly implements the `Cloneable` interface (all the instance variables are shown):

```
public class Record
{
    private StockItem item1;
    private StockItem item2;
    private String description;

    ...
}
```

16. Modify the following class definition so it correctly implements the `Cloneable` interface (all the new instance variables are shown):

```
public class BigRecord extends Record
{
    private StockItem item3;

    ...
}
```

17. Modify the definition of the class `Date` (Display 4.13) so it implements the `Cloneable` interface. Be sure to define the method `clone` in the style of Display 13.7.
18. Modify the definition of the class `Employee` (Display 7.2) so it implements the `Cloneable` interface. Be sure to define the method `clone` in the style of Display 13.8.
19. Modify the definition of the class `HourlyEmployee` (Display 7.3) so it implements the `Cloneable` interface. Be sure to define the method `clone` in the style of Display 13.8.

13.2 Simple Uses of Inner Classes

The ruling ideas of each age have ever been the ideas of its ruling class.

KARL MARX and FRIEDRICH ENGELS, *The Communist Manifesto*, 1848.

Inner classes are classes defined within other classes. In this section, we will describe one of the most useful applications of inner classes, namely, inner classes used as helping classes.

Helping Classes

inner class

Defining an **inner class** is straightforward; simply include the definition of the inner class within another class, as follows:

```
public class OuterClass
{
    private class InnerClass
    {
        Declarations_of_InnerClass_Instance_Variables
        Definitions_of_InnerClass_Methods
    }
    Declarations_of_OuterClass_Instance_Variables
    Definitions_of_OuterClass_Methods
}
```

outer class

As this outline suggests, the class that includes the inner class is called an **outer class**. The definition of the inner class (or classes) need not be the first item(s) of the outer class, but it is good to place it either first or last so that it is easy to find. The inner class need not be private, but that is the only case we will consider in this section. We will consider other modifiers besides `private` in Section 13.3.

An inner class definition is a member of the outer class in the same way that the instance variables of the outer class and the methods of the outer class are members of the outer class. Thus, an inner class definition is local to the outer class definition. So you may reuse the name of the inner class for something else outside the definition of the outer class. If the inner class is private, as ours will always be in this section, then the inner class cannot be accessed by name outside the definition of the outer class.

There are two big advantages to inner classes. First, because they are defined within a class, they can be used to make the outer class self-contained or more self-contained than it would otherwise be. The second advantage is that the inner and outer classes' methods have access to each other's private methods and private instance variables.



TIP: Inner and Outer Classes Have Access to Each Other's Private Members

Within the definition of a method of an inner class, it is legal to reference a private instance variable of the outer class and to invoke a private method of the outer class. To facilitate this, Java follows this convention: If a method is invoked in an inner class and the inner class has no such method, then it is assumed to be an invocation of the method by that name in the outer class. (If the outer class also has no method by that name, that is, of course, an error.) Similarly, an inner class can use the name of an instance variable of the outer class.

The reverse situation, invoking a method of the inner class from the outer class, is not so simple. To invoke a (nonstatic) method of the inner class from within a method of the outer class, you need an object of the inner class to use as a calling object, as we do in Display 13.9.

As long as you are within the definition of the inner or outer classes, the modifiers `public` and `private` (used within the inner or outer classes) are equivalent.

These sorts of invocations and variable references that cross between inner and outer classes can get confusing. So, it is best to confine such invocations and variable references to cases that are clear and straightforward. It is easy to tie your code in knots if you get carried away with this sort of thing. ■

Access Privileges between Inner and Outer Classes

Inner and outer classes have access to each other's private members.

EXAMPLE: A Bank Account Class

Display 13.9 contains a simplified bank account program with an inner class for amounts of money. The bank account class uses values of type `String` to obtain or return amounts of money, such as the amount of a deposit or the answer to a query for the account balance. However, inside the class it stores amounts of money as values of type `Money`, which is an inner class. Values of type `Money` are not stored as `Strings`, which would be difficult to do arithmetic on, nor are they stored as values of type `double`, which would allow round-off errors that would not be acceptable in banking transactions. Instead, the class `Money` stores amounts of money as two integers, one for the dollars and one for the cents. In a real banking program, the class `Money` might have a larger collection of methods, such as methods to do addition, subtraction, and compute percentages, but in this simple example we included only the method for adding an amount of money to the calling object. The outer class `BankAccount` would also have more methods in a real class, but here we included only methods to deposit an amount of money to the account and to obtain the account balance. Display 13.10 contains a simple demonstration program using the class `BankAccount`.

(continued)

EXAMPLE: (continued)

The class `Money` is a private inner class of the class `BankAccount`. So, the class `Money` cannot be used outside of the class `BankAccount`. (Public inner classes are discussed in Section 13.3 and have some subtleties involved in their use.) Because the class `Money` is local to the class `BankAccount`, the name `Money` can be used for the name of another class outside of the class `BankAccount`. (This would be true even if `Money` were a public inner class.)

We have made the instance variables in the class `Money` private following our usual conventions for class members. When we discuss public inner classes, this will be important. However, for use within the outer class (and a private inner class cannot be used anywhere else), there is no difference between public and private or other member modifiers. All instance variables and all methods of the inner class are public to the outer class no matter whether they are marked public or private or anything else. Notice the method `closeAccount` of the outer class. It uses the private instance variables `dollars` and `cents` of the inner class.

This is still very much a toy example, but we will have occasion to make serious use of private inner classes when we discuss linked lists in Chapter 15 and when we study Swing GUIs starting in Chapter 17.

Display 13.9 Class with an Inner Class (part 1 of 2)

```

1  public class BankAccount
2  {
3      private class Money ← The modifier private in this line
4      {                               should not be changed to public.
5          private long dollars;       However, the modifiers public and
6          private int cents;          private inside the inner class Money
7                                     can be changed to anything else and
8                                     it would have no effect on the class
9                                     BankAccount.
10         public Money(String stringAmount)
11         {
12             abortOnNull(stringAmount);
13             int length = stringAmount.length();
14             dollars = Long.parseLong(
15                 stringAmount.substring(0, length - 3));
16             cents = Integer.parseInt(
17                 stringAmount.substring(length - 2, length));
18         }
19
20         public String getAmount()
21         {
22             if (cents > 9)

```

Display 13.9 Class with an Inner Class (part 2 of 2)

```

19         return (dollars + "." + cents);
20     else
21         return (dollars + ".0" + cents);
22     }

23     public void addIn(Money secondAmount)
24     {
25         abortOnNull(secondAmount);
26         int newCents = (cents + secondAmount.cents)%100;
27         long carry = (cents + secondAmount.cents)/100;
28         cents = newCents;
29         dollars = dollars + secondAmount.dollars + carry;
30     }
31     private void abortOnNull(Object o)
32     {
33         if (o == null)
34         {
35             System.out.println("Unexpected null argument.");
36             System.exit(0);
37         }
38     }
39 }
40 private Money balance;

41 public BankAccount ()
42 {
43     balance = new Money("0.00");
44 }

45 public String getBalance()
46 {
47     return balance.getAmount();
48 }

49 public void makeDeposit(String depositAmount)
50 {
51     balance.addIn(new Money(depositAmount));
52 }

53 public void closeAccount()
54 {
55     balance.dollars = 0;
56     balance.cents = 0;
57 }
58 }

```

The definition of the inner class ends here, but the definition of the outer class continues in this display.

To invoke a nonstatic method of the inner class outside of the inner class, you need to create an object of the inner class.

This invocation of the inner class method `getAmount()` would be allowed even if the method `getAmount()` were marked as `private`.

Notice that the outer class has access to the private instance variables of the inner class.

This class would normally have more methods, but we have only included the methods we need to illustrate the points covered here.

Display 13.10 Demonstration Program for the Class `BankAccount`

```

1  public class BankAccount
2  {
3      public static void main(String[] args)
4      {
5          System.out.println("Creating a new account.");
6          BankAccount account = new BankAccount( );
7          System.out.println("Account balance now = $"
8                              + account.getBalance( ));

9          System.out.println("Depositing $100.00");
10         account.makeDeposit("100.00");
11         System.out.println("Account balance now = $"
12                             + account.getBalance( ));
13         System.out.println("Depositing $99.99");
14         account.makeDeposit("99.99");
15         System.out.println("Account balance now = $"
16                             + account.getBalance( ));
17         System.out.println("Depositing $0.01");
18         account.makeDeposit("0.01");
19         System.out.println("Account balance now = $"
20                             + account.getBalance( ));
21         System.out.println("Closing account.");
22         account.closeAccount( );
23         System.out.println("Account balance now = $"
24                             + account.getBalance( ));
25     }
26 }

```

Sample Dialogue

```

Creating a new account.
Account balance now = $0.00
Depositing $100.00
Account balance now = $100.00
Depositing $99.99
Account balance now = $199.99
Depositing $0.01
Account balance now = $200.00
Closing account.
Account balance now = $0.00

```

Helping Inner Classes

You may define a class within another class. The inside class is called an **inner class**. A common and simple use of an inner class is to use it as a helping class for the outer class, in which case the inner class should be marked `private`.

Self-Test Exercises

20. Would the following invocation of `getAmount` in the method `getBalance` of the outer class `BankAccount` still be legal if we change the method `getAmount` of the inner class `Money` from `public` to `private`?

```
public String getBalance()
{
    return balance.getAmount();
}
```

21. Because it does not matter if we make the members of a private inner class `public` or `private`, can we simply omit the `public` or `private` modifiers from the instance variables and methods of a private inner class?
22. Would it be legal to add the following method to the inner class `Money` in Display 13.9? Remember, the question is would it be legal, not would it be sensible.

```
public void doubleBalance()
{
    balance.addIn(balance);
}
```

23. Would it be legal to add the following method to the inner class `Money` in Display 13.9? Remember, the question is would it be legal, not would it be sensible.

```
public void doubleBalance2()
{
    makeDeposit(balance.getAmount());
}
```

The `.class` File for an Inner Class

When you compile any class in Java, it produces a `.class` file. When you compile a class with an inner class, this compiles both the outer class and the inner class and produces two `.class` files. For example, when you compile the class `BankAccount` in Display 13.9, this produces the following two `.class` files:

```
BankAccount.class and BankAccount$Money.class
```

If `BankAccount` had two inner classes, then three `.class` files would be produced.



PITFALL: Other Uses of Inner Classes

In this section, we have shown you how to use an inner class in only one way, namely to create and use objects of the inner class from within the outer class method definitions. There are other ways to use inner classes, but they can involve subtleties. If you intend to use inner classes in any of these other ways, you should consult Section 13.3. ■

13.3 More about Inner Classes

Something deeply hidden had to be behind things.

ALBERT EINSTEIN, Note quoted in *New York Times Magazine* (August 2, 1964), 1964.

In this section, we cover some of the more subtle details about using inner classes. It might be best to treat this section as a reference section and look up the relevant cases as you need them. None of the material in this section is used in the rest of this book.

Static Inner Classes

static

A normal (nonstatic) inner class, which is the kind of inner class we have discussed so far, has a connection between each of its objects and the object of the outer class that created the inner class object. Among other things, this allows an inner class definition to reference an instance variable or invoke a method of the outer class. If you do not need this connection, you can make your inner class **static** by adding the `static` modifier to your inner class definition, as illustrated by the following sample beginning of a class definition:

```
public class OuterClass
{
    private static class InnerClass
    {
```

A static inner class can have nonstatic instance variables and methods, but an object of a static inner class has no connection to an object of the outer class.

You may encounter situations where you need an inner class to be static. For example, if you create an object of the inner class within a static method of the outer class, then the inner class must be static. This follows from the fact that a nonstatic inner class object must arise from an outer class object.

Also, if you want your inner class to itself have static members, then the inner class must be static.

Because a static inner class has no connection to an object of the outer class, you cannot reference an instance variable or invoke a nonstatic method of the outer class within the static inner class.

To invoke a static method of a static inner class within the outer class, simply preface the method name with the name of the inner class and a dot. Similarly, to

name a static variable of a static inner class within the outer class, just preface the static variable name with the name of the inner class and a dot.

Static Inner Class

A **static** inner class is one that is not associated with an object of the outer class. It is indicated by including the modifier `static` in its class heading.

Self-Test Exercises

24. Can you have a static method in a nonstatic inner class?
25. Can you have a nonstatic method in a static inner class?

Public Inner Classes

`public inner
class`

If an inner class is marked with the `public` modifier instead of the `private` modifier, then it can be used in all the ways we discussed so far, but it can also be used outside of the outer class.

The way that you create an object of the inner class outside of the outer class is a bit different for static and nonstatic inner classes. We consider the case of a nonstatic inner class first. When creating an object of a nonstatic inner class, you need to keep in mind that every object of the nonstatic inner class is associated with some object of the outer class. To put it another way, to create an object of the inner class, you must start with an object of the outer class. This has to be true, because an object of the inner class may invoke a method of the outer class or reference an instance variable of the outer class, and you cannot have an instance variable of the outer class unless you have an object of the outer class.

For example, if you change the class `Money` in Display 13.9 from `private` to `public`, so that the class definition begins

```
public class BankAccount
{
    public class Money
```

then you can use an object of the nonstatic inner class `Money` outside of the class `BankAccount` as illustrated by the following:

```
BankAccount account = new BankAccount();
BankAccount.Money amount =
    account.new Money("41.99");
System.out.println(amount.getAmount());
```

This code produces the output

```
41.99
```

Note that the object `amount` of the inner class `Money` is created starting with an object, `account`, of the outer class `BankAccount`, as follows:

```
BankAccount.Money amount =
    account.new Money("41.99");
```

Also, note that the syntax of the second line is *not*

```
new account.Money("41.99"); //Incorrect syntax
```

Within the definition of the inner class `Money`, an object of the inner class can invoke a method of the outer class. However, this is not true outside of the inner class. Outside of the inner class, an object of the inner class can only invoke methods of the inner class. So, we could *not* have continued the previous sample code (which is outside the class `BankAccount` and so outside the inner class `Money`) with the following:

```
System.out.println(amount.getBalance()); //Illegal
```

The meaning of `amount.getBalance()` is clear, but it is still not allowed. If you want something equivalent to `amount.getBalance()`, you should use the corresponding object of the class `BankAccount`; in this case, you would use `account.getBalance()`. (Recall that `account` is the `BankAccount` object used to create the inner class object `amount`.)

Now let's consider the case of a static inner class. You can create objects of a public *static* inner class and do so outside of the inner class—in fact, even outside of the outer class. To do so outside of the outer class, the situation is similar to, but not exactly the same as, what we outlined for nonstatic inner classes. Consider the following outline:

```
public class OuterClass
{
    public static class InnerClass
    {
        public void nonstaticMethod()
        { ... }

        public static void staticMethod()
        {...}

        Other_Members_of_InnerClass
    }

    Other_Members_of_OuterClass
}
```

You can create an object of the inner class outside of the outer class as in the following example:

```
OuterClass.InnerClass innerObject =
    new OuterClass.InnerClass();
```

Note that the syntax is *not*

```
OuterClass.new InnerClass();
```

This may seem like an apparent inconsistency with the syntax for creating the object of a nonstatic inner class. It may help to keep in mind that for a static inner class, `OuterClass.InnerClass` is a well-specified class name and all the information for the object is in that class name. To remember the syntax for a nonstatic inner class, remember that for that case, the object of the outer class modifies how the `new` operator works to create an object of the inner class.

Once you have created an object of the inner class, the object can invoke a nonstatic method in the usual way. For example,

```
innerObject.nonstaticMethod();
```

You can also use the object of the inner class to invoke a static method in the same way. For example,

```
innerObject.staticMethod();
```

However, it is more common, and clearer, to use class names when invoking a static method. For example,

```
OuterClass.InnerClass.staticMethod();
```



TIP: Referring to a Method of the Outer Class

As we have already noted, if a method is invoked in an inner class and the inner class has no such method, then it is assumed to be an invocation of the method by that name in the outer class. For example, we could add a method `showBalance` to the inner class `Money` in Display 13.9, as outlined in what follows:

```
public class BankAccount
{
    private class Money
    {
        private long dollars;
        private int cents;

        public void showBalance()
        {
            System.out.println(getBalance());
        }
        ...
    } //End of Money
}
```

(continued)



TIP: (continued)

```

        public String getBalance()
        {...}
        ...
    }    //End of BankAccount

```

This invocation of `getBalance` is within the definition of the inner class `Money`. But the inner class `Money` has no method named `getBalance`, so it is presumed to be the method `getBalance` of the outer class `BankAccount`.

But suppose the inner class did have a method named `getBalance`; then this invocation of `getBalance` would be an invocation of the method `getBalance` defined in the inner class.

If both the inner and outer classes have a method named `getBalance`, then you can specify that you mean the method of the outer class as follows:

```

public void showBalance()
{
    System.out.println(
        BankAccount.this.getBalance());
}

```

The syntax

Outer_Class_Name.this.Method_Name

always refers to a method of the outer class. In the example, `BankAccount.this` means the `this` of `BankAccount`, as opposed to the `this` of the inner class `Money`. ■

Self-Test Exercises

26. Consider the following class definition:

```

public class OuterClass
{
    public static class InnerClass
    {
        public static void someMethod()
        {
            System.out.println("From inside.");
        }
    }
    Other_Members_of_OuterClass
}

```

Write an invocation of the static method `someMethod` that you could use in some class you define.

Self-Test Exercises (continued)

27. Consider the following class definition:

```
public class Outs
{
    private int outerInt = 100;

    public class Ins
    {
        private int innerInt = 25;
        public void specialMethod()
        {
            System.out.println(outerInt);
            System.out.println(innerInt);
        }
    }
    Other_Members_of_OuterClass
}
```

Write an invocation of the method `specialMethod` with an object of the class `Ins`. Part of this exercise is to create the object of the class `Ins`. This should be code that you could use in some class you define.

Nesting Inner Classes

It is legal to nest inner classes within inner classes. The rules are the same as what we have already discussed except that names can get longer. For example, if `A` has a public inner class `B`, and `B` has a public inner class `C`, then the following is valid code:

```
A aObject = new A();
A.B bObject =
    aObject.new B();
A.B.C cObject =
    bObject.new C();
```

Inner Classes and Inheritance

Suppose `OuterClass` has an inner class named `InnerClass`. If you derive `DerivedClass` from `OuterClass`, then `DerivedClass` automatically has `InnerClass` as an inner class just as if it were defined within `DerivedClass`.

Just as with any other kind of class in Java, you can make an inner class a derived class of some other class. You can also make the outer class a derived class of a different (or the same) base class.

It is not possible to override the definition of an inner class when you define a derived class of the outer class.

It is also possible to use an inner class as a base class to derive classes, but we will not go into those details in this book; there are some subtleties to worry about.

anonymous class

Anonymous Classes

If you wish to create an object but have no need to name the object's class, then you can embed the class definition inside the expression with the `new` operator. These sorts of class definitions are called **anonymous classes** because they have no class name. An expression with an anonymous class definition is, like everything in Java, inside of some class definition. Thus, an anonymous class is an inner class. Before we go into the details of the syntax for anonymous classes, let's say a little about where one might use them.

The most straightforward way to create an object is the following:

```
YourClass anObject = new YourClass();
```

If `new YourClass()` is replaced by some expression that defines the class but does not give the class any name, then there is no name `YourClass` to use to declare the variable `anObject`. So, it does not make sense to use an anonymous class in this situation. However, it can make sense in the following scenario:

```
SomeOtherType anObject = new YourClass();
```

Here `SomeOtherType` must be a type such that an object of the class `YourClass` is also an object of `SomeOtherType`. In this case, you can replace `new YourClass()` with an expression including an anonymous class instead of `YourClass`. The type `SomeOtherType` is usually a Java interface.

Here is an example of an anonymous class. Suppose you define the following interface:

```
public interface NumberCarrier
{
    public void setNumber(int value);
    public int getNumber();
}
```

Then the following creates an object using an anonymous class definition:

```
NumberCarrier anObject = new NumberCarrier()
{
    private int number;
    public void setNumber(int value)
    {
        number = value;
    }
    public int getNumber()
    {
        return number;
    }
};
```

The part in the braces is the same as the part inside the main braces of a class definition. The closing brace is followed by a semicolon, unlike a class definition. (This is because the entire expression will be used as a Java statement.) The beginning part, repeated as follows, may seem strange:

```
new NumberCarrier()
```

The `new` is sensible enough but what is the point of `NumberCarrier()`? It looks like this is an invocation of a constructor for `NumberCarrier`. But, `NumberCarrier` is an interface and has no constructors. The meaning of `new NumberCarrier()` is simply

```
implements NumberCarrier
```

So what is being said is that the anonymous class implements the `NumberCarrier` interface and is defined as shown between the main braces.

Display 13.11 shows a simple demonstration with two anonymous class definitions. For completeness, we have also repeated the definition of the `NumberCarrier` interface in this display.

Display 13.11 Anonymous Classes (part 1 of 2)

<pre> 1 public class AnonymousClassDemo 2 { 3 public static void main(String[] args) 4 { 5 NumberCarrier anObject = 6 new NumberCarrier() 7 { 8 private int number; 9 public void setNumber(int value) 10 { 11 number = value; 12 } 13 public int getNumber() 14 { 15 return number; 16 } 17 }; 18 19 NumberCarrier anotherObject = 20 new NumberCarrier() 21 { 22 private int number; 23 public void setNumber(int value) 24 { 25 number = 2*value; </pre>	<i>This is just a toy example to demonstrate the Java syntax for anonymous classes.</i>
--	--

(continued)

Display 13.11 Anonymous Classes (part 2 of 2)

```

26         public int getNumber()
27         {
28             return number;
29         }
30     };

31     anObject.setNumber(42);
32     anotherObject.setNumber(42);
33     showNumber(anObject);
34     showNumber(anotherObject);
35     System.out.println("End of program.");
36 }

37 public static void showNumber(NumberCarrier o)
38 {
39     System.out.println(o.getNumber());
40 }

41 }

```

This is the file AnonymousClassDemo.java.

Sample Dialogue

```

42
84
End of program.

```

```

1  public interface NumberCarrier This is the file NumberCarrier.java.
2  {
3      public void setNumber(int value);
4      public int getNumber();
5  }

```

**TIP: Why Use Inner Classes?**

Most simple situations do not need inner classes. However, there are situations for which inner classes are a good solution. For example, suppose you want to have a class with two base classes. This is not allowed in Java. However, you can have an outer class derived from one base class with an inner class derived from the other base class. Because the inner and outer classes have access to each other's instance variables and methods, this can often serve as if it were a class with two base classes.

As another example, if you need only one object of a class and the class definition is very short, many programmers like to use an anonymous class (but I must admit I am not one of them).

When we study *linked lists* in Chapter 15, you will see cases where using an inner class as a helping class makes the linked list class self-contained in a very natural way. We will also use inner classes when defining Graphical User Interfaces (GUIs) starting in Chapter 17. But until you learn what linked lists and GUIs are, these are not likely to be compelling examples. ■

Self-Test Exercise

28. Suppose we replace

```
NumberCarrier anObject
```

with

```
Object anObject
```

in Display 13.11. What would be the first statement in the program to cause an error message? Would it be a compiler error message or a run-time error message?

Chapter Summary

- An *interface* is a property of a class that says what methods a class that implements the interface must have.
- An interface is defined the same way as a class is defined except that the keyword `interface` is used in place of the keyword `class` and method bodies are replaced by semicolons.
- An interface may not have any instance variables, with one exception: An interface may have defined constants. If you use the syntax for an instance variable in an inner class, the variable is automatically a constant, not a real instance variable.
- An *inner class* is a class defined within another class.
- One simple use of an inner class is as a helping class to be used in the definition of the outer class methods and/or instance variables.
- A *static* inner class is one that is not associated with an object of the *outer class*. It must include the modifier `static` in its class heading.
- To create an object of a nonstatic inner class outside the definition of the outer class, you must first create an object of the outer class and use it to create an object of the inner class.

Answers to Self-Test Exercises

1. Yes to both. An interface is a type and can be used like any other type.
2. Yes. Any of the interface methods that it does not fully define must be made abstract methods.
3. A derived class can have only one base class, but it can implement any number of interfaces.

4. No, but the way to accomplish the same thing is to have one interface extend the other.

These exercises are for the material on the `Comparable` interface.

5. Yes, the dialogue would be the same. The change from the parameter type `Comparable[]` to `Object[]` in the method interchange is in fact a good idea.
6. No. This will compile without any error messages. However, the less-than ordering does not satisfy the semantics of the `Comparable` interface. For example, the trichotomy law does not hold.
7. Yes. The three required conditions are true for objects of the class `Circle`:
 (Irreflexive) By definition, no circle is inside itself.
 (Trichotomy) For any two circles `c1` and `c2` with centers at the origin, one, and only one, of the following holds true: `c1` is inside of `c2`, `c2` is inside of `c1`, or `c1` equals `c2`.
 (Transitivity) If `c1` is inside of `c2` and `c2` is inside of `c3`, then `c1` is inside of `c3`.
8. The class will produce a compiler error message saying that there is an inconsistency in the definitions of `ANSWER`.
9. The class will compile and run with no error messages. Because the named constant `ANSWER` is never used, there is no inconsistency.
10. The class will produce a compiler error message saying that you have not implemented the heading for `getStuff` in `InterfaceA`.
11. They will all compile and the program will run. The two definitions of `getStuff` have different numbers of parameters, so this is overloading. There is no inconsistency.

12.

```
public class StockItem implements Cloneable
{
    private int number;
    private String name;
    public void setNumber(int newNumber)
    {
        number = newNumber;
    }
    ...
    public Object clone()
    {
        try
        {
            return super.clone();
        }
        catch (CloneNotSupportedException e)
        {
            //This should not happen.
            return null; //To keep compiler happy.
        }
    }
}
```

13. Note that you do not catch a `CloneNotSupportedException` because any such thrown exception in `super.clone` is caught inside the base class method `super.clone`.

```
public class PricedItem extends StockItem
    implements Cloneable
{
    private double price;
    ...
    public Object clone()
    {
        return super.clone();
    }
}

14. public class Record implements Cloneable
{
    private StockItem item1;
    private StockItem item2;
    private String description;

    ...
    public Object clone()
    {
        try
        {
            Record copy =
                (Record) super.clone();
            copy.item1 =
                (StockItem) item1.clone();
            copy.item2 =
                (StockItem) item2.clone();
            return copy;
        }
        catch (CloneNotSupportedException e)
        { //This should not happen.
            return null; //To keep compiler happy.
        }
    }
}
```


15. Note that you do not catch a `CloneNotSupportedException` because any such thrown exception in `super.clone` is caught inside the base class method `super.clone`.

```
public class BigRecord extends Record
    implements Cloneable
{
    private StockItem item3;

    ...
    public Object clone()
    {
        BigRecord copy =
            (BigRecord) super.clone();
        copy.item3 =
            (StockItem) item3.clone();
        return copy;
    }
}
```

16. The heading of the class definition changes to what is shown in the following and the method `clone` shown there is added. The version of `Date` for this chapter on the accompanying website includes this definition of `clone`.

```
public class Date implements Cloneable
{
    private String month;
    private int day;
    private int year;

    ...
    public Object clone()
    {
        try
        {
            return super.clone(); //Invocation of
                                //clone in the base class Object
        }
        catch (CloneNotSupportedException e)
        { //This should not happen.
            return null; //To keep compiler happy.
        }
    }
}
```

17. The heading of the class definition changes to what is shown in the following and the method `clone` shown there is added. The version of `Employee` for this chapter on the accompanying website includes this definition of `clone`.

extra code
on website

extra code
on website

```

public class Employee implements Cloneable
{
    private String name;
    private Date hireDate;

    ...
    public Object clone()
    {
        try
        {
            Employee copy =
                (Employee) super.clone();
            copy.hireDate =
                (Date) hireDate.clone();
            return copy;
        }
        catch (CloneNotSupportedException e)
        { // This should not happen.
            return null; // To keep compiler happy.
        }
    }
}

```

18. The heading of the class definition changes to what is shown in the following, and the method `clone` shown there is added. Note that you do not catch a `CloneNotSupportedException` because any such thrown exception in `super.clone` is caught inside the base class method `super.clone`. The version of `HourlyEmployee` for this chapter on the accompanying website includes this definition of `clone`.

extra code
on website

```

public class HourlyEmployee extends Employee
                               implements Cloneable
{
    private double wageRate;
    private double hours;

    ...
    public Object clone()
    {
        HourlyEmployee copy =
            (HourlyEmployee) super.clone();
        return copy;
    }
}

```

19. It would still be legal. An outer class has access to all the private members of an inner class.
20. Yes, they can be omitted, but the reason is that it indicates package access, and in a private inner class, all privacy modifiers, including package access, are equivalent to public. (Note that the situation for public inner classes will be different.)

21. Yes, it is legal to add the method `doubleBalance` to the inner class `Money` because an inner class has access to the instance variables, such as `balance` of the outer class. To test this out, add the following as a method of the outer class:

```
public void test()
{
    balance.doubleBalance();
}
```

22. It would be legal. The method `makeDeposit` is assumed to be the method `makeDeposit` of the outer class. The calling object `balance` is assumed to be the instance variable of the outer class. These sorts of tricks can lead to confusing code. So, use them sparingly. This is just an exercise.
23. No, a nonstatic inner class cannot have any static methods.
24. Yes, you can have a nonstatic method in a static inner class.

25. `OuterClass.InnerClass.someMethod();`

26. `Outs outerObject = new Outs();`
`Outs.Ins innerObject =`
`outerObject.new Ins();`
`innerObject.specialMethod();`

27. You would get your first error on the following statement and it would be a compiler error:

```
anObject.setNumber(42);
```

With the change described in the exercise, `anObject` is of type `Object` and `Object` has no method named `setNumber`.

28. Line 31 will cause a compiler error because at the level of `Object` because there is no `setNumber` method.

Programming Projects



VideoNote
Solution to
Programming
Project 13.1

1. Modify the recursive implementation of binary search from Chapter 11 so that the search method works on any array of type `Comparable[]`. Test the implementation with arrays of different types to see if it works.
2. Listed next is the skeleton for a class named `City`. Each city has a name and temperature:

```
class City
{
    private String cityName;
    private double temperature;
}
```

Flesh out the class with appropriate accessors, constructors, and mutators. The temperatures are assigned by you and can be set from outside the `City` class—your

code does not have to ensure that they are unique. Next, modify the class so that it implements the `Comparable` interface. The class also overrides the `compareTo` method. This method imposes an order between instances of the `City` class depending upon their temperature. Test your class by creating an array of sample temperatures and sort them in an ascending order using a sorting method that takes as input an array of type `Comparable`.

3. Listed next is a code skeleton for an interface called `Enumeration` and a class called `NameCollection`. `Enumeration` provides an interface to sequentially iterate through some type of collection. In this case, the collection will be the class `NameCollection` that simply stores a collection of names using an array of strings.

```
interface Enumeration
{
    // Returns true if another element in the collection exists
    public boolean hasNext();

    // Returns the next element in the collection as an Object
    public Object getNext();
}

/**
 * NameCollection implements a collection of names using
 * a simple array.
 */
class NameCollection
{
    String[] names;

    /**
     * The list of names is initialized from outside
     * and passed in as an array of strings
     */
    NameCollection(String[] names)
    {
        this.names = names;
    }

    /**
     * getEnumeration should return an instance of a class that
     * implements
     * the Enumeration interface where hasNext() and getNext()
     * correspond to data stored within the names array.
     */
    Enumeration getEnumeration ()
    {
        // Complete code here using an inner class
    }
}
```

Complete the method `getEnumeration()` so that it returns an anonymous inner class that corresponds to the `Enumeration` interface for the names array in `NamesCollection`. Then write a main method that creates a `NamesCollection` object with a sample array of strings, retrieves the `Enumeration` for this class via `getEnumeration()`, and then iterates through the enumeration outputting each name using the `getNext()` method.

4. In Display 13.5, we described a sorting method to sort an array of type `Comparable[]`. In Display 12.6, we described a sorting method that used the merge sort algorithm to sort an array of type `double[]` into increasing order. Redo the method in Display 12.6 so it applies to an array of type `Comparable[]`. Also, do a suitable test program.
5. In Display 13.5, we described a sorting method to sort an array of type `Comparable[]`. In Display 12.8, we described a sorting method that used the quick sort algorithm to sort an array of type `double[]` into increasing order. Redo the method in Display 12.8 so it applies to an array of type `Comparable[]`. Also, do a suitable test program.
6. Redo the class `Person` in Display 5.19 so that it implements the `Cloneable` interface. This may require that you also redo the class `Date` so it implements the `Cloneable` interface. Also, do a suitable test program.
7. Redo the class `Person` in Display 5.19 so that the class `Date` is a private inner class of the class `Person`. Also, do a suitable test program. (You need not start from the version produced in Programming Project 13.6. You can ignore Programming Project 13.6 when you do this project.)
8. This is a combination of Programming Projects 13.6 and 13.7. Redo the class `Person` in Display 5.19 so that the class `Date` is a private inner class of the class `Person`, and so that the class `Person` implements the `Cloneable` interface. Also, do a suitable test program.
9. Redo the class `Employee` and the class `HourlyEmployee` in Displays 7.2 and 7.3 so that the class `Date` is an inner class of the class `Employee` and an inherited inner class of the class `HourlyEmployee`. Also, do a suitable test program.
10. Define an interface named `Shape` with a single method named `area` that calculates the area of the geometric shape:

```
public double area();
```

Next, define a class named `Circle` that implements `Shape`. The `Circle` class should have an instance variable for the radius, a constructor that sets the radius, accessor/mutator methods for the radius, and an implementation of the `area` method. Also define a class named `Rectangle` that implements `Shape`. The `Rectangle` class should have instance variables for the height and width, a constructor that sets the height and width, accessor and mutator methods for the height and width, and an implementation of the `area` method.

The following test code should then output the area of the `Circle` and `Rectangle` objects:

```
public static void main(String[] args)
{
    Circle c = new Circle(4); // Radius of 4
    Rectangle r = new Rectangle(4,3); // Height = 4, Width = 3
    ShowArea(c);
    ShowArea(r);
}

public static void ShowArea(Shape s)
{
    double area = s.area();
    System.out.println("The area of the shape is " + area);
}
```



VideoNote

**Solution to
Programming
Project 13.11**

11. Create a `Student` class that has instance variables for the student's last name and ID number, along with appropriate constructors, accessors, and mutators. Make the `Student` class implement the `Comparable` interface. Define the `compareTo` method to order `Student` objects based on the student ID number. In the `main` method, create an array of at least five `Student` objects, sort them using `Arrays.sort`, and output the students. They should be listed by ascending student number. Next, modify the `compareTo` method so it orders `Student` objects based on the lexicographic ordering of their last name. Without modification to the `main` method, the program should now output the students ordered by name.

This page intentionally left blank



Generics and the ArrayList Class **14**

14.1 THE ArrayList CLASS 797

Using the ArrayList Class 798

Methods in the Class ArrayList 803

The “for-each” Loop 806

Example: Golf Scores 809

The Vector Class 813

Parameterized Classes and Generics 814

14.2 GENERICS 814

Generic Basics 815

Example: A Generic Class for Ordered Pairs 817

Bounds for Type Parameters 825

Generic Methods ★ 828

Inheritance with Generic Classes ★ 830

14 Generics and the ArrayList Class

Hamlet: Do you see yonder cloud that's almost in shape of a camel?

Polonius: By the mass, and 'tis like a camel, indeed.

Hamlet: Me think it is like a weasel.

Polonius: It is backed like a weasel.

Hamlet: Or like a whale.

Polonius: Very like a whale.

WILLIAM SHAKESPEARE, *Hamlet*, 1603.

Introduction

generics

Beginning with version 5.0, Java allows class and method definitions that include parameters for types. Such definitions are called **generics**. Generic programming with a type parameter allows you to write code that applies to any class. For example, you can define a class for a list of items of type `T`, where `T` is a type parameter. You can then use this class with the class `String` plugged in for `T` to automatically get a class for a list of `String` objects. Similarly, you can plug in the class `Double` for `T` to obtain a class for a list of `Doubles`, and you can do a comparable thing for any other class. The class `ArrayList` in the standard Java libraries is, in fact, just such a class for a list of items of type `T`, where `T` is a type parameter. We will first show you how to use classes with a type parameter by using the `ArrayList` class as an example. We will then tell you how you can define other classes with a type parameter.

Prerequisites

Section 14.1 covering the `ArrayList` class requires only Chapters 1 through 6 and Chapter 9. It can reasonably be read without first reading Chapter 9 if you ignore all references to “exceptions.”

Section 14.2 on generics requires Chapters 1 through 7 and Chapter 9. (There is one very short Tip section entitled “Generic Interfaces” that requires Section 13.1 on interfaces, but that Tip section can easily be skipped if you have not yet read Section 13.1.) You need not read Section 14.1 before Section 14.2, but you are encouraged to do so; Section 14.1 can serve as a motivation for Section 14.2.

14.1 The ArrayList Class

"Well, I'll eat it," said Alice, "and if it makes me grow larger, I can reach the key; and if it makes me grow smaller, I can creep under the door; so either way I'll get into the garden. . . ."

LEWIS CARROLL, *Alice's Adventures In Wonderland*, Macmillan, 1865.

ArrayList

`ArrayList` is a class in the standard Java libraries. You can think of an `ArrayList` object as an array that can grow (and shrink) in length while your program is running. In Java, you can read in the length of an array when the program is running, but once your program creates an array of that length, it cannot change the length of the array. For example, suppose you write a program to record customer orders for a mail-order house, and suppose you store all the orders for one customer in an array of objects of some class called `Item`. You could ask the user how many items she or he will order, store the number in a variable called `numberOfItems`, and then create the array `item` with the following statement:

```
Item[] item = new Item[numberOfItems];
```

But suppose the customer enters `numberOfItems` and then decides to order another item? There is no way to increase the size of the array `item`. There are ways around this problem with arrays, but they are all rather complicated. `ArrayLists` serve the same purpose as arrays, except that an `ArrayList` can change length while the program is running. So an `ArrayList` could handle the customer's extra order without any problems.

The class `ArrayList` is implemented using an array as a private instance variable. When this hidden array is full, a new larger hidden array is created, and the data is transferred to this new array. However, you need not concern yourself with this implementation detail. All you need to know is how to use the `ArrayList` class, and we are about to tell you that.

If `ArrayLists` are like arrays but have the nice added feature of being able to change length, then why don't we just always use `ArrayLists` instead of arrays? It often seems that every silver lining has a cloud, and this is true of `ArrayLists` as well. There are three main disadvantages of `ArrayLists`: (1) They are less efficient than arrays; (2) they do not have the square bracket notation, and so using an `ArrayList` is sometimes notationally more awkward than using ordinary arrays; and (3) the base type of an `ArrayList` must be class type (or other reference type); it cannot be a primitive type, such as `int`, `double`, or `char`. For example, if you want an `ArrayList` of `int` values, you must simulate this structure with an `ArrayList` of `Integer` values, where `Integer` is the wrapper class whose objects simulate `int` values. Automatic boxing and unboxing (as discussed in Chapter 5) make (3) less of a problem, because an `ArrayList` with base type, for example, `Integer` can, in effect, store values of type `int`.

Using the ArrayList Class

**import
statement**

ArrayLists are used in much the same way as arrays, but there are some important differences. First, the definition of the class `ArrayList` is not provided automatically. The definition is in the package `java.util`, and any code that uses the class `ArrayList` must contain the following, normally at the start of the file:

```
import java.util.ArrayList;
```

An `ArrayList` is created and named in the same way as objects of any class, except that you specify the base type using a new notation. For example,

```
ArrayList<String> list = new ArrayList<String>(20);
```

capacity

This statement makes `list` the name of an `ArrayList` that stores objects of the class `String` and that has an *initial capacity* of 20 items. When we say that an `ArrayList` has a certain capacity, we mean that it has been allocated memory for that many items, but if it needs to hold more items, the system will automatically allocate more memory. By carefully choosing the initial capacity of an `ArrayList`, you can often make your code more efficient, but this capacity has no effect on how many items the `ArrayList` can hold. If you choose your capacity to be large enough, then the system will not need to reallocate memory too often, and as a result, your program should run faster. On the other hand, if you make your capacity too large, you will waste storage space. However, no matter what capacity you choose, you can still do anything you want with the `ArrayList`.

Java 7 supports a slightly shorter but equivalent way to define the `ArrayList`. In this format, the base type in the call to the constructor is not needed. For example,

```
ArrayList<String> list = new ArrayList<>(20);
```

**type
inference
base type**

This feature is called **type inference** and is briefly discussed in Section 14.2.

The type `String` in the previous `ArrayList` example is the **base type** of the `ArrayList` class. An `ArrayList`—that is, an object of the `ArrayList` class—stores objects of its base type. You can use any reference type as the base type of an `ArrayList` class. In particular, you can use any class or interface type. However, you cannot use a primitive type, such as `int` or `double`, as the base type of an `ArrayList` class. This is an example of a type parameter. The `ArrayList` class is defined as having a type parameter for the type of the elements in the list. Create a concrete class by specifying, in angular brackets, a class type to be substituted for this type parameter. For example, the following code, which we saw earlier in this section, substitutes the type `String` for the type parameter to create the class `ArrayList<String>` and an object of this class named `list`:

```
ArrayList<String> list = new ArrayList<String>(20);
```

`ArrayList` objects can be used like arrays, but they do not have the array square-bracket notation. If you use

```
a[index] = "Hi Mom!";
```

set

for an array of strings `a`, then the analogous statement for a suitable `ArrayList` named `list` is

```
list.set(index, "Hi Mom!");
```

Creating and Naming an ArrayList Object

An object of the class `ArrayList` is created and named in the same way as any other object, except that you specify the base type of the `ArrayList`.

SYNTAX

```
ArrayList<Base_Type> Object_Name = new ArrayList<Base_Type>();
ArrayList<Base_Type> Object_Name =
    new ArrayList<Base_Type>(Initial_Capacity);
```

The *Base_Type* must be a reference type, usually a class type; it cannot be a primitive type such as `int` or `double`. When a number is given as an argument to the constructor, that number determines the initial capacity of the `ArrayList`.

EXAMPLES

```
ArrayList<String> list = new ArrayList<String>();
ArrayList<Double> list2 = new ArrayList<Double>(30);
```

If you would use

```
String temp = a[index];
```

for an array of strings `a`, then the analogous statement for a suitable `ArrayList` named `list` would be

```
String temp = list.get(index);
```

Accessing at an Index

If `list` is an `ArrayList`, its elements can be accessed as follows:

EXAMPLES

```
ArrayList<String> list = new ArrayList<String>();
int index;
...

list.set(index, "Here"); //Sets the element
                        //at index to "Here".
String temp = list.get(index); //The expression
                        //list.get(index)
                        //returns the element
                        //at position index.
ArrayList<Integer> list2 = new ArrayList<Integer>();
...
list2.set(42, index); //Sets the element at index to
                    //new Integer(42). This example relies on
                    // automatic boxing.
```

(continued)

```
int temp2 = list2.get(index); //The expression
                             //v.get(index) returns the
                             //element at position index.
                             // This example
                             //relies on automatic unboxing.
```

The index must be greater than or equal to 0 and less than the current size of the ArrayList list.

Creating and Using an ArrayList

The two methods `set` and `get` give ArrayLists approximately the same functionality that square brackets give to arrays. However, you need to be aware of one important point: The method invocation

```
list.set(index, "Hi Mom!");
```

is *not* always completely analogous to

```
a[index] = "Hi Mom!";
```

add

The method `set` can replace any existing element, but you cannot use `set` to put an element at just any index, as you could with an array. The method `set` is used to change the value of elements, not to set them for the first time. To set an element for the first time, you usually use the method `add`. The basic form of the method `add` adds elements at index position 0, position 1, position 2, and so forth in that order. This means that ArrayLists must always be filled in this order. But your code can then go back and change any individual element, just as it can in an array.

For example, suppose `list` is an ArrayList with base type `String`, which has not yet had any elements added to it; that is, `list` is empty. The following statements will add the strings "One", "Two", and "Three" to positions 0, 1, and 2:

```
list.add("One");
list.add("Two");
list.add("Three");
```

The method name `add` is overloaded. There is also a two-argument method named `add` that allows you to add an element at any currently used index position or at the first unused position. When inserting into an ArrayList with this version of `add`, elements at the specified index and higher (if any) are moved up one position to make room for the new element. For example,

```
list.add(0, "Zero");
```

adds the string "Zero" at position 0 and moves elements originally at positions 0, 1, 2, and so forth up one position to positions 1, 2, 3, and so forth.

Suppose `list` starts out empty and your code executes our four `add` invocations, which we repeat below:

```
list.add("One");  
list.add("Two");  
list.add("Three");  
list.add(0, "Zero");
```

After these four invocations of `list.add`, the `list` would contain the strings "Zero", "One", "Two", and "Three" in positions 0, 1, 2, and 3, respectively.

Note that the two-argument version of `add` cannot add an element at just any position. It can only insert an element at some already used position or at the first unused position. The elements in an `ArrayList` always occupy a contiguous set of positions starting at 0; that is, they are always at positions 0, 1, 2, and so forth up to some last position. This is just like a partially filled array (as discussed in Chapter 6), but unlike a partially filled array, you do not need to do anything to keep track of how many elements are on the list. The method `size` automatically takes care of this for any `ArrayList`.

The add Methods

Elements can be added to an `ArrayList` by using the methods named `add`. The elements are added to index position 0, then 1, then 2, and so forth so there are no gaps in the indices of elements.

The most straightforward method to use for adding to an `ArrayList` is the method named `add` that has only one parameter.

EXAMPLES

```
list.add("Salud");  
list.add("Dinero");  
list.add("Java");
```

The object `list` is an `ArrayList` with base type `String`.

A second method named `add` allows you to add an element at any currently used index position or at the first unused position. When inserted into an `ArrayList` with this version of `add`, elements at the specified index and higher (if any) are moved up one position to make room for the new element.

```
list.add(1, "Amor");
```

If `list` starts out empty and all four statements in the two sets of examples are executed, then `list` would contain the following strings in the order given: "Salud", "Amor", "Dinero", and "Java".

`size` You can find out how many indices already have elements by using the method `size`. If `list` is an `ArrayList`, `list.size()` returns the **size** of the `ArrayList`, which is the number of elements stored in it. The indices of these elements go from 0 to one less than `list.size()`.

The size Method

The method `size` returns the number of elements in an `ArrayList`.

EXAMPLE

```
for (int index = 0; index < list.size(); index++)  
    System.out.println(list.get(index));
```

`list` is an `ArrayList` object.



add

TIP: Summary of Adding to an ArrayList

To place an element in an `ArrayList` position (at an `ArrayList` index) for the first time, you usually use the method `add`. The simplest method named `add` has a single parameter for the element to be added and adds elements at index positions 0, 1, 2, and so forth, in that order.

You can add an element at an already occupied list position by using the two-parameter version of `add`. When inserting into an `ArrayList` with this version of `add`, elements at the specified index and higher are moved up one position to make room for the new element. For example, suppose `list` is an `ArrayList` object with base type `String` that has three elements already on its list. Consider the following method invocation:

```
list.add(1, "Amor");
```

Before, there were elements at index positions 0, 1, and 2. When this invocation of `add` is executed, the element "Amor" is inserted at index 1, and the elements at index positions 1 and 2 are moved to positions 2 and 3.

You can also use the two-argument version of `add` to add an element at the first unused position. If `list` has elements at index positions 0, 1, 2, and 3, then the following is legal:

```
list.add(4, "Mucho Amor");
```

Your code can then go back and change any individual element, using `set`. However, `set` can reset only the element at an index that already has an element.

The method `size` can be used to determine how many elements are stored in an `ArrayList`. ■

Self-Test Exercises

1. Suppose `list` is an object of the class `ArrayList<String>`. How do you add the string "Hello" to the `ArrayList` `list`?
2. Suppose `instruction` is an object of the class `ArrayList<String>` that contains the string "Stop" at index position 5. How do you change the string at index position 5 to "Go" (without changing any of the elements at other positions)?
3. Suppose `instruction` is an object of the class `ArrayList<String>` that contains strings at index positions 0 through 10. How do you insert the string "Go" at index position 5 so that no strings are removed from the list `instruction`?

Self-Test Exercises (continued)

4. Can you use the method `set` to place an element in an `ArrayList` at any index you want?
5. Can you use the two-argument version of the method `add` to add an element in an `ArrayList` at any index you want?
6. Consider the following two method invocations. Are there values of `index1` that are allowed but that are not allowed for `index2`? Are there values of `index2` that are allowed but that are not allowed for `index1`? `list` is an object of the class `ArrayList<String>`.

```
list.set(index1, "Hello");
list.add(index2, "Hello");
```

7. If you create an `ArrayList` with the following statement, can the `ArrayList` contain more than 20 elements?

```
ArrayList<Double> myList = new ArrayList<Double>(20);
```

Methods in the Class `ArrayList`

With arrays, the square brackets and the instance variable `length` are the only tools automatically provided for you, the programmer. If you want to use arrays for other things, you must write code to manipulate the arrays. `ArrayLists`, on the other hand, come with a selection of powerful methods that can do many of the things for which you would need to write code in order to do with arrays. For example, the class `ArrayList` has a version of the method `add` that inserts a new element between two elements in the `ArrayList`. Most of these methods are described in Display 14.1.

Display 14.1 Some Methods in the Class `ArrayList` (part 1 of 3)

CONSTRUCTORS

```
public ArrayList<Base_Type>(int initialCapacity)
```

Creates an empty `ArrayList` with the specified *Base_Type* and initial capacity.

```
public ArrayList<Base_Type>()
```

Creates an empty `ArrayList` with the specified *Base_Type* and an initial capacity of 10.

ARRAYLIKE METHODS

```
public Base_Type set(int index, Base_Type newElement)
```

Sets the element at the specified `index` to `newElement`. Returns the element previously at that position, but the method is often used as if it were a `void` method. If you draw an analogy between the `ArrayList` and an array `a`, this statement is analogous to setting a `[index]` to the value `newElement`. The `index` must be a value greater than or equal to 0 and less than the current size of the `ArrayList`. Throws an `IndexOutOfBoundsException` if the `index` is not in this range.

(continued)

Display 14.1 Some Methods in the Class `ArrayList` (part 2 of 3)

```
public Base_Type get(int index)
```

Returns the element at the specified index. This statement is analogous to returning a `a[index]` for an array `a`. The `index` must be a value greater than or equal to 0 and less than the current size of the `ArrayList`. Throws `IndexOutOfBoundsException` if the `index` is not in this range.

METHODS TO ADD ELEMENTS

```
public boolean add(Base_Type newElement)
```

Adds the specified element to the end of the calling `ArrayList` and increases the `ArrayList`'s size by one. The capacity of the `ArrayList` is increased if that is required. Returns `true` if the add is successful. (The return type is `boolean`, but the method is typically used as if it were a `void` method.)

```
public void add(int index, Base_Type newElement)
```

Inserts `newElement` as an element in the calling `ArrayList` at the specified index. Each element in the `ArrayList` with an index greater than or equal to `index` is shifted upward to have an index that is one greater than the value it had previously. The `index` must be a value greater than or equal to 0 and less than or equal to the current size of the `ArrayList`. Throws `IndexOutOfBoundsException` if the `index` is not in this range. Note that you can use this method to add an element after the last element. The capacity of the `ArrayList` is increased if that is required.

METHODS TO REMOVE ELEMENTS

```
public Base_Type remove(int index)
```

Deletes and returns the element at the specified index. Each element in the `ArrayList` with an index greater than `index` is decreased to have an index that is one less than the value it had previously. The `index` must be a value greater than or equal to 0 and less than the current size of the `ArrayList`. Throws `IndexOutOfBoundsException` if the `index` is not in this range. Often used as if it were a `void` method.

```
protected void removeRange(int fromIndex, int toIndex)
```

Deletes all the elements with indices i such that $\text{fromIndex} \leq i < \text{toIndex}$. Elements with indices greater than or equal to `toIndex` are decreased appropriately.

```
public boolean remove(Object theElement)
```

Removes one occurrence of `theElement` from the calling `ArrayList`. If `theElement` is found in the `ArrayList`, then each element in the `ArrayList` with an index greater than the removed element's index is decreased to have an index that is one less than the value it had previously. Returns `true` if `theElement` was found (and removed). Returns `false` if `theElement` was not found in the calling `ArrayList`.

```
public void clear()
```

Removes all elements from the calling `ArrayList` and sets the `ArrayList`'s size to zero.

SEARCH METHODS

```
public boolean contains(Object target)
```

Returns `true` if the calling `ArrayList` contains `target`; otherwise, returns `false`. Uses the method `equals` of the object `target` to test for equality with any element in the calling `ArrayList`.

Display 14.1 Some Methods in the Class `ArrayList` (part 3 of 3)

```
public int indexOf(Object target)
```

Returns the index of the first element that is equal to `target`. Uses the method `equals` of the object `target` to test for equality. Returns `-1` if `target` is not found.

```
public int lastIndexOf(Object target)
```

Returns the index of the last element that is equal to `target`. Uses the method `equals` of the object `target` to test for equality. Returns `-1` if `target` is not found.

MEMORY MANAGEMENT (SIZE AND CAPACITY)

```
public boolean isEmpty()
```

Returns `true` if the calling `ArrayList` is empty (that is, has size 0); otherwise, returns `false`.

```
public int size()
```

Returns the number of elements in the calling `ArrayList`.

```
public void ensureCapacity(int newCapacity)
```

Increases the capacity of the calling `ArrayList`, if necessary, in order to ensure that the `ArrayList` can hold at least `newCapacity` elements. Using `ensureCapacity` can sometimes increase efficiency, but its use is not needed for any other reason.

```
public void trimToSize()
```

Trims the capacity of the calling `ArrayList` to the `ArrayList`'s current size. This method is used to save storage space.

MAKE A COPY

```
public Object[] toArray()
```

Returns an array containing all the elements on the list. Preserves the order of the elements.

```
public Type[] toArray(Type[] a)
```

Returns an array containing all the elements on the list. Preserves the order of the elements. *Type* can be any class types. If the list will fit in `a`, the elements are copied to `a` and `a` is returned. Any elements of `a` not needed for list elements are set to `null`. If the list will not fit in `a`, a new array is created.

(As we will discuss in Section 14.2, the correct Java syntax for this method heading is

```
public <Type> Type[] toArray(Type[] a)
```

However, at this point we have not yet explained this kind of type parameter syntax.)

```
public Object clone()
```

Returns a shallow copy of the calling `ArrayList`. *Warning:* The clone is not an independent copy. Subsequent changes to the clone may affect the calling object and vice versa. (See Chapter 5 for a discussion of shallow copy.)

EQUALITY

```
public boolean equals(Object other)
```

If `other` is another `ArrayList` (of any base type), then `equals` returns `true` if, and only if, both `ArrayList`s are of the same size and contain the same list of elements in the same order. (In fact, if `other` is any kind of *list*, then `equals` returns `true` if, and only if, both the calling `ArrayList` and `other` are of the same size and contain the same list of elements in the same order. Lists are discussed in Chapter 16.)

Why Are Some Parameters of Type *Base_Type* and Others of Type *Object*?

Look at the table of methods in Display 14.1. In some cases, when a parameter is naturally an object of the base type, the parameter type is the base type, but in other cases, it is the type *Object*.

For example, look at the `add` methods and the second `remove` method in the table. The `add` methods have a parameter of the base type; the `remove` method has a parameter of type *Object*. Why the difference in parameter types? The class `ArrayList` implements a number of interfaces and inherits methods from various ancestor classes. These interfaces and ancestor classes specify that certain parameters have type *Object*.

For example, in Chapter 7, we explained that the parameter for the `equals` method is always of type *Object* because the method heading is inherited from the class *Object*. In other cases, the designers of the `ArrayList` class were free to specify the parameter types for the method.

The “for-each” Loop

for-each loop

In Chapter 16, we will cover a family of classes known as *collections*. The `ArrayList` classes are our first examples of collection classes. Starting with version 5.0, Java has added a new kind of `for` loop that can cycle through all the elements in a collection and can, in particular, cycle through all the elements in an `ArrayList`. This new kind of `for` loop is called a **for-each loop** or **enhanced for loop**. A for-each loop can also be used to cycle through all the elements in an array. The for-each loop was introduced for use with arrays in a starred section of Chapter 6, but you need not go back and read that subsection. The presentation of for-each loops here is complete.

For example, the following code ends with a for-each loop that outputs all the elements in the `ArrayList` named `myList`:

```
ArrayList<String> myList = new ArrayList<String>(20);
<Some code to fill myList>
for (String element : myList)
    System.out.println(element);
```

You can read the line beginning with `for` as “for each `element` in `myList`, do the following.” Note that the variable, `element`, has the same type as the elements in the `ArrayList`. The variable (in this case, `element`) must be declared in the for-each loop as we have done. If you attempt to declare `element` before the for-each loop, you will get a compiler error message.

The general syntax for a for-each loop statement is

```
for (Base_Type Variable : Collection_Object)
    Statement
```

Be sure to notice that you use a colon (not a semicolon) after the *Variable*. You may use any legal variable name for the *Variable*; you do not have to use `element`. The only

Collection_Objects we have seen so far are arrays and `ArrayList` objects. Although it is not required, the *Statement* typically contains the *Variable*. When the for-each loop is executed, the *Statement* is executed once for each element of the *Collection_Object*. More specifically, for each element of the *Collection_Object*, the *Variable* is set to the collection element and then the *Statement* is executed.

The program in Display 14.2 includes an example of a for-each loop as well as examples of some of the other `ArrayList` details we have presented.

For-each Loop for ArrayList Objects

SYNTAX

```
for (Array_Base_Type Variable : ArrayList_Object) Statement
```

EXAMPLE

```
for (Integer element : numberList)
    System.out.println(element);
```

`numberList` is an `ArrayList` with base type `Integer`. This for-each loop outputs the value of each element in `numberList`.

A good way to read the first line of the example is "For each element in `numberList`, do the following."

Display 14.2 A for-each Loop Used with an `ArrayList` (part 1 of 2)

```
1  import java.util.ArrayList;
2  import java.util.Scanner;

3  public class ArrayListDemo
4  {
5      public static void main(String[] args)
6      {
7          ArrayList<String> toDoList = new ArrayList<String>(20);
8          System.out.println(
9              "Enter list entries, when prompted.");
10         boolean done = false;
11         String next = null;
12         String answer;
13         Scanner keyboard = new Scanner(System.in);

14         while (!done)
15         {
16             System.out.println("Input an entry:");
17             next = keyboard.nextLine();
18             toDoList.add(next);
```

(continued)

Display 14.2 A for-each Loop Used with an ArrayList (part 2 of 2)

```
19         System.out.print("More items for the list? ");
20         answer = keyboard.nextLine();
21         if (!answer.equalsIgnoreCase("yes"))
22             done = true;
23     }

24     System.out.println("The list contains:");
25     for (String entry : toDoList)
26         System.out.println(entry);
27 }
28 }
```

Sample Dialogue

```
Enter list entries, when prompted.
Input an entry:
Practice Dancing.
More items for the list? yes
Input an entry:
Buy tickets.
More items for the list? yes
Input an entry:
Pack clothes.
More items for the list? no
The list contains:
Practice Dancing.
Buy tickets.
Pack clothes.
```

Self-Test Exercises

8. Suppose `numberList` is an object of the class `ArrayList<Double>`. Give code that will output all the elements in `numberList` to the screen.
9. Write a class for sorting strings into lexicographic order that follows the outline of the class `SelectionSort` in Display 6.11 of Chapter 6. Your definition, however, will use an `ArrayList` of the class `ArrayList<String>`, rather than an array of elements of type `double`. For words, lexicographic order reduces to alphabetic order if all the words are in either all lowercase or all uppercase letters. You can compare two strings to see which is lexicographically first by using the `String` method `compareTo`. For strings `s1` and `s2`, `s1.compareTo(s2)` returns a negative number if `s1` is lexicographically before `s2`, returns 0 if `s1` equals `s2`, and returns a positive number if `s1` is lexicographically after `s2`. Call

Self-Test Exercises (continued)**extra code
on website**

your class `StringSelectionSort`. A test program you can use to test your class follows. (The program is included with the source code provided on the website that accompanies this book.)

```

import java.util.ArrayList;

public class StringSelectionSortDemo
{
    public static void main(String[] args)
    {
        ArrayList<String> b = new ArrayList<String>();
        b.add("time");
        b.add("tide");
        b.add("clouds");
        b.add("rain");
        System.out.println("ArrayList values before sorting:");
        for (String e : b)
            System.out.print(e + " ");
        System.out.println();
        StringSelectionSort.sort(b);
        System.out.println("ArrayList values after sorting:");
        for (String e : b)
            System.out.print(e + " ");
        System.out.println();
    }
}

```

EXAMPLE: Golf Scores

The program in Display 14.3 reads in a list of golf scores and then outputs the average of the scores and how much each differs from the average. The scores are read and stored in an `ArrayList` so that they will be available later in the program to be output along with how much each differs from the average. This is the kind of thing that is well suited to being done with an ordinary array. However, it is much easier and cleaner to use an `ArrayList` as we did in Display 14.3.

Our program deals with a list of values of type `double`. But we use an `ArrayList` with base type `Double` to store these values. We did not use an `ArrayList` with base type `double` because there is no such thing. The base type for an `ArrayList` must be a class type (or other reference type). However, thanks to Java's automatic boxing, we can program as if an object of type `ArrayList<Double>` can store values of type `double`.

The `ArrayList` automatically keeps track of how many elements are stored in the `ArrayList`. If we had used an ordinary partially filled array in our program instead of an `ArrayList`, we would need an extra `int` variable to keep track of how much of the array is used. When we use an `ArrayList`, we are spared all the overhead associated

(continued)

EXAMPLE: (continued)

with partially filled arrays. Those details are taken care of for us automatically. The code for those details is in the definition of the `ArrayList` class, but there is no need to look at that code. That code is all implementation detail that we need not worry about when using an `ArrayList`.

Notice the use of for-each loops in our program. The cleanest and easiest way to cycle through all the elements in an `ArrayList` is to use a for-each loop.

It is instructive to compare the program in Display 14.3, which uses an `ArrayList`, with the program in Display 6.4, which does the same thing but uses an ordinary array. The version that uses an `ArrayList` is much cleaner and even much shorter than the one that uses an ordinary array. This is because an `ArrayList` does so many things for you automatically that you would have to explicitly code for if you used an ordinary array. This is a good example of information hiding and code reuse. The programmers who defined the `ArrayList` class did a lot of programming for you so that your programming task is simpler than it would otherwise be.

Display 14.3 Golf Score Program (part 1 of 3)

```

1  import java.util.ArrayList;
2  import java.util.Scanner;

3  public class GolfScores
4  {
5      /**
6       * Shows differences between each of a list of golf scores and their
7       * average.
8       */
9      public static void main(String[] args)
10     {
11         ArrayList<Double> score = new ArrayList<Double> ();
12
13         System.out.println("This program reads golf scores and shows");
14         System.out.println("how much each differs from the average.");
15
16         System.out.println("Enter golf scores:");
17         fillArrayList(score);
18         showDifference(score);
19     }
20
21     /**
22      * Reads values into the array a.
23      */
24     public static void fillArrayList(ArrayList<Double> a)

```

Parameters of type `ArrayList<Double>()` are handled just like any other class parameter.

Display 14.3 Golf Score Program (part 2 of 3)

```

21     {
22         System.out.println("Enter a list of nonnegative numbers.");
23         System.out.println(
24             "Mark the end of the list with a negative number.");
25         Scanner keyboard = new Scanner(System.in);
26
27         double next;
28         next = keyboard.nextDouble();
29         while (next >= 0)
30         {
31             a.add(next);
32             next = keyboard.nextDouble();
33         }
34     }
35     /**
36     Returns the average of numbers in a.
37     */
38     public static double computeAverage(ArrayList<Double> a)
39     {
40         double total = 0;
41         for (Double element : a)
42             total = total + element;
43         int numberOfScores = a.size();
44         if (numberOfScores > 0)
45         {
46             return (total/numberOfScores);
47         }
48         else
49         {
50             System.out.println("ERROR: Trying to average 0 numbers.");
51             System.out.println("computeAverage returns 0.");
52             return 0;
53         }
54     }
55     /**
56     Gives screen output showing how much each of the elements
57     in a differ from their average.
58     */
59     public static void showDifference(ArrayList<Double> a)
60     {
61         double average = computeAverage(a);
62         System.out.println("Average of the " + a.size()
63             + " scores = " + average);
64         System.out.println("The scores are:");
65         for (Double element : a)
66             System.out.println(element + " differs from average by "
67                 + (element - average));
68     }
69 }

```

Because of automatic boxing, we can treat values of type double as if their type were Double.

A for-each loop is the nicest way to cycle through all the elements in an ArrayList.

(continued)

Display 14.3 Golf Score Program (part 3 of 3)

Sample Dialogue

```
This program reads golf scores and shows
how much each differs from the average.
Enter golf scores:
Enter a list of nonnegative numbers.
Mark the end of the list with a negative number.
69 74 68 -1
Average of the 3 scores = 70.3333
The scores are:
69.0 differs from average by -1.33333
74.0 differs from average by 3.66667
68.0 differs from average by -2.33333
```

**TIP: Use `trimToSize` to Save Memory**`trimToSize`

`ArrayLists` automatically increase their capacity when your program needs them to have additional capacity. However, the capacity may increase beyond what your program requires. Also, when your program needs less capacity in an `ArrayList`, the `ArrayList` does not automatically shrink. If your `ArrayList` has a large amount of excess capacity, you can save memory by using the method `trimToSize` to shrink the capacity of an `ArrayList`. If `list` is an `ArrayList`, an invocation of `list.trimToSize()` will shrink the capacity of the `ArrayList list` down to the size of `list`, so that there is no unused capacity in `list`. Normally, you should use `trimToSize` only when you know that the `ArrayList` will not need its extra capacity later. ■

**PITFALL: The `clone` Method Makes a Shallow Copy ★**

There are situations in which you would like to make an independent copy of an `ArrayList` object; that is, you would like to make a deep copy of the `ArrayList` object. (Deep copying and shallow copying were discussed in Chapter 5; you may want to review that material.) For example, if you define a class with a private instance variable of an `ArrayList` type, then you would like an accessor method to return a deep copy of the `ArrayList` stored in the private instance variable. The reason you want a deep copy is the same as the reason that you want a deep copy of an array instance variables. This was discussed in Chapter 6 in the subsection entitled “Privacy Leaks with Array Instance Variables.” It would be a good idea to review that subsection before going on with reading this subsection.

As we have often observed, the assignment operator merely copies a reference so that you have another name for the object being copied. So, you do not have an independent



PITFALL: (continued)

copy. You have what is known as a *shallow copy*. For example, assume that `Pet` is a class with the usual kinds of accessor methods and consider the following code:

```
ArrayList<Pet> petList1 = new ArrayList<Pet>();
<Some code to set the instance variables of elements of petList1.>
ArrayList<Pet> petList2 = petList1;
```

`petList2` and `petList1` are just two names for the same `ArrayList` object. Making a change to `petList1` or to an element of `petList1` will also change `petList2` because they are the same list.

If you want an independent copy (deep copy) of `petList1`, you might think the following would give you your independent copy:

```
ArrayList<Pet> petList2 = petList1.clone();
```

Unfortunately, the `clone` method also makes a shallow copy. There is no built-in method to give you a deep copy (independent copy) of an `ArrayList`.

When you need a deep copy of an `ArrayList`, you will have to resort to some ad hoc tricks. If you have a way to make a deep copy of objects of the base type, then you can create a deep copy of each element in the `ArrayList` and place them into a new `ArrayList` object. This is the exact same approach as the one we discussed for making a deep copy of an ordinary array in the subsection of Chapter 6 entitled “Privacy Leaks with Array Instance Variables.” The situation with respect to deep copying of an `ArrayList` is exactly the same as the situation with respect to deep copying of an ordinary array. Although the details of this subsection may seem subtle and difficult, they are not new. You have already faced the exact same problem with ordinary arrays. ■

Self-Test Exercises

10. Can you have an `ArrayList` of `ints`?
11. The following for-each loop was used in the method `showDifference` in Display 14.3. Rewrite it as an ordinary `for` loop. This should help you to see how much cleaner it is to use a for-each loop.

```
for (Double element : a)
    System.out.println(element + " differs from average by "
                        + (element - average));
```

The Vector Class

Vector

The Java standard libraries have a class named `Vector` that behaves almost exactly the same as the class `ArrayList`. In fact, everything we have said about the class `ArrayList` holds true for the `Vector` class. Although in almost all situations, you could use either the class `ArrayList` or the class `Vector`, a clear preference seems to be

developing among programmers for the class `ArrayList`. There are some differences between the classes `Vector` and `ArrayList`, but the differences involve material we have not covered. If you encounter the class `Vector` in somebody's code, chances are the class `Vector` could be replaced by the class `ArrayList`, which would require at most cosmetic changes in the code.¹

Parameterized Classes and Generics

parameterized
class

generics

The class `ArrayList` is a **parameterized class**. It has a parameter, which we have been denoting *Base_Type*, that can be replaced by any reference type to obtain a class for `ArrayList`s with the specified base type. `ArrayList` is just a class that somebody defined (and placed in the standard Java library package `java.util`), so you should also be able to define these kinds of classes. Starting with version 5.0, Java allows class definitions with parameters for types. These classes that have type parameters are called parameterized classes or **generic definitions** or, more simply, **generics**. You already know how to use classes with a type parameter, because we have been using the parameterized class `ArrayList`. In Section 14.2, we will show you how to write your own parameterized classes.



PITFALL: Nonparameterized `ArrayList` and `Vector` Classes

The `ArrayList` and `Vector` classes we discussed in this section have a type parameter for the base type. There are also `ArrayList` and `Vector` classes with no parameter for the base type. (They have base type `Object`.) These `ArrayList` and `Vector` classes without type parameters are left over from earlier versions of Java. When checking details in the Java documentation, be sure you get the documentation for the `ArrayList` and `Vector` classes that have a type parameter. Using notation we introduce in Section 14.2, the versions with type parameters are usually written as `ArrayList<E>` and `Vector<E>` or as `ArrayList<T>` and `Vector<T>` in the Java documentation. ■

14.2 Generics

You can have this dish prepared with any type of meat or fish.

Entry on a restaurant menu

Starting with version 5.0, Java allows class definitions that contain a parameter (or parameters) for a type (or types). In this section, we teach you how to write class definitions that contain a type parameter.

¹The biggest difference between the `Vector` and `ArrayList` classes is that `Vectors` are *synchronized* whereas `ArrayLists` are not. However, synchronization is a topic that we do not cover and is not relevant to the kinds of programming we are doing.

Generic Basics

type parameter

Classes and methods can have a type parameter. The type parameter may then have any reference type, and hence any class type, plugged in for the type parameter. This plugging in produces a specific class type or method. For example, Display 14.4 shows a very simple class definition with a **type parameter** `T`. You may use any nonkeyword identifier for the type parameter; you need not use `T`. However, by convention, type parameters start with an uppercase letter, and there is some tradition of using a single letter for a type parameter. Starting with an uppercase letter makes sense because typically a class type is plugged in for the type parameter. The tradition of using a single letter is not so compelling.

A class definition with a type parameter is stored in a file and compiled just like any other class. For example, the parameterized class shown in Display 14.4 would be stored in a file named `Sample.java`. Once the parameterized class is compiled, it can be used like any other class, except that when used in your code, you must specify a class type to be plugged in for the type parameter. For example, the class `Sample` from Display 14.4 could be used as follows:

```
Sample<String> object1 = new Sample<String>();
object1.setData("Hello");
System.out.println(object1.getData());
Sample<Pet> object2 = new Sample<Pet>();
Pet p = new Pet();
<Some code to set the data for the object p>
object2.setData(p);
```

The class `Pet` can be as defined in Chapter 4, but the details do not matter; it could be any class.

instantiate

A class, such as `Sample<String>`, that you obtain from a generic class by plugging in a type for the type parameter is said to **instantiate** the generic class. So, we would say “`Sample<String>` instantiates the generic class `Sample`.”

Notice the angular bracket notation for the type parameter and also for the class type that is plugged in for the type parameter.

Display 14.4 A Class Definition with a Type Parameter

```
1 public class Sample<T>
2 {
3     private T data;

4     public void setData(T newData)
5     {
6         data = newData;
7     }

8     public T getData()
9     {
10         return data;
11     }
12 }
```

T is a parameter for a type.

Class Definition with a Type Parameter

You can define classes with a parameter for a type. Such a class is called a **generic class** or a **parameterized class**. The type parameter is included in angular brackets after the class name in the class definition heading. You may use any nonkeyword identifier for the type parameter, but by convention, the type parameter starts with an uppercase letter. The type parameter may be used like any other type in the definition of the class. (There are some restrictions on where the type parameter can be used. These are discussed later in the Pitfall section entitled “A Type Parameter Cannot Be Used Everywhere a Type Name Can Be Used.”) For an example, see Display 14.4.

A generic class is used like any other class, except that you specify a reference type, typically a class type, to be plugged in for the type parameter. This class type (or other reference type) is given in angular brackets after the name of the generic class, as shown in the following example:

EXAMPLE

```
Sample<String> object1 = new Sample<String>();  
object1.setData("Hello");
```

Sample<String> is said to **instantiate** the generic class Sample.

Type Inference in Java 7

Starting with version 7, Java supports a feature called **type inference**. In type inference, Java is able to infer the base type in the call to the constructor based on the base type used in the variable declaration. That is, the following

```
ClassName<Base_Type> Object_Name = new ClassName<Base_Type>();
```

can equivalently be written in Java 7 as

```
ClassName<Base_Type> Object_Name = new ClassName<>();
```

The new format saves a little bit of typing and is also somewhat cleaner to read. However, programmers have been using the earlier format for many years, so you are likely to see it in existing code. For greater compatibility, most of the examples in this book do not use the new syntax supported in JDK 7.

EXAMPLES

```
ArrayList<String> list = new ArrayList<>();  
ArrayList<Double> list2 = new ArrayList<>(30);
```



TIP: Compile with the `-Xlint` Option

There are many pitfalls that you can encounter when using type parameters. If you compile with the `-Xlint` option, you will receive more informative diagnostics of any problems or potential problems in your code. For example, the class `Sample` in Display 14.4 should be compiled as follows:

```
javac -Xlint Sample.java
```

If you are using an IDE to compile your programs, check your documentation to see how to set compiler options. (For the TextPad environment, you can set compiler options in the `Preferences` box under the `Configure` menu.)

When compiling with the `-Xlint` option, you will get more warnings than you would otherwise get. A warning is not an error, and if the compiler gives only warnings and no error message, then the class has compiled and can be used. However, in most cases, be sure you understand the warning and feel confident that it does not indicate a problem, or else change your code to eliminate the warning. One warning that you may get on some programs in this text is “no definition of `serialVersionUID`.” Discussion of this warning is beyond the scope of this book, but you can safely ignore the warning. ■

EXAMPLE: A Generic Class for Ordered Pairs

In Display 14.5, we have given a parameterized class for ordered pairs of values.

Notice that the constructor heading does not include the type parameter `T`. This is counter to many people’s intuition, but that is the way it is done. A constructor can use the type parameter, such as `T`, as the type for a parameter for the constructor, but the constructor heading does not include the type parameter in angular brackets, such as `<T>`.

By using this parameterized class with the type `String` plugged in for the type parameter `T`, as shown next, you get a class whose objects are pairs of `String` values:

```
Pair<String> secretPair =
    new Pair<String>("Happy", "Day");
```

By using this parameterized class with the type `Integer` plugged in for the type parameter `T`, as shown next, you get a class whose objects are pairs of `Integer` objects:

```
Pair<Integer> rollofDice =
    new Pair<Integer>(new Integer(2), new Integer(3));
```

If `Pet` is some class you defined, you can plug in `Pet` for the type parameter `T`, as shown next, to get a class whose objects are pairs of objects of type `Pet`:

```
Pet male = new Pet();
Pet female = new Pet();
<Some code to set the data for the objects male and female.>
Pair<Pet> breedingPair =
    new Pair<Pet>(male, female);
```

Display 14.6 contains a simple example of using our generic class `Pair`.

Display 14.5 A Generic Ordered Pair Class (part 1 of 2)

```
1  public class Pair<T>
2  {
3      private T first;
4      private T second;
5
6      public Pair()
7      {
8          first = null;
9          second = null;
10     }
11
12     public Pair(T firstItem, T secondItem)
13     {
14         first = firstItem;
15         second = secondItem;
16     }
17
18     public void setFirst(T newFirst)
19     {
20         first = newFirst;
21     }
22
23     public void setSecond(T newSecond)
24     {
25         second = newSecond;
26     }
27
28     public T getFirst()
29     {
30         return first;
31     }
32
33     public T getSecond()
34     {
35         return second;
36     }
37
38     public String toString()
39     {
40         return ( "first: " + first.toString() + "\n"
41                + "second: " + second.toString() );
42     }
43 }
```

Constructor headings do not include the type parameter in angular brackets.

Display 14.5 A Generic Ordered Pair Class (part 2 of 2)

```
37     public boolean equals(Object otherObject)
38     {
39         if (otherObject == null)
40             return false;
41         else if (getClass() != otherObject.getClass())
42             return false;
43         else
44         {
45             Pair<T> otherPair = (Pair<T>)otherObject;
46             return (first.equals(otherPair.first)
47                 && second.equals(otherPair.second));
48         }
49     }
50 }
```

Terminology

The terms *generic class* and *parameterized class* mean the same thing, namely a class with one or more type parameters.

Display 14.6 Using Our Ordered Pair Class (part 1 of 2)

```
1  import java.util.Scanner;

2  public class GenericPairDemo
3  {
4      public static void main(String[] args)
5      {
6          Pair<String> secretPair =
7              new Pair<String>("Happy", "Day");
8
9          Scanner keyboard = new Scanner(System.in);
10         System.out.println("Enter two words:");
11         String word1 = keyboard.next();
12         String word2 = keyboard.next();
13         Pair<String> inputPair =
14             new Pair<String>(word1, word2);
15         if (inputPair.equals(secretPair))
16         {
17             System.out.println("You guessed the secret words");
18             System.out.println("in the correct order!");
19         }
20     }
21 }
```

(continued)

Display 14.6 Using Our Ordered Pair Class (part 2 of 2)

```
19         }
20         else
21         {
22             System.out.println("You guessed incorrectly.");
23             System.out.println("You guessed");
24             System.out.println(inputPair);
25             System.out.println("The secret words are");
26             System.out.println(secretPair);
27         }
28     }
29 }
```

Sample Dialogue

```
Enter two words:
two words
You guessed incorrectly.
You guessed
first: two
second: words
The secret words are
first: Happy
second: Day
```



PITFALL: A Generic Constructor Name Has No Type Parameter

The class name in a parameterized class definition has a type parameter attached, such as `Pair<T>` in Display 14.5. This can mislead you into thinking you need to use the type parameter in the heading of the constructor definition, but you do not repeat the type parameter specification `<T>` in the heading of the constructor definition. For example, use

```
public Pair()
```

Do not use

```
public Pair<T>()
```

A constructor can use the type parameter, such as `T`, as the type for a parameter for the constructor, as in the following, but the constructor heading does not include the type parameter in angular brackets, such as `<T>`:

```
public Pair(T firstItem, T secondItem)
```

For a complete example, see Display 14.5.

Sometimes it seems that people stay up late at night thinking of ways to make things confusing. As we just noted, in the definition of a parameterized class, a

**PITFALL: (continued)**

constructor has no type parameter in angular brackets. So, you see the following in Display 14.5:

```
public Pair(T firstItem, T secondItem)
```

But as shown in Display 14.6, when you instantiate a generic class by specifying a type for the type parameter, you do specify the type in angular brackets when writing the constructor name, as in the following from Display 14.6:

```
Pair<String> secretPair =  
    new Pair<String>("Happy", "Day");
```

However, this second case is not hard to remember. If you leave out the `<String>`, Java would not know which `Pair` class you meant. If you leave out the `<String>`, the compiler could not tell if you meant `Pair<String>`, `Pair<Double>`, or some other `Pair` class. ■

**PITFALL: You Cannot Plug in a Primitive Type for a Type Parameter**

The type plugged in for a type parameter must be a reference type. It cannot be a primitive type such as `int`, `double`, or `char`. However, now that Java has automatic boxing, this is not a big restriction in practice. For example, if you want `Pair<int>`, you cannot have it, but you can have `Pair<Integer>`, and thanks to automatic boxing, you can use `Pair<Integer>` with `int` values. This is illustrated by the program in Display 14.7.

The most typical type to plug in for a type parameter is a class type. However, you can plug in any reference type. So, in particular, you can plug in an array type for a type parameter. ■

**PITFALL: A Type Parameter Cannot Be Used Everywhere a Type Name Can Be Used**

Within the definition of a parameterized class definition, there are places where an ordinary class name would be allowed but a type parameter is not allowed. In particular, you cannot use the type parameter in simple expressions using `new` to create a new object. For example, the following are all illegal within the definition of a parameterized class definition with type parameter `T`:

```
T object = new T(); //The first T is legal,  
                //the second one is illegal.  
T[] a = new T[10]; //The first T is legal,  
                //the second one is illegal.
```

This restriction is not as arbitrary as it might at first appear. In the first case, `T` is not being used as a type name; it is being used as a constructor name. In the second case, `T` is being used as something like a constructor, although it is not officially a constructor. ■



PITFALL: An Instantiation of a Generic Class Cannot be an Array Base Type

Arrays such as the following are illegal (the generic class `Pair` is the one defined in Display 14.5):

```
Pair<String>[] a = new Pair<String>[10]; //Illegal
```

This is a reasonable thing to want to do, but it is not allowed because of technical details having to do with how Java implements generic classes. The full explanation for this restriction is beyond the scope of this book. ■

Display 14.7 Using Our Ordered Pair Class and Automatic Boxing (part 1 of 2)

```
1  import java.util.Scanner;

2  public class GenericPairDemo2
3  {
4      public static void main(String[] args)
5      {
6          Pair<Integer> secretPair =
7              new Pair<Integer>(42, 24);
8
9          Scanner keyboard = new Scanner(System.in);
10         System.out.println("Enter two numbers:");
11         int n1 = keyboard.nextInt();
12         int n2 = keyboard.nextInt();
13         Pair<Integer> inputPair =
14             new Pair<Integer>(n1, n2);
15         if (inputPair.equals(secretPair))
16         {
17             System.out.println("You guessed the secret numbers");
18             System.out.println("in the correct order!");
19         }
20         else
21         {
22             System.out.println("You guessed incorrectly.");
23             System.out.println("You guessed");
24             System.out.println(inputPair);
25             System.out.println("The secret numbers are");
26             System.out.println(secretPair);
27         }
28     }
29 }
```

Automatic boxing allows you to use an int argument for an Integer parameter.

Display 14.7 Using Our Ordered Pair Class and Automatic Boxing (part 2 of 2)

Sample Dialogue

```
Enter two numbers:
42 24
You guessed the secret numbers
in the correct order!
```

**TIP: A Class Definition Can Have More Than One Type Parameter**

A generic class definition can have any number of type parameters. The multiple type parameters are listed in angular brackets just as in the single type parameter case, but are separated by commas. For example, in Display 14.8, we have rewritten the class `Pair` so the first and second items of a pair can be of different types. In Display 14.9, we give a simple example of using our generic class with two type parameters. ■

Display 14.8 Multiple Type Parameters (part 1 of 2)

```
1  public class TwoTypePair<T1, T2>
2  {
3      private T1 first;
4      private T2 second;
5
6      public TwoTypePair()
7      {
8          first = null;
9          second = null;
10
11      }
12
13      public TwoTypePair(T1 firstItem, T2 secondItem)
14      {
15          first = firstItem;
16          second = secondItem;
17      }
18
19      public void setFirst(T1 newFirst)
20      {
21          first = newFirst;
22      }
23      public void setSecond(T2 newSecond)
24      {
25          second = newSecond;
26      }
27      public T1 getFirst()
28      {
29          return first;
30      }
31      public T2 getSecond()
32      {
33          return second;
34      }
35  }
```

(continued)

Display 14.8 Multiple Type Parameters (part 2 of 2)

```

24    {
25        return first;
26    }

27    public T2 getSecond()
28    {
29        return second;
30    }

31    public String toString()
32    {
33        return ( "first: " + first.toString() + "\n"
34                + "second: " + second.toString() );
35    }
36
37    public boolean equals(Object otherObject)
38    {
39        if (otherObject == null)
40            return false;
41        else if (getClass() != otherObject.getClass())
42            return false;
43        else
44        {
45            TwoTypePair<T1, T2> otherPair =
46                (TwoTypePair<T1, T2>)otherObject;
47            return (first.equals(otherPair.first)
48                    && second.equals(otherPair.second));
49        }
50    }
51 }

```

*The first equals is the equals of the type T1.
The second equals is the equals of the type T2.*



PITFALL: A Generic Class Cannot Be an Exception Class

If you begin an exception class definition as follows, you will get a compiler error message:

```
public class MyException<T> extends Exception //Illegal
```

It is still illegal if you replace `Exception` with `Error`, `Throwable`, or any descendent class of `Throwable`. You cannot create a generic class whose objects are throwable. ■

Display 14.9 Using a Generic Class with Two Type Parameters

```
1  import java.util.Scanner;

2  public class TwoTypePairDemo
3  {
4      public static void main(String[] args)
5      {
6          TwoTypePair<String, Integer> rating =
7              new TwoTypePair<String, Integer>("The Car Guys", 8);

8          Scanner keyboard = new Scanner(System.in);
9          System.out.println(
10              "Our current rating for " + rating.getFirst());
11          System.out.println(" is " + rating.getSecond());

12          System.out.println("How would you rate them?");
13          int score = keyboard.nextInt();
14          rating.setSecond(score);
15          System.out.println(
16              "Our new rating for " + rating.getFirst());
17          System.out.println(" is " + rating.getSecond());
18      }
19  }
```

Sample Dialogue

```
Our current rating for The Car Guys
is 8
How would you rate them?
10
Our new rating for The Car Guys
is 10
```

Bounds for Type Parameters

Sometimes it does not make sense to plug in just any reference type for the type parameter in a generic class definition. For example, consider the generic class `Pair` defined in Display 14.5. Suppose we want to add a method that returns the maximum of the two elements in an ordered pair. We could add the following method definition to the class `Pair` in Display 14.5:

```
max      public T max()
         {
           if (first.compareTo(second) <= 0)
             return first;
           else
             return second;
         }
```

compareTo

Recall that the method `compareTo` is required to be a member of any class that implements the `Comparable` interface. The `Comparable` interface is a standard Java interface that was discussed in Chapter 13. Recall that the `Comparable` interface has only the following method heading that must be implemented:

```
public int compareTo(Object other);
```

When defining a class that implements the `Comparable` interface, the programmer is expected to define `compareTo` so that it returns

- a negative number if the calling object “comes before” the parameter `other`,
- a zero if the calling object “equals” the parameter `other`,
- and a positive number if the calling object “comes after” the parameter `other`.

This all works fine, except for one problem: This makes sense only if the type plugged in for the type parameter `T` satisfies the `Comparable` interface, but Java allows you to plug in any type for the type parameter `T`.

You can have Java enforce this restriction on the possible types that can be plugged in for `T`. To ensure that only classes that implement the `Comparable` interface are plugged in for `T`, begin the class definition as follows:

```
public class Pair<T extends Comparable>
```

**extends
bound**

The part `extends Comparable` is called a **bound** on the type parameter `T`. If you attempt to plug in a type for `T` that does not implement the `Comparable` interface, you will get a compiler error message. Note that you use the keyword `extends`, not the keyword `implements` as you would naturally expect.

Note that the bound `extends Comparable` is not just an optional little nicety. If you omit it, you will get an error message from the compiler saying it does not know about the method `compareTo`.

This version of the generic class `Pair` with the method `max` is summarized in Display 14.10. On the accompanying website, this version of `Pair` is in a subdirectory named `Bounded Pair`.

A bound on a type may be a class name (rather than an interface name) in which case only descendent classes of the bounding class may be plugged in for the type parameters. For example, the following says that only descendent classes of the class `Employee` may be plugged in for `T`, where `Employee` is some class:

```
public class SameGenericClass<T extends Employee>
```

Display 14.10 A Bounded Type Parameter

```
1 public class Pair<T extends Comparable>
2 {
3     private T first;
4     private T second;
5
6     public T max()
7     {
8         if (first.compareTo(second) <= 0)
9             return first;
10        else
11            return second;
12    }
13 }
```

<All the constructors and methods given in Display 14.5
are also included as part of this generic class definition.>

```
12 }
```

This explains why the keyword `extends` is used in the bounds expression rather than `implements`.

multiple bounds

You can have multiple interfaces and possibly one class in a bounds expression. Just separate the entries with the ampersand sign, `&`, as in the following example:

```
public class AnotherGenericClass<T extends Employee & Comparable>
```

If you have more than one type parameter, the syntax follows the following example:

```
public class YetAnotherGeneric
    <T1 extends Employee & Comparable, T2 extends Comparable>
```

You can list any number of interfaces in a bounds expression, but you may list only one class in a bounds expression. Moreover, if you do list a class and some interfaces, the class must be first in the list.

Type Parameter Bounds

You can specify that the class plugged in for a type parameter must be a descendent class of a specified class, must implement specified interfaces, or both.

(continued)

SYNTAX (FOR CLASS DEFINITION HEADINGS)

```
public class Class_Name
    <Type extends Ancestor_Class & Interface_1 & Interface_2 & ...
                                     ...& Last_Interface>
```

If there are multiple type parameters, they are separated by commas. There can be any number of interfaces but only one ancestor class for each type parameter.

EXAMPLES

```
public class Pair<T extends Comparable>
public class MyClass<T extends Employee & Comparable>
public class YourClass <T1 extends Employee & Comparable
                        & Cloneable, T2 extends Comparable>
```

Employee is a class. Comparable and Cloneable are interfaces.

**TIP: Generic Interfaces**

An interface can have one or more type parameters. The details and notation are the same as they are for classes with type parameters. ■

Generic Methods ★

(This is a starred subsection because it is needed only for Chapter 16, which covers the Java collection classes. If you choose to read Chapter 16, you will need to read this subsection first.)

When you define a generic class, you can use the type parameter in the definitions of the methods for that generic class. You also can define a generic method that has its own type parameter that is not the type parameter of any class. This generic method can be a member of an ordinary (nongeneric) class or a member of some generic class with some other type parameter. For example,

```
public class Utility
{
    ...
    public static <T> T getMidpoint(T[] a)
    {
        return a[a.length/2];
    }
    public static <T> T getFirst(T[] a)
    {
        return a[0];
    }
    ...
}
```

In this case, the class (`Utility`) has no type parameters, but the methods `getMidpoint` and `getFirst` each have a type parameter. Note that the type parameter in angular brackets, `<T>`, is placed after all the modifiers—in this case, `public static`—and before the returned type.

When you invoke one of these generic methods, preface the method name with the type to be plugged in, given in angular brackets, as in the following examples:

```
String midString = Utility.<String>getMidpoint(b);  
  
double firstNumber = Utility.<Double>getFirst(c);
```

Note that the dot is before the type in angular brackets; the type is part of the method name, not part of the class name. Also note that the methods `getMidpoint` and `getFirst` use different types plugged in for their type parameter. The type parameter is local to the method, not to the class. (The argument `b` is an array with base type `String`. The argument `c` is an array with base type `Double`.)

You can also define such generic methods inside of generic classes, as in the following example:

```
public class Sample<T>  
{  
    private T data;  
  
    public Sample(T forData)  
    {  
        data = forData;  
    }  
    public <ViewerType> void showTo(ViewerType viewer)  
    {  
        System.out.println("Hello " + viewer);  
        System.out.println("Data is " + data);  
    }  
    ...  
}
```

Note that `T` and `ViewerType` are different type parameters. `T` is a type parameter for the entire class, but `ViewerType` is a type parameter only for the method `showTo`. What follows is a sample use of these generic methods:

```
Sample<Integer> object = new Sample<Integer>(42);  
object.<String>showTo("Friend");
```

This produces the output

```
Hello Friend  
Data is 42
```

Inheritance with Generic Classes ★

You can define a generic class to be a derived class of an ordinary class or a derived class of another generic class. Display 14.11 contains the definition of a generic class called `UnorderedPair`, which is a derived class of the generic class `Pair` (which we gave in Display 14.5). The class `UnorderedPair` overrides the definition of `equals` that it inherits from `Pair`. To a programmer using the class, `UnorderedPair` is just like the class `Pair` with one exception. In `UnorderedPair`, the two components do not have to be in the same order for two pairs to be equal. Less formally, in the `Pair<String>` world, "beer" and "peanuts" is not the same as "peanuts" and "beer". In the `UnorderedPair<String>` world, they are the same. This is illustrated by the demonstration program in Display 14.12.

Just as you would expect, an object of type `UnorderedPair<String>` is also of type `Pair<String>`. As we have seen so far, inheritance with generic classes is straightforward in most cases. However, there are some situations with subtle pitfalls. We discuss those next.

Display 14.11 A Derived Generic Class (part 1 of 2)

```
1  public class UnorderedPair<T> extends Pair<T>
2  {
3      public UnorderedPair()
4      {
5          setFirst(null);
6          setSecond(null);
7      }

8      public UnorderedPair(T firstItem, T secondItem)
9      {
10         setFirst(firstItem);
11         setSecond(secondItem);
12     }

13     public boolean equals(Object otherObject)
14     {
15         if (otherObject == null)
16             return false;
17         else if (getClass() != otherObject.getClass())
18             return false;
19         else
20         {
21             UnorderedPair<T> otherPair =
22                 (UnorderedPair<T>)otherObject;
23             return (getFirst().equals(otherPair.getFirst())
24                 && getSecond().equals(otherPair.getSecond()))
25                 ||
```

Display 14.11 A Derived Generic Class (part 2 of 2)

```

26             (getFirst().equals(otherPair.getSecond())
27             && getSecond().equals(otherPair.getFirst()));
28         }
29     }
30 }

```

Suppose `HourlyEmployee` is a derived class of the class `Employee`. You might think that an object of type `Pair<HourlyEmployee>` is also of type `Pair<Employee>`. You might think that, but you would be wrong. If `G` is a generic class, there is no relationship between `G<A>` and `G<B>`, no matter what the relationship is between the classes `A` and `B`.

Display 14.12 Using `UnorderedPair`

```

1  public class UnorderedPairDemo
2  {
3      public static void main(String[] args)
4      {
5          UnorderedPair<String> p1 =
6              new UnorderedPair<String>("peanuts", "beer");
7          UnorderedPair<String> p2 =
8              new UnorderedPair<String>("beer", "peanuts");
9          if (p1.equals(p2))
10         {
11             System.out.println(p1.getFirst() + " and " +
12                               p1.getSecond() + " is the same as");
13             System.out.println(p2.getFirst() + " and "
14                               + p2.getSecond());
15         }
16     }
17 }

```

Sample Dialogue²

```

peanuts and beer is the same as
beer and peanuts

```

²A note to the grammar police: I intentionally used “is” instead of “are.” If you read and understand the text, you will realize that “peanuts and beer” is a single item. Starting the sentence with a lowercase letter and the absence of a period are also intentional.

Self-Test Exercises

12. (This question refers to the starred section “Generic Methods.” You should skip this question if you have not yet read that subsection.) Define a generic method named `getMidindex`, which is like `getMidpoint`, but returns the index of the array midpoint as opposed to the element at the midpoint.
13. (This question refers to the starred section “Inheritance with Generic Classes.” You should skip this question if you have not yet read that subsection.) Is an array of type `UnorderedPair<String>[]` also of type `Pair<String>[]`?

Chapter Summary

- `ArrayList` is a *parameterized class* that is like an array that can grow (and shrink) while the program is running.
- An `ArrayList` has a number of methods that allow you to use it as a kind of automated partially filled array.
- You can cycle through all the elements in an `ArrayList` using a *for-each loop*.
- You can define classes with one or more type parameters. Such classes are known as *generic classes*.

Answers to Self-Test Exercises

1. `list.add("Hello");`
2. `instruction.set(5, "Go");`
3. `instruction.add(5, "Go");`
4. No. The index for `set` must be greater than or equal to 0 and less than the size of the `ArrayList`. Thus, you can replace any existing element, but you cannot place the element at any higher index. This situation is unlike that of an array. If an array is partially filled to index 10, you can add an element at index 20, as long as the array is that large. With an `ArrayList`, you cannot add an element beyond the last-used index.
5. No. The index for `add` must be greater than or equal to 0 and less than or equal to the size of the `ArrayList`. Thus, you can replace any existing element or add an element to the end of the list, but you cannot place the element at any higher index. This situation is unlike that of an array. If an array is partially filled to index 10, you can add an element at index 20, as long as the array is that large. With an `ArrayList`, you cannot add an element beyond one more than the last-used index.

6. The index for add (that is, index2) is allowed to be one larger than the index for set (that is, index1). The index for set must be strictly less than the size of the ArrayList. The index for add can also be equal to the size of the ArrayList.
7. Yes. The ArrayList can contain more than 20 elements. The number 20 used as an argument to the constructor merely gives the initial memory allocation for the ArrayList. More memory is automatically allocated when it is needed.

```

8. for (Double element : numberList)
    System.out.println(element);

9. import java.util.ArrayList;
   /**
    * Class for sorting an ArrayList of Strings lexicographically
    * (approximately alphabetically).
    */
   public class StringSelectionSort
   {
       /**
        * Sorts the ArrayList a so that a.get(0), a.get(1), ...,
        * a.get(a.size() - 1) are in lexicographic order.
        */
       public static void sort(ArrayList<String> a)
       {
           int index, indexOfNextSmallest;
           for (index = 0; index < a.size() - 1; index++)
           { //Place the correct value in position index:
               indexOfNextSmallest =
                   indexOfSmallest(index, a);
               interchange(index, indexOfNextSmallest, a);
               //a.get(0), a.get(1), ..., a.get(index)
               //are sorted. The rest of the
               //elements are in the remaining positions.
           }
       }
       /**
        * Precondition: i and j are legal indices for the ArrayList a.
        * Postcondition: The values of a.get(i) and
        * a.get(j) have been interchanged.
        */
       private static void interchange(
           int i, int j, ArrayList<String> a)

```

```

        {
            String temp;
            temp = a.get(i);
            a.set(i, a.get(j));
            a.set(j, temp);
        }
    /**
     * Returns the index of the lexicographically first value among
     * a.get(startIndex), a.get(startIndex+1), ..., a.get(a.size()
     * - 1)
     */
    private static int indexOfSmallest(
        int startIndex, ArrayList<String> a)
    {
        String min = a.get(startIndex);
        int indexOfMin = startIndex;
        int index;
        for (index = startIndex + 1; index < a.size(); index++)
            if ((a.get(index)).compareTo(min) < 0)
            {
                min = a.get(index);
                indexOfMin = index;
            }
        return indexOfMin;
    }
}

```

10. No, the base type of an `ArrayList` cannot be a primitive type, such as `int`, `double`, or `char`. You can, however, have an `ArrayList` with base type `Integer` that can be used to store integers.
11. Notice that the following, while correct, is not as easy to understand as the for-each loop.

```

for (int i; i < a.size(); i++)
    System.out.println(a.get(i) + " differs from average by "
                        + (a.get(i) - average));

```

12.

```
public static <T> int getMidindex(T[] a)
{
    return a.length/2;
}
```

13. This is a trick question. As we explained in the text, you cannot have an array of type `UnorderedPair<String>[]` or of type `Pair<String>[]`.

Programming Projects

1. Write a computer program to make use of the dynamic data structure `ArrayList` provided in Java. The program should create a list of items available in a departmental store. First add at least five items in the list. Then display those items. Make use of the methods that can be used with an `ArrayList` to perform the following operations:
 - a. Fetch the current number of elements available in the list.
 - b. Check for a particular item in the list.
 - c. Replace an item of a list with a new item.

Also, in the end convert the latest `ArrayList` in to an `Array` and display the items that are available in the array.

2. Write a program that uses an `ArrayList` of parameter type `Dictionary` to store a database of words. The `Dictionary` class should store the term and its synonyms. Add appropriate accessor and mutator methods.

Your database program should present a menu that allows the user to add a term, delete a term, display all terms, search for a specific term, or search for a specific term and give the user the option to delete it. The search should find any term where any instance variable contains a target search string. For example, if “Legacy” is the search target, then the term should be displayed or deleted. Use the “for-each” loop to iterate through the `ArrayList`. The program should also check if the user inputs only text and should prompt the user for “Invalid Input” if the input is other than text.

3. Many Global Positioning Satellite (GPS) units can record waypoints. The waypoint marks the coordinates of a location on a map along with a timestamp. Consider a GPS unit that stores waypoints in terms of an (X, Y) coordinate on a map together with a timestamp t that records the number of seconds that have elapsed since the unit was turned on. Write a program that allows the user to enter as many waypoints as desired, storing each waypoint in an `ArrayList`, where each waypoint is represented by a class that you design. Each waypoint represents a successive sample point during a hike along some route. The coordinates should be input as doubles, and the timestamp as an integer. Have your program compute the total distance traveled and the average speed in miles per hour. Use the map scaling factor of $1 = 0.1$ miles. For example, if the only two waypoints are $(X=1, Y=1, T=0)$ and $(X=2, Y=1, T=3600)$, then the hiker traveled a distance of 0.1 miles in 3,600 seconds, or 0.1 miles per hour.
4. Write a generic class, `Marks`, with a type parameter `M` where `M` is a numeric object type (e.g., `Integer`, `Double`, or any class that extends `java.lang.Number`). Add a method named `orderAverage` that takes an `ArrayList` of type `M` and returns a double type data which is the average of the values in the `ArrayList`. Use the `doubleValue()` method in the `Number` class to retrieve the value of each number as a double. Test your method with suitable data. Your program should generate a compile-time error if your average method is invoked on an `ArrayList` and a nonnumeric element is encountered (e.g., `Strings`).

5. Create a generic class with a type parameter that simulates drawing an item at random out of a box. This class could be used for simulating a random drawing. For example, the box might contain `Strings` representing names written on a slip of paper, or the box might contain `Integers` representing a random drawing for a lottery based on numeric lottery picks. Create an `add` method that allows the user of the class to add an object of the specified type along with an `isEmpty` method that determines whether or not the box is empty. Finally, your class should have a `drawItem` method that randomly selects an object from the box and returns it. If the user attempts to draw an item out of an empty box, return `null`. Write a `main` method that tests your class.
6. Implement a priority queue capable of holding objects of an arbitrary type, `T`, by defining a `PriorityQueue` class that implements the queue with an `ArrayList`. A priority queue is a type of list where every item added to the queue also has an associated priority. Define priority in your application so that those items with the largest numerical value have the highest priority. Your class should support the following methods:
 - `Add(item, priority)`—Adds a new item to the queue with the associated priority.
 - `Remove()`—Returns the item with the highest priority and removes it from the queue. If the user attempts to remove from an empty queue, return `null`.

For example, if `q` is a priority queue defined to take `Strings`

```
q.add("X", 10);
q.add("Y", 1);
q.add("Z", 3);

System.out.println(q.remove()); // Returns X
System.out.println(q.remove()); // Returns Z
System.out.println(q.remove()); // Returns Y
```

Test your queue on data with priorities in various orders (e.g., ascending, descending, mixed). You can implement the priority queue by performing a linear search through the `ArrayList`. In future courses, you may study a data structure called a *heap* that is a more efficient way to implement a priority queue.

7. Programming Project 6.13 implemented a simple trivia game using an array of `Trivia` objects. Redo this project but use an `ArrayList` of `Trivia` objects instead of an array. The run-time behavior should remain identical to before.
8. In Programming Project 11.9, you were asked to implement a recursive algorithm to find all files that matched a target file name. Redo this Programming Project where the recursive method returns an `ArrayList` of `String` objects. Each string should store the pathname to the matching file. Return `null` if no matching files are found.



9. Use inheritance and classes to represent a deck of playing cards. Create a `Card` class that stores the suit (e.g., Clubs, Diamonds, Hearts, Spades) and name (e.g., Ace, 2, 10, Jack) of each card along with appropriate accessors, constructors, and mutators.

Next, create a `Deck` class that stores an `ArrayList` of `Card` objects. The default constructor should create objects that represent the standard 52 cards and store them in the `ArrayList`. The `Deck` class should have methods to do the following:

- Print every card in the deck.
- Shuffle the cards in the deck. You can implement this by randomly swapping every card in the deck.
- Add a new card to the deck. This method should take a `Card` object as a parameter and add it to the `ArrayList`.
- Remove a card from the deck. This removes the first card stored in the `ArrayList` and returns it.
- Sort the cards in the deck ordered by name.

Next, create a `Hand` class that represents the cards in a hand. `Hand` should be derived from `Deck`. This is because a hand is like a more specialized version of a deck; we can print, shuffle, add, remove, and sort cards in a hand just like cards in a deck. The default constructor should set the hand to an empty set of cards.

Finally, write a main method that creates a deck of cards, shuffles the deck, and creates two hands of 5 cards each. The cards should be removed from the deck and added to the hand. Test the sort and print functions for the hands and the deck. Finally, return the cards in the hand to the deck and test to ensure that the cards have been properly returned.

You may add additional methods or class variables as desired to implement your solution.

10. Do Programming Project 14.9 and extend the program to play blackjack, where the computer plays the role of the house and the user is a single player playing against the house. Use standard house rules for hitting or standing. Add more methods as necessary to implement your program. For an additional challenge, incorporate a betting component and additional blackjack rules, such as splitting or insurance.
11. Do Programming Project 14.9 and extend the program to play five-card stud poker between two hands. Add more methods as necessary to implement your program. For an additional challenge, incorporate a betting component.

This page intentionally left blank



Linked Data Structures **15**

15.1 JAVA LINKED LISTS 842

Example: A Simple Linked List Class 842
Working with Linked Lists 846
Node Inner Classes 852
Example: A Generic Linked List 855
The `equals` Method for Linked Lists 860

15.2 COPY CONSTRUCTORS AND THE `clone` METHOD ★ 862

Simple Copy Constructors and `clone` Methods ★ 862
Exceptions ★ 863
Example: A Linked List with a Deep Copy `clone` Method ★ 870

15.3 ITERATORS 873

Defining an Iterator Class 874
Adding and Deleting Nodes 879

15.4 VARIATIONS ON A LINKED LIST 884

Doubly Linked List 884
The Stack Data Structure 893
The Queue Data Structure 895
Running Times and Big-O Notation 898
Efficiency of Linked Lists 903

15.5 HASH TABLES WITH CHAINING 904

A Hash Function for Strings 905
Efficiency of Hash Tables 908

15.6 SETS 909

Fundamental Set Operations 910
Efficiency of Sets Using Linked Lists 915

15.7 TREES 916

Tree Properties 916
Example: A Binary Search Tree Class ★ 919
Efficiency of Binary Search Trees ★ 924

*If somebody there chanced to be
Who loved me in a manner true
My heart would point him out to me
And I would point him out to you.*

GILBERT AND SULLIVAN, *Ruddigore*, 1887.

Introduction

node and link

linked list

head node

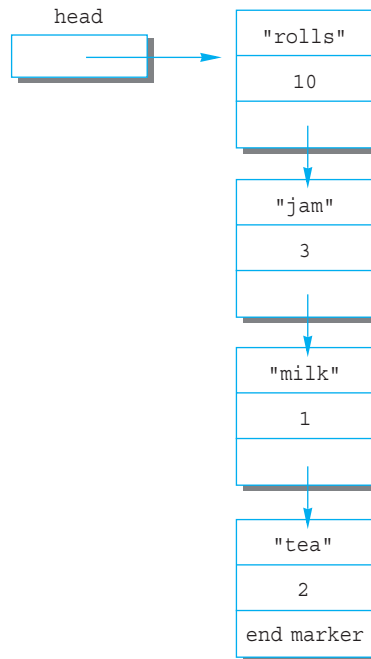
A linked data structure consists of capsules of data, known as **nodes**, which are connected via what are known as **links**. These links can be viewed as arrows and thought of as one-way passages from one node to another. The simplest kind of linked data structure consists of a single chain of nodes, each connected to the next by a link; this is known as a **linked list**. A sample linked list can be depicted as shown in Display 15.1. In Display 15.1, the nodes are represented by boxes that can each hold two kinds of data, a string and an integer, as in a shopping list. The links are depicted as arrows, which reflect the fact that your code must traverse the linked list in one direction without backing up. So there is a first node, a second node, and so on up to the last node. The first node is called the **head node**.

That information is all very vague but provides the general picture of what is going on in a linked list. It becomes concrete when you realize a linked list in some programming language. In Java, the nodes are realized as objects of a node class. The data in a node is stored via instance variables. The links are realized as references. Recall that a reference is simply a memory address. A reference is what is stored in a variable of a class type. So the link is realized as an instance variable of the type of the node class itself. In Java, a node in a linked list is connected to the next node by having an instance variable of the node type contain a reference (that is, a memory address) of where in memory the next node is stored.

Java comes with a `LinkedList` library class as part of the `java.util` package. It makes sense to use this library class, because it is well designed and well tested and will save you a lot of work. However, using the library class will not teach you how to implement linked data structures in Java. To do that, you need to see an implementation of a simple linked data structure, such as a linked list. So to let you see how this sort of thing is done in Java, we will construct our own simplified example of a linked list.

After discussing linked lists, we then go on to discuss more elaborate linked data structures, including sets, hash tables, and trees.

Display 15.1 Nodes and Links in a Linked List



Prerequisites

If you prefer, you may skip this chapter and go directly to Chapter 16 on collection classes or to Chapter 17 to begin your study of windowing interfaces using the Swing library. You have a good deal of flexibility in how you order the later chapters of this book.

This chapter requires material from Chapters 1 through 5, Chapter 14, and simple uses of inner classes (Section 13.2 of Chapter 13). Section 15.7 on trees additionally requires Chapter 11 on recursion.

Sections 15.2 through 15.7 do not depend on each other in any essential way. In particular, you may omit Section 15.2 on cloning and still read the following sections. Sections 15.2 through 15.7 do not depend in any essential way on the material on generic linked lists in subsections of Section 15.1.

15.1 Java Linked Lists

A chain is only as strong as its weakest link.

Thomas Reid's Essays on the Intellectual Powers of Man, 1786.

A linked list is a linked data structure consisting of a single chain of nodes, each connected to the next by a link. This is the simplest kind of linked data structure, but it is nevertheless widely used. In this section, we give examples of linked lists and develop techniques for defining and working with linked lists in Java.

EXAMPLE: A Simple Linked List Class

Display 15.1 is a diagram of a linked list. In the display, the nodes are the boxes. In your Java code, a node is an object of some node class, such as the class `Node1` given in Display 15.2. Each node has a place (or places) for some data and a place to hold a link to another node. The links are shown as arrows that point to the node they “link” to. In Java, the links will be implemented as references to a node stored in an instance variable of the `node` type.

The `Node1` class is defined by specifying, among other things, an instance variable of type `Node1` that is named `link`. This allows each node to store a reference to another node of the same type. There is a kind of circularity in such definitions, but this circularity is allowed in Java. (One way to see that this definition is not logically inconsistent is to note that we can draw pictures, or even build physical models, of our linked nodes.)

The first node, or start node, in a linked list is called the *head node*. If you start at the head node, you can traverse the entire linked list, visiting each node exactly once. As you will see shortly, in Java your code must intuitively “follow the link arrows.” In Display 15.1, the box labeled `head` is not itself the head node; it is not even a node. The box labeled `head` is a variable of type `Node1` that contains a reference to the first node in the linked list—that is, a reference to the head node. The function of the variable `head` is that it allows your code to find that first or head node. The variable `head` is declared in the obvious way:

```
Node1 head;
```

In Java, a linked list is an object that in some sense contains all the nodes of the linked list. Display 15.3 contains a definition of a linked list class for a linked list such as the one in Display 15.1. Notice that a linked list object does not directly contain all the nodes in the linked list. It contains only the instance variable `head` that contains a reference to the first or head node. However, every node can be reached from this first or head node. The `link` instance variable of the first and every `Node1` of the linked list contains a reference to the next `Node1` in the linked list. Thus, the arrows shown in the diagram in Display 15.1 are realized as references in Java. Each node object of a linked list contains (in its `link` instance variable) a reference to another object of the class `Node1`, and this other object contains a reference to another object of the class `Node1`, and so on until the end of the linked list. Thus, a linked list object, indirectly at least, contains all the nodes in the linked list.

Display 15.2 A Node Class

```
public class Node1
{
    private String item;
    private int count;
    private Node1 link;

    public Node1( )
    {
        link = null;
        item = null;
        count = 0;
    }
    public Node1(String newItem, int newCount, Node1 linkValue)
    {
        setData(newItem, newCount);
        link = linkValue;
    }

    public void setData(String newItem, int newCount)
    {
        item = newItem;
        count = newCount;
    }
    public void setLink(Node1 newLink)
    {
        link = newLink;
    }

    public String getItem( )
    {
        return item;
    }

    public int getCount( )
    {
        return count;
    }

    public Node1 getLink( )
    {
        return link;
    }
}
```

*A node contains a reference to another node.
That reference is the link to the next node.*

*We will define a number of node classes so we
numbered the names as in Node1.*

*We will give a better definition of a
node class later in this chapter.*

Display 15.3 A Linked List Class (part 1 of 2)

```
1 public class LinkedList1
2 {
3     private Node1 head;
4
5     public LinkedList1( )
6     {
7         head = null;
8     }
9
10    /**
11     Adds a node at the start of the list with the specified data.
12     The added node will be the first node in the list.
13    */
14    public void addToStart(String itemName, int itemCount)
15    {
16        head = new Node1(itemName, itemCount, head);
17    }
18
19    /**
20     Removes the head node and returns true if the list contains at
21     least one node. Returns false if the list is empty.
22    */
23    public boolean deleteHeadNode( )
24    {
25        if (head != null)
26        {
27            head = head.getLink( );
28            return true;
29        }
30        else
31            return false;
32    }
33
34    /**
35     Returns the number of nodes in the list.
36    */
37    public int size( )
38    {
39        int count = 0;
40        Node1 position = head;
```

We will define a letter linked list class later in this chapter.

Display 15.3 A Linked List Class (part 2 of 2)

```

39     while (position != null)
40     {
41         count++;
42         position = position.getLink( );
43     }
44     return count;
45 }

46 public boolean contains(String item)
47 {
48     return (find(item) != null);
49 }

50 /**
51  * Finds the first node containing the target item, and returns a
52  * reference to that node. If target is not in the list, null is
53  * returned.
54  */
55 private Node1 find(String target)
56 {
57     Node1 position = head;
58     String itemAtPosition;
59     while (position != null)
60     {
61         itemAtPosition = position.getItem( );
62         if (itemAtPosition.equals(target))
63             return position;
64         position = position.getLink( );
65     }
66     return null; //target was not found
67 }
68 public void outputList( )
69 {
70     Node1 position = head;
71     while (position != null)
72     {
73         System.out.println(position.getItem( ) + " "
74                             + position.getCount( ));
75         position = position.getLink( );
76     }
77 }

```

This last node is indicated by the link field being equal to null.

This is the way you traverse an entire linked list.

Working with Linked Lists

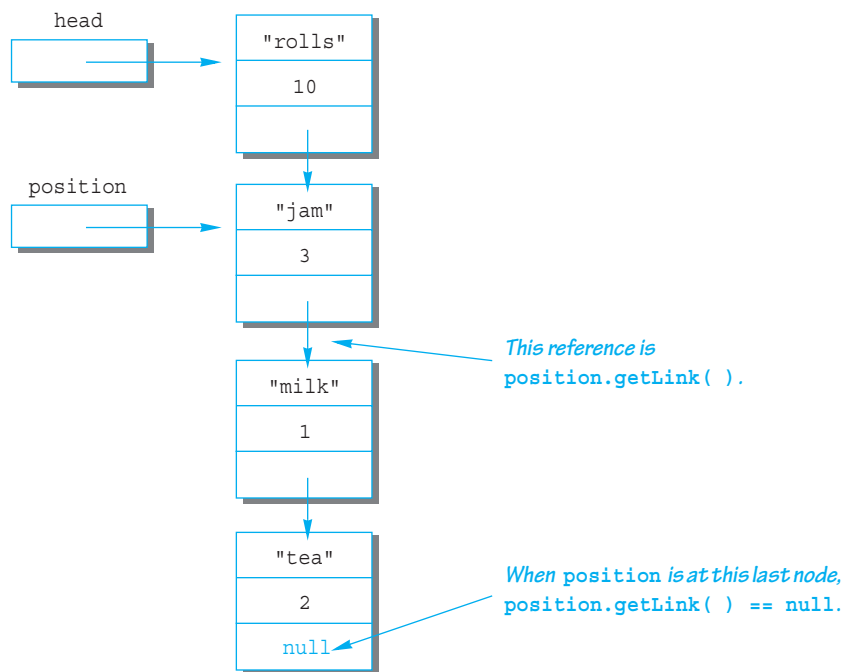
When dealing with a linked list, your code needs to be able to “get to” that first or head node, and you need some way to detect when the last node is reached. To get your code to the first node, use a variable of type `Node1` that always contains a reference to the first node. In Display 15.3, the variable with a reference to the first node is named `head`. From that first or head node, your code can follow the links through the linked list. But how does your code know when it is at the last node in a linked list?

In Java, indicate the end of a linked list by setting the `link` instance variable of the last node in the linked list to `null`, as shown in Display 15.4. That way your code can test whether or not a node is the last node in a linked list by testing whether its `link` instance variable contains `null`. Remember that you check for a link being “equal” to `null` by using `==`, and not using any equals method.

Also use `null` to indicate an empty linked list. The `head` instance variable contains a reference to the first node in the linked list, or it contains `null` if the linked list is empty (that is, if the linked list contains no nodes). The only constructor sets this `head` instance variable to `null`, indicating that a newly created linked list is empty.

empty list

Display 15.4 Traversing a Linked List



Indicating the End of a Linked List

The last node in a linked list should have its `link` instance variable set to `null`. That way, your code can check whether a node is the last node by checking whether its `link` instance variable is equal to `null`.

An Empty List Is Indicated by `null`

Suppose the variable `head` is supposed to contain a reference to the first node in a linked list. Linked lists usually start out empty. To indicate an empty linked list, give the variable `head` the value `null`. This is traditional and works out nicely for many linked list manipulation algorithms.

traversing a linked list

Before we go on to discuss how nodes are added and removed from a linked list, let us suppose that the linked list already has a few nodes and that you want to write out the contents of all the nodes to the screen. You can do this with the method `outputList` (Display 15.3), whose body is reproduced here:

```
Node1 position = head;
while (position != null)
{
    System.out.println(position.getItem( ) + " "
                       + position.getCount( ));
    position = position.getLink( );
}
```

The method uses a local variable named `position` that contains a reference to one node. The variable `position` starts out with the same reference as the `head` instance variable, so it starts out positioned at the first node. The `position` variable then has its position moved from one node to the next with the assignment

```
position = position.getLink( );
```

This is illustrated in Display 15.4. To see that this assignment “moves” the `position` variable to the next node, note that the `position` variable contains a reference to the node pointed to by the `position` arrow in Display 15.4. So, `position` is a name for that node, and `position.link` is a name for the link to the next node. The value of `link` is produced with the accessor method `getLink`. Thus, a reference to the next node in the linked list is `position.getLink( )`. You “move” the `position` variable by giving it the value of `position.getLink( )`.

The method `outputList` continues to move the `position` variable down the linked list and outputs the data in each node as it goes along. When `position` reaches the last node, it outputs the data in that node and then again executes

```
position = position.getLink( );
```

If you study Display 15.4, you will see that when `position` leaves the last node, its value is set to `null`. At this point, we want to stop the loop, so we iterate the loop

```
while (position != null)
```

A similar technique is used to traverse the linked list in the methods `size` and `find`.

adding a node

Next let us consider how the method `addToStart` adds a node to the start of the linked list so that the new node becomes the first node in the list. It does this with the single statement

```
head = new Node1(itemName, itemCount, head);
```

The new node is created with

```
new Node1(itemName, itemCount, head)
```

which returns a reference to this new node. The assignment statement sets the variable `head` equal to a reference to this new node, making the new node the first node in the linked list. To link this new node to the rest of the list, we need only set the `link` instance variable of the new node equal to a reference to the *old first node*. But we have already done that: `head` used to point to the old first node, so if we use the name `head` on *the right-hand side of the assignment operator*, `head` will denote a reference to the old first node. Therefore, the new node produced by

```
new Node1(itemName, itemCount, head)
```

points to the old first node, which is just what we wanted. This is illustrated in Display 15.5.

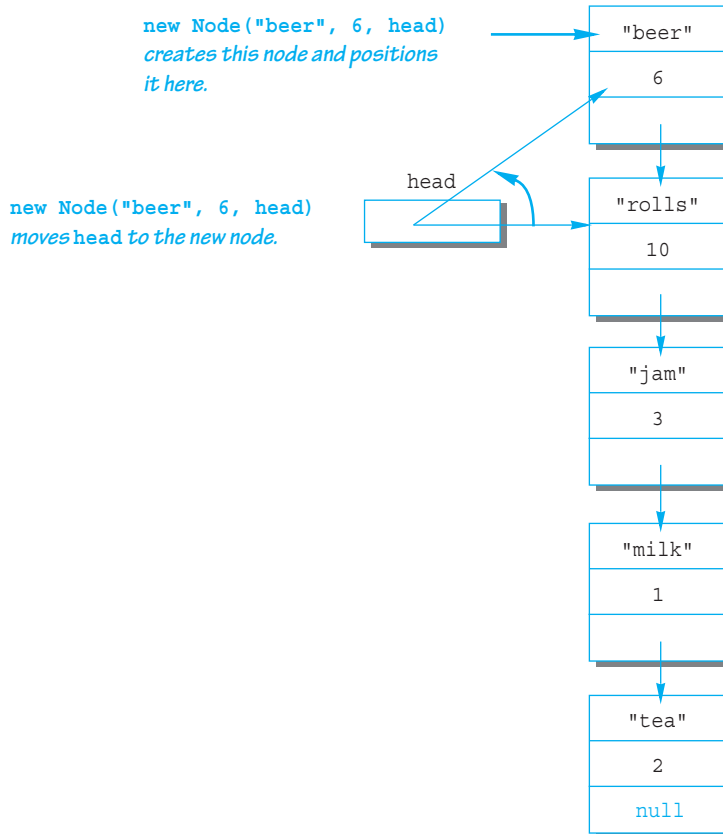
Later, we will discuss adding nodes at other places in a linked list, but the easiest place to add a node is at the start of the list. Similarly, the easiest place to delete a node is at the start of the linked list.

removing a node

The method `deleteHeadNode` removes the first node from the linked list and leaves the `head` variable pointing to (that is, containing a reference to) the old second node (which is now the first node) in the linked list. This is done with the following assignment:

```
head = head.getLink( );
```

Display 15.5 Adding a Node at the Start



This removes the first node from the linked list and leaves the linked list one node shorter. But what happens to the deleted node? At some point, Java will automatically collect it, along with any other nodes that are no longer accessible, and recycle the memory they occupy. This is known as **automatic garbage collection**.

Display 15.6 contains a simple program that demonstrates how some of the methods in the class `LinkedList1` behave.

automatic
garbage
collection

Display 15.6 A Linked List Demonstration

```

1  public class LinkedList1Demo
2  {
3      public static void main(String[] args)
4      {
5          LinkedList1 list = new LinkedList1( );
6          list.addToStart("Apples", 1);
7          list.addToStart("Bananas", 2);
8          list.addToStart("Cantaloupe", 3);
9          System.out.println("List has " + list.size( )
10                           + " nodes.");
11          list.outputList( );
12          if (list.contains("Cantaloupe"))
13              System.out.println("Cantaloupe is on list.");
14          else
15              System.out.println("Cantaloupe is NOT on list.");
16
17          list.deleteHeadNode( );
18
19          if (list.contains("Cantaloupe"))
20              System.out.println("Cantaloupe is on list.");
21          else
22              System.out.println("Cantaloupe is NOT on list.");
23
24          while (list.deleteHeadNode( ))
25              ; //Empty loop body
26          System.out.println("Start of list:");
27          list.outputList( );
28          System.out.println("End of list.");
29      }
30  }

```

Cantaloupe is now in the head node.

Empties the list. There is no loop body because the method deleteHeadNode both performs an action on the list and returns a Boolean value.

Sample Dialogue

```

List has 3 nodes.
Cantaloupe 3
Bananas 2
Apples 1
Cantaloupe is on list.
Cantaloupe is NOT on list.
Start of list:
End of list.

```

Self-Test Exercises

1. What output is produced by the following code?

```
LinkedList1 list = new LinkedList1( );  
list.addToStart("apple pie", 1);  
  
list.addToStart("hot dogs", 12);  
list.addToStart("mustard", 1);  
list.outputList( );
```

2. Define a boolean valued method named `isEmpty` that can be added to the class `LinkedList1` (Display 15.3). The method returns `true` if the list is empty and `false` if the list has at least one node in it.
3. Define a void method named `clear` that can be added to the class `LinkedList1` (Display 15.3). The method has no parameters and it empties the list.



PITFALL: Privacy Leaks

It may help you to understand this section if you first review the Pitfall section of the same name in Chapter 5.

Consider the method `getLink` in the class `Node1` (Display 15.2). It returns a value of type `Node1`. That is, it returns a reference to a `Node1`. In Chapter 5, we said that if a method (such as `getLink`) returns a reference to an instance variable of a (mutable) class type, then the `private` restriction on the instance variable can easily be defeated because getting a reference to an object may allow a programmer to change the private instance variables of the object. There are a number of ways to fix this, the most straightforward of which is to make the class `Node1` a private inner class in the method `Node1`, as discussed in the next subsection.

There is no danger of a privacy leak with the class `Node1` when it is used in the class definition for `LinkedList1`. However, there is no way to guarantee that the class `Node1` will be used only in this way unless you take some precaution, such as making the class `Node1` a private inner class in the class `LinkedList1`.

An alternate solution is to place both of the classes `Node1` and `LinkedList1` into a package, and change the `private` instance variable restriction to the package restriction as discussed in Chapter 7.

Note that this privacy problem can arise in any situation in which a method returns a reference to a private instance variable of a class type. The method `getItem( )` of the class `Node1` comes very close to having this problem. In this case, the method `getItem` causes no privacy leak, but only because the class `String` is not a mutable class (that is, it has no methods that will allow the user to change the value of the string without changing the reference). If instead of storing data of type `String` in our list we had stored data of some mutable class type, then defining an accessor method similarly to `getItem` would produce a privacy leak. ■

Node Inner Classes

You can make a linked list, or any similar data structures, self-contained by making the node class an inner class. In particular, you can make the class `LinkedList1` more self-contained by making `Node1` an inner class, as follows:

```
public class LinkedList1
{
    private class Node1
    {
        <The rest of the definition of Node1 can be
        the same as in Display 15.2.>
    }

    private Node1 head;
    <The constructor and methods in Display 15.3 are inserted here.>
}
```

Note that we have made the class `Node1` a private inner class. If an inner class is not intended to be used elsewhere, it should be made private. Making `Node1` a private inner class hides all objects of the inner class and avoids a privacy leak.

If you are going to make the class `Node1` a private inner class in the definition of `LinkedList1`, then you can safely simplify the definition of `Node1` by eliminating the accessor and mutator methods (the `set` and `get` methods) and just allowing direct access to the instance variables (`item`, `count`, and `link`) from methods of the outer class. In Display 15.7, we have written a class similar to `LinkedList1` in this way. The rewritten version, named `LinkedList2`, is like the class `LinkedList1` in Display 15.3 in that it has the same methods that perform basically the same actions. To keep the discussion simple, `LinkedList2` has only one data field instead of two. We could easily have retained the two data fields, but we wanted a notationally simple example without any distracting details. (See Self-Test Exercise 8 for a version that has the same kind of data in each node as in the nodes of `LinkedList1`.)

Display 15.7 A Linked List Class with a Node Inner Class (part 1 of 3)

```
1 public class LinkedList2
2 {
3     private class Node
4     {
5         private String item;
6         private Node link;
7
8         public Node( )
9         {
10             item = null;
11             link = null;
12         }
13     }
14 }
```

Display 15.7 A Linked List Class with a Node Inner Class (part 2 of 3)

```
12     public Node(String newItem, Node linkValue)
13     {
14         item = newItem;
15         link = linkValue;
16     }
17 } //End of Node inner class

18 private Node head;

19 public LinkedList2( )
20 {
21     head = null;
22 }
23 /**
24  * Adds a node at the start of the list with the specified data.
25  * The added node will be the first node in the list.
26  */
27 public void addToStart(String itemName)
28 {
29     head = new Node(itemName, head);
30 }

31 /**
32  * Removes the head node and returns true if the list contains at
33  * least one node. Returns false if the list is empty.
34  */
35 public boolean deleteHeadNode( )
36 {
37     if (head != null)
38     {
39         head = head.link;
40         return true;
41     }
42     else
43         return false;
44 }

45 /**
46  * Returns the number of nodes in the list.
47  */
48 public int size( )
49 {
50     int count = 0;
51     Node position = head;
52     while (position != null)
53     {
54         count++;
```

(continued)

Display 15.7 A Linked List Class with a Node Inner Class (part 3 of 3)

```
55         position = position.link;
56     }
57     return count;
58 }

59 public boolean contains(String item)
60 {
61     return (find(item) != null);
62 }

63 /**
64  * Finds the first node containing the target item, and returns a
65  * reference to that node. If target is not in the list, null is
66  * returned.
67  */
68 private Node find(String target)
69 {
70     Node position = head;
71     String itemAtPosition;
72     while (position != null)
73     {
74         itemAtPosition = position.item;
75         if (itemAtPosition.equals(target))
76             return position;
77         position = position.link;
78     }
79     return null; //target was not found
80 }
81 public void outputList( )
82 {
83     Node position = head;
84     while (position != null)
85     {
86         System.out.println(position.item );
87         position = position.link;
88     }
89 }

89 public boolean isEmpty( )
90 {
91     return (head == null);
92 }

93 public void clear( )
94 {
95     head = null;
96 }
97 }
```

Note that the outer class has direct access to the inner class's instance variables, such as link.

Self-Test Exercises

4. Would it make any difference if we changed the `Node` inner class in Display 15.7 from a private inner class to a public inner class?
5. Keeping the inner class `Node` in Display 15.7 as `private`, what difference would it make if any of the instance variables or methods in the class `Node` had its access modifiers changed from `private` to `public` or `package` access?
6. Why does the definition of the inner class `Node` in Display 15.7 not have the accessor and mutator methods `getLink`, `setLink`, or other `get` and `set` methods for the link fields similar to those in the class definition of `Node1` in Display 15.2?
7. Would it be legal to add the following method to the class `LinkedList2` in Display 15.7?

```
public Node startNode( )
{
    return head;
}
```

8. Rewrite the definition of the class `LinkedList2` in Display 15.7 so that it has data of a type named `Entry`, which is a public inner class. Objects of type `Entry` have two instance variables defined as follows:

```
private String item;
private int count;
```

This rewritten version of `LinkedList2` will be equivalent to `LinkedList1` in that it has the same methods doing the same things, and it will hold equivalent data in its nodes.

EXAMPLE: A Generic Linked List

Display 15.8 shows a generic linked list with a type parameter `T` for the type of data stored in a node. This generic linked list has the same methods, coded in basically the same way, as our previous linked list (Display 15.7), but we used a type parameter for the type of data in the nodes.

Display 15.10 contains a demonstration program for our generic linked list. The demonstration program uses the class `Entry`, defined in Display 15.9, as the type plugged in for the type parameter `T`. Note that if you want multiple pieces of data in each node, you simply use a class type that has multiple instance variables and plug in this class for the type parameter `T`.

Display 15.8 A Generic Linked List Class (part 1 of 3)

```

1  public class LinkedList3<T>
2  {
3      private class Node<T>
4      {
5          private T data;
6          private Node<T> link;
7          public Node( )
8          {
9              data = null;
10             link = null;
11         }

12         public Node (T newData, Node<T> linkValue)
13         {
14             data = newData;
15             link = linkValue;
16         }
17     } //End of Node<T> inner class
18     private Node<T> head;

19     public LinkedList3( )
20     {
21         head = null;
22     }

23     /**
24      * Adds a node at the start of the list with the specified data.
25      * The added node will be the first node in the list.
26      */
27     public void addToStart(T itemData)
28     {
29         head = new Node<T> (itemData, head);
30     }

31     /**
32      * Removes the head node and returns true if the list contains at
33      * least one node. Returns false if the list is empty.
34      */
35     public boolean deleteHeadNode( )
36     {
37         if (head != null)
38         {
39             head = head.link;
40             return true;
41         }
42         else
43             return false;
44     }
45     /**

```

*This linked list holds objects of type T.
The type T should have well-defined
equals and toString methods.*

Display 15.8 A Generic Linked List Class (part 2 of 3)

```

46     Returns the number of nodes in the list.
47     */
48     public int size( )
49     {
50         int count = 0;
51         Node<T> position = head;
52         while (position != null)
53         {
54             count++;
55             position = position.link;
56         }
57         return count;
58     }

59     public boolean contains(T item)
60     {
61         return (find(item) != null);
62     }
63     /**
64      Finds the first node containing the target item, and returns a
65      reference to that node. If target is not in the list, null is
66      returned.
67     */
68     private Node find(T target)
69     {
70         Node<T> position = head;
71         T itemAtPosition;
72         while (position != null)
73         {
74             itemAtPosition = position.data;
75             if (itemAtPosition.equals(target))
76                 return position;
77             position = position.link;
78         }
79         return null; //target was not found

80     /**
81      Finds the first node containing the target and returns a reference
82      to the data in that node. If target is not in the list, null is
83      returned.
84     */
85     public T findData(T target)
86     {
87         Node<T> result = find(target);
88         if (result == null)
89             return null;
90         else
91             return result.data;
92     }

```

Type T must have a well-defined equals for this method to work.

(continued)

Display 15.8 A Generic Linked List Class (part 3 of 3)

```

92     public void outputList( )
93     {
94         Node<T> position = head;
95         while (position != null)
96         {
97             System.out.println(position.data);
98             position = position.link;
99         }
100    }

101    public boolean isEmpty( )
102    {
103        return (head == null);
104    }

105    public void clear( )
106    {
107        head = null;
108    }
109    /*
110     For two lists to be equal they must contain the same data items in
111     the same order. The equals method of T is used to compare data
112     items.
113     */
114    public boolean equals(Object otherObject)
115    {
116        if (otherObject == null)
117            return false;
118        else if (getClass( ) != otherObject.getClass( ))
119            return false;
120        else
121        {
122            LinkedList3<T> otherList = (LinkedList3<T>)otherObject;
123            if (size( ) != otherList.size( ))
124                return false;
125            Node<T> position = head;
126            Node<T> otherPosition = otherList.head;
127            while (position != null)
128            {
129                if (!(position.data.equals(otherPosition.data)))
130                    return false;
131                position = position.link;
132                otherPosition = otherPosition.link;
133            }
134            return true; //no mismatch was not found
135        }
136    }

```

Type T must have a well-defined toString methods for this to work.

This clears the entire list. Any nodes that were referenced by head are reclaimed through automatic garbage collection.

Display 15.9 A Sample Class for the Data in a Generic Linked List

```

1  public class Entry
2  {
3      private String item;
4      private int count;

5      public Entry(String itemData, int countData)
6      {
7          item = itemData;
8          count = countData;
9      }

10     public String toString( )
11     {
12         return (item + " " + count);
13     }

14     public boolean equals(Object otherObject)
15     {
16         if (otherObject == null)
17             return false;
18         else if (getClass( ) != otherObject.getClass( ))
19             return false;
20         else
21         {
22             Entry otherEntry = (Entry)otherObject;
23             return (item.equals(otherEntry.item)
24                 && (count == otherEntry.count));
25         }
26     }
27 }

```

<There should be other constructors and methods, including accessor and mutator methods, but we do not use them in this demonstration.>

Display 15.10 A Generic Linked List Demonstration (part 1 of 2)

```

1  public class GenericLinkedListDemo
2  {
3      public static void main(String[] args)
4      {
5          LinkedList3<Entry> list = new LinkedList3<Entry>( );

6          Entry entry1 = new Entry("Apples", 1);
7          list.addToStart(entry1);
8          Entry entry2 = new Entry("Bananas", 2);
9          list.addToStart(entry2);
10         Entry entry3 = new Entry("Cantaloupe", 3);
11         list.addToStart(entry3);
12         System.out.println("List has " + list.size( )
13             + " nodes.");

```

(continued)

Display 15.10 A Generic Linked List Demonstration (part 2 of 2)

```

14         list.outputList ( );
15         System.out.println("End of list.");
16     }
17 }

```

Sample Dialogue

```

List has 3 nodes.
Cantaloupe 3
Bananas 2
Apples 1
End of list.

```

**PITFALL: Using Node Instead of Node<T>**

This pitfall is explained by example, using the `LinkedList3<T>` class in Display 15.8. However, the lesson applies to any generic linked structure with a node inner class. The type parameter need not be `T` and the node class name need not be `Node`, but for simplicity, we will use `T` and `Node`.

When defining the `LinkedList3<T>` class in Display 15.8, the type for a node is `Node<T>`; it is not `Node`. However, it is easy to forget the type specification `<T>` and write `Node` instead of `Node<T>`. If you omit the `<T>`, you may or may not get a compiler error message, depending on other details of your code. If you do get a compiler error message, it is likely to seem bewilderingly strange. The problem is that `Node` actually means something. (We do not have time to stop and explain what `Node` means, but it means something similar to a node with data type `Object`, rather than data type `T`.) Your only defense against this pitfall is to be very careful; if you do get a bewildering compiler error message, look for a missing `<T>`.

Sometimes a compiler warning message can be helpful when you make this mistake. If you get a warning that mentions a type cast from `Node` to `Node<T>`, look for an omitted `<T>`.

Finally, we should note that sometimes your code will compile and even run correctly if you omit the `<T>` from `Node<T>`. ■

The equals Method for Linked Lists

The linked lists we presented in Displays 15.3 and 15.7 do not have an `equals` method. We did that to keep the examples simple and not detract from the main message. However, a linked list class should normally have an `equals` method.

There is more than one approach to defining a reasonable `equals` method for a linked list. The two most obvious are the following:

1. Two linked lists are equal if they contain the same data entries (possibly ordered differently).

equals

2. Two linked lists are equal if they contain the same data entries in the same order; that is, the data in the first node of the calling object equals the data in the first node of the other linked list, the data in the two second nodes are equal, and so forth.

It is not true that one of these is the correct approach to defining an `equals` method and the other is incorrect. In different situations, you might want different definitions of `equals`. However, the most common way to define `equals` for a linked list is approach 2. A definition of `equals` that follows approach 2 and that can be added to the class `LinkedList2` in Display 15.7 is given in Display 15.11. The generic linked list in Display 15.8 also contains an `equals` method that follows approach 2.

Note that when we define `equals` for our linked list with type parameter `T`, we trust the programmer who wrote the definition for the type plugged in for `T`. We are assuming the programmer has redefined the `equals` method so that it provides a reasonable test for equality. Situations such as this are the reason it is so important to always include an `equals` method in the classes you define.

Display 15.11 An `equals` Method for the Linked List in Display 15.7

```

1  /*
2   For two lists to be equal they must contain the same data items in
3   the same order.
4  */
5  public boolean equals(Object otherObject)
6  {
7      if (otherObject == null)
8          return false;
9      else if (getClass( ) != otherObject.getClass( ))
10         return false;
11     else
12     {
13         LinkedList2 otherList = (LinkedList2)otherObject;
14         if (size( ) != otherList.size( ))
15             return false;
16         Node position = head;
17         Node otherPosition = otherList.head;
18         while (position != null)
19         {
20             if ( !(position.item.equals(otherPosition.item)) )
21                 return false;
22             position = position.link;
23             otherPosition = otherPosition.link;
24         }
25         return true; //A mismatch was not found
26     }
27 }
```

15.2 Copy Constructors and the `clone` Method ★

There are three ways to do anything:

The right way,

the wrong way,

and the army way.

FILM A Walk in the Sun, 20th Century Fox, Written by Harry Brown and Robert Rossen, 1945.

The way Java handles cloning, and object copying in general, is complicated and can be both subtle and difficult. Some authorities think that the `clone` method was done so poorly in Java that they prefer to ignore it completely and define their own methods for copying objects. I have some sympathy for that view, but before you dismiss Java's approach to cloning, it might be a good idea to see what the approach entails. Linked data structures, such as linked lists, are an excellent setting for discussing cloning because they are an excellent setting for discussing deep versus shallow copying.

This section first presents a relatively simple way to define copy constructors and the `clone` method, but this approach unfortunately produces only shallow copies. We then go on to present one way to produce a deep copy `clone` method and to do so within the official prescribed rules of the Java documentation.

Readers with very little programming experience may be better off skipping this entire section until they become more comfortable with Java. Other readers may prefer to read only the first subsection and possibly the Pitfall “The `clone` Method Is Protected in Object ★.”

Simple Copy Constructors and `clone` Methods ★

Display 15.12 contains a copy constructor and `clone` method definitions that could be added to the definition of the generic linked list class in Display 15.8. The real work is done by the private helping method `copyOf`, so our discussion focuses on the method `copyOf`.

The private method `copyOf` takes an argument that is a reference to the head node of a linked list and returns a reference to the head node of a copy of that linked list. The easiest way to do this is to return the argument. This would, however, simply produce another name for the argument list. We do not want another name; we want another list. So, the method goes down the argument list one node at a time (with `position`) and makes a copy of each node. The linked list of the calling object is built up node by node by adding these new nodes to its linked list. However, there is a complication. We cannot simply add the new nodes at the head (start) end of the list being built. If we did, then the nodes would end up in the reverse of the desired order. So, the new nodes are added to the end of the linked list being built. The variable `end` of type `Node<T>` is kept positioned at the last node so that it is possible to add nodes at the end of the linked list being built. In this way, a copy of the list in the calling object is created so that the order of the nodes is preserved.

The copy constructor is defined by using the private helping method `copyOf` to create a copy of the list of nodes. Other details of the copy constructor and the `clone` method are done in the standard way.

Although the copy constructor and the `clone` method each produce a new linked list with all new nodes, the new list is not truly independent because the data objects are not cloned. See the Pitfall “The `clone` Method Is Protected in Object ★” for a discussion of this point. One way to fix this shortcoming is discussed later in the Programming Tip entitled “Use a Type Parameter Bound for a Better `clone`.”

Exceptions ★

A generic data structure, such as the class `LinkedList` in Display 15.12, is likely to have methods that throw exceptions. Situations such as a `null` argument to the copy constructor might be handled differently in different situations, so it is best to throw a `NullPointerException` if this happens and let the programmer who is using the linked list handle the exception. This is what we did with the copy constructor in Display 15.12. A `NullPointerException` is an unchecked exception, which means that it need not be caught or declared in a `throws` clause. When thrown by a method of a linked list class, it can be treated simply as a run-time error message. The exception can instead be caught in a `catch` block if there is some suitable action that can be taken.

Display 15.12 A Copy Constructor and `clone` Method for a Generic Linked List (part 1 of 3)

```

1  public class LinkedList3<T> implements Cloneable
2  {
3      private class Node<T>
4      {
5          private T data;
6          private Node<T> link;
7
8          public Node( )
9          {
10             data = null;
11             link = null;
12
13             public Node(T newData, Node<T> linkValue)
14             {
15                 data = newData;
16                 link = linkValue;
17             }
18         } //End of Node<T> inner class

```

This copy constructor and this clone method do not make deep copies. We discuss one way to make a deep copy in the Programming Tip “Use a Type Parameter Bound for a Better clone.”

(continued)

Display 15.12 A Copy Constructor and clone Method for a Generic Linked List (part 2 of 3)

```

18     private Node<T> head;
        <All the methods from Display 15.8 are in the class definition,
           but they are not repeated in this display.>

19     /**
20      * Produces a new linked list, but it is not a true deep copy.
21      * Throws a NullPointerException if other is null.
22      */
23     public LinkedList3(LinkedList3<T> otherList)
24     {
25         if (otherList == null)
26             throw new NullPointerException( );
27         if (otherList.head == null)
28             head = null;
29         else
30             head = copyOf(otherList.head);
31     }
32
33
34     public LinkedList3<T> clone( )
35     {
36         try
37         {
38             LinkedList3<T> copy =
39                 (LinkedList3<T>)super.clone( );
40             if (head == null)
41                 copy.head = null;
42             else
43                 copy.head = copyOf(head);
44             return copy;
45         }
46         catch(CloneNotSupportedException e)
47         { //This should not happen.
48             return null; //To keep the compiler happy.
49         }
50     }

51     /**
52      * Precondition: otherHead != null
53      * Returns a reference to the head of a copy of the list
54      * headed by otherHead. Does not return a true deep copy.
55     */

```

Display 15.12 A Copy Constructor and clone Method for a Generic Linked List (part 3 of 3)

```

56     private Node<T> copyOf(Node<T> otherHead)
57     {
58         Node<T> position = otherHead; //moves down other's list.
59         Node<T> newHead; //will point to head of the copy list.
60         Node<T> end = null; //positioned at end of new growing list.
61                                     Invoking clone with position.data would be illegal.
62         //Create first node:
63         newHead =
64             new Node<T>(position.data, null);
65         end = newHead;
66         position = position.link;
67
68         while (position != null)
69         { //copy node at position to end of new list.
70             end.link =
71                 new Node<T>(position.data, null);
72             end = end.link;
73             position = position.link;
74         }
75
76         return newHead;
77     }

```

Invoking clone with position.data would be illegal.



PITFALL: The clone Method Is Protected in Object ★

When defining the copy constructor and clone method for our generic linked list (Display 15.12), we would have liked to have cloned the data in the list being copied. We would have liked to change the code in the helping method copyOf by adding invocations of the clone method as follows:

```

newHead =
    new Node((T) (position.data).clone( ), null);
end = newHead;
position = position.link;

while (position != null)
{ //copy node at position to end of new list.
    end.link =
        new Node((T) (position.data).clone( ), null);
    end = end.link;
    position = position.link;
}

```

This code is identical to code in copyOf except for the addition of the invocations of clone and the type casts. (The type casts are needed because Java thinks clone returns a value of type Object.) If this modified code (with the clone method) would

(continued)

**PITFALL: (continued)**

compile (and if the type plugged in for `T` has a well-defined `clone` method that makes a deep copy), then this modified code would produce a truly independent linked list with no references in common with the list being copied. Unfortunately, this code will not compile.

If you try to compile this code, you will get an error message saying that the method `clone` is protected in the class `Object`. True, we used the type `T`, not the type `Object`, but any class can be plugged in for `T`. So when the generic linked list is compiled, all Java knows about the type `T` is that it is a descendent class of `Object`. Because the designers of the `Object` class chose to make the method `clone` protected, you simply cannot use the `clone` method in the definition of methods such as `copyOf`.

Why was the `clone` method labeled protected in `Object`? Apparently, this was done for security reasons. If a class could use the `clone` method unchanged from `Object`, then that would open the possibility of copying sections of memory unchanged and unchecked and so might give unauthorized memory access. The problem is made more serious by the fact that Java is used to run programs on other machines across the Internet.

The way Java defines the `clone` method in `Object` and the way it specifies how `clone` should be defined in other classes are controversial. Do not be surprised if some future version of Java handles the `clone` method differently. But for now, you are stuck with these `clone` problems.

In many situations, the version of `copyOf` in Display 15.12 (without the use of `clone`) is good enough, but there is a way to get a true deep copy. One way to get a deep copy is to somehow restrict the type `T` to classes that do have a public `clone` method that makes a deep copy. Something such as this can be done and is discussed in the Programming Tip “Use a Type Parameter Bound for a Better `clone`.” ■

**TIP: Use a Type Parameter Bound for a Better `clone` ★**

One way to overcome the problem discussed in the previous Pitfall section is to place a bound on the type parameter `T` (in Display 15.12) so that it must satisfy some suitable interface. There is no standard interface that does the job, but it is very easy to define such an interface. The interface `PubliclyCloneable` given in Display 15.13 is just the interface we need. This short, simple interface guarantees all that we need to define generic linked lists whose `clone` method returns a deep copy.

Note that any class that implements the `PubliclyCloneable` interface has the following three properties:

1. The class implements the `Cloneable` interface. (This happens automatically because `PubliclyCloneable` extends `Cloneable`.)
2. The class has a public `clone` method.
3. The `clone` method for the class makes a deep copy (in the officially sanctioned way).

Condition 3 is not enforced by the Java compiler or run-time software, but like all interface semantics, it is the responsibility of the programmer defining the class to ensure that condition 3 is satisfied.

**TIP: (continued)**

It is now easy to define our generic linked list whose clone method produces a deep copy. The definition is given in Display 15.14. We have already discussed the main points involved in this definition. The Programming Example subsection, “A Linked List with a Deep Copy clone Method ★,” discusses some of the minor, but possibly unclear, details of the definition.¹ ■

Display 15.13 The PubliclyCloneable Interface

```

1  /*
2   The programmer who defines a class implementing this interface
3   has the responsibility to define clone so it makes a deep copy
4   (in the officially sectioned way).
5  */

6  public interface PubliclyCloneable extends Cloneable
7  {
8      public Object clone( );
9  }
```

Any class that implements PubliclyCloneable must have a public clone method.

Any class that implements PubliclyCloneable automatically implements Cloneable.

Display 15.14 A Generic Linked List with a Deep Copy clone Method (part 1 of 3)

```

1  public class LinkedList<T extends PubliclyCloneable>
2      implements PubliclyCloneable
3  {
4      private class Node<T>
5      {
6          private T data;
7          private Node<T> link;

8          public Node( )
9          {
10             data = null;
11             link = null;
12         }

```

(continued)

¹You might wonder whether we could use a type parameter in the PubliclyCloneable interface and so avoid some type casts in the definition copyOf. We could do that, but that may be more trouble than it is worth and, at this introductory level of presentation, would be an unnecessary distraction.

Display 15.14 A Generic Linked List with a Deep Copy clone Method (part 2 of 3)

```

13         public Node(T newData, Node<T> linkValue)
14         {
15             data = newData;
16             link = linkValue;
17         }
18     } //End of Node<T> inner class

19     private Node<T> head;

20     public LinkedList( )
21     {
22         head = null;
23     }
24     /**
25      * Produces a new linked list, but it is not a true deep copy.
26      * Throws a NullPointerException if other is null.
27      */
28     public LinkedList(LinkedList<T> otherList)
29     {
30         if (otherList == null)
31             throw new NullPointerException( );
32         if (otherList.head == null)
33             head = null;
34         else
35             head = copyOf(otherList.head);
36     }

37     public LinkedList<T> clone( )
38     {
39         try
40         {
41             {
42                 LinkedList<T> copy =
43                     (LinkedList<T>)super.clone( );
44                 if (head == null)
45                     copy.head = null;
46                 else
47                     copy.head = copyOf(head);
48                 return copy;
49             }
50             catch(CloneNotSupportedException e)
51             { //This should not happen.
52                 return null; //To keep the compiler happy.
53             }
54         }

55     /*
56      * Precondition: otherHead != null
57      * Returns a reference to the head of a copy of the list
58      * headed by otherHead. Returns a true deep copy.
59      */
60     private Node<T> copyOf(Node<T> otherHead)

```

Display 15.14 A Generic Linked List with a Deep Copy clone Method (part 3 of 3)

```

61     {
62         Node<T> position = otherHead; //moves down other's list.
63         Node<T> newHead; //will point to head of the copy list.
64         Node<T> end = null; //positioned at end of new growing list.
65         //Create first node:
66         newHead =
67             new Node<T>((T) (position.data).clone( ), null);
68         end = newHead;
69         position = position.link;
70         while (position != null)
71             //copy node at position to end of new list.
72             end.link =
73                 new Node<T>((T) (position.data).clone( ), null);
74             end = end.link;
75             position = position.link;
76         }

77     return newHead;
78 }
79
80 public boolean equals(Object otherObject)
81 {
82     if (otherObject == null)
83         return false;
84     else if (getClass( ) != otherObject.getClass( ))
85         return false;
86     else
87     {
88         LinkedList<T> otherList = (LinkedList<T>)otherObject;

<The rest of the definition is the same as in Display 15.8. The only difference
between this definition of equals and the one in Display 15.8 is that we
have replaced the class name LinkedList3<T> with LinkedList<T>.>

89     }

<All the other methods from Display 15.8 are in the class definition,
but are not repeated in this display.>

90     public String toString( )
91     {
92         Node<T> position = head;
93         String theString = "";
94         while (position != null)
95         {
96             theString = theString + position.data + "\n";
97             position = position.link;
98         }
99         return theString;
100     }
101 }

```

This definition of copyOf gives a deep copy of the linked list.

We added a toString method so LinkedList<T> would have all the properties we want T to have.

EXAMPLE: A Linked List with a Deep Copy `clone` Method ★

We have already discussed how and why the `clone` method of the generic linked list class in Display 15.14 returns a deep copy. Let us now look at some of the other details and see an example of using this linked list class.

Note the definition of the `clone` method. Why did we not simplify it to the following?

```
public LinkedList<T> clone( )
{
    return new LinkedList<T>(this);
}
```

This simple, alternative definition would still return a deep copy of the linked list and would work fine in most situations. It is likely that you would not notice any difference if you used this definition of `clone` in place of the one given in Display 15.14.

The only reason for all the other detail in the `clone` method definition given in Display 15.14 is to define the `clone` method as specified in the Java documentation. The reason that the Java documentation asks for those details has to do with security issues. (Some might say that there are three ways to define a `clone` method: the right way, the wrong way, and the Java way. This extra detail is the Java way.)

If you look only quickly at Display 15.14, you might think the following at the start of the definition is an unimportant detail:

```
implements PubliclyCloneable
```

However, it ensures that the linked list class implements the `Cloneable` interface. In order for a class to have a Java-approved `clone` method, it must implement the `Cloneable` interface. It also allows you to make linked lists of linked lists and have a deep copy `clone` method in the linked list of linked lists.

A sample class that implements the `PubliclyCloneable` interface is given in Display 15.15. Display 15.16 shows a demonstration program that makes a deep copy `clone` of a linked list of objects of this sample class.

Display 15.15 A `PubliclyCloneable` Class (part 1 of 2)

```
1 public class StockItem implements PubliclyCloneable
2 {
3     private String name;
4     private int number;
5
6     public StockItem( )
7     {
8         name = null;
9         number = 0;
10    }
```

Display 15.15 A PubliclyCloneable Class (part 2 of 2)

```
10     public StockItem(String nameData, int numberData)
11     {
12         name = nameData;
13         number = numberData;
14     }

15     public void setNumber(int newNumber)
16     {
17         number = newNumber;
18     }
19     public void setName(String newName)
20     {
21         name = newName;
22     }

23     public String toString( )
24     {
25         return (name + " " + number);
26     }

27     public Object clone( )
28     {
29         try
30         {
31             return super.clone( );
32         }
33         catch(CloneNotSupportedException e)
34         { //This should not happen.
35             return null; //To keep compiler happy.
36         }
37     }

38

39     public boolean equals(Object otherObject)
40     {
41         if (otherObject == null)
42             return false;
43         else if (getClass( ) != otherObject.getClass( ))
44             return false;
45         else
46         {
47             StockItem otherItem = (StockItem) otherObject;
48             return (name.equalsIgnoreCase(otherItem.name)
49                 && number == otherItem.number);
50         }
51     }
52 }
```

Display 15.16 Demonstration of Deep Copy clone

```
1  public class DeepDemo
2  {
3      public static void main(String[] args)
4      {
5          LinkedList<StockItem> originalList =
6              new LinkedList<StockItem>( );
7          originalList.addToStart(new StockItem("red dress", 1));
8          originalList.addToStart(new StockItem("black shoe", 2));
9
10         LinkedList<StockItem> copyList = originalList.clone( );
11         if (originalList.equals(copyList))
12             System.out.println("OK, Lists are equal.");
13
14         System.out.println("Now we change copyList.");
15         StockItem dataEntry =
16             copyList.findData(new StockItem("red dress", 1));
17         dataEntry.setName("orange pants");
18
19         System.out.println("originalList:");
20         originalList.outputList( );
21
22         System.out.println("copyList:");
23         copyList.outputList( );
24
25         System.out.println("Only one list is changed.");
26     }
27 }
```

Sample Dialogue

```
OK, Lists are equal.
Now we change copyList.
originalList:
black shoe 2
red dress 1
copyList:
black shoe 2
orange pants 1
Only one list is changed.
```



TIP: Cloning Is an “All or Nothing” Affair

If you define a `clone` method, then you should do so following the official Java guidelines, as we did in Display 15.14. In particular, you should always have the class implement the `Cloneable` interface. If you define a `clone` method in any other way, you may encounter problems in some situations. If you want to have a method for producing copies of objects but do not want to follow the official guidelines on how to define a `clone` method, then use some other name for your “clone-like” method, such as `copier`, or make do with just a copy constructor. ■

Self-Test Exercises

9. In the definition of `copyOf` in Display 15.14, can we replace

```
newHead =
    new Node<T>((T) (position.data).clone(), null);
```

with the following, which uses the copy constructor of `T` instead of the `clone` method of `T`?

```
newHead =
    new Node<T>(new T(position.data), null);
```

10. The definition of the `clone` method in Display 15.14 returns a value of type `LinkedList<T>`. But the class being defined implements the `PubliclyCloneable` interface, and that interface says the value returned must be of type `Object`. Is something wrong?

15.3 Iterators

Play it again, Sam.

Attributed (incorrectly) to the movie *Casablanca*, which contains similar lines.²

When you have a collection of objects, such as the nodes of a linked list, you often need to step through all the objects in the collection one at a time and perform some action on each object, such as writing it out to the screen or in some way editing the data in each object. An **iterator** is any object that allows you to step through the list in this way.

iterator

²There is a Woody Allen movie with this title, but it is based on the misquote from *Casablanca*, which was in common use before the movie came out.

Defining an Iterator Class

In Display 15.17, we have rewritten the class `LinkedList2` from Display 15.7 so that it has an inner class for iterators and a method `iterator()` that returns an iterator for its calling object. We have made the inner class `List2Iterator` public so that we can have variables of type `List2Iterator` outside the class `LinkedList2`, but we do not otherwise plan to use the inner class `List2Iterator` outside of the outer class `LinkedList2`.

Use of iterators for the class `LinkedList2` is illustrated by the program in Display 15.18. Note that, given a linked list named `list`, an iterator for `list` is produced by the method `iterator` as follows:

```
LinkedList2.List2Iterator i = list.iterator();
```

The iterator `i` produced in this way can only be used with the linked list named `list`. Be sure to notice that outside of the class, the type name for the inner class iterator must include the name of the outer class as well as the inner iterator class. The class name for one of these iterators is

```
LinkedList2.List2Iterator
```

Display 15.17 A Linked List with an Iterator (part 1 of 3)

```

1  import java.util.NoSuchElementException;

2  public class LinkedList2
3  {
4      private class Node
5      {
6          private String item;
7          private Node link;

      <The rest of the definition of the Node inner class is given in Display 15.7.>

8          } //End of Node inner class

9      /**
10       * If the list is altered any iterators should invoke restart or
11       * the iterator's behavior may not be as desired.
12       */
13     public class List2Iterator
14     {
15         private Node position;
16         private Node previous; //previous value of position
17         public List2Iterator()
18         {
19             position = head; //Instance variable head of outer class.
20             previous = null;
21         }

```

This is the same as the class in Displays 15.7 and 15.11 except that the `List2Iterator` inner class and the `iterator()` method have been added.

An inner class for iterators for `LinkedList2`

Display 15.17 A Linked List with an Iterator (part 2 of 3)

```

22     public void restart( )
23     {
24         position = head; //Instance variable head of outer class.
25         previous = null;
26     }

27     public String next( )
28     {
29         if (!hasNext( ))
30             throw new NoSuchElementException( );

31         String toReturn = position.item;
32         previous = position;
33         position = position.link;
34         return toReturn;
35     }

36     public boolean hasNext( )
37     {
38         return (position != null);
39     }
40     /**
41     Returns the next value to be returned by next( ).
42     Throws an IllegalStateException if hasNext( ) is false.
43     */
44     public String peek( )
45     {
46         if (!hasNext( ))
47             throw new IllegalStateException( );
48         return position.item;
49     }

50     /**
51     Adds a node before the node at location position.
52     previous is placed at the new node. If hasNext( ) is
53     false, then the node is added to the end of the list.
54     If the list is empty, inserts node as the only node.
55     */
56     public void addHere(String newData)
57     {
58         if (position == null && previous != null)
59             // at end of the list, add to end
60             previous.link = new Node(newData, null);
61         else if (position == null || previous == null)
62             // list is empty or position is head node
63             LinkedList2.this.addToStart(newData);

```

(continued)

Display 15.17 A Linked List with an Iterator (part 3 of 3)

```

64         else
65         { // previous and position are consecutive nodes
66             Node temp = new Node(newData, position)
67             previous.link = temp;
68             previous = temp;
69         }
70     }
71     /**
72     Changes the String in the node at location position.
73     Throws an IllegalStateException if position is not at a node,
74     */
75     public void changeHere(String newData)

```

< Self-Test Exercise 13 asks you to complete the rest of the method changeHere.>

```

76     /**
77     Deletes the node at location position and
78     moves position to the "next" node.
79     Throws an IllegalStateException if the list is empty.
80     */
81     public void delete( )
82     {
83         if (position == null)
84             throw new IllegalStateException( );
85         else if (previous == null)
86         { // remove node at head
87             head = head.link;
88             position = head;
89         }
90         else // previous and position are consecutive nodes
91         {
92             previous.link = position.link;
93             position = position.link;
94         }
95     }
96     private Node head;

```

*If list is an object of the class
LinkedList2, then
list.iterator() returns an
iterator for list.*

```

97     public List2Iterator iterator( )
98     {
99         return new List2Iterator( );
100    }

```

<The other methods and constructors are identical to those in Displays 15.7 and 15.11.>

```

101 }

```

Display 15.18 Using an Iterator (part 1 of 2)

```
1 public class IteratorDemo
2 {
3     public static void main(String[] args)
4     {
5         LinkedList2 list = new LinkedList2( );
6         LinkedList2.List2Iterator i = list.iterator( );
7
8         list.addToStart("shoes");
9         list.addToStart("orange juice");
10        list.addToStart("coat");
11
12        System.out.println("List contains:");
13        i.restart( );
14        while(i.hasNext( ))
15            System.out.println(i.next( ));
16        System.out.println( );
17
18        i.restart( );
19        i.next( );
20        System.out.println("Will delete the node for " + i.peek( ));
21        i.delete( );
22
23        System.out.println("List now contains:");
24        i.restart( );
25        while(i.hasNext( ))
26            System.out.println(i.next( ));
27        System.out.println( );
28
29        i.restart( );
30        i.next( );
31        System.out.println("Will add one node before " + i.peek( ));
32        i.addHere("socks");
33        System.out.println("List now contains:");
34        i.restart( );
35        while(i.hasNext( ))
36            System.out.println(i.next( ));
37        System.out.println( );
38        System.out.println("Changing all items to credit card.");
39        i.restart( );
40        while(i.hasNext( ))
41        {
42            i.changeHere("credit card");
43            i.next( );
44        }
45        System.out.println( );
46        System.out.println("List now contains:");
```

(continued)

Display 15.18 Using an Iterator (part 2 of 2)

```
42         i.restart( );
43         while(i.hasNext( ))
44             System.out.println(i.next( ));
45         System.out.println( );
46     }
47 }
```

Sample Dialogue

```
List contains:
coat
orange juice
shoes

Will delete the node for orange juice
List now contains:
coat
shoes

Will add one node before shoes
List now contains:
coat
socks
shoes

Changing all items to credit card.

List now contains:
credit card
credit card
credit card
```

The basic method for cycling through the elements in the linked list using an iterator is illustrated by the following code from the demonstration program:

```
System.out.println("List now contains:");
i.restart( );
while(i.hasNext( ))
    System.out.println(i.next( ));
```

The iterator is named *i* in this code. The iterator *i* is reset to the beginning of the list with the method invocation *i.restart()*, and each execution of *i.next()* produces the next data item in the linked list. After all the data items in all the nodes have been returned by *i.next()*, the Boolean *i.hasNext()* becomes false, and the while loop ends.

Internally, the local variable `position` references the current node in the linked list, whereas the local variable `previous` references the node linking to the current node. The purpose of the `previous` variable will be seen when adding and deleting nodes. In the constructor and the `restart()` method, `position` is set to `head` and `previous` is set to `null`.

To determine if the end of the list has been reached, `hasNext()` returns whether or not `position` is null:

```
return (position != null);
```

To step through the list, the `next()` method first throws an exception if we have reached the end of the list:

```
if (!hasNext())  
    throw new NoSuchElementException();
```

Otherwise, the method retrieves the string value of the iterator referenced by `position` in the variable `toReturn`, advances `previous` to reference the current `position`, advances `position` to the next node in the list, and returns the string:

```
String toReturn = position.item;  
previous = position;  
position = position.link;  
return toReturn;
```

The definition of the method `changeHere` is left to Self-Test Exercise 13. (If necessary, you can look up the definition in the answer to Self-Test Exercise 13.) The techniques for adding and deleting nodes are discussed in the next subsection.

The Java Iterator Interface

Java has an interface named `Iterator` that specifies how Java would like an iterator to behave. It is in the package `java.util` (and so requires that you import this package). Our iterators do not quite satisfy this interface, but they are in the same general spirit as that interface and could be easily redefined to satisfy the `Iterator` interface.

The `Iterator` interface is discussed in Chapter 16.

Adding and Deleting Nodes

To add or delete a node in a linked list, you normally use an iterator and add or delete a node at the (approximate) location of the iterator. Because deleting is a little easier than adding a node, we will discuss deleting first.

Display 15.19 shows the technique for deleting a node. The linked list is an object of the class `LinkedList2` (Display 15.17). The variables `position` and `previous` are the instance variables of an iterator for the linked list object. These variables each hold a reference to a node, indicated with an arrow. Each time `next( )` is invoked, `previous` and `position` reference subsequent nodes in the list. As indicated in Display 15.19, the node at location `position` is deleted by the following two lines of code:

```
previous.link = position.link;
position = position.link;
```

In Display 15.19, `next( )` has been invoked twice, so `position` is referencing the node with "shoes" and `previous` is referencing the node with "socks".

To delete the node referenced by `position`, the link from the `previous` node is set to `position`'s link. As shown in Display 15.19, this removes the linked list's reference to that node. The variable `position` is then set to the next node in the list to remove any references to the deleted node. As far as the linked list is concerned, the old node is no longer on the linked list. But the node is still in the computer's memory. If there are no longer any references to the deleted node, then the storage that it occupies should be made available for other uses. In many programming languages, you, the programmer, must keep track of items such as deleted nodes and must give explicit commands to return their memory for recycling. This is called **garbage collecting** or **explicit memory management**. In Java, this is done for you automatically, or, as it is ordinarily phrased, Java has automatic garbage collection.

Note that there are special cases that must be handled for deletion. First, if the list is empty, then nothing can be deleted and the `delete( )` method throws an exception. Second, if the node to delete is the head of the list, then there is no previous node to update. Instead, `head` is set to `head.link` to bypass the first node in the list and set a new head node.

Display 15.20 shows the technique for adding a node. We want to add a new node between the nodes named by `previous` and `position`. In Display 15.20, `previous` and `position` are variables of type `Node`, and each contains a reference to a node indicated with an arrow. Thus, the new node goes between the two nodes referenced by `previous` and `position`. In Display 15.20, the method `next( )` has been invoked twice to advance `previous` to "orange juice" and `position` to "shoes".

A constructor for the class `Node` does a lot of the work for us: It creates the new node, adds the data, and sets the link field of the new node to reference the node named by `position`. All this is done with the following:

```
new Node(newData, position)
```

So that we can recognize the node with `newData` in it when we study Display 15.20, let us assume that `newData` holds the string "socks". The following gets us from the first to the second picture:

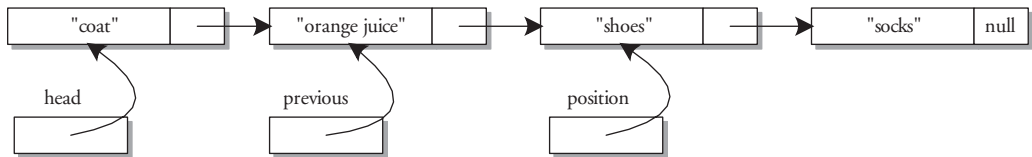
```
temp = new Node(newData, position);
```

garbage
collecting

explicit
memory
management

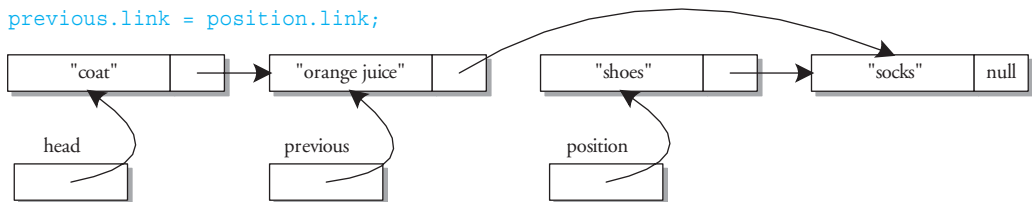
Display 15.19 Deleting a Node

1. Existing list with the iterator positioned at “shoes”



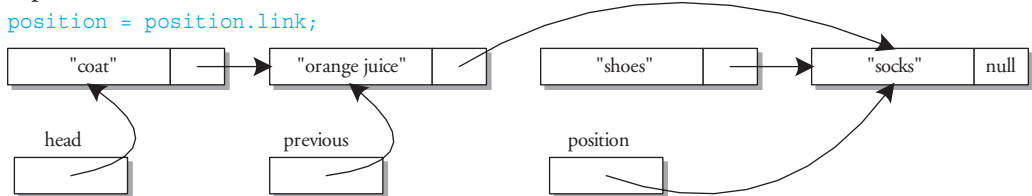
2. Bypass the node at position from previous

```
previous.link = position.link;
```



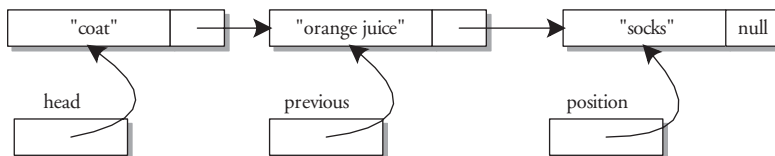
3. Update position to reference the next node

```
position = position.link;
```



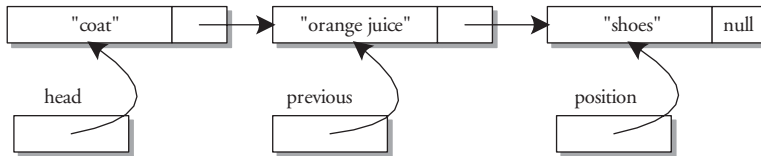
Since no variable references the node “shoes”, Java will automatically recycle the memory allocated for it.

4. Same picture with deleted node not shown



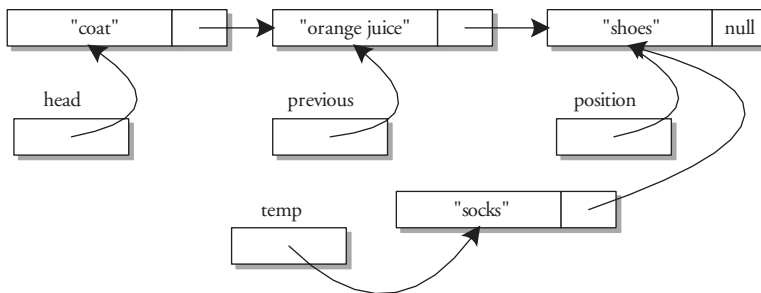
Display 15.20 Adding a Node between Two Nodes

- Existing list with the iterator positioned at “shoes”



- Create new Node with “socks” linked to “shoes”

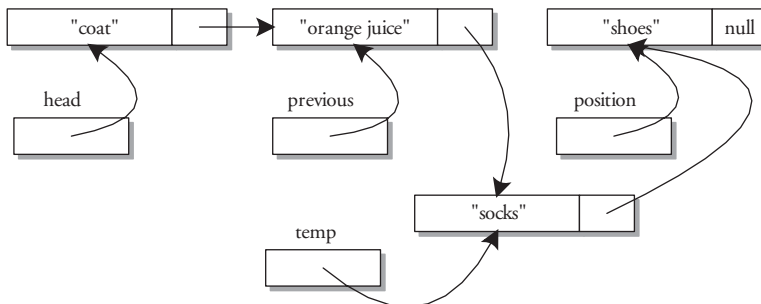
```
temp = new Node(newData, position); // newData is "socks"
```



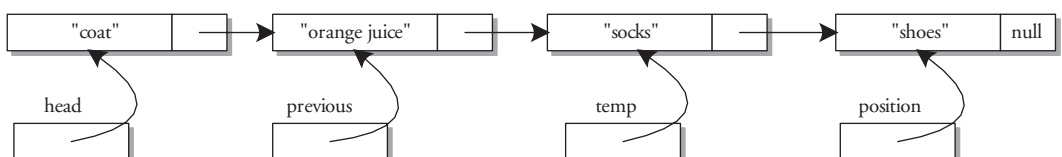
Local variable of type Node

- Make previous link to the Node temp

```
previous.link = temp;
```



- Picture redrawn for clarity, but structurally identical to picture 3



To finish the job, all we need to do is link the previous node to the new node. We want to move the arrow to the node named by `temp`. The following finishes our job:

```
previous.link = temp;
```

The new node is inserted in the desired place, but the picture is not too clear. The fourth picture is the same as the third one; we have simply redrawn it to make it neater.

To summarize, the following two lines insert a new node with `newData` as its data. The new node is inserted between the nodes named by `previous` and `position`.

```
temp = new Node(newData, position);  
previous.link = temp;
```

`previous`, `position`, and `temp` are all variables of type `Node`. (When we use this code, `previous` and `position` will be instance variables of an iterator and `temp` will be a local variable.)

Just like deletion, special cases exist for insertion that must be handled. If the list is empty, then addition is done by adding to the front of the list. If the `position` variable is null, then the new node should be added to the end of the list.

Self-Test Exercises

11. Consider a variant of the class in Display 15.17 with no `previous` local variable. In other words, there is no reference kept to the node that links to the current node `position`. How could we modify the `delete` method to delete the `position` node and still maintain a correct list? The solution is less efficient than the version that uses `previous`.
12. Consider a variant of the class in Display 15.17 with no `previous` local variable. In other words, there is no reference kept to the node that links to the current node `position`. Write a method `addAfterHere(String newData)` that adds a new node after the node in `position`.
13. Complete the definition of the method `changeHere` in the inner class `List2Iterator` in Display 15.17.
14. Given an iterator pointing somewhere in a linked list, does `i.next()` return the value that `i` is referencing prior to the invocation of `i.next()` or does it return the value of the next node in the list?

15.4 Variations on a Linked List

I have called this principle, by which each slight variation, if useful, is preserved, by the term Natural Selection.

CHARLES DARWIN, *On the Origin of Species*, UK, 1859.

In this section, we discuss some variations on linked lists, including the two data structures known as stacks and queues. Stacks and queues need not involve linked lists, but one common way to implement a stack or a queue is to use a linked list.

Doubly Linked List

doubly linked list

An ordinary linked list allows you to move down the list in one direction only (following the links). A **doubly linked list** has one link that has a reference to the next node and one that has a reference to the previous node. In some cases, the link to the previous node can simplify our code. For example, we will no longer need to have a previous instance variable to remember the node that links to the current position. Diagrammatically, a doubly linked list looks like the sample list in Display 15.21.

The node class for a doubly linked list can begin as follows:

```
private class TwoWayNode
{
    private String item;
    private TwoWayNode previous;
    private TwoWayNode next;
    ...
}
```

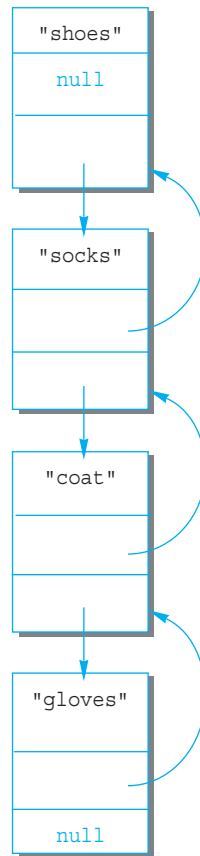
The constructors and some of the methods in the doubly linked list class will require changes (from the singly linked case) in their definitions to accommodate the extra link. The major changes are to the methods that add and delete nodes. To make our code a little cleaner, we can add a new constructor that sets the previous and next nodes:

```
public TwoWayNode(String newItem, TwoWayNode previousNode,
                    TwoWayNode nextNode)
{
    item = newItem;
    next = nextNode;
    previous = previousNode;
}
```

To add a new `TwoWayNode` to the front of the list requires setting links on two nodes instead of one. The general process is shown in Display 15.22. In the `addToStart` method, we first create a new `TwoWayNode`. Because the new node will go on the front of the list, we set the previous link to `null` and the `next` link to the current head:

```
TwoWayNode newHead = new TwoWayNode(itemName, null, head);
```

Display 15.21 A Doubly Linked List



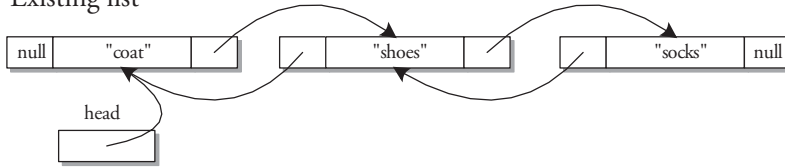
Next, we must set the previous link on the old head node to reference the new head. We can do this by setting `head.previous = newHead`, but we must take care to ensure that `head` is not `null` (i.e., the list is not empty). Finally, we can set `head` to `newHead`:

```
if (head != null)
{
    head.previous = newHead;
}
head = newHead;
```

To delete a node from the doubly linked list also requires updating the references on both sides of the node to delete. Thanks to the backward link, there is no need for an instance variable to keep track of the previous node in the list, as was required for the singly linked list. The general process of deleting a node referenced by position is shown in Display 15.23. Note that some cases must be handled separately, such as deleting a node from the beginning or the end of the list.

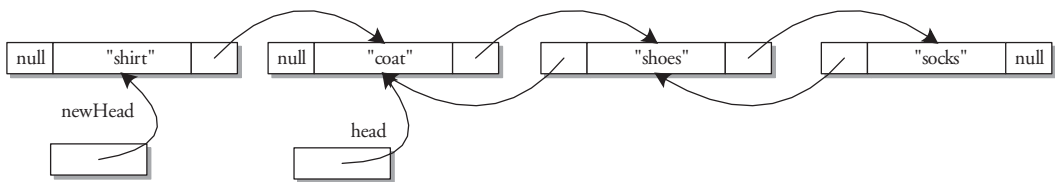
Display 15.22 Adding a Node to the Front of a Doubly Linked List

1. Existing list



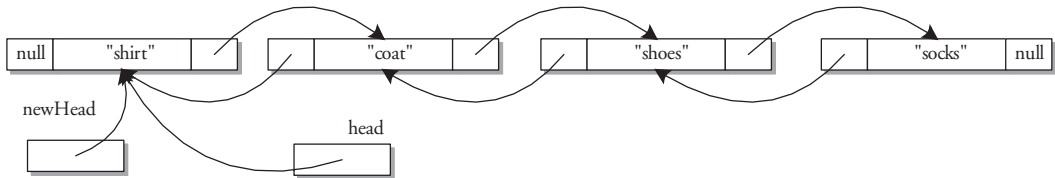
2. Create new TwoWayNode linked to “coat”

```
TwoWayNode newHead = new TwoWayNode(itemName, null, head) // itemName = "shirt"
```



3. Set backward link and set new head

```
head.previous = newHead;  
head = newHead;
```

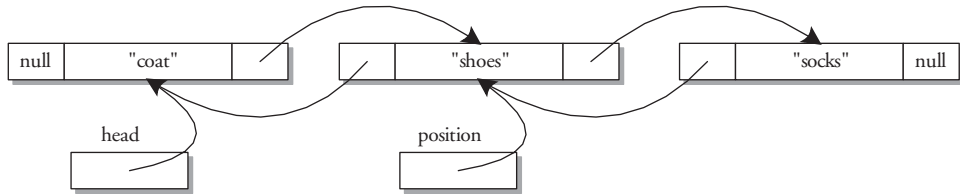


The process of inserting a new node into the doubly linked list is shown in Display 15.24. In this case, we will insert the new node in front of the iterator referenced by `position`. Note that there are also special cases for the insert routine when inserting to the front or adding to the end of the list. Only the general case of inserting between two existing nodes is shown in Display 15.24.

A complete example of a doubly linked list is shown in Display 15.25. The code in Display 15.25 is modified from the code in Display 15.17. Use of the doubly linked list is virtually identical to use of a singly linked list. Display 15.26 demonstrates addition, deletion, and insertion into the doubly linked list.

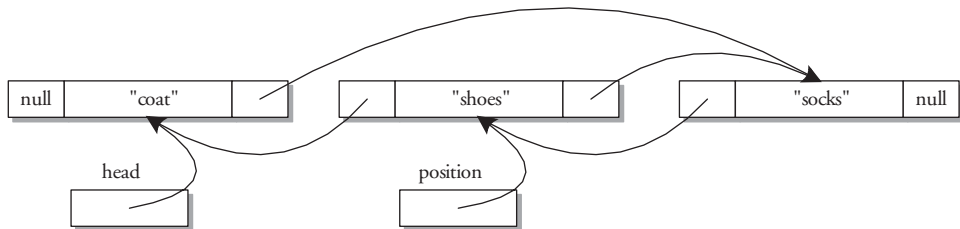
Display 15.23 Deleting a Node from a Doubly Linked List

1. Existing list with an iterator referencing “shoes”



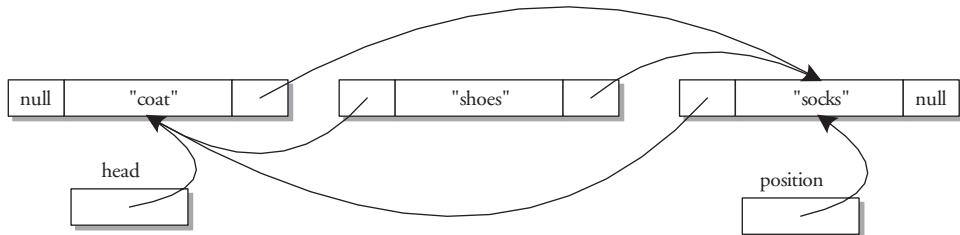
2. Bypass the “shoes” node from the next link of the previous node

```
position.previous.next = position.next;
```

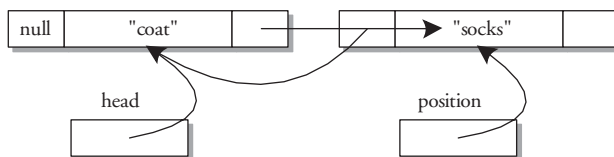


3. Bypass the “shoes” node from the previous link of the next node and move position off the deleted node

```
position.next.previous = position.previous;
position = position.next;
```

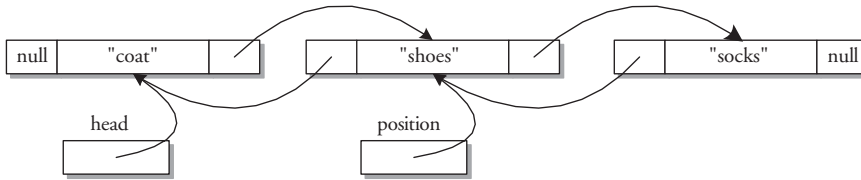


4. Picture redrawn for clarity with the “shoes” node removed since there are no longer references pointing to this node



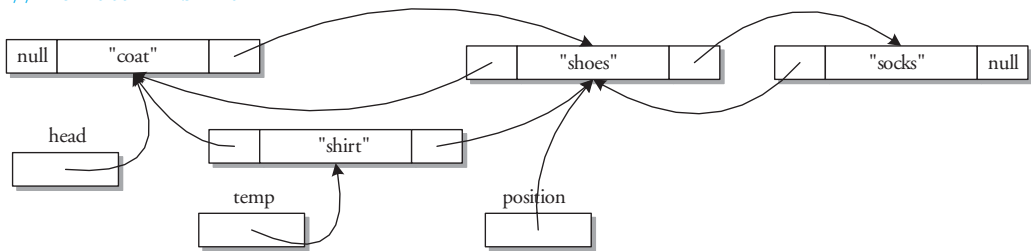
Display 15.24 Inserting a Node into a Doubly Linked List

1. Existing list with an iterator referencing “shoes”



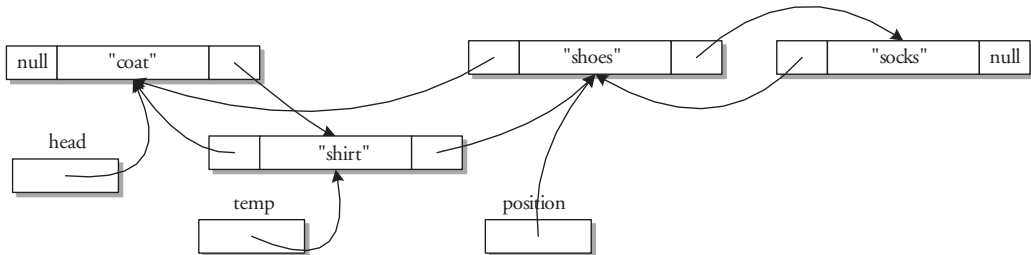
2. Create new TwoWayNode with previous linked to “coat” and next to “shoes”

```
TwoWayNode temp = newTwoWayNode(newData, position.previous, position);
// newData = "shirt"
```



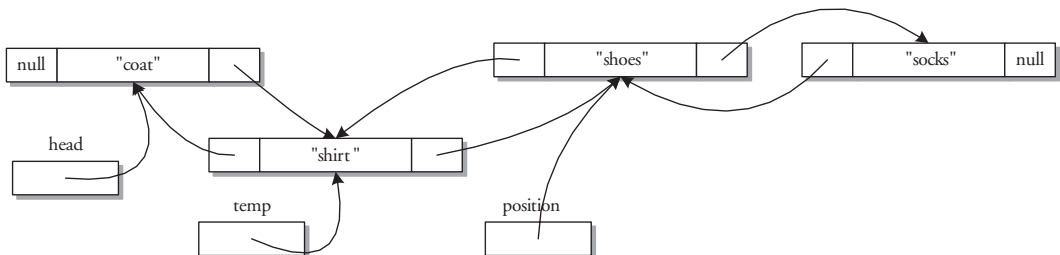
3. Set next link from “coat” to the new node of “shirt”

```
position.previous.next = temp;
```



4. Set previous link from “shoes” to the new node of “shirt”

```
position.previous = temp;
```



Display 15.25 A Doubly Linked List with an Iterator (part 1 of 3)

```

1  import java.util.NoSuchElementException;

2  public class DoublyLinkedList
3  {
4      private class TwoWayNode
5      {
6          private String item;
7          private TwoWayNode previous;
8          private TwoWayNode next;

9          public TwoWayNode()
10         {
11             item = null;
12             next = null;
13             previous = null;
14         }
15         public TwoWayNode(String newItem, TwoWayNode previousNode,
16                               TwoWayNode nextNode)
17         {
18             item = newItem;
19             next = nextNode;
20             previous = previousNode;
21         }
22     } //End of TwoWayNode inner class

23     public class DoublyLinkedListIterator
24     {
25         // We do not need a previous node when using a doubly linked
26         // list
27         private TwoWayNode position = null;

28         public DoublyLinkedListIterator( )
29         {
30             position = head;
31         }
32         public void restart( )
33         {
34             position = head;
35         }
36         public String next( )
37         {
38             if (!hasNext( ))
39                 throw new IllegalStateException( );
40             String toReturn = position.item;
41             position = position.next;
42             return toReturn;
43         }
44     }

```

(continued)

Display 15.25 A Doubly Linked List with an Iterator (part 2 of 3)

```
42     public void insertHere(String newData)
43     {
44         if (position == null && head != null)
45         {
46             // Add to end. First move a temp
47             // pointer to the end of the list
48             TwoWayNode temp = head;
49             while (temp.next != null)
50                 temp = temp.next;
51             temp.next = new TwoWayNode(newData, temp, null);
52         }
53         else if (head == null || position.previous == null)
54             // at head of list
55             DoublyLinkedList.this.addToStart (newData);
56         else
57         {
58             // Insert before the current position
59             TwoWayNode temp = new TwoWayNode(newData,
60                 position.previous, position);
61             position.previous.next = temp;
62             position.previous = temp;
63         }
64     }
65
66     public void delete( )
67     {
68         if (position == null)
69             throw new IllegalStateException( );
70         else if (position.previous == null)
71         { // Deleting first node
72             head = head.next;
73             position = head;
74         }
75         else if (position.next == null)
76         { // Deleting last node
77             position.previous.next = null;
78             position = null;
79         }
80         else
81         {
82             position.previous.next = position.next;
83             position.next.previous = position.previous;
84             position = position.next;
85         }
86     }
87 } // DoublyLinkedListIterator
```

Display 15.25 A Doubly Linked List with an Iterator (part 3 of 3)

```

86     private TwoWayNode head;

87     public DoublyLinkedListIterator iterator( )
88     {
89         return new DoublyLinkedListIterator( );
90     }

91     public DoublyLinkedList( )
92     {
93         head = null;
94     }

95     /**
96      * The added node will be the first node in the list.
97      */
98     public void addToStart(String itemName)
99     {
100         TwoWayNode newHead = new TwoWayNode(itemName, null, head);
101         if (head != null)
102         {
103             head.previous = newHead;
104         }
105         head = newHead;
106     }

```

<The methods hasNext, peek, clear, and isEmpty are identical to those in Display 15.17. Other methods would also normally be defined here, such as deleteHeadNode, size, outputList, equals, clone, find, or contains. They have been left off to simplify the example.>

```

107 } // DoublyLinkedList

```

Display 15.26 Using a Doubly Linked List with an Iterator (part 1 of 2)

```

1  public class DoublyLinkedListDemo
2  {
3      public static void main(String[] args)
4      {
5          DoublyLinkedList list = new DoublyLinkedList( );
6          DoublyLinkedList.DoublyLinkedListIterator i = list.iterator( );

7          list.addToStart("shoes");
8          list.addToStart("orange juice");
9          list.addToStart("coat");
10         System.out.println("List contains:");
11         i.restart( );

```

(continued)

Display 15.26 Using a Doubly Linked List with an Iterator (part 2 of 2)

```
12         while (i.hasNext( ))
13             System.out.println(i.next( ));
14         System.out.println( );

15         i.restart( );
16         i.next( );
17         i.next( );
18         System.out.println("Delete " + i.peek( ));
19         i.delete( );

20         System.out.println("List now contains:");
21         i.restart( );
22         while (i.hasNext( ))
23             System.out.println(i.next( ));
24         System.out.println( );

25         i.restart( );
26         i.next( );
27         System.out.println("Inserting socks before " + i.peek( ));
28         i.insertHere("socks");

29         i.restart( );
30         System.out.println("List now contains:");
31         while (i.hasNext( ))
32             System.out.println(i.next( ));
33         System.out.println( );
34     }
35 }
```

Sample Dialogue

```
List contains:
coat
orange juice
shoes

Delete shoes
List now contains:
Coat
Orange juice

Inserting socks before orange juice
List now contains:
coat
socks
orange juice
```

Self-Test Exercises

15. What operations are easier to implement with a doubly linked list compared with a singly linked list? What operations are more difficult?
16. If the `addToStart` method from Display 15.25 were removed, how could we still add a new node to the head of the list?

The Stack Data Structure

stack

A **stack** is not necessarily a linked data structure, but it can be implemented as a linked list. A stack is a data structure that removes items in the reverse of the order in which they were inserted. So if you insert “one”, then “two”, and then “three” into a stack and then remove them, they will come out in the order “three”, then “two”, and finally “one”. Stacks are discussed in more detail in Chapter 11. A linked list that inserts and deletes only at the head of the list (such as those in Displays 15.3 and 15.8) is, in fact, a stack.

push and pop

You can imagine the stack data structure like a stack of trays in a cafeteria. You can **push** a new tray on top of the stack to make a taller stack. Alternately, you can **pop** the topmost tray off the stack until there are no more trays to remove. A definition of a `Stack` class is shown in Display 15.27 that is based on the linked list from Display 15.3. A short demonstration program is shown in Display 15.28. The `addToStart` method has been renamed to `push` to use stack terminology. Similarly, the `deleteHeadNode` method has been renamed to `pop` and returns the `String` from the top of the stack. Although not shown here to keep the definition simple, it would be appropriate to add other methods such as `peek`, `clone`, or `equals` or to convert the class to use a generic data type.

Stacks

A **stack** is a last-in/first-out data structure; that is, the data items are retrieved in the opposite order to which they were placed in the stack.

Display 15.27 A `Stack` Class (part 1 of 2)

```

1  import java.util.NoSuchElementException;

2  public class Stack
3  {
4      private class Node
5      {
6          private String item;
7          private Node link;

```

(continued)

Display 15.27 A `Stack` Class (part 2 of 2)

```
8         public Node( )
9         {
10             item = null;
11             link = null;
12         }
13         public Node(String newItem, Node linkValue)
14         {
15             item = newItem;
16             link = linkValue;
17         }
18     } //End of Node inner class

19     private Node head;

20     public Stack( )
21     {
22         head = null;
23     }

24     /**
25      * This method replaces addToStart
26      */
27     public void push(String itemName)
28     {
29         head = new Node(itemName, head);
30     }
31     /**
32      * This method replaces deleteHeadNode and
33      * also returns the value popped from the list
34      */
35     public String pop( )
36     {
37         if (head == null)
38             throw new IllegalStateException( );
39         else
40         {
41             String returnItem = head.item;
42             head = head.link;
43             return returnItem;
44         }
45     }
46     public boolean isEmpty( )
47     {
48         return (head == null);
49     }
50 }
```

Display 15.28 Stack Demonstration Program

```

1  public class StackExample
2  {
3      public static void main(String[] args)
4      {
5          Stack stack = new Stack( );
6          Stack.push("Billy Rubin");
7          Stack.push("Lou Pole");
8          Stack.push("Polly Ester");

9          while (!stack.isEmpty( ))
10         {
11             String s = stack.pop( );
12             System.out.println(s);
13         }
14     }
15 }

```

Items come out of the stack in the reverse order that they were added.

Sample Dialogue

```

Polly Ester
Lou Pole
Billy Rubin

```

Self-Test Exercise

17. Display 15.27 does not contain a `peek( )` method. Normally this method would return the data on the top of the stack without popping it off. How could a user of the `Stack` class get the same functionally as `peek( )` even though it is not defined?

The Queue Data Structure

queue

A stack is a last-in/first-out data structure. Another common data structure is a **queue**, which handles data in a first-in/first-out fashion. A queue is like a line at the bank. Customers add themselves to the back of the line and are served from the front of the line. A queue can be implemented with a linked list. However, a queue needs a pointer at both the head of the list and at the **tail** (that is, the other end) of the linked list, because action takes place in both locations. It is easier to remove a node from the head of a linked list than from the tail of the linked list. So, a simple implementation will remove nodes from the head of the list (which we will now call the **front** of the list) and we will add nodes to the tail end of the list, which we will now call the **back** of the list (or the back of the queue).

tail

front

back

The definition of a simple `Queue` class that is based on a linked list is given in Display 15.29. A short demonstration program is given in Display 15.30. We have not made our queue a generic queue to keep the definition simple, but it would be routine to replace the data type `String` with a type parameter.

Queue

A **queue** is a first-in/first-out data structure; that is, the data items are removed from the queue in the same order that they were added to the queue.

Display 15.29 A `Queue` Class (part 1 of 2)

```
1  public class Queue
2  {
3      private class Node
4      {
5          private String item;
6          private Node link;
7
8          public Node( )
9          {
10             item = null;
11             link = null;
12
13             public Node(String newItem, Node linkValue)
14             {
15                 item = newItem;
16                 link = linkValue;
17             } //End of Node inner class
18
19             private Node front;
20             private Node back;
21
22             public Queue( )
23             {
24                 front = null;
25                 back = null;
26             }
27         }
28     }
```

Display 15.29 A Queue Class (part 2 of 2)

```
25     /**
26      * Adds a String to the back of the queue.
27      */
28     public void addToBack(String itemName)
    <The definition of this method is defined in Self-Test Exercise 18.>

29     public boolean isEmpty( )
30     {
31         return (front == null);
32     }

33     public void clear( )
34     {
35         front = null;
36         back = null;
37     }
38     /**
39      * Returns the String in the front of the queue.
40      * Returns null if queue is empty.
41      */
42     public String whoIsNext( )
43     {
44         if (front == null)
45             return null;
46         else
47             return front.item;
48     }
49
50     /**
51      * Removes a String from the front of the queue.
52      * Returns false if the list is empty.
53      */
54     public boolean removeFront( )
55     {
56         if (front != null)
57         {
58             front = front.link;
59             return true;
60         }
61         else
62             return false;
63     }
64 }
```

Display 15.30 Demonstration of the Queue Class

```
1 public class QueueDemo
2 {
3     public static void main(String[] args)
4     {
5         Queue q = new Queue( );
6
7         q.addToBack("Tom");
8         q.addToBack("Dick");
9         q.addToBack("Harriet");
10        while(!q.isEmpty( ))
11        {
12            System.out.println(q.whoIsNext( ));
13            q.removeFront( );
14        }
15        System.out.println("The queue is empty.");
16    }
```

Items come out of the queue in the same order that they went into the queue.

Sample Dialogue

```
Tom
Dick
Harriet
The queue is empty.
```

Items come out of the queue in the same order that they went into the queue.

Self-Test Exercise

18. Complete the definition of the method `addToBack` in Display 15.29.

In order to have some terminology to discuss the efficiency of our `Queue` class and linked list algorithms, we first present some background on how the efficiency of algorithms is usually measured.

Running Times and Big-O Notation

If you ask a programmer how fast his or her program is, you might expect an answer such as “two seconds.” However, the speed of a program cannot be given by a single number. A program will typically take a longer amount of time on larger inputs than it will on smaller inputs. You would expect that a program for sorting numbers would take less time to sort 10 numbers than it would to sort 1,000 numbers. Perhaps it takes 2 seconds to sort 10 numbers, but 10 seconds to sort 1,000 numbers. How, then, should the programmer answer the question “How fast is your program?” The programmer would have to give a table of values showing how long the program took

Display 15.31 Some Values of a Running-Time Function

Input Size	Running Time
10 numbers	2 seconds
100 numbers	2.1 seconds
1,000 numbers	10 seconds
10,000 numbers	2.5 minutes

for different sizes of input. For example, the table might be as shown in Display 15.31. This table does not give a single time, but instead gives different times for a variety of different input sizes.

function

The table is a description of what is called a **function** in mathematics. Just as a (non-void) Java method takes an argument and returns a value, so too does this function take an argument, which is an input size, and returns a number, which is the time the program takes on an input of that size. If we call this function T , then $T(10)$ is 2 seconds, $T(100)$ is 2.1 seconds, $T(1,000)$ is 10 seconds, and $T(10,000)$ is 2.5 minutes. The table is just a sample of some of the values of this function T . The program will take some amount of time on inputs of every size. So although they are not shown in the table, there are also values for $T(1)$, $T(2)$, $\dots$, $T(101)$, $T(102)$, and so forth. For any positive integer N , $T(N)$ is the amount of time it takes for the program to sort N numbers. The function T is called the **running time** of the program.

running time

So far we have been assuming that this sorting program will take the same amount of time on any list of N numbers. That need not be true. Perhaps it takes much less time if the list is already sorted or almost sorted. In this case, $T(N)$ is defined to be the time taken by the “hardest” list—that is, the time taken on that list of N numbers that makes the program run the longest. This is called the **worst-case running time**. In this chapter, we will always mean worst-case running time when we give a running time for an algorithm or for some code.

worst-case running time

The time taken by a program or algorithm is often given by a formula, such as $4N + 3$, $5N + 4$, or N^2 . If the running time $T(N)$ is $5N + 5$, then on inputs of size N , the program will run for $5N + 5$ time units.

Presented next is some code to search an array a with N elements to determine whether a particular value `target` is in the array:

```
int i = 0;
boolean found = false;
while ((i < N) && !(found))
{
    if (a[i] == target)
        found = true;
    else
        i++;
}
```


We want to compute some estimate of how long it will take a computer to execute this code. We would like an estimate that does not depend on which computer we use, either because we do not know which computer we will use or because we might use several different computers to run the program at different times.

One possibility is to count the number of “steps,” but it is not easy to decide what a step is. In this situation, the normal thing to do is count the number of **operations**. The term *operations* is almost as vague as the term *step*, but there is at least some agreement in practice about what qualifies as an operation. Let us say that, for this Java code, each application of any of the following will count as an operation: `=`, `<`, `&&`, `!`, `[]`, `==`, and `++`. The computer must do other things besides carry out these operations, but these seem to be the main things that it is doing, and we will assume that they account for the bulk of the time needed to run this code. In fact, our analysis of time will assume that everything else takes no time at all and that the total time for our program to run is equal to the time needed to perform these operations. Although this is an idealization that clearly is not completely true, it turns out that this simplifying assumption works well in practice, and so it is often made when analyzing a program or algorithm.

Even with our simplifying assumption, we still must consider two cases: Either the value `target` is in the array, or it is not. Let us first consider the case when `target` is not in the array. The number of operations performed will depend on the number of array elements searched. The operation `=` is performed two times before the loop is executed. Because we are assuming that `target` is not in the array, the loop will be executed N times, one for each element of the array. Each time the loop is executed, the following operations are performed: `<`, `&&`, `!`, `[]`, `==`, and `++`. This adds five operations for each of N loop iterations. Finally, after N iterations, the Boolean expression is again checked and found to be `false`. This adds a final three operations (`<`, `&&`, `!`).³ If we tally all these operations, we get a total of $6N + 5$ operations when the target is not in the array. We will leave it as an exercise for the reader to confirm that if the target is in the array, then the number of operations will be $6N + 5$ or less. Thus, the worst-case running time is $T(N) = 6N + 5$ operations for any array of N elements and any value of `target`.

We just determined that the worst-case running time for our search code is $6N + 5$ operations. But an operation is not a traditional unit of time, such as a nanosecond, second, or minute. If we want to know how long the algorithm will take on some particular computer, we must know how long it takes that computer to perform one operation. If an operation can be performed in one nanosecond, then the time will be $6N + 5$ nanoseconds. If an operation can be performed in one second, the time will be $6N + 5$ seconds. If we use a slow computer that takes 10 seconds to perform an operation, the time will be $60N + 50$ seconds. In general, if it takes the computer c nanoseconds to perform one operation, then the actual running time will be approximately $c(6N + 5)$ nanoseconds. (We said *approximately* because we

³Because of short-circuit evaluation, `!(found)` is not evaluated, so we actually get two, not three, operations. However, the important thing is to obtain a good upper bound. If we add in one extra operation, that is not significant.

big-*O* notation

are making some simplifying assumptions and therefore the result may not be the absolutely exact running time.) This means that our running time of $6N + 5$ is a very crude estimate. To get the running time expressed in nanoseconds, you must multiply by some constant that depends on the particular computer you are using. Our estimate of $6N + 5$ is only accurate to within a constant multiple.

Estimates on running time, such as the one we just went through, are normally expressed in something called **big-*O* notation**. (The *O* is the letter “Oh,” not the digit zero.) Suppose we estimate the running time to be, say, $6N + 5$ operations, and suppose we know that no matter what the exact running time of each different operation may turn out to be, there will always be some constant factor c such that the real running time is less than or equal to $c(6N + 5)$. Under these circumstances, we say that the code (or program or algorithm) runs in time $O(6N + 5)$. This is usually read as “big-*O* of $6N + 5$.” We need not know what the constant c will be. In fact, it will undoubtedly be different for different computers, but we must know that there is one such c for any reasonable computer system. If the computer is very fast, the c might be less than 1—say, 0.001. If the computer is very slow, the c might be very large—say, 1,000. Moreover, because changing the units (say from nanosecond to second) involves only a constant multiple, there is no need to give any units of time.

Be sure to notice that a big-*O* estimate is an upper-bound estimate. We always approximate by taking numbers on the high side rather than the low side of the true count. Also notice that when performing a big-*O* estimate, we need not determine an exact count of the number of operations performed. We need only an estimate that is correct up to a constant multiple. If our estimate is twice as large as the true number, that is good enough.

An order-of-magnitude estimate, such as the previous $6N + 5$, contains a parameter for the size of the task solved by the algorithm (or program or piece of code). In our sample case, this parameter N was the number of array elements to be searched. Not surprisingly, it takes longer to search a larger number of array elements than it does to search a smaller number of array elements. Big-*O* running-time estimates are always expressed as a function of the size of the problem. In this chapter, all our algorithms will involve a range of values in some container. In all cases, N will be the number of elements in that range.

The following is an alternative, pragmatic way to think about big-*O* estimates:

*Only look at the term with the highest exponent and
do not pay attention to constant multiples.*

For example, all of the following are $O(N^2)$:

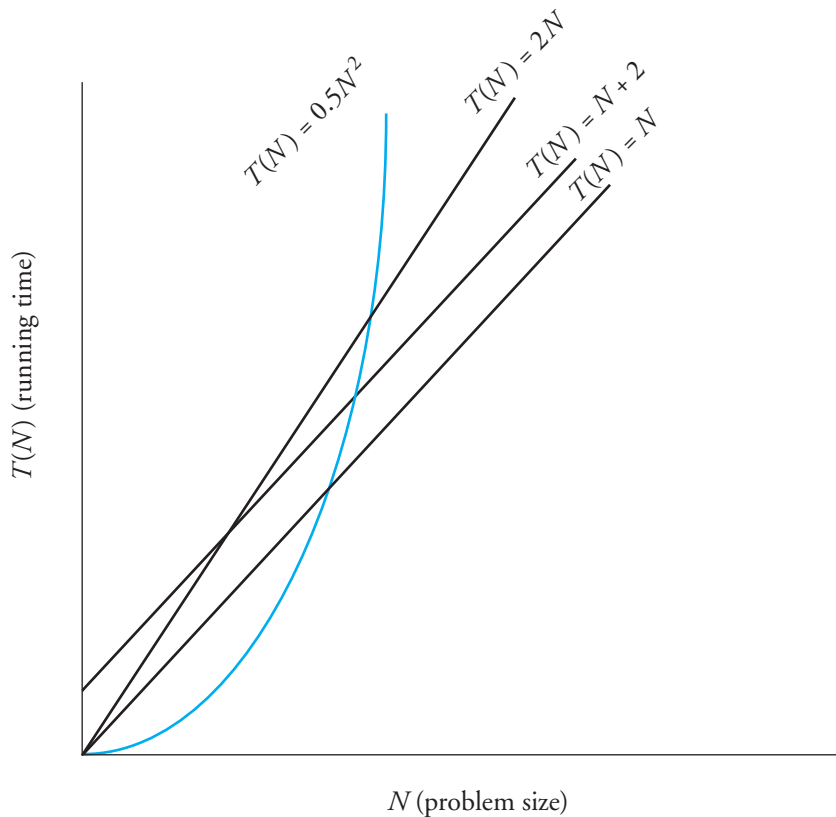
$$N^2 + 2N + 1, \quad 3N^2 + 7, \quad 100N^2 + N$$

All of the following are $O(N^3)$:

$$N^3 + 5N^2 + N + 1, \quad 8N^3 + 7, \quad 100N^3 + 4N + 1$$

These big- O running-time estimates are admittedly crude, but they do contain some information. They will not distinguish between a running time of $5N + 5$ and a running time of $100N$, but they do let us distinguish between some running times and so determine that some algorithms are faster than others. Look at the graphs in Display 15.32 and notice that all the graphs for functions that are $O(N)$ eventually fall below the graph for the function $0.5N^2$. The result is inevitable: An $O(N)$ algorithm will always run faster than any $O(N^2)$ algorithm, provided we use large enough values of N . Although an $O(N^2)$ algorithm could be faster than an $O(N)$ algorithm for the problem size you are handling, programmers have found that, in practice, $O(N)$ algorithms perform better than $O(N)$ algorithms for most practical applications that are intuitively “large.” Similar remarks apply to any other two different big- O running times.

Display 15.32 Comparison of Running Times



linear
running time

quadratic
running time

Some terminology will help with our descriptions of generic algorithm running times. **Linear running time** means a running time of $T(N) = aN + b$. A linear running time is always an $O(N)$ running time. **Quadratic running time** means a running time with a highest term of N^2 . A quadratic running time is always an $O(N^2)$ running time. We will also occasionally have logarithms in running-time formulas. Those normally are given without any base, because changing the base is just a constant multiple. If you see $\log N$, think log base 2 of N , but it would not be wrong to think log base 10 of N . Logarithms are very slow-growing functions. So, an $O(\log N)$ running time is very fast.

In many cases, our running-time estimates will be better than big- O estimates. In particular, when we specify a linear running time, that is a tight upper bound and you can think of the running time as being exactly $T(N) = cN$, although the c is still not specified.

Self-Test Exercises

19. Show that a running time $T(N) = aN + b$ is an $O(N)$ running time. (*Hint:* The only issue is the plus b . Assume N is always at least 1.)
20. Show that for any two bases a and b for logarithms, if a and b are both greater than 1, then there is a constant c such that $\log_a N \leq c (\log_b N)$. Thus, there is no need to specify a base in $O(\log N)$. That is, $O(\log_a N)$ and $O(\log_b N)$ mean the same thing.

Efficiency of Linked Lists

Now that we know about big- O notation, we can express the efficiency of various methods for our linked data structures. As an example of analyzing the run-time efficiency of an algorithm, consider the `find` method for the linked list class in Display 15.3. This method starts at the head of the list and sequentially iterates through each node to see whether it matches the target. If the linked list contains many nodes, then we might get lucky if the target is found at the head of the list. In this case, the computer had only to execute one step: Check the head of the list for the target. In the worst case, the computer might have to search through all n nodes before finding (or not finding) the target. In this case, the computer had to execute n steps. The worst case will obviously take longer to execute than the best case. On average, we might expect to search through about half of the list before finding the target. This would require $n/2$ steps. In our big- O notation, the find operation is $O(n)$. However, the `addToStart` method requires linking only a new node to the head of the list. This runs in $O(1)$ steps (that is, a constant upper bound on the running time that is independent of the size of the input).

Next we shall briefly examine more elaborate data structures that are capable of performing find operations in fewer steps. However, a detailed treatment of these more advanced data structures is beyond the scope of this chapter. The goal of this chapter is to teach you the basic techniques for constructing and manipulating data structures based on nodes and links (that is, nodes and references). The linked lists served as good examples for our discussion.

15.5 Hash Tables with Chaining

Seek, and ye shall find.

MATTHEW 7:7

hash table

hash map

A **hash table** or **hash map** is a data structure that efficiently stores and retrieves data from memory. There are many ways to construct a hash table; in this section, we will use an array in combination with singly linked lists. In the previous section, we saw that a linked list generally requires linear, or $O(n)$, steps to determine if a target is in the list. In contrast, a hash table has the potential to execute a fixed number of steps to look up a target, regardless of the size of n . We saw that a constant-time lookup is written $O(1)$. However, the hash table we will present may still require n steps, but such a case is unlikely.

hash function

An object is stored in a hash table by associating it with a *key*. Given the key, we can retrieve the object. Ideally, the key is unique to each object. If the object has no intrinsically unique key, then we can use a **hash function** to compute one. In most cases, the hash function computes a number.

For example, let us use a hash table to store a dictionary of words. Such a hash table might be useful to make a spell-checker—words missing from the hash table might not be spelled correctly. We will construct the hash table with a fixed array in which each array element references a linked list. The key computed by the hash function will map to the index of the array. The actual data will be stored in a linked list at the hash value's index. Display 15.33 illustrates the idea with a fixed array of 10 entries. Initially, each entry of the array `hashArray` contains a reference to an empty singly linked list. First, we add the word "cat", which has been assigned the key or hash value of 2 (we will show how this was computed shortly). Next, we add "dog" and "bird", which are assigned hash values of 4 and 7, respectively. Each of these strings is inserted as the head of the linked list using the hash value as the index in the array. Finally, we add "turtle", which also has a hash of 2. Because "cat" is already stored at index 2, we now have a **collision**. Both "turtle" and "cat" map to the same index in the array. When this occurs in a hash table with **chaining**, we simply insert the new node onto the existing linked list. In our example, there are now two nodes at index 2: "turtle" and "cat".

collision

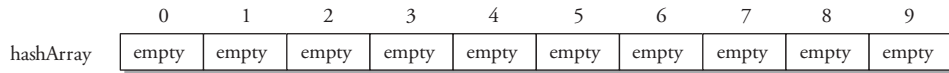
chaining

To retrieve a value from the hash table, we first compute the hash value of the target. Next we search the linked list that is stored at `hashArray[hashValue]` for the target, using an iterator to sequentially search the linked list. If the target is not found in this linked list, then the target is not stored in the hash table. If the size of the linked list is small, then the retrieval process will be quick.

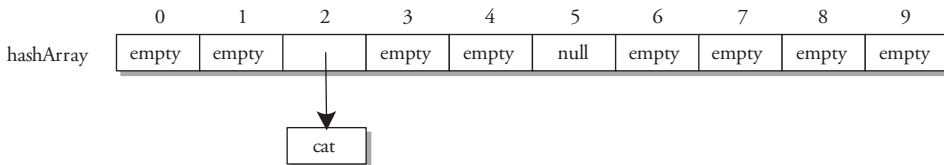
Display 15.33 Constructing a Hash Table

- Existing hash table initialized with 10 empty linked lists

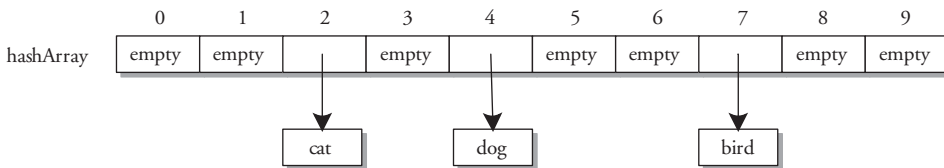
```
hashArray = new LinkedList 3[SIZE]; // SIZE = 10
```



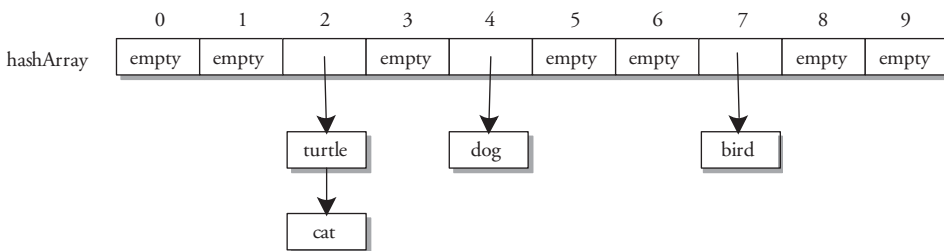
- After adding “cat” with hash of 2



- After adding “dog” with hash of 4 and “bird” with hash of 7



- After adding “turtle” with hash of 2 – collision and chained to linked list with “cat”



A Hash Function for Strings

A simple way to compute a numeric hash value for a string is to sum the ASCII value of every character in the string and then compute the modulus of the sum using the size of the fixed array. A subset of ASCII codes is given in Appendix 3. Code to compute the hash value is shown next:

```
private int computeHash(String s)
{
    int hash = 0;
```

```

    for (int i = 0; i < s.length(); i++)
    {
        hash += s.charAt(i);
    }
    return hash % SIZE; // SIZE=10 in example
}

```

For example, the ASCII codes for the string "dog" are as follows:

```

d ->100
o ->111
g ->103

```

The hash function is computed as follows:

```

Sum                = 100 + 111 + 103 = 314
Hash = Sum % 10    = 314 % 10 = 4

```

In this example, we first compute an unbounded value, the sum of the ASCII values in the string. However, the array was defined to only hold a finite number of elements. To scale the sum to the size of the array, we compute the modulus of the sum with respect to the size of the array, which is 10 in the example. In practice, the size of the array is generally a prime number larger than the number of items that will be put into the hash table.⁴ The computed hash value of 4 serves like a fingerprint for the string "dog". However, different strings may map to the same value. We can verify that "cat" maps to $(99 + 97 + 116) \% 10 = 2$ and also that "turtle" maps to $(116 + 117 + 114 + 116 + 108 + 101) \% 10 = 2$.

A complete code listing for a hash table class is given in Display 15.34, and a demonstration is provided in Display 15.35. The hash table definition in Display 15.34 uses an array in which each element is a `LinkedList2` class defined in Display 15.7.



VideoNote
Walkthrough of
the Hash Table
Class

Display 15.34 A Hash Table Class (part 1 of 2)

```

1  public class HashTable
2  {
3      // Uses the generic LinkedList2 class from Display 15.7
4      private LinkedList2[] hashArray;
5      private static final int SIZE = 10;
6
7      public HashTable( )
8      {
9          hashArray = new LinkedList2[SIZE];
10         for (int i=0; i < SIZE; i++)
11             hashArray[i] = new LinkedList2( );
12     }

```

⁴A prime number avoids common divisors after modulus that can lead to collisions.

Display 15.34 A Hash Table Class (part 2 of 2)

```
12     private int computeHash(String s)
13     {
14         int hash = 0;
15         for (int i = 0; i < s.length( ); i++)
16         {
17             hash += s.charAt(i);
18         }
19         return hash % SIZE;
20     }

21     /**
22      Returns true if the target is in the hash table,
23      false if it is not.
24     */
25     public boolean containsString(String target)
26     {
27         int hash = computeHash(target);
28         LinkedList2 list = hashArray[hash];
29         if (list.contains(target))
30             return true;
31         return false;
32     }
33     /**
34     Stores or puts string s into the hash table
35     */
36     public void put(String s)
37     {
38         int hash = computeHash(s); // Get hash value
39         LinkedList2 list = hashArray[hash];
40         if (!list.contains(s))
41         {
42             // Only add the target if it's not already
43             // on the list.
44             hashArray[hash].addToStart(s);
45         }
46     }
47 } // End HashTable class
```

Display 15.35 Hash Table Demonstration

```
1  public class HashTableDemo
2  {
3      public static void main(String[] args)
4      {
5          HashTable h = new HashTable( );
6
7          System.out.println("Adding dog, cat, turtle, bird");
8          h.put("dog");
9          h.put("cat");
10         h.put("turtle");
11         h.put("bird");
12         System.out.println("Contains dog? " +
13             h.containsString("dog"));
14         System.out.println("Contains cat? " +
15             h.containsString("cat"));
16         System.out.println("Contains turtle? " +
17             h.containsString("turtle"));
18         System.out.println("Contains bird? " +
19             h.containsString("bird"));
20
21         System.out.println("Contains fish? " +
22             h.containsString("fish"));
23         System.out.println("Contains cow? " +
24             h.containsString("cow"));
25     }
26 }
```

Sample Dialogue

```
Adding dog, cat, turtle, bird
Contains dog? true
Contains cat? true
Contains turtle? true
Contains bird? true
Contains fish? false
Contains cow? false
```

Efficiency of Hash Tables

The efficiency of our hash table depends on several factors. First, let us examine some extreme cases. The worst-case run-time performance occurs if every item inserted into the table has the same hash key. Everything will then be stored in a single linked list. With n items, the find operation will require $O(n)$ steps. Fortunately, if the items that we insert are somewhat random, the probability that all of them will hash to the same key is highly unlikely. In contrast, the best-case run-time performance occurs if every

item inserted into the table has a different hash key. This means that there will be no collisions, so the find operation will require constant, or $O(1)$, steps because the target will always be the first node in the linked list.

We can decrease the chance of collisions by using a better hash function. For example, the simple hash function that sums each letter of a string ignores the ordering of the letters. The words "rat" and "tar" would hash to the same value. A better hash function for a string *s* is to multiply each letter by an increasing weight depending upon the position in the word. For example,

```
int hash = 0;
for (int i = 0; i < s.length(); i++)
{
    hash = 31 * hash + s.charAt(i);
}
```

Another way to decrease the chance of collisions is by making the hash table bigger. For example, if the hash table array stored 10,000 entries but we are only inserting 1,000 items, then the probability of a collision is much smaller than if the hash table array stored only 1,000 entries. However, a drawback to creating an extremely large hash table array is wasted memory. If only 1,000 items are inserted into the 10,000-entry hash table, then at least 9,000 memory locations will go unused. This illustrates the **time-space tradeoff**. It is usually possible to increase run-time performance at the expense of memory space, and vice versa.

time-space
tradeoff

Self-Test Exercises

21. Suppose that every student in your university is assigned a unique nine-digit ID number. You would like to create a hash table that indexes ID numbers to an object representing a student. The hash table has a size of N , where N has less than nine digits. Describe a simple hash function that you can use to map from a ID number to a hash index.
22. Write an `outputHashTable()` method for the `HashTable` class that outputs every item stored in the hash table.

15.6 Sets

There are two classes in good society in England. The equestrian classes and the neurotic classes.

GEORGE BERNARD SHAW, *Heartbreak House*, 1919.

A set is a collection of elements in which order and multiplicity are ignored. Many problems in computer science can be solved with the aid of a set data structure. A variation on linked lists is a straightforward way to implement a set. In this implementation, the items in each set are stored using a singly linked list. The data

variable contains a reference to an object we wish to store in the set, whereas the `link` variable refers to the next `Node<T>` in the list (which in turn contains a reference to the next object to store in the set). The node class for a generic set of objects can begin as follows:

```
private class Node<T>
{
    private T data;
    private Node<T> link;
    ...
}
```

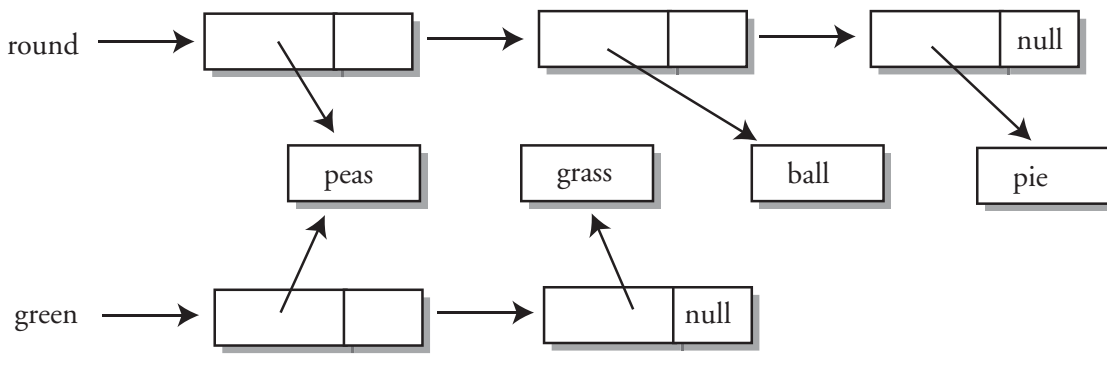
A complete listing is provided in Display 15.37. The `Node` class is a private inner class, similar to how we constructed the generic `LinkedList3<T>` class in Display 15.8. In fact, the set operations of `add`, `contains`, `output`, `clear`, `size`, and `isEmpty` are virtually identical to those from Display 15.8. The `add` method (which was `addToStart`) has been slightly changed to prevent duplicate items from being added into the set. Display 15.36 illustrates two sample sets stored using this data structure. The set `round` contains "peas", "ball", and "pie", whereas the set `green` contains "peas" and "grass". Because the linked list is storing a reference to each object in the set, it is possible to place an item in multiple sets by referencing it from multiple linked lists. In Display 15.36, "peas" is in both sets because it is round and green.

Fundamental Set Operations

The fundamental operations that our set class should support are as follows:

- **Add Element.** Add a new item into a set.
- **Contains.** Determine if a target item is a member of the set.
- **Union.** Return a set that is the union of two sets.
- **Intersection.** Return a set that is the intersection of two sets.

Display 15.36 Sets Using Linked Lists



We should also make an iterator so that every element can be retrieved from a set. This is left as a programming project for the reader (Programming Project 15.7). Other useful set operations include methods to retrieve the cardinality of the set and to remove items from the set.

Code to implement sets is provided in Display 15.37. The `add` method is similar to adding a node to the front of a linked list. The `head` variable always references the first node in the list. The `contains` method is identical to the `find` method for a singly linked list. We simply loop through every item in the list looking for the target.

The `union` method combines the elements in the calling object's set with the elements from the set of the input argument, `otherSet`. To union these sets, we first create a new empty `Set<T>` object. Next, we iterate through both the calling object's set and `otherSet`'s set. All elements are added (which creates new references to the items in the set) to the new set. The `add` method enforces uniqueness, so we do not have to check for duplicate elements in the `union` method.

The `intersection` method is similar to the `union` method in that it also creates a new empty `Set<T>` object. In this case, we populate the set with items that are common to both the calling object's set and `otherSet`'s set. This is accomplished by iterating through every item in the calling object's set. For each item, we invoke the `contains` method for `otherSet`. If `contains` returns `true`, then the item is in both sets and can be added to the new set.

A short demonstration program is shown in Display 15.38.

Display 15.37 `Set<T>` Class (part 1 of 3)

```

1  // Uses a linked list as the internal data structure
2  // to store items in a set.
3  public class Set<T>
4  {
5      private class Node<T>
6      {
7          private T data;
8          private Node<T> link;
9          public Node( )
10         {
11             data = null;
12             link = null;
13         }
14         public Node(T newData, Node<T> linkValue)
15         {
16             data = newData;
17             link = linkValue;
18         }
19     } //End of Node<T> inner class

```

(continued)

Display 15.37 Set<T> Class (part 2 of 3)

```
20     private Node<T> head;

21     public Set ( )
22     {
23         head = null;
24     }
25     /**
26      * Add a new item to the set. If the item
27      * is already in the set, false is returned;
28      * otherwise, true is returned.
29      */
30     public boolean add(T newItem)
31     {
32         if (!contains(newItem))
33         {
34             head = new Node<T>(newItem, head);
35             return true;
36         }
37         return false;
38     }

39     public boolean contains(T item)
40     {
41         Node<T> position = head;
42         T itemAtPosition;
43         while (position != null)
44         {
45             itemAtPosition = position.data;
46             if (itemAtPosition.equals(item))
47                 return true;
48             position = position.link;
49         }
50         return false; //target was not found
51     }

52     public void output ( )
53     {
54         Node position = head;
55         while (position != null)
56         {
57             System.out.print(position.data.toString( ) + " ");
58             position = position.link;
59         }
60         System.out.println( );
61     }
```

Display 15.37 Set<T> Class (part 3 of 3)

```

62     /**
63      * Returns a new set that is the union
64      * of this set and the input set.
65      */
66     public Set<T> union(Set<T> otherSet)
67     {
68         Set<T> unionSet = new Set<T>( );
69         // Copy this set to unionSet.
70         Node<T> position = head;
71         while (position != null)
72         {
73             unionSet.add(position.data);
74             position = position.link;
75         }
76         // Copy otherSet items to unionSet.
77         // The add method eliminates any duplicates.
78         position = otherSet.head;
79         while (position != null)
80         {
81             unionSet.add(position.data);
82             position = position.link;
83         }
84         return unionSet;
85     }

86     /**
87      * Returns a new set that is the intersection
88      * of this set and the input set.
89      */
90     public Set<T> intersection(Set<T> otherSet)
91     {
92         Set<T> interSet = new Set<T>( );
93         // Copy only items in both sets.
94         Node<T> position = head;
95         while (position != null)
96         {
97             if (otherSet.contains(position.data))
98                 interSet.add(position.data);
99             position = position.link;
100         }
101         return interSet;
102     }
103 }

```

The clear, size, and isEmpty methods are identical to those in Display 15.8 for the LinkedList3 class.

Display 15.38 Set Class Demo (part 1 of 2)

```
1  public class SetDemo
2  {
3      public static void main(String[] args)
4      {
5          // Round things
6          Set round = new Set<String>( );
7          // Green things
8          Set green = new Set<String>( );
9
10         // Add some data to both sets
11         round.add("peas");
12         round.add("ball");
13         round.add("pie");
14         round.add("grapes");
15
16         green.add("peas");
17         green.add("grapes");
18         green.add("garden hose");
19         green.add("grass");
20
21         System.out.println("Contents of set round: ");
22         round.output( );
23         System.out.println("Contents of set green: ");
24         green.output( );
25         System.out.println( );
26
27         System.out.println("ball in set round? " +
28             round.contains("ball"));
29         System.out.println("ball in set green? " +
30             green.contains("ball"));
31         System.out.println("ball and peas in same set? " +
32             ((round.contains("ball") &&
33              (round.contains("peas"))) ||
34              (green.contains("ball") &&
35               (green.contains("peas")))));
36
37         System.out.println("pie and grass in same set? " +
38             ((round.contains("pie") &&
39              (round.contains("grass"))) ||
40              (green.contains("pie") &&
41               (green.contains("grass")))));
42
43         System.out.print("Union of green and round: ");
44         round.union(green).output( );
```

Display 15.38 Set Class Demo (part 2 of 2)

```
39         System.out.print("Intersection of green and round: ");
40         round.intersection(green).output( );
41     }
42 }
```

Sample Dialogue

```
Contents of set round:
grapes pie ball peas
Contents of set green:
grass garden hose grapes peas

ball in set round? true
ball in set green? false
ball and peas in same set? true
pie and grass in same set? false
Union of green and round: garden hose grass peas ball pie grapes
Intersection of green and round: peas grapes
```

Efficiency of Sets Using Linked Lists

We can analyze the efficiency of our set data structure in terms of the fundamental set operations. Adding an item to the set always inserts a new node on the front of the list. This requires constant, or $O(1)$, steps. The `contains` method iterates through the entire set looking for the target, which requires $O(n)$ steps. When we invoke the `union` method for sets A and B , it iterates through both sets and adds each item into a new set. If there are n items in set A and m items in set B , then $n + m$ add methods are invoked. However, there is a hidden cost because the `add` method searches through its entire list for any duplicates before a new item is added. Although beyond the scope of this text, the additional cost results in $O(m + n)^2$ steps. Finally, the `intersection` method applied to sets A and B invokes the `contains` method of set B for each item in set A . Because the `contains` method requires $O(m)$ steps for each item in set A , then this requires $O(m) \times O(n)$ steps, or $O(mn)$ steps. These are inefficient methods in our implementation of sets. A different approach to represent the set—for example, one that used hash tables instead of a linked list—could result in an `intersection` method that runs in $O(n + m)$ steps. Nevertheless, our linked list implementation would probably be fine for an application that uses small sets or for an application that does not frequently invoke the `intersection` method, and we have the benefit of relatively simple code that is easy to understand.

If we really need the efficiency, then we could maintain the same interface to the `Set<T>` class but replace our linked list implementation with something else. If we used the hash table implementation from Section 15.5, then the `contains` method

could run in $O(1)$ steps instead of $O(n)$ steps. It might seem like the `intersection` method will now run in $O(n)$ steps, but by switching to a hash table, it becomes more difficult to iterate through the set of items. Instead of traversing a single linked list to retrieve every item in the set, the hash table version must now iterate through the hash table array and then for each index in the array iterate through the linked list at that index. If the array is size N and the number of items in the hash table is n , then the iteration time becomes $O(N + n)$. In practice, we would expect N to be larger than n . So although we have decreased the number of steps it takes to look up an item, we have increased the number of steps it takes to iterate over every item. If this is troublesome, you could overcome this problem with an implementation of `Set<T>` that uses both a linked list (to facilitate iteration) and a hash table (for fast lookup). However, the complexity of the code is significantly increased using such an approach. You are asked to explore the hash table implementation in Programming Project 15.10.

Self-Test Exercises

23. Write a method named `difference` that returns the difference between two sets. The method should return a new set that has items from the first set that are not in the second set. For example, if `setA` contains $\{1, 2, 3, 4\}$ and `setB` contains $\{2, 4, 5\}$, then `setA.difference(setB)` should return the set $\{1, 3\}$.
24. What is the run time of the `difference` method for the previous exercise? Give your answer using big- O notation.

15.7 Trees

I think that I shall never see a data structure as useful as a tree.

ANONYMOUS

The tree data structure is an example of a more complicated data structure made with links. Moreover, trees are a very important and widely used data structure. So, we will briefly outline the general techniques used to construct and manipulate trees. This section is only a very brief introduction to trees to give you the flavor of the subject.

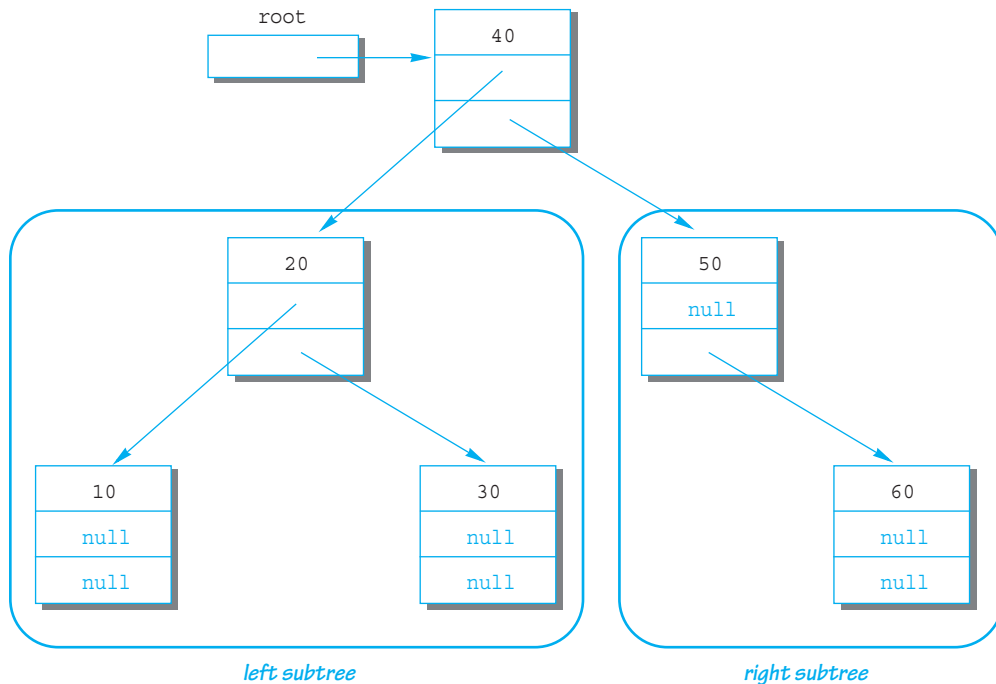
This section uses recursion, which is covered in Chapter 11.

Tree Properties

A tree is a data structure that is structured as shown in Display 15.39. In particular, in a tree you can reach any node from the top (root) node by some path that follows the links. Note that there are no cycles in a tree. If you follow the links, you eventually get to an “end.” A definition for a tree class for this sort of tree of `ints` is outlined in Display 15.39. Note that each node has two references to other nodes (two links) coming from it. This sort of tree is called a **binary tree**, because each node has exactly

two link instance variables. There are other kinds of trees with different numbers of link instance variables, but the binary tree is the most common case.

Display 15.39 A Binary Tree



```

1 public class IntTree
2 {
3     public class IntTreeNode
4     {
5         private int data;
6         private IntTreeNode leftLink;
7         private IntTreeNode rightLink;
8     } //End of IntTreeNode inner class
9
10    private IntTreeNode root;
    <The methods and other inner classes are not shown.>
10 }
  
```

root node

The instance variable named `root` serves a purpose similar to that of the instance variable `head` in a linked list (Display 15.3). The node whose reference is in the `root` instance variable is called the **root node**. Any node in the tree can be reached from the root node by following the links.

The term *tree* may seem like a misnomer. The root is at the top of the tree, and the branching structure looks more like a root branching structure than a tree branching structure. The secret to the terminology is to turn the picture (Display 15.39) upside down. The picture then does resemble the branching structure of a tree, and the root node is where the tree's root would begin. The nodes at the ends of the branches with both link instance variables set to `null` are known as **leaf nodes**, a terminology that may now make some sense. By analogy to an empty linked list, an **empty tree** is denoted by setting the link variable `root` equal to `null`.

Note that a tree has a recursive structure. Each tree has, in effect, two subtrees whose root nodes are the nodes pointed to by the `leftLink` and `rightLink` of the root node. These two subtrees are circled in Display 15.39. This natural recursive structure makes trees particularly amenable to recursive algorithms. For example, consider the task of searching the tree in such a way that you visit each node and do something with the data in the node (such as writing it out to the screen). There is a general plan of attack that goes as follows:

Preorder Processing

1. Process the data in the root node.
2. Process the left subtree.
3. Process the right subtree.

Obtain a number of variants on this search process by varying the order of these three steps. Two more versions follow:

inorder

Inorder Processing

1. Process the left subtree.
2. Process the data in the root node.
3. Process the right subtree.

postorder

Postorder Processing

1. Process the left subtree.
2. Process the right subtree.
3. Process the data in the root node.

Binary Search Tree Storage Rule

The tree in Display 15.39 has numbers that were stored in the tree in a special way known as the **Binary Search Tree Storage Rule**. The rule is summarized in the following box.

Binary Search Tree Storage Rule

1. All the values in the left subtree are less than the value in the root node.
 2. All the values in the right subtree are greater than or equal to the value in the root node.
 3. This rule applies recursively to each of the two subtrees.
- (The base case for the recursion is an empty tree, which is always considered to satisfy the rule.)

**binary search
tree**

A tree that satisfies the Binary Search Tree Storage Rule is referred to as a **binary search tree**.

Note that if a tree satisfies the Binary Search Tree Storage Rule and you output the values using the Inorder Processing method, then the numbers will be output in order from smallest to largest.

For trees that follow the Binary Search Tree Storage Rule and that are short and fat rather than tall and thin, values can be very quickly retrieved from the tree using a binary search algorithm that is similar in spirit to the binary search algorithm we presented in Display 11.6. The topic of searching and maintaining a binary storage tree to realize this efficiency is a large topic that goes beyond what we have room for here. However, we give one example of a class for trees that satisfy the Binary Search Tree Storage Rule.

EXAMPLE: A Binary Search Tree Class ★

Display 15.40 contains the definition of a class for a binary search tree that satisfies the Binary Search Tree Storage Rule. For simplicity, this tree stores integers, but a routine modification can produce a similar tree class that stores objects of any class that implements the `Comparable` interface. Display 15.41 demonstrates the use of this tree class. Note that no matter in which order the integers are inserted into the tree, the output, which uses inorder traversal, outputs the integers in sorted order.

The methods in this class make extensive use of the recursive nature of binary trees. If `aNode` is a reference to any node in the tree (including possibly the root node), then the entire tree with root `aNode` can be decomposed into three parts:

1. The node `aNode`.
2. The left subtree with root node `aNode.leftLink`.
3. The right subtree with root node `aNode.rightLink`.

The left and right subtrees do themselves satisfy the Binary Search Tree Storage Rule, so it is natural to use recursion to process the entire tree by doing the following:

1. Processing the left subtree with root node `aNode.leftLink`
2. Processing the node `aNode`
3. Processing the right subtree with root node `aNode.rightLink`

Note that we processed the root node after the left subtree (inorder traversal). This guarantees that the numbers in the tree are output in the order smallest to largest. The method `showElementsInSubtree` uses a very straightforward implementation of this technique.

Other methods are a bit more subtle in that only one of the two subtrees needs to be processed. For example, consider the method `isInSubtree`, which returns `true` or `false` depending on whether or not the parameter `item` is in the tree with root node `subTreeRoot`. To see if the item is anyplace in the tree, set `subTreeRoot` equal

(continued)

EXAMPLE: (continued)

to the root of the entire tree, as we did in the method `contains`. However, to express our recursive algorithm for `isInSubtree`, we need to allow for the possibility of subtrees other than the entire tree.

The algorithm for `isInSubtree` expressed in pseudocode is as follows:

```
if (The root node subTreeRoot is empty.)
    return false;
else if (The node subTreeRoot contains item.)
    return true;
else if (item < subTreeRoot.data)
    return (The result of searching the tree
            with root node subTreeRoot.leftLink);
else
    //item > link.data
    return (The result of searching the tree
            with root node subTreeRoot.rightLink);
```

The reason this algorithm gives the correct result is that the tree satisfies the Binary Search Tree Storage Rule, so we know that if

```
item < subTreeRoot.data
```

then `item` is in the left subtree (if it is anywhere in the tree), and if

```
item > subTreeRoot.data
```

then `item` is in the right subtree (if it is anywhere in the tree).

The method with the following heading uses techniques very much like those used in `isInSubtree`:

```
private IntTreeNode insertInSubtree(
    int item, IntTreeNode subTreeRoot)
```

However, there is something new here. We want the method `insertInSubtree` to insert a new node with the data `item` into the tree with root node `subTreeRoot`. But in this case, we want to deal with `subTreeRoot` as a variable and not use it only as the value of the variable `subTreeRoot`. For example, if `subTreeRoot` contains `null`, then we want to change the value of `subTreeRoot` to a reference to a new node containing `item`. However, Java parameters cannot change the value of a variable given as an argument. (Review the discussion of parameters in Chapter 5 if this sounds unfamiliar.) So, we must do something a little different. To change the value of the variable `subTreeRoot`, we return a reference to what we want the new value to be, and we invoke the method `subTreeRoot` as follows:

```
subTreeRoot = insertInSubtree(item, subTreeRoot);
```

That explains why the method `insertInSubtree` returns a reference to a tree node, but we still have to explain why we know it returns a reference to the desired (modified) subtree.

EXAMPLE: (continued)

Note that the method `insertInSubtree` searches the tree just as the method `isInSubtree` does, but it does not stop if it finds `item`; instead, it searches until it reaches a leaf node—that is, a node containing `null`. This `null` is where the item belongs in the tree, so it replaces `null` with a new subtree containing a single node that contains `item`. You may need to think about the method `insertInSubtree` a bit to see that it works correctly; allow yourself some time to study the method `insertInSubtree` and be sure you are convinced that after the addition, like the following,

```
subTreeRoot = insertInSubtree(item, subTreeRoot);
```

the tree with root node `subTreeRoot` still satisfies the Binary Search Tree Storage Rule.

The rest of the definition of the class `IntTree` is routine.

Display 15.40 A Binary Search Tree for Integers (part 1 of 2)

```

1  /**
2   * Class invariant: The tree satisfies the Binary Search Tree Storage Rule.
3   */
4  public class IntTree
5  {
6      private static class IntTreeNode ← The only reason this inner
7      {                                     class is static is that it is
8          private int data;                used in the static methods
9          private IntTreeNode leftLink;    insertInSubtree,
10         private IntTreeNode rightLink;   isInSubtree, and
11                                         showElementsInSubtree.
12
13         public IntTreeNode(int newData, IntTreeNode newLeftLink,
14                                IntTreeNode newRightLink)
15         {
16             data = newData;
17             leftLink = newLeftLink;
18             rightLink = newRightLink;
19         }
20     } //End of IntTreeNode inner class
21
22     private IntTreeNode root;
23
24     public IntTree( )
25     {
26         root = null; ← This class should have more methods.
27                                     This is just a sample of possible methods.
28     }
29
30     public void add(int item)
31     {
32         root = insertInSubtree(item, root);
33     }

```

(continued)

Display 15.40 A Binary Search Tree for Integers (part 2 of 2)

```

29     public boolean contains(int item)
30     {
31         return isInSubtree(item, root);
32     }

33     public void showElements( )
34     {
35         showElementsInSubtree(root);
36     }
37     /**
38      Returns the root node of a tree that is the tree with root node
39      subTreeRoot, but with a new node added that contains item.
40      */
41     private static IntTreeNode insertInSubtree(int item,
42                                               IntTreeNode subTreeRoot)
43     {
44         if (subTreeRoot == null)
45             return new IntTreeNode(item, null, null);
46         else if (item < subTreeRoot.data)
47         {
48             subTreeRoot.leftLink = insertInSubtree(item, subTreeRoot.
49                                                         leftLink);
50             return subTreeRoot;
51         }
52         else //item >= subTreeRoot.data
53         {
54             subTreeRoot.rightLink = insertInSubtree(item, subTreeRoot.
55                                                         rightLink);
56             return subTreeRoot;
57         }
58     }

59     private static boolean isInSubtree(int item, IntTreeNode
60                                         subTreeRoot)
61     {
62         if (subTreeRoot == null)
63             return false;
64         else if (subTreeRoot.data == item)
65             return true;
66         else if (item < subTreeRoot.data)
67             return isInSubtree(item, subTreeRoot.leftLink);
68         else //item >= link.data
69             return isInSubtree(item, subTreeRoot.rightLink);
70     }

71     private static void showElementsInSubtree(IntTreeNode subTreeRoot)
72     { //Uses inorder traversal.
73         if (subTreeRoot != null)
74         {
75             showElementsInSubtree(subTreeRoot.leftLink);
76             System.out.print(subTreeRoot.data + " ");
77             showElementsInSubtree(subTreeRoot.rightLink);
78         } //else do nothing. Empty tree has nothing to display.
79     }

```

Display 15.41 Demonstration Program for the Binary Search Tree

```
1  import java.util.Scanner;

2  public class BinarySearchTreeDemo
3  {
4      public static void main(String[] args)
5      {
6          Scanner keyboard = new Scanner(System.in);
7          IntTree tree = new IntTree( );

8          System.out.println("Enter a list of nonnegative integers.");
9          System.out.println("Place a negative integer at the end.");
10         int next = keyboard.nextInt( );
11         while (next >= 0)
12         {
13             tree.add(next);
14             next = keyboard.nextInt( );
15         }

16         System.out.println("In sorted order:");
17         tree.showElements( );
18     }
19 }
```

Sample Dialogue

```
Enter a list of nonnegative integers.
Place a negative integer at the end.
40
30
20
10
11
22
33
44
-1
In sorted order:
10 11 20 22 30 33 40 44
```

Efficiency of Binary Search Trees ★

When searching a tree that is as short as possible (all paths from root to a leaf differ by at most one node), the search method `isInSubtree`, and hence also the method `contains`, is about as efficient as the binary search on a sorted array (Display 11.6). This should not be a surprise because the two algorithms are in fact very similar. In big- O notation, the worst-case running time is $O(\log n)$, where n is the number of nodes in the tree. This means that searching a short, fat binary tree is very efficient. To obtain this efficiency, the tree does not need to be as short as possible so long as it comes close to being as short as possible. As the tree becomes less short and fat and more tall and thin, the efficiency falls off until, in the extreme case, the efficiency is the same as that of searching a linked list with the same number of nodes.

Maintaining a tree so that it remains short and fat as nodes are added is a topic that is beyond the scope of what we have room for in this book. (The technical term for short and fat is *balanced*.) We will note only that if the numbers that are stored in the tree arrive in random order, then with very high probability the tree will be short and fat enough to realize the efficiency discussed in the previous paragraph.

Self-Test Exercises

25. Suppose that the code for the method `showElementsInSubtree` in Display 15.40 were changed so that

```
showElementsInSubtree(subTreeRoot.leftLink);  
System.out.print(subTreeRoot.data + " ");  
showElementsInSubtree(subTreeRoot.rightLink);
```

were changed to

```
System.out.print(subTreeRoot.data + " ");  
showElementsInSubtree(subTreeRoot.leftLink);  
showElementsInSubtree(subTreeRoot.rightLink);
```

Will the numbers still be output in ascending order?

26. How can you change the code for the method `showElementsInSubtree` in Display 15.40 so that the numbers are output from largest to smallest instead of from smallest to largest?

Chapter Summary

- A *linked list* is a data structure consisting of objects known as *nodes*, such that each node contains data and also a reference to one other node so that the nodes link together to form a list.
- Setting a link instance variable to `null` indicates the end of a linked list (or other linked data structure). `null` is also used to indicate an empty linked list (or other empty linked data structure).
- You can make a linked list (or other linked data structure) self-contained by making the node class an inner class of the linked list class.
- In many situations, a `clone` method or copy constructor is best defined so that it makes a deep copy.
- You can use an *iterator* to step through the elements of a collection, such as the elements in a linked list.
- Nodes in a *doubly linked list* have two links—one to the previous node in the list and one to the next node. This makes some operations, such as insertion and deletion, slightly easier.
- A *stack* is a data structure in which elements are removed in the reverse of the order they were added to the stack. A *queue* is a data structure in which elements are removed in the same order that they were added to the queue.
- *Big-O notation* specifies an upper bound for how many steps or how long a program will take to run based on the size of the input to the program. This can be used to analyze the efficiency of an algorithm.
- A *hash table* is a data structure that is used to store objects and retrieve them efficiently. A *hash function* is used to map an object to a value that can then be used to index the object.
- Linked lists can be used to implement sets, including common operations such as union, intersection, and set membership.
- A *binary tree* is a branching linked data structure consisting of nodes that each have two link instance variables. A tree has a special node called the *root node*. Every node in the tree can be reached from the root node by following links.
- If values are stored in a binary tree in such a way that the *Binary Search Tree Storage Rule* is followed, then there are efficient algorithms for reaching values stored in the tree.

Answers to Self-Test Exercises

1. mustard 1
hot dogs 12
apple pie 1
2. This method has been added to the class `LinkedList1` on the accompanying website.

```
public boolean isEmpty( )
{
    return (head == null);
}
```

3. This method has been added to the class `LinkedList1` on the accompanying website.

```
public void clear( )
{
    head = null;
}
```

If you defined your method to remove all nodes using the `deleteHeadNode` method, your method is doing wasted work.

4. Yes. If we make the inner class `Node` a public inner class, it could be used outside the definition of `LinkedList2`, whereas leaving it as private means it cannot be used outside the definition of `LinkedList2`.
5. It would make no difference. Within the definition of an outer class, there is full access to the members of an inner class whatever the inner class member's access modifier is. To put it another way, inside the private inner class `Node`, the modifiers `private` and `package` access are equivalent to `public`.
6. Because the outer class has direct access to the instance variables of the inner class `Node`, no access or mutator methods are needed for `Node`.
7. It would be legal, but it would be pretty much a useless method, because you cannot use the type `Node` outside of the class `LinkedList2`. For example, outside of the class `LinkedList2`, the following is illegal (`listObject` is of type `LinkedList2`):

```
Node v = listObject.startNode( ); //Illegal
```

whereas the following would be legal outside of the class `LinkedList2` (although it is hard to think of anyplace you might use it):

```
Object v = listObject.startNode( );
```

8.

```
public class LinkedList2
{
    public class Entry
    {
        private String item;
        private int count;

        public Entry( )
```

```
{
    item = null;
    count = 0;
}

public Entry(String itemData, int countData)
{
    item = itemData;
    count = countData;
}

public void setItem(String itemData)
{
    item = itemData;
}

public void setCount(int countData)
{
    count = countData;
}

public String getItem( )
{
    return item;
}

public int getCount( )
{
    return count;
}
} // End of Entry inner class

private class Node
{
    private Entry item;
    private Node link;

    public Node( )
    {
        item = null;
        link = null;
    }
}
```

```

    public Node(Entry newItem, Node linkValue)
    {
        item = newItem;
        link = linkValue;
    }
} //End of Node inner class

private Node head;

<Other definitions from LinkedList2 go here>

} //End of LinkedList2 class

```

The rest of the definition of `LinkedList2` is essentially the same as in Display 15.7, but with the type `String` replaced by `Entry`. A complete definition is given in the subdirectory named “Exercise 8” on the website that accompanies this text.

9. No, T is not guaranteed to have a copy constructor. Even if T has a copy constructor, it is illegal to use T with `new` like this.
10. No, you can use any descendent class of `Object` (which means any class type) as the returned type, because the value returned will still be of type `Object`.
11. The `delete` method must now search through the list to find the previous node and then change the link to bypass the current position. This is less efficient than the code in Display 15.17 because the reference to the previous node is already set.

```

public void delete( )
{
    if (position == null)
    {
        throw new IllegalStateException( );
    }
    else
    {
        Node current = head;
        Node previous = null;
        while (current != null)
        {
            if (current == position)
            {
                // Found the node to delete
                // Check if we're at the head
                if (previous == null)
                {
                    head = head.link;
                    position = head;
                }
            }
        }
    }
}

```

```

        else // Delete in middle of list
        {
            previous.link = position.link;
            position = position.link;
        }
        return;
    }
    previous = current; // Advance references
    current = current.link;
}
}
}

```

12. One problem with adding after the iterator's position is that there is no way to add to the front of the list. It would be possible to make a special case in which the new node were added to the front (e.g., if position is null, add the new data to the head) if desired.

```

public void addAfterHere(String newData)
{
    if (position == null && head != null)
    {
        // At end of list; can't add here
        throw new IllegalStateException( );
    }
    else if (head == null)
        // At head of empty list, add to front
        LinkedList2Iter.this.addToStart(newData);
    else
    {
        // Add after current position
        Node temp = new Node(newData, position.link);
        position.link = temp;
    }
}

```

13.

```
public void changeHere(String newData)
{
    if (position == null)
        throw new IllegalStateException( );
    else
        position.item = newData;
}
```

14. When invoking `i.next( )`, the value of the node that `i` is referencing is copied to a local variable, the iterator moves to the next node in the link, and then the value of the local variable is returned. Therefore, the value that `i` is referencing prior to the invocation is returned.

15. Insertion and deletion are slightly easier with the doubly linked list because we no longer need a separate instance variable to keep track of the previous node due to the previous link. However, all operations require updating more links (e.g., both the next and previous instead of just the previous).

16. Use the iterator:

```
DoublyLinkedList.DoublyLinkedListIterator i = list.iterator( );
i.restart( );
i.insertHere("Element At Front");
```

17. Pop the top of the stack and then push it back on:

```
String s = stack.pop( );
Stack.push(s);
// s contains the string on the top of the stack
```

18. `public void addToBack(String itemName)`

```
{
    Node newEntry =
        new Node(itemName, null);
    if (front == null) //empty queue
    {
        back = newEntry;
        front = back;
    }
    else
    {
        back.link = newEntry;
        back = back.link;
    }
}
```

19. Just note that $aN + b \leq (a + b)N$, as long as $1 \leq N$.

20. This is mathematics, not Java. So, = will mean *equals*, not assignment.

First note that $\log_a N = (\log_a b)(\log_b N)$.

To see this first identity, just note that if you raise a to the power $\log_a N$, you get N , and if you raise a to the power $(\log_a b)(\log_b N)$, you also get N .

If you set $c = (\log_a b)$, you get $\log_a N = c (\log_b N)$.

21. The simplest hash function is to map the ID number to the range of the hash table using the modulus operator:

```
hash = ID % N; // N is the hash table size
```

```

22. public void outputHashTable( )
    {
        for (int i=0; i< SIZE; i++)
        {
            if (hashArray[i].size( ) > 0)
                hashArray[i].outputList( );
        }
    }

```

23. This code is similar to intersection, but adds elements if they are not in otherSet:

```

public Set<T> difference(Set<T> otherSet)
{
    Set<T> diffSet = new Set<T>( );
    // Copy only items in this set but not otherSet
    Node<T> position = head;
    while (position != null)
    {
        if (!otherSet.contains(position.data))
            diffSet.add(position.data);
        position = position.link;
    }
    return diffSet;
}

```

24. As implemented in Programming Problem 15.23, the complexity is identical to the intersection method. For every element in the set, we invoke the contains method of otherSet. This requires $O(nm)$ steps, where n is the number of items in the calling object's set and m is the number of items in otherSet's set.

25. No.

26. Change

```

showElementsInSubtree(subTreeRoot.leftLink);
System.out.print(subTreeRoot.data + " ");
showElementsInSubtree(subTreeRoot.rightLink);

to

showElementsInSubtree(subTreeRoot.rightLink);
System.out.print(subTreeRoot.data + " ");
showElementsInSubtree(subTreeRoot.leftLink);

```

Programming Projects

1. In an ancient land, the beautiful princess Eve had many suitors. She decided on the following procedure to determine which suitor she would marry. First, all of the suitors would be lined up one after the other and assigned numbers. The first suitor would be number 1, the second number 2, and so on up to the last suitor, number n . Starting at the suitor in the first position, she would then count three suitors down the line

(because of the three letters in her name), and the third suitor would be eliminated from winning her hand and removed from the line. Eve would then continue, counting three more suitors, and eliminate every third suitor. When she reached the end of the line, she would continue counting from the beginning.

For example, if there were six suitors, the elimination process would proceed as follows:

123456	Initial list of suitors; start counting from 1.
12456	Suitor 3 eliminated; continue counting from 4.
1245	Suitor 6 eliminated; continue counting from 1.
125	Suitor 4 eliminated; continue counting from 5.
15	Suitor 2 eliminated; continue counting from 5.
1	Suitor 5 eliminated; 1 is the lucky winner.

Write a program that creates a circular linked list of nodes to determine which position you should stand in to marry the princess if there are n suitors. Your program should simulate the elimination process by deleting the node that corresponds to the suitor that is eliminated for each step in the process.

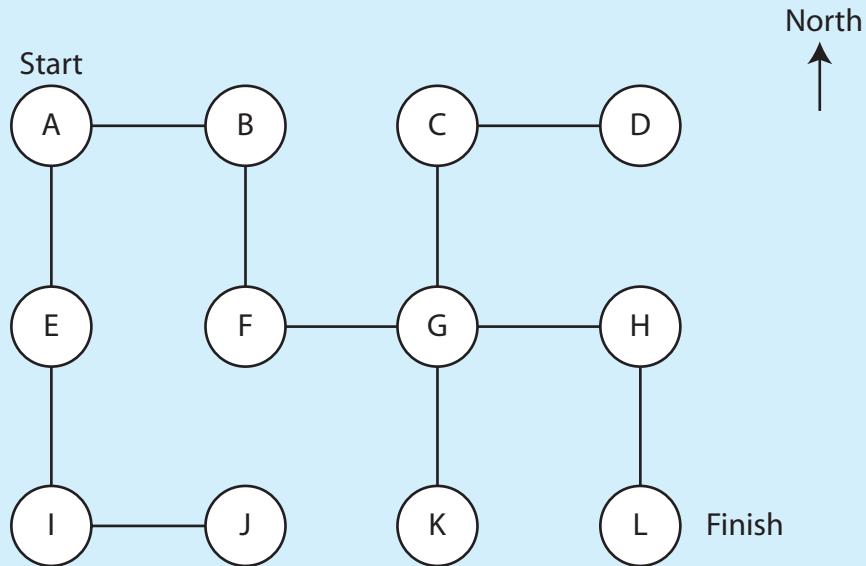
2. A record contains data as well as reference to the next record. We should be able to insert or remove data within the data records. Create a class named as `WordLinkedList` that uses the `LinkedList` class discussed in this chapter to contain lists of words. Create a menu-driven program that allows a user to choose from the following list of operations:
 1. Insert word in the beginning
 2. Insert word at a given position
 3. Delete word from the beginning
 4. Delete word from a given position
 5. Display complete list
 6. Search a specific word
 7. Exit
3. Extend the previous problem to create a list that will act as a stack. A stack is a data structure that follows the last-in first-out order for the addition and deletion of elements. This list will contain the following items of integer type values. Modify the method implemented in the previous problem for addition of an item in the stack to create a push method. Also, modify the remove method for deleting items from the stack to create a pop method. Do remember that the elements are inserted or deleted from only one end of the stack.
4. In reference to the previous two problems, again modify the program to make use of the `LinkedList` class to implement a double-ended queue. In a double-ended queue, insertion and deletion can be carried out at both the ends. Add methods to add and remove elements in this double-ended queue.
5. Complete the definition of the binary search tree class `IntTree` in Display 15.39 by adding the following: Make `IntTree` implement the `Cloneable` interface, including the definition of a `clone` method; add a copy constructor; add an

`equals` method; add a method named `sameContents` as described later in this project; add a `toString` method; and add a method to produce an iterator. Define `equals` so that two trees are equal if (and only if) the two trees have the exact same shape and have the same numbers in corresponding nodes. The `clone` method and the copy constructor should each produce a deep copy that is equal to the original list according to the `equals` method. The boolean valued method `sameContents` has one parameter of type `IntTree` and returns `true` if the calling object and the argument tree contain exactly the same numbers, and returns `false` otherwise. Note that `equals` and `sameContents` are not the same. Also, write a suitable test program.

6. Write an `addSorted` method for the generic linked list from Display 15.8 such that the method adds a new node in the correct location so that the list remains in sorted order. Note that this will require that the type parameter `T` extend the `Comparable` interface. Write a suitable test program.
7. Add a `remove` method and an iterator for the `Set` class in Display 15.37. Write a suitable test program.
8. The hash table from Display 15.34 hashed a string to an integer and stored the same string in the hash table. Modify the program so that instead of storing strings, it stores `Employee` objects as defined in Display 7.2. Use the `name` instance variable as the input to the hash function. The modification will require changes to the linked list, because the `LinkedList2` class created only linked lists of strings. For the most generality, modify the hash table so that it uses the generic `LinkedList3` class defined in Display 15.8. You will also need to add a `get` method that returns the `Employee` object stored in the hash table that corresponds to the input name. Test your program by adding and retrieving several names, including names that hash to the same slot in the hash table.
9. Displays 15.34 and 15.35 provide the beginnings of a spell-checker. Refine the program to make it more useful. The modified program should read in a text file, parse each word, see if it is in the hash table, and, if not, output the line number and word of the potentially misspelled word. Discard any punctuation in the original text file. Use the `words.txt` file as the basis for the hash table dictionary. This file can be found on the book's website. The file contains 87,314 words in the English language. Test your spell-checker on a short text document.
10. Change the `Set<T>` class of Display 15.37 so that internally it uses a hash table to store its data instead of a linked list. The headers of the public methods should remain the same so that a program such as the demonstration in Display 15.38 should still work without requiring any changes. Add a constructor that allows the user of the new `Set<T>` class to specify the size of the hash table array.

For an additional challenge, implement the set using both a hash table and a linked list. Items added to the set should be stored using both data structures. Any operation requiring lookup of an item should use the hash table, and any operation requiring iteration through the items should use the linked list.

11. The following figure is called a *graph*. The circles are called *nodes* and the lines are called *edges*. An edge connects two nodes. You can interpret the graph as a maze of rooms and passages. The nodes can be thought of as rooms, and an edge connects one room to another. Note that each node has at most four edges in the graph that follows.



Write a program that implements the previous maze using references to instances of a `Node` class. Each node in the graph will correspond to an instance of `Node`. The edges correspond to links that connect one node to another and can be represented in `Node` as instance variables that reference another `Node` class. Start the user in node A. The user's goal is to reach the finish in node L. The program should output possible moves in the north, south, east, or west direction. Sample execution is shown next.

You are in room A of a maze of twisty little passages, all alike. You can go east or south.

E

You are in room B of a maze of twisty little passages, all alike. You can go west or south.

S

You are in room F of a maze of twisty little passages, all alike. You can go north or east.

E



Collections, Maps, and Iterators **16**

16.1 COLLECTIONS 936

Wildcards 938

The Collection Framework 938

Concrete Collection Classes 946

Differences between `ArrayList<T>` and
`Vector<T>` 956

Nonparameterized Version of the Collection
Framework ★ 956

16.2 MAPS 957

Concrete Map Classes 960

16.3 ITERATORS 964

The Iterator Concept 964

The `Iterator<T>` Interface 964

List Iterators 968

16 Collections, Maps, and Iterators

Science is built up with facts, as a house is with stones. But a collection of facts is no more science than a heap of stones is a house.

Ch. IX: Hypotheses in Nature, as translated by George Bruce Halsted, 1913.

Introduction

collection

iterator

A **collection** is a data structure for holding elements. For example, an `ArrayList<T>` object is a collection. Java has a repertoire of interfaces and classes that give a uniform treatment of collections. An **iterator** is an object that cycles through all the elements in a collection. In this chapter, we discuss collections and iterators.

Prerequisites

Sections 16.1 to 16.3 can be considered one single large topic. These three sections require Chapters 1 through 9, Section 13.1 of Chapter 13, which covers interfaces, Chapter 14 on generics and the `ArrayList<T>` class, and Chapter 15 on linked data structures. The material on inner classes in Chapter 13 (Sections 13.2 and 13.3) is not needed except for a brief reference in the Programming Tip entitled “Defining Your Own Iterator Classes,” which requires Section 13.2 (but not 13.3).

None of the material in this chapter is needed to understand Swing and GUIs. So, you may skip this and go directly to Chapter 17 if you prefer to cover Swing GUIs before considering the material of this chapter.

16.1 Collections

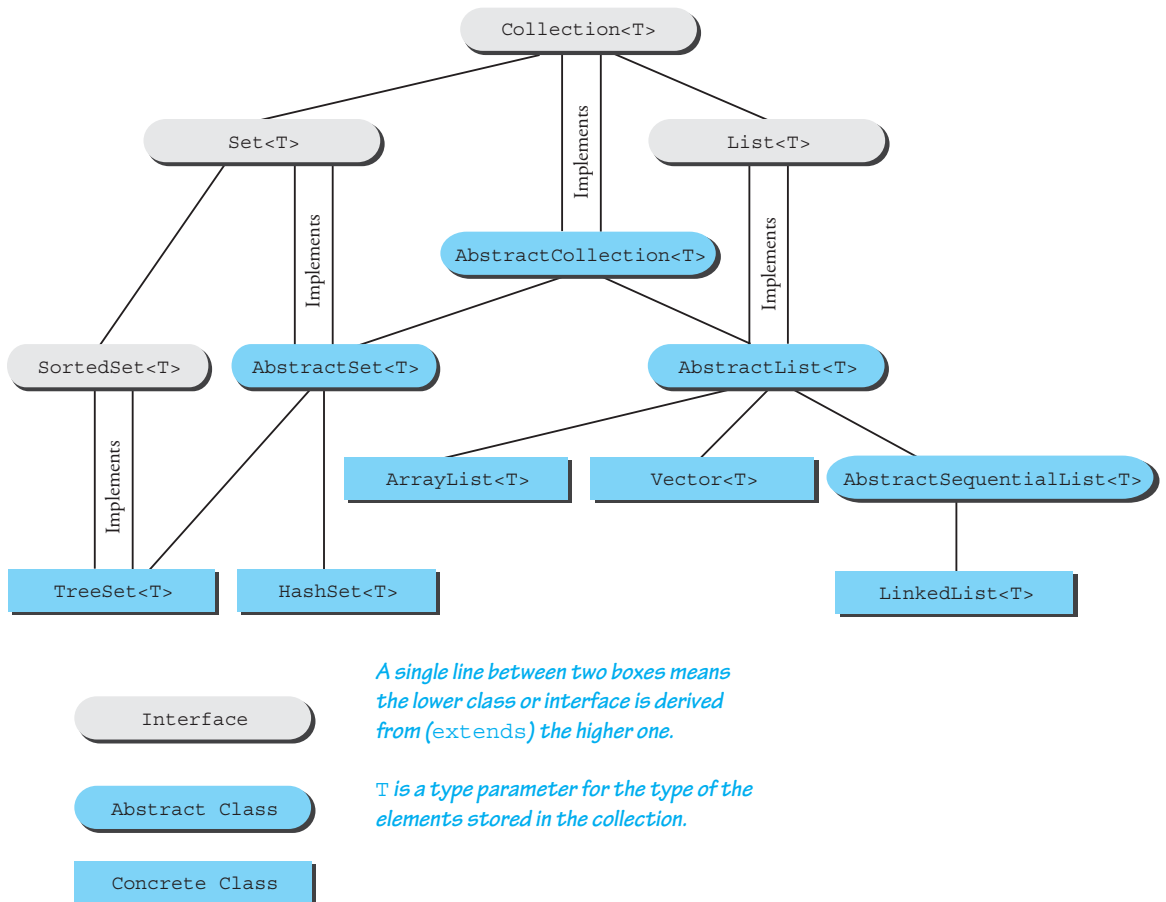
*Put all your eggs in one basket and
—WATCH THAT BASKET.*

MARK TWAIN, Pudd'nhead Wilson, Charles L. Webster & Company, 1894.

A Java collection is a class that holds objects. This concept is made precise by the `Collection<T>` interface. A Java collection is any class that implements the `Collection<T>` interface. As we shall see, many of these classes can be used as predefined data structures similar to those we defined ourselves in Chapter 15. One example of a Java collection class, which you saw in Chapter 14, is the `ArrayList<T>` class. The `Collection<T>` interface allows you to write code that applies to all Java collections so that you do not have to rewrite the code for each specific collection type. There are other interfaces and abstract classes that are in

some sense or another produced from the `Collection<T>` interface. Some of these are shown in Display 16.1. In this section, we give you an introduction to this Java collection framework. The topic is too large to treat exhaustively in this book, so this can only be an introductory treatment.

Display 16.1 The Collection Landscape



Collections are used along with *iterators*, which are discussed in Section 16.3. Separating collections and iterators into two sections turns out to be a handy way of organizing the material, but the two topics are intimately intertwined. In practice, you normally use them together.

Before we discuss the `Collection<T>` interface, we need a brief detour to learn a bit more about parameter type specifications.

Wildcards

Classes and interfaces in the collection framework use some parameter type specifications that we have not seen before. For example, they allow you to say things such as, “The argument must be a `ArrayList<T>`, but it can have any base type.” More generally these new parameter type specifications use generic classes but do not fully specify the type plugged in for the type parameter. Because they specify a wide range of argument types, they are known as **wildcards**.

wildcard<?>

The easiest wildcard to understand is `<?>`, which says that you can use any type in place of the type parameter. For example,

```
public void sampleMethod(String arg1, ArrayList<?> arg2)
```

is invoked with two arguments. The first argument must be of type `String`. The second argument can be a `ArrayList<T>` with any base type.

Note that `ArrayList<?>` is different from `ArrayList<Object>`. For example, if the type specification is `ArrayList<?>`, then you can plug in an argument of type `ArrayList<String>` (as well as other types); you cannot plug in an argument of type `ArrayList<String>` if the type specification is `ArrayList<Object>`.

extends

You can place a bound on a wildcard saying the type used in place of the wildcard must be an ancestor type or a descendent type of some class or interface. For example, `<? extends String>` says that the argument plugged in can be an object of any descendent class of the class `String`. The notation, restrictions, and meaning are the same as what we described for type bounds such as `<T extends String>`, which we discussed in Chapter 14.

For example,

```
public void
    anotherMethod(String arg1, ArrayList<? extends String> arg2)
```

is invoked with two arguments. The first argument must be of type `String`, but the second argument can be of any `ArrayList<T>` object provided the base type of the `ArrayList<T>` is a descendent type of `String`.

super

To specify that the wildcard type be an ancestor type of some class or interface, use `super` rather than `extends`. For example, `ArrayList<? super String>` specifies an `ArrayList<T>` whose base type can be any ancestor class of the class `String`. As it turns out, we will have no occasion to use wildcard types involving `super`.

The Collection Framework

Collection<T>
interface

The `Collection<T>` interface is the highest level of Java’s framework for collection classes. This interface describes the basic operations that all collection classes should implement. A summary of these operations (method headings) for the `Collection<T>` interface are given in Display 16.2. A more complete description can be found in Appendix 5. Because an interface is a type, you can define methods with a parameter of type `Collection<T>`. That parameter can be filled in with an argument that is an object of any class in the collection framework (that is, any class that implements the

`Collection<T>` interface). This turns out to be a very powerful tool. Let us explore the possibilities. So far, we have seen one class that implements the `Collection<T>` interface, namely the class `ArrayList<T>`. In addition to the methods given in Chapter 14 for the `ArrayList<T>` class, the `ArrayList<T>` class also implements all the methods given in Display 16.2. There are a number of different predefined classes that implement the `Collection<T>` interface, and you can define your own classes to do this. If you write a method to manipulate a parameter of type `Collection<T>`, it will work for all of these classes. Also, the methods in the `Collection<T>` interface ensure that you can intermix the use of different collection classes. For example, consider the method

```
public boolean containsAll(Collection<?> collectionOfTargets)
```

You can use this with two `ArrayList<T>` objects (the calling object and the argument) to see if one contains all the elements of the other. The two `ArrayList<T>` objects do not even have to have the same base type. Moreover, you can also use it with an `ArrayList<T>` object and an object of any other class that implements the `Collection<T>` interface to compare the elements in these two different kinds of `Collection<T>` objects.

Display 16.2 Method Headings in the `Collection<T>` Interface (part 1 of 3)

The `Collection<T>` interface is in the `java.util` package.

CONSTRUCTORS

Although not officially required by the interface, any class that implements the `Collection<T>` interface should have at least two constructors: a no-argument constructor that creates an empty `Collection<T>` object, and a constructor with one parameter of type `Collection<? extends T>` that creates a `Collection<T>` object with the same elements as the constructor argument. The interface does not specify whether the copy produced by the one-argument constructor is a shallow copy or a deep copy of its argument.

METHODS

```
boolean isEmpty()
```

Returns true if the calling object is empty; otherwise returns false.

```
public boolean contains(Object target)
```

Returns true if the calling object contains at least one instance of `target`. Uses `target.equals` to determine if `target` is in the calling object.

```
public boolean containsAll(Collection<?> collectionOfTargets)
```

Returns true if the calling object contains all of the elements in `collectionOfTargets`. For an element in `collectionOfTargets`, this method uses `element.equals` to determine if `element` is in the calling object.

(continued)

Display 16.2 Method Headings in the `Collection<T>` Interface (part 2 of 3)

```
public boolean equals(Object other)
```

This is the `equals` of the collection, not the `equals` of the elements in the collection. Overrides the inherited method `equals`. Although there are no official constraints on `equals` for a collection, it should be defined as we have described in Chapter 7 and also to satisfy the intuitive notion of collections being equal.

```
public int size()
```

Returns the number of elements in the calling object. If the calling object contains more than `Integer.MAX_VALUE` elements, returns `Integer.MAX_VALUE`.

```
Iterator<T> iterator()
```

Returns an iterator for the calling object. (Iterators are discussed in Section 16.3.)

```
public Object[] toArray()
```

Returns an array containing all of the elements in the calling object. If the calling object makes any guarantees as to what order its elements are returned by its iterator, this method must return the elements in the same order.

The array returned should be a new array so that the calling object has no references to the returned array. (You might also want the elements in the array to be clones of the elements in the collection. However, this is apparently not required by the interface, because library classes, such as `Vector<T>`, return arrays that contain references to the elements in the collection.)

```
public <E> E[] toArray(E[] a)
```

Note that the type parameter `E` is not the same as `T`. So, `E` can be any reference type; it need not be the type `T` in `Collection<T>`. For example, `E` might be an ancestor type of `T`.

Returns an array containing all of the elements in the calling object. The argument `a` is used primarily to specify the type of the array returned. The exact details are described next.

The type of the returned array is that of `a`. If the elements in the calling object fit in the array `a`, then `a` is used to hold the elements of the returned array; otherwise a new array is created with the same type as `a`.

If `a` has more elements than the calling object, the element in `a` immediately following the end of the copied elements is set to `null`.

If the calling object makes any guarantees as to what order its elements are returned by its iterator, this method must return the elements in the same order. (Iterators are discussed in Section 16.3.)

```
public int hashCode()
```

Returns the hash code value for the calling object. The hash code is a numeric key that is ideally a unique identifier for the calling object. (Hash codes are discussed in Section 15.5.)

Display 16.2 Method Headings in the `Collection<T>` Interface (part 3 of 3)**OPTIONAL METHODS**

The following methods are optional, which means they still must be implemented, but the implementation can simply throw an `UnsupportedOperationException` if, for some reason, you do not want to give them a "real" implementation. An `UnsupportedOperationException` is a `RuntimeException` and so is not required to be caught or declared in a `throws` clause.

```
public boolean add(T element) (Optional)
```

Ensures that the calling object contains the specified `element`. Returns `true` if the calling object changes as a result of the call. Returns `false` if the calling object does not permit duplicates and already contains `element`; also returns `false` if the calling object does not change for any other reason.

```
public boolean addAll(Collection<? extends T> collectionToAdd) (Optional)
```

Ensures that the calling object contains all the elements in `collectionToAdd`. Returns `true` if the calling object changes as a result of the call; returns `false` otherwise.

```
public boolean remove(Object element) (Optional)
```

Removes a single instance of the element from the calling object, if it is present. Returns `true` if the calling object contained the element; returns `false` otherwise.

```
public boolean removeAll(Collection<?> collectionToRemove) (Optional)
```

Removes all the calling object's elements that are also contained in `collectionToRemove`. Returns `true` if the calling object is changed; otherwise returns `false`.

```
public void clear() (Optional)
```

Removes all the elements from the calling object.

```
public boolean retainAll(Collection<?> saveElements) (Optional)
```

Retains only the elements in the calling object that are also contained in the collection `saveElements`. In other words, removes from the calling object all of its elements that are not contained in the collection `saveElements`. Returns `true` if the calling object is changed; otherwise returns `false`.

Packages

All the collection classes and interfaces discussed in this chapter are in the `java.util` package.

**`Set<T>` and
`List<T>`
interfaces**

The relationships between some of the classes and interfaces that implement or extend the `Collection<T>` interface are given in Display 16.1. There are two main interfaces that extend the `Collection<T>` interface: the `Set<T>` interface and the `List<T>` interface. Classes that implement the `Set<T>` interface do not allow an

element in the class to occur more than once. Classes that implement the `List<T>` interface have their elements ordered on a list, so there is a zeroth element, a first element, a second element, and so forth. A class that implements the `List<T>` interface allows elements to occur more than once. The `ArrayList<T>` class implements the `List<T>` interface.

The `Set<T>` interface has the same method headings as the `Collection<T>` interface, but in some cases the semantics (intended meanings) are different. For example, the semantics of adding new elements to the set do not allow duplicates. The `add` methods are described in Display 16.3. A complete list of the `Set<T>` interface is given in Appendix 5.

Display 16.3 Adding Elements in the `Set<T>` Interface

The `Set<T>` interface is in the `java.util` package.

The `Set<T>` interface extends the `Collection<T>` interface and has all the same method headings given in Display 16.2. However, the semantics of the `add` methods vary as described below.

```
public boolean add(T element) (Optional)
```

If `element` is not already in the calling object, `element` is added to the calling object and `true` is returned. If `element` is in the calling object, the calling object is unchanged and `false` is returned.

```
public boolean addAll(Collection<? extends > collectionToAdd) (Optional)
```

Ensures that the calling object contains all the elements in `collectionToAdd`. Returns `true` if the calling object changed as a result of the call; returns `false` otherwise. Thus, if `collectionToAdd` is a `Set<T>`, then the calling object is changed to the union of itself with `collectionToAdd`.

The `List<T>` interface has more method headings than the `Collection<T>` interface, and some of the methods inherited from the `Collection<T>` interface receive somewhat different semantics. For example, the semantics of adding new elements to the set allow duplicates, and rules must be made about which element should be removed when there are duplicates. These methods, along with new method definitions, are described in Display 16.4. A complete list of the `List<T>` interface is given in Appendix 5.

Display 16.4 Selected Methods in the `List<T>` Interface (part 1 of 3)

The `List<T>` interface is in the `java.util` package.

The `List<T>` interface extends the `Collection<T>` interface.

ADDING AND REMOVING ELEMENTS

```
public boolean add(T element) (Optional)
```

Adds `element` to the end of the calling object's list. Normally returns `true`. Returns `false` if the operation failed, but if the operation failed, something is seriously wrong and you will probably get a run-time error anyway.

Display 16.4 Selected Methods in the `List<T>` Interface (part 2 of 3)

```
public boolean addAll(Collection<? extends T> collectionToAdd) (Optional)
```

Adds all of the elements in `collectionToAdd` to the end of the calling object's list. The elements are added in the order they are produced by an iterator for `collectionToAdd`.

```
public boolean remove(Object element) (Optional)
```

Removes the first occurrence of `element` from the calling object's list, if it is present. Returns `true` if the calling object contained the element; returns `false` otherwise.

```
public boolean removeAll(Collection<?> collectionToRemove) (Optional)
```

Removes all the calling object's elements that are also in `collectionToRemove`. Returns `true` if the calling object was changed; otherwise returns `false`.

NEW METHOD HEADINGS

The following methods are in the `List<T>` interface but were not in the `Collection<T>` interface. Those that are optional are noted.

```
public void add(int index, T newElement) (Optional)
```

Inserts `newElement` in the calling object's list at location `index`. The old elements at location `index` and higher are moved to higher indices.

```
public boolean addAll(int index,
                     Collection<? extends T> collectionToAdd) (Optional)
```

Inserts all of the elements in `collectionToAdd` to the calling object's list starting at location `index`. The old elements at location `index` and higher are moved to higher indices. The elements are added in the order they are produced by an iterator for `collectionToAdd`.

```
public T get(int index)
```

Returns the object at position `index`.

```
public T set(int index, T newElement) (Optional)
```

Sets the element at the specified index to `newElement`. The element previously at that position is returned.

```
public T remove(int index) (Optional)
```

Removes the element at position `index` in the calling object. Shifts any subsequent elements to the left (subtracts one from their indices). Returns the element that was removed from the calling object.

```
public int indexOf(Object target)
```

Returns the index of the first element that is equal to `target`. Uses the method `equals` of the object `target` to test for equality. Returns `-1` if `target` is not found.

(continued)

Display 16.4 Selected Methods in the `List<T>` Interface (part 3 of 3)

```
public int lastIndexOf(Object target)
```

Returns the index of the last element that is equal to `target`. Uses the method `equals` of the object `target` to test for equality. Returns `-1` if `target` is not found.

```
public List<T> subList(int fromIndex, int toIndex)
```

Returns a *view* of the elements at locations `fromIndex` to `toIndex` of the calling object; the object at `fromIndex` is included; the object, if any, at `toIndex` is not included. The *view* uses references into the calling object; so, changing the view can change the calling object. The returned object will be of type `List<T>` but need not be of the same type as the calling object. Returns an empty `List<T>` if `fromIndex` equals `toIndex`.

```
ListIterator<T> listIterator()
```

Returns a list iterator for the calling object. (Iterators are discussed in Section 16.3.)

```
ListIterator<T> listIterator(int index)
```

Returns a list iterator for the calling object starting at `index`. The first element to be returned by the iterator is the one at `index`. (Iterators are discussed in Section 16.3.)

Collection Interfaces

The primary interfaces for collection classes are the `Collection<T>`, `Set<T>`, and `List<T>` interfaces. Both the `Set<T>` and the `List<T>` interfaces are derived from the `Collection<T>` interface. The `Set<T>` interface is for collections that do not allow repetition of elements and do not impose an order on their elements. The `List<T>` interface is for collections that do allow repetition of elements and do impose an order on their elements.

For-Each Loops

You can use a for-each loop with any of the collections discussed in this chapter.



PITFALL: Optional Operations

What is the point of an optional method heading in an interface? The whole purpose of an interface is to specify what methods can be used with an object of the interface type so that you can write code for an arbitrary object of the interface type. The reasoning behind these optional methods is that they normally would be implemented, but in unusual situations a programmer may leave them “unsupported.” (The alternative would be to have two interfaces, one with and one without the optional operations. Uncharacteristically, Java designers opted for a smaller number of interfaces.) But there is still more to the story.

**PITFALL: (continued)**

The optional methods are not, strictly speaking, optional. Like the other methods in an interface, the optional methods must have a method body so that the optional method heading is converted to a complete method definition. So, what is optional? The “optional” refers to the semantics of the method. If the method is optional, then you may give it a trivial implementation, and you will not have shirked your responsibility to follow the (unenforced) semantics for the interface.

To keep these optional methods from producing unexplained failures, the interface semantics say that if you do not give an optional method a “real” implementation, then you should have the method body throw an `UnsupportedOperationException`. For example, the `add` method of the `Collection<T>` interface is optional and so can be implemented as follows (provided you have good reason for this):

```
public boolean add(T element)
{
    throw new UnsupportedOperationException();
}
```

The `UnsupportedOperationException` class is a derived class of the `RuntimeException` class, so an `UnsupportedOperationException` is an unchecked exception, meaning it need not be caught in a `catch` block or declared in a `throws` clause.

The intention is that the code for a class that implements an interface with optional methods would be written and used in such a way that this `UnsupportedOperationException` would only be thrown during debugging. These rules on optional methods are part of the semantics of the interface, and like all other parts of the semantics of an interface, they depend entirely on the good will and responsibility of the programmer defining the class that implements the interface. ■

Optional Methods

When an interface lists a method as “optional,” you still need to implement it when defining a class that implements the interface. However, if you do not want to give it a “real” definition, you can simply have the method body throw an `UnsupportedOperationException`.

**TIP: Dealing with All Those Exceptions**

If you examine the `Collection<T>`, `Set<T>`, and `List<T>` interfaces in Appendix 5, you will see that many of the methods are liberally sprinkled with statements that certain exceptions are thrown. All these exception classes are unchecked exceptions, meaning that they need not be caught in a `catch` block and need not be declared in a `throws` clause. They are there primarily for debugging. If you are using an existing

(continued)

**TIP: (continued)**

collection class, you can view them as run-time error messages. If you are defining a class as a derived class of some other collection class, then most or all of the exception throwing will be inherited, so you need not worry too much about it. If you are defining a collection class from scratch and want your class to implement one of the collection interfaces, then you do need to throw suitable exceptions as specified for the interface.

With one exception (no pun intended), all the exception classes mentioned in this chapter are in the package `java.lang` and so do not require any import statement. The one exception is the `NoSuchElementException`, which is used with iterators in Section 16.3. The `NoSuchElementException` is in the `java.util` package, which requires an import statement if your code mentions the `NoSuchElementException` class. ■

Self-Test Exercises

1. Give the definition of a `boolean` valued static generic method named `inSome`. The method `inSome` has two parameters of type `Collection<T>` and one parameter of type `T`. The method returns `true` if the parameter of type `T` is in either (or both) collections; it returns `false` otherwise.
2. Give the definition of a static generic method named `getFirst` that has one parameter of type `List<T>` and a return type of `T`. The method returns the first element in the list or `null` if the list is empty.
3. Give the definition of a static `boolean` valued method named `noNull`. The method `noNull` has one parameter of type `Set<?>` and removes `null` from the set if `null` is in the set; otherwise it leaves the set unchanged. The method returns `true` if the set is changed and `false` if it is not changed.

Concrete Collection Classes

**Abstract
Set<T>**

**Abstract
List<T>**

The abstract classes `AbstractSet<T>` and `AbstractList<T>` are there for convenience when implementing the `Set<T>` and `List<T>` interfaces, respectively. They have almost no methods beyond those in the interfaces they implement. Although these two abstract classes have only a few abstract methods, the other (nonabstract) methods have fairly useless implementations that must be overridden. When defining a derived class of either `AbstractSet<T>` or `AbstractList<T>`, you need to define not just the abstract methods but also all the methods you intend to use. It usually makes more sense to simply use (or define derived classes of) the `HashSet<T>`, `ArrayList<T>`, or `Vector<T>` classes, which are derived classes of `AbstractSet<T>` and `AbstractList<T>` and are full implementations of the `Set<T>` and `List<T>` interfaces.

`Abstract
Collection<T>`

The abstract class `AbstractCollection<T>` is a skeleton class for the `Collection<T>` interface. Although it is perfectly legal, you seldom, if ever, need to define a derived class of the `AbstractCollection<T>` class. Instead, you normally define a derived class of one of the descendent classes of the `AbstractCollection<T>` class.

`HashSet<T>`

If you want a class that implements the `Set<T>` interface and do not need any methods beyond those in the `Set<T>` interface, you can use the concrete class `HashSet<T>`. So, after all is said and done, if all you need is a collection class that does not allow elements to occur more than once, then you can use the `HashSet<T>` class and need not worry about all the other classes and interfaces in Display 16.1. The word *Hash* refers to the fact that the `HashSet<T>` class is implemented using a hash table, which was introduced in Section 15.5. The `HashSet<T>`, of course, implements all the methods in the `Set<T>` interface and adds no other methods beyond constructors. A summary of the `HashSet<T>` constructors and other methods is given in Display 16.5. If you want to define your own class that implements the `Set<T>` interface, you are probably better off using the `HashSet<T>` class rather than the `AbstractSet<T>` class as a base class.

Display 16.5 Methods in the `HashSet<T>` Class

The `HashSet<T>` class is in the `java.util` package.

The `HashSet<T>` class extends the `AbstractSet<T>` class and implements the `Set<T>` interface. The `HashSet<T>` class implements all of the methods in the `Set<T>` interface (Display 16.3). The only other methods in the `HashSet<T>` class are the constructors. The three constructors that do not involve concepts beyond the scope of this book are given next.

All the exception classes mentioned are the kind that are not required to be caught in a `catch` block or declared in a `throws` clause.

All the exception classes mentioned are in the package `java.lang` and so do not require any import statement.

```
public HashSet ()
```

Creates a new, empty set.

```
public HashSet (Collection<? extends T> c)
```

Creates a new set that contains all the elements of `c`. Throws a `NullPointerException` if `c` is null.

```
public HashSet (int initialCapacity)
```

Creates a new, empty set with the specified capacity.

Throws an `IllegalArgumentException` if `initialCapacity` is less than zero.

The methods are the same as those described for the `Set<T>` interface (Display 16.3).



VideoNote

Using
HashMap
with a
Custom Class

It is important to note that if you intend to use the `HashSet<T>` class with your own class as the parameterized type `T`, then your class must override the following methods:

```
public int hashCode();
public boolean equals(Object obj);
```

The `hashCode()` method should return a numeric key that is ideally a unique identifier for an object in your class. See Section 15.5 for a discussion about hash codes. It is always a good idea to override the `equals()` method for any class you write, but you must override it in this scenario. Java will use the hash code to index the object and then use the `equals()` method to check if an object exists in the set. If the hash code for two different objects is identical, the objects will still be indexed correctly as long as `equals()` indicates they are unique. However, the identical hash codes will cause a collision that will decrease performance.

Display 16.6 shows a sample program that uses the `HashSet<T>` class. This program is conceptually similar to the program in Display 15.38, in which sets containing strings of round things and green things were manipulated in various

Display 16.6 `HashSet<T>` Class Demo (part 1 of 3)

```
1  import java.util.HashSet;
2  import java.util.Iterator;
3  public class HashSetDemo
4  {
5      private static void outputSet(HashSet<String> set) The outputSet
6      { method uses an iterator
7          Iterator<String> i = set.iterator(); to print the contents of
8          while (i.hasNext()) a HashSet<T> object.
9              System.out.print(i.next() + " "); Iterators are described
10         System.out.println(); in Section 16.3.
11     }
12
13     public static void main(String[] args)
14     {
15         HashSet<String> round = new HashSet<String>();
16         HashSet<String> green = new HashSet<String>();
17
18         // Add some data to each set
19         round.add("peas");
20         round.add("ball");
21         round.add("pie");
22         round.add("grapes");
23
24         green.add("peas");
25         green.add("grapes");
26         green.add("garden hose");
27         green.add("grass");
```

Display 16.6 `HashSet<T>` Class Demo (part 2 of 3)

```
25         System.out.println("Contents of set round: ");
26         outputSet(round);
27         System.out.println("\nContents of set green: ");
28         outputSet(green);

29         System.out.println("\nball in set 'round'? " +
30             round.contains("ball"));
31         System.out.println("ball in set 'green'? " +
32             green.contains("ball"));

33         System.out.println("\nball and peas in same set? "+
34             ((round.contains("ball") &&
35             (round.contains("peas")) ||
36             (green.contains("ball") &&
37             (green.contains("peas")))));
38         System.out.println("pie and grass in same set? "+
39             ((round.contains("pie") &&
40             (round.contains("grass")) ||
41             (green.contains("pie") &&
42             (green.contains("grass")))));

43         // To union two sets we use the addAll method.
44         HashSet<String>setUnion = new HashSet<String>(round);
45         round.addAll(green);
46         System.out.println("\nUnion of green and round:");
47         outputSet(setUnion);

48         // To intersect two sets we use the removeAll method.
49         HashSet<String> setInter = new HashSet<String>(round);
50         setInter.removeAll(green);
51         System.out.println("\nIntersection of green and round:");
52         outputSet(setInter);
53         System.out.println();
54     }
55 }
```

Sample Dialogue

```
Contents of set round:
grapes pie ball peas

Contents of set green:
grass garden hose grapes peas
```

(continued)

Display 16.6 `HashSet<T>` Class Demo (part 3 of 3)

```
ball in set round? true
ball in set green? false

ball and peas in same set? True
pie and grass in same set? false

Union of green and round:
garden hose grass peas ball pie grapes

Intersection of green and round:
peas grapes
```

ways using our own `Set<T>` class implemented with linked lists. However, the code listing in Display 16.6 uses the `HashSet<T>` class in place of our custom `Set<T>` class. Nevertheless, most of the code is identical because the `Set<T>` class was designed to have an interface similar to the `HashSet<T>` class. Both have `add` and `contains` methods. Functionality similar to our `union` and `intersection` methods can be achieved by using the `HashSet<T>` `addAll` and `removeAll` methods. To output the items in a `HashSet<T>` object, we define an `outputSet` method. This method uses iterators, which are not discussed until Section 16.3, so for now you can ignore the details of how `outputSet` works.

In general, it is recommended that you use the collection classes unless they do not provide the functionality you need for your program. For example, say that you want every item added to the set to have a reference to the set that contains it. This could be useful if you want to determine whether two items are in the same set—you could just follow the reference to the containing set for each item and see whether they are the same. Without such a reference, you would have to invoke the `contains` method for every set to learn whether the items are in the same set. If this were an important feature for your program, you might want to develop your own class instead of using one of the collection classes. If the collection classes were sufficient, the result would be shorter code that is generally easier to develop and maintain. Moreover, the collection classes such as `HashSet<T>` have been designed with efficiency and scalability in mind.

`Vector<T>` If you want a class that implements the `List<T>` interface and do not need any methods beyond those in this interface, you can use the `ArrayList<T>` or `Vector<T>` class. So, after all is said and done, if all you need is a collection class that allows elements to occur more than once, or you need a collection that orders its elements as on a list (that is, as in an array), or you need a class that has both of these properties, then you can use the `ArrayList<T>` or `Vector<T>` class and need not worry about all the other classes and interfaces in Display 16.1. The `ArrayList<T>` and `Vector<T>` classes implement all the methods in the `List<T>` interface. A table of methods for the `ArrayList<T>` class was given in Chapter 14 and a more complete table is given in

Display 16.7. A table of methods for the `Vector<T>` class is also given in Display 16.7. A more complete list of the methods in these classes is given in Appendix 5. If you want to define your own class that implements the `List<T>` interface, you would probably be better off using either the `ArrayList<T>` or the `Vector<T>` class rather than the `AbstractList<T>` class as a base class.

**Abstract
Sequential
List<T>**

The abstract class `AbstractSequentialList<T>` is derived from the `AbstractList<T>` class. Although it does override some methods inherited from the class `AbstractList<T>`, it adds no completely new methods. The point of the `AbstractSequentialList<T>` class is to provide for efficient implementation of sequentially moving through the list at the expense of having inefficient implementation of random access to elements (that is, inefficient implementation of the `get` method). The `LinkedList<T>` class is a concrete derived class of the abstract class `AbstractSequentialList<T>`. (The implementation of the `LinkedList<T>` class is similar to that of the linked list classes we discussed in Chapter 15.) If you need a `List<T>` with efficient random access to elements (that is, efficient implementation of

LinkedList<T>

Display 16.7 Methods in the Classes `ArrayList<T>` and `Vector<T>` (part 1 of 4)

The `ArrayList<T>` and `Vector<T>` classes and the `Iterator<T>` and `ListIterator<T>` interfaces are in the `java.util` package.

All the exception classes mentioned are unchecked exceptions, which means they are not required to be caught in a catch block or declared in a throws clause. (If you have not yet studied exceptions, you can consider the exceptions to be run-time error messages.)

`NoSuchElementException` is in the `java.util` package, which requires an import statement if your code mentions the `NoSuchElementException` class. All the other exception classes mentioned are in the package `java.lang` and so do not require any import statement.

CONSTRUCTORS

```
public ArrayList(int initialCapacity)
```

Creates an empty `ArrayList<T>` with the specified initial capacity. When the `ArrayList<T>` needs to increase its capacity, the capacity doubles.

```
public ArrayList()
```

Creates an empty `ArrayList<T>` with an initial capacity of 10. When the `ArrayList<T>` needs to increase its capacity, the capacity doubles.

```
public ArrayList(Collection<? extends T> c)
```

Creates an `ArrayList<T>` that contains all the elements of the collection `c`, in the same order. In other words, the elements have the same index in the `ArrayList<T>` created as they do in `c`. This is not quite a true copy constructor because it does not preserve capacity. The capacity of the created list will be `c.size()`, not `c.capacity`.

The `ArrayList<T>` created is only a shallow copy of the collection argument. The `ArrayList<T>` created contains references to the elements in `c` (not references to clones of the elements in `c`).

```
public Vector(int initialCapacity)
```

Creates an empty vector with the specified initial capacity. When the vector needs to increase its capacity, the capacity doubles.

(continued)

Display 16.7 Methods in the Classes `ArrayList<T>` and `Vector<T>` (part 2 of 4)

```
public Vector()
```

Creates an empty vector with an initial capacity of 10. When the vector needs to increase its capacity, the capacity doubles.

```
public Vector(Collection<? extends T> c)
```

Creates a vector that contains all the elements of the collection `c`, in the same order. In other words, the elements have the same index in the vector created as they do in `c`. This is not quite a true copy constructor because it does not preserve capacity. The capacity of the created vector will be `c.size()`, not `c.capacity`.

The vector created is only a shallow copy of the collection argument. The vector created contains references to the elements in `c` (not references to clones of the elements in `c`).

```
public Vector(int initialCapacity, int capacityIncrement)
```

Constructs an empty vector with the specified initial capacity and capacity increment. When the vector needs to grow, it will add room for `capacityIncrement` more items.

(`ArrayList<T>` does not have a corresponding constructor.)

ARRAYLIKE METHODS FOR BOTH `ArrayList<T>` AND `Vector<T>`

```
public T set(int index, T newElement)
```

Sets the element at the specified index to `newElement`. The element previously at that position is returned. If you draw an analogy to an array `a`, this is analogous to setting `a[index]` to the value `newElement`. The `index` must be a value greater than or equal to zero and strictly less than the current size of the list.

```
public T get(int index)
```

Returns the element at the specified index. This is analogous to returning `a[index]` for an array `a`. The `index` must be a value greater than or equal to 0 and less than the current size of the calling object.

METHODS TO ADD ELEMENTS FOR BOTH `ArrayList<T>` AND `Vector<T>`

```
public boolean add(T newElement)
```

Adds `newElement` to the end of the calling object's list and increases its size by one. The capacity of the calling object is increased if that is required. Returns `true` if the add was successful. This method is often used as if it were a void method.

```
public void add(int index, T newElement)
```

Inserts `newElement` as an element in the calling object at the specified index and increases the size of the calling object by one. Each element in the calling object with an index greater than or equal to `index` is shifted upward to have an index that is one greater than the value it had previously.

The `index` must be a value greater than or equal to zero and less than or equal to the size of the calling object (before this addition).

Note that you can use this method to add an element after the last current element. The capacity of the calling object is increased if that is required.

Display 16.7 Methods in the Classes `ArrayList<T>` and `Vector<T>` (part 3 of 4)

```
public boolean addAll(Collection<? extends T> c)
```

Appends all the elements in `c` to the end of the elements in the calling object in the order that they are enumerated by a `c` iterator. The behavior of this method is not guaranteed if the collection `c` is the calling object or any collection including the calling object either directly or indirectly.

```
public boolean addAll(int index, Collection<? extends T> c)
```

Inserts all the elements in `c` into the calling object starting at position `index`. Elements are inserted in the order that they are enumerated by a `c` iterator. Elements previously at positions `index` or higher are shifted to higher numbered positions.

METHODS TO REMOVE ELEMENTS FOR BOTH `ArrayList<T>` AND `Vector<T>`

```
public T remove(int index)
```

Deletes the element at the specified index and returns the element deleted. The size of the calling object is decreased by one. The capacity of the calling object is not changed. Each element in the calling object with an index greater than or equal to `index` is decreased to have an index that is one less than the value it had previously.

The `index` must be a value greater than or equal to zero and less than the size of the calling object (before this removal).

```
public boolean remove(Object theElement)
```

Removes the first occurrence of `theElement` from the calling object. If `theElement` is found in the calling object, then each element in the calling object with an index greater than or equal to `theElement`'s index is decreased to have an index that is one less than the value it had previously. Returns `true` if `theElement` was found (and removed). Returns `false` if `theElement` is not found in the calling object. If the element was removed, the size is decreased by one. The capacity is not changed.

```
protected void removeRange(int fromIndex, int toIndex)
```

Removes all elements with index greater than or equal to `fromIndex` and strictly less than `toIndex`. Be sure to note that this method is `protected`, not `public`.

```
public void clear()
```

Removes all elements from the calling object and sets its size to zero.

SEARCH METHODS FOR BOTH `ArrayList<T>` AND `Vector<T>`

```
public boolean isEmpty()
```

Returns `true` if the calling object is empty (that is, has size 0); otherwise returns `false`.

```
public boolean contains(Object target)
```

Returns `true` if `target` is an element of the calling object; otherwise returns `false`. Uses the method `equals` of the object `target` to test for equality.

```
public int indexOf(Object target)
```

Returns the index of the first element that is equal to `target`. Uses the method `equals` of the object `target` to test for equality. Returns 1 if `target` is not found.

```
public int lastIndexOf(Object target)
```

Returns the index of the last element that is equal to `target`. Uses the method `equals` of the object `target` to test for equality. Returns 1 if `target` is not found.

(continued)

Display 16.7 Methods in the Classes `ArrayList<T>` and `Vector<T>` (part 4 of 4)**ITERATORS FOR BOTH `ArrayList<T>` AND `Vector<T>`**

```
public Iterator<T> iterator()
```

Returns an iterator for the calling object. Iterators are discussed in Section 16.3.

```
public ListIterator<T> listIterator()
```

Returns a `ListIterator<T>` for the calling object. `ListIterator<T>` is discussed in Section 16.3.

```
ListIterator<T> listIterator(int index)
```

Returns a list iterator for the calling object starting at `index`. The first element to be returned by the iterator is the one at `index`. (Iterators are discussed in Section 16.3.)

CONVERTING TO AN ARRAY FOR BOTH `ArrayList<T>` AND `Vector<T>`

```
public Object[] toArray()
```

Returns an array containing all of the elements in the calling object. The elements of the array are indexed the same as in the calling object.

```
public <E> E[] toArray(E[] a)
```

Note that the type parameter `E` is not the same as `T`. So, `E` can be any reference type; it need not be the type `T` in `Collection<T>`. For example, `E` might be an ancestor type of `T`.

Returns an array containing all of the elements in the calling object. The elements of the array are indexed the same as in the calling object.

The argument `a` is used primarily to specify the type of the array returned. The exact details are described next.

The type of the returned array is that of `a`. If the collection fits in the array `a`, then `a` is used to hold the elements of the returned array; otherwise a new array is created with the same type as `a`.

If `a` has more elements than the calling object, then the element in `a` immediately following the end of the elements copied from the calling object is set to `null`.

MEMORY MANAGEMENT FOR BOTH `ArrayList<T>` AND `Vector<T>`

```
public int size()
```

Returns the number of elements in the calling object.

```
public int capacity()
```

Returns the current capacity of the calling object.

```
public void ensureCapacity(int newCapacity)
```

Increases the capacity of the calling object to ensure that it can hold at least `newCapacity` elements. Using `ensureCapacity` can sometimes increase efficiency, but it is not needed for any other reason.

```
public void trimToSize()
```

Trims the capacity of the calling object to be the calling object's current size. This is used to save storage.

MAKE A COPY FOR BOTH `ArrayList<T>` AND `Vector<T>`

```
public Object clone()
```

Returns a shallow copy of the calling object.

the `get` method), then use the `ArrayList<T>` or `Vector<T>` class or a class derived from one of these two classes. If you do not need efficient random access but need to efficiently move sequentially through the list, then use the `LinkedList<T>` class or a class derived from the `LinkedList<T>` class.

`SortedSet<T>`

`TreeSet<T>`

The interface `SortedSet<T>` and the concrete class `TreeSet<T>` are designed for implementations of the `Set<T>` interface that provide for rapid retrieval of elements (efficient implementation of the `contains` and similar methods). The implementation of the class is similar to the binary tree class discussed in Chapter 15 but with more sophisticated ways to do inserting that keep the tree balanced. We will not discuss the `SortedSet<T>` interface or the `TreeSet<T>` class in this text, but you should be aware of their existence so you know what to look for in the Java documentation should you need them.

Self-Test Exercises

4. Can an object that instantiates the `HashSet<T>` class contain multiple copies of some element?
5. Suppose you want to define a class that orders its elements like a `List<T>` but does not allow multiple occurrences of an element like a `Set<T>`. Would it be better to make it a derived class of the `ArrayList<T>` class or a derived class of the `HashSet<T>` class?
6. You would like to use the following class as the type in a `HashSet<T>` collection. What is missing and how would you fix it?

```
public class Customer
{
    private String name;
    private String address;
    public Customer(String newName, String newAddress)
    {
        name = newName;
        address = newAddress;
    }
    public String toString()
    {
        return name + " : " + address;
    }
}
```


Differences between `ArrayList<T>` and `Vector<T>`

For most purposes, `ArrayList<T>` and `Vector<T>` are equivalent. There are only minor differences between the classes. The methods that are in both classes are given in Display 16.7. The class `Vector<T>` has more methods than `ArrayList<T>` does; these methods are not given in Display 16.7. However, most of the extra methods are little more than alternate names for methods that are in both `ArrayList<T>` and `Vector<T>`. None of the methods in `Vector<T>` do anything that cannot easily be done with an `ArrayList<T>`. The class `ArrayList<T>` is reputed to be more efficient than `Vector<T>`. The biggest difference between these two classes is that `ArrayList<T>` is newer than `Vector<T>` and was created as part of the Java collection framework, whereas `Vector<T>` is an older class that was retrofitted with extra method names to make it fit into the collection framework. You are encouraged to use `ArrayList<T>` rather than `Vector<T>`. However, a lot of existing code uses `Vector<T>`, so you should be familiar with it.¹

Nonparameterized Version of the Collection Framework ★

Before version 5.0, Java did not have type parameters. So, the collection framework consisted of ordinary classes and interfaces, such as `Collection`, `List`, `ArrayList`, and so forth, all of which had no type parameters. Although this older collection framework has been supplanted by the new, generic version, the old version's classes and interfaces, without type parameters, are still in the standard libraries and in a lot of older code. There is no longer any need for the older classes and interfaces that do not have type parameters. They can sometimes be harder to use and are less versatile than the new, generic classes with type parameters. You should not use the older classes and interfaces without type parameters. However, you will often see them used in older code. When reading older code, you will not go too far wrong in thinking of `Collection` as meaning `Collection<Object>`, `ArrayList` as meaning `ArrayList<Object>`, and so forth. This is not, strictly speaking, correct. For example, the classes `ArrayList` and `ArrayList<Object>` are not the same, but they are very similar.

¹The biggest difference between `Vector<T>` and `ArrayList<T>` classes is that `Vector<T>` objects are *synchronized* while `ArrayList<T>` objects are not. Synchronization is briefly discussed in Chapter 19.



PITFALL: Omitting the `<T>`

If you omit `<T>` or a corresponding class name, such as using `ArrayList` instead of `ArrayList<String>`, then you may get a compiler error message. If you do get a compiler error message, it is likely to seem bewilderingly strange. The problem is that `ArrayList` and other class and interface names with `<T>` omitted actually mean something. (We do not have time to stop and explain what they mean, but a hint is given in the starred subsection “Nonparameterized Version of the Collection Framework.”) Your only defense against this pitfall is to be very careful; if you do get a bewildering compiler error message, look for a missing `<T>` or a missing `<Class_Name>`.

Sometimes a compiler warning message can be helpful when you make this mistake. If you get a warning that mentions a type case from a class name without a `<T>` to a class name with a `<T>` or with a `<Class_Name>`, look for an omitted `<T>` or an omitted `<Class_Name>`.

Finally, we should note that sometimes your code will compile and even run correctly if you omit the `<T>` from a class name in the collection framework. ■

16.2 Maps

A man has but one mother. But, a mother may have any number of sons.

Saying on a Wall Sampler.

map

The Java **map** framework is similar in character to the collection framework, except that it deals with collections of ordered pairs. Objects in the map framework can implement mathematical functions and relations and so can be used to construct database classes. Think of the pair as consisting of a key `k` (to search for) and an associated value `v`. For example, the key might be a student ID number and the value might be an object storing information about the student (such as the name, major, address, or phone number) associated with that ID number. Commonly used interfaces and classes in this framework are shown in Display 16.8. In this chapter, we will focus on the `Map<K, V>` interface, the `AbstractMap<K, V>` class, and the `HashMap<K, V>` class.

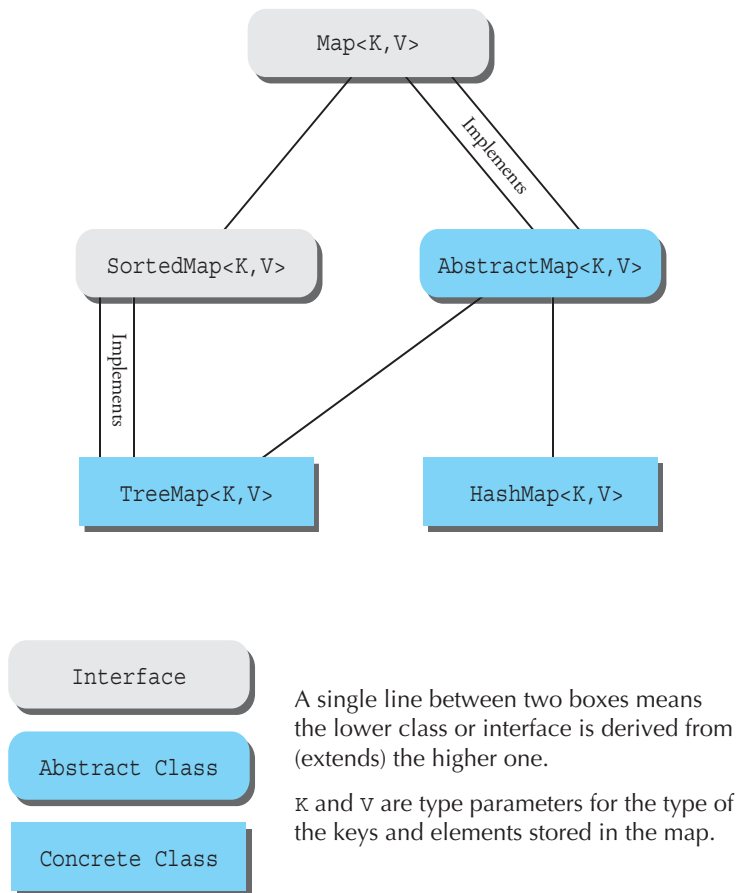
Because the map interface will map a key to a value, we must now specify two types of parameters instead of one as we did with collections. The `Map<K, V>` interface specifies the basic operations that all map classes should implement. A summary of these operations is given in Display 16.9. A more detailed description is in Appendix 5. Note that there are many similarities to the `Collection<T>` interface.

Abstract
`Map<K, V>`

Hash
`Map<K, V>`

`Map<K, V>`
interface

Display 16.8 The Map Landscape

Display 16.9 Method Headings in the `Map<K, V>` Interface (part 1 of 3)

The `Map<K, V>` interface is in the `java.util` package.

CONSTRUCTORS

Although not officially required by the interface, any class that implements the `Map<K, V>` interface should have at least two constructors: a no-argument constructor that creates an empty `Map<K, V>` object, and a constructor with one `Map<K, V>` parameter that creates a `Map<K, V>` object with the same elements as the constructor argument. The interface does not specify whether the copy produced by the one-argument constructor is a shallow copy or a deep copy of its argument.

Display 16.9 Method Headings in the `Map<K, V>` Interface (part 2 of 3)**METHODS**`boolean isEmpty()`

Returns `true` if the calling object is empty; otherwise returns `false`.

`public boolean containsValue(Object value)`

Returns `true` if the calling object contains at least one or more keys that map to an instance of `value`.

`public boolean containsKey(Object key)`

Returns `true` if the calling object contains `key` as one of its keys.

`public boolean equals(Object other)`

This is the `equals` of the map, not the `equals` of the elements in the map. Overrides the inherited method `equals`.

`public int size()`

Returns the number of (key, value) mappings in the calling object.

`public int hashCode()`

Returns the hash code value for the calling object.

`public Set<Map.Entry<K, V>> entrySet()`

Returns a set *view* consisting of (key, value) mappings for all entries in the map. Changes to the map are reflected in the set and vice versa.

`public Collection<V> values()`

Returns a collection *view* consisting of all values in the map. Changes to the map are reflected in the collection and vice versa.

`public V get(Object key)`

Returns the value to which the calling object maps `key`. If `key` is not in the map, then `null` is returned. Note that this does not always mean that the key is not in the map because it is possible to map a key to `null`. The `containsKey` method can be used to distinguish the two cases.

OPTIONAL METHODS

The following methods are optional, which means they still must be implemented, but the implementation can simply throw an `UnsupportedOperationException` if, for some reason, you do not want to give the methods a “real” implementation. An `UnsupportedOperationException` is a `RuntimeException` and so is not required to be caught or declared in a `throws` clause.

`public V put(K key, V value) (Optional)`

Associates `key` to `value` in the map. If `key` is associated with an existing value, then the old value is overwritten and returned. Otherwise `null` is returned.

(continued)

Display 16.9 Method Headings in the `Map<K, V>` Interface (part 3 of 3)

```
public void putAll(Map<? extends K, ? extends V> mapToAdd) (Optional)
```

Adds all mappings of `mapToAdd` into the calling object's map.

```
public V remove (Object key) (Optional)
```

Removes the mapping for the specified key. If the key is not found in the map, then `null` is returned; otherwise the previous value for the key is returned.

Concrete Map Classes

The abstract class `AbstractMap<K, V>` is convenient for implementing the `Map<K, V>` interface, just as the `AbstractSet<T>` class served the same purpose for the `Set<T>` interface. When defining a derived class of `AbstractMap<K, V>`, you need to define not just the abstract methods but also all the methods you intend to use. It usually makes more sense to use (or define derived classes of) the `HashMap<K, V>` or `TreeMap<K, V>` classes, which are derived classes of `AbstractMap<K, V>` and are full implementations of the `Map<K, V>` interfaces. However, if you wish to implement your own map with your own data structures, then it would be appropriate to derive classes from `AbstractMap<K, V>`.

In this chapter, we will focus only on the `HashMap<K, V>` class, which is a concrete implementation of the `Map<K, V>` interface. Internally, the class uses a hash table similar to what we discussed in Chapter 15. Note that this class does not make any guarantee as to the order of elements placed in the map. If you require order, then you should use the `TreeMap<K, V>` class (which internally uses a tree to store its elements) or the `LinkedHashMap<K, V>` class, which uses a doubly linked list to maintain order inside a `HashMap<K, V>` object. The `LinkedHashMap<K, V>` class is derived from the `HashMap<K, V>` class.

Knowing how hash tables operate is helpful in optimizing a program that uses a `HashMap`. When we created a hash table in Chapter 15, we used a fixed-sized array where each array entry referenced a linked list. A hash function mapped an input value, such as a `String`, to an index in the array. If the size of the array is much smaller than the number of elements added, then there will be lots of collisions and execution performance will be low. On the other hand, if the size of the array is much larger than the number of elements added, then memory will be wasted. A similar trade-off exists with the `HashMap<K, V>` class. One of the constructors allows us to specify an **initial capacity** and a **load factor**. The initial capacity specifies how many “buckets” exist in the hash table. This would be analogous to the size of the array of the hash table. The load factor is a number between 0 and 1. This variable specifies a percentage such that if the number of elements added to the hash table exceeds the load factor, then the capacity of the hash table will automatically increase. The default load factor is 0.75 and the default initial capacity is 16. This means that the capacity will be increased (by roughly double) once 12 elements are added to the map. This process is called **rehashing**; it can be time consuming if you have a large number of elements in the map. Although the capacity will automatically increase when necessary, your program will run more efficiently if the capacity is initially set to the number of elements you expect will be added to the map.

initial
capacity
load factor

rehashing

The `HashMap<K,V>`, of course, implements all the methods in the `Map<K,V>` interface and adds no other methods beyond constructors and an implementation of the `clone()` method. A summary of the `HashMap<K,V>` constructors and `clone()` is given in Display 16.10.

As with the `HashSet<T>` class, if you intend to use your own class as the parameterized type `K` in a `HashMap<K,V>` then your class must override the following methods:

```
public int hashCode();
public boolean equals(Object obj);
```

These methods are required for indexing and checking for uniqueness of the key. See the discussion in Section 16.1 about overriding these methods and Section 15.5 about hash functions.

Display 16.10 Methods in the `HashMap<K,V>` Class

The `HashMap<K,V>` class is in the `java.util` package.

The `HashMap<K,V>` class extends the `AbstractMap<K,V>` class and implements the `Map<K,V>` interface.

The `HashMap<K,V>` class implements all of the methods in the `Map<K,V>` interface (Display 16.9). The only other methods in the `HashMap<K,V>` class are the constructors.

All the exception classes mentioned are the kind that are not required to be caught in a `catch` block or declared in a `throws` clause.

All the exception classes mentioned are in the package `java.lang` and so do not require any import statement.

```
public HashMap()
```

Creates a new, empty map with a default initial capacity of 16 and load factor of 0.75.

```
public HashMap(int initialCapacity)
```

Creates a new, empty map with a default capacity of `initialCapacity` and load factor of 0.75. Throws an `IllegalArgumentException` if `initialCapacity` is negative.

```
public HashMap(int initialCapacity, float loadFactor)
```

Creates a new, empty map with the specified capacity and load factor. Throws an `IllegalArgumentException` if `initialCapacity` is negative or `loadFactor` is nonpositive.

```
public HashMap(Map<? extends K, ? extends V> m)
```

Creates a new map with the same mappings as `m`. The `initialCapacity` is set to the same size as `m` and the `loadFactor` to 0.75.

Throws a `NullPointerException` if `m` is null.

```
public Object clone()
```

Creates a shallow copy of this instance and returns it. The keys and values are not cloned.

The remainder of the methods are the same as those described for the `Map<K,V>` interface (Display 16.9).

A program that demonstrates the `HashMap<K,V>` class is given in Display 16.11. This is a variant of Programming Project 15.8 from Chapter 15. In this example, the program uses the `name` instance variable as the key to map to an `Employee` object as defined in Display 7.2. Several sample `Employee` objects are created and added to the map using the first name as the key. The user is given the opportunity to type in names until enter is pressed on a blank line. Any name that exists in the map is retrieved, and its information is output to the screen.

Display 16.11 `HashMap<K,V>` Class Demo (part 1 of 2)

```

1  // This class uses the Employee class defined in Chapter 7.
2  import java.util.HashMap;
3  import java.util.Scanner;
4  public class HashMapDemo
5  {
6      public static void main(String[] args)
7      {
8          // First create a hashmap with an initial size of 10 and
9          // the default load factor
10         HashMap<String,Employee> employees =
11             new HashMap<String,Employee>(10);

12         // Add several employees objects to the map using
13         // their name as the key
14         employees.put("Joe",
15             new Employee("Joe",new Date("September", 15, 1970)));
16         employees.put("Andy",
17             new Employee("Andy",new Date("August", 22, 1971)));
18         employees.put("Greg",
19             new Employee("Greg",new Date("March", 9, 1972)));
20         employees.put("Kiki",
21             new Employee("Kiki",new Date("October", 8, 1970)));
22         employees.put("Antoinette",
23             new Employee("Antoinette",new Date("May", 2, 1959)));
24         System.out.print("Added Joe, Andy, Greg, Kiki, ");
25         System.out.println("and Antoinette to the map.");

26         // Ask the user to type a name. If found in the map,
27         // print it out.
28         Scanner keyboard = new Scanner(System.in);
29         String name = "";
30         do
31         {
32             System.out.print("\nEnter a name to look up in the map. ");
33             System.out.println("Press enter to quit.");

```

Display 16.11 `HashMap<K,V>` Class Demo (part 2 of 2)

```
34         name = keyboard.nextLine();
35         if (employees.containsKey(name))
36         {
37             Employee e = employees.get(name);
38             System.out.println("Name found: " + e.toString());
39         }
40         else if (!name.equals(""))
41         {
42             System.out.println("Name not found.");
43         }
44     } while (!name.equals(""));
45 }
46 }
```

Sample Dialogue

```
Added Joe, Andy, Greg, Kiki, and Antoinette to the map.

Enter a name to look up in the map. Press enter to quit.
Joe
Name found: Joe September 15, 1970

Enter a name to look up in the map. Press enter to quit.
Andy
Name found: Andy August 22, 1971

Enter a name to look up in the map. Press enter to quit.
Kiki
Name found: Kiki October 8, 1970

Enter a name to look up in the map. Press enter to quit.
Myla
Name not found.
```

Self-Test Exercises

7. Can an object that instantiates the `HashMap<K,V>` class contain multiple copies of some element as a key? How about multiple copies of some element as a value?
8. Suppose that you want a `HashMap<K,V>` that maps a unique employee ID number to an `Employee` object. Give the definition for a `HashMap<K,V>` variable that defines and allocates the `HashMap`. Expect to have 100 employees in your organization. If the employee ID number is an integer between 0 and 100, is a map a good choice for a data structure to store this information?

16.3 Iterators

The White Rabbit put on his spectacles. "Where shall I begin, please your Majesty?" he asked.

"Begin at the beginning," the King said, very gravely, "And go on till you come to the end: then stop."

LEWIS CARROLL, *Alice's Adventures in Wonderland*, Macmillan, 1865.

iterator

An **iterator** is an object that is used with a collection to provide sequential access to the elements in the collection. In this section, we discuss iterators in general, and iterators in the Java collection framework in particular.

The Iterator Concept

In the next subsection, we will discuss the Java `Iterator` interface, but before that, let us consider the intuitive idea of an iterator. An iterator is something that allows you to examine and possibly modify the elements in a collection in some sequential order. So, an iterator imposes an order on the elements of a collection even if the collection, such as the class `HashSet<T>`, does not impose any order on the elements it contains.

Something that is not an object—and thus not, strictly speaking, a Java `Iterator`—but that satisfies the intuitive idea of an iterator is an `int` variable `i` used with an array `a`. This iterator `i` can be made to start out at the first array as follows:

```
i = 0;
```

The iterator can give you the current element; the current element is simply `a[i]`. The iterator can go to the next element and give you the next element as follows:

```
i++;
"Gives you a[i]"
```

The concept of an iterator is simple but powerful enough to be used frequently.

The `Iterator<T>` Interface

`Iterator<T>` interface

Java formalizes the concept of an iterator with the `Iterator<T>` generic interface. Any object of any class that satisfies the `Iterator<T>` interface is an `Iterator<T>`. So, an array index is not a Java `Iterator<T>`. However, the index could be an instance variable in an object of an `Iterator<T>` class.

An `Iterator<T>` object does not stand on its own. It must be associated with some collection object. How is the association accomplished? In Java, any class that satisfies the `Collection<T>` interface must have a method, named `iterator()`, that returns an `Iterator<T>`. For example, let us say `c` is an instance of the `HashSet<T>` collection class with some class plugged in for `T`. To make things concrete, let us plug in `String` for `T`; so `c` is an instance of the `HashSet<String>` collection class. You can obtain an iterator for `c` as follows:

```
Iterator<String> iteratorForC = c.iterator();
```

You may not know what class the `iteratorForC` is an instance of, but you do know it satisfies the `Iterator<String>` interface, so you know it has the methods in the `Iterator<T>` interface. These methods are given in Display 16.12.

Display 16.13 contains a simple demonstration of using an iterator with a `HashSet<T>` object. A `HashSet<T>` object imposes no order on the elements in the `HashSet<T>` object, but the iterator imposes an order on the elements—namely, the order in which they are produced by `next()`. There are no requirements on this ordering. If you run the program in Display 16.13 twice, the order of the elements' output will almost certainly be the same each time. However, it would not be an error if they are output in a different order each time the program runs.

If the collection used with an `Iterator<T>` imposes an order on its elements, such as an `ArrayList<T>` does, then the `Iterator<T>` will output the elements in that order. If you require an order with a `HashSet<T>` object, then you may use the `LinkedHashSet<T>` class, which uses an internal doubly linked list to store the items in the order that they are added.

Display 16.12 Methods in the `Iterator<T>` Interface

The `Iterator<T>` interface is in the `java.util` package.

All the exception classes mentioned are the kind that are not required to be caught in a `catch` block or declared in a `throws` clause.

`NoSuchElementException` is in the `java.util` package, which requires an import statement if your code mentions the `NoSuchElementException` class. All the other exception classes mentioned are in the package `java.lang` and so do not require any import statement.

```
public T next()
```

Returns the next element of the collection that produced the iterator.

Throws a `NoSuchElementException` if there is no next element.

```
public boolean hasNext()
```

Returns `true` if `next()` has not yet returned all the elements in the collection; returns `false` otherwise.

```
public void remove() (Optional)
```

Removes from the collection the last element returned by `next`.

This method can be called only once per call to `next`. If the collection is changed in any way, other than by using `remove`, the behavior of the iterator is not specified (and thus should be considered unpredictable).

Throws `IllegalStateException` if the `next` method has not yet been called, or if the `remove` method has already been called after the last call to the `next` method.

Throws an `UnsupportedOperationException` if the `remove` operation is not supported by this `Iterator<T>`.

Display 16.13 An Iterator

```

1  import java.util.HashSet;
2  import java.util.Iterator;

3  public class HashSetIteratorDemo
4  {
5      public static void main(String[] args)
6      {
7          HashSet<String> s = new HashSet<String>();

8          s.add("health");
9          s.add("love");
10         s.add("money");

11         System.out.println("The set contains:");

12         Iterator<String> i = s.iterator();
13         while (i.hasNext())
14             System.out.println(i.next());

15         i.remove();

16         System.out.println();
17         System.out.println("The set now contains:");

18         i = s.iterator();
19         while (i.hasNext())
20             System.out.println(i.next());

21         System.out.println("End of program.");
22     }
23 }

```

You cannot “reset” an iterator “to the beginning.” To do a second iteration, you create another iterator.

Sample Dialogue

The set contains:
money
love
health

The set now contains:
money
love
End of program.

The HashSet<T> object does not order the elements it contains, but the iterator imposes an order on the elements.

Iterators

An iterator is something that allows you to examine and possibly modify the elements in a collection in some sequential order. Java formalizes this concept with the two interfaces `Iterator<T>` and `ListIterator<T>`.



TIP: For-Each Loops as Iterators

A for-each loop is not, strictly speaking, an iterator (because, among other things, it is not an object), but a for-each loop serves the same purpose as an iterator: It lets you cycle through the elements in a collection. When dealing with collections, you can often use a for-each loop in place of an iterator, and the for-each loop is usually simpler and easier to use than an iterator. For example, in Display 16.14, we have rewritten the program in Display 16.13 using for-each loops in place of iterators. Note that we needed to do some extra programming with the variable `last` in order to simulate `i.remove()`. Sometimes an iterator works best, and sometimes a for-each loop works best. Many authorities would say that our code would be better if we had not replaced the first iterator loop in Display 16.13 with a for-each loop. ■

Display 16.14 For-Each Loops as Iterators (part 1 of 2)

```
1  import java.util.HashSet;
2  import java.util.Iterator;

3  public class ForEachDemo
4  {
5      public static void main(String[] args)
6      {
7          HashSet<String> s = new HashSet<String>();

8          s.add("health");
9          s.add("love");
10         s.add("money");

11         System.out.println("The set contains:");

12         String last = null;
13         for (String e : s)
14         {
15             last = e;
16             System.out.println(e);
17         }
```

(continued)

Display 16.14 For-Each Loops as Iterators (part 2 of 2)

```

18         s.remove(last);

19         System.out.println();
20         System.out.println("The set now contains:");

21         for (String e : s)
22             System.out.println(e);

23         System.out.println("End of program.");
24     }
25 }
```

The output is the same as in Display 16.13.

List Iterators

List Iterator<T>

The collection framework has two iterator interfaces: the `Iterator<T>` interface, which you have already seen and which works with any collection class that implements the `Collection<T>` interface; and the `ListIterator<T>` interface, which is designed to work with collections that implement the `List<T>` interface. The `ListIterator<T>` interface extends the `Iterator<T>` interface. A `ListIterator<T>` has all the methods that an `Iterator<T>` has, plus more methods that provide two new abilities: A `ListIterator<T>` can move in either direction along the list of elements in the collection, and a `ListIterator<T>` has methods, such as `set` and `add`, that can be used to change the elements in the collection. The methods for the `ListIterator<T>` interface are given in Display 16.15. See Appendix 5 for a more detailed description that includes all exceptions thrown.

The map framework does not directly support the iterable interface, but you can use the map's `keySet()`, `values()`, or `entrySet()` methods, which return iterable sets containing the keys, values, or (key, value) mappings of the map.

Display 16.15 Methods in the `ListIterator<T>` Interface (part 1 of 2)

The `ListIterator<T>` interface is in the `java.util` package.

The *cursor position* is explained in the text and in Display 16.16.

All the exception classes mentioned are the kind that are not required to be caught in a `catch` block or declared in a `throws` clause.

`NoSuchElementException` is in the `java.util` package, which requires an import statement if your code mentions the `NoSuchElementException` class. All the other exception classes mentioned are in the package `java.lang` and so do not require any import statement.

```
public T next()
```

Returns the next element of the list that produced the iterator. More specifically, returns the element immediately after the cursor position.

Throws a `NoSuchElementException` if there is no next element.

Display 16.15 Methods in the `ListIterator<T>` Interface (part 2 of 2)

```
public T previous()
```

Returns the previous element of the list that produced the iterator. More specifically, returns the element immediately before the cursor position.

Throws a `NoSuchElementException` if there is no previous element.

```
public boolean hasNext()
```

Returns `true` if there is a suitable element for `next()` to return; returns `false` otherwise.

```
public boolean hasPrevious()
```

Returns `true` if there is a suitable element for `previous()` to return; returns `false` otherwise.

```
public int nextIndex()
```

Returns the index of the element that would be returned by a call to `next()`. Returns the list size if the cursor position is at the end of the list.

```
public int previousIndex()
```

Returns the index that would be returned by a call to `previous()`. Returns 1 if the cursor position is at the beginning of the list.

```
public void add(T newElement) (Optional)
```

Inserts `newElement` at the location of the iterator cursor (that is, before the value, if any, that would be returned by `next()` and after the value, if any, that would be returned by `previous()`). Cannot be used if there has been a call to `add` or `remove` since the last call to `next()` or `previous()`.

Throws `IllegalStateException` if neither `next()` nor `previous()` has been called, or if the `add` or `remove` method has already been called after the last call to `next()` or `previous()`.

```
public void remove() (Optional)
```

Removes from the collection the last element returned by `next()` or `previous()`.

This method can be called only once per call to `next()` or `previous()`.

Cannot be used if there has been a call to `add` or `remove` since the last call to `next()` or `previous()`.

Throws `IllegalStateException` if neither `next()` nor `previous()` has been called, or if the `add` or `remove` method has already been called after the last call to `next()` or `previous()`.

```
public void set(T newElement) (Optional)
```

Replaces the last element returned by `next()` or `previous()` with `newElement`.

Cannot be used if there has been a call to `add` or `remove` since the last call to `next()` or `previous()`.

Throws `IllegalStateException` if neither `next()` nor `previous()` has been called, or if the `add` or `remove` method has been called since the last call to `next()` or `previous()`.

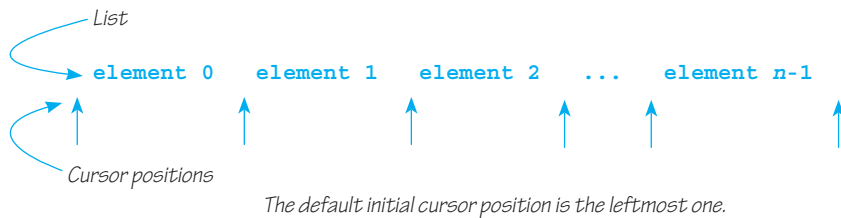
cursor

The general idea of *next* and *previous* is clear, but we need to make it precise if you are to understand the `next()` and `previous()` methods of the `ListIterator<T>` interface. Every `ListIterator<T>` has a position marker in the list known as the **cursor**. If the list has n elements, they are numbered by indices 0 through $n-1$, but there are $n+1$ cursor positions, as indicated in Display 16.16. When `next()` is invoked, the element immediately following the cursor position is returned and the cursor is moved to the next cursor position. When `previous()` is invoked, the element immediately before the cursor position is returned, and the cursor is moved back to the preceding cursor position.

The `ListIterator<T>` Interface

The `ListIterator<T>` interface extends the `Iterator<T>` interface. The `ListIterator<T>` interface differs from the `Iterator<T>` interface by adding the following abilities: A `ListIterator<T>` can move in either direction along the list of elements in the collection, and a `ListIterator<T>` has methods, such as `set` and `add`, that can be used to change the elements in the collection.

Display 16.16 `ListIterator<T>` Cursor Positions



PITFALL: `next` Can Return a Reference

If `i` is an iterator, then `i.next()` returns an element of the collection that created `i`, but there are two meanings of “return an element.”

1. The invocation `i.next()` could return a copy of the element in the collection (for example, using a copy constructor or a `clone` method).
2. Alternatively, `i.next()` could return a reference to the element in the collection.

In case (1), modifying `i.next()` *will not* change the element in the collection (provided the copy was a deep copy). In case (2), modifying `i.next()` *will* change the element in the collection. The APIs for both the `Iterator<T>` and `ListIterator<T>` interfaces are vague on whether you should follow policy (1) or (2), but the iterators



PITFALL: (continued)

for the standard predefined collection classes, such as `ArrayList<T>` and `HashSet<T>`, return references. So, you can modify the elements in the collection by using mutator methods on `i.next()`. This is illustrated in Display 16.17. The comments we made about `i.next()` also apply to `i.previous()`.

The fact that `next` and `previous` return references to elements in the collection is not necessarily bad news. It means you must be careful, but it also means you can cycle through all the elements in the collection and perform some processing that might modify the elements. For example, if the elements in the collection are records of some sort, you can use mutator methods to update the records.

If you read the APIs for the `Iterator<T>` and `ListIterator<T>` interfaces, they say that a `ListIterator<T>` can change the collection, but presumably, a plain old `Iterator<T>` cannot. These API comments do not refer to whether or not a reference is returned by `i.next()`. They simply refer to the fact that the `ListIterator<T>` interface has a `set` method, whereas the `Iterator<T>` interface does not. Do not confuse this with the point discussed in the previous paragraph. ■

Display 16.17 An Iterator Returns a Reference (part 1 of 2)

```

1  import java.util.ArrayList;
2  import java.util.Iterator;

3  public class IteratorReferenceDemo
4  {
5      public static void main(String[] args)
6      {
7          ArrayList<Date> birthdays = new ArrayList<Date>();

8          birthdays.add(new Date(1, 1, 1990));
9          birthdays.add(new Date(2, 2, 1990));
10         birthdays.add(new Date(3, 3, 1990));

11         System.out.println("The list contains:");

12         Iterator<Date> i = birthdays.iterator();
13         while (i.hasNext())
14             System.out.println(i.next());

15         i = birthdays.iterator();
16         Date d = null; //To keep the compiler happy.
17         System.out.println("Changing the references.");
18         while (i.hasNext())
19         {
20             d = i.next();
21             d.setDate(4, 1, 1990);
22         }

```

The class `Date` is defined in Display 4.13, but you can easily guess all you need to know about `Date` for this example.

(continued)

Display 16.17 An Iterator Returns a Reference (part 2 of 2)

```

23         System.out.println("The list now contains:");

24         i = birthdays.iterator();
25         while (i.hasNext())
26             System.out.println(i.next());

27         System.out.println("April fool!");
28     }
29 }

```

Sample Dialogue

```

The list contains:
January 1, 1990
February 2, 1990
March 3, 1990
Changing the references.
The list now contains:
April 1, 1990
April 1, 1990
April 1, 1990
April fool!

```

**TIP: Defining Your Own Iterator Classes**

There really is little need to define your own `Iterator<T>` or `ListIterator<T>` classes. The most common and easiest way to define a collection class is to make it a derived class of one of the library collection classes, such as `ArrayList<T>` or `HashSet<T>`. When you do this, you automatically get the method `iterator()`, and if need be, the method `listIterator()`, which takes care of iterators. However, if you should need to define a collection class in some other way, then the best way to define your iterator class or classes is to define them as inner classes of your collection class. ■

Self-Test Exercises

9. Does a `HashSet<T>` object have a method to produce a `ListIterator<T>`?
Does an `ArrayList<T>` object have a method to produce a `ListIterator<T>`?
10. Suppose `i` is a `ListIterator<T>`. Will an invocation of `i.next()` followed by `i.previous()` return the same element for each of the two invocations or might they return two different elements? What about `i.previous()` followed by `i.next()`?

Chapter Summary

- The main *collection* interfaces are `Collection<T>`, `Set<T>`, and `List<T>`. The `Set<T>` and `List<T>` interfaces extend the `Collection<T>` interface. The library classes that are standard to use and that implement these interfaces are `HashSet<T>`, which implements the `Set<T>` interface, and `ArrayList<T>`, which implements the `List<T>` interface.
- A `Set<T>` does not allow repeated elements and does not order its elements. A `List<T>` allows repeated elements and orders its elements.
- The `Map<K, V>` interface is used to store a mapping between a key `K` and a value `V`. It is commonly used to store databases in memory. The `HashMap<K, V>` class is a standard library class that implements a map.
- An *iterator* is something that allows you to examine and possibly modify the elements in a collection in some sequential order. Java formalizes this concept with the two interfaces `Iterator<T>` and `ListIterator<T>`.
- An `Iterator<T>` (with only the required methods implemented) goes through the elements of the collection in one direction only, from the beginning to the end. A `ListIterator<T>` can move through the collection list in both directions, forward and back. A `ListIterator<T>` has a `set` method; the `Iterator<T>` interface does not require a `set` method.

Answers to Self-Test Exercises

1.

```
public static <T> boolean inSome(T target,
                                Collection<T> c1, Collection<T> c2)
{
    return (c1.contains(target) || c2.contains(target));
}
```
2.

```
public static <T> T getFirst(List<T> aList)
{
    if (aList.isEmpty())
        return null;
    else
        return aList.get(0);
}
```
3.

```
public static boolean noNull(Set<?> s)
{
    return (s.remove(null));
}
```
4. No.

5. It would make more sense to make it a derived class of the `ArrayList<T>` class. Then the elements are ordered. You can ensure against repeated elements by redefining all methods that add elements so that the methods check to see if the element is already in the class before entering it. A derived class of the `HashSet<T>` class would automatically ensure that no element is repeated, but it would seem to take a good deal of work to maintain the elements in order.
6. The `Customer` class must override `hashCode` and `equals`. A simple technique to implement `hashCode` when the class contains strings is to return the string's `hashCode` method. One possible implementation of these methods follows.

```
public int hashCode()
{
    return this.toString().hashCode();
}
public boolean equals(Object obj)
{
    Customer other = (Customer) obj;
    return (other.toString().equals(this.toString()));
}
```

7. Multiple copies of some element are not allowed as a key, but are allowed as values.
8. The variable would be defined as


```
HashMap<Integer,Employee> employeeMap =
new HashMap<Integer,Employee>(100);
```

 If the ID numbers are between 0 and 100, then the map will work, but a simple array or `ArrayList` might be a more appropriate data structure.
9. A `HashSet<T>` does not. An `ArrayList<T>` does.
10. The answer to both questions is the same: They will return the same element.

Programming Projects

1. Redo Programming Project 6.8 in Chapter 6, but this time do it for a vector of strings to be sorted into lexicographic order.
2. The Sieve of Eratosthenes is an ancient algorithm that generates prime numbers. Consider the list of numbers from 2 to 10 as follows:

2 3 4 5 6 7 8 9 10

The algorithm starts with the first prime number in the list, which is 2, and then iterates through the remainder of the list, removing any number that is a multiple of 2 (in this case, 4, 6, 8, and 10), leaving

2 3 5 7 9

We then repeat the process with the second prime number in the list, which is 3, and then iterate through the remainder of the list, removing any number that is a multiple of 3 (in this case 9), leaving

2 3 5 7

We then repeat starting with each successive prime number, but no elements are removed because there are no multiples of 5 or 7 (a more efficient implementation of the algorithm would stop without examining 5 or 7). The numbers that remain in the list are all prime numbers.

Implement this algorithm using an `ArrayList` of integers that is initialized to the values from 2 to 100. Your program can iterate numerically through the `ArrayList` from index 0 to index `size() - 1` to get the current prime number, but should use an `Iterator` to scan through the remainder of the list to eliminate the multiples. You can use the `listIterator` method to retrieve the iterator starting at a specified index into the `ArrayList`. Output all remaining prime numbers to the console.

3. The birthday paradox is that there is a surprisingly high probability that two or more people in the same room happen to share the same birthday. By birthday, we mean the same day of the year (ignoring leap years), but not the exact birthday that includes the birth year or time of day. Write a program that approximates the probability that 2 or more people in the same room have the same birthday, for 2 to 50 people in the room.

The program should use simulation to approximate the answer. Over many trials (say, 5,000), randomly assign birthdays (i.e., a number from 1–365) to everyone in the room. Use a `HashSet` to store the birthdays. As the birthdays are randomly generated, use the `contains` method of a `HashSet` to see if someone with the same birthday is already in the room. If so, increment a counter that tracks how many times at least two people have the same birthday and then move on to the next trial. After the trials are over, divide the counter by the number of trials to get an estimated probability that two or more people share the same birthday for a given room size.

Your output should look something like the following. It will not be exactly the same due to the random numbers:

For 2 people, the probability of two birthdays is about 0.002

For 3 people, the probability of two birthdays is about 0.0082

For 4 people, the probability of two birthdays is about 0.0163

...

For 49 people, the probability of two birthdays is about 0.9654

For 50 people, the probability of two birthdays is about 0.969

4. The text files `boynames.txt` and `girlnames.txt`, which are included in the source code for this book, contain lists of the 1,000 most popular boy and girl names in the United States for the year 2005, as compiled by the Social Security Administration.

These are blank-delimited files where the most popular name is listed first, the second most popular name is listed second, and so on to the 1,000th most popular name, which is listed last. Each line consists of the first name followed by a blank



VideoNote
Solution to
Programming
Project 16.3

space followed by the number of registered births in the year using that name. For example, the `girlnames.txt` file begins with

```
Emily 25494
Emma 22532
```

This indicates that Emily is the most popular name with 25,494 registered namings, Emma is the second most popular with 22,532, and so on.

Write a program that determines how many names are on both the boys' and the girls' list. Use the following algorithm:

- Read each girl name as a `String`, ignoring the number of namings, and add it to a `HashSet` object.
 - Read each boy name as a `String`, ignoring the number of namings, and add it to the same `HashSet` object. If the name is already in the `HashSet`, then the `add` method returns `false`. If you count the number of `false` returns, then this gives you the number of common namings.
 - Add each common name to an `ArrayList` and output all of the common names from this list before the program exits.
5. Repeat the previous problem except create your own class, `Name`, that is added to a `HashMap` instead of a `HashSet`. The `Name` class should have three private variables, a `String` to store the name, an integer to store the number of namings for girls, and an integer to store the number of namings for boys. Use the first name as the key to the `HashMap`. The value to store is the `Name` object. Instead of ignoring the number of namings, as in the previous project, store the number in the `Name` class. Make the `ArrayList` a list of `Name` objects; each time you find a common name, add the entire `Name` object to the list. Your program should then iterate through the `ArrayList` and output each common name, along with the number of boy and girl namings.
 6. In a fairyland, the beautiful daughter Laura of the King Charles decided to marry. To help her choose from the many suitors she decided on the following procedure. First, all of the suitors would be lined up one after the other and assigned numbers. The first suitor would be number 1, the second number 2, and so on up to the last suitor, number n . For our implementation let's consider the value of n to be 56. Starting from the first suitor, she would then count five suitors down the line and the fifth suitor would be eliminated from winning her hand and removed from the line. Laura would then continue, counting five more suitors, and eliminating every fifth suitor. When she reaches the end of the line, she would reverse direction and work her way back to the beginning. Similarly on reaching the first person in line, she would reverse direction and make her way to the end of the line.

For example, if there were five suitors, then the elimination process would proceed as follows:

```
12345 Initial list of suitors; start counting from 1.
1234 Suitor 5 eliminated; bounce from end back to 1 and keep counting.
```



VideoNote
Solution to
Programming
Project 16.5

- 234 Suitor 1 eliminated; continue counting back from 2.
- 24 Next, Suitor 3 is eliminated; the counting is continued.
- 2 Suitor 4 is eliminated; 2 is the lucky winner.

Write a program that uses an `ArrayList` or `Vector` to determine which position you should stand in to marry Laura if there are n suitors. Your program should use the `ListIterator` interface to traverse the list of suitors and remove a suitor. Be sure that you iterate references the proper object while bouncing back to the beginning of the list of suitors. The suitor at the start of the list should only be counted once when Laura reverses the count.

7. In social networking websites, people link to their friends to form a social network. Write a program that uses `HashMaps` to store the data for such a network. Your program should read from a file that specifies the network connections for different usernames. The file should have the following format to specify a link:

```
source_username friend_username
```

There should be an entry for each link, one per line. Here is a sample file for five usernames:

```
iba            java_guru
iba            crisha
iba            ducky
crisha         java_guru
crisha         iba
ducky          java_guru
ducky          iba
java_guru      iba
java_guru      crisha
java_guru      ducky
wittless       java_guru
```

In this network, everyone links to `java_guru` as a friend. `iba` is friends with `java_guru`, `crisha`, and `ducky`. Note that links are not bidirectional; `wittless` links with `java_guru` but `java_guru` does not link with `wittless`.

First, create a `User` class that has an instance variable to store the user's name and another instance variable that is of type `HashSet<User>`. The `HashSet<User>` variable should contain references to the `User` objects that the current user links to. For example, for the user `iba` there would be three entries, for `java_guru`, `crisha`, and `ducky`. Second, create a `HashMap<String, User>` instance variable in your main class that is used to map from a username to the corresponding `User` object. Your program should do the following:

- Upon startup, read the data file and populate the `HashMap` and `HashSet` data structures according to the links specified in the file.

- Allow the user to enter a name.
- If the name exists in the map, then output all usernames that are one link away from the user entered.
- If the name exists in the map, then output all usernames that are two links away from the user entered. To accomplish this in a general way, you might consider writing a recursive subroutine.

Do not forget that your `User` class must override the `hashCode` and `equals` methods.

8. You have collected a file of faculty where each faculty member is rated 1 (lowest) to 5 (highest). The first line of the file is a number that identifies how many rating entries are in the file. Each rating then consists of two lines: the unique ID of the faculty followed by the numeric rating from 1 to 5. Here is a sample rating file with three unique faculty IDs and six ratings:

```
8
Eve_8640
5
Kate_6721
5
Eve_8640
3
Rex_5432
4
Eve_8640
1
Kate_6721
2
Rex_5432
2
Kate_6721
1
```

Write a program that reads a file in this format, calculates the average rating for each faculty member, and outputs the average along with the number of reviews. Here is the desired output for the sample data:

```
Eve_8640: 3 reviews, average of 3.0 / 5
Kate_6721: 3 reviews, average of 2.6 / 5
Rex_5432: 2 reviews, average of 3.0 / 5
```

Use an `ArrayList` to calculate the output. Your map(s) should index from a string representing each faculty member's name to integers that store the number of reviews for the faculty ID and the sum of the ratings for the faculty member.

9. The file `words.txt` included on the website contains a list of 87,314 English words. Write a program that uses this word list to implement a simple spell-checker. First, read all of the words into a `HashSet<String>` object. Then, allow the user to enter the name of a text file that contains written English. The program should output all of the words that are not in the set as potentially misspelled words.

10. You have a list of student ID numbers followed by the course number (separated by a space) that each student is enrolled in. The listing is in no particular order. For example, if student 1 is in CS100 and CS200 while student 2 is in CS105 and MATH210, then the list might look like this:

```
1 CS100
2 MATH210
2 CS105
1 CS200
```

Write a program that reads data in this format from the console. If the ID number is `-1`, then stop inputting data. Use the `HashMap` class to map from an `Integer` (the student ID number) to an `ArrayList` of type `String` that holds each course that the student is enrolled in. The declaration should look like this:

```
HashMap<Integer, ArrayList<String>> students =
    new HashMap<Integer, ArrayList<String>>();
```

After all the data is input, iterate through the map and output the student ID number and all courses stored in the vector for that student. The result should be a list of courses organized by student ID number.

This page intentionally left blank



Swing I 17

17.1 EVENT-DRIVEN PROGRAMMING 983

Events and Listeners 983

17.2 BUTTONS, EVENTS, AND OTHER SWING BASICS 984

Example: A Simple Window 985

Buttons 991

Action Listeners and Action Events 992

Example: A Better Version of Our First Swing GUI 995

Labels 998

Color 999

Example: A GUI with a Label and Color 1000

17.3 CONTAINERS AND LAYOUT MANAGERS 1002

Border Layout Managers 1003

Flow Layout Managers 1006

Grid Layout Managers 1007

Panels 1011

Example: A Tricolor Built with Panels 1012

The `Container` Class 1016

The Model-View-Controller Pattern ★ 1020

17.4 MENUS AND BUTTONS 1021

Example: A GUI with a Menu 1021

Menu Bars, Menus, and Menu Items 1021

Nested Menus ★ 1026

The `AbstractButton` Class 1026

The `setActionCommand` Method 1029

Listeners as Inner Classes ★ 1030

17.5 TEXT FIELDS AND TEXT AREAS 1033

Text Areas and Text Fields 1034

A Swing Calculator 1041

It Don't Mean a Thing If It Ain't Got That Swing.

SONG TITLE *Duke Ellington, Brunswick Records, 1931.*

Introduction

Swing

GUI

This is the first of two chapters that present the basic classes in the **Swing** package and teach the basic techniques for using these classes to define *GUIs*. **GUIs** are windowing interfaces that handle user input and output. *GUI* is pronounced “gooey” and stands for **graphical user interface**. Entire books have been written on Swing, so we will not be able to give you a complete description of Swing in just three chapters. However, we will teach you enough to allow you to write a variety of windowing interfaces.

GUI

Windowing systems that interact with the user are often called **GUIs**. *GUI* is pronounced “gooey” and stands for **graphical user interface**.

AWT

The **AWT (Abstract Window Toolkit)** package is an older package designed for doing windowing interfaces. Swing can be viewed as an improved version of the AWT. However, Swing did not completely replace the AWT package. Some AWT classes are replaced by Swing classes, but other AWT classes are needed when using Swing. We will use classes from both Swing and the AWT.

Swing GUIs are designed using a particular form of object-oriented programming that is known as *event-driven programming*. Our first section begins with a brief overview of event-driven programming.

Prerequisites

Before covering this chapter (and the two chapters on applets¹ and more Swing), you need to have read Chapters 1 through 5, Chapter 7 (inheritance), Chapter 13 (interfaces and inner classes), and Section 8.2 of Chapter 8 (abstract classes). (Section 8.2 of Chapter 8 does not require Section 8.1.) Except for one subsection at the end of this chapter, you need not have read any of the other chapters that precede this chapter.

To cover the last subsection of this chapter, entitled “A Swing Calculator,” you need to first read Chapter 9, which covers exceptions. If you have not yet read Chapter 9, you can skip that last section.

¹The chapter on applets is on the website that accompanies this book.

17.1 Event-Driven Programming

My duty is to obey orders.

MARY ANNA JACKSON *Life and Letters of General Thomas J. Jackson (1891), Ch. 4: The War with Mexico—1846–1848, p. 45, Harper & Brothers, 1891.*

event-driven programming

Event-driven programming is a programming style that uses a signal-and-response approach to programming. Signals to objects are things called *events*, a concept we explain in this section.

Events and Listeners

event listener firing an event

Swing programs use events and event handlers. An **event** is an object that acts as a signal to another object known as a **listener**. The sending of the event is called **firing the event**. The object that fires the event is often a GUI component, such as a button. The button fires the event in response to being clicked. The listener object performs some action in response to the event. For example, the listener might place a message on the screen in response to a particular button being clicked. A given component may have any number of listeners, from zero to several listeners. Each listener might respond to a different kind of event, or multiple listeners might respond to the same events.

If you have read Chapter 9 on exception handling, then you have already seen one specialized example of event-driven programming.² An exception object is an event. The throwing of an exception is an example of firing an event (in this case, firing the exception event). The listener is the `catch` block that catches the event.

event handler

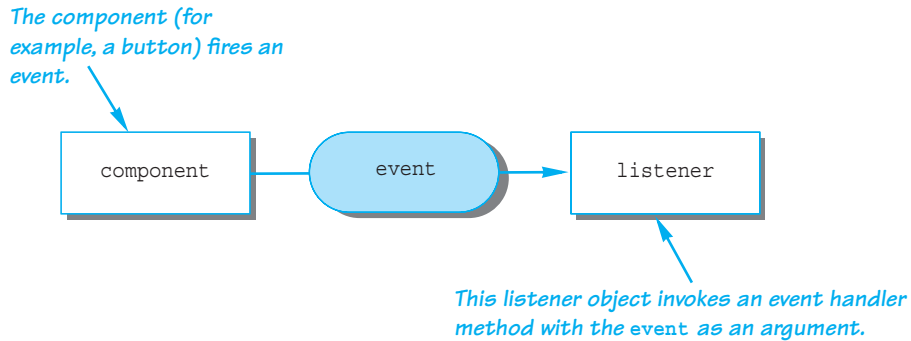
In Swing GUIs, an event often represents some action such as clicking a mouse, dragging the mouse, pressing a key on the keyboard, clicking the close-window button on a window, or any other action that is expected to elicit a response. A listener object has methods that specify what will happen when events of various kinds are received by the listener. These methods that handle events are called **event handlers**. You the programmer will define (or redefine) these event-handler methods. The relationship between an event-firing object, such as a button, and its event-handling listener is shown diagrammatically in Display 17.1.

Event-driven programming is very different from most programming you have seen before now. All our previous programs consisted of a list of statements executed in order. There were loops that repeat statements and branches that choose one of a list of statements to execute next. However, at some level, each run of a program consists of a list of statements performed by one agent (the computer) that executes the statements one after the other in order.

Event-driven programming is a very different game. In event-driven programming, you create objects that can fire events, and you create listener objects to react to the events. For the most part, your program does not determine the order in which things happen. The events determine that order. When an event-driven program is running,

²If you have not yet covered Chapter 9 on exceptions, you can safely ignore this paragraph.

Display 17.1 Event Firing and an Event Listener



the next thing that happens depends on the next event. It is as though the listeners are robots that interact with other objects (possibly other robots) in response to events (signals) from these other objects. You program the robots, but the environment and other robots determine what any particular robot will actually end up doing.

If you have never done event-driven programming before, one aspect of it may seem strange to you: *You will be writing definitions for methods that you will never invoke in any program.* This will likely feel a bit strange at first, because a method is of no value unless it is invoked. So, somebody or something other than you, the programmer, must be invoking these methods. That is exactly what does happen. The Swing system automatically invokes certain methods when an event signals that the method needs to be called.

Event-driven programming with the Swing library makes extensive use of inheritance. The classes you define will be derived classes of some basic Swing library classes. These derived classes will inherit methods from their base class. For many of these inherited methods, library software will determine when these methods are invoked, but you will override the definition of the inherited method to determine what will happen when the method is invoked.

17.2 Buttons, Events, and Other Swing Basics

One button click is worth a thousand key strokes.

ANONYMOUS

In this section, we present enough about Swing to allow you to do some simple GUI programs.

EXAMPLE: A Simple Window

Display 17.2 contains a Swing program that produces a simple window. The window contains nothing but a button on which is written "Click to end program." If the user follows the instructions and clicks the button with his or her mouse, the program ends.

The `import` statements give the names of the classes used and which package they are in. What we and others call the *Swing library* is the package named `javax.swing`. The *AWT library* is the package `java.awt`. Note that one package name contains an "x" and one does not.

This program is a simple class definition with only a `main` method. The first line in the `main` method creates an object of the class `JFrame`. That line is reproduced as follows:

```
JFrame firstWindow = new JFrame();
```

This is an ordinary declaration of a variable named `firstWindow` and an invocation of the no-argument constructor for the class `JFrame`. A `JFrame` object is a basic window that includes a border and the usual three buttons for minimizing the window down to an icon, changing the size of the window, and closing the window. These buttons are shown in the upper-right corner of the window, which is typical, but if your operating system normally places these buttons someplace else, that is where they will likely be located in a `JFrame` on your computer.

The initial size of the `JFrame` window is set using the `JFrame` method `setSize`, as follows:

```
firstWindow.setSize(WIDTH, HEIGHT);
```

In this case, `WIDTH` and `HEIGHT` are defined `int` constants. The units of measure are pixels, so the window produced is 300 pixels by 200 pixels. (The term *pixel* is defined in the box entitled "Pixel.") As with other windows, you can change the size of a `JFrame` by using your mouse to drag a corner of the `JFrame` window.

The buttons for minimizing the window down to an icon and for changing the size of the window behave as they do in any of the other windows you have used. The minimization button shrinks the window down to an icon. (To restore the window, click the icon.) The second button changes the size of the window back and forth from full screen to a smaller size. The close-window button can behave in different ways depending on how it is set by your program.

The behavior of the close-window button is set with the `JFrame` method `setDefaultCloseOperation`. The line of the program that sets the behavior of the close-window button is reproduced next:

```
firstWindow.setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
```

(continued)

EXAMPLE: (continued)

In this case, the argument `JFrame.DO_NOTHING_ON_CLOSE` is a defined constant named `DO_NOTHING_ON_CLOSE`, which is defined in the `JFrame` class. This sets the close-window button so that when it is clicked, nothing happens (unless we programmed something to happen, which we have not done). Other possible arguments are given in Display 17.3.

The method `setDefaultCloseOperation` takes a single `int` argument; each of the constants described in Display 17.3 is an `int` constant. However, do not think of them as `int` values. Think of them as policies for what happens when the user clicks the close-window button. It was convenient to name these policies by `int` values. However, they could just as well have been named by `char` values or `String` values or something else. The fact that they are `int` values is an incidental detail of no real importance.

Descriptions of some of the most important methods in the class `JFrame` are given in Display 17.3. Some of these methods will not be explained until later in this chapter. A more complete list of methods for the class `JFrame` is given in Appendix 5.

A `JFrame` can have components added, such as buttons, menus, and text labels. For example, the following line from Display 17.2 adds the `JButton` object named `endButton` to the `JFrame` named `firstWindow`:

```
firstWindow.add(endButton);
```

The description of how the `JButton` named `endButton` is created and programmed will be given in the two subsections entitled “Buttons” and “Action Listeners and Action Events” a little later in this section.

We end this subsection by jumping ahead to the last line of the program, which is

```
firstWindow.setVisible(true);
```

This makes the `JFrame` window visible on the screen. At first glance, this may seem strange. Why not have windows automatically become visible? Why would you create a window if you did not want it to be visible? The answer is that you may not want it to be visible at all times. You have certainly experienced windows that disappear and reappear. To hide the window, which is not desirable in this example, you would replace the argument `true` with `false`.

JFrame

An object of the class `JFrame` is what you think of as a window. It automatically has a border and some basic buttons for minimizing the window and similar actions. As you will see, a `JFrame` object can have buttons and many other components added to the window and programmed for action.

Display 17.2 A First Swing Demonstration Program (part 1 of 2)

```

1  import javax.swing.JFrame;
2  import javax.swing.JButton;

3  public class FirstSwingDemo
4  {
5      public static final int WIDTH = 300;
6      public static final int HEIGHT = 200;

7      public static void main(String[] args)
8      {
9          JFrame firstWindow = new JFrame( );
10         firstWindow.setSize(WIDTH, HEIGHT);

11         firstWindow.setDefaultCloseOperation(
12             JFrame.DO_NOTHING_ON_CLOSE);

13         JButton endButton = new JButton("Click to end program.");
14         EndingListener buttonEar = new EndingListener();
15         endButton.addActionListener(buttonEar);
16         firstWindow.add(endButton);

17         firstWindow.setVisible(true);
18     }
19 }
```

This program is not typical of the style we will use in Swing programs.

This is the file FirstSwingDemo.java.

```

1  import java.awt.event.ActionListener;
2  import java.awt.event.ActionEvent;

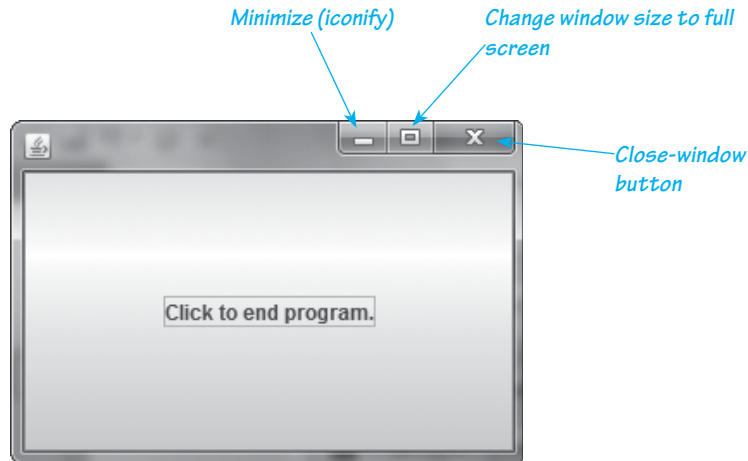
3  public class EndingListener implements ActionListener
4  {
5      public void actionPerformed(ActionEvent e)
6      {
7          System.exit(0);
8      }
9  }
```

This is the file EndingListener.java.

(continued)

Display 17.2 A First Swing Demonstration Program (part 2 of 2) (Source: Oracle Corporation)

RESULTING GUI



Pixel

pixel

A **pixel** is the smallest unit of space on which your screen can write. With Swing, both the size and the position of objects on the screen are measured in pixels. The more pixels you have on a screen, the greater the screen resolution.

Resolution's Relationship to Object Size

The relationship between resolution and size can seem confusing at first. A high-resolution screen is a screen of better quality than a low-resolution screen, so why does an object look smaller on a high-resolution screen and larger on a low-resolution screen? Consider a very simple case—namely, a one-pixel “dot.” For a screen of fixed size, if there are very many pixels (high resolution), then the one-pixel dot will be very small. If there are fewer pixels (low resolution) for the same size screen, then each pixel must be larger because the smaller number of pixels cover the same screen. So, if there are fewer pixels, the one-pixel dot will be larger. Similarly, a two-pixel figure or a figure of any number of pixels will look larger on a low-resolution (fewer pixels) screen.

Display 17.3 Some Methods in the Class `JFrame` (part 1 of 2)

The class `JFrame` is in the `javax.swing` package.

```
public JFrame ()
```

Constructor that creates an object of the class `JFrame`.

Display 17.3 Some Methods in the Class JFrame (part 2 of 2)

```
public JFrame(String title)
```

Constructor that creates an object of the class `JFrame` with the title given as the argument.

```
public void setDefaultCloseOperation(int operation)
```

Sets the action that will happen by default when the user clicks the close-window button. The argument should be one of the following defined constants:

`JFrame.DO_NOTHING_ON_CLOSE`: Do nothing. The `JFrame` does nothing, but if there are any registered window listeners, they are invoked. (Window listeners are explained in Chapter 18.)

`JFrame.HIDE_ON_CLOSE`: Hide the frame after invoking any registered `WindowListener` objects.

`JFrame.DISPOSE_ON_CLOSE`: Hide and *dispose* the frame after invoking any registered window listeners. When a window is **disposed**, it is eliminated but the program does not end. To end the program, use the next constant as an argument to `setDefaultCloseOperation`.

`JFrame.EXIT_ON_CLOSE`: Exit the application using the `System.exit` method. (Do not use this for frames in applets. Applets are discussed in Chapter 20 on the website.)

If no action is specified using the method `setDefaultCloseOperation`, then the default action taken is `JFrame.HIDE_ON_CLOSE`.

Throws an `IllegalArgumentException` if the argument is not one of the values listed above.³

Throws a `SecurityException` if the argument is `JFrame.EXIT_ON_CLOSE` and the Security Manager will not allow the caller to invoke `System.exit`. (You are not likely to encounter this case.)

```
public void setSize(int width, int height)
```

Sets the size of the calling frame so that it has the width and height specified. Pixels are the units of length used.

```
public void setTitle(String title)
```

Sets the title for this frame to the argument string.

```
public void add(Component componentAdded)
```

Adds a component to the `JFrame`.

```
public void setLayout(LayoutManager manager)
```

Sets the layout manager. Layout managers are discussed later in this chapter in Section 17.3.

```
public void setJMenuBar(JMenuBar menubar)
```

Sets the menu bar for the calling frame. (Menus and menu bars are discussed later in this chapter in Section 17.4.)

```
public void dispose()
```

Eliminates the calling frame and all its subcomponents. Any memory they use is released for reuse. If there are items left (items other than the calling frame and its subcomponents), then this does not end the program. (The method `dispose` is discussed in Chapter 19.)

³If you have not yet covered Chapter 9 on exceptions, you can safely ignore all references to “throwing exceptions.”

The setVisible Method

Many classes of Swing objects have a `setVisible` method. The `setVisible` method takes one argument of type `boolean`. If `w` is an object, such as a `JFrame` window, that can be displayed on the screen, then the call

```
w.setVisible(true);
```

will make `w` visible. The call

```
w.setVisible(false);
```

will hide `w`.

SYNTAX

```
Object_For_Screen.setVisible(Boolean_Expression);
```

EXAMPLE (FROM DISPLAY 17.2)

```
public static void main(String[] args)
{
    JFrame firstWindow = new JFrame();
    .
    .
    .
    firstWindow.setVisible(true);
}
```



PITFALL: Forgetting to Program the Close-Window Button

The following lines from Display 17.2 ensure that when the user clicks the close-window button, nothing happens:

```
firstWindow.setDefaultCloseOperation(
    JFrame.DO_NOTHING_ON_CLOSE);
```

If you forget to program the close-window button, then the default action is as if you had set it the following way:

```
firstWindow.setDefaultCloseOperation(
    JFrame.HIDE_ON_CLOSE);
```

In the program in Display 17.2, this would mean that if the user clicks the close-window button, the window will hide (become invisible and inaccessible), but the program will not end, which is a pretty bad situation. Because the window would be hidden, there would be no way to click the "Click to end program." button. You would need to use some operating system command that forces the program to end. That is an operating system topic, not a Java topic, and the exact command depends on which operating system you are using. ■

Self-Test Exercises

1. What Swing class do you normally use to define a window? Any window class that you define would normally be an object of this class.
2. What units of measure are used in the following call to `setSize` that appeared in the `main` method of the program in Display 17.2? In other words, 300 what? Inches? Centimeters? Light years? And similarly, 200 what?

```
firstWindow.setSize(WIDTH, HEIGHT);
```

which is equivalent to

```
firstWindow.setSize(300, 200);
```

3. What is the method call to set the close-window button of the `JFrame` `someWindow` so that nothing happens when the user clicks the close-window button in `someWindow`?
4. What is the method call to set the close-window button of the `JFrame` `someWindow` so that the program ends when the user clicks the close-window button in `someWindow`?
5. What happens when you click the minimizing button of the `JFrame` shown in Display 17.2?
6. Suppose `someWindow` is a `JFrame` and `n` is an `int` variable with some value. Give a Java statement that will make `someWindow` visible if `n` is positive and hide `someWindow` otherwise.

Buttons

`JButton`

A button object is created in the same way that any other object is created, but you use the class `JButton`. For example, the following example from Display 17.2 creates a button:

```
JButton endButton = new JButton("Click to end program.");
```

The argument to the construct, in this case, "Click to end program.", is a string that will be written on the button when the button is displayed. If you look at the picture of the GUI in Display 17.2, you will see that the button is labeled "Click to end program."

adding a button

We have already discussed adding components, such as buttons, to a `JFrame`. The button is added to the `JFrame` by the following line from Display 17.2:

```
firstWindow.add(endButton);
```

In the next subsection, we explain the lines from Display 17.2 involving the method `addActionListener`.

The JButton Class

An object of the class `JButton` is displayed in a GUI as a component that looks like a button. Click the button with your mouse to simulate pushing it. When creating an object of the class `JButton` using `new`, you can give a string argument to the constructor and the string will be displayed on the button.

You can add a `JButton` object to a `JFrame` by using the method `add` with the `JFrame` as the calling object and the `JButton` object as the argument. You will later see that you can also add buttons to other GUI objects (known as “containers”) in a similar way.

A button’s action is programmed by registering a listener with the button using the method `addActionListener`.

EXAMPLE

```
JButton niceButton = new JButton("Click here");
niceButton.addActionListener(new SomeActionListenerClass());
someJFrame.add(niceButton);
```

The Close-Window Button Is Not in the Class JButton

The buttons that you add to a GUI are all objects of the class `JButton`. The close-window button and the other two accompanying buttons on a `JFrame` are not objects of the class `JButton`. They are part of the `JFrame` object.

Action Listeners and Action Events

Clicking a button with your mouse (or activating certain other items in a GUI) creates an object known as an event and sends the event object to another object (or objects) known as the listener(s). This is called **firing the event**. The listener then performs some action. When we say that the event is “sent” to the listener object, what we really mean is that some method in the listener object is invoked with the event object as the argument. This invocation happens automatically. Your Swing GUI class definition will not normally contain an invocation of this method. However, your Swing GUI class definition does need to do two things:

- First, for each button, it needs to specify what objects are listeners that will respond to events fired by that button; this is called **registering** the listener.
- Second, it must define the methods that will be invoked when the event is sent to the listener. Note that these methods will be defined by you, but in normal circumstances, you will never write an invocation of these methods. The invocations will take place automatically.

The following lines from Display 17.2 create an `EndingListener` object named `buttonEar` and register `buttonEar` as a listener to receive events from the button named `endButton`:

```
EndingListener buttonEar = new EndingListener();
endButton.addActionListener(buttonEar);
```

registering a
listener

addAction
Listener

The second line says that `buttonEar` is registered as a listener to `endButton`, which means `buttonEar` will receive all events fired by `endButton`.

action event

Action Listener

Different kinds of components require different kinds of listener classes to handle the events they fire. A button fires events known as **action events**, which are handled by listeners known as **action listeners**.

An action listener is an object whose class implements the `ActionListener` interface. For example, the class `EndingListener` in Display 17.2 implements the `ActionListener` interface. The `ActionListener` interface has only one method heading that must be implemented, namely the following:

action Performed

```
public void actionPerformed(ActionEvent e)
```

In the class `EndingListener` in Display 17.2, the `actionPerformed` method is defined as follows:

```
public void actionPerformed(ActionEvent e)
{
    System.exit(0);
}
```

If the user clicks the button `endButton`, it sends an action event to the action listener for that button. But `buttonEar` is the action listener for the button `endButton`, so the action event goes to `buttonEar`. When an action listener receives an action event, the event is automatically passed as an argument to the method `actionPerformed` and the method `actionPerformed` is invoked. If the event is called `e`, then the following invocation takes place in response to `endButton` firing `e`:

```
buttonEar.actionPerformed(e);
```

In this case, the parameter `e` is ignored by the method `actionPerformed`. The method `actionPerformed` simply invokes `System.exit` and thereby ends the program. So, if the user clicks `endButton` (the one labeled "Click to end program."), the net effect is to end the program and so the window goes away.

Note that you never write any code that says

```
buttonEar.actionPerformed(e);
```

This action does happen, but the code for this is embedded in some class definition inside the Swing and/or AWT libraries. Somewhere the code says something like

```
bla.actionPerformed(e);
```

and somehow `buttonEar` gets plugged in for the parameter `bla` and this invocation of `actionPerformed` is executed. But, all this is done for you. All you do is define the method `actionPerformed` and register `buttonEar` as a listener for `endButton`.

Note that the method `actionPerformed` must have a parameter of type `ActionEvent`, even if your definition of `actionPerformed` does not use this parameter. This is because the invocations of `actionPerformed` were already programmed for you and so must allow the possibility of using the `ActionEvent` parameter `e`. As you will see, in other Swing GUIs the method `actionPerformed` does often use the event `e` to determine which button was clicked. This first example is a special, simple case

because there is only one button. Later in this chapter, we will say more about defining the `actionPerformed` method in more complicated situations.



PITFALL: Changing the Heading for `actionPerformed`

When you define the method `actionPerformed` in an action listener, you are implementing the method heading for `actionPerformed` that is specified in the `ActionListener` interface. Thus, the header for the method `actionPerformed` is determined for you, and you cannot change the heading. It must have exactly one parameter, and that parameter must be of type `ActionEvent`, as in the following:

```
public void actionPerformed(ActionEvent e)
```

If you change the type of the parameter or if you add (or subtract) a parameter, you will not have given a correct definition of an action listener.⁴ The only thing you can change is the name of the parameter `e`, because it is just a placeholder. So the following change is acceptable:

```
public void actionPerformed(ActionEvent theEvent)
```

Of course, if you make this change, then inside the body of the method `actionPerformed`, you will use the identifier `theEvent` in place of the identifier `e`.

You also cannot add a `throws` clause to the method `actionPerformed`.⁵ If a checked exception is thrown in the definition of `actionPerformed`, then it must be caught in the method `actionPerformed`. (Recall that a checked exception is one that must be either caught in a `catch` block or declared in a `throws` clause.) ■



TIP: Ending a Swing Program

A GUI program is normally based on a kind of infinite loop. There may not be a Java loop statement in a GUI program, but nonetheless the GUI program need not ever end. The windowing system normally stays on the screen until the user indicates that it should go away (for example, by clicking the "Click to end program." button in Display 17.2). If the user never asks the windowing system to go away, it will never go away. When you write a Swing GUI program, you need to use `System.exit` to end the program when the user (or something else) says it is time to do so. Unlike the kinds of programs we saw before this chapter, a Swing program will not end after it has executed all the code in the program. A Swing program does not end until it executes a `System.exit`. (In some cases, the `System.exit` may be in some library code and need not be explicitly given in your code.) ■

`System.exit`

⁴Although it would be rather questionable style, you can overload the method named `actionPerformed` so that you have multiple versions of the method `actionPerformed`, each with a different parameter list. But only the version of `actionPerformed` shown here has anything to do with making a class into an action listener.

⁵If you have not yet covered exception handling (Chapter 9), you can safely ignore this paragraph.

Self-Test Exercises

7. What kind of event is fired when you click a `JButton`?
8. What method heading must be implemented in a class that implements the `ActionListener` interface?
9. Change the program in Display 17.2 so that the window displayed has the title "My First Window". *Hint:* Consult the description of constructors in Display 17.3.

EXAMPLE: A Better Version of Our First Swing GUI

Display 17.4 is a rewriting of the demonstration program in Display 17.2 that includes a few added features. This new version produces a window that is similar to the one produced by the program in Display 17.2. However, this new version is done in the style you should follow in writing your own GUIs. Notice that the window is produced by defining a class (`FirstWindow`) whose objects are windows of the kind we want. The window is then displayed by a program (`DemoWindow`) that uses the class `FirstWindow`.

Observe that `FirstWindow` is a derived class of the class `JFrame`. This is the normal way to define a windowing interface. The base class `JFrame` gives some basic window facilities, and then the derived class adds whatever additional features you want in your window interface.

Note that the constructor in Display 17.4 starts by calling the constructor for the parent class `JFrame` with the line

```
super();
```

As we noted in Chapter 7, this ensures that any initialization that is normally done for all objects of type `JFrame` will in fact be done. If the base class constructor you call has no arguments, then it will be called automatically, so we could have omitted the invocation of `super()` in Display 17.4. However, if the base class constructor needs an argument, as it may in some other situations, then you must include a call to the base class constructor, `super`.

Note that almost all the initializing for the window `FirstWindow` in Display 17.4 is placed in the constructor for the class. That is as it should be. The initialization, such as setting the initial window size, should be part of the class definition and not actions performed by objects of the class (as they were in Display 17.2). All the initializing methods, such as `setSize` and `setDefaultCloseOperation`, are inherited from the class `JFrame`. Because they are invoked in the constructor for the window, the window itself is the calling object. In other words, a method invocation such as

```
setSize(WIDTH, HEIGHT);
```

is equivalent to

```
this.setSize(WIDTH, HEIGHT);
```

(continued)

EXAMPLE: (continued)

Similarly, the method invocations

```
setDefaultCloseOperation(  
    JFrame.DO_NOTHING_ON_CLOSE);
```

and

```
add(endButton);
```

are equivalent to

```
this.setDefaultCloseOperation(  
    JFrame.DO_NOTHING_ON_CLOSE);
```

and

```
this.add(endButton);
```

In the class `FirstWindow` (Display 17.4), we added the title "First Window Class" to the window as follows:

```
setTitle("First Window Class");
```

You can see where the title is displayed in a `JFrame` by looking at the picture of the GUI given in Display 17.4.

One thing we did differently in Display 17.4 than in Display 17.2 is use an anonymous object in the following line:

```
endButton.addActionListener(new EndingListener());
```

The same action was performed by the following lines in Display 17.2:

```
EndingListener buttonEar = new EndingListener();  
endButton.addActionListener(buttonEar);
```

In Display 17.2, we were trying to be extra clear and so we used these two steps. However, it makes more sense to use the anonymous object `new EndingListener()` because this listener object is never referenced again and so does not need a name.

The program `DemoWindow` in Display 17.4 simply displays an object of the class `FirstWindow` on the screen.

Almost all of the initialization details for the window in Display 17.4 have been moved to the constructor for the class `FirstWindow`. However, we have placed the invocations of the method `setVisible` in the application program that uses the window class `FirstWindow`. We could have placed an invocation of `setVisible` in the constructor for `FirstWindow` and omitted the invocation of `setVisible` from the application program `DemoWindow` (Display 17.4). If we had done so, we would have produced the same results when we ran the application program. However, in normal situations, the application program knows when the window should be displayed, so it is normal to put the invocation of the method `setVisible` in the application program. The programmer writing the class `FirstWindow` cannot anticipate when a programmer who uses the window will want to make it visible (or hide it).

Display 17.4 The Normal Way to Define a JFrame (Source: Oracle Corporation)

```
1  import javax.swing.JFrame;
2  import javax.swing.JButton;

3  public class FirstWindow extends JFrame
4  {
5      public static final int WIDTH = 300;
6      public static final int HEIGHT = 200;

7      public FirstWindow()
8      {
9          super();
10         setSize(WIDTH, HEIGHT);

11         setTitle("First Window Class");
12         setDefaultCloseOperation(
13             JFrame.DO_NOTHING_ON_CLOSE);

14         JButton endButton = new JButton("Click to end program.");
15         endButton.addActionListener(new EndingListener());
16         add(endButton);
17     }
18 }
```

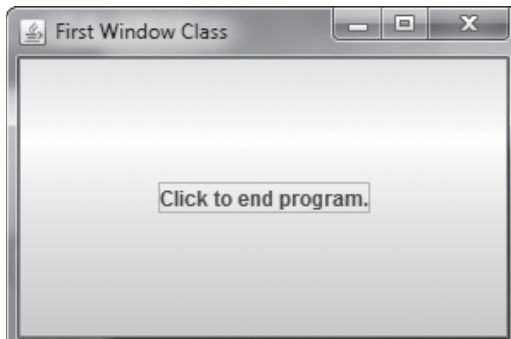
The class `EndingListener` is defined in Display 17.2.

This is the file `FirstWindow.java`.

```
1  public class DemoWindow
2  {
3      public static void main(String[] args)
4      {
5          FirstWindow w = new FirstWindow();
6          w.setVisible(true);
7      }
8  }
```

This is the file `DemoWindow.java`.

RESULTING GUI



JFrame Classes

When we say that a class is a **JFrame class**, we mean the class is a descendent class of the class `JFrame`. For example, the class `FirstWindow` in Display 17.4 is a `JFrame` class. When we say an object is a **JFrame**, we mean that it is an object of some `JFrame` class.

Self-Test Exercises

10. Change the program in Display 17.4 so that the title of the `JFrame` is not set by the method `setTitle` but is instead set by the call to the base class constructor. *Hint*: Recall Self-Test Exercise 9.
11. Change the program in Display 17.4 so that there are two ways to end the GUI program: The program can be ended by either clicking the "Click to end program." button or clicking the close-window button.

Labels

label

JLabel

We have seen how to add a button to a `JFrame`. If you want to add some text to your `JFrame`, use a label instead of a button. A **label** is an object of the class `JLabel`. A label is little more than a line of text. The text for the label is given as an argument to the `JLabel` constructor as follows:

```
JLabel greeting = new JLabel("Hello");
```

The label `greeting` can then be added to a `JFrame` just as a button is added. For example, the following might appear in a constructor for a derived class of `JFrame`:

```
JLabel greeting = new JLabel("Hello");  
add(greeting);
```

The next Programming Example, "A GUI with a Label and Color," includes a label in a `JFrame` GUI.

The JLabel Class

An object of the class `JLabel` is little more than one line of text that can be added to a `JFrame` (or, as we will see, added to certain other objects).

EXAMPLE (INSIDE A CONSTRUCTOR FOR A DERIVED CLASS OF JFrame)

```
JLabel myLabel = new JLabel("Hi Mom!");  
add(myLabel);
```

Color

You can set the color of a `JFrame` (or other GUI object). To set the background color of a `JFrame`, use

```
getContentPane().setBackground(Color);
```

For example, the following will set the color of the `JFrame` named `someFrame` to blue:

```
someFrame.getContentPane().setBackground(Color.BLUE);
```

`Color.BLUE`

Alternatively, if you set the color in the constructor for the `JFrame`, the invocation takes the form

```
getContentPane().setBackground(Color.BLUE);
```

which is equivalent to

```
this.getContentPane().setBackground(Color.BLUE);
```

The next Programming Example, “A GUI with a Label and Color,” shows a `JFrame` object (in fact, two of them) with color.

content pane

The method invocation `getContentPane()` returns something called the **content pane** of the `JFrame`. So,

```
getContentPane().setBackground(Color.BLUE);
```

actually sets the color of the content pane to blue. The content pane is the “inside” of the `JFrame`, so coloring the content pane has the effect of coloring the inside of the `JFrame`. However, you can think of

```
getContentPane().setBackground(Color);
```

as a peculiarly spelled method invocation that sets the color of the `JFrame`. (In this book, we will not be referring to the content pane of a `JFrame` except when we want to color the `JFrame`, so we will explain the content pane no further.)

**get
Content
Pane**

Use `getContentPane` only when you give color to a `JFrame`. As you will see, to set the color of some component in a `JFrame`, such as a button, simply use the method `setBackground` with the button or other component as the calling object. You will see examples of adding color to components in Section 17.3.

color

Color

What kind of thing is a **color** when used in a Java Swing class? Like everything else in Java, a color is an object—in this case, an object that is an instance of the class `Color`. The class `Color` is in the `java.awt` package. (Note that the package name is `java.awt`, not `javax.awt`.)

In a later chapter, you will see how you can define your own colors, but for now we will use the colors that are already defined for you, such as `Color.BLUE`, which is a constant named `BLUE` that is defined in the class `Color`. The constant, of course, represents the color blue. If you set the background of a `JFrame` to `Color.BLUE`, then the `JFrame` will have a blue background. The type of the constant `Color.BLUE` and other such constants is `Color`. The list of color constants that are defined for you is given in Display 17.5. The next Programming Example, “A GUI with a Label and Color,” has an example of a constructor with one parameter of type `Color`.

Display 17.5 The Color Constants

Color.BLACK	Color.MAGENTA
Color.BLUE	Color.ORANGE
Color.CYAN	Color.PINK
Color.DARK_GRAY	Color.RED
Color.GRAY	Color.WHITE
Color.GREEN	Color.YELLOW
Color.LIGHT_GRAY	

The class `Color` is in the `java.awt` package.

EXAMPLE: A GUI with a Label and Color

Display 17.6 shows a class for GUIs with a label and a background color. We have already discussed the use of color for this window. The label is used to display the text string "Close-window button works." The label is created as follows:

```
JLabel aLabel = new JLabel("Close-window button works.");
```

The label is added to the `JFrame` with the method `add` as shown in the following line from Display 17.6:

```
add(aLabel);
```

The GUI class `ColoredWindow` in Display 17.6 programs the close-window button as follows:

```
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

This way, when the user clicks the close-window button, the program ends. Note that if the program has more than one window, as it does in Display 17.6, and the user clicks the close-window button in any one window of the class `ColoredWindow`, then the entire program ends and all windows go away.

Note that we set the title of the `JFrame` by making it an argument to `super` rather than an argument to `setTitle`. This is another common way to set the title of a `JFrame`.

If you run the program `DemoColoredWindow` in Display 17.6, then the two windows will be placed one on top of the other. To see both windows, you need to use your mouse to move the top window.

Setting the Title of a JFrame

The two most common ways to set the title of a `JFrame` are to use the method `setTitle`, as illustrated in Display 17.4, or to give the title as an argument to the base class constructor `super`, as illustrated in Display 17.6.

Self-Test Exercises

12. How would you modify the class definition in Display 17.6 so that the window produced by the no-argument constructor is magenta instead of pink?
13. Rewrite the following two lines from Display 17.6 so that the label does not have the name `aLabel` or any other name. *Hint:* Use an anonymous object.

```
JLabel aLabel = new JLabel("Close-window button works.");
add(aLabel);
```

Display 17.6 A `JFrame` with Color (part 1 of 2)

```
1  import javax.swing.JFrame;
2  import javax.swing.JLabel;
3  import java.awt.Color;

4  public class ColoredWindow extends JFrame
5  {
6      public static final int WIDTH = 300;
7      public static final int HEIGHT = 200;

8      public ColoredWindow(Color theColor)
9      {

10         super("No Charge for Color");
11         setSize(WIDTH, HEIGHT);
12         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

13         getContentPane().setBackground(theColor);

14         JLabel aLabel = new JLabel("Close-window button works.");
15         add(aLabel);
16     }
17     public ColoredWindow()
18     {
19         this(Color.BLUE);
20     }
21 }
```

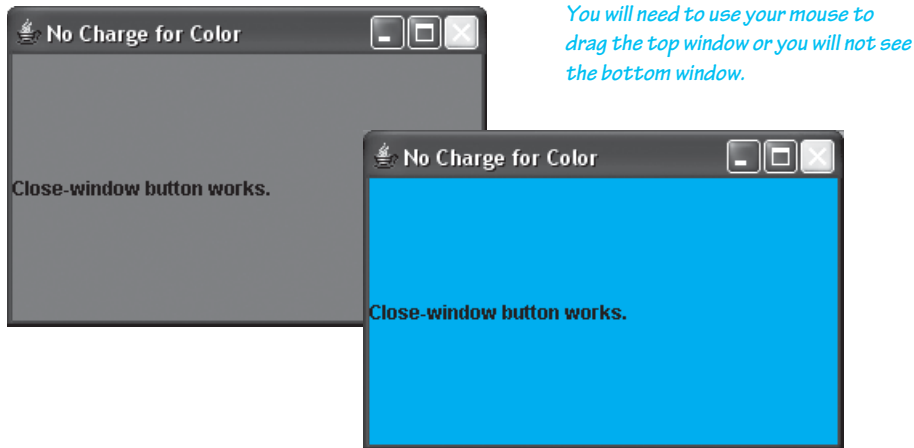
This is an invocation of the other constructor.

This is the file ColoredWindow.java.

```
1  import java.awt.Color;
2
3  public class DemoColoredWindow
4  {
5      public static void main(String[] args)
6      {
7          ColoredWindow w1 = new ColoredWindow();
8          w1.setVisible(true);
9
10         ColoredWindow w2 = new ColoredWindow(Color.GRAY);
11         w2.setVisible(true);
12     }
13 }
```

This is the file ColoredWindow.java.

(continued)

Display 17.6 A `JFrame` with Color (part 2 of 2) (Source: Oracle Corporation)**RESULTING GUI**

17.3 Containers and Layout Managers

Don't put all your eggs in one basket.

PROVERB

There are two main ways to create new classes from old classes. One way is to use inheritance; this is known as the *Is-A relationship*. For example, an object of the class `ColoredWindow` in Display 17.6 is a `JFrame` because `ColoredWindow` is a derived class of the class `JFrame`. The second way to create a new class from an existing class (or classes) is to have instance variables of an already existing class type; this is known as *composition* or the *Has-A relationship*. The Swing library has already set things up so you can easily use composition. The actual code for declaring instance variables is in the Swing library classes, such as the class `JFrame`. Rather than declaring instance variables, add components to a `JFrame` using the `add` method. This does ultimately set some instance variables, but this is done automatically when you use the `add` method. In this section, we discuss adding and arranging components in a GUI or subpart of a GUI.

Thus far, we have only added one component, either a button or a label, to a `JFrame`. You can add more than one component to a `JFrame`. To do so, use the `add` method multiple times, but the `add` method simply tells which components are added to the `JFrame`; it does not say how they are arranged, such as side by side or one above the other. To describe how the components are arranged, you need to use a **layout manager**.

In this section, we will see that there are other classes of objects besides `JFrames` that can have components added with the `add` method and arranged by a layout manager. All these classes are known as **container classes**.

layout
manager

container
class

Border Layout Managers

If you do not specify a layout, then Java will use a `BorderLayout` by default. Display 17.7 contains an example of a GUI that uses a layout manager to arrange three labels in a `JFrame`. The labels are arranged one below the other on three lines.

A layout manager is added to the `JFrame` class in Display 17.7 with the following line:

```
setLayout(new BorderLayout());
```

Border Layout

`BorderLayout` is a layout manager class, so `new BorderLayout()` produces a new anonymous object of the class `BorderLayout`. This `BorderLayout` object is given the task of arranging components (in this case, labels) that are added to the `JFrame`.

It may help to note that the previous invocation of `setLayout` is equivalent to the following:

```
BorderLayout manager = new BorderLayout();
setLayout(manager);
```

Display 17.7 The `BorderLayout` Manager (part 1 of 2)

```
1  import javax.swing.JFrame;
2  import javax.swing.JLabel;
3  import java.awt.BorderLayout;

4  public class BorderLayoutJFrame extends JFrame
5  {
6      public static final int WIDTH = 500;
7      public static final int HEIGHT = 400;

8      public BorderLayoutJFrame()
9      {
10         super("BorderLayout Demonstration");
11         setSize(WIDTH, HEIGHT);
12         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

13         setLayout(new BorderLayout());

14         JLabel label1 = new JLabel("First label");
15         add(label1, BorderLayout.NORTH);

16         JLabel label2 = new JLabel("Second label");
17         add(label2, BorderLayout.SOUTH);

18         JLabel label3 = new JLabel("Third label");
19         add(label3, BorderLayout.CENTER);
20     }
21 }
```

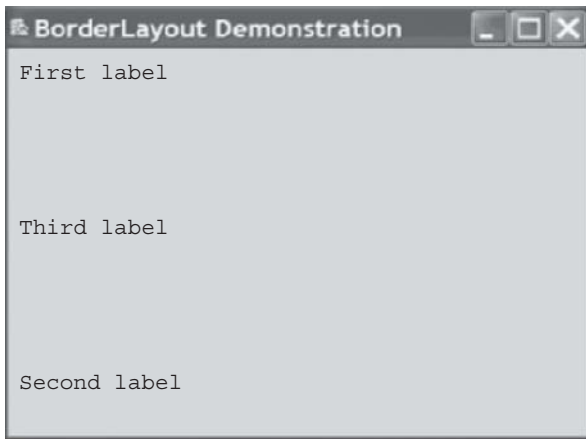
This is the file `BorderLayoutJFrame.java`.

(continued)

Display 17.7 The BorderLayout Manager (part 2 of 2) (Source: Oracle Corporation)

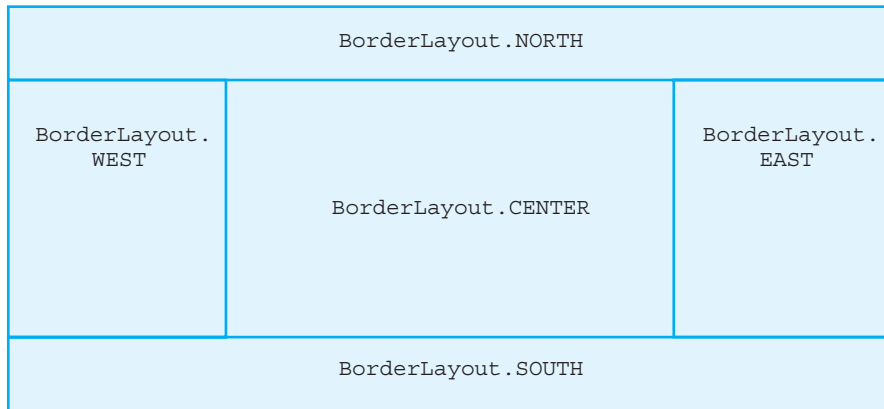
This is the file BorderLayoutDemo.java.

```
1 public class BorderLayoutDemo
2 {
3     public static void main(String[] args)
4     {
5         BorderLayoutJFrame gui = new BorderLayoutJFrame();
6         gui.setVisible(true);
7     }
8 }
```

RESULTING GUI

A BorderLayout manager places labels (or other components) into the five regions BorderLayout.NORTH, BorderLayout.SOUTH, BorderLayout.EAST, BorderLayout.WEST, and BorderLayout.CENTER. These five regions are arranged as shown in Display 17.8. The outside box represents the JFrame (or other container to which you will add things). None of the lines in the diagram will be visible unless you do something to make them visible. We drew them in to show you where each region is located.

Display 17.8 BorderLayout Regions



In Display 17.7, we added labels as follows:

```
JLabel label1 = new JLabel("First label");  
add(label1, BorderLayout.NORTH);  
  
JLabel label2 = new JLabel("Second label");  
add(label2, BorderLayout.SOUTH);  
  
JLabel label3 = new JLabel("Third label");  
add(label3, BorderLayout.CENTER);
```

When you use a BorderLayout manager, you give the location of the component added as a second argument to the method add, as in the following:

```
add(label1, BorderLayout.NORTH);
```

The labels (or other components to be added) need not be added in any particular order, because the second argument completely specifies where the label is placed.

BorderLayout.NORTH, BorderLayout.SOUTH, BorderLayout.EAST, BorderLayout.WEST, and BorderLayout.CENTER are five string constants defined in the class BorderLayout. The values of these constants are "North", "South", "East", "West", and "Center". Although you can use a quoted string such as "North" as the second argument to add, it is more consistent with our general style rules to use a defined constant such as BorderLayout.NORTH.

You need not use all five regions. For example, in Display 17.7 we did not use the regions BorderLayout.EAST and BorderLayout.WEST. If some regions are not used, any extra space is given to the BorderLayout.CENTER region, which is the largest region.

(The space is divided between regions as follows: Regions are allocated space in the order first north and south, second east and west, and last center. So, in particular, if there is nothing in the north region, then the east and west regions will extend to the top of the space.)

From this discussion, it sounds as though you can place only one item in each region, but later in this chapter, when we discuss *panels*, you will see that there is a way to group items so that more than one item can (in effect) be placed in each region.

There are some standard layout managers defined for you in the `java.awt` package, and you can also define your own layout managers. However, for most purposes, the layout managers defined in the standard libraries are all that you need, and we will not discuss how you can create your own layout manager classes.

Flow Layout Managers

The `FlowLayout` **manager** is the simplest layout manager. It arranges components one after the other, going from left to right, in the order in which you add them to the `JFrame` (or other container class) using the method `add`. For example, if the class in Display 17.7 had used the `FlowLayout` manager instead of the `BorderLayout` manager, it would have used the following code:

```
setLayout(new FlowLayout());

JLabel label1 = new JLabel("First label");
add(label1);

JLabel label2 = new JLabel("Second label");
add(label2);

JLabel label3 = new JLabel("Third label");
add(label3);
```

Layout Managers

The components that you add to a container class are arranged by an object known as a **layout manager**. Add a layout manager with the method `setLayout`, which is a method of every container class, such as a `JFrame` or an object of any of the other container classes that we will introduce later in this chapter. If you do not add a layout manager, a default layout manager will be provided for you.

SYNTAX

```
Container_Object.setLayout(new Layout_Manager_Class());
```

EXAMPLE (WITHIN A CONSTRUCTOR FOR A CLASS CALLED `BorderLayoutJFrame`)

```
public BorderLayoutJFrame()
{
    ...
    setLayout(new BorderLayout());

    JLabel label1 = new JLabel("First label");
    add(label1, BorderLayout.NORTH);

    JLabel label2 = new JLabel("Second label");
    add(label2, BorderLayout.SOUTH);

    ...
}
```

Note that if we had used the `FlowLayout` manager, as in the preceding code, then the `add` method would have only one argument. With a `FlowLayout` manager, the items are displayed in the order they are added, so that the labels above would be displayed all on one line as follows:

```
First label Second label Third label
```

[extra code
on website](#)

The full program is in the files `FlowLayoutJFrame.java` and `FlowLayoutDemo.java` on the accompanying website. You will see a number of examples of GUIs that use the `FlowLayout` manager class later in this chapter.

Grid Layout Managers

`GridLayout`

A **GridLayout manager** arranges components in a two-dimensional grid with some number of rows and columns. With a `GridLayout` manager, each entry is the same size. For example, the following says to use a `GridLayout` manager with `aContainer`, which can be a `JFrame` or other container:

```
aContainer.setLayout(new GridLayout(2, 3));
```

The two numbers given as arguments to the constructor `GridLayout` specify the number of rows and columns. This would produce the following sort of layout:

The lines will not be visible unless you do something special to make them so. They are just included here to show you the region boundaries.

When using a `GridLayout` manager, each component is stretched so that it completely fills its grid position.

Although you specify a number of rows and columns, the rules for the number of rows and columns is more complicated than what we have said so far. If the values for the number of rows and the number of columns are both nonzero, then the number of columns will be ignored. For example, if the specification is `new GridLayout(2, 3)`, then some sample sizes are as follows: If you add six items, the grid will be as shown. If you add seven or eight items, a fourth column is automatically added, and so forth. If you add fewer than six components, there will be two rows and a reduced number of columns.

There is another way to specify that the number of columns is to be ignored. You can do this by setting the number of columns to zero, which will allow any number of columns. So a specification of `(2, 0)` is equivalent to `(2, 3)`, and in fact is equivalent to `(2, n)` for any nonnegative value of `n`. Similarly, you can specify that the number of rows is to be ignored by setting the number of rows to zero, which will allow any number of rows.

When using the `GridLayout` class, the method `add` has only one argument. The items are placed in the grid from left to right, first filling the top row, then the second row, and so forth. You are not allowed to skip any grid position (although you will later see that you can add something that does not show and so gives the illusion of skipping a grid position).

A sample use of the `GridLayout` class is given in Display 17.9.

Display 17.9 The GridLayout Manager (part 1 of 2)

```
1  import javax.swing.JFrame;
2  import javax.swing.JLabel;
3  import java.awt.GridLayout;

4  public class GridLayoutJFrame extends JFrame
5  {
6      public static final int WIDTH = 500;
7      public static final int HEIGHT = 400;

8      public static void main(String[] args)
9      {
10         GridLayoutJFrame gui = new GridLayoutJFrame(2, 3);
11         gui.setVisible(true);
12     }

13     public GridLayoutJFrame(int rows, int columns )
14     {
15         super();
16         setSize(WIDTH, HEIGHT);
17         setTitle("GridLayout Demonstration");
18         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
19         setLayout(new GridLayout(rows, columns ));

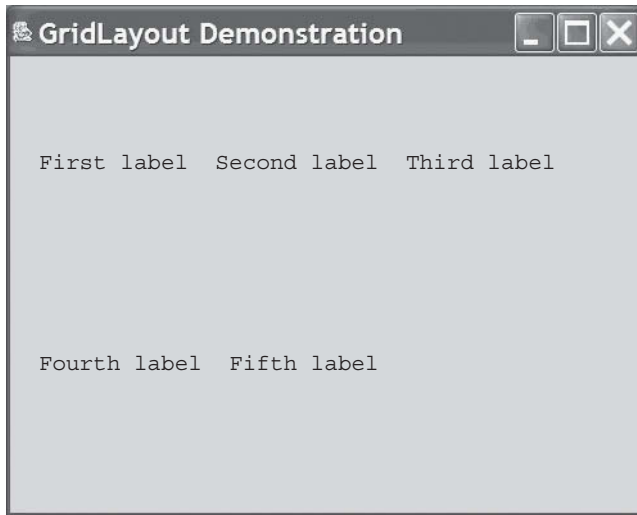
20         JLabel label1 = new JLabel("First label");
21         add(label1);

22         JLabel label2 = new JLabel("Second label");
23         add(label2);

24         JLabel label3 = new JLabel("Third label");
25         add(label3);

26         JLabel label4 = new JLabel("Fourth label");
27         add(label4);

28         JLabel label5 = new JLabel("Fifth label");
29         add(label5);
30     }
31 }
```

Display 17.9 The `GridLayout` Manager (part 2 of 2) (Source: Oracle Corporation)**RESULTING GUI**

Note that we have placed a demonstration `main` method in the class definition in Display 17.9. This is handy, but is not typical. Normally, a Swing GUI is created and displayed in a `main` method (or other method) in some class other than the class that defines the GUI. However, it is perfectly legal and sometimes convenient to place a `main` method in the GUI class definition so that it is easy to display a sample of the GUI. Note that the `main` method that is given in the class itself is written in the same way as a `main` method that is in some other class. In particular, you need to construct an object of the class, as in the following line from the `main` method in Display 17.9:

```
GridLayoutJFrame gui = new GridLayoutJFrame(2, 3);
```

The three layout managers we have discussed are summarized in Display 17.10.

Next we will discuss *panels*, which will let you realize the full potential of layout managers.



VideoNote
GUI Layout
using an IDE

Display 17.10 Some Layout Managers

LAYOUT MANAGER	DESCRIPTION
These layout manager classes are in the <code>java.awt</code> package.	
<code>FlowLayout</code>	Displays components from left to right in the order in which they are added to the container.
<code>BorderLayout</code>	Displays the components in five areas: north, south, east, west, and center. You specify the area a component goes into in a second argument of the <code>add</code> method.
<code>GridLayout</code>	Lays out components in a grid, with each component stretched to fill its box in the grid.

Self-Test Exercises

14. In Display 17.7, would it be legal to replace

```
JLabel label1 = new JLabel("First label");
add(label1, BorderLayout.NORTH);
JLabel label2 = new JLabel("Second label");
add(label2, BorderLayout.SOUTH);
JLabel label3 = new JLabel("Third label");
add(label3, BorderLayout.CENTER);
```

with the following?

```
JLabel aLabel = new JLabel("First label");
add(aLabel, BorderLayout.NORTH);
aLabel = new JLabel("Second label");
add(aLabel, BorderLayout.SOUTH);
aLabel = new JLabel("Third label");
add(aLabel, BorderLayout.CENTER);
```

In other words, can we reuse the variable `aLabel` or must each label have its own variable name?

15. How would you modify the class definition in Display 17.7 so that the three labels are displayed as follows?

```
First label
Second label
Third label
```

(There may be space between each pair of lines.)

Self-Test Exercises (continued)

16. How would you modify the class definition in Display 17.7 so that the three labels are displayed as follows?

First label

Second label

Third label

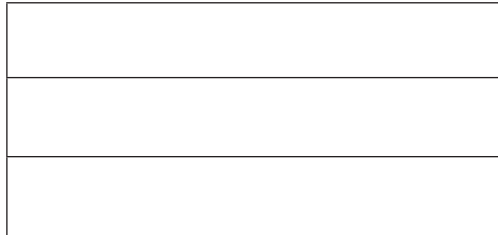
(There may be space between each pair of lines.)

17. Suppose you are defining a windowing GUI class in the usual way, as a derived class of the class `JFrame`, and suppose you want to specify a layout manager for the `JFrame` so as to produce the following sort of layout (that is, a one-row layout, typically having three columns):



What should the argument to `setLayout` be?

18. Suppose the situation is as described in Self-Test Exercise 17, except that you want the following sort of layout (that is, a one-column layout, typically having three rows):



What should the argument to `setLayout` be?

Panels

panel

A GUI is often organized in a hierarchical fashion, with window-like containers, known as *panels*, inside of other window-like containers. A **panel** is an object of the class `JPanel`, which is a very simple container class that does little more than group objects. It is one of the simplest container classes, but an extremely useful one. A `JPanel` object is analogous to the braces used to combine a number of simpler Java statements into a single larger Java statement. It groups smaller objects, such as buttons and labels, into a larger component (the `JPanel`). You can then put the `JPanel` object

in a `JFrame`. Thus, one of the main functions of `JPanel` objects is to subdivide a `JFrame` (or other container) into different areas.

For example, when you use a `BorderLayout` manager, you can place components in each of the five locations `BorderLayout.NORTH`, `BorderLayout.SOUTH`, `BorderLayout.EAST`, `BorderLayout.WEST`, and `BorderLayout.CENTER`. But what if you want to put two components at the bottom of the screen in the `BorderLayout.SOUTH` position? To do this, you would put the two components in a panel and then place the panel in the `BorderLayout.SOUTH` position.

You can give different layout managers to a `JFrame` and to each panel in the `JFrame`. Because you can add panels to other panels and each panel can have its own layout manager, this enables you to produce almost any kind of overall layout of the items in your GUI.

For example, if you want to place two buttons at the bottom of your `JFrame` GUI, you might add the following to the constructor of your `JFrame` GUI:

```
setLayout(new BorderLayout());

JPanel buttonPanel = new JPanel();
buttonPanel.setLayout(new FlowLayout());

JButton firstButton = new JButton("One");
buttonPanel.add(firstButton);

JButton secondButton = new JButton("Two");
buttonPanel.add(secondButton);

add(buttonPanel, BorderLayout.SOUTH);
```

The next Programming Example makes use of panels within panels.

EXAMPLE: A Tricolor Built with Panels

When first run, the GUI defined in Display 17.11 looks as shown in the first view. The entire background is light gray, and there are three buttons at the bottom of the GUI labeled "Blue", "White", and "Gray". If you click any one of the buttons, a vertical stripe with the color written on the button appears. You can click the buttons in any order. In the last three views in Display 17.11, we show what happens if you click the buttons in left-to-right order.

The blue, white, and gray stripes are the `JPanels` named `bluePanel`, `whitePanel`, and `grayPanel`. At first the panels are not visible because they are all light gray, so no borders are visible. When you click a button, the corresponding panel changes color and so is clearly visible.

Notice how the action listeners are set up. Each button registers the `this` parameter as a listener, as in the following line:

```
blueButton.addActionListener(this);
```

Because this line appears inside of the constructor for the class `PanelDemo`, the `this` parameter refers to `PanelDemo`, which is the entire GUI. Thus, the entire `JFrame`

(continued)

EXAMPLE: (continued)

(the entire GUI) is the listener, not the `JPanel`. So when you click one of the buttons, it is the `actionPerformed` method in `PanelDemo` that is executed.

When a button is clicked, the `actionPerformed` method is invoked with the action event fired as the argument to `actionPerformed`. The method `actionPerformed` recovers the string written on the button with the following line:

```
String buttonString = e.getActionCommand();
```

The method `actionPerformed` then uses a multiway if-else statement to determine if `buttonString` is "Blue", "White", or "Gray" and changes the color of the corresponding panel accordingly. It is common for an `actionPerformed` method to be based on such a multiway if-else statement, although we will see another approach in the subsection entitled "Listeners as Inner Classes" later in this chapter.

Display 17.11 also introduces one other small but new technique. We gave each button a color. We did this with the method `setBackground`, using basically the same technique that we used in previous examples. You can give a button or almost any other item a color using `setBackground`. Note that you do not use `getContentPane` when adding color to any component other than a `JFrame`.

Display 17.11 Using Panels (part 1 of 4)

```

1  import javax.swing.JFrame;
2  import javax.swing.JPanel;
3  import java.awt.BorderLayout;
4  import java.awt.GridLayout;
5  import java.awt.FlowLayout;
6  import java.awt.Color;
7  import javax.swing.JButton;
8  import java.awt.event.ActionListener;
9  import java.awt.event.ActionEvent;

10 public class PanelDemo extends JFrame implements ActionListener
11 {
12     public static final int WIDTH = 300;
13     public static final int HEIGHT = 200;

14     private JPanel bluePanel;
15     private JPanel whitePanel;
16     private JPanel grayPanel;

17     public static void main(String[] args)
18     {
19         PanelDemo gui = new PanelDemo();
20         gui.setVisible(true);
21     }

```

In addition to being the GUI class, the class `PanelDemo` is the action listener class. An object of the class `PanelDemo` is the action listener for the buttons in that object.

We made these instance variables because we want to refer to them in both the constructor and the method `actionPerformed`.

(continued)

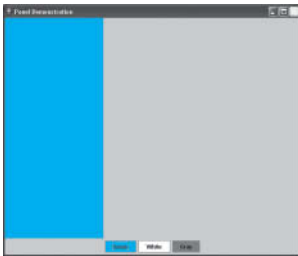
Display 17.11 Using Panels (part 2 of 4)

```
22     public PanelDemo()
23     {
24         super("Panel Demonstration");
25         setSize(WIDTH, HEIGHT);
26         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
27         setLayout(new BorderLayout());
28
29         JPanel biggerPanel = new JPanel();
30         biggerPanel.setLayout(new GridLayout(1, 3));
31
32         bluePanel = new JPanel();
33         bluePanel.setBackground(Color.LIGHT_GRAY);
34         biggerPanel.add(bluePanel);
35
36         whitePanel = new JPanel();
37         whitePanel.setBackground(Color.LIGHT_GRAY);
38         biggerPanel.add(whitePanel);
39
40         grayPanel = new JPanel();
41         grayPanel.setBackground(Color.LIGHT_GRAY);
42         biggerPanel.add(grayPanel);
43
44         add(biggerPanel, BorderLayout.CENTER);
45
46         JPanel buttonPanel = new JPanel();
47         buttonPanel.setBackground(Color.LIGHT_GRAY);
48         buttonPanel.setLayout(new FlowLayout());
49
50         JButton blueButton = new JButton("Blue");
51         blueButton.setBackground(Color.BLUE);
52         blueButton.addActionListener(this);
53         buttonPanel.add(blueButton);
54
55         JButton whiteButton = new JButton("White");
56         whiteButton.setBackground(Color.WHITE);
57         whiteButton.addActionListener(this);
58         buttonPanel.add(whiteButton);
59
60         JButton grayButton = new JButton("Gray");
61         grayButton.setBackground(Color.GRAY);
62         grayButton.addActionListener(this);
63         buttonPanel.add(grayButton);
64
65         add(buttonPanel, BorderLayout.SOUTH);
66     }
67     public void actionPerformed(ActionEvent e)
68     {
69         String buttonString = e.getActionCommand();
70
71         if (buttonString.equals("Blue"))
72             bluePanel.setBackground(Color.BLUE);
73     }
74 }
```

An object of the class `PanelDemo` is the action listener for the buttons in that object.

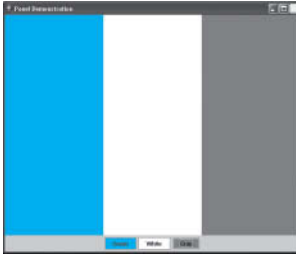
Display 17.11 Using Panels (part 3 of 4) (Source: Oracle Corporation)

```
62         else if (buttonString.equals("White"))
63             whitePanel.setBackground(Color.WHITE);
64         else if (buttonString.equals("Gray"))
65             grayPanel.setBackground(Color.GRAY);
66         else
67             System.out.println("Unexpected error.");
68     }
69 }
```

RESULTING GUI (When first run)**RESULTING GUI (After clicking Blue button)****RESULTING GUI (After clicking White button)**

(continued)

Display 17.11 Using Panels (part 4 of 4) (Source: Oracle Corporation)

RESULTING GUI (After clicking Gray button)**Adding Color**

Color a JFrame as follows:

```
JFrame_Object.getContentPane().setBackground(Color);
```

If this is inside a constructor for the JFrame, then the expression simplifies to

```
getContentPane().setBackground(Color);
```

or the equivalent

```
this.getContentPane().setBackground(Color);
```

Color a button, label, or any other component (which is not a JFrame) as follows:

```
Component_Object.setBackground(Color);
```

Note that `getContentPane()` is only used with a JFrame.

EXAMPLE (INSIDE A CONSTRUCTOR FOR A DERIVED CLASS OF JFrame)

```
getContentPane().setBackground(Color.WHITE);  
JButton redButton = new JButton("Red");  
redButton.setBackground(Color.RED);
```

The Container Class**Container**

The class called `Container` is in the `java.awt` package. Any descendent class of the class `Container` can have components added to it (or, more precisely, can have components added to objects of the class). The class `JFrame` is a descendent class of the

class `Container`, so any descendent class of the class `JFrame` can serve as a container to hold labels, buttons, panels, or other components.

Similarly, the class `JPanel` is a descendent of the class `Container`, and any object of the class `JPanel` can serve as a container to hold labels, buttons, other panels, or other components. Display 17.12 shows a portion of the hierarchy of Swing and AWT classes. Note that the `Container` class is in the AWT library and not in the Swing library. This is not a major issue, but it does mean that the `import` statement for the `Container` class is

```
import java.awt.Container;
```

container
class
component

A **container class** is any descendent class of the class `Container`. The class `JComponent` serves a similar role for components. Any descendent class of the class `JComponent` is called a `JComponent` or sometimes simply a **component**. You can add any `JComponent` object to any container class object.

The class `JComponent` is derived from the class `Container`, so you can add a `JComponent` to another `JComponent`. Often, this will turn out to be a viable option; occasionally it is something to avoid.⁶

The classes `Component`, `Frame`, and `Window` shown in Display 17.12 are AWT classes that some readers may have heard of. We include them for reference value, but we will have no need for these classes. We will eventually discuss all the other classes shown in Display 17.12.

When you are dealing with a Swing container class, you have three kinds of objects to deal with:

1. The container itself, probably some sort of panel or window-like object
2. The components you add to the container, such as labels, buttons, and panels
3. A layout manager, which positions the components inside the container

You have seen examples of these three kinds of objects in almost every `JFrame` class we have defined. Almost every complete GUI you build, and many subparts of the GUIs you build, will be made up of these three kinds of objects.

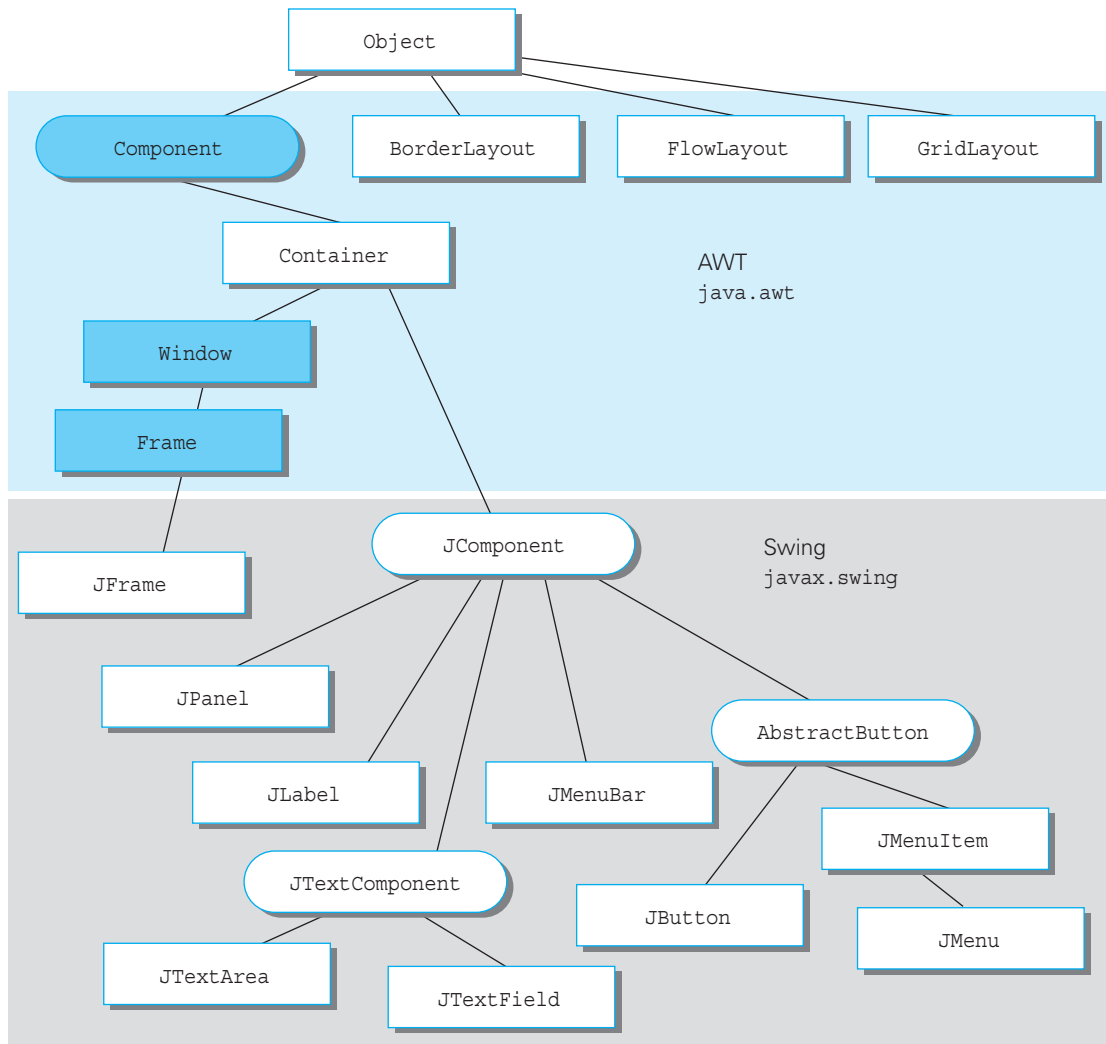
Self-Test Exercises

19. What standard Java package contains the layout manager classes discussed in this chapter?
20. Is an object of the class `JPanel` a container class? Is it a component class?
21. With a `GridLayout` manager, you cannot leave any grid element empty, but you can do something that will make a grid element look empty to the user. What can you do?

(continued)

⁶ In particular, it is legitimate and sometimes useful to add `JComponents` to a `JButton`. We do not have space in this book to develop techniques for doing this effectively, but you may want to give it a try. You have covered enough material to do it.

Display 17.12 Hierarchy of Swing and AWT Classes



Abstract Class

Concrete Class

A line between two boxes means the lower class is derived from (extends) the higher one.

This blue color indicates a class that is not used in this text but is included here for reference. If you have not heard of any of these classes, you can safely ignore them. (The class `Component` does receive very brief treatment in Chapter 19.)

Self-Test Exercises (continued)

22. You are used to defining derived classes of the Swing class `JFrame`. You can also define derived classes of other Swing classes. Define a derived class of the class `JPanel` that is called `PinkJPanel`. An object of the class `PinkJPanel` can be used just as we used objects of the class `JPanel`, but an object of the class `PinkJPanel` is pink in color (unless you explicitly change its color). The class `PinkJPanel` will have only one constructor—namely, the no-argument constructor. (*Hint:* This is very easy.)



TIP: Code a GUI's Look and Actions Separately

You can divide the task of designing a Swing GUI into two main subtasks: (1) designing and coding the appearance of the GUI on the screen; (2) designing and coding the actions performed in response to button clicks and other user actions. This dividing of one big task into two simpler tasks makes the big task easier and less error prone.

For example, consider the program in Display 17.11. Your first version of this program might use the following definition of the method `actionPerformed`:

```
public void actionPerformed(ActionEvent e)
{ }
```

This version of the method `actionPerformed` does nothing, but your program will run and will display a window on the screen, just as shown in Display 17.11. If you click any of the buttons, nothing will happen, but you can use this version of your GUI to adjust details, such as the order and location of buttons.

After you get the GUI to look the way you want it to look, you can define the action parts of the GUI, typically using the method `actionPerformed`.

If you include the phrase `implements ActionListener` at the start of your `JFrame` definition, then you must include some definition of the method `actionPerformed`. A method definition, such as

```
public void actionPerformed(ActionEvent e)
{ }
```

stub

which does nothing (or does very little) is called a **stub**. Using stubs is a good programming technique in many contexts, not just in Swing programs.

Alternatively, when writing your first version of a Swing GUI like the one in Display 17.11, you could omit the definition of the method `actionPerformed` completely, *provided you also omit the phrase `implements ActionListener` and omit the invocations of `addActionListener`.* ■

The Model-View-Controller Pattern ★

Model-View-Controller

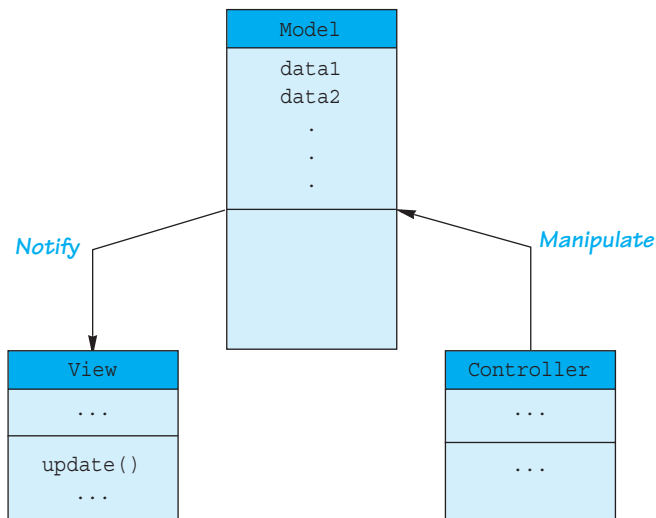
The technique we advocated in the previous Programming Tip is an example of a general technique known as the **Model-View-Controller** pattern. Display 17.13 gives a diagram of this pattern. The Model part of the pattern performs the heart of the application. The View part is the output part; it displays a picture of the Model's state. The Controller is the input part; it relays commands from the user to the Model. Each of the three interacting parts is realized as an object with responsibility for its own tasks. In a simple task such as the `JFrame` in Display 17.11, you can have a single object with different methods to realize each of the roles Model, View, and Controller.

To simplify the discussion, we have presented the Model-View-Controller pattern as if the user interacts directly with the Controller. The Controller need not be under the direct control of the user, but could be controlled by some other software or hardware component. In a Swing GUI, the View and Controller parts might be separate classes or separate methods combined into one larger class that displays a single window for all user interactions.

Self-Test Exercises

23. Suppose you omit the method `actionPerformed` from the class in Display 17.11 and make no other changes. Would the class compile? If it compiles, will it run with no error messages?
24. Suppose you omit the method `actionPerformed` and the phrase `implements ActionListener` from the class in Display 17.11 and make no other changes. Would the class compile? If it compiles, will it run with no error messages?

Display 17.13 The Model-View-Controller Pattern



17.4 Menus and Buttons

For hours and location press 1.

For a recorded message describing services press 2.

For instructions on using our website press 3.

To use our automated information system press 4.

To speak to an operator between 8 am and noon Monday through

Thursdays press 7.

PHONE ANSWERING MACHINE

In this section, we describe the basics of Swing menus. Swing menu items (menu choices) behave essentially the same as Swing buttons. They generate action events that are handled by action listeners, just as buttons do.

EXAMPLE: A GUI with a Menu

Display 17.14 contains a program that is essentially the same as the GUI in Display 17.11 except that this GUI uses a menu instead of buttons. This GUI has a menu bar at the top of the window. The menu bar lists the names of all the pull-down menus. This GUI has only one pull-down menu, which is named "Add Colors". However, there could be more pull-down menus in the same menu bar.

The user can pull down a menu by clicking its name in the menu bar. Display 17.14 contains three pictures of the GUI. The first is what you see when the GUI originally appears. In that picture, the menu name "Add Colors" can be seen in the menu bar, but you cannot see the menu. If you click the words "Add Colors" with your mouse, the menu drops down, as shown in the second picture of the GUI. If you click "Green", "White", or "Gray" on the menu, then a vertical strip of the named color appears in the GUI.

In the next subsection, we go over the details of the program in Display 17.14.

Menus, Menu Items, and Menu Bars

When adding menus as we did in Display 17.14, use the three Swing classes `JMenu`, `JMenuItem`, and `JMenuBar`. Entries on a menu are objects of the class `JMenuItem`. These `JMenuItems` are placed in `JMenus`, and then the `JMenus` are typically placed in a `JMenuBar`. Let us look at the details.

A **menu** is an object of the class `JMenu`. A choice on a menu is called a **menu item** and is an object of the class `JMenuItem`. A menu item is identified by the string that labels it, such as "Blue", "White", or "Gray" in the menu in Display 17.14. You can add as many `JMenuItems` as you wish to a menu. The menu lists the items in the order

menu

menu item

Display 17.14 A GUI with a Menu (part 1 of 3)

```
1  import javax.swing.JFrame;
2  import javax.swing.JPanel;
3  import java.awt.GridLayout;
4  import java.awt.Color;
5  import javax.swing.JMenu;
6  import javax.swing.JMenuItem;
7  import javax.swing.JMenuBar;
8  import java.awt.event.ActionListener;
9  import java.awt.event.ActionEvent;

10 public class MenuDemo extends JFrame implements ActionListener
11 {
12     public static final int WIDTH = 300;
13     public static final int HEIGHT = 200;

14     private JPanel bluePanel;
15     private JPanel whitePanel;
16     private JPanel grayPanel;

17     public static void main(String[] args)
18     {
19         MenuDemo gui = new MenuDemo();
20         gui.setVisible(true);
21     }

22     public MenuDemo()
23     {
24         super("Menu Demonstration");
25         setSize(WIDTH, HEIGHT);
26         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
27         setLayout(new GridLayout(1, 3));

28         bluePanel = new JPanel();
29         bluePanel.setBackground(Color.LIGHT_GRAY);
30         add(bluePanel);

31         whitePanel = new JPanel();
32         whitePanel.setBackground(Color.LIGHT_GRAY);
33         add(whitePanel);
34         grayPanel = new JPanel();
```

Display 17.14 A GUI with a Menu (part 2 of 3)

```
35         grayPanel.setBackground(Color.LIGHT_GRAY);
36         add(grayPanel);

37         JMenu colorMenu = new JMenu("Add Colors");

38         JMenuItem blueChoice = new JMenuItem("Blue");
39         blueChoice.addActionListener(this);
40         colorMenu.add(blueChoice);

41         JMenuItem whiteChoice = new JMenuItem("White");
42         whiteChoice.addActionListener(this);
43         colorMenu.add(whiteChoice);

44         JMenuItem grayChoice = new JMenuItem("Gray");
45         grayChoice.addActionListener(this);
46         colorMenu.add(grayChoice);

47         JMenuBar bar = new JMenuBar();
48         bar.add(colorMenu);
49         setJMenuBar(bar);
50     }
```

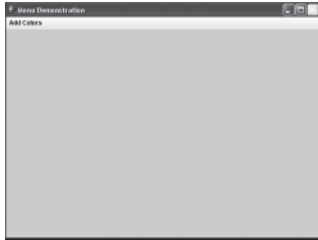
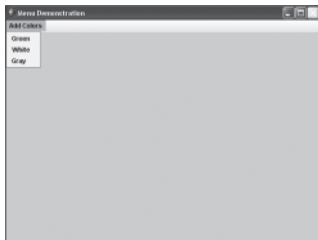
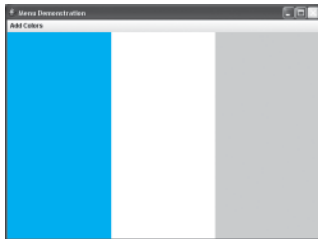
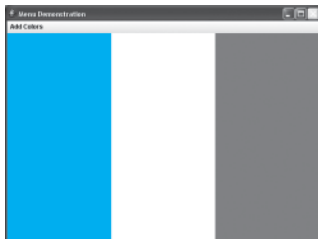
The definition of `actionPerformed` is identical to the definition given in Display 17.11 for a similar GUI using buttons instead of menu items.

```
51     public void actionPerformed(ActionEvent e)
52     {
53         String buttonString = e.getActionCommand();

54         if (buttonString.equals("Blue"))
55             bluePanel.setBackground(Color.BLUE);
56         else if (buttonString.equals("White"))
57             whitePanel.setBackground(Color.WHITE);
58         else if (buttonString.equals("Gray"))
59             grayPanel.setBackground(Color.GRAY);
60         else
61             System.out.println("Unexpected error.");
62     }
63 }
```

(continued)

Display 17.14 A GUI with a Menu (part 3 of 3) (Source: Oracle Corporation)

RESULTING GUI**RESULTING GUI (after clicking Add Colors in the menu bar)****RESULTING GUI (after choosing Blue and White on the menu)****RESULTING GUI (after choosing all the colors on the menu)**

in which they are added. The following code, taken from the constructor in Display 17.14, creates a new `JMenu` object named `colorMenu` and then adds a `JMenuItem` labeled "Red". Other menu items are added in a similar way.

```
JMenu colorMenu = new JMenu("Add Colors");

JMenuItem blueChoice = new JMenuItem("Blue");
blueChoice.addActionListener(this);
colorMenu.add(blueChoice);
```

listeners

Note that, just as we did for buttons in Display 17.11, in Display 17.14 we registered the `this` parameter as an action listener for each menu item. Defining action listeners and registering listeners for menu items are done in the exact same way as for buttons. In fact, the syntax is even the same. If you compare Displays 17.14 and 17.11, you will see that the definition of the method `actionPerformed` is the same in both classes.

Add a `JMenuItem` to an object of the class `JMenu` using the method `add` in exactly the same way that you add a component, such as a button, to a container object. Moreover, if you look at the preceding code, you will see that you specify a string for a `JMenuItem` in the same way that you specify a string to appear on a button.

menu bar

A **menu bar** is a container for menus, typically placed near the top of a windowing interface. Add a menu to a menu bar using the method `add` in the same way that you add menu items to a menu. The following code from the constructor in Display 17.14 creates a new menu bar named `bar` and then adds the menu named `colorMenu` to this menu bar:

```
JMenuBar bar = new JMenuBar();
bar.add(colorMenu);
```

There are two different ways to add a menu bar to a `JFrame`. You can use the method `setJMenuBar`, as shown in the following code from the constructor in Display 17.14:

```
setJMenuBar(bar);
```

This sets an instance variable of type `JMenuBar` so that it names the menu bar named `bar`. Saying it less formally, this adds the menu bar named `bar` to the `JFrame` and places the menu bar at the top of the `JFrame`.

Alternatively, you can use the `add` method to add a menu bar to a `JFrame` (or to any other container). You do so in the same way that you add any other component, such as a label or a button. An example of using `add` to add a `JMenuBar` to a `JFrame` is given in the file `MenuAdd.java` on the accompanying website.

extra code
on website

Menus

A **menu** is an object of the class `JMenu`. A choice on a menu is an object of the class `JMenuItem`. Menus are collected together in a **menu bar** (or menu bars). A menu bar is an object of the class `JMenuBar`.

Events and listeners for menu items are handled in exactly the same way as they are for buttons.

extra code
on website

Nested Menus ★

As shown in Display 17.12, the class `JMenu` is a descendent of the `JMenuItem` class. So, every `JMenu` object is also a `JMenuItem` object. Thus, a `JMenu` can be a menu item in another menu. This means that you can nest menus. For example, the outer menu might give you a list of menus. You can display one of the menus on that list by clicking the name of the desired menu. You can then choose an item from that menu by using your mouse again. There is nothing new you need to know to create these nested menus. Simply add menus to menus just as you add other menu items. There is an example of nested menus in the file `NestedMenus.java` on the accompanying website.

The `AbstractButton` Class

As shown in Display 17.12, the classes `JButton` and `JMenuItem` are derived classes of the abstract class named `AbstractButton`. All of the basic properties and methods of the classes `JButton` and `JMenuItem` are inherited from the class `AbstractButton`. That is why objects of the class `JButton` and objects of the class `JMenuItem` are so similar. Some of the methods for the class `AbstractButton` are listed in Display 17.15. All these methods are inherited by both the class `JButton` and the class `JMenuItem`. (Some of these methods were inherited by the class `AbstractButton` from the class `JComponent`, so you may sometimes see some of the methods listed as “inherited from `JComponent`.”)

Adding Menus to a `JFrame`

In the following, we assume that all code is inside a constructor for a (derived class of a) `JFrame`. To see the following examples put together to produce a complete GUI, see the constructor in Display 17.14.

Creating Menu Items

A menu item is an object of the class `JMenuItem`. Create a new menu item in the usual way, as illustrated by the following example. The string in the argument position is the displayed text for the menu item.

```
JMenuItem redChoice = new JMenuItem("Red");
```

Adding Menu Item Listeners

Events and listeners for menu items are handled in the exact same way as they are for buttons: Menu items fire action events that are received by objects of the class `ActionListener`.

SYNTAX

```
JMenu_Item_Name.addActionListener(Action_Listener);
```

EXAMPLE

```
redChoice.addActionListener(this);
```

Creating a Menu

A menu is an object of the class `JMenu`. Create a new menu in the usual way, as illustrated by the following example. The string argument is the displayed text that identifies the menu.

```
JMenu colorMenu = new JMenu("Add Colors");
```

Adding Menu Items to a Menu

Use the method `add` to add menu items to a menu.

SYNTAX

```
JMenu_Name.add(JMenu_Item);
```

EXAMPLE (colorMenu IS AN OBJECT OF THE CLASS JMenu)

```
colorMenu.add(redChoice);
```

Creating a Menu Bar

A menu bar is an object of the class `JMenuBar`. Create a new menu bar in the usual way, as illustrated by the following example:

```
JMenuBar bar = new JMenuBar();
```

Adding a Menu to a Menu Bar

Add a menu to a menu bar using the method `add` as follows:

SYNTAX

```
JMenu_Bar_Name.add(JMenu_Name);
```

EXAMPLE (bar IS AN OBJECT OF THE CLASS JMenuBar)

```
bar.add(colorMenu);
```

Adding a Menu Bar to a Frame

There are two different ways to add a menu bar to a `JFrame`. You can use the method `add` to add the menu bar to a `JFrame` (or to any other container). Another common way of adding a menu bar to a `JFrame` is to use the method `setJMenuBar` as follows:

SYNTAX

```
setJMenuBar(JMenu_Bar_Name);
```

EXAMPLE

```
setJMenuBar(bar);
```


Display 17.15 Some Methods in the Class `AbstractButton`

The abstract class `AbstractButton` is in the `javax.swing` package.

All of these methods are inherited by both of the classes `JButton` and `JMenuItem`.

```
public void setBackground(Color theColor)
```

Sets the background color of this component.

```
public void addActionListener(ActionListener listener)
```

Adds an `ActionListener`.

```
public void removeActionListener(ActionListener listener)
```

Removes an `ActionListener`.

```
public void setActionCommand(String actionCommand)
```

Sets the action command.

```
public String getActionCommand()
```

Returns the action command for this component.

```
public void setText(String text)
```

Makes text the only text on this component.

```
public String getText()
```

Returns the text written on the component, such as the text on a button or the string for a menu item.

```
public void setPreferredSize(Dimension preferredSize)
```

Sets the preferred size of the button or label. Note that this is only a suggestion to the layout manager. The layout manager is not required to use the preferred size. The following special case will work for most simple situations. The `int` values give the width and height in pixels.

```
public void setPreferredSize(  
    new Dimension(int width, int height))
```

```
public void setMaximumSize(Dimension maximumSize)
```

Sets the maximum size of the button or label. Note that this is only a suggestion to the layout manager. The layout manager is not required to respect this maximum size. The following special case will work for most simple situations. The `int` values give the width and height in pixels.

```
public void setMaximumSize(  
    new Dimension(int width, int height))
```

```
public void setMinimumSize(Dimension minimumSize)
```

Sets the minimum size of the button or label. Note that this is only a suggestion to the layout manager. The layout manager is not required to respect this minimum size.

Although we do not discuss the `Dimension` class, the following special case is intuitively clear and will work for most simple situations. The `int` values give the width and height in pixels.

```
public void setMinimumSize(  
    new Dimension(int width, int height))
```

The Dimension Class

Objects of the class `Dimension` are used with buttons, menu items, and other objects to specify a size. The `Dimension` class is in the package `java.awt`. The parameters in the following constructor are pixels.

CONSTRUCTOR

```
Dimension(int width, int height)
```

EXAMPLE

```
aButton.setPreferredSize(new Dimension(30, 50));
```

The `setActionCommand` Method

When the user clicks a button or menu item, it fires an action event that normally goes to one or more action listeners where it becomes an argument to an `actionPerformed` method. This action event includes a `String` instance variable that is known as the **action command** for the button or menu item and that is retrieved with the accessor method `getActionCommand`. The action command in the event is copied from an instance variable in the button or menu item object. If you do nothing to change it, the action command is the string written on the button or the menu item. The method `setActionCommand` given in Display 17.15 for the class `AbstractButton` can be used with any `JButton` or `JMenuItem` to change the action command for that component. Among other things, this will allow you to have different action commands for two buttons, two menu items, or a button and menu item even though they have the same string written on them.

The method `setActionCommand` takes a `String` argument that becomes the new action command for the calling button or menu item. For example, consider the following code:

```
JButton nextButton = new JButton("Next");
nextButton.setActionCommand("Next Button");
JMenuItem chooseNext = new JMenuItem("Next");
chooseNext.setActionCommand("Next Menu Item");
```

If we had not used `setActionCommand` in the preceding code, then the button `nextButton` and the menu item `chooseNext` would both have the action command `"Next"` and so we would have no way to tell which of the two components `nextButton` and `chooseNext` an action event `"Next"` came from. However, using the method `setActionCommand`, we can give them the different action commands `"Next Button"` and `"Next Menu Item"`.

The action command for a `JButton` or `JMenuItem` is kept as the value of a private instance variable for the `JButton` or `JMenuItem`. The method `setActionCommand` is simply an ordinary mutator method that changes the value of this instance variable.

action
command

setAction
Command

setActionCommand and getActionCommand

Every button and every menu item has a string associated with it that is known as the **action command** for that button or menu item. When the button or menu item is clicked, it fires an action event `e`. The following invocation returns the action command for the button or menu item that fired `e`:

```
e.getActionCommand( )
```

The method `actionPerformed` typically uses this action command string to decide which button or menu item was clicked.

The default action command for a button or menu item is the string written on it, but if you want, you can change the action command with an invocation of the method `setActionCommand`. For example, the menu item `chooseNext` created by the following code will display the string "Next" when it is a menu choice, but will have the string "Next Menu Item" as its action command.

EXAMPLE

```
JMenuItem chooseNext = new JMenuItem("Next");  
chooseNext.setActionCommand("Next Menu Item");
```

An alternate approach to defining action listeners is given in the next subsection. That technique is, among other things, another way to deal with multiple buttons or menu items that have the same thing written on them.

Listeners as Inner Classes ★

In all of our previous examples, our GUIs had only one action listener object to deal with all action events from all buttons and menus in the GUI. The opposite extreme also has much to recommend it. You can have a separate `ActionListener` class for each button or menu item, so that each button or menu item has its own unique action listener. There is then no need for a multiway `if-else` statement. The listener knows which button or menu item was clicked because it listens to only one button or menu item.

The approach outlined in the previous paragraph does have one down side: You typically need to give a lot of definitions of `ActionListener` classes. Rather than putting each of these classes in a separate file, it is much cleaner to make them private inner classes. This has the added advantage of allowing the `ActionListener` classes to have access to private instance variables and methods of the outer class.

In Display 17.16, we re-created the GUI in Display 17.14 using the techniques of this subsection.

Display 17.16 Listeners as Inner Classes (part 1 of 2)

```

    <Import statements are the same as in Display 17.14.>

1  public class InnerListenersDemo extends JFrame
2  {
3      public static final int WIDTH = 300;
4      public static final int HEIGHT = 200;

5      private JPanel bluePanel;
6      private JPanel whitePanel;
7      private JPanel grayPanel;

8      private class BlueListener implements ActionListener
9      {
10         public void actionPerformed(ActionEvent e)
11         {
12             bluePanel.setBackground(Color.BLUE);
13         }
14     } //End of BlueListener inner class

15     private class WhiteListener implements ActionListener
16     {
17         public void actionPerformed(ActionEvent e)
18         {
19             whitePanel.setBackground(Color.WHITE);
20         }
21     } //End of WhiteListener inner class
22     private class GrayListener implements ActionListener
23     {
24         public void actionPerformed(ActionEvent e)
25         {
26             grayPanel.setBackground(Color.GRAY);
27         }
28     } //End of GrayListener inner class

29     public static void main(String[] args)
30     {
31         InnerListenersDemo gui = new InnerListenersDemo();
32         gui.setVisible(true);
33     }

34     public InnerListenersDemo()
35     {
36         super("Menu Demonstration");
37         setSize(WIDTH, HEIGHT);
38         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
39         setLayout(new GridLayout(1, 3));

```

*The resulting GUI is the same as in
Display 17.14.*

(continued)

Display 17.16 Listeners as Inner Classes (part 2 of 2)

```
40         bluePanel = new JPanel();
41         greenPanel.setBackground(Color.LIGHT_GRAY);
42         add(bluePanel);

43         whitePanel = new JPanel();
44         whitePanel.setBackground(Color.LIGHT_GRAY);
45         add(whitePanel);

46         grayPanel = new JPanel();
47         grayPanel.setBackground(Color.LIGHT_GRAY);
48         add(grayPanel);

49         JMenu colorMenu = new JMenu("Add Colors");

50         JMenuItem blueChoice = new JMenuItem("Blue");
51         blueChoice.addActionListener(new BlueListener());
52         colorMenu.add(blueChoice);

53         JMenuItem whiteChoice = new JMenuItem("White");
54         whiteChoice.addActionListener(new WhiteListener());
55         colorMenu.add(whiteChoice);
56         JMenuItem grayChoice = new JMenuItem("Gray");
57         grayChoice.addActionListener(new GrayListener());
58         colorMenu.add(grayChoice);

59         JMenuBar bar = new JMenuBar();
60         bar.add(colorMenu);
61         setJMenuBar(bar);
62     }

63 }
```

Self-Test Exercises

25. What type of event is fired when you click a `JMenuItem`? How does it differ from the type of event fired when you click a `JButton`?
26. Write code to create a `JButton` with "Hello" written on it but with "Bye" as its action command.
27. Write code to create a `JMenuItem` with "Hello" as its displayed text (when it is a choice in a menu) but with "Bye" as its action command.

Self-Test Exercises (continued)

28. If you want to change the action command for a `JButton`, use the method `setActionCommand`. What method do you use to change the action command for a `JMenuItem`?
29. Is the following legal in Java?
- ```
JMenu aMenu = new JMenu();
...
JMenu aSubMenu = new JMenu();
...
aMenu.add(aSubMenu);
```
30. How many `JMenuBar` objects can you have in a `JFrame`?
31. A `JFrame` has a private instance variable of type `JMenuBar`. What is the name of the mutator method to change the value of this instance variable?
32. Write code to create a new menu item named `aChoice` that has the label "Exit".
33. Suppose you are defining a class called `MenuGUI` that is a derived class of the class `JFrame`. Write code to add the menu item `mItem` to the menu `m`. Then add `m` to the menu bar `mBar`, and then add the menu bar to the `JFrame` `MenuGUI`. Assume that this all takes place inside a constructor for `MenuGUI`. Also assume that everything has already been constructed with `new`, and that all necessary listeners are registered. You just need to do the adding.
34. ★How can you modify the program in Display 17.16 so that when the `Blue` menu item is clicked, all three colors are shown? The `Gray` and `White` choices remain the same. (Remember the menu items may be clicked in any order, so the `Blue` menu item can be the first or second item clicked.)
35. ★Rewrite the Swing GUI in Display 17.16 so that there is only one action listener inner class. The inner class constructor will have two parameters, one for a panel and one for a color.

## 17.5 Text Fields and Text Areas

*Write your answers in the spaces provided.*

### COMMON INSTRUCTION FOR AN EXAMINATION

You have undoubtedly interacted with windowing systems that provide spaces for you to enter text information such as your name, address, and credit card number. In this section, we show you how to add these fields for text input and text output to your Swing GUIs.

## Text Fields and Text Areas

### text field

#### JTextField

A **text field** is an object of the class `JTextField` and is displayed as a field that allows the user to enter a single line of text. In Display 17.17, the following creates a text field named `name` in which the user will be asked to enter his or her name:

```
private JTextField name;
...
name = new JTextField(NUMBER_OF_CHAR);
```

In Display 17.17, the variable `name` is a private instance variable. The creation of the `JTextField` in the last of the previous lines takes place inside the class constructor. The number `NUMBER_OF_CHAR` that is given as an argument to the `JTextField` constructor specifies that the text field will have room for at least `NUMBER_OF_CHAR` characters to be visible. The defined constant `NUMBER_OF_CHAR` is 30, so the text field is guaranteed to have room for at least 30 characters. You can type any number of characters into a text field but only a limited number will be visible; in this case, you know that at least 30 characters will be visible.

A Swing GUI can read the text in a text field and so receive text input; if this is desired, it can produce output by causing text to appear in the text field. The method `getText` returns the text written in the text field. For example, the following will set a variable named `inputString` to whatever string is in the text field `name` at the time that the `getText` method is invoked:

```
String inputString = name.getText();
```

The method `getText` is an input method, and the method `setText` is an output method. The method `setText` can be used to display a new text string in a text field. For example, the following will cause the text field `name` to change the text it displays to the string "This is some output":

```
name.setText("This is some output");
```

Display 17.17 A Text Field (part 1 of 3)

---

```
1 import javax.swing.JFrame;
2 import javax.swing.JTextField;
3 import javax.swing.JPanel;
4 import javax.swing.JLabel;
5 import javax.swing.JButton;
6 import java.awt.GridLayout;
7 import java.awt.BorderLayout;
8 import java.awt.FlowLayout;
9 import java.awt.Color;
10 import java.awt.event.ActionListener;
11 import java.awt.event.ActionEvent;

12 public class TextFieldDemo extends JFrame
13 implements ActionListener
14 {
15 public static final int WIDTH = 400;
16 public static final int HEIGHT = 200;
```

Display 17.17 A Text Field (part 2 of 3)

```

17 public static final int NUMBER_OF_CHAR = 30;

18 private JTextField name;

19 public static void main(String[] args)
20 {
21 TextFieldDemo gui = new TextFieldDemo();
22 gui.setVisible(true);
23 }

24 public TextFieldDemo()
25 {
26 super("Text Field Demo");
27 setSize(WIDTH, HEIGHT);
28 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
29 setLayout(new GridLayout(2, 1));

30 JPanel namePanel = new JPanel();
31 namePanel.setLayout(new BorderLayout());
32 namePanel.setBackground(Color.WHITE);

33 name = new JTextField(NUMBER_OF_CHAR);
34 namePanel.add(name, BorderLayout.SOUTH);
35 JLabel nameLabel = new JLabel("Enter your name here:");
36 namePanel.add(nameLabel, BorderLayout.CENTER);

37 add(namePanel);

38 JPanel buttonPanel = new JPanel();
39 buttonPanel.setLayout(new FlowLayout());
40 buttonPanel.setBackground(Color.BLUE);
41 JButton actionButton = new JButton("Click me");
42 actionButton.addActionListener(this);
43 buttonPanel.add(actionButton);

44 JButton clearButton = new JButton("Clear");
45 clearButton.addActionListener(this);
46 buttonPanel.add(clearButton);

47 add(buttonPanel);
48 }

49 public void actionPerformed(ActionEvent e)
50 {
51 String actionCommand = e.getActionCommand();

52 if (actionCommand.equals("Click me"))
53 name.setText("Hello " + name.getText());
54 else if (actionCommand.equals("Clear"))
55 name.setText("");
56 else
57 name.setText("Unexpected error.");
58 }
59 }

```

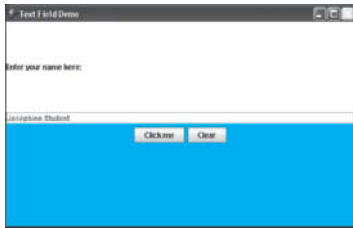
*This sets the text field equal to the empty string, which makes it blank.*

(continued)



Display 17.17 A Text Field (part 3 of 3) (Source: Oracle Corporation)

**RESULTING GUI** (When program is started and a name entered)



**RESULTING GUI** (After clicking the "Click me" button)



The following line from the method `actionPerformed` in Display 17.17 uses both `getText` and `setText`:

```
name.setText("Hello" + name.getText());
```

This line changes the string in the text field `name` to "Hello" followed by the old string value in the text field. The net effect is to insert the string "Hello" in front of the string displayed in the text field.

### **getText and setText**

The classes `JTextField` and `JTextArea` both contain methods called `getText` and `setText`. The method `getText` can be used to retrieve the text written in the text field or text area. The method `setText` can be used to change the text written in the text field or text area.

#### **SYNTAX**

*Name\_of\_Text\_Component.getText()* returns the text currently displayed in the text field or text area.

*Name\_of\_Text\_Component.setText(New\_String\_To\_Display);*

**EXAMPLES**

```
String inputString = ioComponent.getText();
ioComponent.setText("Hello out there!");
```

`ioComponent` may be an instance of either of the classes `JTextField` or `JTextArea`.

**text area****JTextArea**

A **text area** is an object of the class `JTextArea`. A text area is the same as a text field except that it allows multiple lines. Two parameters to the constructor for `JTextArea` specify the minimum number of lines and the minimum number of characters per line that are guaranteed to be visible. You can enter any amount of text in a text area, but only a limited number of lines and a limited number of characters per line will be visible. For example, the following creates a `JTextArea` named `theText` that will have at least 5 lines and at least 20 characters per line visible:

```
JTextArea theText = new JTextArea(5, 20);
```

There is also a constructor with one additional `String` parameter for the string initially displayed in the text area. For example,

```
JTextArea theText = new JTextArea("Enter\ntext here.", 5, 20);
```

Note that a string value can be multiple lines because it can contain the new-line character `'\n'`.

A `JTextField` has a similar constructor with a `String` parameter, as in the following example:

```
JTextField ioField =
 new JTextField("Enter numbers here.", 30);
```

If you look at Display 17.12, you will see that both `JTextField` and `JTextArea` are derived classes of the abstract class `JTextComponent`. Most of the methods for `JTextField` and `JTextArea` are inherited from `JTextComponent` and so `JTextField` and `JTextArea` have mostly the same methods with the same meanings except for minor redefinitions to account for having just one line or multiple lines. Display 17.18 describes some methods in the class `JTextComponent`. All of these methods are inherited and have the described meaning in both `JTextField` and `JTextArea`.

**setLineWrap**

You can set the line-wrapping policy for a `JTextArea` using the method `setLineWrap`. The method takes one argument of type `boolean`. If the argument is `true`, then at the end of a line, any additional characters for that line will appear on the following line of the text area. If the argument is `false`, the extra characters will be on the same line and will not be visible. For example, the following sets the line wrap policy for the `JTextArea` object named `theText` so that at the end of a line, any additional characters for that line will appear on the following line:

```
theText.setLineWrap(true);
```

**output-only****setEditable**

You can specify that a `JTextField` or `JTextArea` cannot be written in by the user. To do so, use the method `setEditable`, which is a method in both the `JTextField`

and `JTextArea` classes. If `theText` names an object in either of the classes `JTextField` or `JTextArea`, then the following

```
theText.setEditable(false);
```

will set `theText` so that only your GUI program can change the text in the text component `theText`; the user cannot change the text. After this invocation of `setEditable`, if the user clicks the mouse in the text component named `theText` and then types at the keyboard, the text in the text component will not change.

To reverse things and make `theText` so that the user can change the text in the text component, use `true` in place of `false`, as follows:

```
theText.setEditable(true);
```

If no invocation of `setEditable` is made, then the default state allows the user to change the text in the text component.

#### Display 17.18 Some Methods in the Class `JTextComponent`

---

All these methods are inherited by the classes `JTextField` and `JTextArea`.

The abstract class `JTextComponent` is in the package `javax.swing.text`. The classes `JTextField` and `JTextArea` are in the package `javax.swing`.

```
public String getText()
```

Returns the text that is displayed by this text component.

```
public boolean isEditable()
```

Returns `true` if the user can write in this text component. Returns `false` if the user is not allowed to write in this text component.

```
public void setBackground(Color theColor)
```

Sets the background color of this text component.

```
public void setEditable(boolean argument)
```

If `argument` is `true`, then the user is allowed to write in the text component. If `argument` is `false`, then the user is not allowed to write in the text component.

```
public void setText(String text)
```

Sets the text that is displayed by this text component to be the specified text.

---

### The Classes `JTextField` and `JTextArea`

The classes `JTextField` and `JTextArea` can be used to add areas for changeable text to a GUI. An object of the class `JTextField` has one line that displays some specified number of characters. An object of the class `JTextArea` has a size consisting of a specified number of lines and a specified number of characters per line. More text can be typed into

a `JTextField` or `JTextArea` than is specified in its size, but the extra text may not be visible.

The number of characters per line and the number of lines are a guaranteed minimum. More lines and especially more characters per line may be visible. (The space per line is actually guaranteed to be *Characters\_Per\_Line* times the space for one uppercase letter M.)

### SYNTAX

```
JTextField Name_of_Text_Field = new JTextField(Characters_Per_Line);
JTextArea Name_of_Text_Area =
 new JTextArea(Number_of_Lines, Characters_Per_Line);
```

### EXAMPLES

```
JTextField name = new JTextField(30);
JTextArea someText = new JTextArea(10, 30);
```

There are also constructors that take an additional `String` argument that specifies an initial string to display in the text component.

### SYNTAX

```
JTextField Name_of_Text_Field =
 new JTextField(Initial_String, Characters_Per_Line);
JTextArea Name_of_Text_Area =
 new JTextArea(Initial_String, Number_of_Lines, Characters_Per_Line);
```

### EXAMPLES

```
JTextField name = new JTextField("Enter name here.", 30);
JTextArea someText =
 new JTextArea("Enter story here.\nClick button.", 10, 30);
```

## Number of Characters per Line

The number of characters per line (given as an argument to constructors for `JTextField` or `JTextArea`) is not the number of just any characters. The number gives the number of em spaces in the line. An **em space** is the space needed to hold one uppercase letter M, which is the widest letter in the alphabet. So a line that is specified to hold 20 characters will always be able to hold at least 20 characters and will almost always hold more than 20 characters.

## Scroll Bars

Scroll bars for text areas and text fields are discussed in Chapter 18. They are a nice touch, but until you reach Chapter 18, your GUI programs will work fine without them.



### TIP: Labeling a Text Field

Sometimes you want to label a text field. For example, suppose the GUI asks for a name and a credit card number and expects the user to enter these in two text fields. In this case, the GUI needs to label the two text fields so that the user knows in which field to type the name and in which field to type the credit card number. You can use an object of the class `JLabel` to label a text field or any other component in a Swing GUI. Simply place the label and text field in a `JPanel` and treat the `JPanel` as a single component. For example, we did this with the text field name in Display 17.17. ■

### Self-Test Exercises

36. What is the difference between an object of the class `JTextArea` and an object of the class `JTextField`?
37. What would happen if when running the GUI in Display 17.17 you were to enter your name and click the "Click me" button three times?
38. Rewrite the program in Display 17.17 so that it uses a text area in place of a text field. Change the label "Enter your name here:" to "Enter your story here:". When the user clicks the "Click me" button, your GUI should change the string displayed in the text area to "Your story is " + `lineCount` + "lines long.". The variable `lineCount` is a variable of type `int` that your program sets equal to the number of lines currently displayed in the text area. Use a `BorderLayout` manager and place your text area in the region `BorderLayout.CENTER` so that there is room for it. You can assume the user enters at least one line before clicking the "Click me" button. The last line in the text area will have no `'\n'` so you may need to add one if you are counting the number of occurrences of `'\n'`. Blank lines are counted.



### TIP: Inputting and Outputting Numbers

When you want to input numbers using a Swing GUI, your GUI must convert input text to numbers. For example, when you input the number 42 in a `JTextField`, your program will receive the string "42", not the number 42. Your program must convert the input string value "42" to the integer value 42. When you want to output numbers using a GUI constructed with Swing, you must convert the numbers to a string and then output that string. For example, if you want to output the number 40, your program would convert the integer value 40 to the string value "40". With Swing, all input typed by the user is string input and all displayed output is string output. The techniques for converting back and forth between strings and numbers were given in Chapter 5 in the subsection titled "Static Methods in Wrapper Classes" and the Programming Example entitled "Another Approach to Keyboard Input." A simple example of a Swing GUI with numeric input and output is given in the next subsection. ■

## A Swing Calculator

Designing a realistic Swing calculator is the subject of Programming Project 17.3. In this programming example, we will develop a simplified calculator to get you started on that Programming Project. Display 17.19 contains a GUI for a calculator that keeps a running total of numbers. The user enters a number in the text field and then clicks either the + or − button. The number in the text field is then added into or subtracted from a running total that is kept in the instance variable `result`, and then the new total (the new value of `result`) is given in the text field. If the user clicks the "Reset" button, then the running total—that is, the value of `result`—is set to zero. When the GUI is first run, the running total, that is, the value of `result`, is set to zero.

Most of the details are similar to things you have already seen, but one new element is the use of exception handling. If the user enters a number in an incorrect format, such as placing a comma in a number, then one of the methods throws a `NumberFormatException`. If the user enters a number in an incorrect format, such as 2,000 with a comma instead of 2000, the method `assumingCorrectNumberFormats` invokes the method `stringToDouble` with the alleged number string "2,000" as an argument. Then `stringToDouble` calls `Double.parseDouble`, but `Double.parseDouble` throws a `NumberFormatException` because no Java number string can contain a comma. Because the invocation of `Double.parseDouble` takes place within an invocation of the method `stringToDouble`, `stringToDouble` in turn throws a `NumberFormatException`. The invocation of `stringToDouble` takes place inside the invocation of `assumingCorrectNumberFormats`, so `assumingCorrectNumberFormats` throws the `NumberFormatException` that it received from the invocation of `stringToDouble`. However, the invocation of `assumingCorrectNumberFormats` is inside a `try` block. The exception is caught in the following `catch` block. At this point, the `JTextField` (named `ioField`) is set to the error message "Error: Reenter Number."

Notice that if a `NumberFormatException` is thrown, the value of the instance variable `result` is not changed. A `NumberFormatException` can be thrown by an invocation of `stringToDouble` in either of the following lines of code from the method `assumingCorrectNumberFormats`:

```
result = result + stringToDouble(ioField.getText());
```

or

```
result = result - stringToDouble(ioField.getText());
```

If the exception is thrown, execution of the method `stringToDouble` ends immediately and control passes to the `catch` block. Thus, control passes to the `catch` block before the previous addition or subtraction is performed. So `result` is unchanged, and the user can reenter the last number and proceed with the GUI as if that incorrect number were never entered.

### Uncaught Exceptions

In a Swing program, throwing an uncaught exception does not end the GUI, but it may leave it in an unpredictable state. It is best to always catch any exception that is thrown even if all that the `catch` block does is output an instruction to redo something, such as reentering some input or just outputting an error message.

Display 17.19 A Simple Calculator (part 1 of 4)

```
1 import javax.swing.JFrame;
2 import javax.swing.JTextField;
3 import javax.swing.JPanel;
4 import javax.swing.JLabel;
5 import javax.swing.JButton;
6 import java.awt.BorderLayout;
7 import java.awt.FlowLayout;
8 import java.awt.Color;
9 import java.awt.event.ActionListener;
10 import java.awt.event.ActionEvent;

11 /**
12 * A simplified calculator.
13 * The only operations are addition and subtraction.
14 */
15 public class Calculator extends JFrame
16 implements ActionListener
17 {
18 public static final int WIDTH = 400;
19 public static final int HEIGHT = 200;
20 public static final int NUMBER_OF_DIGITS = 30;

21 private JTextField ioField;
22 private double result = 0.0;

23 public static void main(String[] args)
24 {
25 Calculator aCalculator = new Calculator();
26 aCalculator.setVisible(true);
27 }

28 public Calculator()
29 {
30 setTitle("Simplified Calculator");
31 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
32 setSize(WIDTH, HEIGHT);
33 setLayout(new BorderLayout());

34 JPanel textPanel = new JPanel();
35 textPanel.setLayout(new FlowLayout());
36 ioField =
37 new JTextField("Enter numbers here.", NUMBER_OF_DIGITS);
38 ioField.setBackground(Color.WHITE);
39 textPanel.add(ioField);
40 add(textPanel, BorderLayout.NORTH);
41 JPanel buttonPanel = new JPanel();
42 buttonPanel.setBackground(Color.BLUE);
43 buttonPanel.setLayout(new FlowLayout());
```

Display 17.19 A Simple Calculator (part 2 of 4)

```

44 JButton addButton = new JButton("+");
45 addButton.addActionListener(this);
46 buttonPanel.add(addButton);
47 JButton subtractButton = new JButton("-");
48 subtractButton.addActionListener(this);
49 buttonPanel.add(subtractButton);
50 JButton resetButton = new JButton("Reset");
51 resetButton.addActionListener(this);
52 buttonPanel.add(resetButton);

53 add(buttonPanel, BorderLayout.CENTER);
54 }

55 public void actionPerformed(ActionEvent e)
56 {
57 try
58 {
59 assumingCorrectNumberFormats(e);
60 }
61 catch (NumberFormatException e2)
62 {
63 ioField.setText("Error: Reenter Number.");
64 }
65 }

```

*A `NumberFormatException` does not need to be declared or caught in a catch block.*

```

66 //Throws NumberFormatException.
67 public void assumingCorrectNumberFormats(ActionEvent e)
68 {
69 String actionCommand = e.getActionCommand();

70 if (actionCommand.equals("+"))
71 {
72 result = result + stringToDouble(ioField.getText());
73 ioField.setText(Double.toString(result));
74 }
75 else if (actionCommand.equals("-"))
76 {
77 result = result - stringToDouble(ioField.getText());
78 ioField.setText(Double.toString(result));
79 }
80 else if (actionCommand.equals("Reset"))
81 {
82 result = 0.0;
83 ioField.setText("0.0");
84 }
85 else
86 ioField.setText("Unexpected error.");
87 }

```

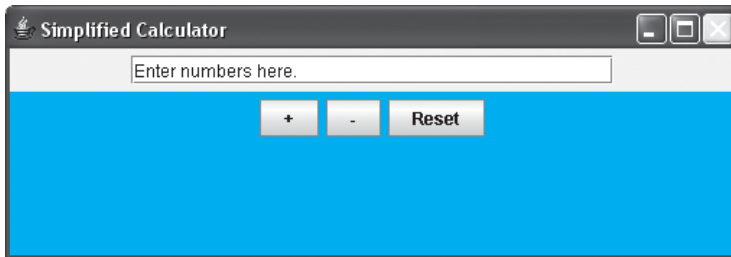
(continued)



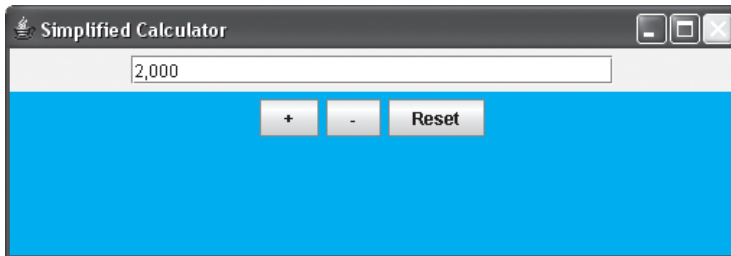
Display 17.19 A Simple Calculator (part 3 of 4) (Source: Oracle Corporation)

```
88 //Throws NumberFormatException.
89 private static double stringToDouble(String stringObject)
90 {
91 return Double.parseDouble(stringObject.trim());
92 }
93 }
```

**RESULTING GUI** (When started)



**RESULTING GUI** (After entering 2,000)



**RESULTING GUI** (After clicking +)



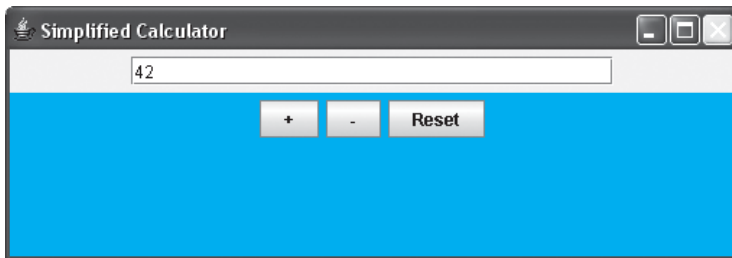
Display 17.19 A Simple Calculator (part 4 of 4) (Source: Oracle Corporation)

---

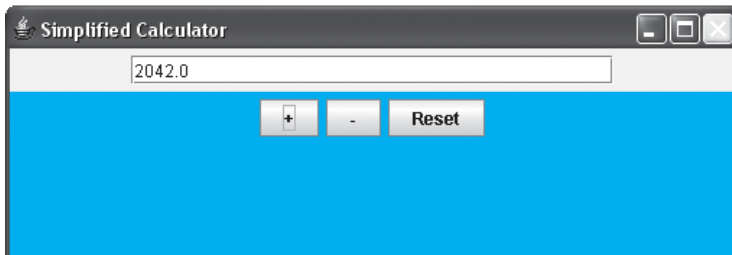
**RESULTING GUI** (After entering 2000.0 and clicking +)



**RESULTING GUI** (After entering 42)



**RESULTING GUI** (After clicking +)



---

## Self-Test Exercises

39. In the GUI in Display 17.19, why did we make the text field `ioField` an instance variable but did not make instance variables of any of the buttons `addButton`, `subtractButton`, or `resetButton`?
40. What would happen if the user running the GUI in Display 17.19 were to run the GUI and simply click the addition button without typing anything into the text field?

(continued)

**Self-Test Exercises** (continued)

41. What would happen if the user running the GUI in Display 17.19 were to type the number 10 into the text field and then click the addition button three times? Explain your answer.
42. Suppose you change the `main` method in Display 17.19 to the following:

```
public static void main(String[] args)
{
 Calculator calculator1 = new Calculator();
 calculator1.setVisible(true);
 Calculator calculator2 = new Calculator();
 calculator2.setVisible(true);
}
```

This will cause two calculator windows to be displayed. (If one is on top of the other, you can use your mouse to move the top one.) If you add numbers in one of these calculators, will anything change in the other calculator?

43. Suppose you change the `main` method in Display 17.19 as we described in Self-Test Exercise 42. This will cause two calculator windows to be displayed. If you click the close-window button in one of the windows, will one window go away or will both windows go away?

## Chapter Summary

- Swing GUIs (graphical user interfaces) are programmed using event-driven programming. In event-driven programming, a user action, such as a button click, generates an event, which is automatically passed to an event-handling method that performs the appropriate action.
- There are two main techniques for designing a Swing GUI class. You can use inheritance to create a derived class of one of the library classes such as `JFrame` or you can build a GUI by adding components to a container class. You normally use both of these techniques when defining a Swing GUI class.
- A *windowing GUI* is usually defined as a derived class of the class `JFrame`.
- A *button* is an object of the class `JButton`. Clicking a button fires an action event that is handled by an action listener. An *action listener* is any class that implements the `ActionListener` interface.
- A *label* is an object of the class `JLabel`. You can use a label to add text to a GUI.
- When adding components to an object of a container class, such as adding a button to a panel or `JFrame`, use the method `add`. The components in a container are arranged by an object called a *layout manager*.

- A *panel* is a container object that is used to group components inside of a larger container. Panels are objects in the class `JPanel`.
- A *menu item* is a choice on a menu. A menu item is realized in your code as an object of the class `JMenuItem`. A *menu* is an object of the class `JMenu`. A menu item is added to a `JMenu` with the method `add`. A *menu bar* is an object of the class `JMenuBar`. A menu is added to a `JMenuBar` with the method `add`.
- A `JMenuBar` can be added to a `JFrame` with the method `setJMenuBar`. It can also be added using the method `add`, just as any other component can be added.
- Both buttons and menu items fire action events and so normally have one or more action listeners registered with them to respond to the events.

## Answers to Self-Test Exercises

1. The `JFrame` class.
2. Sizes in Swing are measured in pixels.
3. `someWindow.setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);`
4. `someWindow.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);`
5. When you click the minimizing button, the `JFrame` is reduced to an icon, usually at the bottom of your monitor screen.
6. `someWindow.setVisible(n > 0);`

The following also works but is not good style:

```
if (n > 0)
 someWindow.setVisible(true);
else
 someWindow.setVisible(false);
```

7. An action event.
8. `public void actionPerformed(ActionEvent e)`
9. Change

```
JFrame firstWindow = new JFrame();
```

to

```
JFrame firstWindow = new JFrame("My First Window");
```

Alternatively, you can add the following:

```
firstWindow.setTitle("My First Window");
```

## 10. Delete

```
setTitle("First Window Class");
and replace
super();
with
super("First Window Class");
```

## 11. Change

```
setDefaultCloseOperation(
 JFrame.DO_NOTHING_ON_CLOSE);
to
setDefaultCloseOperation(
 JFrame.EXIT_ON_CLOSE);
```

## 12. Change the following line in the no-argument constructor in Display 17.6 from

```
this(Color.PINK);
to
this(Color.MAGENTA);
```

## 13. add(new JLabel("Close-window button works."));

## 14. Yes, it is legal. It is OK to reuse a variable name such as aLabel.

## 15. You need to change the add statements, as in the following rewritten section of code:

```
JLabel label1 = new JLabel("First label");
add(label1, BorderLayout.NORTH);

JLabel label2 = new JLabel("Second label");
add(label2, BorderLayout.CENTER);

JLabel label3 = new JLabel("Third label");
add(label3, BorderLayout.SOUTH);
```

## 16. You need to change the add statements, as in the following rewritten section of code:

```
JLabel label1 = new JLabel("First label");
add(label1, BorderLayout.NORTH);

JLabel label2 = new JLabel("Second label");
add(label2, BorderLayout.EAST);

JLabel label3 = new JLabel("Third label");
add(label3, BorderLayout.SOUTH);
```

## 17. The argument should be new GridLayout(1, 3). So, the entire method invocation is

```
setLayout(new GridLayout(1, 3));
```

Alternatively, you could use `new GridLayout(1, 0)`. It is also possible to do something similar with a `BorderLayout` manager or a `FlowLayout` manager, but a `GridLayout` manager will work nicer here.

18. The argument should be `new GridLayout(0, 1)`. So, the entire method invocation is `setLayout(new GridLayout(0, 1));`

Alternatively, you could use `new GridLayout(3, 1)`, if you know there will be at most three components added, but if more than three components are added, then a second column will be added. It is also possible to do something similar with a `BorderLayout` manager, but a `GridLayout` manager will work nicer here.

19. `java.awt`
20. An object of the class `JPanel` is both a container class and a component class.
21. To make it look as though you have an empty grid element, add an empty panel to the grid element.
22. `import javax.swing.JPanel;`  
`import java.awt.Color;`

```
public class PinkJPanel extends JPanel
{
 public PinkJPanel()
 {
 setBackground(Color.PINK);
 }
}
```

extra code  
on website

The class `PinkJPanel` is on the website that accompanies this text.

23. It will not compile, but will give a compiler error message saying that `actionPerformed` is not defined (because it claims to implement the `ActionListener` interface).
24. It will not compile, but will give compiler error messages saying that, in effect, the invocations of `addActionListener` such as `redButton.addActionListener(this);` have arguments of an incorrect type.
25. Clicking a `JMenuItem` fires an action event (that is, an object of the class `ActionEvent`). This is the same as with a `JButton`.
26. `JButton b = new JButton("Hello");`  
`b.setActionCommand("Bye");`
27. `JMenuItem m = new JMenuItem("Hello");`  
`m.setActionCommand("Bye");`
28. To change the action command for a `JMenuItem`, use the method `setActionCommand`, just as you would for a `JButton`.
29. Yes, it is legal.

30. As many as you want. Only one can be added with the method `setJMenuBar`, but any number of others can be added to using the `add` method.

31. `setJMenuBar`

32. `JMenuItem aChoice = new  
JMenuItem("Exit");`

33. `m.add(mItem);  
mBar.add(m);  
setJMenuBar(mBar);`

You could use the following instead of using `setJMenuBar`:

`add(mBar);`

This will all take place inside a constructor named `MenuGUI`.

34. Register all three types of listeners with `blueChoice`, as follows:

```
blueChoice.addActionListener(new GrayListener());
blueChoice.addActionListener(new WhiteListener());
blueChoice.addActionListener(new BlueListener());
```

35. Replace the three inner classes with the following inner class:

```
private class ColorListener implements ActionListener
{
 private JPanel thePanel;
 private Color theColor;

 public ColorListener(Color c, JPanel p)
 {
 theColor = c;
 thePanel = p;
 }

 public void actionPerformed(ActionEvent e)
 {
 thePanel.setBackground(theColor);
 }
} //End of ColorListener inner class
```

Replace

```
blueChoice.addActionListener(new BlueListener());
```

with

```
blueChoice.addActionListener(
 new ColorListener(Color.BLUE, bluePanel));
```

Also make similar changes to the menu items `whiteChoice` and `grayChoice`, with the obvious changes to colors and panels.

This is not really preferable to what we did in Display 17.16, but it is a good exercise. The complete program done this way is on the accompanying website in the file named `InnerListenersDemo2.java`.

36. A `TextField` object displays only a single line. A `TextArea` object can display more than one line of text.
37. The contents of the text field would change to "Hello Hello Hello " followed by your name.
38. This program is on the website that accompanies this text.

extra code  
on website

```
import javax.swing.JFrame;
import javax.swing.JTextArea;
import javax.swing.JPanel;
import javax.swing.JLabel;
import javax.swing.JButton;
import java.awt.GridLayout;
import java.awt.BorderLayout;
import java.awt.FlowLayout;
import java.awt.Color;
import java.awt.event.ActionListener;
import java.awt.event.ActionEvent;

public class TextAreaDemo extends JFrame
 implements ActionListener
{
 public static final int WIDTH = 400;
 public static final int HEIGHT = 200;
 public static final int NUMBER_OF_LINES = 10;
 public static final int NUMBER_OF_CHAR = 30;

 private JTextArea story;

 public static void main(String[] args)
 {
 TextAreaDemo gui = new TextAreaDemo();
 gui.setVisible(true);
 }

 public TextAreaDemo()
 {
 setTitle("Text Area Demo");
 setSize(WIDTH, HEIGHT);
 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
 setLayout(new GridLayout(2, 1));
 JPanel storyPanel = new JPanel();
 storyPanel.setLayout(new BorderLayout());
 storyPanel.setBackground(Color.WHITE);

 story = new JTextArea(NUMBER_OF_LINES, NUMBER_OF_CHAR);
```



```

 storyPanel.add(story, BorderLayout.CENTER);
 JLabel storyLabel = new JLabel("Enter your story here:");
 storyPanel.add(storyLabel, BorderLayout.NORTH);

 add(storyPanel);

 JPanel buttonPanel = new JPanel();
 buttonPanel.setLayout(new FlowLayout());
 buttonPanel.setBackground(Color.PINK);
 JButton actionButton = new JButton("Click me");
 actionButton.addActionListener(this);
 buttonPanel.add(actionButton);

 JButton clearButton = new JButton("Clear");
 clearButton.addActionListener(this);
 buttonPanel.add(clearButton);

 add(buttonPanel);
 }

 public void actionPerformed(ActionEvent e)
 {
 String actionCommand = e.getActionCommand();

 if (actionCommand.equals("Click me"))
 {
 int lineCount = getLineCount();
 story.setText("Your story is "
 + lineCount + " lines long.");
 }
 else if (actionCommand.equals("Clear"))
 story.setText("");
 else
 story.setText("Unexpected error.");
 }

 private int getLineCount()
 {
 String storyString = story.getText();
 int count = 0;

 for (int i = 0; i < storyString.length(); i++)
 if (storyString.charAt(i) == '\n')
 count++;

 return count + 1; //The last line has no '\n'.
 }
}

```

39. We made the text field an instance variable because we needed to refer to it in the definition of the method `actionPerformed`. On the other hand, the only direct reference we had to the buttons was in the constructor. So, we need names for the buttons only in the constructor definition.
40. The GUI would try to add the string `"Enter numbers here."` as if it were a string for a number. This will cause a `NumberFormatException` to be thrown and the string `"Error: Reenter Number."` would be displayed in the text field.
41. Every time the user clicks the addition button, the following assignment is executed:

```
result = result + stringToDouble(ioField.getText());
```

So, the number in the text field is added to the total as many times as the user clicks the addition button. But, the value in the text field is the running total, so the running total is added to itself. Thus, the running total is added to the total as many times as the user clicks the addition button.

Let us say that the user starts the GUI, types in 10, and clicks the addition button. That adds 10 to `result`, so the value of `result` is then 0 plus 10, which is 10, and 10 is displayed. Now the user clicks the addition button a second time. That adds 10 to `result` again, so the value of `result` is 10 plus 10, which is 20, and 20 is displayed. Next the user clicks the addition button a third time. This time, 20 is in the text field, and so it is added to `result`, which is also 20. Thus, the value of `result` is now 40, and 40 is displayed. Note that it is always the number in the text field that is added in.

42. The two calculator windows are completely independent. Each has its own instance variable `result`, which has no effect on the other's instance variable `result`.
43. If you click the close-window button in either calculator window, the entire program ends because that causes an invocation of `System.exit`. There is no invocation of `System.exit` in Display 17.19, but the following ensures that a `System.exit` that is in some library class will be invoked:

```
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

## Programming Projects



VideoNote  
Solution to  
Programming  
Project 17.1

1. Design and code a Swing GUI to translate text that is input in English into Pig Latin. You can assume that the sentence contains no punctuation. The rules for Pig Latin are as follows:
  - a. For words that begin with consonants, move the leading consonant to the end of the word and add "ay." Thus, "ball" becomes "allbay"; "button" becomes "uttonbay"; and so forth.
  - b. For words that begin with vowels, add "way" to the end of the word. Thus, "all" becomes "allway"; "one" becomes "oneway"; and so forth.

Use a `FlowLayout` with a `JTextArea` for the source text and a separate `JTextArea` for the translated text. Add a `JButton` with an event to perform the translation. A sample application is shown next with the text translated to Pig Latin.

To parse the source text, note that you can use the `Scanner` class on a `String`. For example the following code:

```
Scanner scan = new Scanner("foo bar zot");
while (scan.hasNext())
{
 System.out.println(scan.next());
}
```

will output

```
foo
bar
zot
```

2. Develop a simple number game using `JFrame` that will display a rectangular grid of nine buttons displayed in a  $3 \times 3$  game board. Nine random numbers will be generated, one corresponding to each button. The numbers can be anything in the range of 1 to 50. The buttons will be initially blank and the player will be asked to select any three buttons. As soon as the player selects a button, the corresponding number should be displayed at the bottom of the applet, finally displaying all three numbers selected by the player. If one of the numbers selected is odd, the player gets a prize of \$20; if two are odd, the player gets a prize of \$50; and if all three numbers are odd, the player gets a prize of \$100. Also, if any of the three selected numbers is 50, then there will be a bumper prize of \$1000. If any even number is selected then the system should output "Better Luck Next Time!"
3. Design and code a Swing GUI calculator. You can use Display 17.19 as a starting point, but your calculator will be more sophisticated. Your calculator will have two text fields that the user cannot change: One labeled "Result" will contain the result of performing the operation, and the other labeled "Operand" will be for the user to enter a number to be added, subtracted, and so forth from the result. The user enters the number for the "Operand" text field by clicking buttons labeled with the digits 0 through 9 and a decimal point, just as in a real calculator. Allow the operations of addition, subtraction, multiplication, and division. Use a `GridLayout` manager to produce a button pad that looks similar to the keyboard on a real calculator.

When the user clicks a button for an operation, the following occurs: the operation is performed, the "Result" text field is updated, and the "Operand" text field is cleared. Include a button labeled "Reset" that resets the "Result" to 0.0. Also include a button labeled "Clear" that resets the "Operand" text field so it is blank.

*Hint:* Define an exception class named `DivisonByZeroException`. Have your code throw and catch a `DivisonByZeroException` if the user attempts to "divide by zero." Your code will catch the `DivisonByZeroException` and output a suitable message to the "Operand" text field. The user may then enter a new substitute

number in the "Operand" text field. Because values of type `double` are, in effect, approximate values, it makes no sense to test for equality with `0.0`. Consider an operand to be "equal to zero" if it is in the range  $-1.0e-10$  to  $+1.0e-10$ .

4. Create a class named as `Driving`. The constructor of this class will contain two buttons and two labels. The buttons will be displayed with the caption 'Yes' and 'No'. The first label object will contain the text 'Are you an excellent driver?' If the user clicks 'Yes', the first label should change to 'We need you!' If the user clicks 'No', the first label should change to 'You need to practice!' The second label shows how many users respond with 'Yes' and how many users respond with 'No'.
5. Redo or do for the first time the trivia game described in Programming Projects 6.12 and 6.13, except create a GUI for the game interface. Use a layout of your choice with the appropriate text fields, labels, and buttons to implement your design. The game should ask only one question at a time and output the correct answer if the player answers a question incorrectly. When all questions have been answered, show the final score and exit the program.
6. Create a simple text editor by using the component `JTextArea` that allows the user to enter text. The program should have two buttons. The first button should save the content of editor to a file, while the other should be to clear the contents of the editor.

This page intentionally left blank



# Swing II 18

## 18.1 WINDOW LISTENERS 1058

Example: A Window Listener Inner Class 1060

The `dispose` Method 1063

The `WindowAdapter` Class 1064

## 18.2 ICONS AND SCROLL BARS 1066

Icons 1066

Scroll Bars 1072

Example: Components with Changing Visibility 1077

## 18.3 THE `Graphics` CLASS 1081

Coordinate System for Graphics Objects 1081

The Method `paint` and the Class `Graphics` 1082

Drawing Ovals 1087

Drawing Arcs 1087

Rounded Rectangles ★ 1091

`paintComponent` for Panels 1092

Action Drawings and `repaint` 1092

Some More Details on Updating a GUI ★ 1098

## 18.4 COLORS 1098

Specifying a Drawing Color 1099

Defining Colors 1100

The `JColorChooser` Dialog Window 1102

## 18.5 FONTS AND THE `drawString` METHOD 1105

The `drawString` Method 1105

Fonts 1108

# 18 Swing II

*Window listeners?*

*I thought windows were for looking not for listening.*

Student answer on an exam

## Introduction

This chapter is a continuation of Chapter 17, presenting more details about designing regular Swing GUIs. Chapter 20 on applets is a side issue that may be read after this chapter if you prefer.

## Prerequisites

This chapter uses material from Chapter 17 (and its prerequisites).

Section 18.2 on icons and scroll bars is not used in subsequent sections and so may be skipped or postponed.

## 18.1 Window Listeners

*A man may see how this world goes with no eyes.*

*Look with thine ears... .*

WILLIAM SHAKESPEARE, *King Lear*, 1607.

In Chapter 17, we used the method `setDefaultCloseOperation` to program the close-window button in a `JFrame`. This allows for only a limited number of possibilities for what happens when the close-window button is clicked. When the user clicks the close-window button (or either of the two accompanying buttons), the `JFrame` fires an event known as a **window event**. A `JFrame` can use the method `setWindowListener` to register a **window listener** to respond to such window events. A window listener can be programmed to respond to a window event in any way you wish. Window events are objects of the class `WindowEvent`. A window listener is any class that satisfies the `WindowListener` interface.

The method headings in the `WindowListener` interface are given in Display 18.1. If a class implements the `WindowListener` interface, it must have definitions for all seven of these method headings. If you do not need all of these methods, then you can define the ones you do not need to have empty bodies, like this:

```
public void windowDeiconified(WindowEvent e)
{ }
```

window event

window  
listener

WindowEvent

Window  
Listener

---

**Display 18.1   Methods in the `WindowListener` Interface**

---

The `WindowListener` interface and the `WindowEvent` class are in the package `java.awt.event`.

```
public void windowOpened(WindowEvent e)
```

Invoked when a window is opened.

```
public void windowClosing(WindowEvent e)
```

Invoked when a window is in the process of being closed. Clicking the close-window button causes an invocation of this method.

```
public void windowClosed(WindowEvent e)
```

Invoked when a window has been closed.

```
public void windowIconified(WindowEvent e)
```

Invoked when a window is iconified. When you click the minimize button in a `JFrame`, it is iconified.

```
public void windowDeiconified(WindowEvent e)
```

Invoked when a window is deiconified. When you activate a minimized window, it is deiconified.

```
public void windowActivated(WindowEvent e)
```

Invoked when a window is activated. When you click in a window, it becomes the activated window. Other actions can also activate a window.

```
public void windowDeactivated(WindowEvent e)
```

Invoked when a window is deactivated. When a window is activated, all other windows are deactivated. Other actions can also deactivate a window.

---

### **The `WindowListener` Interface**

When the user clicks any of the three standard `JFrame` buttons (for closing the window, minimizing the window, and resizing the window), it generates a window event. Window events are sent to window listeners. In order to be a window listener, a class must implement the `WindowListener` interface. The method headings for the `WindowListener` interface are given in Display 18.1.



**EXAMPLE: A Window Listener Inner Class**

Display 18.2 gives an example of a `JFrame` class with a window listener class that is an inner class. The window listener inner class is named `CheckOnExit`. A window listener class need not be an inner class, but it is frequently convenient to make a window listener class (or other kind of listener class) an inner class.

The main `JFrame` in Display 18.2 simply displays a message. What is interesting is how the window listener programs the close-window button. You can apply the window listener used in this `JFrame` to any `JFrame`. When the close-window button is clicked, a second, smaller window appears and asks "Are you sure you want to exit?" If the user clicks the "Yes" button, the entire program ends, and so both windows go away. If the user clicks the "No" button, only the smaller window disappears; the program and the main window continue. Let us look at the programming details.

When the close-window button in the main window is clicked, this fires a window event. The only registered window listener is the anonymous object that is the argument to `addWindowListener`. Next we repeat the relevant line of code, which is in the constructor for `WindowListenerDemo`:

```
addWindowListener(new CheckOnExit());
```

This anonymous window listener object receives the window event fired when the close-window button is clicked and then invokes the method `windowClosing`. The method `windowClosing` creates and displays a window object of the class `ConfirmWindow`, which contains the message "Are you sure you want to exit?" as well as the two buttons labeled "Yes" and "No".

If the user clicks the "Yes" button, the action event fired by that button goes to the `actionPerformed` method, which ends the program with a call to `System.exit`. If the user clicks the "No" button, then the `actionPerformed` method invokes the method `dispose`. The method `dispose`, discussed in the next subsection, makes its calling object go away but does not end the program. The calling object for `dispose` is the smaller window (which is an object of the class `ConfirmWindow`), so this smaller window goes away but the main window remains.

Notice that even though we have registered a window listener, which says what should happen when the close-window button is clicked, we still need to invoke the method `setDefaultCloseOperation`. When the close-window button is clicked, the policy set by `setDefaultCloseOperation` is always carried out in addition to any action by window listeners. If we do not include any invocation of `setDefaultCloseOperation`, then the default action is to make the window go away (but not to end the program). We do not want our main window to go away, so we set the policy as follows:

```
setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
```

This means that clicking the close-window button causes no action other than the actions of any window listeners. If you are using a window listener to set the action of the close-window button, you invariably want an invocation of `setDefaultCloseOperation` with the argument `JFrame.DO_NOTHING_ON_CLOSE`.

## Display 18.2 A Window Listener (part 1 of 3)

```

1 import javax.swing.JFrame;
2 import javax.swing.JPanel;
3 import java.awt.BorderLayout;
4 import java.awt.FlowLayout;
5 import java.awt.Color;
6 import javax.swing.JLabel;
7 import javax.swing.JButton;
8 import java.awt.event.ActionListener;
9 import java.awt.event.ActionEvent;
10 import java.awt.event.WindowListener;
11 import java.awt.event.WindowEvent;

12 public class WindowListenerDemo extends JFrame
13 {
14 public static final int WIDTH = 300; //for main window
15 public static final int HEIGHT = 200; //for main window
16 public static final int SMALL_WIDTH = 200; //for confirm window
17 public static final int SMALL_HEIGHT = 100; //for confirm window

18 private class CheckOnExit implements WindowListener
19 {
20 public void windowOpened(WindowEvent e)
21 {}

22 public void windowClosing(WindowEvent e)
23 {
24 ConfirmWindow checkers = new ConfirmWindow();
25 checkers.setVisible(true);
26 }

27 public void windowClosed(WindowEvent e)
28 {}

29 public void windowIconified(WindowEvent e)
30 {}

31 public void windowDeiconified(WindowEvent e)
32 {}

33 public void windowActivated(WindowEvent e)
34 {}

35 public void windowDeactivated(WindowEvent e)
36 {}
37 } //End of inner class CheckOnExit

38 private class ConfirmWindow extends JFrame implements
39 ActionListener
40 {
41 public ConfirmWindow()
42 {
43 setSize(SMALL_WIDTH, SMALL_HEIGHT);
44 getContentPane().setBackground(Color.YELLOW);

```

*This WindowListener class is an inner class.*

*A window listener must define all the method headings in the WindowListener interface, even if some are trivial implementations.*

*Another inner class.*

(continued)

## Display 18.2 A Window Listener (part 2 of 3)

```

44 setLayout(new BorderLayout());
45 JLabel confirmLabel = new JLabel(
46 "Are you sure you want to exit?");
47 add(confirmLabel, BorderLayout.CENTER);

48 JPanel buttonPanel = new JPanel();
49 buttonPanel.setBackground(Color.ORANGE);
50 buttonPanel.setLayout(new FlowLayout());

51 JButton exitButton = new JButton("Yes");
52 exitButton.addActionListener(this);
53 buttonPanel.add(exitButton);

54 JButton cancelButton = new JButton("No");
55 cancelButton.addActionListener(this);
56 buttonPanel.add(cancelButton);

57 add(buttonPanel, BorderLayout.SOUTH);
58 }

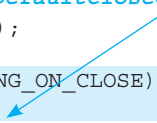
59 public void actionPerformed(ActionEvent e)
60 {
61 String actionCommand = e.getActionCommand();

62 if (actionCommand.equals("Yes"))
63 System.exit(0);
64 else if (actionCommand.equals("No"))
65 dispose(); //Destroys only the ConfirmWindow.
66 else
67 System.out.println(
68 "Unexpected Error in Confirm Window.");
69 } //End of inner class ConfirmWindow
70
71 public static void main(String[] args)
72 {
73 WindowListenerDemo demoWindow = new WindowListenerDemo();
74 demoWindow.setVisible(true);
75 }

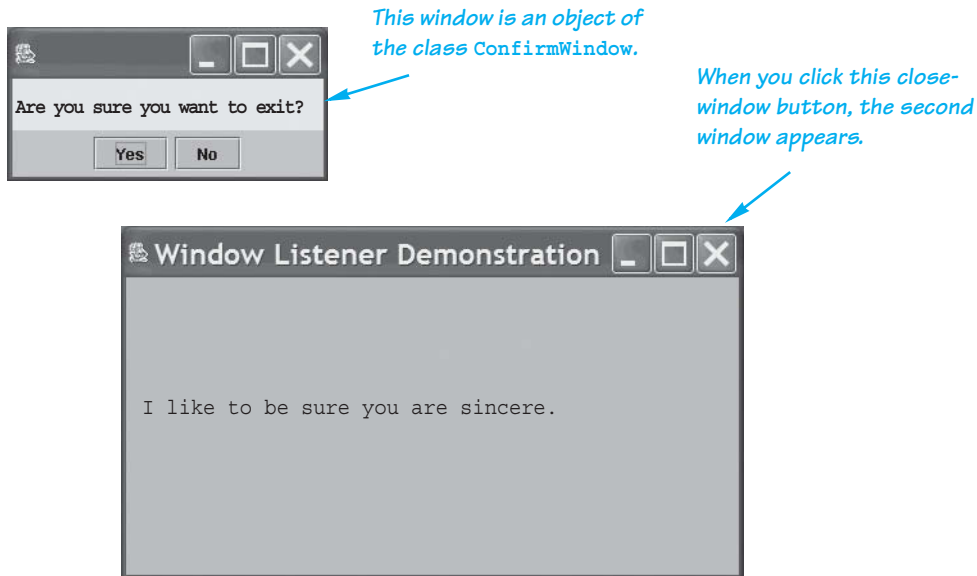
76
77 public WindowListenerDemo()
78 {
79 setSize(WIDTH, HEIGHT);
80 setTitle("Window Listener Demonstration");
81
82 setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
83 addWindowListener(new CheckOnExit());
84
85 getContentPane().setBackground(Color.LIGHT_GRAY);
86 JLabel aLabel = new JLabel(
87 "I like to be sure you are sincere.");
88 add(aLabel);
89 }

```

Even if you have a window listener, you normally must still invoke `setDefaultCloseOperation`.



Display 18.2 A Window Listener (part 3 of 3) (Source: Oracle Corporation)

**RESULTING GUI**

## The dispose Method

**dispose**

The method `dispose` is a method in the class `JFrame` that releases any resources used by the `JFrame` or any of its components. So, a call to `dispose` eliminates the `JFrame` and its components, but if the program has items that are not components of the `JFrame`, then the program does not end. For example, in Display 18.2, the smaller window of the class `ConfirmWindow` invokes `dispose` (if the user clicks the "No" button). That causes that smaller window to go away, but the larger window remains.

### The dispose Method

The class `JFrame` has a method named `dispose` that will eliminate the invoking `JFrame` without ending the program. When `dispose` is invoked, the resources consumed by the `JFrame` and its components are returned for reuse, so the `JFrame` is gone, but the program does not end (unless `dispose` eliminates all elements in the program, as in a one-window program). The method `dispose` is often used in a program with multiple windows to eliminate one window without ending the program.

**SYNTAX**

```
JFrame_Object.dispose();
```

The `JFrame_Object` is often an implicit `this`. A complete example of using `dispose` can be seen in Display 18.2.

**PITFALL: Forgetting to Invoke `setDefaultCloseOperation`**

If you register a window listener to respond to window events from a `JFrame`, you should also include an invocation of the method `setDefaultCloseOperation`, typically in the `JFrame` constructor. This is because the default or other behavior set by `setDefaultCloseOperation` takes place even if there is a window listener. If you do not want any actions other than those provided by the window listener(s), you should include the following in the `JFrame` constructor:

```
setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
```

If you do not include any invocation of `setDefaultCloseOperation`, the default action is the same as if you had included the invocation

```
setDefaultCloseOperation(JFrame.HIDE_ON_CLOSE);
```

which hides the `JFrame` when the close-window button is clicked. (The actions of any registered window listener are also performed.) ■

**The `WindowAdapter` Class**

In Display 18.2, we gave empty bodies to most of the method headings in the `WindowListener` interface. The abstract class `WindowAdapter` is a way to avoid all those empty method bodies. The class `WindowAdapter` does little more than provide trivial implementations of the method headings in the `WindowListener` interface. So, if you make a window listener a derived class of the class `WindowAdapter`, then you have only to define the method headings in the `WindowListener` interface that you need. The other method headings inherit trivial implementations from `WindowAdapter`. (`WindowAdapter` is unusual in that it is an abstract class with no abstract methods.)

For example, in Display 18.3 we have rewritten the inner class `CheckOnExit` from Display 18.2, but this time we made it a derived class of the `WindowAdapter` class. This definition of `CheckOnExit` is much shorter and cleaner than the one in Display 18.2, but the two implementations of `CheckOnExit` are equivalent. Thus, you can replace the definition of `CheckOnExit` in Display 18.2 with the shorter one in Display 18.3. The file `WindowListenerDemo2` on the accompanying website contains a version of Display 18.2 with this shorter definition of `CheckOnExit`.

The class `WindowAdapter` is in the `java.awt.event` package and so requires an `import` statement such as the following:

```
import java.awt.event.WindowAdapter;
```

You cannot always define your window listeners as derived classes of `WindowAdapter`. For example, suppose you want a `JFrame` class to be its own window listener. To accomplish this, the class must be a derived class of `JFrame` and so cannot be a derived

extra code  
on website

class of any other class such as `WindowAdapter`. In such cases, you make the class a derived class of `JFrame` and have it implement the `WindowListener` interface. See Self-Test Exercise 4 for an example.

### Display 18.3 Using `WindowAdapter`

---

*This requires the following import*

```
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;

1 private class CheckOnExit extends WindowAdapter
2 {
3 public void windowClosing(WindowEvent e)
4 {
5 ConfirmWindow checkers = new ConfirmWindow();
6 checkers.setVisible(true);
7 }
8 } //End of inner class CheckOnExit
```

*If the definition of the inner class `CheckOnExit` in Display 18.2 were replaced with this definition of `CheckOnExit`, there would be no difference in how the outer class or any class behaves.*

---

### Self-Test Exercises

1. When you define a class and make it implement the `WindowListener` interface, what methods must you define? What do you do if there is no particular action that you want one of these methods to take?
2. The GUI in Display 18.2 has a main window. When the user clicks the close-window button in the main window, a smaller window appears that says "Are you sure you want to exit?" What happens if the user clicks the close-window button in this smaller window? Explain your answer.
3. If you want a Swing program to end completely, you can invoke the method `System.exit`. What if you want a `JFrame` window to go away, but you do not want the program to end? What method can you have the `JFrame` invoke?
4. Rewrite the class in Display 18.2 so that the class is its own window listener.  
*Hint:* The constructor will contain

```
addWindowListener(this);
```

## 18.2 Icons and Scroll Bars

I ♥ ICONS.

ANONYMOUS

### Icons

**icon**

JLabels, JButtons, and JMenuItems can have icons. An **icon** is simply a small picture, although it is not required to be small. The label, button, or menu item may have just a string displayed on it, just an icon, or both (or nothing at all). An icon is an instance of the ImageIcon class and is based on a digital picture file. The picture file can be in almost any standard format, such as .gif, .jpg, or .tiff.

**ImageIcon**

The class ImageIcon is used to convert a picture file to a Swing icon. For example, if you have a picture in a file named duke\_waving.gif, the following will produce an icon named dukeWavingIcon for the picture duke\_waving.gif:

```
ImageIcon dukeIcon = new ImageIcon("duke_waving.gif");
```

The file duke\_waving.gif should be in the same directory as the class in which this code appears. Alternatively, you can use a complete or relative pathname to specify the picture file. Note that the picture file is given as a value of type String that names the picture file. The file duke\_waving.gif and other picture files we will use in this chapter are all provided on the website that accompanies this text.

**setIcon**

You can add an icon to a label with the method setIcon, as follows:

```
JLabel dukeLabel = new JLabel("Mood check");
dukeLabel.setIcon(dukeIcon);
```

Alternatively, you give the icon as an argument to the JLabel constructor, as follows:

```
JLabel dukeLabel = new JLabel(dukeIcon);
```

You can leave the label as created and it will have an icon but no text, or you can add text with the method setText, as follows:

**setText**

```
dukeLabel.setText("Mood check");
```

Icons and text may be added to JButtons and JMenuItems in the same way as they are added to a JLabel. For example, the following is taken from Display 18.4, which is a demonstration of the use of icons:

```
JButton happyButton = new JButton("Happy");
ImageIcon happyIcon = new ImageIcon("smiley.gif");
happyButton.setIcon(happyIcon);
```

## Display 18.4 Using Icons (part 1 of 2)

```
1 import javax.swing.JFrame;
2 import javax.swing.JPanel;
3 import javax.swing.JTextField;
4 import javax.swing.ImageIcon;
5 import java.awt.BorderLayout;
6 import java.awt.FlowLayout;
7 import java.awt.Color;
8 import javax.swing.JLabel;
9 import javax.swing.JButton;
10 import java.awt.event.ActionListener;
11 import java.awt.event.ActionEvent;

12 public class IconDemo extends JFrame implements ActionListener
13 {
14 public static final int WIDTH = 500;
15 public static final int HEIGHT = 200;
16 public static final int TEXT_FIELD_SIZE = 30;

17 private JTextField message;
18 public static void main(String[] args)
19 {
20 IconDemo iconGui = new IconDemo();
21 iconGui.setVisible(true);
22 }

23 public IconDemo()
24 {
25 super("Icon Demonstration");
26 setSize(WIDTH, HEIGHT);
27 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

28 setBackground(Color.WHITE);
29 setLayout(new BorderLayout());

30 JLabel dukeLabel = new JLabel("Mood check");
31 ImageIcon dukeIcon = new ImageIcon("duke_waving.gif");
32 dukeLabel.setIcon(dukeIcon);
33 add(dukeLabel, BorderLayout.NORTH);

34 JPanel buttonPanel = new JPanel();
35 buttonPanel.setLayout(new FlowLayout());
36 JButton happyButton = new JButton("Happy");
37 ImageIcon happyIcon = new ImageIcon("smiley.gif");
38 happyButton.setIcon(happyIcon);
39 happyButton.addActionListener(this);
40 buttonPanel.add(happyButton);

41 JButton sadButton = new JButton("Sad");
42 ImageIcon sadIcon = new ImageIcon("sad.gif");
43 sadButton.setIcon(sadIcon);
44 sadButton.addActionListener(this);
```

(continued)



Display 18.4 Using Icons (part 2 of 2) (Source: Oracle Corporation)

```

45 buttonPanel.add(sadButton);
46 add(buttonPanel, BorderLayout.SOUTH);

47 message = new JTextField(TEXT_FIELD_SIZE);
48 add(message, BorderLayout.CENTER);
49 }

50 public void actionPerformed(ActionEvent e)
51 {
52 String actionCommand = e.getActionCommand();
53 if (actionCommand.equals("Happy"))
54 message.setText(
55 "Smile and the world smiles with you!");
56 else if (actionCommand.equals("Sad"))
57 message.setText(
58 "Cheer up. It can't be that bad.");
59 else
60 message.setText("Unexpected Error.");
61 }
62 }

```

**RESULTING GUI<sup>1</sup>***View after clicking the "Sad" button.***button with  
only an icon**

You can produce a button or menu item with (just) an icon on it by giving the `ImageIcon` object as an argument to the `JButton` or `JMenuItem` constructor. For example,

```

ImageIcon happyIcon = new ImageIcon("smiley.gif");
JButton smileButton = new JButton(happyIcon);
JMenuItem happyChoice = new JMenuItem(happyIcon);

```

If you create a button or menu item in this way and do not add text with the method `setText`, you should use `setActionCommand` to explicitly give the button or menu item an action command, because there is no string on the button or menu item.

<sup>1</sup>Java, Duke, and all Java-based trademarks and logos are trademarks or registered trademarks of Oracle, Inc. in the United States and other countries.

All of the classes `JButton`, `JMenuItem`, and `JLabel` have constructors that let you specify text and an icon to appear on the button, menu item, or label. The constructor can specify no text or icon, text only, an icon only, or both text and an icon. When you specify both text and an icon, the text is the first argument and the icon is the second argument; also, the constructor for a `JLabel` requires a third argument, as described in Display 18.5. If you omit either text or an icon (or both) from the constructor, you can add them later with the methods `setText` and `setIcon`. Some of these methods for the classes `JButton`, `JMenuItem`, and `JLabel` are given in Display 18.5.

### Icons and the Class `ImageIcon`

An **icon** is simply a small picture, although it is not really required to be small. The class `ImageIcon` is used to convert a picture file to a Swing icon.

#### SYNTAX

```
ImageIcon Name_Of_ImageIcon =
 new ImageIcon(Picture_File_Name);
```

The *Picture\_File\_Name* is a string giving either a relative or absolute pathname to the picture file. (So if the picture file is in the same directory as your program, you need give only the name of the picture file.)

#### EXAMPLE

```
ImageIcon happyIcon =
 new ImageIcon("smiley.gif");
```

Display 18.5 Some Methods in the Classes `JButton`, `JMenuItem`, and `JLabel` (part 1 of 2)

```
public JButton()
public JMenuItem()
public JLabel()
```

Creates a button, menu item, or label with no text or icon on it. (Typically, you will later use `setText` and/or `setIcon` with the button, menu item, or label.)

```
public JButton(String text)
public JMenuItem(String text)
public JLabel(String text)
```

Creates a button, menu item, or label with the text on it.

```
public JButton(ImageIcon picture)
public JMenuItem(ImageIcon picture)
public JLabel(ImageIcon picture)
```

Creates a button, menu item, or label with the icon `picture` on it and no text.

(continued)

Display 18.5 Some Methods in the Classes `JButton`, `JMenuItem`, and `JLabel` (part 2 of 2)

---

```
public JButton(String text, ImageIcon picture)
public JMenuItem(String text, ImageIcon picture)
public JLabel(
 String text, ImageIcon picture, int horizontalAlignment)
```

Creates a button, menu item, or label with both the text and the icon picture on it. `horizontalAlignment` is one of the constants `SwingConstants.LEFT`, `SwingConstants.CENTER`, `SwingConstants.RIGHT`, `SwingConstants.LEADING`, or `SwingConstants.TRAILING`.

The interface `SwingConstants` is in the `javax.swing` package.

```
public void setText(String text)
```

Makes `text` the only text on the button, menu item, or label.

```
public void setIcon(ImageIcon picture)
```

Makes `picture` the only icon on the button, menu item, or label.

```
public void setMargin(Insets margin)
```

`JButton` and `JMenuItem` have the method `setMargin`, but `JLabel` does not.

The method `setMargin` sets the size of the margin around the text and icon in the button or menu item. The following special case will work for most simple situations. The `int` values give the number of pixels from the edge of the button or menu item to the text and/or icon.

```
public void setMargin(new Insets(
 int top, int left, int bottom, int right))
```

The class `Insets` is in the `java.awt` package. (We will not be discussing any other uses for the class `Insets`.)

```
public void setVerticalTextPosition(int textPosition)
```

Sets the vertical position of the text relative to the icon. The `textPosition` should be one of the constants `SwingConstants.TOP`, `SwingConstants.CENTER` (the default position), or `SwingConstants.BOTTOM`.

The interface `SwingConstants` is in the `javax.swing` package.

```
public void setHorizontalTextPosition(int textPosition)
```

Sets the horizontal position of the text relative to the icon. The `textPosition` should be one of the constants `SwingConstants.RIGHT`, `SwingConstants.LEFT`, `SwingConstants.CENTER`, `SwingConstants.LEADING`, or `SwingConstants.TRAILING`.

The interface `SwingConstants` is in the `javax.swing` package.

---

## The Insets Class

Objects of the class `Insets` are used to specify the size of the margin in a button or menu item. The `Insets` class is in the package `java.awt`. The parameters in the following constructors are in pixels.

### CONSTRUCTOR

```
public Insets(int top, int left, int bottom, int right)
```

### EXAMPLES

```
aButton.setMargin(new Insets(10, 20, 10, 20));
```

## setIcon and setText

The method `setIcon` can be used to add an icon to a `JButton`, `JMenuItem`, or `JLabel`. The argument to `setIcon` must be an `ImageIcon` object.

### SYNTAX

```
Component.setIcon (ImageIcon_Object) ;
```

The *Component* can be a `JButton`, `JMenuItem`, or `JLabel`.

### EXAMPLE

```
JLabel helloLabel = new JLabel("Hello");
ImageIcon dukeIcon = new ImageIcon("duke_waving.gif");
helloLabel.setIcon(dukeIcon);
```

The method `setText` can be used to add text to a `JButton`, `JMenuItem`, or `JLabel`.

### SYNTAX

```
Component.setText (Text_String) ;
```

The *Component* can be a `JButton`, `JMenuItem`, or `JLabel`.

### EXAMPLE

```
ImageIcon dukeIcon = new ImageIcon("duke_waving.gif");
JLabel helloLabel = new JLabel(dukeIcon);
helloLabel.setText("Hello");
```

The two examples are equivalent.

## Self-Test Exercises

5. Write code to create a button that has on it both the text "Magic Button" and the picture in the file `wizard.gif`.
6. Write code to add the picture in the file `wizard.gif` to the `JPanel` named `picturePanel`. Assume that `picturePanel` has a `FlowLayout` manager.
7. Suppose you want to create a button that has the picture in the file `wizard.gif` on it and no text. Suppose further that you want the button to have the action command "Kazam". How would you create the button and set up the action command?

## Scroll Bars

When you create a text area, you specify the number of lines that are visible and the number of characters per line, as in the following example:

```
JTextArea memoDisplay = new JTextArea(15, 30);
```

The text area `memoDisplay` will have room for 15 lines of text, and each line will have room for at least 30 characters. The user can enter more text, but only a limited amount of text will be visible. It would be better not to have a firm limit on the number of lines or the number of characters per line that the user can see in some convenient way. The way to accomplish this is to add scroll bars to the text area, although, as you will see, the Java code looks more like adding the text area to the scroll bars rather than the other way around.

### view port

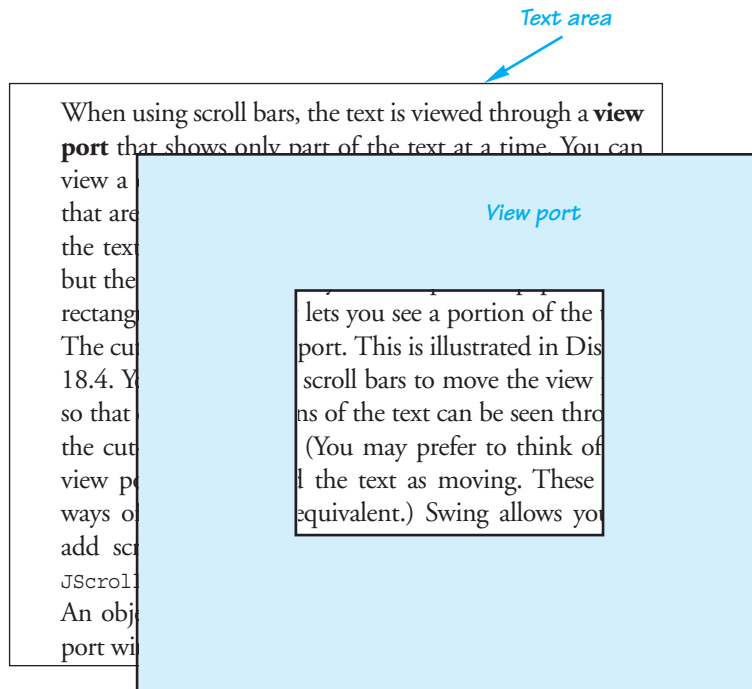
When using scroll bars, the text is viewed through a **view port** that shows only part of the text at a time. You can view a different part of the text by using the scroll bars that are placed along the side and bottom of the view port. It is as if the text were written on an unbounded sheet of paper, but the paper is covered by another piece of paper with a rectangular cutout that lets you see only a portion of the text. The cutout is the view port. This is illustrated in Display 18.6. You use the scroll bars to move the view port so that different portions of the text can be seen through the cutout view port. (You may prefer to think of the view port as fixed and the text as moving. These two ways of thinking are equivalent.) Swing allows you to add scroll bars to your text areas by using the class `JScrollPane`.

### JScrollPane

An object of the class `JScrollPane` is essentially a view port with scroll bars. When you create a `JScrollPane`, you give the text area as an argument to the `JScrollPane` constructor. For example, if `memoDisplay` is an object of the class `JTextArea` (as created in the line of code at the start of this subsection), you can place `memoDisplay` in a `JScrollPane` as follows:

```
JScrollPane scrolledText = new JScrollPane(memoDisplay);
```

Display 18.6 View Port for a Text Area

Display 18.7 Some Methods in the Class `JScrollPane` (part 1 of 2)

The `JScrollPane` class is in the `javax.swing` package.

```
public JScrollPane(Component objectToBeScrolled)
```

Creates a new `JScrollPane` for the `objectToBeScrolled`. Note that the `objectToBeScrolled` need not be a `JTextArea`, although that is the only type of argument considered in this book.

```
public void setHorizontalScrollBarPolicy(int policy)
```

Sets the policy for showing the horizontal scroll bar. The `policy` should be one of the following:

```
JScrollPane.HORIZONTAL_SCROLLBAR_ALWAYS
```

```
JScrollPane.HORIZONTAL_SCROLLBAR_NEVER
```

```
JScrollPane.HORIZONTAL_SCROLLBAR_AS_NEEDED
```

The phrase `AS_NEEDED` means the scroll bar is shown only when it is needed. This is explained more fully in the text. The meanings of the other policy constants are obvious from their names.

(As indicated, these constants are defined in the class `JScrollPane`. You should not need to even be aware of the fact that they have `int` values. Think of them as policies, not as `int` values.)

```
public void setVerticalScrollBarPolicy(int policy)
```

(continued)

Display 18.7 Some Methods in the Class `JScrollPane` (part 2 of 2)

Sets the policy for showing the vertical scroll bar. The `policy` should be one of the following:

```
JScrollPane.VERTICAL_SCROLLBAR_ALWAYS
JScrollPane.VERTICAL_SCROLLBAR_NEVER
JScrollPane.VERTICAL_SCROLLBAR_AS_NEEDED
```

The phrase `AS_NEEDED` means the scroll bar is shown only when it is needed. This is explained more fully in the text. The meanings of the other policy constants are obvious from their names. (As indicated, these constants are defined in the class `JScrollPane`. You should not need to even be aware of the fact that they have `int` values. Think of them as policies, not as `int` values.)

The `JScrollPane` can then be added to a container, such as a `JPanel` or `JFrame`, as follows:

```
textPanel.add(scrolledTextBox);
```

This is illustrated by the program in Display 18.8.

Note the following two lines in the constructor definition in Display 18.8:

```
scrolledTextBox.setHorizontalScrollBarPolicy(
 JScrollPane.HORIZONTAL_SCROLLBAR_ALWAYS);
scrolledTextBox.setVerticalScrollBarPolicy(
 JScrollPane.VERTICAL_SCROLLBAR_ALWAYS);
```

### setting scroll bar policies

Despite the imposing length of these two method invocations, they perform a very simple task. The first merely specifies that the horizontal scroll bar will always be present. The second specifies that the vertical scroll bar will always be present.

If you omit the invocation of the two methods `setHorizontalScrollBarPolicy` and `setVerticalScrollBarPolicy`, the scroll bars will be visible only when you need them. In other words, if you omit these two method invocations and all the text fits in the view port, then no scroll bars will be visible. When you add enough text to need scroll bars, the needed scroll bars will appear automatically.

Display 18.7 summarizes what we have said about the class `JScrollPane`. We are interested in using `JScrollPane` only with text areas. However, as we note in Display 18.7, `JScrollPane` can be used with almost any sort of component.

## Display 18.8 A Text Area with Scroll Bars (part 1 of 3)

```
1 import javax.swing.JFrame;
2 import javax.swing.JTextArea;
3 import javax.swing.JPanel;
4 import javax.swing.JLabel;
5 import javax.swing.JButton;
6 import javax.swing.JScrollPane;
7 import java.awt.BorderLayout;
8 import java.awt.FlowLayout;
9 import java.awt.Color;
10 import java.awt.event.ActionListener;
11 import java.awt.event.ActionEvent;
```

## Display 18.8 A Text Area with Scroll Bars (part 2 of 3)

```
12 public class ScrollBarDemo extends JFrame
13 implements ActionListener
14 {
15 public static final int WIDTH = 600;
16 public static final int HEIGHT = 400;
17 public static final int LINES = 15;
18 public static final int CHAR_PER_LINE = 30;

19 private JTextArea memoDisplay;
20 private String memo1;
21 private String memo2;

22 public static void main(String[] args)
23 {
24 ScrollBarDemo gui = new ScrollBarDemo();
25 gui.setVisible(true);
26 }

27 public ScrollBarDemo()
28 {
29 super("Scroll Bars Demo");
30 setSize(WIDTH, HEIGHT);
31 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
32 JPanel buttonPanel = new JPanel();
33 buttonPanel.setBackground(Color.LIGHT_GRAY);
34 buttonPanel.setLayout(new FlowLayout());
35 JButton memo1Button = new JButton("Save Memo 1");
36 memo1Button.addActionListener(this);
37 buttonPanel.add(memo1Button);

38 JButton memo2Button = new JButton("Save Memo 2");
39 memo2Button.addActionListener(this);
40 buttonPanel.add(memo2Button);

41 JButton clearButton = new JButton("Clear");
42 clearButton.addActionListener(this);
43 buttonPanel.add(clearButton);

44 JButton get1Button = new JButton("Get Memo 1");
45 get1Button.addActionListener(this);
46 buttonPanel.add(get1Button);

47 JButton get2Button = new JButton("Get Memo 2");
48 get2Button.addActionListener(this);
49 buttonPanel.add(get2Button);

50 add(buttonPanel, BorderLayout.SOUTH);

51 JPanel textPanel = new JPanel();
52 textPanel.setBackground(Color.BLUE);
```

(continued)



Display 18.8 A Text Area with Scroll Bars (part 3 of 3) (Source: Oracle Corporation)

```

53 memoDisplay = new JTextArea(LINES, CHAR_PER_LINE);
54 memoDisplay.setBackground(Color.WHITE);

55 JScrollPane scrolledText = new JScrollPane(memoDisplay);
56 scrolledText.setHorizontalScrollBarPolicy(
57 JScrollPane.HORIZONTAL_SCROLLBAR_ALWAYS);
58 scrolledText.setVerticalScrollBarPolicy(
59 JScrollPane.VERTICAL_SCROLLBAR_ALWAYS);

60 textPanel.add(scrolledText);

61 add(textPanel, BorderLayout.CENTER);
62 }
63 public void actionPerformed(ActionEvent e)
64 {
65 String actionCommand = e.getActionCommand();

66 if (actionCommand.equals("Save Memo 1"))
67 memo1 = memoDisplay.getText();
68 else if (actionCommand.equals("Save Memo 2"))
69 memo2 = memoDisplay.getText();
70 else if (actionCommand.equals("Clear"))
71 memoDisplay.setText("");
72 else if (actionCommand.equals("Get Memo 1"))
73 memoDisplay.setText(memo1);
74 else if (actionCommand.equals("Get Memo 2"))
75 memoDisplay.setText(memo2);
76 else
77 memoDisplay.setText("Error in memo interface");
78 }
79 }

```

**RESULTING GUI**

## Scroll Bars

The class `JScrollPane` is used to add scroll bars to a `JTextArea` (and certain other components). The `JTextArea` object is given as an argument to the constructor that creates the `JScrollPane`. The `JScrollPane` class is in the `javax.swing` package.

### SYNTAX

```
JScrollPane Identifier = new JScrollPane(Text_Area_Object);
```

### EXAMPLES

```
JTextArea memoDisplay = new JTextArea(LINES, CHAR_PER_LINE);
JScrollPane scrolledText = new JScrollPane(memoDisplay);
textPanel.add(scrolledText);
```

## Self-Test Exercises

8. When setting up a `JScrollPane`, do you have to invoke both of the methods `setHorizontalScrollBarPolicy` and `setVerticalScrollBarPolicy`?
9. In Display 18.7, we listed the constructor for `JScrollPane` as follows:

```
public JScrollPane(Component objectToBeScrolled)
```

This indicates that the argument to the constructor must be of type `Component`. But we used the constructor with an argument of type `JTextArea`. Isn't this some sort of type violation?

## EXAMPLE: Components with Changing Visibility

The GUI in Display 18.9 has labels that change from visible to invisible and back again. Because the labels contain nothing but an icon each, it appears as if the icons also change roles from visible to invisible and back again. When the GUI is first run, the label with Duke not waving is shown. When the "Wave" button is clicked, the label with Duke not waving disappears and the label with Duke waving appears. When the button labeled "Stop" is clicked, the label with Duke waving disappears and the label with Duke not waving returns. Note that you can make a component invisible without making the entire GUI invisible.

(continued)

**EXAMPLE:** (continued)

In this GUI, a label becomes visible or invisible when a button is clicked. For example, the following code from the method `actionPerformed` in Display 18.9 determines what happens when the button with the text "Wave" on it is clicked:

```
if (actionCommand.equals("Wave"))
{
 wavingLabel.setVisible(true);
 standingLabel.setVisible(false);
}
```

We used the `setVisible` method on the labels containing the icons rather than directly on the icons because the class `ImageIcon` does not have the `setVisible` method.

The two statements

```
wavingLabel.setVisible(true);
standingLabel.setVisible(false);
```

make `wavingLabel` visible and `standingLabel` invisible.

Display 18.9 Labels with Changing Visibility (part 1 of 3)

```
1 import javax.swing.JFrame;
2 import javax.swing.ImageIcon;
3 import javax.swing.JPanel;
4 import javax.swing.JLabel;
5 import javax.swing.JButton;
6 import java.awt.BorderLayout;
7 import java.awt.FlowLayout;
8 import java.awt.Color;
9 import java.awt.event.ActionListener;
10 import java.awt.event.ActionEvent;

11 public class VisibilityDemo extends JFrame
12 implements ActionListener
13 {
14 public static final int WIDTH = 300;
15 public static final int HEIGHT = 200;
16 private JLabel wavingLabel;
17 private JLabel standingLabel;
```

## Display 18.9 Labels with Changing Visibility (part 2 of 3)

```
18 public static void main(String[] args)
19 {
20 VisibilityDemo demoGui = new VisibilityDemo();
21 demoGui.setVisible(true);
22 }

23 public VisibilityDemo()
24 {
25 setSize(WIDTH, HEIGHT);
26 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
27 setTitle("Visibility Demonstration");

28 setLayout(new BorderLayout());

29 JPanel picturePanel = new JPanel();
30 picturePanel.setBackground(Color.WHITE);
31 picturePanel.setLayout(new FlowLayout());

32 ImageIcon dukeStandingIcon =
33 new ImageIcon("duke_standing.gif");
34 standingLabel = new JLabel(dukeStandingIcon);
35 standingLabel.setVisible(true);
36 picturePanel.add(standingLabel);

37 ImageIcon dukeWavingIcon = new ImageIcon("duke_waving.gif");
38 wavingLabel = new JLabel(dukeWavingIcon);
39 wavingLabel.setVisible(false);
40 picturePanel.add(wavingLabel);

41 add(picturePanel, BorderLayout.CENTER);

42 JPanel buttonPanel = new JPanel();
43 buttonPanel.setBackground(Color.LIGHT_GRAY);
44 buttonPanel.setLayout(new FlowLayout());

45 JButton waveButton = new JButton("Wave");
46 waveButton.addActionListener(this);
47 buttonPanel.add(waveButton);

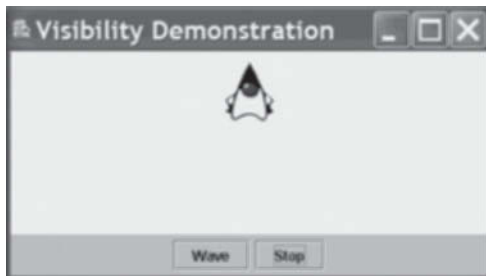
48 JButton stopButton = new JButton("Stop");
49 stopButton.addActionListener(this);
50 buttonPanel.add(stopButton);
51 add(buttonPanel, BorderLayout.SOUTH);
52 }
```

(continued)

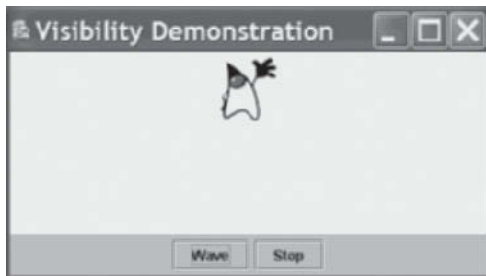
Display 18.9 Labels with Changing Visibility (part 3 of 3) (Source: Oracle Corporation)

```
53 public void actionPerformed(ActionEvent e)
54 {
55 String actionCommand = e.getActionCommand();
56 if (actionCommand.equals("Wave"))
57 {
58 wavingLabel.setVisible(true);
59 standingLabel.setVisible(false);
60 }
61 else if (actionCommand.equals("Stop"))
62 {
63 standingLabel.setVisible(true);
64 wavingLabel.setVisible(false);
65 }
66 else
67 System.out.println("Unanticipated error.");
68 }
69 }
```

**Resulting GUI (After clicking Stop button)**



**Resulting GUI (After clicking Wave button)**



## 18.3 The Graphics Class

*Drawing is my life!*

### THE GRAPHICS CLASS

In this section, we show you how to produce drawings for your GUIs using the `Graphics` class.

### Coordinate System for Graphics Objects

**origin**

**( $x$ ,  $y$ )**

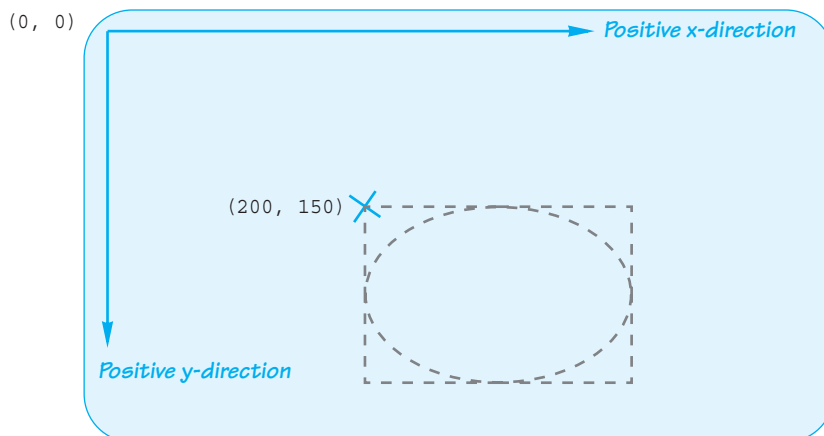
When drawing objects on the screen, Java uses the coordinate system shown in Display 18.10. The **origin** point (0, 0) is the upper-left corner of the screen area used for drawing (usually a `JFrame` or `JPanel`). The  $x$ -coordinate, or horizontal coordinate, is positive and increasing to the right. The  $y$ -coordinate, or vertical coordinate, is positive and increasing in the downward direction. The point ( $x$ ,  $y$ ) is located  $x$  pixels in from the left edge of the screen and down  $y$  pixels from the top of the screen. All coordinates are normally positive. Units as well as sizes of figures are in pixels. When placing a rectangle on the screen, Java often uses a coordinate such as (200, 150) to specify where the rectangle is located.

Note that, when specifying the location of a rectangle or other figure, the coordinates do not indicate the center of the rectangle, but instead indicate the location of the upper-left corner of the rectangle. In Display 18.10, the X marks the location of the point (200, 150) and the rectangle shown is at location (200, 150).

**bounding box**

When placing a figure other than a rectangle on the screen, Java encloses the figure in an imaginary tightly fitting rectangle, sometimes called a **bounding box**, and positions the upper-left corner of the imaginary rectangle. For example, in Display 18.10, the oval displayed is located at point (200, 150).

Display 18.10 Screen Coordinate System



paint



VideoNote

Walkthrough of a  
Simple Drawing  
Program

## The Method `paint` and the Class `Graphics`

Almost all Swing and Swing-related components and containers have a method named `paint`. The method `paint` draws the component or container on the screen. Up until now, we have had no need to redefine this method or to even mention it. It is defined for you and is called automatically when the figure is displayed on the screen. However, to draw geometric figures, such as circles and boxes, you need to redefine the method `paint`. It is this method that draws the figures.

Display 18.11 shows a GUI program that displays a `JFrame` with a rather primitive face drawn inside of it. The mouth and eyes are just straight line segments. We will soon see how to get round eyes and a smile (and more), but the basic technique can be seen more clearly in this simple figure. The code for drawing the face is given in the method `paint`.

Display 18.11 Drawing a Very Simple Face (part 1 of 2)

```
1 import javax.swing.JFrame;
2 import java.awt.Graphics;
3 import java.awt.Color;
4 public class Face extends JFrame

5 {
6 public static final int WINDOW_WIDTH = 400;
7 public static final int WINDOW_HEIGHT = 400;

8 public static final int FACE_DIAMETER = 200;
9 public static final int X_FACE = 100;
10 public static final int Y_FACE = 100;

11 public static final int EYE_WIDTH = 20;
12 public static final int X_RIGHT_EYE = X_FACE + 55;
13 public static final int Y_RIGHT_EYE = Y_FACE + 60;
14 public static final int X_LEFT_EYE = X_FACE + 130;
15 public static final int Y_LEFT_EYE = Y_FACE + 60;

16 public static final int MOUTH_WIDTH = 100;
17 public static final int X_MOUTH = X_FACE + 50;
18 public static final int Y_MOUTH = Y_FACE + 150;

19 public static void main(String[] args)
20 {
21 Face drawing = new Face();
22 drawing.setVisible(true);
23 }
24 public Face()
25 {
```

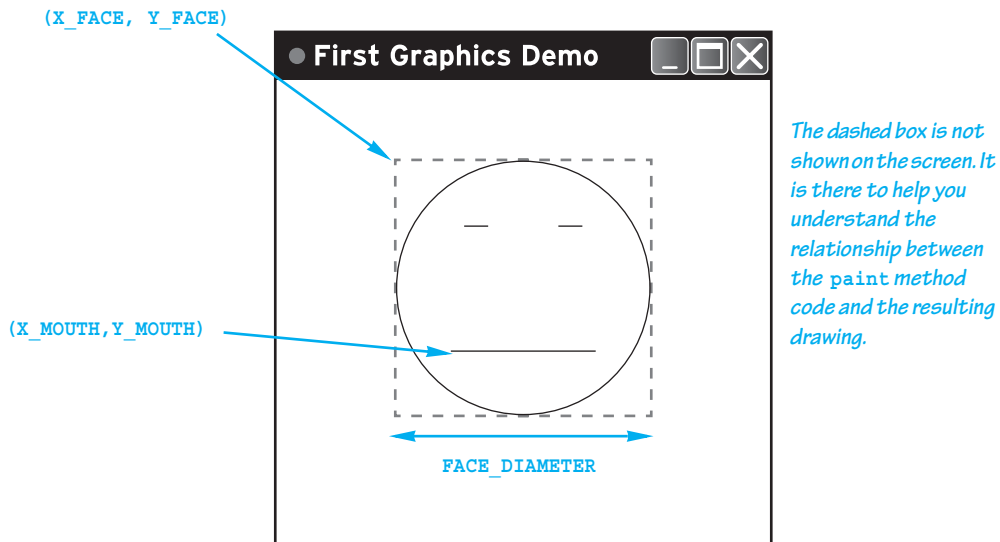
Display 18.11 Drawing a Very Simple Face (part 2 of 2)

```

26 super("First Graphics Demo");
27 setSize(WINDOW_WIDTH, WINDOW_HEIGHT);
28 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
29 getContentPane().setBackground(Color.white);
30 }

31 public void paint(Graphics g)
32 {
33 super.paint(g);
34 g.drawOval(X_FACE, Y_FACE, FACE_DIAMETER, FACE_DIAMETER);
35 //Draw Eyes:
36 g.drawLine(X_RIGHT_EYE, Y_RIGHT_EYE,
37 X_RIGHT_EYE + EYE_WIDTH, Y_RIGHT_EYE);
38 g.drawLine(X_LEFT_EYE, Y_LEFT_EYE,
39 X_LEFT_EYE + EYE_WIDTH, Y_LEFT_EYE);
40 //Draw Mouth:
41 g.drawLine(X_MOUTH, Y_MOUTH, X_MOUTH + MOUTH_WIDTH, Y_MOUTH);
42 }
43 }

```

**Resulting GUI**

The method `paint` is called automatically, and you normally should not invoke it in your code. If you do not redefine it, the method `paint` for a `JFrame` object simply draws a frame border, title, and other standard features, and then asks the components to all invoke their `paint` methods. If we do not redefine the method `paint`, then the



`JFrame` would have a border and title but would contain nothing. The code in the redefinition of `paint` explains how to draw the face. Let us look at the details.

#### Graphics

Notice that the method `paint` has a parameter `g` of type `Graphics`. `Graphics` is an abstract class in the `java.awt` package. Every container and component that can be drawn on the screen has an associated `Graphics` object. (To be precise, every `JComponent` has an associated `Graphics` object.) This associated `Graphics` object has data specifying what area of the screen the component or container covers. In particular, the `Graphics` object for a `JFrame` specifies that the drawing takes place inside the borders of the `JFrame` object. (Because `Graphics` is an abstract class, every `Graphics` object is an instance of some concrete descendent class of the `Graphics` class, but we usually do not care about which descendent class. All we normally need to know is that it is of type `Graphics`.)

The `Graphics` class, and so any `Graphics` object `g`, has all the methods that we will use to draw figures, such as circles, lines, and boxes, on the screen. Almost the entire definition of the `paint` method in Display 18.11 consists of invocations of various drawing methods with the parameter `g` as the calling object.

When the `paint` method in Display 18.11 is (automatically) invoked, the parameter `g` will be replaced by the `Graphics` object associated with the `JFrame`, so the figures drawn will be inside the `JFrame`. Let us look at the code in this method `paint`.

Notice the first line in the definition of `paint` in Display 18.11:

```
super.paint(g);
```

Recall that `super` is a name for the parent class of a derived class. The class in Display 18.11 is derived from the class `JFrame`, so `super.paint` is the `paint` method for the class `JFrame`. Whenever you redefine the method `paint`, you should start with this invocation of `super.paint`. This ensures that your definition of `paint` will do all the things the standard `paint` method does, such as draw the title and border for the `JFrame`. (This lesson applies even if the class is derived from some class other than `JFrame`.)

#### drawOval

The following invocation from the method `paint` draws the circle forming the head:

```
g.drawOval(X_FACE, Y_FACE, FACE_DIAMETER, FACE_DIAMETER);
```

The last two arguments give the width and height of the enclosing rectangle, shown in red. The fact that these two arguments are equal is what makes it a circle instead of a typical oval. The first two arguments give  $x$ - and  $y$ -coordinates for the position of the circle. Note that a figure is positioned by giving the position of the upper-left corner of an enclosing rectangle.

#### drawLine

The only other drawing statements in the method `paint` are invocations of `g.drawLine`. The method `g.drawLine` draws a straight line between two points with  $x$ - and  $y$ -coordinates  $(x_1, y_1)$  and  $(x_2, y_2)$ , where the argument positions for the four coordinate numbers are indicated as follows:

```
g.drawLine(x1, y1, x2, y2)
```

For example, the invocation that draws the mouth is as follows:

```
g.drawLine(X_MOUTH, Y_MOUTH, X_MOUTH + MOUTH_WIDTH, Y_MOUTH);
```

Because both  $y$ -coordinates (`Y_MOUTH`) are the same, the line is horizontal. The line for the mouth begins at coordinates (`X_MOUTH`, `Y_MOUTH`) and extends to the right for `MOUTH_WIDTH` pixels.

### The Graphics Class

Every container and component that can be drawn on the screen has an associated `Graphics` object. This associated `Graphics` object has data specifying what area of the screen the component or container covers. In particular, the `Graphics` object for a `JFrame` specifies that the drawing takes place inside the borders of the `JFrame` object.

When an object `g` of the class `Graphics` is used as the calling object for a drawing method, the drawing takes place inside the area of the screen specified by `g`. For example, if `g` is the `Graphics` object for a `JFrame`, the drawing takes place inside the borders of the `JFrame` object.

Some of the commonly used methods of the class `Graphics` are given in Display 18.12.

`Graphics` is an abstract class in the `java.awt` package.

Some of the commonly used methods of the class `Graphics` are given in Display 18.12. Note that most methods come in pairs, one whose name starts with `draw` and one whose name starts with `fill`, such as `drawOval` and `fillOval`. The one that starts with `draw` will draw the outline of the specified figure. The one that starts with `fill` will draw a solid figure obtained by filling the inside of the specified figure. In the next few subsections, we discuss some of these methods.

#### Display 18.12 Some Methods in the Class `Graphics` (part 1 of 3)

`Graphics` is an abstract class in the `java.awt` package.

Although many of these methods are abstract, we always use them with objects of a concrete descendent class of `Graphics`, even though we usually do not know the name of that concrete class.

```
public abstract void drawLine(int x1, int y1, int x2, int y2)
```

Draws a line between points (`x1`, `y1`) and (`x2`, `y2`).

```
public abstract void drawRect(int x, int y,
 int width, int height)
```

Draws the outline of the specified rectangle. (`x`, `y`) is the location of the upper-left corner of the rectangle.

```
public abstract void fillRect(int x, int y,
 int width, int height)
```

Fills the specified rectangle. (`x`, `y`) is the location of the upper-left corner of the rectangle.

(continued)

Display 18.12 Some Methods in the Class `Graphics` (part 2 of 3)

```
public void draw3DRect(int x, int y, int width,
 int height, boolean raised)
```

Draws the outline of the specified rectangle. (x, y) is the location of the upper-left corner. The rectangle is highlighted to look like it has thickness. If `raised` is `true`, the highlight makes the rectangle appear to stand out from the background. If `raised` is `false`, the highlight makes the rectangle appear to be sunken into the background.

```
public void fill3DRect(int x, int y, int width,
 int height, boolean raised)
```

Fills the rectangle specified by

```
draw3DRec(x, y, width, height, raised)
```

```
public abstract void drawRoundRect (int x, int y,
 int width, int height, int arcWidth, int arcHeight)
```

Draws the outline of the specified round-cornered rectangle. (x, y) is the location of the upper-left corner of the enclosing regular rectangle. `arcWidth` and `arcHeight` specify the shape of the round corners. See the text for details.

```
public abstract void fillRoundRect(int x, int y,
 int width, int height, int arcWidth, int arcHeight)
```

Fills the rounded rectangle specified by

```
drawRoundRec(x, y, width, height, arcWidth, arcHeight)
```

```
public abstract void drawOval(int x, int y,
 int width, int height)
```

Draws the outline of the oval with the smallest enclosing rectangle that has the specified width and height. The (imagined) rectangle has its upper-left corner located at (x, y).

```
public abstract void fillOval (int x, int y,
 int width, int height)
```

Fills the oval specified by

```
drawOval(x, y, width, height)
```

```
public abstract void drawArc(int x, int y,
 int width, int height,
 int startAngle, int arcSweep)
```

Draws part of an oval that just fits into an invisible rectangle described by the first four arguments. The portion of the oval drawn is given by the last two arguments. See the text for details.

Display 18.12 Some Methods in the Class `Graphics` (part 3 of 3)

```
public abstract void fillArc(int x, int y,
 int width, int height,
 int startAngle, int arcSweep)
```

Fills the partial oval specified by

```
drawArc(x, y, width, height, startAngle, arcSweep)
```

## Drawing Ovals

An oval is drawn by the method `drawOval`. The arguments specify the location, width, and height of the smallest rectangle that encloses the oval. For example, the following line draws an oval:

```
drawOval g.drawOval(100, 50, 300, 200);
```

This draws an oval that just fits into an invisible rectangle whose upper-left corner is at coordinates (100, 50) and that has a width of 300 pixels and a height of 200 pixels. Note that the point that is used to place the oval on the screen is not the center of the oval or anything like the center, but is something like the upper-left corner of the oval.

Note that a circle is a special case of an oval in which the width and height of the rectangle are equal. For example, the following line from the definition of `paint` in Display 18.11 draws a circle for the outline of the face:

```
g.drawOval(X_FACE, Y_FACE, FACE_DIAMETER, FACE_DIAMETER);
```

Because the enclosing rectangle has the same width and height, this produces a circle.

Some of the methods you can use to draw simple figures are shown in Display 18.12. A similar table is given in Appendix 5.

## Drawing Arcs

Arcs, such as the smile on the happy face in Display 18.13, are described by giving an oval and then specifying what portion of the oval will be used for the arc. For example, the following statement from Display 18.13 draws the smile on the happy face:

```
drawArc g.drawArc(X_MOUTH, Y_MOUTH, MOUTH_WIDTH, MOUTH_HEIGHT,
 MOUTH_START_ANGLE, MOUTH_ARC_SWEEP);
```

which is equivalent to

```
g.drawArc(X_MOUTH, Y_MOUTH, MOUTH_WIDTH, MOUTH_HEIGHT, 180, 180);
```

The arguments `MOUTH_WIDTH` and `MOUTH_HEIGHT` determine the size of an invisible rectangle. The arguments `X_MOUTH` and `Y_MOUTH` determine the location of the rectangle. The upper-left corner of the rectangle is located at the point `(X_MOUTH, Y_MOUTH)`. Inside this invisible rectangle, envision an invisible oval that just fits inside the invisible rectangle. The last two arguments specify the portion of this invisible oval that is made visible.

Display 18.14 illustrates how these last two arguments specify an arc of the invisible oval to be made visible. The next-to-last argument specifies a start angle in degrees. The last argument specifies how many degrees of the oval's arc will be made visible. If the last argument is 360 (degrees), then the full oval is made visible.

Display 18.13 Drawing a Happy Face (part 1 of 2)

---

```
1 import javax.swing.JFrame;
2 import java.awt.Graphics;
3 import java.awt.Color;

4 public class HappyFace extends JFrame
5 {
6 public static final int WINDOW_WIDTH = 400;
7 public static final int WINDOW_HEIGHT = 400;

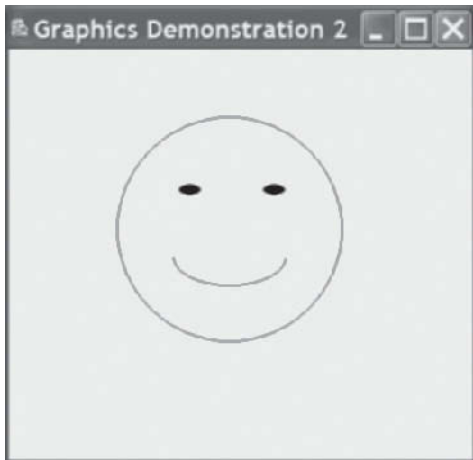
8 public static final int FACE_DIAMETER = 200;
9 public static final int X_FACE = 100;
10 public static final int Y_FACE = 100;

11 public static final int EYE_WIDTH = 20;
12 public static final int EYE_HEIGHT = 10;
13 public static final int X_RIGHT_EYE = X_FACE + 55;
14 public static final int Y_RIGHT_EYE = Y_FACE + 60;
15 public static final int X_LEFT_EYE = X_FACE + 130;
16 public static final int Y_LEFT_EYE = Y_FACE + 60;

17 public static final int MOUTH_WIDTH = 100;
18 public static final int MOUTH_HEIGHT = 50;
19 public static final int X_MOUTH = X_FACE + 50;
20 public static final int Y_MOUTH = Y_FACE + 100;
21 public static final int MOUTH_START_ANGLE = 180;
22 public static final int MOUTH_ARC_SWEEP = 180;
23 public static void main(String[] args)
24 {
25 HappyFace drawing = new HappyFace();
26 drawing.setVisible(true);
27 }
```

Display 18.13 Drawing a Happy Face (part 2 of 2) (Source: Oracle Corporation)

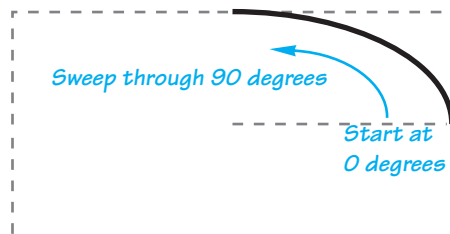
```
28 public HappyFace()
29 {
30 super("Graphics Demonstration 2");
31 setSize(WINDOW_WIDTH, WINDOW_HEIGHT);
32 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
33 getContentPane().setBackground(Color.white);
34 }
35 public void paint(Graphics g)
36 {
37 super.paint(g);
38 g.drawOval(X_FACE, Y_FACE, FACE_DIAMETER, FACE_DIAMETER);
39 //Draw Eyes:
40 g.fillOval(X_RIGHT_EYE, Y_RIGHT_EYE, EYE_WIDTH, EYE_HEIGHT);
41 g.fillOval(X_LEFT_EYE, Y_LEFT_EYE, EYE_WIDTH, EYE_HEIGHT);
42 //Draw Mouth:
43 g.drawArc(X_MOUTH, Y_MOUTH, MOUTH_WIDTH, MOUTH_HEIGHT,
44 MOUTH_START_ANGLE, MOUTH_ARC_SWEEP);
45 }
46 }
```

**Resulting GUI**

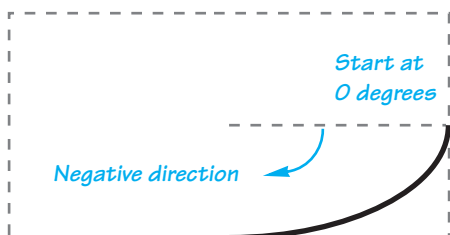
Display 18.14 Specifying an Arc

---

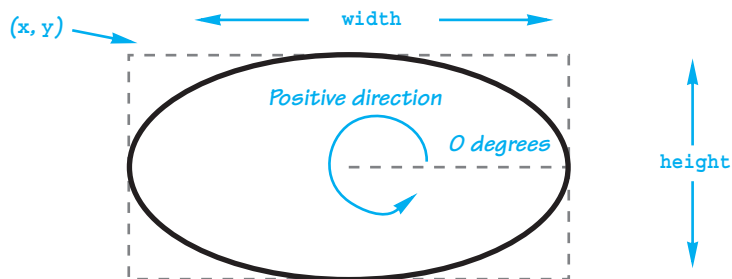
```
g.drawArc(x, y, width, height, 0, 90);
```



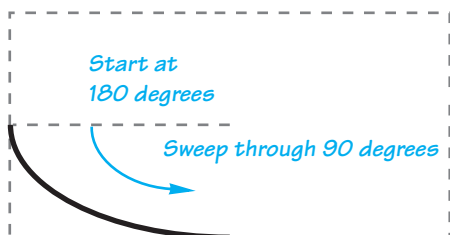
```
g.drawArc(x, y, width, height, 0, -90);
```



```
g.drawArc(x, y, width, height, 0, 360);
```



```
g.drawArc(x, y, width, height, 180, 90);
```



The angles are numbered with zero degrees, as shown in Display 18.14. In the first figure, the start angle is zero degrees. The counterclockwise direction is positive. So a start angle of 90 degrees would start at the top of the oval. A start angle of -90 degrees would start at the bottom of the oval. For example, the smile on the happy face in Display 18.13 has a start angle of 180 degrees, so it starts on the left end of the invisible oval. The last argument is also 180, so the arc is made visible through a counterclockwise direction of 180 degrees, or halfway around the oval in the counterclockwise direction.

### Self-Test Exercises

10. Give an invocation of a method to draw a horizontal line from point (30, 40) to point (100, 60). The calling object of type `Graphics` is named `g`.
11. Give an invocation of a method to draw a horizontal line of length 100 starting at position (30, 40) and extending to the right. The calling object of type `Graphics` is named `g`.
12. Give an invocation of a method that draws a vertical line of length 100 starting at position (30, 40) and extending downward. Use `graphicsObject` (of type `Graphic`) as the calling object.
13. Give an invocation of a method to draw a solid rectangle of width 100 and height 50 with the upper-left corner at position (20, 30). The calling object of type `Graphics` is named `graphicsObject`.
14. Give an invocation of a method to draw a solid rectangle of width 100 and height 50 with the upper-right corner at position (200, 300). The calling object of type `Graphics` is named `g`.
15. Give an invocation of a method to draw a circle of diameter 100 with the center at position (300, 400). The calling object of type `Graphics` is named `g`.
16. Give an invocation of a method to draw a circle of radius 100 with the center at position (300, 400). The calling object of type `Graphics` is named `g`.

### Rounded Rectangles ★

#### rounded rectangle

A **rounded rectangle** is a rectangle whose corners have been replaced by arcs so that the corners are rounded. For example, suppose `g` is of type `Graphics` and consider what would be drawn by the following:

```
g.drawRoundRect(x, y, width, height, arcWidth, arcHeight)
```

The arguments `x`, `y`, `width`, and `height` determine a regular rectangle in the usual way. The upper-left corner is at the point (`x`, `y`). The rectangle has the specified `width` and



height. The last two arguments, `arcWidth` and `arcHeight`, specify the arcs that will be used for the corners so as to produce a rounded rectangle. Each corner is replaced with a quarter of an oval that is `arcWidth` pixels wide and `arcHeight` pixels high. This is illustrated in Display 18.15. To obtain corners that are arcs of circles, just make `arcWidth` and `arcHeight` equal.

### paintComponent for Panels

You can draw figures on a `JPanel` and place the `JPanel` in a `JFrame`. When defining a `JPanel` class that contains a graphics drawing, use the method `paintComponent` instead of the method `paint`, but otherwise the details are similar to what we have seen for `JFrames`. `JFrames` use the method `paint`. However, `JPanels`—and in fact all `JComponents`—use the method `paintComponent`. A very simple example of using `paintComponent` with a `JPanel` is given in Display 18.16.

If you look back at Display 17.12 in Chapter 17, you will see that a `JPanel` is a `JComponent`, but a `JFrame` is not a `JComponent`. A `JFrame` is only a `Component`. This is why they use different methods to paint the screen.

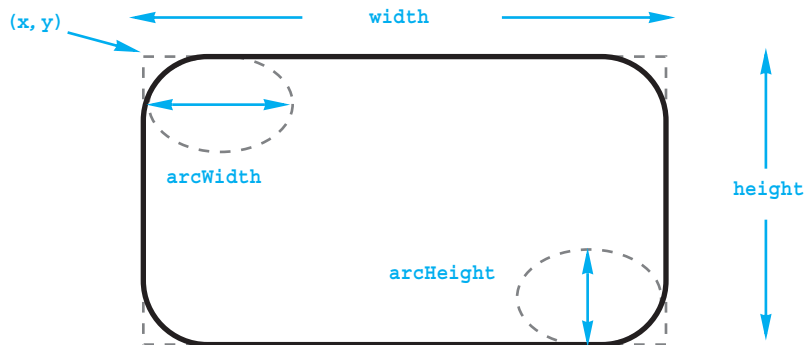
### Action Drawings and repaint

The program in Display 18.17 is similar to the program in Display 18.13. It draws a happy face similar to the happy face given in Display 18.13, but with one difference: There is a button at the bottom of the GUI that says `Click for a Wink`. When you click that button, the left eye winks. (Remember the left eye is on your right.) Let us see the details.

Display 18.15 A Rounded Rectangle

```
g.drawRoundRect(x, y, width, height, arcWidth, arcHeight);
```

*produces:*



Display 18.16 `paintComponent` Demonstration (part 1 of 2)

```
1 import javax.swing.JFrame;
2 import javax.swing.JPanel;
3 import java.awt.GridLayout;
4 import java.awt.Graphics;
5 import java.awt.Color;

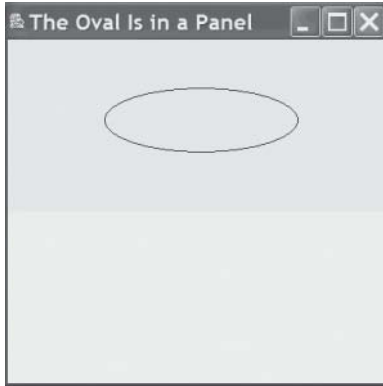
6 public class PaintComponentDemo extends JFrame
7 {
8 public static final int FRAME_WIDTH = 400;
9 public static final int FRAME_HEIGHT = 400;

10 private class FancyPanel extends JPanel
11 {
12 public void paintComponent(Graphics g)
13 {
14 super.paintComponent(g);
15 setBackground(Color.YELLOW);
16 g.drawOval(FRAME_WIDTH/4, FRAME_HEIGHT/8,
17 FRAME_WIDTH/2, FRAME_HEIGHT/6);
18 }
19 }

20 public static void main(String[] args)
21 {
22 PaintComponentDemo w = new PaintComponentDemo();
23 w.setVisible(true);
24 }

25 public PaintComponentDemo()
26 {
27 setSize(FRAME_WIDTH, FRAME_HEIGHT);
28 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
29 setTitle("The Oval Is in a Panel");
30 setLayout(new GridLayout(2, 1));
31 FancyPanel p = new FancyPanel();
32 add(p);
33 JPanel whitePanel = new JPanel();
34 whitePanel.setBackground(Color.WHITE);
35 add(whitePanel);
36 }
37 }
```

(continued)

Display 18.16 `paintComponent` Demonstration (part 2 of 2) (Source: Oracle Corporation)**RESULTING GUI**

The program in Display 18.17 has a private instance variable `wink` of type `boolean`. When the value of `wink` is `false`, the `paint` method draws an ordinary happy face. When the value of `wink` is `true`, the `paint` method draws the face the same except that the left eye is just a straight line, which looks like the eye is closed. The variable `wink` is initialized to `false`.

When the button labeled `Click for a Wink` is clicked, this sends an action event to the method `actionPerformed`. The method `actionPerformed` then changes the value of the variable `wink` to `true` and invokes the method `repaint`. This use of the method `repaint` is new, so let us discuss it a bit.

Every `JFrame` (in fact, every `Component` and every `Container`) has a method named `repaint`. The method `repaint` will repaint the screen so that any changes to the graphics being displayed will show on the screen. If you omit the invocation of `repaint` from the method `actionPerformed`, then the variable `wink` will change to `true`, but the screen will not change. Without an invocation of `repaint`, the face will not change, because the method `paint` must be called again with the new value of `wink` before the change takes effect. The method `repaint` does a few standard things and, most importantly, will also invoke the method `paint`, which redraws the screen. Be sure to note that you should invoke `repaint` and not `paint`.

Now we explain why, when `wink` has the value `true`, the method `paint` draws the face with the left eye changed. The relevant part of the code is the following, which draws the left eye:

```
if (wink)
 g.drawLine(X_LEFT_EYE, Y_LEFT_EYE,
 X_LEFT_EYE + EYE_WIDTH, Y_LEFT_EYE);
else
 g.fillOval(X_LEFT_EYE, Y_LEFT_EYE, EYE_WIDTH, EYE_HEIGHT);
```

Display 18.17 An Action Drawing (part 1 of 3)

```
1 import javax.swing.JFrame;
2 import javax.swing.JButton;
3 import java.awt.event.ActionListener;
4 import java.awt.event.ActionEvent;
5 import java.awt.BorderLayout;
6 import java.awt.Graphics;
7 import java.awt.Color;

8 public class ActionFace extends JFrame
9 {
10 public static final int WINDOW_WIDTH = 400;
11 public static final int WINDOW_HEIGHT = 400;

12 public static final int FACE_DIAMETER = 200;
13 public static final int X_FACE = 100;
14 public static final int Y_FACE = 100;

15 public static final int EYE_WIDTH = 20;
16 public static final int EYE_HEIGHT = 10;
17 public static final int X_RIGHT_EYE = X_FACE + 55;
18 public static final int Y_RIGHT_EYE = Y_FACE + 60;
19 public static final int X_LEFT_EYE = X_FACE + 130;
20 public static final int Y_LEFT_EYE = Y_FACE + 60;

21 public static final int MOUTH_WIDTH = 100;
22 public static final int MOUTH_HEIGHT = 50;
23 public static final int X_MOUTH = X_FACE + 50;
24 public static final int Y_MOUTH = Y_FACE + 100;
25 public static final int MOUTH_START_ANGLE = 180;
26 public static final int MOUTH_ARC_SWEEP = 180;

27 private boolean wink;

28 private class WinkAction implements ActionListener
29 {
30 public void actionPerformed(ActionEvent e)
31 {
32 wink = true;
33 repaint();
34 }
35 } // End of WinkAction inner class
```

(continued)

Display 18.17 An Action Drawing (part 2 of 3)

```
36 public static void main(String[] args)
37 {
38 ActionFace drawing = new ActionFace();
39 drawing.setVisible(true);
40 }

41 public ActionFace()
42 {
43 setSize(WINDOW_WIDTH, WINDOW_HEIGHT);
44 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
45 setTitle("Hello There!");
46 setLayout(new BorderLayout());
47 getContentPane().setBackground(Color.white);

48 JButton winkButton = new JButton("Click for a Wink.");
49 winkButton.addActionListener(new WinkAction());
50 add(winkButton, BorderLayout.SOUTH);
51 wink = false;
52 }

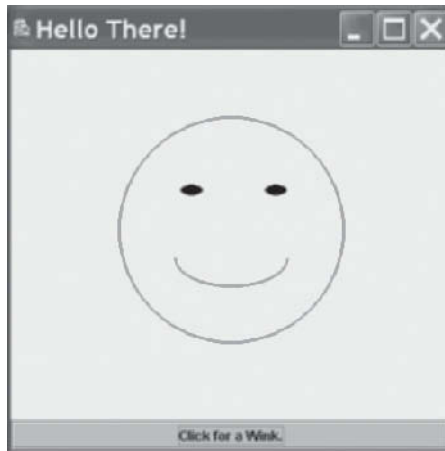
53 public void paint(Graphics g)
54 {
55 super.paint(g);
56 g.drawOval(X_FACE, Y_FACE, FACE_DIAMETER, FACE_DIAMETER);
57 //Draw Right Eye:
58 g.fillOval(X_RIGHT_EYE, Y_RIGHT_EYE, EYE_WIDTH, EYE_HEIGHT);
59 //Draw Left Eye:
60 if (wink)
61 g.drawLine(X_LEFT_EYE, Y_LEFT_EYE,
62 X_LEFT_EYE + EYE_WIDTH, Y_LEFT_EYE);
63 else
64 g.fillOval(X_LEFT_EYE, Y_LEFT_EYE, EYE_WIDTH, EYE_HEIGHT);
65 //Draw Mouth:
66 g.drawArc(X_MOUTH, Y_MOUTH, MOUTH_WIDTH, MOUTH_HEIGHT,
67 MOUTH_START_ANGLE, MOUTH_ARC_SWEEP);
68 }

69 }
```

Display 18.17 An Action Drawing (part 3 of 3) (Source: Oracle Corporation)

---

**Resulting GUI (When started)**



**Resulting GUI (After clicking the button)**



---

If `wink` has the value `true`, then the eye is drawn as a line, which looks like a closed eye. If `wink` has the value `false`, then the eye is drawn as an oval, which looks like an open eye.

### The repaint and paint Methods

When you change the graphic's contents in a window and want to update the window so that the new contents show on the screen, do not call `paint`; call `repaint`. The `repaint` method takes care of some overhead and then calls the `paint` method. Normally, you do not define `repaint`. As long as you define the `paint` method correctly, the `repaint` method should work correctly. Note that you often define `paint`, but you normally do not call it. On the other hand, normally you do not define `repaint`, but you do sometimes call it.

### Some More Details on Updating a GUI ★

Most of the changes to a GUI windowing system that we have seen are updated automatically so that they are visible on the screen. This is done by an object known as the **repaint manager**. The repaint manager works automatically, and you need not even be aware of its presence. However, there are a few updates that the repaint manager will not do for you. You have already learned that you need an invocation of `repaint` when your GUI changes the figure drawn in the `JFrame` as in Display 18.17.

Two other updating methods that you will often see when looking at Swing code are `validate` and `pack`.

Every container class has the method `validate`, which has no arguments. An invocation of `validate` causes the container to lay out its components again. An invocation of `validate` is a kind of “update” action that makes changes in the components actually happen on the screen. Many simple changes that are made to a Swing GUI, such as changing color or changing the text in a text field, happen automatically. Other changes, such as some kinds of addition of components or changes in visibility, may require an invocation of `validate` or some other “update” method. Sometimes it is difficult to decide whether an invocation of `validate` is necessary. When in doubt, include an invocation of `validate`. Although invoking `validate` when it is not needed can make your program a little less efficient, it will have no other ill effects on your GUI.

The method `pack` causes the window to be resized, usually to a smaller size, but more precisely to an approximation of a size known as the *preferred size*. (Yes, you can change the preferred size, but we do not have room to cover all of the Swing library in these few chapters.)

We do not have enough space in this book to go into all the details of how a GUI is updated on the screen, but these few remarks may make some code you find in more advanced books a little less puzzling.

## 18.4 Colors

*One colored picture is worth a thousand black and white pictures.*

ANONYMOUS

In this section, we tell you how to specify colors for the figures you draw with the graphics methods. We also show you how to define your own colors using the class `Color`.

## Specifying a Drawing Color

`setColor`

[extra  
code on  
website](#)

When drawing figures with methods such as `drawLine` inside of the definition of the `paint` method, you can think of your drawing as being done with a pen that can change colors. The method `setColor` will change the color of the pen.

For example, consider the happy face that is drawn by the GUI in Display 18.13. If you change the definition of the `paint` method to the version shown in Display 18.18, the eyes will be blue and the mouth will be red. (The file `HappyFaceColor.java` on the accompanying website contains a version of the changed program. It consists of the program in Display 18.13 with the definition of the `paint` method replaced by the one in Display 18.18 and with the class name changed from `HappyFace` to `HappyFaceColor`.)

### The `setColor` Method

When you are doing drawings with an object of the class `Graphics`, you can set the color of the drawing with an invocation of `setColor`. The color specified can later be changed with another invocation of `setColor`, so a single drawing can have multiple colors.

#### SYNTAX

```
Graphics_Object.setColor(Color_Object);
```

#### EXAMPLE

```
g.setColor(Color.BLUE);
```

Display 18.18 Adding Color

```
1 public void paint(Graphics g)
2 {
3 super.paint(g);
4 //Default is equivalent to: g.setColor(Color.black);
5 g.drawOval(X_FACE, Y_FACE, FACE_DIAMETER, FACE_DIAMETER);
6 //Draw Eyes:
7 g.setColor(Color.BLUE);
8 g.fillOval(X_RIGHT_EYE, Y_RIGHT_EYE, EYE_WIDTH, EYE_HEIGHT);
9 g.fillOval(X_LEFT_EYE, Y_LEFT_EYE, EYE_WIDTH, EYE_HEIGHT);
10 //Draw Mouth:
11 g.setColor(Color.RED);
12 g.drawArc(X_MOUTH, Y_MOUTH, MOUTH_WIDTH, MOUTH_HEIGHT,
13 MOUTH_START_ANGLE, MOUTH_ARC_SWEEP);
14 }
```

*If you replace the method `paint` in Display 18.13 with this version of `paint`, then the happy face will have blue eyes and red lips.*



**RGB color  
system****Color  
constructors**

## Defining Colors

Display 17.5 in Chapter 17 lists the standard colors in the class `Color`, which are defined for you. If that table does not have the colors you want, you can use the class `Color` to define your own colors. To understand how this is done, you need to first know a few basic facts about colors. By mixing red, green, and blue light in varying amounts, the human eye can be given the sensation of viewing any color the eye is capable of seeing. This is what an ordinary television set does to produce all the colors it displays. The television mixes red, green, and blue light and shines these lights on the screen in differing amounts. This is often called the **RGB color system**, for obvious reasons. Because a computer monitor is basically the same thing as a television set, colors for computer monitors can be produced in the same way. The Java `Color` class mixes amounts of red, green, and blue to produce any new color you might want.

When specifying the amount of each of the colors red, green, and blue, you can use either integers in the range 0 to 255 (inclusive) or `float` values in the range 0.0 to 1.0 (inclusive). For example, brown is formed by mixing red and green. So, the following defines a color called `brown` that will look like a shade of brown:

```
Color brown = new Color(200, 150, 0);
```

This color `brown` will have a 200.0/255 fraction of the maximum amount of red possible, a 150.0/255 fraction of the maximum amount of green possible, and no blue. If you want to use fractions to express the color, you can. The following is an equivalent way of defining the same color `brown`:

```
Color brown =
new Color((float)(200.0/255), (float)(150.0/255), (float)0.0);
```

You need the type casts `(float)` because the constructors for the class `Color` accept only arguments of type `int` or `float`, and numbers such as 200.0/255 and 0.0 are considered to be of type `double`, not of type `float`.

Some constructors for the class `Color` and some of the commonly used methods for the class `Color` are summarized in Display 18.19.

### RGB Colors

The class `Color` uses the RGB method of creating colors. That means that every color is a combination of the three colors red, green, and blue.

Display 18.19 Some Methods in the Class `Color` (part 1 of 2)

The class `Color` is in the `java.awt` package.

```
public Color(int r, int g, int b)
```

Constructor that creates a new `Color` with the specified RGB values. The parameters `r`, `g`, and `b` must each be in the range 0 to 255 (inclusive).

Display 18.19 Some Methods in the Class `Color` (part 2 of 2)

```
public Color(float r, float g, float b)
```

Constructor that creates a new `Color` with the specified RGB values. The parameters `r`, `g`, and `b` must each be in the range 0.0 to 1.0 (inclusive).

```
public int getRed()
```

Returns the red component of the calling object. The returned value is in the range 0 to 255 (inclusive).

```
public int getGreen()
```

Returns the green component of the calling object. The returned value is in the range 0 to 255 (inclusive).

```
public int getBlue()
```

Returns the blue component of the calling object. The returned value is in the range 0 to 255 (inclusive).

```
public Color brighter()
```

Returns a brighter version of the calling object color.

```
public Color darker()
```

Returns a darker version of the calling object color.

```
public boolean equals(Object c)
```

Returns `true` if `c` is equal to the calling object color; otherwise, returns `false`.



### PITFALL: Using `doubles` to Define a Color

Suppose you want to make a color that is made of half the possible amount of red, half the possible amount of blue, and no green. The following seems reasonable:

```
Color purple = new Color(0.5, 0.0, 0.5);
```

However, this will produce a compiler error. The numbers `0.5` and `0.0` are considered to be of type `double`, and this constructor requires arguments of type `float` (or of type `int`). So, an explicit type cast is required, as follows:

```
Color purple = new Color((float)0.5, (float)0.0, (float)0.5);
```

Java does allow the following method of specifying that a number is of type `float`, which can be simpler than the previous line of code:

```
Color purple = new Color(0.5f, 0.0f, 0.5f);
```

(continued)

**PITFALL: (continued)**

An even easier way to avoid these problems is to simply use `int` arguments, as in the following:

```
purple = new Color(127, 0, 127);
```

(You may feel that the values of 127 should be replaced by 128, but that is a minor point. You are not likely to even notice the difference in color between, say, 127 red and 128 red.)

In any final code produced, these `float` numbers should normally be replaced by defined constants, such as

```
public static final float RED_VALUE = (float)0.5;
public static final float GREEN_VALUE = (float)0.0;
public static final float BLUE_VALUE = (float)0.5;
```

Note that even though the defined constants are specified to be of type `float`, you still need a type cast. ■

## The JColorChooser Dialog Window

The class `JColorChooser` can be used to produce a dialog window that allows you to choose a color by looking at color samples or by choosing RGB values. The static method `showDialog` in the class `JColorChooser` produces a window that allows the user to choose a color. A sample program using this method is given in Display 18.20. The statement that launches the `JColorChooser` dialog window is the following:

```
sampleColor =
 JColorChooser.showDialog(this, "JColorChooser", sampleColor);
```

When this statement is executed, the window shown in the second GUI picture in Display 18.20 is displayed for the user to choose a color. Once the user has chosen a color and clicked the OK button, the window goes away and the chosen color is returned as the value of the `JColorChooser.showDialog` method invocation. So, in this example, the `Color` object returned is assigned to the variable `sampleColor`. If the user clicks the Cancel button, then the method invocation returns `null` rather than a color.

The method `JColorChooser.showDialog` takes three arguments. The first argument is the parent component, which is the component from which it was launched. In most simple cases, it is likely to be `this`, as it is in our example. The second argument is a title for the color chooser window. The third argument is the initial color for the color chooser window. The window shows the user samples of what the color he or she chooses will look like. The user can choose colors repeatedly, and each will be displayed in turn until the user clicks the OK button. The color displayed when the color chooser window first appears is that third argument.

The color chooser window has three tabs at the top labeled `Swatches`, `HSB`, and `RGB`. This gives the user three different ways to choose colors. If the `Swatches` tab is clicked, the window displays color samples for the user to choose from. This is the

## Display 18.20 JColorChooser Dialog (part 1 of 2)

```
1 import javax.swing.JFrame;
2 import javax.swing.JPanel;
3 import javax.swing.JButton;
4 import javax.swing.JColorChooser;
5 import java.awt.event.ActionListener;
6 import java.awt.event.ActionEvent;
7 import java.awt.BorderLayout;
8 import java.awt.FlowLayout;
9 import java.awt.Color;

10 public class JColorChooserDemo extends JFrame
11 implements ActionListener
12 {
13 public static final int WIDTH = 400;
14 public static final int HEIGHT = 200;

15 private Color sampleColor = Color.LIGHT_GRAY;

16 public static void main(String[] args)
17 {
18 JColorChooserDemo gui = new JColorChooserDemo();
19 gui.setVisible(true);
20 }

21 public JColorChooserDemo()
22 {
23 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
24 getContentPane().setBackground(sampleColor);
25 setLayout(new BorderLayout());
26 setTitle("JColorChooser Demo");
27 setSize(WIDTH, HEIGHT);
28 JPanel buttonPanel = new JPanel();
29 buttonPanel.setBackground(Color.WHITE);
30 buttonPanel.setLayout(new FlowLayout());
31 JButton chooseButton = new JButton("Choose a Color");
32 chooseButton.addActionListener(this);
33 buttonPanel.add(chooseButton);
34 add(buttonPanel, BorderLayout.SOUTH);
35 }
```

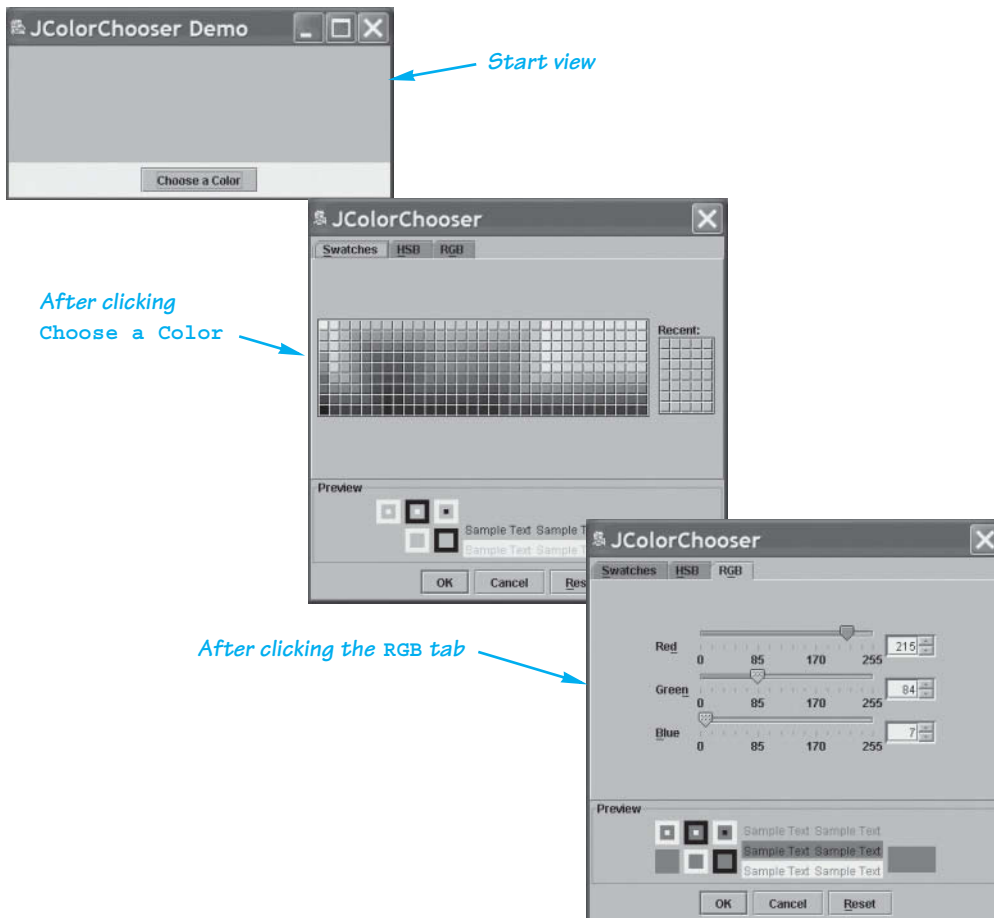
(continued)

Display 18.20 JColorChooser Dialog (part 2 of 2) (Source: Oracle Corporation)

```

36 public void actionPerformed(ActionEvent e)
37 {
38 if (e.getActionCommand().equals("Choose a Color"))
39 {
40 sampleColor =
41 JColorChooser.showDialog(this,
42 "JColorChooser", sampleColor);
43 if (sampleColor != null) // If a color was chosen
44 getContentPane().setBackground(sampleColor);
45 }
46 else
47 System.out.println("Unanticipated Error");
48 }

```

**Resulting GUI (Three views of one GUI)**

way the window first comes up. So, if the user clicks no tab, it is the same as clicking the `Swatches` tab. The `RGB` tab allows the user to choose a color by specifying the red, green, and blue values. The `HSB` tab gives the user a chance to choose colors in a way we will not discuss. To really understand the `JColorChooser` dialog window, you need to run the program in Display 18.20 to see it in action.

### Self-Test Exercises

17. How would you change the method `paint` in Display 18.18 so that the happy face has one blue eye (the right eye) and one green eye (the left eye)?
18. How would you change the method `paint` in Display 18.18 so that the happy face not only has blue eyes and a red mouth, but also has brown skin?

## 18.5 Fonts and the drawString Method

*It is not of so much consequence what you say,  
as how you say it.*

ALEXANDER SMITH, *Dreamthorp. On the Writing of Essays*, Strahan, 1863.

Java has facilities to add text to drawings and to modify the font of the text. We will show you enough to allow you to do most things you might want to do with text and fonts.

### The drawString Method

Display 18.21 contains a demonstration program for the method `drawString`. When the program is run, the GUI displays the text "Push the button." When the user clicks the button, the string is changed to "Thank you. I needed that." The text is written with the method `drawString`.

The method `drawString` is similar to the drawing methods in the class `Graphics`, but it displays text rather than a drawing. For example, the following line from Display 18.21 writes the string stored in the variable `theText` starting at the  $x$ - and  $y$ -coordinates `X_START` and `Y_START`:

```
g.drawString(theText, X_START, Y_START);
```

The string is written in the current font. A default font is used if no font is specified. The details about fonts are discussed in the next subsection.

Display 18.21 Using `drawString` (part 1 of 2)

```
1 import javax.swing.JFrame;
2 import javax.swing.JPanel;
3 import javax.swing.JButton;
4 import java.awt.event.ActionListener;
5 import java.awt.event.ActionEvent;
6 import java.awt.BorderLayout;
7 import java.awt.Graphics;
8 import java.awt.Color;
9 import java.awt.Font;

10 public class DrawStringDemo extends JFrame
11 implements ActionListener
12 {
13 public static final int WIDTH = 350;
14 public static final int HEIGHT = 200;
15 public static final int X_START = 20;
16 public static final int Y_START = 100;
17 public static final int POINT_SIZE = 24;

18 private String theText = "Push the button.";
19 private Color penColor = Color.BLACK;
20 private Font fontObject =
21 new Font("SansSerif", Font.PLAIN, POINT_SIZE);

22 public static void main(String[] args)
23 {
24 DrawStringDemo gui = new DrawStringDemo();
25 gui.setVisible(true);
26 }

27 public DrawStringDemo()
28 {
29 setSize(WIDTH, HEIGHT);
30 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
31 setTitle("drawString Demonstration");

32 getContentPane().setBackground(Color.WHITE);
33 setLayout(new BorderLayout());

34 JPanel buttonPanel = new JPanel();
35 buttonPanel.setBackground(Color.GRAY);
36 buttonPanel.setLayout(new BorderLayout());
37 JButton theButton = new JButton("The Button");
38 theButton.addActionListener(this);

39 buttonPanel.add(theButton, BorderLayout.CENTER);

40 add(buttonPanel, BorderLayout.SOUTH);
41 }
```

Display 18.21 Using drawString (part 2 of 2) (Source: Oracle Corporation)

```
42 public void paint(Graphics g)
43 {
44 super.paint(g);
45 g.setFont(fontObject);
46 g.setColor(penColor);
47 g.drawString(theText, X_START, Y_START);
48 }

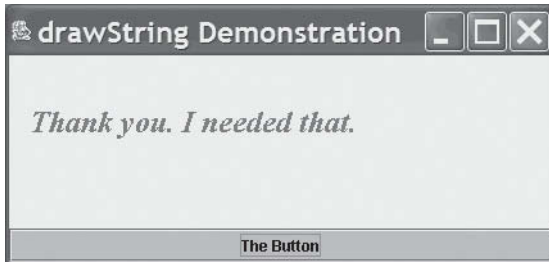
49 public void actionPerformed(ActionEvent e)
50 {
51 penColor = Color.RED;
52 fontObject =
53 new Font("Serif", Font.BOLD|Font.ITALIC, POINT_SIZE);
54 theText = "Thank you. I needed that.";

55 repaint();
56 }
57 }
```

RESULTING GUI (Start view)



RESULTING GUI (After clicking the button)





## Fonts

The program in Display 18.21 illustrates how the font for the method `drawString` is set. That program sets the font with the following line in the definition of the method `paint`:

```
setFont g.setFont(fontObject);
```

In this program, `fontObject` is a private instance variable of type `Font`. `Font` is a class in the `java.awt` package. Objects of the class `Font` represent fonts.

In Display 18.21, the variable `fontObject` is set using a constructor for the class `Font`. The initial font is set as part of the instance variable declaration in the following lines taken from Display 18.21:

```
private Font fontObject =
 new Font("SansSerif", Font.PLAIN, POINT_SIZE);
```

### Font

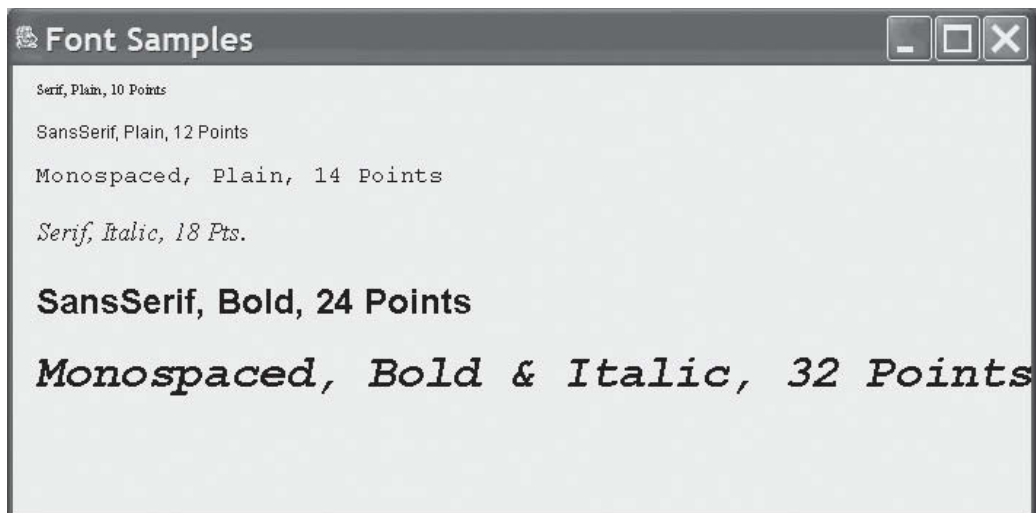
The constructor for the class `Font` creates a font in a given style and size. The first argument, in this case `"SansSerif"`, is a string that gives the name of the font (that is, the basic style). Some typical font names are `"Times"`, `"Courier"`, and `"Helvetica"`. You may use any font currently available on your system. Java guarantees that you will have at least the three fonts `"Monospaced"`, `"SansSerif"`, and `"Serif"`. To see what these fonts look like on your system, run the program `FontDisplay.java` on the accompanying website. It will produce the window shown in Display 18.22.

extra code  
on website

Most font names have no real meaning. The names just sounded right to the creator. However, the terms `"Serif"`, `"Sans Serif"`, and `"Monospaced"` do mean something,

Display 18.22 Result of Running `FontDisplay.java` (Source: Oracle Corporation)

*Fonts may look somewhat different on your system.*



which may help you keep the names of the three guaranteed fonts clear in your mind. **Serifs** are those small lines that sometimes finish off the ends of the lines in letters. For example, **S** has serifs (at the two ends of the curved line), but **s** does not have serifs. The "Serif" font will always have these decorative little lines. *Sans* means without, so the "SansSerif" font will not have these decorative little lines. As you might guess, "Monospaced" means that all the characters have equal width.

Fonts can be given style modifiers, such as bold or italic, and they can come in different sizes. The second and third arguments to the constructor for `Font` specify the style modifications and size for the font, as in the following, which occurs in the `actionPerformed` method in Display 18.21:

```
new Font("Serif", Font.BOLD|Font.ITALIC, POINT_SIZE);
```

The second argument specifies style modifications. Note that you can specify multiple style modifications by connecting them with the symbol `|` as in `Font.BOLD|Font.ITALIC`.<sup>2</sup> The last argument specifies the size of the letters in the version of the font created.

### point size

Character sizes are specified in units known as *points*, so the size of a particular version of a font is called a **point size**. One **point** is 1/72 of an inch, but measurements of font sizes are not as precise as might be ideal; two different fonts of the same point size may be slightly different in size.

The method `setFont` sets the font for the `Graphics` object, which is named `g` in Display 18.21. The font remains in effect until it is changed. If you do not specify any font, then a default font is used.

There is no simple way to change the properties of the current font, such as making it italic. Every change in a font normally requires that you define a new `Font` object and use it as an argument to `setFont`.

Display 18.23 gives some useful details about constructors, methods, and constants that are members of, or are related to, the class `Font`.

## The drawString Method

The `drawString` method writes the text given by the *String* at the point (*X*, *Y*) of the *Graphics\_Object*. The text is written in the current font, color, and font size.

### SYNTAX

```
Graphics_Object.drawString(String, X, Y);
```

### EXAMPLE

```
g.drawString("I love you madly.", X_START, Y_START);
```

<sup>2</sup>The symbol `|` produces a "bitwise or of the numbers," but that detail need not concern you. You need not even know what is meant by a "bitwise or of the numbers." Just think of `|` as a special way to connect style specifications.

Display 18.23 Some Methods and Constants for the Class `Font`

The class `Font` is in the `java.awt` package.

#### CONSTRUCTOR FOR THE CLASS `Font`

```
public Font(String fontName, int styleModifications, int size)
```

Constructor that creates a version of the font named by `fontName` with the specified `styleModifications` and `size`.

#### CONSTANTS IN THE CLASS `Font`

```
Font.BOLD
```

Specifies bold style.

```
Font.ITALIC
```

Specifies italic style.

```
Font.PLAIN
```

Specifies plain style—that is, not bold and not italic.

#### NAMES OF FONTS (These three are guaranteed by Java. Your system will probably have others as well as these.)

```
"Monospaced"
```

See Display 18.22 for a sample.

```
"SansSerif"
```

See Display 18.22 for a sample.

```
"Serif"
```

See Display 18.22 for a sample.

#### METHOD THAT USES `Font`

```
public abstract void setFont(Font fontObject)
```

This method is in the class `Graphics`. Sets the current font of the calling `Graphics` object to `fontObject`.

### Self-Test Exercises

19. Suppose `g` is an object of type `Graphics`. Write a line of code that will set the font for `g` to Sans Serif bold of size 14 points.
20. Suppose `g` is an object of type `Graphics`. Write a line of code that will set the font for `g` to Sans Serif bold and italic of size 14 points.

## Chapter Summary

- You can define a *window listener* class by having it implement the `WindowListener` interface.
- An *icon* is an object of the class `ImageIcon` and is created from a digital picture. You can add icons to `JButtons`, `JLabels`, and `JMenuItems`.
- You can use the class `JScrollPane` to add scroll bars to a text area.
- You can draw figures such as lines, ovals, and rectangles using methods in the class `Graphics`.
- You can use the method `setColor` to specify the color of each figure or text drawn with the method of the class `Graphics`.
- You can define your own colors using the class `Color`.
- Colors are defined using the *RGB* (red/green/blue) system.
- You can use the method `drawString` of the class `Graphics` to add text to a `JFrame` or `JPanel`.
- You can use the method `setFont` to set the font, style modifiers, and point size for text written with the `drawString` method of the `Graphics` class.

## Answers to Self-Test Exercises

1. All the methods in Display 18.1. If there is no particular action that you want the method to perform, you can give the method an empty body.
2. The smaller window goes away but the larger window stays. This is the default action for the close-window button and we did not change it for the smaller window.
3. `dispose`
4. The import statements are the same as in Display 18.2. The rest of the definition follows. This definition is in the file `WindowListenerDemo3` on the accompanying website.

```
public class WindowListenerDemo3 extends JFrame
 implements WindowListener
{
 public static final int WIDTH = 300; //for main window
 public static final int HEIGHT = 200; //for main window
 public static final int SMALL_WIDTH = 200;
 //for confirm window
 public static final int SMALL_HEIGHT = 100;
 //for confirm window

 private class ConfirmWindow extends JFrame
 implements ActionListener
```

extra code  
on website

```

 {
 public ConfirmWindow()
 {
 setSize(SMALL_WIDTH, SMALL_HEIGHT);
 getContentPane().setBackground(Color.YELLOW);
 setLayout(new BorderLayout());

 JLabel confirmLabel = new JLabel(
 "Are you sure you want to exit?");
 add(confirmLabel, BorderLayout.CENTER);

 JPanel buttonPanel = new JPanel();
 buttonPanel.setBackground(Color.ORANGE);
 buttonPanel.setLayout(new FlowLayout());

 JButton exitButton = new JButton("Yes");
 exitButton.addActionListener(this);
 buttonPanel.add(exitButton);

 JButton cancelButton = new JButton("No");
 cancelButton.addActionListener(this);
 buttonPanel.add(cancelButton);

 add(buttonPanel, BorderLayout.SOUTH);
 }

 public void actionPerformed(ActionEvent e)
 {
 String actionCommand = e.getActionCommand();

 if (actionCommand.equals("Yes"))
 System.exit(0);
 else if (actionCommand.equals("No"))
 dispose();//Destroys only the ConfirmWindow.
 else
 System.out.println(
 "Unexpected Error in Confirm Window.");
 }
 } //End of inner class ConfirmWindow

 public static void main(String[] args)
 {
 WindowListenerDemo3 demoWindow =
 new WindowListenerDemo3();
 demoWindow.setVisible(true);
 }

```

```
public WindowListenerDemo3()
{
 setSize(WIDTH, HEIGHT);
 setTitle("Window Listener Demonstration");

 setDefaultCloseOperation(
 JFrame.DO_NOTHING_ON_CLOSE);
 addWindowListener(this);

 getContentPane().setBackground(Color.LIGHT_GRAY);
 JLabel aLabel =
 new JLabel("I like to be sure you are sincere.");
 add(aLabel);
}

//The following are now methods of the class
WindowListenerDemo3:
public void windowOpened(WindowEvent e)
{}

public void windowClosing(WindowEvent e)
{
 ConfirmWindow checkers = new ConfirmWindow();
 checkers.setVisible(true);
}

public void windowClosed(WindowEvent e)
{}

public void windowIconified(WindowEvent e)
{}

public void windowDeiconified(WindowEvent e)
{}

public void windowActivated(WindowEvent e)
{}

public void windowDeactivated(WindowEvent e)
{}
}
```

5. JButton magicButton = new JButton("Magic Button");  
ImageIcon wizardIcon = new ImageIcon("wizard.gif");  
magicButton.setIcon(wizardIcon);

There are a number of other ways to accomplish the same thing. Here are two of a number of valid alternatives:

```

JButton magicButton = new JButton("Magic Button");
magicButton.setIcon(new ImageIcon("wizard.gif"));

ImageIcon wizardIcon = new ImageIcon("wizard.gif");
JButton magicButton =
 new JButton("Magic Button", wizardIcon);

```

6. 

```

ImageIcon wizardIcon = new ImageIcon("wizard.gif");
JLabel wizardPicture = new JLabel(wizardIcon);
picturePanel.add(wizardPicture);

```

There are a number of other ways to accomplish the same thing. Here is one valid alternative:

- ```

picturePanel.add(new JLabel(
    new ImageIcon("wizard.gif")));

```
7.

```

ImageIcon wizardIcon = new ImageIcon("wizard.gif");
JButton magicButton = new JButton(wizardIcon);
magicButton.setActionCommand("Kazam");

```

There are a number of other ways to accomplish the same thing. Here is one valid alternative:

- ```

JButton magicButton =
 new JButton(new ImageIcon("wizard.gif"));
magicButton.setActionCommand("Kazam");

```
8. No. You can invoke none, one, or both methods.
  9. No. The class `JTextArea` is a descendent class of the class `Component`. So, every `JTextArea` is also a `Component`.
  10. 

```
g.drawLine(30, 40, 100, 60);
```
  11. 

```
g.drawLine(30, 40, 130, 40);
```
  12. 

```
graphicsObject.drawLine(30, 40, 30, 140);
```
  13. 

```
graphicsObject.fillRect(20, 30, 100, 50);
```
  14. 

```
g.fillRect(200, 300, 100, 50);
```
  15. 

```
g.drawOval(250, 350, 100, 100);
```
  16. 

```
g.drawOval(200, 300, 200, 200);
```
  17. Insert `g.setColor(Color.GREEN)` as indicated next:  

```

//Draw Eyes:
g.setColor(Color.BLUE);
g.fillOval(X_RIGHT_EYE, Y_RIGHT_EYE, EYE_WIDTH, EYE_HEIGHT);
g.setColor(Color.GREEN);
g.fillOval(X_LEFT_EYE, Y_LEFT_EYE, EYE_WIDTH, EYE_HEIGHT);

```

18. Replace the following line in the `paint` method

```
g.drawOval(X_FACE, Y_FACE, FACE_DIAMETER, FACE_DIAMETER);
```

with

```
Color brown =
 new Color(200, 150, 0);
g.setColor(brown);
g.filloval(X_FACE, Y_FACE, FACE_DIAMETER, FACE_DIAMETER);
```

Note that there is no predefined color constant `Color.BROWN`, so you need to define a color for brown. You may prefer some other arguments instead of `(200, 150, 0)` so that you get a shade of brown that is more to your liking.

19. `g.setFont(new Font("SansSerif", Font.BOLD, 14));`

20. `g.setFont(new Font("SansSerif", Font.BOLD|Font.ITALIC, 14));`

## Programming Projects

1. A Sierpinski Gasket or Triangle is a type of fractal named after the Polish mathematician Waclaw Sierpinski who described some of its interesting properties in 1916. It is a nice example of how an orderly structure can be created as a result of random, chaotic behavior.

One way to create the fractal is to start with an equilateral triangle. Let us say that the corners are labeled `x`, `y`, and `z`.

1. Set `current` equal to point `x`.
2. Repeat many times (you can try 10,000).
  - a. Randomly pick `target` as one of the three `x`, `y`, or `z`.
  - b. Calculate the point halfway between `current` and `target`.
  - c. Set `current` to this halfway point.
  - d. Draw a pixel at location `current`. One way to do this is to fill or draw a tiny rectangle at this coordinate.

Write a program that draws a Sierpinski Gasket. You can pick the coordinates for the corners of the triangle. It may seem like you should get a random mess of dots but instead you get a very orderly picture!

To draw a single pixel at coordinate `(X,Y)`, use the `drawLine` method where the start and endpoints are both `(X,Y)`.



2. The file named `humphrey-img.txt` contained with the website for this book holds raw image data<sup>3</sup> of a Martian rock called “Humphrey” that was taken by the Mars Exploration Rover Spirit. The format of this text file is as follows:

First line: single number indicating the height and width of the image (in this case, 461).

Lines 2–462: A row of 461 numbers each separated by a space. Each number represents a pixel in grayscale and ranges from 0 to 255 where 0 is black and 255 is white.

For example, the following data describes a  $3 \times 3$  square where every pixel is white except for a black line along the diagonal from the upper-left corner to the bottom-right corner:

```
3
0 255 255
255 0 255
255 255 0
```

- Write a program to read in the data from the file and display it in a `JFrame` window. To draw a single pixel at coordinate  $(X,Y)$ , use the `drawLine` method where the start and endpoints are both  $(X,Y)$ . For speed, the contents of the file should be read into an array once and the array data used in the `paint()` method to draw the image.
- In this particular image, only about  $2/3$  of the shades of gray are used. For example, if the image consists entirely of shades in the range from 150–160, then the entire image would appear to be almost the same shade of gray. One method to enhance such an image is to scale the shade of each pixel to the entire range from 0 to 255. Pixels that were originally at value 150 would be drawn with the value 0, pixels that were originally 151 would be drawn with the value 25, and so on up to pixels of the shade 160, which would be drawn with the value 255. This technique spaces out the shading so the details are easier to see.

To compute the new shade for a pixel at coordinate  $(i, j)$ , do the following:

$$\text{NewShade}(i, j) = \frac{255 \times (\text{OriginalShade}(i, j) - \text{MinOriginalShade})}{(\text{MaxOriginalShade} - \text{MinOriginalShade})}$$

`MinOriginalShade` is the smallest scale of gray in the original image and `MaxOriginalShade` is the largest scale of gray in the original image.

Modify your program so that the image is drawn using the scaling technique described above. The brightness and details in the resulting image should be a little bit easier to distinguish.

3. Write a GUI program that uses the methods in the `Graphics` class to draw three faces—happy, frowny, scary—when three corresponding buttons labeled ‘Happy’, ‘Frowny’, ‘Scary’ are clicked.

<sup>3</sup>The original raw image data has been cropped and converted to a textual format for purposes of this project.

4. Write a “skeleton” GUI program that implements the `WindowListener` interface. Write code for each of the methods in Display 18.1 that simply prints out a message identifying which event occurred. Print the message out in a text field. Note that your program will not end when the close-window button is clicked (but will instead simply send a message to the text field saying that the `windowClosing` method has been invoked). Include a button labeled `Exit` that the user can click to end the program.
5. Enhance the face drawing in Display 18.17 in the following ways: Add color so the eyes are blue and the mouth is red. When the face winks, the line that represents a closed eye is black, not blue. Add a nose and a brown handlebar mustache. Add buttons labeled “Smile” and “Frown”. When the “Frown” button is clicked, the face shows a frown (upside down smile); when the “Smile” button is clicked, the face shows a smile. When the user clicks the close-window button, a window pops up to ask if the user is sure he or she wants to exit, as in Display 18.2.
6. Create a swing program to make use of `FontDemo` class. The program will display text in at least three different fonts such as `SansSerif`, `Arial`, and `Serif`. Allow the user to enter a sentence in a `TextField`. Add a `Preview` button to display the text in different fonts, point size and style (bold or italics).
7. The `MouseListener` interface allows you to retrieve mouse events. A program implements this interface in a manner similar to the `WindowListener` interface. For example, the following program creates a `JFrame` and outputs the X and Y coordinates of any mouse clicks within the `JFrame`. The `MouseListener` interface requires the implementing class to define the `mouseClicked`, `mouseEntered`, `mousePressed`, `mouseReleased`, and `mouseExited` methods. In the example, only the `mouseClicked` method has been completed.

```
import javax.swing.JFrame;
import java.awt.event.MouseListener;
import java.awt.event.MouseEvent;

public class MouseDemo extends JFrame implements MouseListener
{
 public void mouseClicked (MouseEvent e)
 {
 System.out.println(e.getX() + " " + e.getY());
 }
 public void mouseEntered (MouseEvent e) {}
 public void mousePressed (MouseEvent e) {}
 public void mouseReleased (MouseEvent e) {}
 public void mouseExited (MouseEvent e) {}
}
```



VideoNote  
Solution to  
Programming  
Project 18.7

```
public MouseDemo()
{
 super();
 setSize(600,400);
 setTitle("Mouse Demo");
 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
 addMouseListener(this); // Add listener for this object
}

public static void main(String[] args)
{
 MouseDemo m = new MouseDemo();
 m.setVisible(true);
}
}
```

Modify this program to create a simple drawing program. When the mouse button is clicked, a solid circle with a radius of three pixels should be drawn in the `JFrame` centered at the mouse coordinates. Draw the circle in the color of your choice. Make sure that the drawing is correctly redrawn if the `JFrame` is minimized and then displayed again.

8. Write a program that graphically displays a vertical bar chart to display the analysis of your last five months' mobile bill. The first input is an array of `String` that represents month name. The second input is an array of `double` that represents billing amount. The Y axis will represent Billing Amount and the X axis will represent Month. Display the following data using the bar chart.

| Month    | Billing Amount |
|----------|----------------|
| January  | 230.5          |
| February | 310.7          |
| March    | 370.0          |
| April    | 245.9          |
| May      | 117.1          |



# Java Never Ends **19**

## **19.1 MULTITHREADING** 1120

Example: A Nonresponsive GUI 1121

`Thread.sleep` 1121

The `getGraphics` Method 1125

Fixing a Nonresponsive Program Using Threads 1126

Example: A Multithreaded Program 1126

The Class `Thread` 1127

The `Runnable` Interface ★ 1130

Race Conditions and Thread Synchronization ★ 1133

## **19.2 NETWORKING WITH STREAM SOCKETS** 1138

Sockets 1138

Sockets and Threading 1142

The `URL` Class 1143

## **19.3 JAVABEANS** 1143

The Component Model 1144

The JavaBeans Model 1144

## **19.4 JAVA AND DATABASE CONNECTIONS** 1145

Relational Databases 1145

Java DB and JDBC 1146

SQL 1147

## **19.5 WEB PROGRAMMING WITH JAVA SERVER PAGES** 1158

Applets, Servlets, and Java Server Pages 1158

Oracle GlassFish Enterprise Server 1160

HTML Forms—the Common Gateway Interface 1161

JSP Declarations, Expressions, Scriptlets,  
and Directives 1163

## **19.6 INTRODUCTION TO FUNCTIONAL PROGRAMMING IN JAVA 8** 1172

## **19.7 INTRODUCTION TO JAVAFX** 1180

# 19 Java Never Ends

*And thick and fast they came at last,  
And more, and more, and more —*

LEWIS CARROLL, *Through the Looking-Glass*, Macmillan, 1871.

## Introduction

Of course there is only a finite amount of Java, but when you consider all the standard libraries and other accompanying software, the amount of power and the amount to learn seem to be endless. In this chapter, we give you a brief introduction to seven topics to give you a flavor of some of the directions you can take in extending your knowledge of Java. The seven topics are multithreading, networking with stream sockets, JavaBeans, the interaction of Java with database systems, Web programming with Java Server Pages, functional programming, and JavaFX.

## Prerequisites

You really should cover most of the book before covering this chapter. However, Section 19.1 requires only Chapters 17 and 18 and their prerequisites. Section 19.2 requires Chapters 1 through 5, 9, and 10. Sections 19.3 and 19.4 require only Chapters 1 through 6. Section 19.5 requires an understanding of HTML, which is given in Chapter 20. Chapter 20 is distributed as a file on the website included in this book. Aside from references to Section 19.1 in Section 19.2, all sections are independent of each other and may be read in any order.

## 19.1 Multithreading

*“Can you do two things at once?”  
“I have trouble doing one thing at once.”*

ANONYMOUS

### thread

A **thread** is a separate computation process. In Java, you can have programs with multiple threads. You can think of the threads as computations that execute in parallel. On a computer with enough processors, the threads might indeed execute in parallel. However, in most normal computing situations, the threads do not really do this. Instead, the computer switches resources between threads so that each thread in turn does a little bit of computing. To the user, this looks like the processes are executing in parallel.

You have already experienced threads. Modern operating systems allow you to run more than one program at the same time. For example, rather than waiting for your virus

scanning program to finish its computation, you can go on to, say, read your e-mail while the virus scanning program is still executing. The operating system is using threads to make this happen. There may or may not be some work being done in parallel depending on your computer and operating system. Most likely, the two computation threads are simply sharing computer resources so that they take turns using the computer's resources. When reading your e-mail, you may or may not notice that response is slower because resources are being shared with the virus scanning program. Your e-mail reading program is indeed slowed down, but because humans are so much slower than computers, any apparent slowdown is likely to be unnoticed.

### EXAMPLE: A Nonresponsive GUI

Display 19.1 contains a very simple action GUI. When the "Start" button is clicked, the GUI draws circles one after the other until a large portion of the window is filled with circles. There is 1/10 of a second pause between the drawing of each circle, so, you can see the circles appear one after the other. If you are interested in Java programming, this can be pretty exciting for the first few circles, but it quickly becomes boring. You are likely to want to end the program early, but if you click the close-window button, nothing will happen until the program is finished drawing all its little circles. We will use threads to fix this problem, but first let us understand this program, which does not really use threads in any essential way, despite the occurrence of the word `Thread` in the program. We explain this Swing program in the next few subsections.

#### `Thread.sleep`

In Display 19.1, the following method invocation produces a 1/10 of a second pause after drawing each of the circles:

```
doNothing(PAUSE);
```

which is equivalent to

```
doNothing(100);
```

The method `doNothing` is a private helping method that does nothing except call the method `Thread.sleep` and take care of catching any thrown exception. So, the pause is really created by the method invocation

```
Thread.sleep Thread.sleep(100);
```

This is a static method in the class `Thread` that pauses whatever thread includes the invocation. It pauses for the number of milliseconds (thousandths of a second) given as an argument. So, this pauses the computation of the program in Display 19.1 for 100 milliseconds or 1/10 of a second.

“Wait a minute,” you may think, “the program in Display 19.1 was not supposed to use threads in any essential way.” That is basically true, but every Java program uses threads in some way. If there is only one stream of computation, as in Display 19.1, then that is treated as a single thread by Java. So, threads are always used by Java, but not in an interesting way until more than one thread is used.

You can safely think of the invocation of

```
Thread.sleep(milliseconds);
```

as a pause in the computation that lasts (approximately) the number of milliseconds given as the argument. (If this invocation is in a thread of a multithreaded program, then the pause, like anything else in the thread, applies only to the thread in which it occurs.)

The method `Thread.sleep` can sometimes be handy even if you do not do any multithreaded programming. The class `Thread` is in the package `java.lang` and so requires no import statement.

#### Display 19.1 Nonresponsive GUI (part 1 of 3)

---

```

1 import javax.swing.JFrame;
2 import javax.swing.JPanel;
3 import javax.swing.JButton;
4 import java.awt.BorderLayout;
5 import java.awt.FlowLayout;
6 import java.awt.Graphics;
7 import java.awt.event.ActionListener;
8 import java.awt.event.ActionEvent;

9 /**
10 Packs a section of the frame window with circles, one at a time.
11 */
12 public class FillDemo extends JFrame implements ActionListener
13 {
14 public static final int WIDTH = 300;
15 public static final int HEIGHT = 200;
16 public static final int FILL_WIDTH = 300;
17 public static final int FILL_HEIGHT = 100;
18 public static final int CIRCLE_SIZE = 10;
19 public static final int PAUSE = 100; //milliseconds

20 private JPanel box;

21 public static void main(String[] args)
22 {
23 FillDemo gui = new FillDemo();
24 gui.setVisible(true);
25 }

```

## Display 19.1 Nonresponsive GUI (part 2 of 3)

```

26 public FillDemo()
27 {
28 setSize(WIDTH, HEIGHT);
29 setTitle("FillDemo");
30 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
31 setLayout(new BorderLayout());
32 box = new JPanel();
33 add(box, "Center");

34 JPanel buttonPanel = new JPanel();
35 buttonPanel.setLayout(new FlowLayout());
36 JButton startButton = new JButton("Start");
37 startButton.addActionListener(this);
38 buttonPanel.add(startButton);
39 add(buttonPanel, "South");
40 }

41 public void actionPerformed(ActionEvent e)
42 {
43 fill();
44 }

45 public void fill()
46 {
47 Graphics g = box.getGraphics();

48 for (int y = 0; y < FILL_HEIGHT; y = y + CIRCLE_SIZE)
49 for (int x = 0; x < FILL_WIDTH; x = x + CIRCLE_SIZE)
50 {
51 g.fillOval(x, y, CIRCLE_SIZE, CIRCLE_SIZE);
52 doNothing(PAUSE);
53 }
54 }

55 public void doNothing(int milliseconds)
56 {
57 try
58 {
59 Thread.sleep(milliseconds);
60 }
61 catch (InterruptedException e)
62 {
63 System.out.println("Unexpected interrupt");
64 System.exit(0);
65 }
66 }
67 }

```

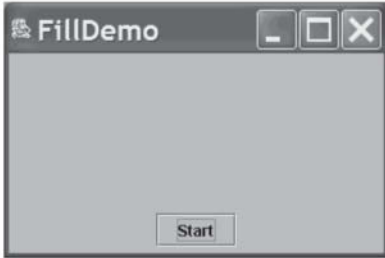
*Nothing else can happen until  
actionPerformed returns, which  
does not happen until fill returns.*

*Everything stops for  
100 milliseconds  
(1/10 of a second).*

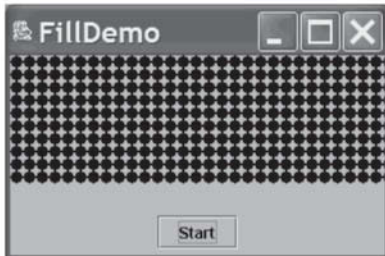
(continued)



Display 19.1 Nonresponsive GUI (part 3 of 3) (Source: Oracle Corporation)

**RESULTING GUI** (When started)**RESULTING GUI** (While drawing circles)

*If you click the close-window button while the circles are being drawn, the window will not close until all the circles are drawn.*

**RESULTING GUI** (After all circles are drawn)

The method `Thread.sleep` can throw an `InterruptedException`, which is a checked exception—that is, it must be either caught in a catch block or declared in a throws clause. We do not discuss `InterruptedException` in this book, leaving it for more advanced books on multithreaded programming, but it has to do with one thread interrupting another thread. We will simply note that an `InterruptedException` may be thrown by `Thread.sleep` and so must be accounted for—in our case, by a simple catch block. The class `InterruptedException` is in the `java.lang` package and so requires no import statement.

### Thread.sleep

`Thread.sleep` is a static method in the class `Thread` that pauses the thread that includes the invocation. It pauses for the number of milliseconds (thousandths of a second) given as an argument.

The method `Thread.sleep` may throw an `InterruptedException`, which is a checked exception and so must be either caught in a `catch` block or declared in a `throws` clause.

The classes `Thread` and `InterruptedException` are both in the package `java.lang`, so neither requires any import statement.

Note that `Thread.sleep` can be invoked in an ordinary (single thread) program of the kind we have seen before this chapter. It will insert a pause in the single thread of that program.

#### SYNTAX

```
Thread.sleep(Number_Of_Milliseconds) ;
```

#### EXAMPLE

```
try
{
 Thread.sleep(100); //Pause of 1/10 of a second
}
catch(InterruptedException e)
{
 System.out.println("Unexpected interrupt");
}
```

### The `getGraphics` Method

The other new method in Display 19.1 is the `getGraphics` method, which is used in the following line from the method `fill`:

```
getGraphics Graphics g = box.getGraphics();
```

The `getGraphics` method is almost self-explanatory. As we already noted in Chapter 18, almost every item displayed on the screen (more precisely, every `JComponent`) has an associated `Graphics` object. The method `getGraphics` is an accessor method that returns the associated `Graphics` object (of the calling object for `getGraphics`)—in this case, the `Graphics` object associated with the panel `box`. This gives us a `Graphics` object that can draw circles (or anything else) in the panel box.

We still need to say a bit more about why the program in Display 19.1 makes you wait before it will respond to the close-window button, but otherwise this concludes our explanation of Display 19.1. The rest of the code consists of standard things we have seen before.

**getGraphics**

Every `JComponent` has an associated `Graphics` object. The method `getGraphics` is an accessor method that returns the associated `Graphics` object of its calling object.

**SYNTAX**

```
Component.getGraphics();
```

**EXAMPLE (see Display 19.1 for context)**

```
Graphics g = box.getGraphics();
```

## Fixing a Nonresponsive Program Using Threads

Now that we have discussed the new items in the program in Display 19.1, we are ready to explain why it is nonresponsive and to show you how to use threads to write a responsive version of that program.

Recall that when you run the program in Display 19.1, it draws circles one after the other to fill a portion of the frame. Although there is only a 1/10 of a second pause between drawing each circle, it can still seem like it takes a long time to finish. So, you are likely to want to abort the program and close the window early. But, if you click the close-window button, the window will not close until the GUI is finished drawing all the circles.

Here is why the close-window button is nonresponsive: The method `fill`, which draws the circles, is invoked in the body of the method `actionPerformed`. So, the method `actionPerformed` does not end until after the method `fill` ends. And, until the method `actionPerformed` ends, the GUI cannot go on to do the next thing, which is probably to respond to the close-window button.

Here is how we fixed the problem: We have the method `actionPerformed` create a new (independent) thread to draw the circles. Once `actionPerformed` does this, the new thread is an independent process that proceeds on its own. The method `actionPerformed` has nothing more to do with this new thread; the work of `actionPerformed` is ended. So, the main thread (the one with `actionPerformed`) is ready to move on to the next thing, which will probably be to respond promptly to a click of the close-window button. At the same time, the new thread draws the circles. So, the circles are drawn, but at the same time a click of the close-window button will end the program. The program that implements this multithreaded solution is given in the next Programming Example.

### EXAMPLE: A Multithreaded Program

Display 19.2 contains a program that uses a main thread and a second thread to implement the technique discussed in the previous subsection. The general approach was outlined in the previous subsection, but we need to explain the Java code details. We do that in the next few subsections.

## The Class Thread

### Thread

In Java, a thread is an object of the class `Thread`. The normal way to program a thread is to define a class that is a derived class of the class `Thread`. An object of this derived class will be a thread that follows the programming given in the definition of the derived (thread) class.

### run()

Where do you do the programming of a thread? The class `Thread` has a method named `run`. The definition of the method `run` is the code for the thread. When the thread is executed, the method `run` is executed. Of course, the method defined in the class `Thread` and inherited by any derived class of `Thread` does not do what you want your thread to do. So, when you define a derived class of `Thread`, you override the definition of the method `run` to do what you want the thread to do.

In Display 19.2, the inner class `Packer` is a derived class of the class `Thread`. The method `run` for the class `Packer` is defined to be exactly the same as the method `fill` in our previous, unresponsive GUI (Display 19.1). So, an object of the class `Packer` is a thread that will do what `fill` does, namely draw the circles to fill up a portion of the window.

### start()

The method `actionPerformed` in Display 19.2 differs from the method `actionPerformed` in our older, nonresponsive program (Display 19.1) in that the invocation of the method `fill` is replaced with the following:

```
Packer packerThread = new Packer();
packerThread.start();
```

This creates a new, independent thread named `packerThread` and starts it processing. Whatever `packerThread` does, it does as an independent thread. The main thread can then allow `actionPerformed` to end, and the main thread will be ready to respond to any click of the close-window button.

Display 19.2 Threaded Version of `FillDemo` (part 1 of 3)

```
1 import javax.swing.JFrame;
2 import javax.swing.JPanel;
3 import javax.swing.JButton;
4 import java.awt.BorderLayout;
5 import java.awt.FlowLayout;
6 import java.awt.Graphics;
7 import java.awt.event.ActionListener;
8 import java.awt.event.ActionEvent;

9 public class ThreadedFillDemo extends JFrame implements ActionListener
10 {
11 public static final int WIDTH = 300;
12 public static final int HEIGHT = 200;
13 public static final int FILL_WIDTH = 300;
14 public static final int FILL_HEIGHT = 100;
15 public static final int CIRCLE_SIZE = 10;
16 public static final int PAUSE = 100; //milliseconds
```

*The GUI produced is identical to the GUI produced by Display 19.1 except that in this version the close-window button works even while the circles are being drawn, so you can end the GUI early if you get bored.*

(continued)

Display 19.2 Threaded Version of FillDemo (part 2 of 3)

```

17 private JPanel box;

18 public static void main(String[] args)
19 {
20 ThreadedFillDemo gui = new ThreadedFillDemo();
21 gui.setVisible(true);
22 }

23 public ThreadedFillDemo()
24 {
25 setSize(WIDTH, HEIGHT);
26 setTitle("Threaded Fill Demo");
27 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

28 setLayout(new BorderLayout());

29 box = new JPanel();
30 add(box, "Center");

31 JPanel buttonPanel = new JPanel();
32 buttonPanel.setLayout(new FlowLayout());
33 JButton startButton = new JButton("Start");
34 startButton.addActionListener(this);
35 buttonPanel.add(startButton);
36 add(buttonPanel, "South");
37 }

38 public void actionPerformed(ActionEvent e)
39 {
40 Packer packerThread = new Packer();
41 packerThread.start();
42 }

43 private class Packer extends Thread
44 {
45 public void run()
46 {
47 Graphics g = box.getGraphics();
48 for (int y = 0; y < FILL_HEIGHT; y = y + CIRCLE_SIZE)
49 for (int x = 0; x < FILL_WIDTH; x = x + CIRCLE_SIZE)
50 {
51 g.fillOval(x, y, CIRCLE_SIZE, CIRCLE_SIZE);
52 doNothing(PAUSE);
53 }
54 }

```

*You need a thread object, even if there are no instance variables in the class definition of Packer.*

*start "starts" the thread and calls run.*

*run is inherited from Thread but needs to be overridden. This definition of run is identical to that of fill in Display 19.1.*

## Display 19.2 Threaded Version of FillDemo (part 3 of 3)

```
55 public void doNothing(int milliseconds)
56 {
57 try
58 {
59 Thread.sleep(milliseconds);
60 }
61 catch (InterruptedException e)
62 {
63 System.out.println("Unexpected interrupt");
64 System.exit(0);
65 }
66 }
67 } //End Packer inner class
68 }
```

`run()` We need only to discuss the method `start`, and we will be through with our explanation. The method `start` initiates the computation (process) of the calling thread. It performs some overhead associated with starting a thread, and then it invokes the `run` method for the thread. As we have already seen, the `run` method of the class `Packer` in Display 19.2 draws the circles we want, so the invocation

```
packerThread.start();
```

does this as well, because it calls `run`. Note that you do not invoke `run` directly. Instead, you invoke `start`, which does some other needed things and then invokes `run`.

This ends our explanation of the multithreaded program in Display 19.2, but there is still one, perhaps puzzling, thing about the class `Packer` that we should explain. The definition of the class `Packer` includes no instance variables. So, why do we need to bother with an object of the class `Packer`? Why not simply make all the methods static and call them with the class name `Packer`? The answer is that the only way to get a new thread is to create a new `Thread` object. The things inherited from the class `Thread` are what the object needs to be a thread. Static methods do not a thread make. In fact, not only will static methods not work, the compiler will not even allow you to define `run` to be static. This is because `run` is inherited from `Thread` as a nonstatic method; this cannot be changed to static when overriding a method definition. The compiler will not let you even try to do this without creating an object of the class `Packer`.

### The Thread Class

A thread is an object of the class `Thread`. The normal way to program a thread is to define a class that is a derived class of the class `Thread`. An object of this derived class will be a thread that follows the programming given in the definition of the derived (thread) class's method named `run`.

Any thread class inherits the method `start` from the class `Thread`. An invocation of `start` by an object of a thread class will start the thread and invoke the method `run` for that thread.

See Display 19.2 for an example.

### The Runnable Interface ★

There are times when you would rather not make a thread class a derived class of the class `Thread`. The alternative to making your class a derived class of the class `Thread` is to have your class instead implement the `Runnable` interface. The `Runnable` interface has only one method heading:

```
public void run()
```

A class that implements the `Runnable` interface must still be run from an instance of the class `Thread`. This is usually done by passing the `Runnable` object as an argument to the thread constructor. The following is an outline of one way to do this:

```
public class ClassToRun extends SomeClass implements Runnable
{

 public void run()
 {
 //Fill this just as you would if ClassToRun
 //were derived from Thread.
 }

 public void startThread()
 {
 Thread theThread = new Thread(this);
 theThread.start();
 }

}
```

The previous method `startThread` is not compulsory, but it is one way to produce a thread that will in turn run the `run` method of an object of the class `ClassToRun`. In Display 19.3, we have rewritten the program in Display 19.2 using the `Runnable` interface. The program behaves exactly the same as the one in Display 19.2.

## Display 19.3 The Runnable Interface (part 1 of 2)

```
1 import javax.swing.JFrame;
2 import javax.swing.JPanel;
3 import javax.swing.JButton;
4 import java.awt.BorderLayout;
5 import java.awt.FlowLayout;
6 import java.awt.Graphics;
7 import java.awt.event.ActionListener;
8 import java.awt.event.ActionEvent;

9 public class ThreadedFillDemo2 extends JFrame
10 implements ActionListener, Runnable
11 {
12 public static final int WIDTH = 300;
13 public static final int HEIGHT = 200;
14 public static final int FILL_WIDTH = 300;
15 public static final int FILL_HEIGHT = 100;
16 public static final int CIRCLE_SIZE = 10;
17 public static final int PAUSE = 100; //milliseconds
18 private JPanel box;

19 public static void main(String[] args)
20 {
21 ThreadedFillDemo2 gui = new ThreadedFillDemo2();
22 gui.setVisible(true);
23 }

24 public ThreadedFillDemo2()
25 {
26 setSize(WIDTH, HEIGHT);
27 setTitle("Threaded Fill Demo");
28 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

29 setLayout(new BorderLayout());

30 box = new JPanel();
31 add(box, "Center");

32 JPanel buttonPanel = new JPanel();
33 buttonPanel.setLayout(new FlowLayout());

34 JButton startButton = new JButton("Start");
35 startButton.addActionListener(this);
36 buttonPanel.add(startButton);
37 add(buttonPanel, "South");
38 }
```

(continued)



## Display 19.3 The Runnable Interface (part 2 of 2)

```
39 public void actionPerformed(ActionEvent e)
40 {
41 startThread();
42 }

43 public void run()
44 {
45 Graphics g = box.getGraphics();
46 for (int y = 0; y < FILL_HEIGHT; y = y + CIRCLE_SIZE)
47 for (int x = 0; x < FILL_WIDTH; x = x + CIRCLE_SIZE)
48 {
49 g.fillOval(x, y, CIRCLE_SIZE, CIRCLE_SIZE);
50 doNothing(PAUSE);
51 }
52 }

53 public void startThread()
54 {
55 Thread theThread = new Thread(this);
56 theThread.start();
57 }

58 public void doNothing(int milliseconds)
59 {
60 try
61 {
62 Thread.sleep(milliseconds);
63 }
64 catch (InterruptedException e)
65 {
66 System.out.println("Unexpected interrupt");
67 System.exit(0);
68 }
69 }
70 }
```

**Self-Test Exercises**

1. Because `sleep` is a static method, how can it possibly know what thread it needs to pause?
2. Where was polymorphism used in the program in Display 19.2? (*Hint:* We are looking for an answer involving the class `Packer`.)

## race condition

## Race Conditions and Thread Synchronization ★

When multiple threads change a shared variable, it is sometimes possible that the variable will end up with the wrong (and often unpredictable) value. This is called a **race condition** because the final value depends on the sequence in which the threads access the shared value.

For example, consider two threads where each thread runs the following code:

```
int local;
local = sharedVariable;
local++;
sharedVariable = local;
```

The intent is for each thread to increment `sharedVariable` by one, so if there are two threads, then `sharedVariable` should be incremented by two. However, consider the case where `sharedVariable` is 0. The first thread runs and executes the first two statements, so its variable `local` is set to 0. Now there is a context switch to the second thread. The second thread executes all four statements, so its variable `local` is set to 0 and incremented, and `sharedVariable` is set to 1. Now we return to the first thread and it continues where it left off, which is the third statement. The variable `local` is 0 so it is incremented to 1 and then the value 1 is copied into `sharedVariable`. The end result after both threads are done is that `sharedVariable` has the value 1, and we lost the value written by thread two!

You might think that this problem could be avoided by replacing our code with a single statement such as

```
sharedVariable++;
```

Unfortunately, this will not solve our problem because the statement is not guaranteed to be an “atomic” action and there could still be a context switch to another thread “in the middle” of executing the statement.

To demonstrate this problem, consider the `Counter` class shown in Display 19.4. This simple class merely stores a variable that increments a counter. It uses the somewhat roundabout way to increment the counter on purpose to increase the likelihood of a race condition.

The way we will demonstrate the race condition is to do the following:

1. Create a single instance of the `Counter` class.
2. Create an array of many threads (30,000 in the example) where each thread references the single instance of the `Counter` class.
3. Each thread runs and invokes the `increment()` method.
4. Wait for each thread to finish and then output the value of the counter. If there are no race conditions, then its value should be 30,000. If there are race conditions, then the value will be less than 30,000.

We create many threads to increase the likelihood that the race condition occurs. With only a few threads, it is not likely that there will be a switch to another thread inside the `increment()` method at the right point to cause a problem.

Display 19.4 The Counter Class

```
1 public class Counter
2 {
3 private int counter;
4 public Counter()
5 {
6 counter = 0;
7 }
8 public int value()
9 {
10 return counter;
11 }
12 public void increment()
13 {
14 int local;
15 local = counter;
16 local++;
17 counter = local;
18 }
19 }
```

The only new tool that we need for our demonstration program is a way to wait for all the threads to finish. If we do not wait, then our program might output the counter before all the threads have had a chance to increment the value. We can wait by invoking the `join()` method for every thread we create. This method waits for the thread to complete. The `join()` method throws `InterruptedException`. This is a checked exception so we must use the try/catch mechanism.



**VideoNote**  
Walkthrough  
of a Program  
with Race  
Conditions

The class `RaceConditionTest` in Display 19.5 illustrates the race condition. You may have to run the program several times before you get a value less than 30,000. Problems as a result of race conditions are often rare occurrences. This makes them extremely hard to find and debug!

Display 19.5 The `RaceConditionTest` Class (part 1 of 2)

```
1 public class RaceConditionTest extends Thread
2 {
3 private Counter countObject;
4 public RaceConditionTest(Counter ctr)
5 {
6 countObject = ctr;
7 }
```

*Stores a reference to a single Counter object*

## Display 19.5 The RaceConditionTest Class (part 2 of 2)

```

8 public void run()
9 {
10 countObject.increment();
11 }

12 public static void main(String[] args)
13 {
14 int i;
15 Counter masterCounter = new Counter();
16 RaceConditionTest[] threads = new RaceConditionTest[30000];

17 System.out.println("The counter is " + masterCounter.value());
18 for (i = 0; i < threads.length; i++)
19 {
20 threads[i] = new RaceConditionTest(masterCounter);
21 threads[i].start();
22 }

23 // Wait for the threads to finish
24 for (i = 0; i < threads.length; i++)
25 {
26 try
27 {
28 threads[i].join();
29 }
30 catch (InterruptedException e)
31 {
32 System.out.println(e.getMessage());
33 }
34 }
35 System.out.println("The counter is " + masterCounter.value());
36 }
37 }
38 }

```

*Invokes the code in Display 19.4 where the race condition occurs*

*The single instance of the Counter object*

*Array of 30,000 threads*

*Gives each thread a reference to the single Counter object and starts each thread*

*Waits for the thread to complete*

Sample Dialogue (output will vary)

```

The counter is 0
The counter is 29998

```

**critical region**  
**synchronized**

So how do we fix this problem? The solution is to make each thread wait so only one thread can run the code in `increment()` at a time. This section of code is called a **critical region**. Java allows you to add the keyword **synchronized** around a critical region to enforce the requirement that only one thread is allowed to execute in this region at a time. All other threads will wait until the thread inside the region is finished.

In this particular case, we can add the keyword `synchronized` to either the method or around the specific code. If we add `synchronized` to the `increment()` method in the `Counter` class, then it looks like this:

```
public synchronized void increment()
{
 int local;
 local = counter;
 local++;
 counter = local;
}
```

If we add `synchronized` inside the code, then we can write

```
public void increment()
{
 int local;
 synchronized (this)
 {
 local = counter;
 local++;
 counter = local;
 }
}
```

Either version will result in a counter whose final value is always 30,000. There are many other issues involved in thread management, such as concurrency and synchronization. These concepts are often covered in more detail in an operating systems or parallel programming course.

## Self-Test Exercises

3. In the `run()` method of Display 19.5, make the thread sleep a random amount of time between one and five milliseconds. You should see an increase in the number of problems caused by race conditions. Can you explain why?
4. Here is some code that synchronizes thread access to a shared variable. How come it is not guaranteed to output 30,000 every time it is run?

```
public class Counter
{
 private int counter;
 public Counter()
 {
 counter = 0;
 }
}
```

**Self-Test Exercises** (continued)

```
 public int value()
 {
 return counter;
 }
 public synchronized void increment()
 {
 counter++;
 }
 }
 public class RaceConditionTest extends Thread
 {
 private Counter countObject;
 public RaceConditionTest(Counter ctr)
 {
 countObject = ctr;
 }
 public void run()
 {
 countObject.increment();
 }
 public static void main(String[] args)
 {
 int i;
 Counter masterCounter = new Counter();
 RaceConditionTest[] threads = new RaceConditionTest[30000];
 System.out.println("The counter is " + masterCounter.
 value());
 for (i = 0; i < threads.length; i++)
 {
 threads[i] = new RaceConditionTest(masterCounter);
 threads[i].start();
 }
 System.out.println("The counter is " + masterCounter.
 value());
 }
 }
```

## 19.2 Networking with Stream Sockets

*Since in order to speak, one must first listen, learn to speak by listening.*

RUMI DAYLIGHT *A Daybook of Spiritual Guidance*, Shambhala, 1999,  
trans. Camille Helminski, 1999.

### Transmission Control Protocol (TCP)

server

client

### User Datagram Protocol (UDP)

When computers want to communicate with each other over a network, each computer must speak the same “language.” In other words, the computers need to communicate using the same *protocol*. One of the most common protocols today is **TCP**, or the **Transmission Control Protocol**. For example, the HTTP protocol used to transmit Web pages is based on TCP. TCP is a stream-based protocol in which a stream of data is transmitted from the sender to the receiver. TCP is considered a reliable protocol because it guarantees that data from the sender is received in the same order in which it was sent. An analogy to TCP is the telephone system. A connection is made when the phone is dialed and the participants communicate by speaking back and forth. In TCP, the receiver must first be listening for a connection, the sender initiates the connection, and then the sender and receiver can transmit data. The program that is waiting for a connection is called the **server** and the program that initiates the connection is called the **client**.

An alternate protocol is **UDP**, or the **User Datagram Protocol**. In UDP, packets of data are transmitted but no guarantee is made regarding the order in which the packets are received. An analogy to UDP is the postal system. Letters that are sent might be received in an unpredictable order, or lost entirely with no notification. Although Java provides support for UDP, we will only introduce TCP in this section.

### Sockets

sockets

port

Network programming is implemented in Java using **sockets**. A socket describes one end of the connection between two programs over the network. A socket consists of an address that identifies the remote computer and a **port** for both the local and remote computer. The port is assigned an integer value between 0 and 65,535 that is used to identify which program should handle data received from the network. Two applications may not bind to the same port. Typically, ports 0 to 1,024 are reserved for use by well-known services implemented by your operating system.

The process of client/server communication is shown in Display 19.6. First, the server waits for a connection by listening on a specific port. When a client connects to this port, a new socket is created that identifies the remote computer, the remote port, and the local port. A similar socket is created on the client. Once the sockets are created on both the client and the server, data can be transmitted using streams in a manner very similar to the way we implemented file I/O in Chapter 10.

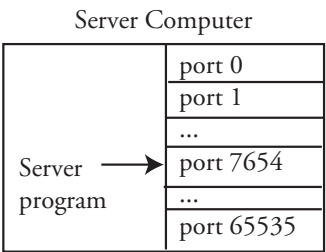
Display 19.7 shows how to create a simple server that listens on port 7654 for a connection. Once it receives a connection, a new socket is returned by the `accept()` method. From this socket, we create a `BufferedReader`, just as if we were reading from a text file described in Chapter 10. Data is transmitted to the socket using a `DataOutputStream`, which is similar to a `FileOutputStream`. The `ServerSocket`



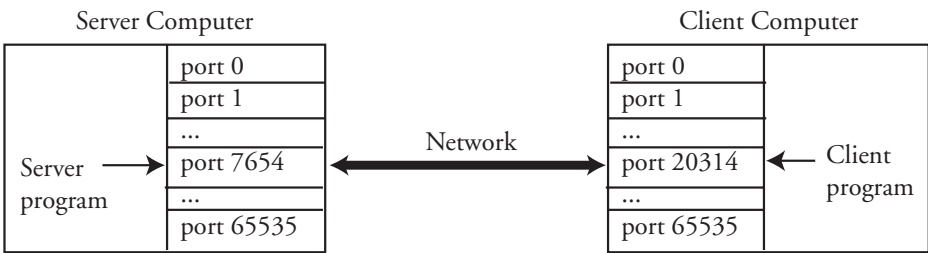
VideoNote  
Networking  
with Streams

Display 19.6 Client/Server Network Communication through Sockets

1. The server listens and waits for a connection on port 7654.



2. The client connects to the server on port 7654. It uses a local port that is assigned automatically, in this case, port 20314.



The server program can now communicate over a socket bound locally to port 7654 and remotely to the client's address at port 20314.

The client program can now communicate over a socket bound locally to port 20314 and remotely to the server's address at port 7654.

and `Socket` classes are in the `java.net` package, while the `BufferedReader` and `DataOutputStream` classes are in the `java.io` package. Once the streams are created, the server expects the client to send a name. The server waits for the name with a call to `readLine()` on the `BufferedReader` object and then sends back the name concatenated with the current date and time. Finally, the server closes the streams and sockets.

Display 19.6 shows how to create a client that connects to our date and time server. First, we create a socket with the name of the computer running the server along with the corresponding port of 7654. If the server program and client program are running on the same computer, then you can use `localhost` as the name of the machine. Your computer understands that any attempt to connect across a network to the machine named `localhost` really corresponds to a connection with itself. Otherwise, the hostname should be set to the name of the computer (e.g., `my.server.com`). After a connection is made, the client creates stream objects, sends its name, waits for a reply, and prints the reply.

`localhost`



Display 19.7 Date and Time Server (part 1 of 2)

---

```
1 import java.util.Date;
2 import java.net.ServerSocket;
3 import java.net.Socket;
4 import java.io.DataOutputStream;
5 import java.io.BufferedReader;
6 import java.io.InputStreamReader;
7 import java.io.IOException;

8 public class DateServer
9 {
10 public static void main(String[] args)
11 {
12 Date now = new Date();

13 try
14 {
15 System.out.println("Waiting for a connection on port 7654.");
16 ServerSocket serverSock = new ServerSocket(7654);
17 Socket connectionSock = serverSock.accept();

18 BufferedReader clientInput = new BufferedReader(
19 new InputStreamReader(connectionSock.getInputStream()));
20 DataOutputStream clientOutput = new DataOutputStream(
21 connectionSock.getOutputStream());

22 System.out.println("Connection made, waiting for client " +
23 "to send their name.");
24 String clientText = clientInput.readLine();
25 String replyText = "Welcome, " + clientText +
26 ", Today is " + now.toString() + "\n";
27 clientOutput.writeBytes(replyText);
28 System.out.println("Sent: " + replyText);

29 clientOutput.close();
30 clientInput.close();
31 connectionSock.close();
32 serverSock.close();
33 }
34 catch (IOException e)
35 {
36 System.out.println(e.getMessage());
37 }
38 }
}
```

## Display 19.7 Date and Time Server (part 2 of 2)

Sample Dialogue      *Output when the client program in Display 19.8 connects to the server program*

```
Waiting for a connection on port 7654.
Connection made, waiting for client to send their name.
Sent: Welcome, Dusty Rhodes, Today is Sun Mar 1 12:18:21 AKDT 2015
```

## Display 19.8 Date and Time Client (part 1 of 2)

```
1 import java.net.Socket;
2 import java.io.DataOutputStream;
3 import java.io.BufferedReader;
4 import java.io.InputStreamReader;
5 import java.io.IOException;

6 public class DateClient
7 {
8 public static void main(String[] args)
9 {
10 try
11 {
12 String hostname = "localhost";
13 int port = 7654;
14
15 System.out.println("Connecting to server on port " + port);
16 Socket connectionSock = new Socket(hostname, port);
17
18 BufferedReader serverInput = new BufferedReader(
19 new InputStreamReader(connectionSock.getInputStream()));
20 DataOutputStream serverOutput = new DataOutputStream(
21 connectionSock.getOutputStream());
22
23 System.out.println("Connection made, sending name.");
24 serverOutput.writeBytes("Dusty Rhodes\n");
25
26 System.out.println("Waiting for reply.");
27 String serverData = serverInput.readLine();
28 System.out.println("Received: " + serverData);
29
30 serverOutput.close();
31 serverInput.close();
32 connectionSock.close();
33 }
34 }
35 }
```

(continued)

## Display 19.8 Date and Time Client (part 2 of 2)

```
29 catch (IOException e)
30 {
31 System.out.println(e.getMessage());
32 }
33 }
34 }
```

## Sample Dialogue

*Output when client program connects to the server program in Display 19.7*

```
Connecting to server on port 7654
Connection made, sending name.
Waiting for reply.
Received: Welcome, Dusty Rhodes, Today is Sun Mar 1 12:18:21 AKDT 2015
```

Note that the socket and stream objects throw checked exceptions. This means that their exceptions must be caught or declared in a `throws` block.

## Sockets and Threading

### blocking

If you run the program in Display 19.7, then you will notice that the server waits, or **blocks**, at the `serverSock.accept()` call until a client connects to it. Both the client and server also block at the `readLine()` call if data from the socket is not yet available. In a client with a GUI, you would notice this as a nonresponsive program while it is waiting for data. For the server, this behavior makes it difficult to handle connections with more than one client. After a connection is made with the first client, the server will become nonresponsive to the client's requests while it waits for a second client.

The solution to this problem is to use threads. One thread will listen for new connections while another thread handles an existing connection. Section 19.1 describes how to create threads and make a GUI program responsive. On the server, the `accept()` call is typically placed in a loop and a new thread is created to handle each client connection:

```
while (true)
{
 Socket connectionSock = serverSock.accept();
 ClientHandler handler = new ClientHandler(connectionSock);
 Thread theThread = new Thread(handler);
 theThread.start();
}
```

In this code, `ClientHandler` is a class that implements `Runnable`. The constructor keeps a reference to the socket in an instance variable, and the `run()` method would handle all communications. A complete implementation of a threaded server is left as Programming Projects 19.7 and 19.8.

## The URL Class

Java's `URL` class will retrieve the HTML from a website into a stream while eliminating several details involved in creating a socket. The `URL` class also illustrates the flexibility of streams and the power of polymorphism. Code that reads from the keyboard or from a file can be used almost verbatim to read from a website; all we need to do is change the source of the stream that is connected to the `Scanner` object. To use the `URL` class, import `java.net.URL`, create a `URL` object, and then use the stream when creating a `Scanner` object. From that point on, reading from the `Scanner` will read data from the `URL` specified in the `URL` object. The following code listing will output the HTML of `www.wikipedia.org`:

```
URL website = new
 URL("http://www.wikipedia.org");
Scanner inputStream = new Scanner(
 new InputStreamReader(
 website.openStream()));

while (inputStream.hasNextLine())
{
 String s = inputStream.nextLine();
 System.out.println(s);
}
inputStream.close();
```

### Self-Test Exercises

5. What is the purpose of a port in the context of a socket?
6. Consider a threaded server that is expected to have up to 100 clients connected to it at one time. Why might this server require a large amount of resources such as memory, disk space, or processor time?

## 19.3 JavaBeans

*Insert tab A into slot B.*

ANONYMOUS

### JavaBeans

**JavaBeans** refers to a framework that facilitates software building by connecting software components from diverse sources. Some of the components might be standard existing pieces of software. Some might be designed for the particular application. Typically, the various components were designed and coded by different teams. If the components are all designed within the JavaBeans framework, it simplifies the process of integrating the components and means that the components produced can more easily be reused for future software projects. JavaBeans have been widely used. For example, the AWT and Swing packages were built within the JavaBeans framework.

## The Component Model

You are most likely to have heard the word *component* when shopping for a home entertainment system. The individual pieces, such as a receiver/amplifier, DVD player, speakers, and so forth, are called *components*. Connect the components to produce a working system, but do not connect them in just any way. You must connect them following the interface rules for each component. The speaker wire must connect to the correct plug, and there had better be a plug for it to connect to. You may think it is obvious that a receiver/amplifier needs to have connections for speakers; it is obvious if the receiver/amplifier design is going to be used to make many identical units for use by many different people. However, if you are only making one receiver/amplifier for one home entertainment system, you might just “open the box” and connect the wire inside. Software systems, unfortunately, are often constructed using the “open the box” approach. The component model says that components should always have well-defined connections for other components—which in our case will be other software components, not speakers—but the idea is the same.

## The JavaBeans Model

A component model specifies how components interact with one another. In the case of JavaBeans, the software components (classes) are required to provide at least the following interface services or abilities:

1. **Rules to Ensure Consistency in Writing Interfaces:**

For example, the rules say, among other things, that the name of an accessor method must begin with `get` and that the name of a mutator method must start with `set`. This same rule has always been a style rule for us in this book, but it was a “should do.” In the JavaBeans framework, it becomes a “must do.” Of course, there are other rules as well. This is just a sample rule.

2. **An Event Handling Model:**

This is essentially the event-handling model we presented for the AWT and Swing. (Remember the AWT and Swing were done within the JavaBeans framework.)

3. **Persistence:**

This means that an object has an identity that extends beyond one session. For example, a `JFrame` of the kind we have seen so far may be used; when you are finished using it, it goes away. The next time you use it, it starts out completely new, born again just as it started before. Persistence means the `JFrame` or other component can retain information about its former use; its state is saved, for example, in a database someplace.

4. **Introspection:**

This is an enhancement of simple accessor and mutator methods. It includes facilities to find out what access to a component is available as well as providing access.

5. **Builder Support:**

These are primarily IDEs (Integrated Development Environments) designed to connect JavaBean components to produce a final application. Some examples are the open source NetBeans, the Eclipse Foundation’s Eclipse, and JetBrains’s IntelliJ IDEA.

event  
handling

persistence

introspection

### What Is a JavaBean?

A **JavaBean** (often called a *JavaBean* component or simply a *Bean*) is a reusable software component (Java class or classes) that satisfies the requirements of the JavaBeans framework and that can be manipulated in an IDE designed for building applications out of Beans.

### What Are Enterprise JavaBeans?

The **Enterprise JavaBean** framework extends the JavaBeans framework to more readily accommodate business applications.

### Self-Test Exercises

7. What is meant by *persistence*?
8. What event-handling model is used with JavaBeans?

## 19.4 Java and Database Connections

*It is a capital mistake to theorize before one has data.*

SIR ARTHUR CONAN DOYLE (*Sherlock Holmes*), *Scandal in Bohemia*, 1891.

As an example of how Java has been extended to interact with other software systems, in this section, we will briefly describe how Java interacts with database management systems. This introduction is just enough for you to construct and manipulate databases at a fairly basic level. The intent of this section is to introduce some common database manipulations to let you know what kinds of things are available to you for database programming in Java.

### Relational Databases

A **database** is a structured collection of data, and the software that manages a database is known as a **database management system (DBMS)**. A DBMS is especially useful when dealing with a large amount of data, because it simplifies the data creation, storage, and retrieval from the perspective of an application. Consequently, a database system is almost universally employed with any large-scale application that requires the storage of information. For example, applications that manage financial transactions, employee records, products, or customer data will typically use a database.

The most common database model used today is the **relational database**, which was defined by Edgar Codd at the IBM Almaden Research Center in 1970. A relational database refers to a collection of relations, which are more commonly referred to as **tables**. A table consists of records that comprise the rows of the table. The fields for

database  
management  
system  
(DBMS)

relational  
database

each record comprise the columns of the table. Tables may be related to one another through their fields, hence the term *relational database*.

## Java DB and JDBC

### Java DB

### Apache Derby

In this section, we will give examples based on Oracle's relational database management system called **Java DB**. Java DB is a version of the open source database known as **Apache Derby**. It is packaged with version 6 or higher of the Java Software Development Kit (SDK). Depending on what version of the Java SDK that is installed on your system, you may not have the complete Derby package (e.g., one that includes documentation and source code examples), so you may wish to download it from Oracle's Web page. Alternately, other database systems (such as Oracle, SQL Server, or MySQL) will also work with Java, but they will require appropriate drivers and minor changes to the source code. These changes are primarily in the specification of the connection string.

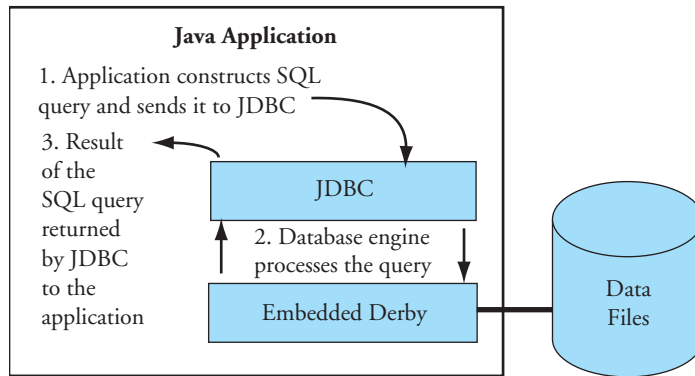
Installing Derby may require some additional configuration of your system. It will require setting several environment variables in your operating system. The `DERBY_INSTALL` variable should be set to the pathname of the folder that contains the installed Derby files, the `CLASSPATH` variable should include `derby.jar`, which is in the `lib` folder of the main Derby folder, and your `PATH` variable should contain the `bin` folder of the main Derby folder. See the documentation that comes with the Derby software for more detailed instructions.

Derby runs in one of two modes: network server mode or embedded mode. In network server mode, the Derby server program stores and processes the data while a client program connects to the server using sockets. Queries, commands, and results are transmitted between the client and server over sockets. This mode allows the database server to run on one machine, while database clients run on separate machines. In embedded mode, the classes for the Derby engine are embedded in your program and executed in the same thread as your application. In this section, we will only use embedded mode, but only minor changes are required to create a program that runs under server mode.

Application programs typically access databases through several layers of abstraction. The lowest level is the database engine, which handles tasks such as indexing, storing, and retrieving data from the disk drive. In our case, these tasks are performed by the Derby engine. A set of higher-level classes provides a connection to Derby along with a consistent interface so that few code changes are necessary at higher levels if the underlying database provider is changed. **Java Database Connectivity (JDBC)** is a common API used to access databases. JDBC is included in version 5 or higher of the Java SDK. Various Microsoft and Oracle database systems are among the many commercially available database systems that are compatible with JDBC, along with Derby. Typically, you need to download and install a JDBC driver for your database system. Conceptually, JDBC is simple: Establish a connection to a database system (either on your computer or over the Internet) and execute database commands, and do this all within your Java code.

Finally, applications send commands and queries to JDBC using strings formatted in **SQL**. SQL is a standard for database access that has been adopted by virtually all

Display 19.9 Relationships between JDBC, Embedded Derby, and SQL



relational database vendors. The initials *SQL* stand for *Structured Query Language*. Display 19.9 illustrates the relationships among these components for an embedded Derby database.

## SQL

SQL is pronounced either by saying the letters or by saying the word “sequel.” SQL is a language for formulating queries for a relational database. SQL is not part of Java, but JDBC allows you to embed SQL commands in your Java code.

SQL works with relational databases. As an example, suppose we were organizing a catalog of books and authors. A relational database can be thought of as a collection of named tables with rows and columns, such as those shown in Display 19.10. In this case, we have created a table with author information (author name, unique author ID, and URL), a table with book information (title and ISBN), and a table that identifies which author has written which book (unique author ID and ISBN).

In this brief introduction, we will not go into the details of the constraints on tables. However, to see that there are some constraints, note that in the three tables in Display 19.10, no relationship is repeated. If we had one entry for each book with all the information—title, author, ISBN number,<sup>1</sup> and author’s URL—then there would be two entries giving Dan Simmons’ URL, because he has two books in our database.

To manipulate the database, we issue SQL commands, also known as SQL queries, to the database. The following is a sample SQL command:

```
SELECT Titles.Title, Titles.ISBN, BooksAuthors.Author_ID
FROM Titles, BooksAuthors
WHERE Titles.ISBN = BooksAuthors.ISBN
```

<sup>1</sup>The ISBN number is a unique identification number assigned to (almost) every book published.



Display 19.10 Relational Database Tables

---

**Names**

| AUTHOR           | AUTHOR_ID | URL         |
|------------------|-----------|-------------|
| Adams, Douglas   | 1         | http:// ... |
| Simmons, Dan     | 2         | http:// ... |
| Stephenson, Neal | 3         | http:// ... |

**Titles**

| TITLE                               | ISBN          |
|-------------------------------------|---------------|
| Snow Crash                          | 0-553-38095-8 |
| Endymion                            | 0-553-57294-6 |
| The Hitchhikers Guide to the Galaxy | 0-671-46149-4 |
| The Rise of Endymion                | 0-553-57298-9 |

**BooksAuthors**

| ISBN          | AUTHOR_ID |
|---------------|-----------|
| 0-553-38095-8 | 3         |
| 0-553-57294-6 | 2         |
| 0-671-46149-4 | 1         |
| 0-553-57298-9 | 2         |

---

Display 19.11 Result of SQL Command in Text

---

**Result**

| TITLE                               | ISBN          | AUTHOR_ID |
|-------------------------------------|---------------|-----------|
| Snow Crash                          | 0-553-38095-8 | 3         |
| Endymion                            | 0-553-57294-6 | 2         |
| The Hitchhikers Guide to the Galaxy | 0-671-46149-4 | 1         |
| The Rise of Endymion                | 0-553-57298-9 | 2         |

---

This will produce the table shown in Display 19.11. That table contains all titles with matching ISBN number and author ID. The ISBN number is the bridge that connects the tables `Titles` and `BooksAuthors`.

As a more detailed example, let us connect to an embedded Derby database and issue the SQL commands to create a new table, insert a new row (record) into the table, select rows, and modify rows. Note that this is not a comprehensive list of commands but rather a small subset of SQL.

First, our Java program must import the SQL libraries `java.sql.Connection`, `java.sql.DriverManager`, `java.sql.SQLException`, and `java.sql.Statement`. When processing results, we will also need to import `java.sql.ResultSet`. Next we must load the database drivers:

```
String driver = "org.apache.derby.jdbc.EmbeddedDriver";
Class.forName(driver).newInstance();
```

**connection  
string**

This code throws three checked exceptions that must be caught: `ClassNotFoundException`, `InstantiationException`, and `IllegalAccessException`. For simplicity, we catch only the superclass `Exception` in the code shown in Display 19.12.

Once the database drivers are loaded, we can connect to the database. This is done by passing a **connection string** to the `DriverManager.getConnection` method. The connection string specifies the protocol, database name, and other parameters, such as whether or not a new database should be created. For example, to connect to and create a database named `BookDatabase` using Derby, we would use

```
Connection conn = null;
conn = DriverManager.getConnection(
 "jdbc:derby:BookDatabase; create = true");
```

This creates a subdirectory named `BookDatabase` in the active working directory that will contain the database, files. You could specify a pathname in front of `BookDatabase` if you want to create the database somewhere else on the file system. If you ever wish to delete the database, then simply delete the `BookDatabase` directory. Note that if the database already exists, then the attribute `create=true` will not delete the existing database but will instead connect to the existing database.

Additional parameters are specified in the command string by separating them with semicolons. For example, if the database requires a username and password, then the connection string would look like this:

```
conn = DriverManager.getConnection
 ("jdbc:derby:BookDatabase;create=true;user=username;" +
 "password = pass");
```

When we are finished accessing the database, invoke the `close( )` method to close the connection. The `DriverManager.getConnection( )` method requires that `SQLException` be caught, so this code should be placed inside an appropriate try/catch block.

SQL commands or queries can be issued to JDBC once the database connection is established. First, a `Statement` object must be constructed and then invoked by calling the `execute` or `executeQuery` method with a SQL string as its argument. The `execute` method can be used to execute any SQL statement, but it is generally used for SQL commands where return values are not needed or ignored (e.g., creating a new table or deleting a row). It returns `true` if the command results in a set of data and `false` if there is no result or an update count. The `executeQuery` method is used with a SQL query that is expected to return some rows from the database. It returns a `ResultSet` object that contains the data produced by the query. These methods throw `SQLException` if there is a database error. Display 19.12 illustrates how to create a new table and insert three rows using the `CREATE TABLE` and `INSERT` commands. The result is identical to the `names` table in Display 19.10. Note that the program should only be run once. If you attempt to run it a second time, the program will throw an exception when executing the `CREATE TABLE` command, because it is invalid to create a new table that matches an existing name.

## Common SQL Statements

SQL Statements are constructed as strings and passed to JDBC. The syntax and examples for the `CREATE TABLE`, `INSERT`, `UPDATE`, and `SELECT` statements follow.

### SYNTAX

|                           |                                                                                                                                                                                                                                                                                     |                                                                                                          |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <code>CREATE TABLE</code> | Create a new table named newtable with fields field1, field2, etc. Data types are similar to Java and include: int, bigint, float, double, and var(size) which is equivalent to a String of maximum length size.                                                                    | <code>CREATE TABLE newtable (field1 datatype, field2 datatype, ...)</code>                               |
| <code>INSERT</code>       | Insert a new row into the table tableName where field1 has the value field1Value, field2 has the value field2Value, etc. The data types for the values must match those for the corresponding fields when the table was created. String values should be enclosed in single quotes. | <code>INSERT INTO tableName VALUES (field1Value, field2Value, ...)</code>                                |
| <code>UPDATE</code>       | Change the specified fields to the new values for any rows that match the WHERE clause. Op is a comparison operator such as =, <> (not equal to), <, >, etc.                                                                                                                        | <code>UPDATE tableName SET field1 = newValue, field2 = newValue, ... WHERE fieldName Op someValue</code> |
| <code>SELECT</code>       | Retrieve the specified fields for the rows that match the WHERE clause. The * may be used to retrieve all fields. Omit the WHERE clause to retrieve all rows from the table.                                                                                                        | <code>SELECT field1, field2 FROM tableName WHERE fieldname Op someValue</code>                           |

### EXAMPLE

```
CREATE TABLE names(author varchar(50), author_id int, url
varchar(80))
INSERT INTO names VALUES ('Adams, Douglas', 1, 'http://
www.douglasadams.com')
UPDATE names SET url = 'http://www.douglasadams.com/dna/bio.html'
WHERE author_id = 1
SELECT author, author_id, url FROM names
SELECT author, author_id, url FROM names WHERE author_id > 1
```

## Display 19.12 Creating a Derby Embedded Database and Table (part 1 of 2)

```

1 import java.sql.Connection;
2 import java.sql.DriverManager;
3 import java.sql.SQLException;
4 import java.sql.Statement;

5 public class CreateDB
6 {
7 private static final String driver =
8 "org.apache.derby.jdbc.EmbeddedDriver";
9 private static final String protocol = "jdbc:derby:";
10 public static void main(String[] args)
11 {
12 try  Load the embedded Derby driver.
13 {
14 Class.forName(driver).newInstance();
15 System.out.println("Loaded the embedded driver.");
16 }
17 catch (Exception err)  Must catch ClassNotFoundException,
18 InstantiationException, Illegal
19 AccessException
20 {
21 System.err.println("Unable to load the embedded driver.");
22 err.printStackTrace(System.err);
23 System.exit(0);
24 }

25 String dbName = "BookDatabase";
26 Connection conn = null;
27 try
28 {
29 System.out.println(
30 "Connecting to and creating the database...");
31 conn = DriverManager.getConnection(protocol + dbName +
32 ";create=true");
33 System.out.println("Database created.");
34 Statement s = conn.createStatement();  Create a statement object
35 to run SQL statements.
36 s.execute("CREATE TABLE names" +
37 " (author varchar(50), author_id " +
38 "int, url varchar(80))");  Create a table called
39 "names" with three
40 fields, 50 characters
41 for an author and an
42 integer author ID,
43 and 80 characters
44 for a URL, then insert
45 sample data.
46 System.out.println("Created 'names' table.");
47 System.out.println("Inserting authors.");
48 s.execute("INSERT INTO names " +
49 "VALUES ('Adams, Douglas', 1, " +
50 "'http://www.douglasadams.com')");

```

(continued)

## Display 19.12 Creating a Derby Embedded Database and Table (part 2 of 2)

```
36 s.execute("INSERT INTO names " +
37 "VALUES ('Simmons, Dan', 2, 'http://www.dansimmons.com')");
38 s.execute("INSERT INTO names " +
39 "VALUES ('Stephenson, Neal', 3, " +
40 "'http://www.nealstephenson.com')");
41 System.out.println("Authors inserted.");
42 conn.close();
43 }
44 catch (SQLException err)
45 {
46 System.err.println("SQL error.");
47 err.printStackTrace(System.err);
48 System.exit(0);
49 }
50 }
```

## Sample Dialogue

```
Loaded the embedded driver.
Connecting to and creating the database.
Database created.
Created 'names' table.
Inserting authors.
Authors inserted.
```

Display 19.13 shows how to retrieve rows from an existing database and table using the `SELECT` statement. The syntax of the `SELECT` statement is given in the box “Common SQL Statements.” The desired fields must be specified or an `*` placed after the `SELECT` statement to retrieve all fields. After specifying the name of the table, an optional `WHERE` clause may be inserted that contains conditions that must be met, much like the Boolean condition you might place after an `if` statement. If the `WHERE` clause is left off, then all rows from the table will be retrieved. Otherwise, only those rows that match the conditions of the `WHERE` clause are returned.

A `SELECT` statement can be executed by invoking the `executeQuery( )` method of a `Statement` object. The return value is an object of type `ResultSet`. The `ResultSet` object maintains a cursor to each matching row in the database. Initially, the cursor is positioned before the first row. The `next( )` method is used to advance the cursor to the next row. If there is no next row, then `false` is returned. Otherwise, `true` is returned. Typically, a `while` loop is used to iterate over all rows in the set by looping until the `next( )` method returns `false`. Note that iteration is forward

only, similar to reading data from an input stream. Once the cursor is positioned over a row, we can use one of following methods to retrieve data from a specific column in the current row:

```
intVal = resultSet.getInt("name of int field");
lngVal = resultSet.getLong("name of bigint field");
strVal = resultSet.getString("name of varchar field");
dblVal = resultSet.getDouble("name of double field");
fltVal = resultSet.getFloat("name of float field");
```

The program in Display 19.13 can run once the database is created with the program given in Display 19.12. It retrieves and outputs all rows in the table by executing a `SELECT` query with no `WHERE` clause and then retrieves and outputs only those rows with any `author_id` greater than 1. Invoke the `close()` method to free resources when finished using a `ResultSet` object.

Finally, we can change the contents of an existing row using the SQL `UPDATE` command. In the `UPDATE` command, we specify the table, field(s) to change, new value(s), and a `WHERE` clause to indicate which rows should be changed. If the `WHERE` clause is omitted, then every row in the table is updated. The syntax is described in the box “Common SQL Statements.” Display 19.14 shows an example where the `URL` field is changed to a new value entered by the user. The `URL` field is only changed for the row that matches the `author_id` entered by the user.

Display 19.13 Retrieving Rows with the `SELECT` Statement (part 1 of 3)

```
1 import java.sql.Connection;
2 import java.sql.DriverManager;
3 import java.sql.ResultSet;
4 import java.sql.SQLException;
5 import java.sql.Statement;

6 public class ReadDB
7 {
8 private static final String driver =
9 "org.apache.derby.jdbc.EmbeddedDriver";
10 private static final String protocol = "jdbc:derby:";

11 /*
12 * Outputs the author, ID, and URL of the current
13 * author in the ResultSet
14 */
15 public static void displayNameRow(ResultSet rs) throws SQLException
16 {
17 int id = rs.getInt("author_id");
18 String author = rs.getString("author");
19 String url = rs.getString("url");
```

*The database must be created (Display 19.12) before running the program.*

*The accessor methods throw the checked exception SQLException.*

(continued)

Display 19.13 Retrieving Rows with the **SELECT** Statement (part 2 of 3)

```

19 System.out.println("ID = " + id + ", Author = "
20 + author + ", URL = " + url);
21 }

22 public static void main(String[] args)
23 {
24 try
25 {
26 Class.forName(driver).newInstance();
27 System.out.println("Loaded the embedded driver.");
28 }
29 catch (Exception err)
30 {
31 System.err.println("Unable to load the embedded driver.");
32 err.printStackTrace(System.err);
33 System.exit(0);
34 }

35 String dbName = "BookDatabase";
36 Connection conn = null;
37 try
38 {
39 System.out.println("Connecting to the database...");
40 conn = DriverManager.getConnection(protocol + dbName);
41 System.out.println("Connected.");
42 Statement s = conn.createStatement();

43 ResultSet rs = null;
44 System.out.println("All rows:");
45 rs = s.executeQuery("SELECT author, author_id, url FROM names");
46 while(rs.next())
47 {
48 displayNameRow(rs);
49 }
50 rs.close();

51 System.out.println();
52 System.out.println("All rows with an ID > 1:");
53 rs = s.executeQuery("SELECT author, author_id, url " +
54 "FROM names WHERE author_id > 1");
55 while(rs.next())
56 {
57 displayNameRow(rs);
58 }

```

*The text "create=true;" has been left off the connection string to connect to an existing database.*

Display 19.13 Retrieving Rows with the **SELECT** Statement (part 3 of 3)

```

59 rs.close();
60 conn.close();
61 }
62 catch (SQLException err)
63 {
64 System.err.println("SQL error.");
65 err.printStackTrace(System.err);
66 System.exit(0);
67 }
68 }
69 }

```

## Sample Dialogue

```

Loaded the embedded driver.
Connecting to the database.
Connected.
All rows:
ID = 1, Author = Adams, Douglas, URL = http://www.douglasadams.com
ID = 2, Author = Simmons, Dan, URL = http://www.dansimmons.com
ID = 3, Author = Stephenson, Neal, URL = http://www.nealstephenson.com

All rows with an ID > 1:
ID = 2, Author = Simmons, Dan, URL = http://www.dansimmons.com
ID = 3, Author = Stephenson, Neal, URL = http://www.nealstephenson.com

```

Display 19.14 Updating Rows with the **UPDATE** Statement (part 1 of 3)

```

1 import java.sql.Connection;
2 import java.sql.DriverManager;
3 import java.sql.ResultSet;
4 import java.sql.SQLException;
5 import java.sql.Statement;
6 import java.util.Scanner;

7 public class UpdateDB
8 {
9 private static final String driver =
10 "org.apache.derby.jdbc.EmbeddedDriver";
11 private static final String protocol = "jdbc:derby:";

```

*The database must be created (Display 19.12) before running this program. Use the program in Display 19.13 to view the changes made by this program.*

(continued)



Display 19.14 Updating Rows with the UPDATE Statement (part 2 of 3)

```
11 public static void main(String[] args)
12 {
13 try
14 {
15 Class.forName(driver).newInstance();
16 System.out.println("Loaded the embedded driver.");
17 }
18 catch (Exception err)
19 {
20 System.err.println("Unable to load the embedded driver.");
21 err.printStackTrace(System.err);
22 System.exit(0);
23 }
24 String dbName = "BookDatabase";
25 Connection conn = null;
26 try
27 {
28 System.out.println("Connecting to the database...");
29 conn = DriverManager.getConnection(protocol + dbName);
30 System.out.println("Connected.");
31
32 System.out.println(
33 "Enter the ID number of the author to change:");
34 Scanner scan = new Scanner(System.in);
35 int id = scan.nextInt();  Skips the newline left after nextInt()
36 scan.nextLine();
37 System.out.println("Enter the new URL for this author.");
38 String newURL = scan.nextLine();
39
40 Statement s = conn.createStatement();
41 s.execute("UPDATE names " +
42 "SET URL = '" + newURL + "' WHERE author_id = " +
43 id);
44 System.out.println("URL changed to " + newURL);
45 conn.close();
46 }
47 catch (SQLException err)
48 {
49 System.err.println("SQL error.");
50 err.printStackTrace(System.err);
51 System.exit(0);
52 }
53 }
```

---

Display 19.14 Updating Rows with the `UPDATE` Statement (part 3 of 3)

---

## Sample Dialogue

```
Loaded the embedded driver.
Connecting to the database...
Connected.
Enter the ID number of the author to change:
2
Enter the new URL for this author:
http://www.dansimmons.com/about/bio.htm
URL changed to http://www.dansimmons.com/about/bio.htm
```

---

There is much more to SQL and JDBC than what we have discussed here. However, this section should give you a good idea about how to integrate SQL into a Java application and provide a starting point to learn more.

### Self-Test Exercises

9. Give the SQL `SELECT` command to produce a table of book titles with corresponding author and author ID from the table `Result` in Display 19.11 and one of the tables in Display 19.10. Follow the example of the SQL command in the text used to produce the `Result` table.
10. What is a connection string?
11. What is the difference between the `execute( )` and `executeQuery( )` methods of the `JDBC Statement` class?
12. Give the SQL statement to create the `BooksAuthors` table shown in Display 19.10.
13. Give the SQL statement to insert the four entries shown in Display 19.10 into the `BooksAuthors` table.

## 19.5 Web Programming with Java Server Pages

*Everything is connected... no one thing can change by itself.*

PAUL HAWKEN

Up to this point, we have used Java to create stand-alone applications. However, Java is also used to create interactive websites. In this section, we briefly introduce ways that Java can be used on the Web with an emphasis on Java Server Pages. This section requires a basic understanding of HTML. An introduction to HTML is given in Chapter 20, which is included on the website with this book. This section is not a complete enough introduction to allow you to immediately start writing Java Web applications. The intent is to introduce the major concepts behind Java Server Pages so you can learn what kinds of things are possible should you wish to learn more with a book or other resource dedicated to the topic.

### Applets, Servlets, and Java Server Pages

When you instruct your Web browser to view a page from a Web server on the Internet, your Web browser requests the page from the Web server, the Web server processes the request (which may involve reading the requested page from a file on the hard drive), and then the Web server sends the requested page to your Web browser. Your Web browser formats, or renders, the received data to fit on your computer screen. This interaction is a specific case of the client/server model described in Section 19.2. Your Web browser is the client program, your computer is the client computer, the remote website is the server computer (e.g., <http://www.remotesite.com>), and the Web server software running on the remote website is the server program.

In the context of a Web application, the client/server model is important because Java code can run in two places: on the client or on the server. There are trade-offs to both approaches. Server-based programs have easy access to information that resides on the server, such as customer orders or inventory data. Because all of the computation is done on the server and results are transmitted to the client as HTML, a client does not need a powerful computer to run a server-based program. On the other hand, a client-based program may require a more powerful client computer, because all computation is performed locally. However, richer interaction is possible, because the client program has access to local resources, such as the graphics display (e.g., perhaps using Swing) or the operating system. Many systems today are constructed using code that runs on both the client and the server to reap the benefit of both approaches.

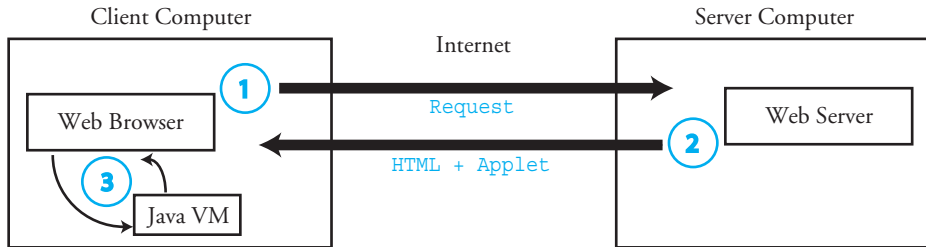
Web applications built with Java include **Java applets**, **Java servlets**, and **Java Server Pages (JSP)**. Java applets run on the client computer and are discussed in Chapter 20. JavaScript, which is a different language than Java despite its similar name, also runs on the client computer as part of the Web browser. Java servlets and Java Server Pages run on the server. In this chapter, we focus primarily on Java Server Pages, which are a dynamic version of Java servlets. Servlets must be compiled before they can run, just like a normal Java program. In contrast, JSP code is embedded with

Java applet

Java servlet

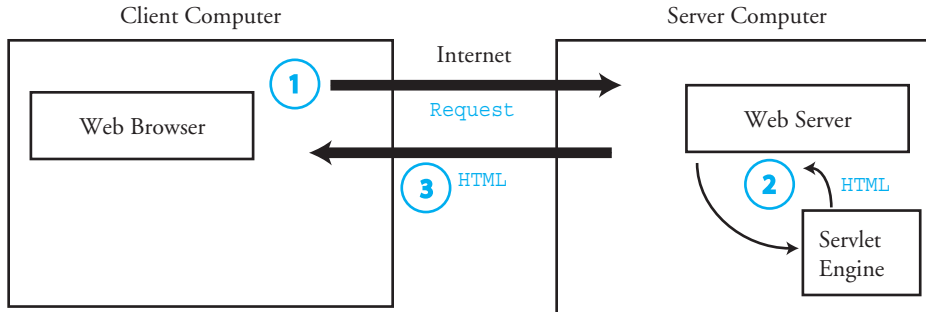
Java Server  
Pages

Display 19.15 Running a Java Applet



- 1 The client's Web browser sends a request to the server for a Web page with a Java applet.
- 2 The server sends the HTML for the Web page and applet class files to the client.
- 3 The client runs the applet using the Java Virtual Machine and displays its output in the Web browser.

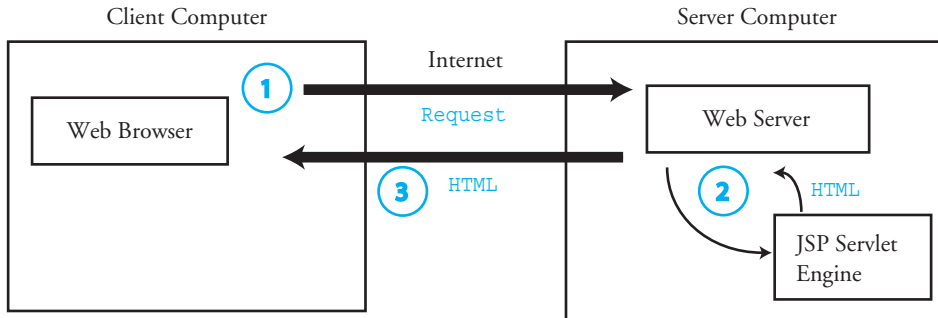
Display 19.16 Running a Java Servlet



- 1 The client's Web browser sends a request to the server for a Web page that runs a Java servlet.
- 2 The Web server instructs the servlet engine to execute the requested servlet, which consists of running precompiled Java code. The servlet outputs HTML that is returned to the Web server.
- 3 The Web server sends the servlet's HTML to the client's Web browser to be displayed.

the corresponding HTML and is compiled “on the fly” into a servlet when the page is requested. This flexibility can make it easier to develop Web applications using JSP than with Java servlets. Displays 19.15, 19.16, and 19.17 illustrate the differences for a website that runs an applet, servlet, or JSP.

Display 19.17 Running a Java Server Page (JSP) Program



- 1 The client's Web browser sends a request to the server for a Web page that contains JSP code.
- 2 The JSP servlet engine dynamically compiles the JSP source code into a Java servlet if a current, compiled servlet doesn't exist. The servlet runs and outputs HTML that is returned to the Web server.
- 3 The Web server sends the servlet's HTML to the client's Web browser to be displayed.

## Oracle GlassFish Enterprise Server

JSP requires a Web server with a JSP servlet engine. In this section, we will use the Oracle GlassFish Enterprise Server, previously known as the Sun Java System Application Server. It is distributed by Oracle as part of the Java Enterprise Edition SDK. However, you may use any Web server that supports Java servlets and JSP. Another popular choice is the open source Apache Tomcat server. Follow the installation instructions that come with the server that you select.

The GlassFish server will ask you to select an administrator username and password during the installation procedure. It will also ask you for a location on the hard drive to place the server files. We will refer to the pathname you select as `<glassfish_home>`. On a Windows machine, this may default to `C:\Sun\SDK`. After the installation is complete, start the GlassFish Web service. If the installation is successful, then you should be able to see the administrator's page by opening `http://localhost:4848` in your Web browser. The default URL for accessing applications is `http://localhost:8080`. Substitute `localhost` with the name of the machine if you are using a remote server.

At this point, you can test your server by creating or copying an HTML file into `<glassfish_home>\domains\domain1\docroot` and opening it with your Web browser. For example, if you create an HTML file named `test.html` in this directory, then you should be able to access it with your Web browser by navigating to `http://localhost:8080/test.html`. If you cannot read the page, check your configuration settings. To run JSP programs, your Web browser must open the page through the GlassFish Web server and not directly load the page from the HTML file on the disk drive.

## HTML Forms—the Common Gateway Interface

### HTML form

### Common Gateway Interface

The HTML form is a common mechanism for users to input data to Web applications. If you have ever visited a Web page in which you enter data using textboxes and buttons, then you have used an HTML form. In this section, we introduce just a handful of tags that can be used in an HTML form and then show how submitted data can be processed by a JSP program. The information you enter into an HTML form is transmitted to the Web server using a protocol called the **Common Gateway Interface (CGI)**. The server processes the information using a program such as a Java servlet and then returns the results back to the user.

An HTML form is created with the `<form>` tag. The syntax is as follows:

```
<FORM ACTION="Path_To_CGI_Program" METHOD="GET or POST">
 Form_Elements
</FORM>
```

The `ACTION` string identifies the program that will process the form input. In our case, this will consist of a JSP program. The `METHOD` string specifies how data will be sent to the server and is either `GET` or `POST`. `GET` means that the data will be sent as part of the URL while `POST` means that the data will be sent as part of the data and will not be visible in the URL.

Elements that can be inserted inside the form include selection lists, textboxes, checkboxes, radio buttons, and several other common GUI widgets. In this section, we will only introduce textboxes and submission buttons. A textbox has the following syntax:

```
<INPUT TYPE="TEXT" NAME="Textbox_Name" VALUE="Default_Text"
 SIZE="Length_In_Characters"
 MAXLENGTH="Maximum_Number_Of_Allowable_Characters">
```

All of the attributes are optional except for the input type. However, the name of the textbox will be required to retrieve the textbox's value from JSP. If you would like a default value to be entered in the textbox, you can put it in the `VALUE` field or leave it blank for no text. The `SIZE` field controls the length of the textbox while `MAXLENGTH` limits the number of characters that may be entered.

The submission button has the following syntax:

```
<INPUT TYPE="SUBMIT" NAME="Name" VALUE="Button_Text">
```

The `NAME` field can be used to identify the submission button in case there are multiple submission buttons and you wish to identify which one was clicked. `VALUE` determines what text is placed in the button's label.

A sample form with two text fields and a submission button is shown in Display 19.18. The form, when opened in a Web browser, is shown in Display 19.19. It asks the user to enter an Author ID and a new URL with the intent of changing the URL to the submitted value for the specified Author ID. Because we have not created a JSP program that processes the form, the Web browser will display an error message if the Submit button is clicked. We will write the JSP program in the next subsection.

Display 19.18 An HTML Form Document

```
<html>
<head>
<title>Change Author's URL</title>
</head>
```

```
<body>
<h1>Change Author's URL</h1>
<p>
Enter the ID of the author you would like to change
along with the new URL.
</p>
```

```
<form ACTION = "EditURL.jsp" METHOD = POST>
```

Author ID:

```
<input TYPE = "TEXT" NAME = "AuthorID"
VALUE = "" SIZE = "4" MAXLENGTH = "4">
```

```


```

New URL:

```
<input TYPE = "TEXT" NAME = "URL"
VALUE = "http://" SIZE = "40" MAXLENGTH = "200">
```

```
<p>
INPUT TYPE="SUBMIT" VALUE="Submit">
```

```
</p>
```

```
</form>
```

```
</body>
```

```
</html>
```

*Invokes the JSP program named `EditURL.jsp`. If this program does not exist, you will see an error message upon clicking the Submit button. `EditURL.jsp` is given in the next subsection.*

*Creates a textbox named `AuthorID` that is empty, displays four characters at once, and accepts at most, four characters*

*Creates a textbox named `URL` that by default contains `"http://"`, displays 40 characters at once, and accepts at most 200 characters*

*Creates a Submit button*

Display 19.19 Browser View of Display 19.18

## Change Author's URL

Enter the ID of the author you would like to change along with the new URL.

Author ID:

New URL:

## JSP Declarations, Expressions, Scriptlets, and Directives

A JSP Web page is created the same way you make an HTML file, except JSP code is added along with the HTML code. Additionally, instead of naming the file with an extension of `.HTM` or `.HTML`, the extension is `.JSP`. The file should be placed in the root folder of your Web server so it can be accessed by and processed through the Web server.

### JSP elements declaration

The JSP elements we will briefly discuss are **declarations**, **expressions**, **scriptlets**, and **directives**. All of these elements are identified by their own tags. The **declarations** tag allows us to define variables and methods. The variables and methods are accessible from any scriptlets and expressions on the same page. Variable declarations are compiled as instance variables for a class that corresponds to the JSP page. Declarations are defined with the syntax

```
<%!
 Declarations
%>
```

For example, the following defines an instance variable named `count` and a method named `incrementCount` that increments the `count` variable:

```
<%!
 private int count = 0;

 private void incrementCount()
 {
 count++;
 }
%>
```

### expression

We can access variables defined in declarations with an **expression**. The syntax to embed an expression is as follows:

```
<%=
 Expression
%>
```

Expressions are embedded directly into the HTML. The Web browser will display the value of the expression in place of the tag. For example, we can output the value of the `count` variable in bold type with the following piece of HTML:

```
The value of count is <%= count %>
```

### scriptlet

Blocks of Java code can be embedded in a **scriptlet**. The syntax for a scriptlet is as follows:

```
<%
 Java code
%>
```

If you wish to output HTML within a scriptlet, then this is done using `out.println()`, which is used in the same manner as `System.out.println()`. The variable `out` is already defined for us and is of type `javax.servlet.jsp.JspWriter`. Also note that



`System.out.println( )` will output to the console, which is useful for debugging purposes, while `out.println( )` will output to the browser. The following scriptlet invokes the `incrementCount( )` method and then outputs the value in `count`:

```
<%
 out.println("The counter's value is " + count + "
");
 incrementCount();
%>
```

Display 19.20 is a JSP page with a declaration, expression, and scriptlet that output text inside a header tag from levels 1 to 6. The identifier `LASTLEVEL` is declared as the last heading level that is to be displayed. `LASTLEVEL` is modified by `static final`, because it is intended as a constant. A loop inside the scriptlet outputs sample text for each level. The HTML that is generated by the JSP page is shown in Display 19.21, and the browser view is shown in Display 19.22.

Display 19.20 JSP Code to Display Heading Levels

---

```

1 <html>
2 <title>
3 Displaying Heading Tags with JSP
4 </title>

5 <body>
6 <%!
7 private static final int LASTLEVEL = 6;
8 %>

9 <p>
10 This page uses JSP to display Heading Tags from
11 Level 1 to Level <%= LASTLEVEL %>
12 </p>

13 <%
14 int i;
15 for (i = 1; i <= LASTLEVEL; i++)
16 {
17 out.println("<H" + i + ">" +
18 "This text is in Heading Level " + i +
19 "</H" + i + ">");
20 }
21 %>

22 </body>
23 </html>
```

---

*JSP declaration* (points to line 7)

*JSP expression that evaluates to the value 6* (points to line 11)

*JSP scriptlet that contains a block of Java code* (points to line 13)

---

**Display 19.21 HTML Generated by JSP in Display 19.20**

---

```
<html>
<title>
Displaying Heading Tags with JSP
</title>

<body>

<p>
This page uses JSP to display Heading Tags from
Level 1 to Level 6
</p>

<H1>This text is in Heading Level 1</H1>
<H2>This text is in Heading Level 2</H2>
<H3>This text is in Heading Level 3</H3>
<H4>This text is in Heading Level 4</H4>
<H5>This text is in Heading Level 5</H5>
<H6>This text is in Heading Level 6</H6>

</body>
</html>
```

---

---

**Display 19.22 Browser View of Display 19.21**

---

This page uses JSP to display Heading Tags from Level 1 to Level 6

**This text is in Heading Level 1**

**This text is in Heading Level 2**

**This text is in Heading Level 3**

**This text is in Heading Level 4**

**This text is in Heading Level 5**

**This text is in Heading Level 6**

---

To make a JSP page more interactive, we can read and process the data entered in an HTML form. One way to read these values is to call the `request.getParameter` method. This method takes a `String` parameter as input that identifies the name of an HTML form element and returns the value entered by the user for that element on the form. For example, if there is a textbox named `AuthorID`, then we can retrieve the value entered in that textbox with the following scriptlet code:

```
String value = request.getParameter("AuthorID");
```

If the user leaves the field blank, then `getParameter` returns an empty string. A simple example is given in Display 19.23. This JSP program echoes back the data entered by the user in Display 19.18. The name of the JSP file must match the value supplied for the `ACTION` tag of the form. In this case, the name is `EditURL.jsp`.

Display 19.23 Echoing Values Submitted by a Browser Viewing Display 19.18

```
<html>
<title>Edit URL: Echo submitted values</title>
<body>
<h2>Edit URL</h2>
```

*This program should be saved as `EditURL.jsp` and must match the value in the `ACTION` field of the HTML form tag.*

```
<p>
This version of EditURL.jsp simply echoes back to the
user the values that were entered in the textboxes.
</p>
```

*The `getParameter` method calls return as Strings the values entered by the user in the URL and AuthorID textboxes from Display 19.18.*

```
<%
String url = request.getParameter("URL");
String stringID = request.getParameter("AuthorID");
int author_id = Integer.parseInt(stringID);
out.println("The submitted author ID is: " + author_id);
out.println("
");
out.println("The submitted URL is: " + url);
%>
</body>
</html>
```

Sample Dialogue

*Submitted on the Web browser when viewing Display 19.18*

```
Author ID:
2
New URL:
http://www.dansimmons.com/about/bio.htm

Edit URL Web browser display after clicking Submit
```

This version of `EditURL.jsp` simply echoes back to the user the values that were entered in the textboxes.

```
The submitted author ID is: 2
The submitted URL is:
http://www.dansimmons.com/about/bio.htm
```

**directive**

Finally, let us introduce one more JSP tag, the **directive**. In general terms, directives instruct the compiler how to process a JSP program. Examples include the definition of our own tags, including the source code of other files, and importing packages. The syntax for directives is as follows:

```
<%@
 Directives
%>
```

**page import**

In this introduction, we will cover only the **page import** directive. The purpose of this directive is the same as the normal Java `import` statement, but the syntax is slightly different. First, we identify the directive and then specify the packages to import inside a string. Multiple packages are separated by a comma, e.g.,

```
<%@
 page import="java.util.*,java.sql.*"
%>
```

A JSP program that uses Derby to create an embedded database is shown in Display 19.24. The page import directive is used to import the necessary SQL packages. The program in Display 19.24 is almost identical to the stand-alone program in Display 19.12, except all of the code has been moved into a scriptlet. If the JSP program is in a file named `CreateDB.jsp`, then it would be invoked by navigating to `http://localhost:8080/CreateDB.jsp`. The database files on a GlassFish server will be created in `<glassfish_home>\domains\domain1\config` if you do not specify a pathname. If you run the program more than once, then it will throw an exception the second time, because the database table names already exists and cannot be created again. If you wish to start over with a new database, then you can delete the `BookDatabase` directory in the `<glassfish_home>\domains\domain1\config` directory.

Display 19.24 JSP Program to Create a Derby Database and Table (part 1 of 3)

```
1 <%@ page import="java.sql.*" %>
2 <html>
3 <title>Create New Database</title>
4 <body>
5 <H2>Create New Database</h2>

6 <p>
7 This program creates a new Derby database named 'BookDatabase'
8 and puts sample data into the 'person' table.
9 </p>

10 <%
11 String driver = "org.apache.derby.jdbc.EmbeddedDriver";
12 String protocol = "jdbc:derby:";
```

*Directive to import the Java SQL packages*

*Refer to Display 19.12 for an explanation of the database code in this scriptlet.*

(continued)

Display 19.24 JSP Program to Create a Derby Database and Table (part 2 of 3)

```

13 try
14 {
15 Class.forName(driver).newInstance();
16 out.println("Loaded the embedded driver.
");
17 }
18 catch (Exception err)
19 {
20 out.println(
21 "Unable to load the embedded driver.</body></html>");
22 return;
23
24 String dbName = "BookDatabase";
25 Connection conn = null;
26 try
27 {
28 out.println(
29 "Connecting to and creating the database...
");
30 conn = DriverManager.getConnection(protocol + dbName +
31 ";create=true");
32 out.println("Connected.
");
33
34 Statement s = conn.createStatement();
35
36 s.execute("CREATE TABLE names " +
37 "(author varchar(50), author_id int, url varchar(80))");
38 out.println("Created 'names' table.
");
39
40 out.println("Inserting authors.
");
41 s.execute("INSERT INTO names " +
42 "VALUES ('Adams, Douglas', 1, 'http://www.douglasadams.com')");
43 s.execute("INSERT INTO names " +
44 "VALUES ('Simmons, Dan', 2, 'http://www.dansimmons.com')");
45 s.execute("INSERT INTO names " +
46 "VALUES ('Stephenson, Neal', 3, " +
47 "'http://www.nealstephenson.com')");
48 out.println("Authors inserted.
");
49
50 conn.close();
51 }
52 catch (Exception err)
53 {
54 out.println("SQL error.
");
55 }
56 %>
57 </body>
58 </html>

```

*A return statement will terminate the JSP program.*

## Display 19.24 JSP Program to Create a Derby Database and Table (part 3 of 3)

## Sample Dialogue

**Create New Database**

This program creates a new Derby database named 'BookDatabase' and puts sample data into the 'names' table.

Loaded the embedded driver.

Connecting to and creating the database...

Connected.

Created 'names' table.

Inserting authors.

Authors inserted.

In this second part, to complete our example, let us modify the `EditURL.jsp` program from Display 19.23 to make it change the contents of the database instead of echoing back values submitted by the HTML form. Our new program will require running the program in Display 19.24 once in order to create and initialize the database. Once this is done, you can open the HTML form in Display 19.18 in a Web browser, which prompts the user to submit a new URL for an author ID, and then the new URL will be updated in the database by the program in Display 19.25. The program in Display 19.25 is a combination of Display 19.14 (updating a Derby database) and Display 19.23 (echoing values submitted by a Web browser). Instead of prompting the user to input an author ID and new URL from the console as in Display 19.14, these values are submitted to the JSP program by the HTML form in Display 19.18. The JSP program then updates the database to the submitted values in the same manner as the program in Display 19.14.

## Display 19.25 JSP Program to Update Database Entries Submitted by a Browser Viewing Display 19.18 (part 1 of 3)

```

1 <%@ page import="java.sql.*" %>
2 <html>
3 <title>Edit URL: Update new URL in a database</title>
4 <body>
5 <h2>Edit URL</h2>

6 <p>
7 This version of EditURL.jsp updates the URL field
8 of a Derby database to the submitted value for the
9 row with a matching Author ID.
10 </p>

11 <%
12 String newURL = request.getParameter("URL");
13 String stringID = request.getParameter("AuthorID");
14 int author_id = Integer.parseInt(stringID);

```

*This program should be saved as `EditURL.jsp` and must match the value in the ACTION field of the HTML form tag.*

*The `getParameter` method calls return as Strings the values entered by the user in the URL and AuthorID textboxes from Display 19.18.*

(continued)

Display 19.25 JSP Program to Update Database Entries Submitted by a Browser Viewing  
Display 19.18 (part 2 of 3)

```

15 String driver = "org.apache.derby.jdbc.EmbeddedDriver";
16 String protocol = "jdbc:derby:";
17 try
18 {
19 Class.forName(driver).newInstance();
20 out.println("Loaded the embedded driver.
");
21 }
22 catch (Exception err)
23 {
24 out.println("Unable to load the embedded driver.</body></html>");
25 return;
26 }

27 String dbName = "BookDatabase";
28 Connection conn = null;
29 try
30 {
31 out.println("Connecting to and creating the database...
");
32 conn = DriverManager.getConnection(protocol + dbName +
33 ";create=true");
34 out.println("Connected.
");

35 Statement s = conn.createStatement();
36 s.execute("UPDATE names " +
37 "SET URL = '" + newURL + "' WHERE author_id = " + author_id);
38 out.println("
URL changed to " + newURL +
39 "for Author ID = " + author_id + "
");

40 out.println("
Displaying all rows:
");
41 ResultSet rs = null;
42 rs = s.executeQuery("SELECT author, author_id, url FROM names");
43 out.println("");
44 while(rs.next())
45 {
46 int id = rs.getInt("author_id");
47 String author = rs.getString("author");
48 String url = rs.getString("url");
49 out.println("ID = " + id + ", Author = "
50 + author + ", URL = " + url + "");
51 }
52 out.println("");
53 rs.close();

54 conn.close();
55 }
56 catch (Exception err)
57 {
58 out.println("SQL error.
");
59 }

```

*Refer to Displays 19.13 and 19.14 for an explanation of the database code in this scriptlet.*

*The URL in the database is changed to the submitted value if the Author IDs match.*

*This loop outputs all rows in the database inside a numbered list.*

Display 19.25 JSP Program to Update Database Entries Submitted by a Browser Viewing  
Display 19.18 (part 3 of 3)

---

```
59 %>
60 </body>
61 </html>
```

Sample Dialogue

*Submitted on the Web browser when viewing Display 19.18*

```
Author ID:
2
New URL:
http://www.dansimmons.com/about/bio.htm
Edit URL Web browser display after clicking Submit

This version of EditURL.jsp updates the URL field of a Derby database to
the submitted value for the row with a matching Author ID.

Loaded the embedded driver.
Connecting to and creating the database...
Connected.

URL changed to http://www.dansimmons.com/about/bio.htm for Author ID = 2

Displaying all rows:
1. ID = 1, Author = Adams, Douglas, URL = http://www.douglasadams.com
2. ID = 2, Author = Simmons, Dan, URL = http://www.dansimmons.com/about/
 bio.htm
3. ID = 3, Author = Stephenson, Neal, URL = http://www.nealstephenson.com
```

---

Although we have covered enough JSP to write fairly sophisticated programs, there is much more that we have not discussed. For example, beans can be used as a convenient way to encapsulate data submitted from a HTML form. Additionally, we have not covered sessions, tag libraries, security, and numerous other topics that are important in the construction of JSP pages. In particular, the technique of generating SQL read and write queries based on user-entered values is not secure—a malicious user could enter values that potentially run arbitrary SQL statements. A more secure solution is to use a precompiled SQL statement that is supported by the `java.sql.PreparedStatement` class. Refer to a textbook dedicated to JSP or database programming to learn more.



### Self-Test Exercises

14. What is a major difference between a website implemented with a Java applet and a website implemented with Java Server Pages?
15. Give the HTML to create a form with two elements: a textbox named `FirstName` that holds a maximum of 50 characters, and a Submit button. The form should submit its data to a JSP program called `ProcessName.jsp` using the `POST` method.
16. Identify the following JSP tags: `<% %>`, `<%@ %>`, `<%! %>`, `<%= %>`
17. Write a JSP scriptlet that handles the form created in Self-Test Exercise 13 and outputs the name in a bold heading font.

## 19.6 Introduction to Functional Programming in Java 8



VideoNote  
Functional  
Programming  
Example

Java 8 is the current version of Java that was released in March 2014. While there are many new features in Java 8, the core addition is functional programming with lambda expressions. In this section we describe the benefits of functional programming and give a few examples of the programming style. Most of the features in Java 8 are more appropriate for an advanced Java text, but the concepts apply to material we have discussed, particularly when we are working with collections.

A lambda expression is a nameless function. In functional programming, a function is the same thing as a method. Related concepts include closures, anonymous functions, and function literals. As a nameless function, a lambda expression is essentially a little chunk of code that you can pass around as data but have it be treated like a function with parameters. Lambda expressions provide a neat way to implement a class that normally has only one function and to modify methods on the spot rather than go through the work of defining a method to perform a specialized task. Additionally, lambda expressions help Java parallelize itself to run more efficiently on multicore or parallel machines. For example, normally we will process elements in an `ArrayList` by creating a `for` loop that accesses each element one by one. This is considered *external* access to the loop. In contrast, with lambda expressions we can *internally* iterate through the `ArrayList` by providing a function that tells Java how to process each element. The Java Virtual Machine can then parallelize the operating by farming out computations on the elements to different processors.

The format to define a lambda expression looks like this:

```
parameters -> body
```

The arrow separates the parameters from the body. In many cases the body is short and just a single line of code. If it were longer, then a traditional method may make more

sense. Here is a lambda expression with a function that takes no parameters and returns the number 68:

```
() -> { return 68; }
```

Here is a lambda expression that returns the sum of two integers *x* and *y*:

```
(int x, int y) -> { return (x+y); }
```

In many cases Java can infer the type of the parameters, in which case we can leave the data type off. We can also simply provide an expression on the right side and it automatically becomes the return value without adding the keyword `return`. The following is equivalent to the previous example:

```
(x, y) -> x+y
```

As an example to motivate the use of lambda functions, consider the case where we want a class to implement the `Runnable` interface. If you recall from Section 19.1, the `Runnable` interface has only one method to implement, the `run()` method. Unlike in Section 19.1, in our simple example we won't be using threads, so we can directly invoke the `run()` method rather than the `start()` method of a thread. The following code illustrates the traditional way we would create an object that implements `Runnable`:

```
public class OldStyleRunnable implements
 Runnable
{
 public void run()
 {
 System.out.println
 ("Running in a class!");
 }
}

public class NotLambda1
{
 public static void main(String[] args)
 {
 OldStyleRunnable r0 = new
 OldStyleRunnable();
 r0.run(); // Not running in a thread
 }
}
```

### Sample Dialogue

```
Running in a class!
```

This is fine for one object, but what if we wanted multiple objects and we wanted different code in the `run()` method for each? Then we would have to explicitly create a separate class for each object. An alternative is to use an anonymous class in which we declare and instantiate the class in a single statement:

```
public class NotLambda2
{
 public static void main(String[] args)
 {
 // Anonymous class that overrides
 // the run() method. Anonymous classes are described in
 // Chapter 13.
 Runnable r = new Runnable()
 {
 public void run()
 {System.out.println
 ("In an anonymous class!");
 }
 };
 r.run();
 }
}
```

### Sample Dialogue

```
In an anonymous class!
```

This is an improvement over the first version because we can now create unique `Runnable` objects with the `run()` method of our choice without the need to assign a name to each derived `Runnable` class. However, lambda functions allow us to assign a function to a `Runnable` object in a single line:

```
public class LambdaRunnable
{
 public static void main(String[] args)
 {
 Runnable r =
 () -> System.out.println
 ("In a lambda expression!");
 r.run();
 }
}
```

### Sample Dialogue

```
In a lambda expression!
```

The lambda format is the simplest of all and lets us directly insert the method where needed. The same concept applies to implementing an `actionListener` for a GUI component. For example, instead of this old style code that uses an anonymous class:

```
button.addActionListener(new ActionListener()
{
 public void actionPerformed(ActionEvent e)
 {
 System.out.println("You clicked me!");
 }
});
```

we can now use the much shorter and easier to read

```
button.addActionListener
 (e -> System.out.println
 ("You clicked me!"));
```

Java's lambda expressions are particularly useful when applied to collections. Three common operations that we typically perform are to *filter*, *map*, or *reduce* the collection. In this section we give a short example of each.

Let's start with the concept of a filter. Consider the following code, which creates a list of doubles:

```
ArrayList<Double> nums = new ArrayList<>();
nums.add(3.5);
nums.add(56.3);
nums.add(81.1);
nums.add(4.8);
```

If we want to output only the values in the array that are greater than 50, then in traditional Java style (external processing) we would make a loop with an if statement:

```
for (int i = 0; i < nums.size(); i++)
 if (nums.get(i) > 50)
 System.out.println(nums.get(i));
```

Using Java 8's lambda expressions, we can do the same thing by creating a stream of the elements in the `ArrayList` and then filtering them. This is accomplished through a sequence of function calls:

```
nums.stream().filter((Double val) -> val >
50).forEach((Double val) ->
 System.out.println(val));
```

For readability purposes it is common to put each function call on a separate line:

```
nums.stream()
 .filter((Double val) -> val > 50)
 .forEach((Double val) ->
 System.out.println(val));
```

The `stream()` method creates a stream that generates a list that we can iterate once. Not to be confused with data streams, this new type of stream can be accessed in parallel or sequentially. In our case we are using only sequential streams. Once the stream is generated, we invoke `filter` and the `forEach`. Inside `filter` we specify a lambda expression. Each element in the `ArrayList` is filtered according to the lambda expression. In this case, the variable `val` is an element in the `ArrayList` that is being processed and the function says to filter only those elements with values greater than 50. Next, the `forEach` iterates through the filtered elements and outputs each one via `println`. In our example, this would output 56.3 and 81.1.

We can simplify the code a little bit more by leaving out the data type because Java is able to infer it from the context. The resulting code becomes

```
nums.stream()
 .filter(val -> val > 50)
 .forEach(val -> System.out.println(val));
```

The new format is quite different from the traditional method but the style is more concise, can be more easily parallelized, and in general requires less code than the old technique.

Next, consider the concept of a map. A map takes elements in the collection and transforms them in some way. First, consider a simple mapping where we would like to add 100 to every element in the `ArrayList`. We can do so as follows:

```
nums.stream()
 .map(val -> (val + 100))
 .forEach(val -> System.out.println(val));
```

This will output 100 added to each value (i.e., 103.5, 156.3, 181.1, 104.8). Note that each function is invoked in sequence. If we add our previous filter to the beginning, then we would get only 156.3 and 181.1:

```
nums.stream()
 .filter(val -> val > 50)
 .map(val -> (val + 100))
 .forEach(val -> System.out.println(val));
```

Finally, consider the concept of collecting. Collecting means that we process all of our elements in some way and collect the final result. The result is often a single value. Examples include summing, averaging, finding the minimum, and finding the

maximum of a set of data. The following code shows how we could compute the sum of all elements in our `ArrayList`:

```
double d = nums.stream()
 .mapToDouble(v -> v)
 .sum();
System.out.println("The sum is " + d);
```

The `mapToDouble` function takes each element and maps it as a double (a bit redundant here, since we are starting with doubles) and then accumulates all the elements into a sum. As you might surmise, there are also the methods `mapToInt()`, `mapToLong()`, and so on as well as methods to compute `min()`, `max()`, `average()`, and other values.

More customization is possible using the `reduce` function. In our case we'll use the version that takes as input a seed value and a binary function. Consider a collection with values  $v_1$ ,  $v_2$ , and  $v_3$ . If we start with a seed value  $s$ , then `reduce` will first apply the binary function to  $s$  and  $v_1$ , producing  $r_1$ . The binary function is then applied to  $r_1$  and  $v_2$ , producing  $r_2$ . Then the binary function applies to  $r_2$  and  $v_3$ , producing  $r_3$ , which is returned as the final value. The following code computes the sum of all values using `reduce`:

```
d = nums.stream()
 .reduce(0.0, (v1, v2) -> v1 + v2);
System.out.println("The sum is " + d);
```

In this case, 0.0 is the seed value and the second parameter is the function that specifies how to accumulate the sum of the values. For the first step,  $v_1$  corresponds to 0.0 and  $v_2$  corresponds to 3.5. This produces the intermediate sum of 3.5. In the second step,  $v_1$  corresponds to 3.5 and  $v_2$  corresponds to 56.3, producing 59.8. In the third step,  $v_1$  corresponds to 59.8 and  $v_2$  to 81.1, and so on until the sum is reached.

For an additional example, consider the following list of names:

```
ArrayList<String> names = new ArrayList<>();
names.add("Paco");
names.add("Enrique");
names.add("Bob");
```

If we want to compute the average length of all the names, then we can map the length to an integer:

```
d = names.stream()
 .mapToInt(name -> name.length())
 .average()
 .getAsDouble();
System.out.println("The average is " + d)
```

In this case we map each name to an int using the `length()` method, compute the average, and get the value as a double.

For the final example, say that we want to get the largest name. We can use the reduction technique:

```
String s = names.stream()
 .reduce("", (n1, n2) ->
 {
 if (n1.length() > n2.length())
 return n1;
 else
 return n2;
 }
);
System.out.println("longest Name: " + s);
```

We use a block in this case, where the function compares the length of the strings and returns the largest one. This is one case where we would commonly use the `?` operator to shorten the code:

```
String s = names.stream()
 .reduce("", (n1, n2) ->
 (n1.length() > n2.length()) ? n1 : n2);
System.out.println("longest Name: " + s);
```

These examples should give you an idea of what Java lambda expressions look like and what they can do. While there is definitely a learning curve, lambda expressions allow you to write code that is more concise while enabling parallel processing. Java 8's new syntax supports both functional programming and object-oriented programming in a way that reaps the benefits of both styles.

## Self-Test Exercises

18. Rewrite the program below so that it uses lambda functions to implement the action listeners.

```
import javax.swing.JButton;
import javax.swing.JFrame;
import java.awt.Color;
import java.awt.Container;
import java.awt.FlowLayout;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class ButtonDemo extends JFrame implements
 ActionListener
{
 public ButtonDemo()
 {
```

**Self-Test Exercises** (continued)

```

 setSize(250, 100);
 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
 setTitle("Button Demo");
 Container contentPane = getContentPane();
 contentPane.setBackground(Color.BLUE);
 contentPane.setLayout(new FlowLayout());

 JButton stopButton = new JButton("Red");
 stopButton.addActionListener(this);
 contentPane.add(stopButton);

 JButton goButton = new JButton("Green");
 goButton.addActionListener(this);
 contentPane.add(goButton);
 }

 public void actionPerformed(ActionEvent e)
 {
 Container contentPane = getContentPane();

 if (e.getActionCommand().equals("Red"))
 contentPane.setBackground(Color.RED);
 else if (e.getActionCommand().equals("Green"))
 contentPane.setBackground(Color.GREEN);
 else
 System.out.println("Error.");
 }

 public static void main(String[] args)
 {
 ButtonDemo buttonGui = new ButtonDemo();
 buttonGui.setVisible(true);
 }
}

```

19. What is the output of the following code?

```

ArrayList<Integer> nums = new ArrayList<>();
nums.add(3);
nums.add(5);
nums.add(1);
nums.stream()
 .filter(val -> val > 1)
 .forEach(val -> System.out.println(val));

```



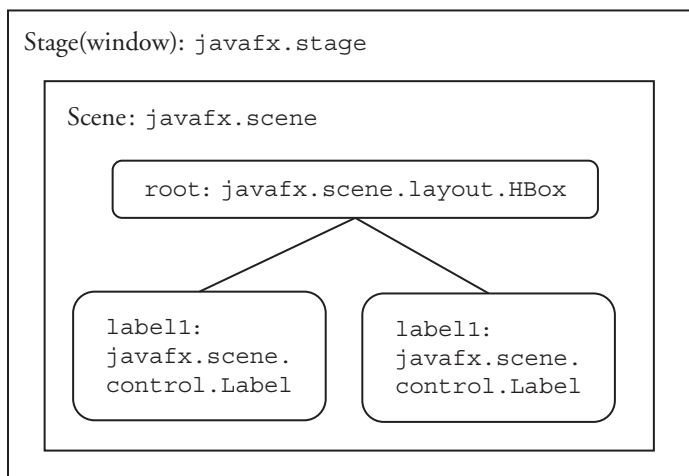
## 19.7 Introduction to JavaFX

JavaFX is a set of packages that allow Java programmers to create rich graphics and media applications. Potential applications include GUI interfaces, 2-D and 3-D games, animations, visual effects, touch-enabled applications, and multimedia applications. At the time of this writing, JavaFX 8 is the latest version. JavaFX has several advantages over other graphical libraries, including hardware-accelerated graphics and a high-performance media engine. The platform includes built-in UI controls and supports XML-based markup for the design and layout of UI components and Cascading Style Sheets for presentation. This separates the controlling code from the UI while simplifying the UI design. Most IDEs assist with many of these details in the construction of a JavaFX application. At some point JavaFX will replace Swing as the standard library for creating graphical interfaces. However, both JavaFX and Swing are expected to be included in Java distributions in the foreseeable future.

A JavaFX application uses the metaphor of a stage and scenes, just like the stage and scenes in a theater. The `Stage` class is a top-level JavaFX container and in our examples will correspond to an OS window. Every `Stage` has an associated `Scene` object. The `Scene` object contains a set of nodes called a **scene graph**. These nodes describe a scene of the application, just as the script, actors, and props describe a scene in a play or movie. In JavaFX the scene graph is a hierarchical set of nodes where the *root* node is at the top of the tree. Underneath the root node we can create subtrees consisting of layout components (e.g., panels), UI controls (e.g., buttons or textfields), shapes, or charts. Nodes have properties that include items such as the text, size, position, and color; they can be styled with Cascading Style Sheets (CSS), generate events, and be associated with visual effects.

The class structure and scene graph for a simple “Hello world” JavaFX application are shown in Display 19.26. In this example the `Stage` contains a `Scene` that is composed of an `HBox` layout pane that simply arranges nodes in a horizontal row. Inside the `HBox` are two labels. One label displays “Hello” and the second label displays “World.”

Display 19.26 Class Structure and Scene Graph for a Simple JavaFX Application



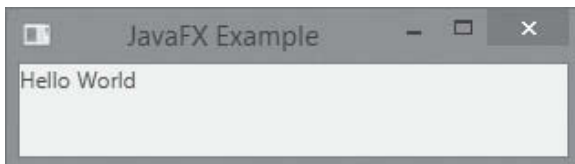
Code that implements the JavaFX program in Display 19.26 is shown in Display 19.27. The JavaFX program must extend `Application`. The entry point is the `start` method, which is invoked through the `launch` method. The JavaFX panes are similar in principle to the Swing layout classes. Other JavaFX layout panes include `BorderPane`, which is like Swing's `BorderLayout`, `HBox` for horizontal layout of nodes and `VBox` for vertical layout of nodes; `StackPane` to place nodes within a single stack on top of previous nodes; `FlowPane`, which is like Swing's `FlowLayout`; `TilePane`, which is like a tiled `FlowPane`; `GridPane`, which is like Swing's `GridLayout`; and `AnchorPane`, which allows nodes to be anchored to edges of the pane.

Display 19.27 Simple JavaFX Application (Source: Oracle Corporation)

```
1 import javafx.application.Application;
2 import javafx.scene.Scene;
3 import javafx.scene.control.Label;
4 import javafx.scene.layout.HBox;
5 import javafx.stage.Stage;

6 public class JavaFXHelloWorld extends Application
7 {
8 public void start(Stage primaryStage)
9 {
10 Label label1 = new Label();
11 Label label2 = new Label();
12 label1.setText("Hello");
13 label2.setText(" World");
14
15 HBox root = new HBox();
16 root.getChildren().add(label1);
17 root.getChildren().add(label2);
18
19 Scene scene = new Scene(root, 300, 50);
20
21 primaryStage.setTitle("JavaFX Example");
22 primaryStage.setScene(scene);
23 primaryStage.show();
24 }
25
26 public static void main(String[] args) {
27 launch(args);
28 }
29 }
```

Sample Dialogue



The programs in Displays 19.28 and 19.29 demonstrate how JavaFX allows us to achieve impressive visual effects with a minimal amount of code through a declarative rather than a procedural programming model. In a declarative program we specify what the program should do rather than the individual steps needed to achieve the end result. Display 19.28 draws a green circle on a black background. The program uses the `AnchorPane` layout with no anchors; we demonstrate anchoring in Display 19.32.

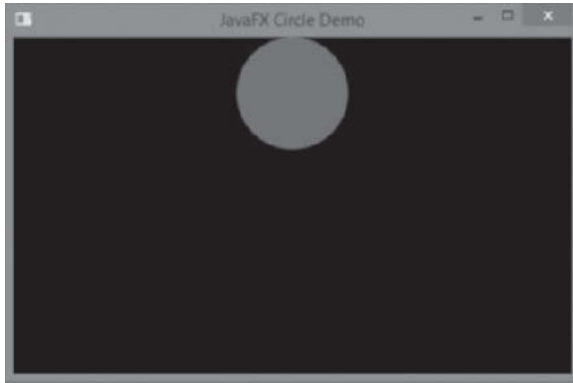
#### Display 19.28 JavaFX Circle Demonstration (part 1 of 2)

---

```
1 import javafx.application.Application;
2 import javafx.scene.Group;
3 import javafx.scene.Scene;
4 import javafx.scene.paint.Color;
5 import javafx.scene.shape.Circle;
6 import javafx.stage.Stage;
7 import javafx.scene.layout.AnchorPane;
8
9 public class JavaFXCircle extends Application
10 {
11 public void start(Stage stage)
12 {
13 Circle c = new Circle(250,50,50);
14 c.setFill(Color.GREEN);
15
16 AnchorPane root = new AnchorPane();
17 root.getChildren().add(c);
18
19 Scene scene = new Scene(root, 500, 300,
20 Color.BLACK);
21
22 stage.setTitle("JavaFX Circle Demo");
23 stage.setScene(scene);
24 stage.show();
25 }
26
27 public static void main(String[] args)
28 {
29 launch(args);
30 }
```

Display 19.28 JavaFX Circle Demonstration (part 2 of 2) (Source: Oracle Corporation)

Sample Dialogue



What if you wanted to animate the circle up and down while changing colors? As shown in Section 19.1, this is not trivial. We need to set up a thread with a timer and redraw the circle inside the timer. JavaFX lets us do this easily by attaching transitions to the circle node. In Display 19.29 we have attached a fill transition from green to blue and a translation transition in the vertical dimension to the  $y$ -coordinate 200 starting from coordinate (250, 50). JavaFX includes many other transitions, such as changing the scale, fade effect, and rotation. The parallel transition tells JavaFX to apply all of the transitions in parallel rather than sequentially. This is an example of declarative programming; we are telling JavaFX the desired end result and then the library handles the sequential details to implement the instructions.

Display 19.29 JavaFX Animated Circle Demonstration (part 1 of 2)

```
1 import javafx.application.Application;
2 import javafx.scene.Group;
3 import javafx.scene.Scene;
4 import javafx.scene.paint.Color;
5 import javafx.scene.shape.Circle;
6 import javafx.stage.Stage;
7 import javafx.scene.layout.AnchorPane;
8 import javafx.animation.FillTransition;
9 import javafx.animation.Timeline;
10 import javafx.animation.ParallelTransition;
11 import javafx.animation.TranslateTransition;
12 import javafx.util.Duration;
13
14 public class JavaFXCircleAnimate extends
 Application
```

(continued)

Display 19.29 JavaFX Animated Circle Demonstration (part 2 of 2)

---

```
15 {
16 public void start(Stage stage)
17 {
18 Circle c = new Circle(250,50,50);
19 c.setFill(Color.GREEN);
20
21 AnchorPane root = new AnchorPane();
22 root.getChildren().add(c);
23
24 FillTransition fill = new
 FillTransition(Duration.millis(500));
25 fill.setToValue(Color.BLUE); // To Blue
26
27 TranslateTransition translate = new
 TranslateTransition(
28 Duration.millis(500));
29 translate.setToY(200); // Move to Y=200
30
31 // Run the fill and translate
32 //transitions in parallel
33 ParallelTransition transition = new
 ParallelTransition(c,
34 fill, translate);
35 transition.setCycleCount(
 Timeline.INDEFINITE);
36 transition.setAutoReverse(true);
37 transition.play();
38
39 Scene scene = new Scene(
 root, 500, 300, Color.BLACK);
40
41 stage.setTitle("JavaFX Circle Demo");
42 stage.setScene(scene);
43 stage.show();
44 }
45
46 public static void main(String[] args)
47 {
48 launch(args);
49 }
```

---

Upon running the program in Display 19.29 you will see the circle move vertically from top to bottom while changing colors from green to blue. Note that the program is responsive! JavaFX handles threading so that the application does not lock up like the program in Display 19.1. If you add other UI controls like buttons or textboxes to the scene, then they will be active while the circle is animated.

JavaFX allows the programmer to attach event handlers to UI controls in a manner similar to Swing. Display 19.30 shows how to attach an event handler to a button. When the button is clicked, the value entered in the text field is read into an integer, incremented by one, and output into the label. The program uses a VBox pane, which vertically stacks each node that is added to the scene.

Display 19.30 JavaFX Event Demonstration (part 1 of 2)

---

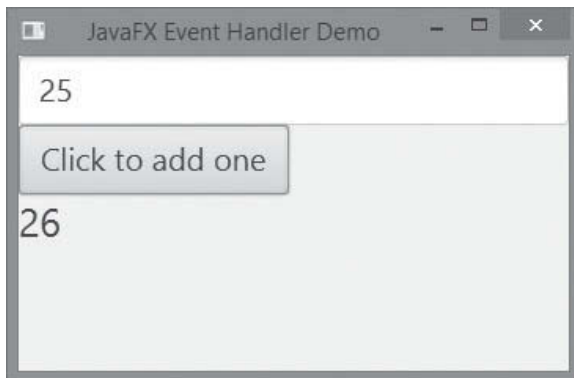
```
1 import javafx.application.Application;
2 import javafx.event.ActionEvent;
3 import javafx.event.EventHandler;
4 import javafx.scene.Scene;
5 import javafx.scene.control.Button;
6 import javafx.scene.control.Label;
7 import javafx.scene.control.TextField;
8 import javafx.scene.layout.VBox;
9 import javafx.scene.text.Font;
10 import javafx.stage.Stage;
11
12 public class JavaFXEvent extends Application
13 {
14 public void start(Stage primaryStage)
15 {
16 TextField txt = new TextField();
17 txt.setText("0");
18 txt.setFont(new Font(20));
19
20 Label lbl = new Label();
21 lbl.setFont(new Font(25));
22
23 Button btn = new Button();
24 btn.setFont(new Font(20));
25 btn.setText("Click to add one");
26 btn.setOnAction(new
27 EventHandler<ActionEvent>()
28 {
29 public void handle(ActionEvent event)
```

(continued)

Display 19.30 JavaFX Event Demonstration (part 2 of 2) (Source: Oracle Corporation)

```
29 {
30 int val = Integer.parseInt
 (txt.getText());
31 val++;
32 lbl.setText
 (Integer.toString(val));
33 }
34 });
35
36 VBox root = new VBox(); // Vertical layout
37 root.getChildren().add(txt);
38 root.getChildren().add(btn);
39 root.getChildren().add(lbl);
40
41 Scene scene = new Scene(root, 350, 200);
42 primaryStage.setTitle
 ("JavaFX Event Handler Demo");
43 primaryStage.setScene(scene);
44 primaryStage.show();
45 }
46
47 public static void main(String[] args)
48 {launch(args);
49 }
50 }
```

Sample Dialogue (after entering 25 and clicking the button)



If you have read Section 19.6 on functional programming, then you can simplify the event handling code by using the following lambda expression:

```
btn.setOnAction(e ->
{
 int val = Integer.parseInt
 (txt.getText());
 val++;
 lbl.setText
 (Integer.toString(val));
});
```

Building complex interfaces can be tedious and difficult to visualize when directly coding the layout panes. To assist with UI development Oracle has released the JavaFX Scene Builder. If you are using an IDE, then the Scene Builder may already be installed on your system. The Scene Builder can be downloaded for free from <http://www.oracle.com/technetwork/java/javase/downloads/sb2download-2177776.html>. Consult your IDE's documentation if configuration is needed to integrate the JavaFX Scene Builder. At the time of this writing, the latest version of the Scene Builder is 2.0.

The Scene Builder allows the programmer or UI designer to graphically construct the interface and quickly test the layout of UI controls. When using the Scene Builder, a JavaFX application will typically be split up into at least three separate files, each handling a different aspect of the program:

- **FXML file.** This is an XML file created by the Scene Builder that describes the layout of nodes in the scene. A sample FXML file similar to the program in Display 19.30 follows. Although you could manually create the file, it is normally generated by the Scene Builder.

```
<?xml version="1.0" encoding="UTF-8"?>
<?import javafx.scene.text.*?>
<?import javafx.scene.control.*?>
<?import java.lang.*?>
<?import javafx.scene.layout.*?>

<VBox maxHeight="-Infinity" maxWidth="-Infinity" minHeight=
"-Infinity" minWidth="-Infinity" prefHeight="200.0"
prefWidth="350.0" xmlns="http://javafx.com/javafx/8"
xmlns:fx="http://javafx.com/fxml/1">
 <children>
 <TextField fx:id="txt" text="0">

 </TextField>
 <Button fx:id="btn"
 mnemonicParsing="false"
 text="Click to add one">

 </Button>
```



```

<Label fx:id="lbl" text="23">

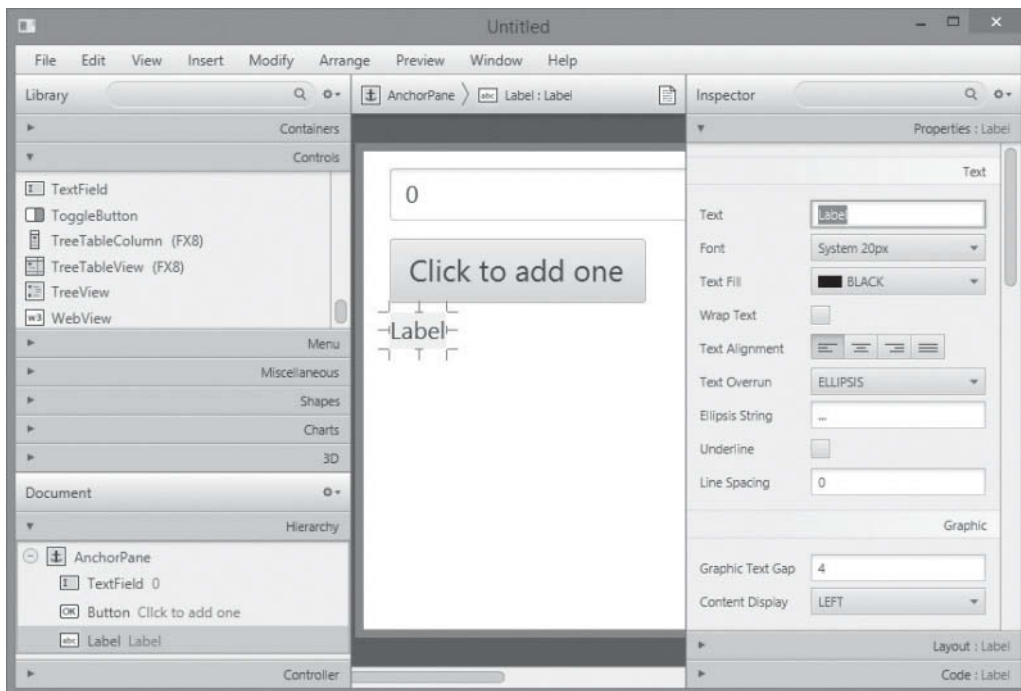
</Label>
</children>
</VBox>

```

- **Application file.** This is the JavaFX source code that contains the `start` method. When used with an FXML file, the `start` method merely loads the FXML file using the `FXMLLoader` class.
- **Controller file.** This file contains a class that implements `javaFX.fxml.Initializable` and contains event handlers that respond to UI controls.

If you are using an IDE that includes the Scene Builder, then consult your IDE's documentation on how to create a new JavaFX FXML Application project. Otherwise, you can directly launch the Scene Builder application after downloading and installing it. Display 19.31 shows the Scene Builder after dragging an `AnchorPane` from the "Containers" section to the middle of the window, followed by dragging a `TextField`, `Button`, and `Label` from the "Controls" section. You can select a control by either clicking it on the form or selecting it by name under "Hierarchy" within the

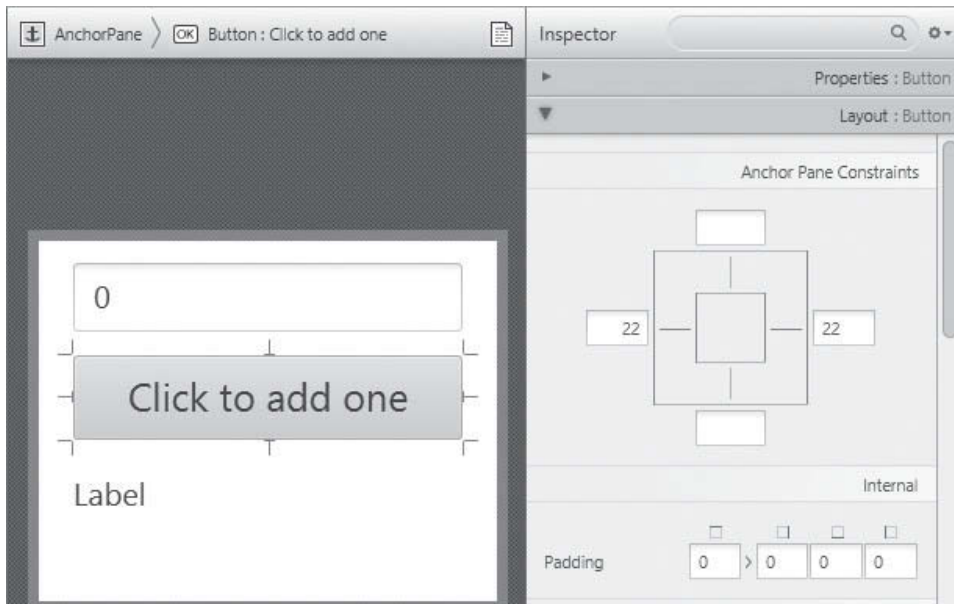
Display 19.31 UI Design with Scene Builder 2.0 (Source: Oracle Corporation)



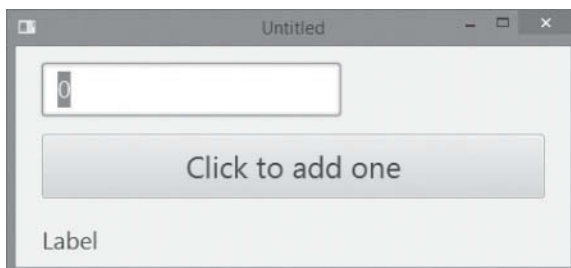
“Document” section on the bottom left. The latter is useful for “invisible” controls such as a label with no text. Once a control is selected, you can edit properties, such as the text or font size, in the “Properties” section in the “Inspector” window on the right. In Display 19.31 the text and font properties have been changed in a similar manner as the program in Display 19.30.

Since we are using an `AnchorPane`, we can anchor the sides of a control to the edges of the pane. This is useful if the window is resized. For example, if we want the button to fit the entire width of the window when it is resized, then we would anchor the left and right edges. This is illustrated in Display 19.32. The button has been selected, and under the “Layout” section of the “Inspector,” anchors have been set on the left and right sides. You can see test the result using the “Preview” command from the main menu.

Display 19.32 Anchoring a Button Using Scene Builder 2.0 (Source: Oracle Corporation)



Sample Dialogue



Display 19.33 JavaFX Application Class for `JavaFXApp.fxml`

---

```
1 import javafx.application.Application;
2 import javafx.fxml.FXMLLoader;
3 import javafx.scene.Parent;
4 import javafx.scene.Scene;
5 import javafx.stage.Stage;
6
7 public class JavaFXApp extends Application
8 {
9 public void start(Stage stage)
10 throws Exception
11 {
12 Parent root = FXMLLoader.load
13 (getClass().
14 getResource("JavaFXApp.fxml"));
15 Scene scene = new Scene(root);
16 stage.setScene(scene);
17 stage.show();
18 }
19 public static void main(String[] args) {
20 launch(args);
21 }
22 }
```

---

To load a saved FXML file created by the Scene Builder we use the `FXMLLoader` class. Display 19.33 shows how to load a FXML file named `JavaFXApp.FXML`. Since the layout details are in the FXML file, very little coding is needed in the application class.

Next we need a Controller class to respond to events. A class named `JavaFXAppController.java` is shown in Display 19.34; it implements the same button handler as the program in Display 19.30. This class implements `Initializable` and must have an `initialize` method that can be used to initialize the controller.

To link variables defined in the `JavaFXAppController` class to UI controls created in the Scene Builder, place the `@FXML` annotation before the variable definition. This injects the necessary values from the FXML loader.

Display 19.34 JavaFX Controller Class for `JavaFXApp.fxml` (part 1 of 2)

---

```
1 import java.net.URL;
2 import java.util.ResourceBundle;
3 import javafx.event.ActionEvent;
4 import javafx.fxml.FXML;
5 import javafx.fxml.Initializable;
6 import javafx.scene.control.TextField;
```

Display 19.34 JavaFX Controller Class for `JavaFXApp.fxml` (part 2 of 2)

```
7 import javafx.scene.control.Label;
8 import javafx.scene.control.Button;
9
10 public class JavaFXAppController implements
 Initializable
11 {
12 // The @FXML annotation looks up the
13 // corresponding ID in the FXML file so
14 // these variables map to the controls in
15 // the UI
16 @FXML
17 private Label lblNumber;
18 @FXML
19 private Button btnClick;
20 @FXML
21 private TextField txtNumber;
22
23 @FXML
24 private void handleButtonAction
 (ActionEvent event)
25 {
26 int val = Integer.parseInt
27 (txtNumber.getText());
28 val++;
29 lblNumber.setText
30 (Integer.toString(val));
31 }
32 public void initialize
 (URL url, ResourceBundle rb)
33 {
34 // Required by Initializable interface
35 // Called to initialize a controller after
36 // the root element has been processed
37 }
38 }
```

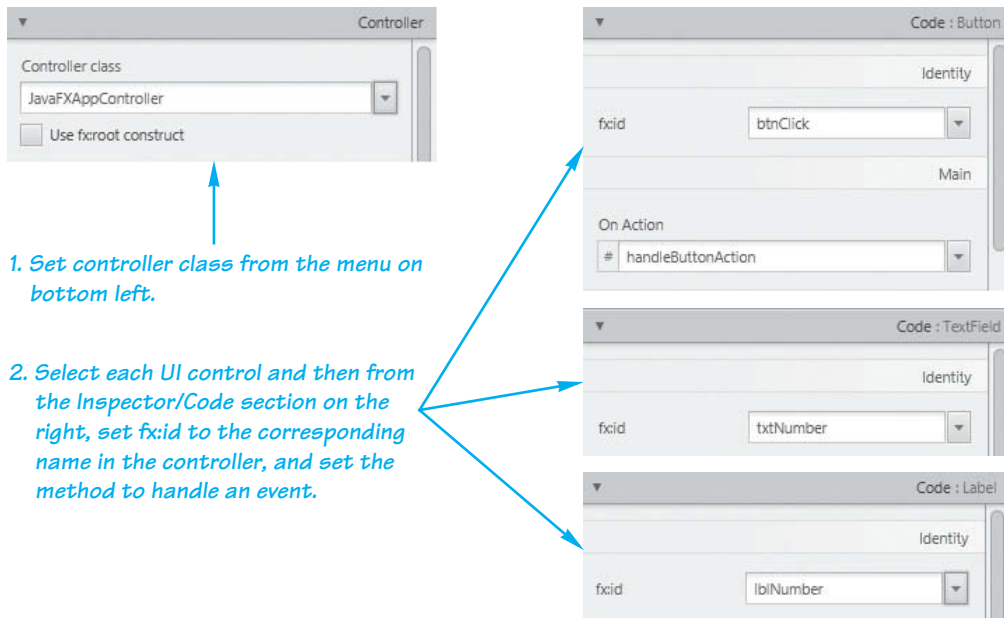
Finally, we need to link the controller to the FXML file. Back in the Scene Builder, select the Java file containing the controller in the “Controller” section located at the bottom left side of the Scene Builder. In our example, the controller is named `JavaFXController.java`.

Next, select each UI control that has a corresponding variable defined in the controller. To link the controls, select the variable name from the “Code” section of the “Inspector”

on the right. You can also select a method for an event handler. For example, on line 17 of Display 19.34 we named the label variable `lblNumber`. In the Scene Builder the same name should be entered in the `fx:id` field for the label on the form.

Display 19.35 depicts the process to link the controller of Display 19.34 to the FXML file constructed by the Scene Builder. Once the linkages are made, the Java programs can be compiled and the main application run to produce output such as that shown in the Sample Dialogue of Display 19.35.

Display 19.35 Linking the Controller in the Scene Builder (Source: Oracle Corporation)



Sample Dialogue (after compiling and running JavaFXApp)



In this section we have presented only a small taste of what JavaFX can do. JavaFX provides a structure and APIs for visual effects, animation, graphics, media, and the construction of graphical user interfaces. In addition, JavaFX supports declarative programming, separates controlling code from display code through FXML, and offers a Scene Builder application to assist with the construction of complex user interfaces. For additional reading about JavaFX, visit the Oracle JavaFX overview page at <http://www.oracle.com/technetwork/java/javase/overview/javafx-overview-2158620.html> and the JavaFX documentation website at <http://docs.oracle.com/javase/8/javase-clienttechnologies.htm>.

### Self-Test Exercises

20. What is the purpose of the FXML file in a JavaFX application?
21. What purpose does the @FXML annotation serve in a controller class?

## Chapter Summary

- A *thread* is a separate computation process. A Java program can have multiple threads.
- Use the class `Thread` to produce multiple threads.
- The static method `Thread.sleep` inserts a pause into the thread in which it is invoked.
- A thread's `join()` method is used to wait for threads to finish. The `synchronized` keyword restricts a critical region of code to a single thread only.
- A *socket* refers to an endpoint that connects two programs over a network.
- *TCP* refers to a reliable, streaming protocol for network communication. It ensures that data is received in the same order it was sent.
- *JavaBeans* refers to a framework for producing Java components that are easy to combine with other JavaBean components to produce new applications.
- A *relational database* organizes data in related tables. Records are stored as rows in a table. The fields for each record are stored as columns in a table.
- *Java DB* or *Derby* is a database that runs in client/server or embedded mode.
- JDBC allows you to insert SQL commands into your Java code to access and manipulate databases.
- The SQL `SELECT`, `UPDATE`, `CREATE TABLE`, and `INSERT` commands allow data to be retrieved, modified, created, or inserted into a database.
- *Java Server Pages (JSP)* refers to a framework that allows a programmer to create Web-based Java applications that run on the server. JSP requires a Web server capable of compiling and running Java servlets and JSP.

- HTML forms are a common mechanism to input user-specified data into a JSP application.
- Java code is added to a JSP page through *directive*, *expression*, *declaration*, and *scriptlet* tags.
- Java 8's lambda functions simplify event handling and provide a new way to efficiently and succinctly manage collections.
- JavaFX uses a set of nodes in a scene graph to create rich graphics and media applications.

## Answers to Self-Test Exercises

1. The invocation of `Thread.sleep` takes place inside a thread. Every action in Java takes place inside some thread. Any action performed by a method invocation in a specific thread takes place in that thread. Even if it does not know where it is, a method's action takes place where the method invocation is; if you are lost and yell out, you might not know where you are but the yell would still be wherever it is you are.
2. The class `Packer` inherits the method `start` from the base class `Thread` and is not overridden. The method `start` invokes the method `run`, but when `start` is invoked by an object of the class `Packer`, it is the definition of `run` that is given in the class `Packer` that is used, not the definition of `run` given in the definition of `Thread`. That is exactly what is meant by *late binding* or *polymorphism*.
3. In this program, the code that runs in each thread is fairly short. As a result, there is a good chance that each thread runs to completion before the next thread begins. The `sleep` method forces a thread to suspend and another thread to begin. This increases the likelihood that there will be contention for the shared variable and a race condition.
4. The code synchronizes access to the shared variable but the `main` method does not wait for all threads to finish before printing out the value of the shared variable. Thus, it is possible that `main` will output a premature value of `counter` if `main` completes before any one of the threads that it creates. To correct this problem, invoke the `join()` method on each thread before outputting `counter` as illustrated in Display 19.5.
5. A port is used to identify which program should receive data from the network. One program only may be bound to a specific port.
6. Each client connection may run in its own thread, requiring a large amount of memory, disk space, or processor time.
7. *Persistence* means that a component's state can be saved so that the next time it is used it remembers what state it was in.
8. The same one that is used for Swing and AWT.
9. 

```
SELECT Result.Title, Names.Author, Result.Author_ID
FROM Result, Names
WHERE Result.Author_ID = Names.Author_ID
```
10. A connection string is used to connect to (and possibly create) a database via JDBC. The protocol, database name, and other parameters (such as the username, password, or flag to create a new database) are specified in the connection string.

11. The `execute` method can be used to execute any SQL statement, but it is generally used for SQL commands where return values are not needed. It returns a Boolean value. The `executeQuery` method returns a `ResultSet` object that contains the rows matching a query.
12. 

```
CREATE TABLE BooksAuthors (ISBN varchar(15), author_id int)
```
13. 

```
INSERT INTO BooksAuthors VALUES ('0-553-38095-8',3)
INSERT INTO BooksAuthors VALUES ('0-553-57294-6',2)
INSERT INTO BooksAuthors VALUES ('0-671-46149-4',1)
INSERT INTO BooksAuthors VALUES ('0-553-57928-9',2)
```
14. A Java applet is downloaded and executed on the client, while a JSP program is written alongside HTML and runs on the server.
15. 

```
<html>
<head>
<title>Submit Firstname</title>
</head>

<body>
<form ACTION = "ProcessName.jsp" METHOD = POST>
First Name:
<input TYPE = "TEXT" NAME = "FirstName" VALUE = "" MAXLENGTH = "50">
<input TYPE="SUBMIT" VALUE="Submit">
</form>
</body>
</html>
```
16. `<% %>` is a scriptlet, `<%@ %>` is a directive, `<%! %>` is a declaration, `<%= %>` is an expression.
17. 

```
<%
String firstName = request.getParameter("FirstName");
out.println("<h1>" + firstName + "</h1>");
%>
```
18. 

```
// We no longer need to implement ActionListener
// since we implement it in the lambda function
public class ButtonDemo2 extends JFrame
{
 public ButtonDemo2()
 {
 setSize(250, 100);

 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
 setTitle("Button Demo");
 Container contentPane = getContentPane();
 contentPane.setBackground(Color.BLUE);
 contentPane.setLayout(new FlowLayout());

 JButton stopButton = new JButton("Red");
 stopButton.addActionListener(
 e -> contentPane.setBackground(Color.RED));
 contentPane.add(stopButton);
 }
}
```



```

 Button goButton = new JButton("Green");
 goButton.addActionListener(
 e -> contentPane.setBackground(Color.GREEN));
 contentPane.add(goButton);
 }
 public static void main(String[] args)
 {
 ButtonDemo2 buttonGui = new ButtonDemo2();
 buttonGui.setVisible(true);
 }
}

```

19. 3  
5

20. The FXML file describes the layout of the nodes in the scene. In our example, the FXML file describes which GUI controls are part of the application, their visual properties, and how the controls are arranged (e.g. stacked horizontally, vertically, etc.) Separate files contain the Java start method and event handlers.
21. The @FXML annotation links a variable in the controller class to a control in the FXML file. The variable in the controller class will be linked to the control in the FXML file with the same fx:id as the variable name.

## Programming Projects

1. Write a GUI program that uses the methods in the `Graphics` class together with threads to draw a ball within a rectangular box. The ball should move inside the box and as it strikes the wall of the box it should bounce back. The application should be responsive and exit immediately if the user decides to close the window.
2. Create a program to simulate the movement of a bicycle wheel. A wheel consists of a rim represented by a circle and several spokes represented by line from the centre of circle to its parameters. The wheel should move from one end of the frame to the other end on the click of start button. Use a thread for the rotation.
3. Write a program to simulate a buzzer. The program should make use of the `Thread` class. Create a class named `Buzzer` that extends the `Thread` class. Create an interface named `MonitorTime` which contains a method `setBuzzerTime()`. Your `Buzzer` class should implement this interface. Override `setBuzzerTime()` method to set the buzzer time delay in milliseconds and the number of times the buzzer should be repeated. Also, include methods `blowBuzzer` to start the buzzer.
4. Modify the GUI in Display 19.2 so that the circles are alternately red, white, and blue, and so that they fill the area from bottom to top instead of top to bottom.
5. Produce a GUI similar to the one in Display 19.2 except that instead of filling an area with circles, it launches a ball (which is just a circle), and the ball bounces



VideoNote  
Solution to  
Programming  
Project 19.3

around inside a rectangle. You can create the illusion of a moving ball by repeatedly erasing the ball and redrawing it at a location a little farther along its path. Look up the method `setXORMode` in the class `Graphics`. It will take care of the erasing.

6. This project shows how to create a simple Web server. Create a server program that listens on port 8000. When a client connects to the server, the program should send the following data to the client:

```
"HTTP/1.0 200 OK\n\n" + body
```

where `body` is the String `"<HTML><TITLE>Java Server</TITLE>This web page was sent by our simple <B>Java Server</B></HTML>"`. If you know HTML, feel free to insert your own content. The header line identifies the message as part of the HTTP protocol that is used to transmit Web pages.

When the server is running, you should be able to start a Web browser and navigate to your machine on port 8000 and view the message. For example, if the server is running on your local machine, you could point your Web browser to `http://localhost:8000` and the message in `body` should be displayed.

7. Modify the server from Programming Project 19.6 so that the content for `body` is read from a file on the local hard drive instead of hard-coded into the program. This file should contain the HTML string from Programming Project 19.6. In addition, modify the server so that a new thread is created for each connection. Test the server by starting up two or more Web browsers and navigate to your site. Each browser should display the message.
8. Create a threaded chat server and a corresponding chat client. Using the port of your choice, create a server that starts a new thread for every client that connects to it. Every message that the server receives from a client should be broadcast back to all other clients. The chat client should allow the user to type in a string of text and have it sent to the server upon pressing enter. Use threads on the client so messages can be retrieved from the server and displayed even while the user is typing. Test your server by connecting to it with multiple clients and verifying that messages are transmitted back and forth.
9. The program in Display 19.12 creates a database with the `Names` table from Display 19.10. Modify this program so it also creates the `Titles` and `BooksAuthors` tables with identical data entries as shown in Display 19.10. Next, create a separate program that prompts the user to input the name of an author and then outputs all book titles written by that author.
10. Create an HTML form that prompts the user to enter a height in feet. When the form is submitted, a JSP web page should run that displays the height converted to inches. The height in inches is `Foot * 12 Inches`.
11. Create an HTML form that serves as a random baby name generator. The form should prompt the user to enter a last name. When the form is submitted, a JSP Web page should run that randomly picks the first name of a boy and the first name of a girl and then outputs the random first names coupled with the entered last name. For example, if the last name entered is "Savitch" and if the JSP program



randomly selects “Emma” as the girl name and “Homer” as the boy name, then the output would be

```
If your baby is a boy, consider the name Homer Savitch.
If your baby is a girl, consider the name Emma Savitch.
```

The boy and girl names should be randomly selected from the files `boynames.txt` and `girlnames.txt` that are included in the source code on the website for this book.

These files contain the 1,000 most popular boy and girl names in the United States for the year 2003 as compiled by the Social Security Administration.

12. Recreate Programming Project 19.9 as JSP pages instead of as a stand-alone application. One JSP page should create the database, tables, and populate the tables with data. Create an HTML form that allows the user to enter the name of an author. The form should invoke another JSP page that displays all titles written by the specified author that are stored in the database.
13. This program simulates what might happen if two people who share the same bank account happen to make a simultaneous deposit or withdrawal and the bank does not account for race conditions by recreating the situation described in Displays 19.4 and 19.5 with a simple `BankAccount` class. The `BankAccount` class should store an account balance and have methods to retrieve the balance, make a deposit, and make a withdrawal. Do not worry about negative balances.

Next, create an array of thousands of threads where each thread has a reference to the same `BankAccount` object. In the `run()` method, even numbered threads deposit one dollar and odd numbered threads withdraw one dollar. If you create an even number of threads, then after all threads are done the account balance should be zero. See if you can find a number of threads so that you consistently end up with a balance that is not zero. If you want to increase the likelihood of a race condition, then make each thread sleep a short random number of milliseconds in the `run()` method.

Add the `synchronized` keyword to fix the problem and ensure a balance of zero after all the threads are done.

14. Write a GUI application using JavaFX with a `button` and a `textarea`. When the button is clicked, the HTML from [www.pearsonhighered.com/savitch](http://www.pearsonhighered.com/savitch) should be retrieved using the `URL` class and output into the `textarea`. Use Java 8’s functional programming paradigm to implement the action listener for the button.

<code>abstract</code>	<code>final</code>	<code>public</code>
<code>assert</code>	<code>finally</code>	<code>return</code>
<code>boolean</code>	<code>float</code>	<code>short</code>
<code>break</code>	<code>for</code>	<code>static</code>
<code>byte</code>	<code>goto</code>	<code>strictfp</code>
<code>case</code>	<code>if</code>	<code>super</code>
<code>catch</code>	<code>implements</code>	<code>switch</code>
<code>char</code>	<code>import</code>	<code>synchronized</code>
<code>class</code>	<code>instanceof</code>	<code>this</code>
<code>const</code>	<code>int</code>	<code>throw</code>
<code>continue</code>	<code>interface</code>	<code>throws</code>
<code>default</code>	<code>long</code>	<code>transient</code>
<code>do</code>	<code>native</code>	<code>true</code>
<code>double</code>	<code>new</code>	<code>try</code>
<code>else</code>	<code>null</code>	<code>void</code>
<code>enum</code>	<code>package</code>	<code>volatile</code>
<code>extends</code>	<code>private</code>	<code>while</code>
<code>false</code>	<code>protected</code>	

This page intentionally left blank

PRECEDENCE	ASSOCIATIVITY
From highest at top to lowest at bottom. Operators in the same group have equal precedence.	
Dot operator, array indexing, and method invocation: <code>.</code> , <code>[ ]</code> , <code>()</code>	Left to right
<code>++</code> (postfix, as in <code>x++</code> ), <code>--</code> (postfix)	Right to left
The unary operators: <code>+</code> , <code>-</code> , <code>++</code> (prefix, as in <code>++x</code> ), <code>--</code> (prefix), <code>!</code> , <code>~</code> (bitwise complement) <sup>1</sup>	Right to left
<code>new</code> and type casts ( <code>Type</code> )	Right to left
The binary operators <code>*</code> , <code>/</code> , <code>%</code>	Left to right
The binary operators <code>+</code> , <code>-</code>	Left to right
The binary operators <code>&lt;&lt;</code> , <code>&gt;&gt;</code> , <code>&gt;&gt;&gt;</code> (shift operators) <sup>1</sup>	Left to right
The binary operators <code>&lt;</code> , <code>&gt;</code> , <code>&lt;=</code> , <code>&gt;=</code> , <code>instanceof</code>	Left to right
The binary operators <code>==</code> , <code>!=</code>	Left to right
The binary operator <code>&amp;</code>	Left to right
The binary operator <code>^</code> (exclusive or) <sup>1</sup>	Left to right
The binary operator <code> </code>	Left to right
The binary operator <code>&amp;&amp;</code>	Left to right
The binary operator <code>  </code>	Left to right
The ternary operator (conditional operator) <code>? :</code>	Right to left
The assignment operators <code>=</code> , <code>*=</code> , <code>/=</code> , <code>%=</code> , <code>+=</code> , <code>-=</code> , <code>&amp;=</code> , <code> =</code> , <code>^=</code> , <code>&lt;&lt;=</code> , <code>&gt;&gt;=</code> , <code>&gt;&gt;&gt;=</code>	Right to left

<sup>1</sup> Not discussed in this book.

This page intentionally left blank

# ASCII Character Set 3

The characters shown here form the ASCII character set, which is the subset of the Unicode character set that is commonly used by English speakers. The numbering is the same whether the characters are considered to be members of the Unicode character set or of the ASCII character set. Character number 32 is the blank. Printable characters only are shown.

32		56	8	80	P	104	h
33	!	57	9	81	Q	105	i
34	"	58	:	82	R	106	j
35	#	59	;	83	S	107	k
36	\$	60	<	84	T	108	l
37	%	61	=	85	U	109	m
38	&	62	>	86	V	110	n
39	'	63	?	87	W	111	o
40	(	64	@	88	X	112	p
41	)	65	A	89	Y	113	q
42	*	66	B	90	Z	114	r
43	+	67	C	91	[	115	s
44	,	68	D	92	\	116	t
45	-	69	E	93	]	117	u
46	.	70	F	94	^	118	v
47	/	71	G	95	_	119	w
48	0	72	H	96	`	120	x
49	1	73	I	97	a	121	y
50	2	74	J	98	b	122	z
51	3	75	K	99	c	123	{
52	4	76	L	100	d	124	
53	5	77	M	101	e	125	}
54	6	78	N	102	f	126	~
55	7	79	O	103	g		



This page intentionally left blank

## SYNTAX

```
System.out.printf (Format_String, Output_1, Output_2, ..., Output_Last);
```

*Format\_String* is a string including one format specifier for each *Output* argument. *Format\_String* is output with each format specifier replaced by its corresponding *Output* argument in the format given by the *Output* argument's format specifier.

Display A4.1 Format Specifiers for `System.out.printf`

CONVERSION CHARACTER	TYPE OF OUTPUT	EXAMPLES
d	Decimal (ordinary) integer.	%5d %d
f	Fixed-point (everyday notation) floating-point.	%6.2f %f
e	E-notation floating point.	%8.3e %e
g	General floating point. (Java decides whether to use E-notation or not.)	%8.3g %g
s	String.	%12s %s
c	Character.	%2c %c
b	Boolean. The corresponding <i>Output</i> argument is a Boolean expression. Outputs <code>true</code> or <code>false</code> .	%6b %b
n	Denotes a line break. This does not correspond to an <i>Output</i> argument. It is approximately equivalent to <code>\n</code> .	%n

A number of the form *N.M* in a format specifier specifies a field width of *N* spaces with *M* digits after the decimal point. If one number *N* is given only, it specifies a field width; if there is a decimal point in the output, then the number of digits after the decimal point is determined by Java.

When the value output does not fill the field width specified, then blanks are added in front of the value output. The output is then said to be *right justified*. If you add a hyphen (-) after the %, then any extra blank space is placed after the value output and the output is said to be *left justified*. For example, `%8.2f` is right justified and `%-8.2f` is left justified.

This page intentionally left blank

This appendix summarizes most of the library classes used in this book. This appendix includes some methods, and even some classes, that are not discussed in the text. The lists of class methods and other class members contain the most commonly used members and the members used in this book, but they are not complete lists of methods for the classes given here.

If a class or interface is derived from another class or interface, respectively, then in some cases, the table for the derived class or interface lists only new methods and does not list all the inherited methods.

## Abstract Button

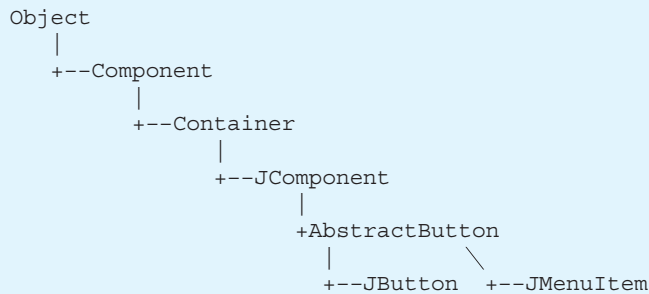
Package: `javax.swing`

The classes `JButton` and `JMenuItem` are also in this package.

All these methods are inherited by the classes `JButton` and `JMenuItem`.

`AbstractButton` is an abstract class.

Ancestor classes:



```
public void addActionListener(ActionListener listener)
```

Adds an `ActionListener`.

```
public String getActionCommand()
```

Returns the action command for this component.

```
public String getText()
```

Returns the text written on the component, such as the text on a button or the string for a menu item.

```
public void removeActionListener(ActionListener listener)
```

Removes an `ActionListener`.

```
public void setActionCommand(String actionCommand)
```

Sets the action command.

```
public void setBackground(Color theColor)
```

Sets the background color of this component.

```
public void setMaximumSize(Dimension maximumSize)
```

Sets the maximum size of the button or label. Note that this is only a suggestion to the layout manager. The layout manager is not required to respect this maximum size. The following special case will work for most simple situations. The `int` values give the width and height in pixels.

```
public void setMaximumSize(
 new Dimension(int width, int height))
```

```
public void setMinimumSize(Dimension minimumSize)
```

Sets the minimum size of the button or label. Note that this is only a suggestion to the layout manager. The layout manager is not required to respect this minimum size. Although we do not discuss the `Dimension` class, the following special case is intuitively clear and will work for most simple situations. The `int` values give the width and height in pixels.

```
public void setMinimumSize(
 new Dimension(int width, int height))
```

```
public void setPreferredSize(Dimension preferredSize)
```

Sets the preferred size of the button or label. Note that this is only a suggestion to the layout manager. The layout manager is not required to use the preferred size. The following special case will work for most simple situations. The `int` values give the width and height in pixels.

```
public void setPreferredSize(
 new Dimension(int width, int height))
```

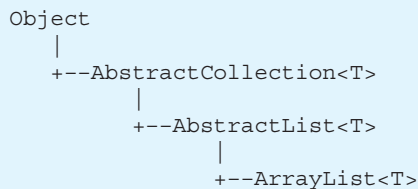
```
public void setText(String text)
```

Makes text the only text on this component.

## ArrayList<T>

Package: `java.util`

Ancestor classes:



All the exception classes mentioned are unchecked exceptions, which means they are not required to be caught in a `catch` block or declared in a `throws` clause. `NoSuchElementException` is in the `java.util` package, which requires an import statement if your code mentions the `NoSuchElementException` class. All the other exception classes mentioned are in the package `java.lang` and so do not require any import statement.

**CONSTRUCTORS**

```
public ArrayList(int initialCapacity)
```

Creates an empty `ArrayList<T>` with the specified initial capacity. When the `ArrayList<T>` needs to increase its capacity, the capacity doubles.  
Throws an `IllegalArgumentException` if `initialCapacity` is negative.

```
public ArrayList()
```

Creates an empty `ArrayList<T>` with an initial capacity of 10. When the `ArrayList<T>` needs to increase its capacity, the capacity doubles.

```
public ArrayList(Collection<? extends <T> c)
```

Creates an `ArrayList<T>` that contains all the elements of the collection `c` in the same order as they have in `c`. In other words, the elements have the same index in the `ArrayList<T>` created as they do in `c`. This is not quite a true copy constructor because it does not preserve capacity. The capacity of the created `ArrayList<T>` will be `c.size()`, not `c.capacity`. The `ArrayList<T>` created is only a shallow copy of the collection argument. The `ArrayList<T>` created contains references to the elements in `c` (not references to clones of the elements in `c`).

**Throws:**

`NullPointerException` if `c` is `null`.

**ARRAYLIKE METHODS**

```
public T set(int index, T newElement)
```

Sets the element at the specified `index` to `newElement`. The element previously at that position is returned. If you draw an analogy to an array `a`, this is analogous to setting `a[index]` to the value `newElement`. The `index` must be a value greater than or equal to 0 and strictly less than the current size of the list.

**Throws:**

`IndexOutOfBoundsException` if the `index` is not in this range.

```
public T get(int index)
```

Returns the element at the specified `index`. This is analogous to returning `a[index]` for an array `a`. The `index` must be a value greater than or equal to 0 and less than the current size of the calling object.

**Throws:**

`IndexOutOfBoundsException` if the `index` is not in the required range.

## METHODS TO ADD ELEMENTS

```
public boolean add(T newElement)
```

Adds `newElement` to the end of the calling object's list and increases its size by one. The capacity of the calling object is increased if that is required. Returns `true` if the add was successful. This method is often used as if it were a `void` method.

```
public void add(int index, T newElement)
```

Inserts `newElement` as an element in the calling object at the specified index and increases the size of the calling object by one. Each element in the calling object with an index greater than or equal to `index` is shifted upward to have an index that is one greater than it had previously. The `index` must be a value greater than or equal to 0 and less than *or equal to* the size of the calling object (before this addition).

Note that you can use this method to add an element after the last current element. The capacity of the calling object is increased if that is required.

**Throws:**

`IndexOutOfBoundsException` if the index is not in the prescribed range.

```
public boolean addAll(Collection<? extends T> c)
```

Appends all the elements in `c` to the end of the elements in the calling object in the order that they are enumerated by a `c` iterator. The behavior of this method is not guaranteed if the collection `c` is the calling object or any collection including the calling object either directly or indirectly.

**Throws:**

`NullPointerException` if `c` is `null`.

```
public boolean addAll(int index, Collection<? extends T> c)
```

Inserts all the elements in `c` into the calling object starting at position `index`. Elements are inserted in the order that they are enumerated by a `c` iterator. Elements previously at positions `index` or higher are shifted to higher numbered positions.

**Throws:**

`IndexOutOfBoundsException` if `index` is not both greater than or equal to zero and less than `size()`.

`NullPointerException` if `c` is `null`.

## METHODS TO REMOVE ELEMENTS

```
public T remove(int index)
```

Deletes the element at the specified index and returns the element deleted. The size of the calling object is decreased by one. The capacity of the calling object is not changed. Each element in the calling object with an index greater than or equal to `index` is decreased to have an index that is one less than the value it had previously.

The `index` must be a value greater than or equal to 0 and less than the size of the calling object (before this removal).

**Throws:**

`IndexOutOfBoundsException` if the `index` is not in the prescribed range.

```
public boolean remove(Object theElement)
```

Removes the first occurrence of `theElement` from the calling object. If `theElement` is found in the calling object, then each element in the calling object with an index greater than or equal to `theElement`'s index is decreased to have an index that is one less than the value it had previously. Returns `true` if `theElement` was found (and removed). Returns `false` if `theElement` was not found in the calling object. If the element was removed, the size is decreased by one. The capacity is not changed.

```
protected void removeRange(int fromIndex, int toIndex)
```

Removes all elements with index greater than or equal to `fromIndex` and strictly less than `toIndex`. Be sure to note that this method is `protected`, not `public`.

```
public void clear()
```

Removes all elements from the calling object and sets its size to zero.

#### SEARCH METHODS

```
public boolean isEmpty()
```

Returns `true` if the calling object is empty (that is, has size 0); otherwise returns `false`.

```
public boolean contains(Object target)
```

Returns `true` if `target` is an element of the calling object; otherwise returns `false`. Uses the method `equals` of the object `target` to test for equality.

```
public int indexOf(Object target)
```

Returns the index of the first element that is equal to `target`. Uses the method `equals` of the object `target` to test for equality. Returns `-1` if `target` is not found.

```
public int lastIndexOf(Object target)
```

Returns the index of the last element that is equal to `target`. Uses the method `equals` of the object `target` to test for equality. Returns `-1` if `target` is not found.

#### ITERATORS

```
public Iterator<T> iterator()
```

Returns an iterator for the calling object. Iterators are discussed in Section 16.3.

```
public ListIterator<T> listIterator()
```

Returns a `ListIterator<T>` for the calling object. `ListIterator<T>` is discussed in Section 16.3.



```
ListIterator<T> listIterator(int index)
```

Returns a list iterator for the calling object starting at `index`. The first element to be returned by the iterator is the one at `index`. (Iterators are discussed in Section 16.3.)

**Throws:**

`IndexOutOfBoundsException` if `index` does not satisfy:  
`0 <= index <= size()`

**CONVERTING TO AN ARRAY**

```
public Object[] toArray()
```

Returns an array containing all of the elements in the calling object. The elements of the array are indexed the same as in the calling object.

```
public <E> E[] toArray(E[] a)
```

Note that the type parameter `E` is not the same as `T`. So, `E` can be any reference type; it need not be the type `T` in `Collection<T>`. For example, `E` might be an ancestor type of `T`.

Returns an array containing all of the elements in the calling object. The elements of the array are indexed the same as in the calling object.

The argument `a` is used primarily to specify the type of the array returned. The exact details are as follows:

The type of the returned array is that of `a`. If the collection fits in the array `a`, then `a` is used to hold the elements of the returned array; otherwise a new array is created with the same type as `a`.

If `a` has more elements than the calling object, then the element in `a` immediately following the end of the elements copied from the calling object are set to `null`.

**Throws:**

`ArrayStoreException` if the type of `a` is not an ancestor type of the type of every element in the calling object.

`NullPointerException` if `a` is `null`.

**MEMORY MANAGEMENT**

```
public int size()
```

Returns the number of elements in the calling object.

```
public int capacity()
```

Returns the current capacity of the calling object.

```
public void ensureCapacity(int newCapacity)
```

Increases the capacity of the calling object to ensure that it can hold at least `newCapacity` elements. Using `ensureCapacity` can sometimes increase efficiency, but its use is not needed for any other reason.

```
public void trimToSize()
```

Trims the capacity of the calling object to be the calling object's current size. This is used to save storage.

#### MAKE A COPY

```
public Object clone()
```

Returns a shallow copy of the calling object.

## Boolean

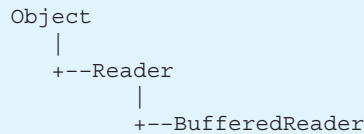
This is a wrapper class for `boolean`. See Section 5.1 in Chapter 5.

## BufferedReader

Package: `java.io`

The `FileReader` class is also in this package.

Ancestor classes:



```
public BufferedReader(Reader readerObject)
```

This is the only constructor you are likely to need. There is no constructor that accepts a file name as an argument. If you want to create a stream using a file name, use

```
new BufferedReader(new FileReader(File_Name))
```

When used in this way, the `FileReader` constructor, and thus the `BufferedReader` constructor invocation, can throw a `FileNotFoundException`, which is a kind of `IOException`.

If you want to create a stream using an object of the class `File`, use

```
new BufferedReader(new FileReader(File_Object))
```

When used in this way, the `FileReader` constructor, and thus the `BufferedReader` constructor invocation, can throw a `FileNotFoundException`, which is a kind of `IOException`.

```
public void close() throws IOException
```

Closes the stream's connection to a file.

```
public int read() throws IOException
```

Reads a single character from the input stream and returns that character as an `int` value. If the read goes beyond the end of the file, then `-1` is returned. Note that the value is returned as an `int`. To obtain a `char`, you must perform a type cast on the value returned. The end of a file is signaled by returning `-1`. (All of the "real" characters return a positive integer.)

```
public String readLine() throws IOException
```

Reads a line of input from the input stream and returns that line. If the read goes beyond the end of the file, `null` is returned. (Note that an `EOFException` is not thrown at the end of a file. The end of a file is signaled by returning `null`.)

```
public long skip(long n) throws IOException
```

Skips `n` characters.

## Byte

This is a wrapper class for `byte`. See Section 5.1 in Chapter 5.

## Character

Package: `java.lang`

Ancestor classes:

```
Object
|
+--Character
```

Implemented interfaces: `Comparable`, `Serializable`

The `Character` class is marked `final`, which means it cannot be used as a base class to derive other classes.

```
public static boolean isDigit(char argument)
```

Returns `true` if its argument is a digit; otherwise returns `false`.

### EXAMPLES

`Character.isDigit('5')` returns `true`. `Character.isDigit('A')` and `Character.isDigit('%')` both return `false`.

```
public static boolean isLetter(char argument)
```

Returns `true` if its argument is a letter; otherwise returns `false`.

### EXAMPLES

`Character.isLetter('A')` returns `true`. `Character.isLetter('%')` and `Character.isLetter('5')` both return `false`.

```
public static boolean isLetterOrDigit(char argument)
```

Returns `true` if its argument is a letter or a digit; otherwise returns `false`.

### EXAMPLES

`Character.isLetterOrDigit('A')` and `Character.isLetterOrDigit('5')` both return `true`. `Character.isLetterOrDigit('&')` returns `false`.

```
public static boolean isLowerCase(char argument)
```

Returns true if its argument is a lowercase letter; otherwise returns false.

#### EXAMPLES

`Character.isLowerCase('a')` returns true. `Character.isLowerCase('A')` and `Character.isLowerCase('%')` both return false.

```
public static boolean isUpperCase(char argument)
```

Returns true if its argument is an uppercase letter; otherwise returns false.

#### EXAMPLES

`Character.isUpperCase('A')` returns true. `Character.isUpperCase('a')` and `Character.isUpperCase('%')` both return false.

```
public static boolean isWhitespace(char argument)
```

Returns true if its argument is a whitespace character; otherwise returns false. Whitespace characters are those that print as whitespace, such as the space character (blank character), the tab character (`'\t'`), and the new-line character (`'\n'`).

#### EXAMPLES

`Character.isWhitespace(' ')` returns true. `Character.isWhitespace('A')` returns false.

```
public static char toLowerCase(char argument)
```

Returns the lowercase version of its argument. If the argument is not a letter, it is returned unchanged.

#### EXAMPLE

`Character.toLowerCase('a')` and `Character.toLowerCase('A')` both return `'a'`.

```
public static char toUpperCase(char argument)
```

Returns the uppercase version of its argument. If the argument is not a letter, it is returned unchanged.

#### EXAMPLE

`Character.toUpperCase('a')` and `Character.toUpperCase('A')` both return `'A'`.

## Collection<T> Interface

Package: `java.util`

Ancestor interfaces: none

All the exception classes mentioned are unchecked exceptions, which means they are not required to be caught in a `catch` block or declared in a `throws` clause.

All the exception classes mentioned are in the package `java.lang` and so do not require any `import` statement.

### CONSTRUCTORS

Although not officially required by the interface, any class that implements the `Collection<T>` interface should have at least two constructors: a no-argument constructor that creates an empty `Collection<T>` object, and a constructor with one parameter of type `Collection<? extends T>` that creates a `Collection<T>` object with the same elements as the constructor argument. The interface does not specify whether the copy produced by the one-argument constructor is a shallow copy or a deep copy of its argument.

```
public boolean contains(Object target)
```

Returns `true` if the calling object contains at least one instance of `target`. Uses `target.equals` to determine if `target` is in the calling object.

#### Throws:

`ClassCastException` if the type of `target` is incompatible with the calling object (optional).

`NullPointerException` if `target` is `null` and the calling object does not support `null` elements (optional).

```
public boolean containsAll(Collection<?> collectionOfTargets)
```

Returns `true` if the calling object contains all of the elements in `collectionOfTargets`. For `element` in `collectionOfTargets`, this method uses `element.equals` to determine if `element` is in the calling object.

#### Throws:

`ClassCastException` if the types of one or more elements in `collectionOfTargets` are incompatible with the calling object (optional).

`NullPointerException` if `collectionOfTargets` contains one or more `null` elements and the calling object does not support `null` elements (optional).

`NullPointerException` if `collectionOfTargets` is `null`.

```
public boolean equals(Object other)
```

This is the `equals` of the collection, not the `equals` of the elements in the collection. Overrides the inherited method `equals`. Although there are no official constraints on `equals` for a collection, it should be defined as we have described in Chapter 7 and also satisfy the intuitive notion of collections being equal.

```
public int hashCode()
```

Returns the hash code value for the calling object. Hash codes are discussed in Chapter 15.

```
public boolean isEmpty()
```

Returns `true` if the calling object is empty; otherwise returns `false`.

```
Iterator<T> iterator()
```

Returns an iterator for the calling object. (Iterators are discussed in Section 16.3.)

```
public Object[] toArray()
```

Returns an array containing all of the elements in the calling object. If the calling object makes any guarantees as to what order its elements are returned by its iterator, this method must return the elements in the same order.

The array returned should be a new array so that the calling object has no references to the returned array. (You might also want the elements in the array to be clones of the elements in the collection. However, this is apparently not required by the interface, because library classes, such as `Vector<T>`, return arrays that contain references to the elements in the collection.)

```
public <E> E[] toArray(E[] a)
```

Note that the type parameter `E` is not the same as `T`. So, `E` can be any reference type; it need not be the type `T` in `Collection<T>`. For example, `E` might be an ancestor type of `T`.

Returns an array containing all of the elements in the calling object. The argument `a` is used primarily to specify the type of the array returned. The exact details are as follows:

The type of the returned array is that of `a`. If the elements in the calling object fit in the array `a`, then `a` is used to hold the elements of the returned array; otherwise a new array is created with the same type as `a`.

If `a` has more elements than the calling object, the element in `a` immediately following the end of the copied elements is set to `null`.

If the calling object makes any guarantees as to what order its elements are returned by its iterator, this method must return the elements in the same order. (Iterators are discussed in Section 16.3.)

#### Throws:

`ArrayStoreException` if the type of `a` is not an ancestor type of the type of every element in the calling object.

`NullPointerException` if `a` is `null`.

```
public int size()
```

Returns the number of elements in the calling object. If the calling object contains more than `Integer.MAX_VALUE` elements, returns `Integer.MAX_VALUE`.

#### OPTIONAL METHODS

The following methods are optional, which means they still must be implemented, but the implementation can simply throw an `UnsupportedOperationException` if for some reason you do not want to give them a “real” implementation. An `UnsupportedOperationException` is a `RuntimeException` and so is not required to be caught or declared in a `throws` clause.

```
public boolean add(T element) (Optional)
```

Ensures that the calling object contains the specified `element`. Returns `true` if the calling object changed as a result of the call. Returns `false` if the calling object does not permit duplicates and already contains `element`; also returns `false` if the calling object does not change for any other reason.

**Throws:**

`UnsupportedOperationException` if this method is not supported by the class that implements this interface.

`ClassCastException` if the class of `element` prevents it from being added to the calling object.

`NullPointerException` if `element` is `null` and the calling object does not support `null`

elements. `IllegalArgumentException` if some other aspect of `element` prevents it from being added to the calling object.

```
public boolean addAll(Collection<? extends T> collectionToAdd) (Optional)
```

Ensures that the calling object contains all the elements in `collectionToAdd`. Returns `true` if the calling object changed as a result of the call; returns `false` otherwise. If the calling object changes during this operation, its behavior is unspecified; in particular, its behavior is unspecified if `collectionToAdd` is the calling object.

**Throws:**

`UnsupportedOperationException` if this method is not supported by the class that implements this interface.

`ClassCastException` if the class of an element of `collectionToAdd` prevents it from being added to the calling object.

`NullPointerException` if `collectionToAdd` contains one or more `null` elements and the calling object does not support `null` elements, or if `collectionToAdd` is `null`.

`IllegalArgumentException` if some aspect of an element of `collectionToAdd` prevents it from being added to the calling object.

```
public void clear() (Optional)
```

Removes all the elements from the calling object.

**Throws:**

`UnsupportedOperationException` if this method is not supported by the class that implements this interface.

```
public boolean remove(Object element) (Optional)
```

Removes a single instance of `element` from the calling object, if it is present. Returns `true` if the calling object contained `element`; returns `false` otherwise.

**Throws:**

`UnsupportedOperationException` if this method is not supported by the class that implements this interface.

`ClassCastException` if the type of `element` is incompatible with the calling object (optional).

`NullPointerException` if `element` is `null` and the calling object does not support `null` elements (optional).

```
public boolean removeAll(Collection<?> collectionToRemove) (Optional)
```

Removes all the calling object's elements that are also contained in `collectionToRemove`. Returns `true` if the calling object was changed; otherwise returns `false`.

**Throws:**

`UnsupportedOperationException` if this method is not supported by the class that implements this interface.

`ClassCastException` if the types of one or more elements in `collectionToRemove` are incompatible with the calling collection (optional).

`NullPointerException` if `collectionToRemove` contains one or more `null` elements and the calling object does not support `null` elements (optional).

`NullPointerException` if `collectionToRemove` is `null`.

```
public boolean retainAll(Collection<?> saveElements) (Optional)
```

Retains only the elements in the calling object that are also contained in the collection `saveElements`. In other words, removes from the calling object all of its elements that are not contained in the collection `saveElements`. Returns `true` if the calling object was changed; otherwise returns `false`.

**Throws:**

`ClassCastException` if the types of one or more elements in `saveElements` are incompatible with the calling object (optional).

`NullPointerException` if `saveElements` contains one or more `null` elements and the calling object does not support `null` elements (optional).

`NullPointerException` if `saveElements` is `null`.

## Color

Package: `java.awt`

Ancestor classes:

```
Object
|
+--Color
```

### CONSTRUCTORS

```
public Color(float r, float g, float b)
```

Constructor that creates a new `Color` with the specified RGB values. The parameters `r`, `g`, and `b` must each be in the range 0.0 to 1.0 (inclusive).

```
public Color(int r, int g, int b)
```

Constructor that creates a new `Color` with the specified RGB values. The parameters `r`, `g`, and `b` must each be in the range 0 to 255 (inclusive).



## METHODS

```
public Color brighter()
```

Returns a brighter version of the calling object color.

```
public Color darker()
```

Returns a darker version of the calling object color.

```
public boolean equals(Object c)
```

Returns true if c is equal to the calling object color; otherwise returns false.

```
public int getBlue()
```

Returns the blue component of the calling object. The returned value is in the range 0 to 255 (inclusive).

```
public int getGreen()
```

Returns the green component of the calling object. The returned value is in the range 0 to 255 (inclusive).

```
public int getRed()
```

Returns the red component of the calling object. The returned value is in the range 0 to 255 (inclusive).

## CONSTANTS

Color.BLACK	Color.MAGENTA
Color.BLUE	Color.ORANGE
Color.CYAN	Color.PINK
Color.DARK_GRAY	Color.RED
Color.GRAY	Color.WHITE
Color.GREEN	Color.YELLOW
Color.LIGHT_GRAY	

## Comparable Interface

Package: java.lang

Ancestor interfaces: none

The Comparable interface has only one method heading that must be implemented.

```
public int compareTo(Object other)
```

The method compareTo should return a negative number if the calling object “comes before” the parameter other, a zero if the calling object “equals” the parameter other, and a positive number if the calling object “comes after” the parameter other.

The “comes before” ordering that underlies compareTo should be a total ordering. Most normal ordering, such as less-than on numbers and lexicographic ordering on strings, are total orderings.

## Double

This is a wrapper class for `double`. See Section 5.1 in Chapter 5.

## File

Package: `java.io`

Ancestor classes:

```

Object
|
+--File

```

Many of these methods throw a `SecurityException` if a security manager exists and is unhappy with the method invocation. This is not likely to be a concern for readers of this book, and we have not noted this in the method descriptions.

The class `SecurityException` is an unchecked exception class, which means you need not catch it or declare it in a `throws` clause.

```
public File(String fileName)
```

Constructor. `fileName` can be either a full or a relative pathname (which includes the case of a simple file name). `fileName` is referred to as the **abstract pathname**.

### Throws:

`NullPointerException` if the pathname `fileName` is `null`.

```
public boolean canRead()
```

Tests whether the program can read from the file. Returns `true` if the file named by the abstract pathname exists and is readable by the program; otherwise returns `false`.

```
public boolean canWrite()
```

Tests whether the program can write to the file. Returns `true` if the file named by the abstract pathname exists and is writable by the program; otherwise returns `false`.

```
public boolean createNewFile()
```

Creates a new empty file named by the abstract pathname, provided that a file of that name does not already exist. Returns `true` if successful; returns `false` otherwise.

### Throws:

`IOException` if an I/O error occurs.

```
public boolean delete()
```

Tries to delete the file or directory named by the abstract pathname. A directory must be empty to be removed. Returns `true` if it was able to delete the file or directory. Returns `false` if it was unable to delete the file or directory.

```
public boolean exists()
```

Tests whether there is a file with the abstract pathname.

```
public String getName()
```

Returns the last name in the abstract pathname (that is, the simple file name). Returns the empty string if the abstract pathname is the empty string.

```
public String getPath()
```

Returns the abstract pathname as a `String` value.

```
public boolean isDirectory()
```

Returns `true` if a directory (folder) exists that is named by the abstract pathname; otherwise returns `false`.

```
public boolean isFile()
```

Returns `true` if a file exists that is named by the abstract pathname and the file is a normal file; otherwise returns `false`. The meaning of *normal* is system dependent. Any file created by a Java program is guaranteed to be normal.

```
public long length()
```

Returns the length in bytes of the file named by the abstract pathname. If the file does not exist or the abstract pathname names a directory, then the value returned is not specified and may be anything.

```
public boolean mkdir()
```

Makes a directory named by the abstract pathname. Will not create parent directories. See `makedirs`. Returns `true` if successful; otherwise returns `false`.

```
public boolean mkdirs()
```

Makes a directory named by the abstract pathname. Will create any necessary but nonexistent parent directories. Returns `true` if successful; otherwise returns `false`. Note that if it fails, then some of the parent directories may have been created.

```
public boolean renameTo(File newName)
```

Renames the file represented by the abstract pathname to `newName`. Returns `true` if successful; otherwise returns `false`. `newName` can be a relative or absolute pathname. This may require moving the file. Whether or not the file can be moved is system dependent.

**Throws:**

`NullPointerException` if parameter `newName` is `null`.

```
public boolean setReadOnly()
```

Sets the file represented by the abstract pathname to be read only. Returns `true` if successful; otherwise returns `false`.

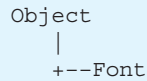
## Float

This is a wrapper class for `float`. See Section 5.1 in Chapter 5.

## Font

Package: `java.awt`

Ancestor classes:



### CONSTRUCTOR

```
public Font(String fontName, int styleModifications, int size)
```

Constructor that creates a version of the font named by `fontName` with the specified `styleModifications` and `size`.

### CONSTANTS

`Font.BOLD`

Specifies bold style.

`Font.ITALIC`

Specifies italic style.

`Font.PLAIN`

Specifies plain style—that is, not bold and not italic.

### NAMES OF Fonts

(These three are guaranteed by Java. Your system will probably have others as well as these.)

`"Monospaced"`

See Chapter 18 for a sample.

`"SansSerif"`

See Chapter 18 for a sample.

`"Serif"`

See Chapter 18 for a sample.

### METHOD THAT USES Font

```
public abstract void setFont(Font fontObject)
```

This method is in the class `Graphics`. Sets the current font of the calling `Graphics` object to `fontObject`.

## Graphics

Package: `java.awt`

Ancestor classes:

```

Object
|
+--Graphics

```

Graphics is an abstract class.

Although many of these methods are abstract, we always use them with objects of a concrete descendent class of Graphics, even though we usually do not know the name of that concrete class.

```

public abstract void drawRect(int x, int y,
 int width, int height)

```

Draws the outline of the specified rectangle. (x, y) is the location of the upper-left corner of the rectangle.

```

public abstract void fillRect(int x, int y,
 int width, int height)

```

Fills the specified rectangle. (x, y) is the location of the upper-left corner of the rectangle.

```

public void draw3DRect(int x, int y, int width,
 int height, boolean raised)

```

Draws the outline of the specified rectangle. (x, y) is the location of the upper-left corner. The rectangle is highlighted to look like it has thickness. If `raised` is `true`, the highlight makes the rectangle appear to stand out from the background. If `raised` is `false`, the highlight makes the rectangle appear to be sunken into the background.

```

public void fill3DRect(int x, int y, int width,
 int height, boolean raised)

```

Fills the rectangle specified by

```

draw3DRect(x, y, width, height, raised)

```

```

public abstract void drawArc(int x, int y,
 int width, int height,
 int startAngle, int arcSweep)

```

Draws part of an oval that just fits into an invisible rectangle described by the first four arguments. The portion of the oval drawn is given by the last two arguments. See Chapter 18 for details.

```

public abstract void drawLine(int x1, int y1, int x2, int y2)

```

Draws a line between points (x1, y1) and (x2, y2).

```

public abstract void drawOval(int x, int y,
 int width, int height)

```

Draws the outline of the oval with the smallest enclosing rectangle that has the specified width and height. The (imagined) rectangle has its upper-left corner located at (x, y).

```
public void drawPolygon(int[] x, int[] y, int points)
```

Draws a polygon through the point

(x[0], y[0]), (x[1], y[1]), ..., (x[points - 1], y[points - 1]).

Always draws a closed polygon. If the first and last points are not equal, it draws a line from the last to the first point.

```
public void drawPolyline(int[] x, int[] y, int points)
```

Draws a polygon through the point

(x[0], y[0]), (x[1], y[1]), ..., (x[points - 1], y[points - 1]).

If the first and last points are not equal, the polygon will not be closed.

```
public abstract void drawRoundRect(int x, int y,
 int width, int height, int arcWidth, int arcHeight)
```

Draws the outline of the specified round-cornered rectangle. (x, y) is the location of the upper-left corner of the enclosing regular rectangle. arcWidth and arcHeight specify the shape of the round corners. See Chapter 18 for details.

```
public abstract void drawString(String text, int x, int y)
```

Draws the text given by the specified string, using this graphics object's current font and color. The baseline of the leftmost character is at position (x, y) in this graphics object's coordinate system.

```
public abstract void fillArc(int x, int y,
 int width, int height,
 int startAngle, int arcSweep)
```

Fills the partial oval specified by

```
drawArc(x, y, width, height, startAngle, arcSweep)
```

```
public abstract void fillOval(int x, int y,
 int width, int height)
```

Fills the oval specified by

```
drawOval(x, y, width, height)
```

```
public void fillPolygon(int[] x, int[] y, int points)
```

Fills (with color) the polygon specified by

```
drawPolygon(x, y, points)
```

```
public abstract void fillRoundRect(int x, int y,
 int width, int height, int arcWidth, int arcHeight)
```

Fills the round rectangle specified by

```
drawRoundRect(x, y, width, height, arcWidth, arcHeight)
```

```
public abstract void setFont(Font fontObject)
```

Sets the current font of the calling Graphics object to fontObject.

**HashMap<K,V> Class**

Package: `java.util`

Ancestor classes:

```

Object
|
+--AbstractMap<K,V>
 |
 +--HashMap<K,V>

```

Implements interfaces: `Map<K,V>`, `Cloneable`, `Serializable`

The `HashMap<K,V>` class implements all of the methods in the `Map<K,V>` interface. The only other methods in the `HashMap<K,V>` class are the constructors.

All the exception classes mentioned are the kind that are not required to be caught in a catch block or declared in a throws clause. All the exception classes mentioned are in the package `java.lang` and so do not require any import statement. The class `K` must implement the `equals` and `hashCode` methods.

```
public HashMap()
```

Creates a new, empty map with a default initial capacity of 16 and load factor of 0.75. The capacity is the number of slots in the hash table. The load factor is the percentage of capacity before the size of the table is automatically increased.

```
public HashMap(int initialCapacity)
```

Creates a new, empty map with a default capacity of `initialCapacity` and load factor of 0.75.

**Throws:**

`IllegalArgumentException` if the initial capacity is negative.

```
public HashMap(int initialCapacity, float loadFactor)
```

Creates a new, empty map with the specified capacity and load factor.

**Throws:**

`IllegalArgumentException` if the initial capacity or the load factor is negative.

```
public HashMap (Map<? extends K,? extends V> m)
```

Creates a new map with the same mappings as `m`. The `initialCapacity` is set to the same size as `m` and the `loadFactor` to 0.75.

**Throws:**

`NullPointerException` if `m` is `null`.

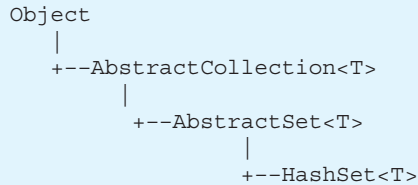
```
public Object clone()
```

Creates a shallow copy of this instance and returns it. The keys and values are not cloned. The remainder of the methods are the same as those described for the `Map<K,V>` interface.

## HashSet<T>

Package: `java.util`

Ancestor classes:



Implements interfaces: `Cloneable`, `Collection<T>`, `Serializable`, `Set<T>`

The `HashSet<T>` class implements all of the methods in the `Set<T>` interface. The only other methods in the `HashSet<T>` class are the constructors. The class `T` must implement the `equals` and `hashCode` methods. The two constructors that do not involve concepts beyond the scope of this book are given as follows:

All the exception classes mentioned are the kind that are not required to be caught in a `catch` block or declared in a `throws` clause.

All the exception classes mentioned are in the package `java.lang` and so do not require any import statement.

```
public HashSet ()
```

Creates a new, empty set.

```
public HashSet (Collection<? extends T> c)
```

Creates a new set that contains all the elements of `c`.

**Throws:**

`NullPointerException` if `c` is `null`.

```
public HashSet (int initialCapacity)
```

Creates a new, empty set with the specified capacity.

**Throws:**

`IllegalArgumentException` if `initialCapacity` is less than zero.

The methods are the same as those described for the `Set<T>` interface.

## Integer

This is a wrapper class for `int`. See Section 5.1 in Chapter 5.



**Iterator<T> Interface**

Package: `java.util`

Ancestor interfaces: none

All the exception classes mentioned are the kind that are not required to be caught in a `catch` block or declared in a `throws` clause.

`NoSuchElementException` is in the `java.util` package, which requires an `import` statement if your code mentions the `NoSuchElementException` class. All the other exception classes mentioned are in the package `java.lang` and so do not require any `import` statements.

```
public boolean hasNext ()
```

Returns `true` if `next ()` has not yet returned all the elements in the collection; returns `false` otherwise.

```
public T next ()
```

Returns the next element of the collection that produced the iterator.

**Throws:**

`NoSuchElementException` if there is no next element.

```
public void remove () (Optional)
```

Removes from the collection the last element returned by `next`.

This method can be called only once per call to `next`.

**Throws:**

`IllegalStateException` if the `next` method has not yet been called, or the `remove` method has already been called after the last call to the `next` method.

`UnsupportedOperationException` if the `remove` operation is not supported by this `Iterator`.

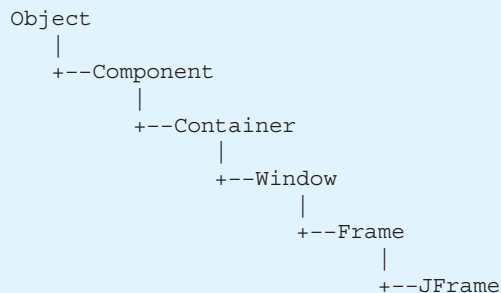
**JButton**

See `AbstractButton`.

**JFrame**

Package: `javax.swing`

Ancestor classes:



```
public JFrame ()
```

Constructor that creates an object of the class `JFrame`.

```
public JFrame (String title)
```

Constructor that creates an object of the class `JFrame` with the title given as the argument.

```
public Component add(Component componentAdded)
```

Adds `componentAdded` to the `JFrame`. Typically used as a void method.

```
public Container getContentPane ()
```

Returns the content pane of the calling `JFrame` object. `Container` is a class in the package `java.awt`.

To set the color of a `JFrame`, use

```
getContentPane ().setBackground (Color c)
```

If you use `setBackground` without the `getContentPane ()`, you will not get any error messages, but you will probably not see the color.

```
public void setDefaultCloseOperation (int operation)
```

Sets the action that will happen by default when the user clicks the close-window button. The argument should be one of the following defined constants:

`JFrame.DO_NOTHING_ON_CLOSE`: Do nothing. The `JFrame` does nothing, but if there are any registered window listeners, they are invoked. (Window listeners are explained in Chapter 18.)

`JFrame.HIDE_ON_CLOSE`: Hide the frame after invoking any registered `WindowListener` objects.

`JFrame.DISPOSE_ON_CLOSE`: Hide and *dispose* the frame after invoking any registered window listeners. When a window is **disposed**, it is eliminated but the program does not end. To end the programs, use the next constant as an argument to `setDefaultCloseOperation`.

`JFrame.EXIT_ON_CLOSE`: Exit the application using the `System.exit` method. (Do not use this for frames in applets. Applets are discussed on the accompanying website.)

If no action is specified using the method `setDefaultCloseOperation`, then the default action taken is `JFrame.HIDE_ON_CLOSE`.

#### Throws:

`IllegalArgumentException` if the argument is not one of the values listed previously.

`SecurityException` if the argument is `JFrame.EXIT_ON_CLOSE` and the Security Manager will not allow the caller to invoke `System.exit`. (You are not likely to encounter this case.)

```
public void setLayout (LayoutManager manager)
```

Makes `manager` the layout manager for the `JFrame`.

```
public void setSize (int width, int height)
```

Sets the size of the calling frame so that it has the width and height specified. Pixels are the units of length used.

```
public void setTitle(String title)
```

Sets the title for this frame to the argument string.

```
public void dispose()
```

Eliminates the calling frame and all its subcomponents. Any memory they use is released for reuse. If there are items left (items other than the calling frame and its subcomponents), then this does not end the program. (The method `dispose` is discussed in Chapter 18.)

```
public void setJMenuBar(JMenuBar menubar)
```

Sets the menu bar for the calling frame.

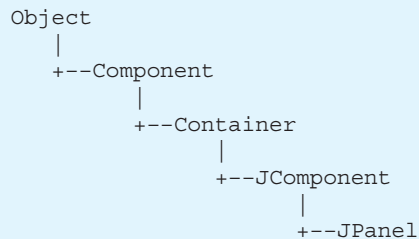
### JMenuItem

See `AbstractButton`.

### JPanel

Package: `javax.swing`

Ancestor classes:



```
public JPanel()
```

Constructor that creates an object of the class `JPanel`.

```
public JPanel(LayoutManager manager)
```

Constructor that creates an object of the class `JPanel` with the given layout manager.

```
public Component add(Component componentAdded)
```

Adds `componentAdded` to the `JPanel`. Typically used as a `void` method.

```
public void setBackground(Color c)
```

Sets the color of the `JPanel`.

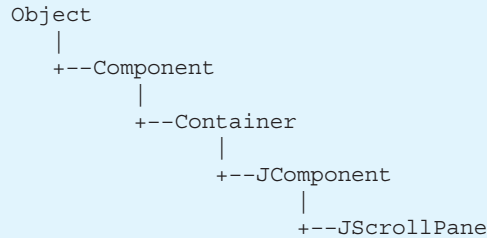
```
public void setLayout(LayoutManager manager)
```

Makes `manager` the layout manager for the `JPanel`.

## JScrollPane

Package: javax.swing

Ancestor classes:



```
public JScrollPane(Component objectToBeScrolled)
```

Creates a new JScrollPane for the objectToBeScrolled. Note that the objectToBeScrolled need not be a JTextArea, although that is the only type of argument considered in this book.

```
public void setHorizontalScrollBarPolicy(int policy)
```

Sets the policy for showing the horizontal scroll bar. The policy should be one of

```
JScrollPane.HORIZONTAL_SCROLLBAR_ALWAYS
```

```
JScrollPane.HORIZONTAL_SCROLLBAR_NEVER
```

```
JScrollPane.HORIZONTAL_SCROLLBAR_AS_NEEDED
```

The phrase AS\_NEEDED means the scroll bar is shown only when it is needed. This is explained more fully in Chapter 17. The meanings of the other policy constants are obvious from their names.

(As indicated, these constants are defined in the class JScrollPane. You should not need to even be aware of the fact that they have int values. Think of them as policies, not as int values.)

```
public void setVerticalScrollBarPolicy(int policy)
```

Sets the policy for showing the vertical scroll bar. The policy should be one of

```
JScrollPane.VERTICAL_SCROLLBAR_ALWAYS
```

```
JScrollPane.VERTICAL_SCROLLBAR_NEVER
```

```
JScrollPane.VERTICAL_SCROLLBAR_AS_NEEDED
```

The phrase AS\_NEEDED means the scroll bar is shown only when it is needed. This is explained more fully in Chapter 18. The meanings of the other policy constants are obvious from their names.

(As indicated, these constants are defined in the class JScrollPane. You should not need to even be aware of the fact that they have int values. Think of them as policies, not as int values.)

**JTextArea**

See `JTextComponent`.

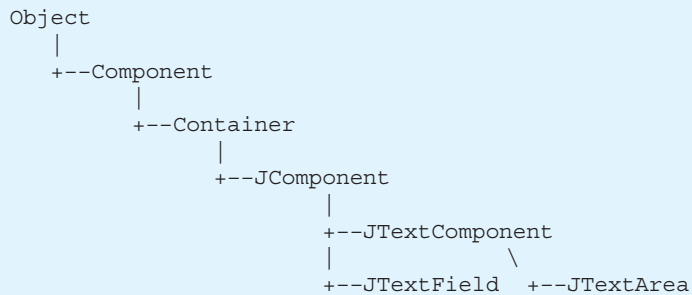
**JTextComponent**

Package: `javax.swing.text`

The classes `JTextField` and `JTextArea` are in the package `javax.swing`.

All these methods are inherited by the classes `JTextField` and `JTextArea`.

Ancestor classes:



```
public String getText()
```

Returns the text that is displayed by this text component.

```
public boolean isEditable()
```

Returns `true` if the user can write in this text component. Returns `false` if the user is not allowed to write in this text component.

```
public void setBackground(Color theColor)
```

Sets the background color of this text component.

```
public void setEditable(boolean argument)
```

If `argument` is `true`, then the user is allowed to write in the text component. If `argument` is `false`, then the user is not allowed to write in the text component.

```
public void setText(String text)
```

Sets the text that is displayed by this text component to be the specified text.

**JTextField**

See `JTextComponent`.

## List<T> Interface

Package: `java.util`

Ancestor interfaces: `Collection<T>`, `Iterable<T>`

All the exception classes mentioned are the kind that are not required to be caught in a `catch` block or declared in a `throws` clause.

All the exception classes mentioned are in the package `java.lang` and so do not require any `import` statement.

### CONSTRUCTORS

Although not officially required by the interface, any class that implements the `List<T>` interface should have at least two constructors: a no-argument constructor that creates an empty `List<T>` object, and a constructor with one parameter of type `Collection<? extends T>` that creates a `List<T>` object with the same elements as the constructor argument. If the argument imposes an ordering on its elements, then the `List<T>` created should preserve this ordering.

```
public boolean contains(Object target)
```

Returns `true` if the calling object contains at least one instance of `target`. Uses `target.equals` to determine if `target` is in the calling object.

#### Throws:

`ClassCastException` if the type of `target` is incompatible with the calling object (optional).

`NullPointerException` if `target` is `null` and the calling object does not support `null` elements (optional).

```
public boolean containsAll(Collection<?> collectionOfTargets)
```

Returns `true` if the calling object contains all of the elements in `collectionOfTargets`. For `element` in `collectionOfTargets`, uses `element.equals` to determine if `element` is in the calling object. The elements need not be in the same order or have the same multiplicity in `collectionOfTargets` and in the calling object.

```
public boolean equals(Object other)
```

If the argument is a `List`, returns `true` if the calling object and the argument contain exactly the same elements in exactly the same order; otherwise returns `false`. If the argument is not a `List`, `false` is returned.

```
public int hashCode()
```

Returns the hash code value for the calling object. Hash codes are discussed in Chapter 15.

```
boolean isEmpty()
```

Returns `true` if the calling object is empty; otherwise returns `false`.

```
Iterator<T> iterator()
```

Returns an iterator for the calling object. (Iterators are discussed in Section 16.3.)

```
public Object[] toArray()
```

Returns an array containing all of the elements in the calling object. The elements in the returned array are in the same order as in the calling object. A new array must be returned so that the calling object has no references to the returned array.

```
public <E> E[] toArray(E[] a)
```

Note that the type parameter `E` is not the same as `T`. So, `E` can be any reference type; it need not be the type `T` in `Collection<T>`. For example, `E` might be an ancestor type of `T`.

Returns an array containing all of the elements in the calling object. The elements in the returned array are in the same order as in the calling object. The argument `a` is used primarily to specify the type of the array returned. The exact details are described in the table for the `Collection<T>` interface.

**Throws:**

`ArrayStoreException` if the type of `a` is not an ancestor type of the type of every element in the calling object.

`NullPointerException` if `a` is `null`.

```
public int size()
```

Returns the number of elements in the calling object. If the calling object contains more than `Integer.MAX_VALUE` elements, returns `Integer.MAX_VALUE`.

#### OPTIONAL METHODS

As with the `Collection<T>` interface, the following methods are optional, which means they still must be implemented, but the implementation can simply throw an `UnsupportedOperationException` if for some reason you do not want to give them a “real” implementation. An `UnsupportedOperationException` is a `RuntimeException` and so is not required to be caught or declared in a `throws` clause.

```
public boolean add(T element) (Optional)
```

Adds `element` to the end of the calling object’s list. Normally returns `true`. Returns `false` if the operation failed, but if the operation failed, something is seriously wrong and you will probably get a run-time error anyway.

**Throws:**

`UnsupportedOperationException` if the `add` method is not supported by the calling object.  
`ClassCastException` if the class of `element` prevents it from being added to the calling object.  
`NullPointerException` if `element` is `null` and the calling object does not support `null` elements.

`IllegalArgumentException` if some aspect of `element` prevents it from being added to the calling object.

```
public boolean addAll(Collection<? extends T> collectionToAdd) (Optional)
```

Adds all of the elements in `collectionToAdd` to the end of the calling object's list. The elements are added in the order they are produced by an iterator for `collectionToAdd`.

**Throws:**

`UnsupportedOperationException` if the `addAll` method is not supported by the calling object.  
`ClassCastException` if the class of an element in `collectionToAdd` prevents it from being added to the calling object.

`NullPointerException` if `collectionToAdd` contains one or more `null` elements and the calling object does not support `null` elements, or if `collectionToAdd` is `null`.

`IllegalArgumentException` if some aspect of an element in `collectionToAdd` prevents it from being added to the calling object.

```
public void clear() (Optional)
```

Removes all the elements from the calling object.

**Throws:**

`UnsupportedOperationException` if the `clear` method is not supported by the calling object.

```
public boolean remove(Object element) (Optional)
```

Removes the first occurrence of `element` from the calling object's list, if it is present. Returns `true` if the calling object contained the `element`; returns `false` otherwise.

**Throws:**

`ClassCastException` if the type of `element` is incompatible with the calling object (optional).

`NullPointerException` if `element` is `null` and the calling object does not support `null` elements (optional).

`UnsupportedOperationException` if the `remove` method is not supported by the calling object.

```
public boolean removeAll(Collection<?> collectionToRemove) (Optional)
```

Removes all the calling object's elements that are also in `collectionToRemove`. Returns `true` if the calling object was changed; otherwise returns `false`.

**Throws:**

`UnsupportedOperationException` if the `removeAll` method is not supported by the calling object.

`ClassCastException` if the types of one or more elements in the calling object are incompatible with `collectionToRemove` (optional).

`NullPointerException` if the calling object contains one or more `null` elements and `collectionToRemove` does not support `null` elements (optional).

`NullPointerException` if `collectionToRemove` is `null`.



```
public boolean retainAll(Collection<?> saveElements) (Optional)
```

Retains only the elements in the calling object that are also in the collection `saveElements`. In other words, removes from the calling object all of its elements that are not contained in the collection `saveElements`. Returns `true` if the calling object was changed; otherwise returns `false`.

**Throws:**

`UnsupportedOperationException` if the `retainAll` method is not supported by the calling object.

`ClassCastException` if the types of one or more elements in the calling object are incompatible with `saveElements` (optional).

`NullPointerException` if the calling object contains one or more null elements and `saveElements` does not support null elements (optional).

`NullPointerException` if the collection `saveElements` is null.

#### NEW METHOD HEADINGS

The following methods are in the `List<T>` interface but were not in the `Collection<T>` interface. Those that are optional are noted.

```
public void add(int index, T newElement) (Optional)
```

Inserts `newElement` in the calling object's list at location `index`. The old elements at location `index` and higher are moved to higher indices.

**Throws:**

`IndexOutOfBoundsException` if the `index` is not in the range:

`0 <= index <= size()`.

`UnsupportedOperationException` if this `add` method is not supported by the calling object.  
`ClassCastException` if the class of `newElement` prevents it from being added to the calling object.

`NullPointerException` if `newElement` is null and the calling object does not support null elements.

`IllegalArgumentException` if some aspect of `newElement` prevents it from being added to the calling object.

```
public boolean addAll(int index,
 Collection<? extends T> collectionToAdd) (Optional)
```

Inserts all of the elements in `collectionToAdd` to the calling object's list starting at location `index`. The old elements at location `index` and higher are moved to higher indices. The elements are added in the order they are produced by an iterator for `collectionToAdd`. Returns `true` if successful; otherwise returns `false`.

**Throws:**

`IndexOutOfBoundsException` if the `index` is not in the range:

`0 <= index <= size()`.

`UnsupportedOperationException` if the `addAll` method is not supported by the calling object.

`ClassCastException` if the class of one of the elements of `collectionToAdd` prevents it from being added to the calling object.

`NullPointerException` if `collectionToAdd` contains one or more null elements and the calling object does not support null elements, or if `collectionToAdd` is null.

`IllegalArgumentException` if some aspect of one of the elements of `collectionToAdd` prevents it from being added to the calling object.

```
public int indexOf(Object target)
```

Returns the index of the first element that is equal to `target`. Uses the method `equals` of the object `target` to test for equality. Returns `-1` if `target` is not found.

**Throws:**

`ClassCastException` if the type of `target` is incompatible with the calling object (optional).  
`NullPointerException` if `target` is `null` and the calling object does not support `null` elements (optional).

```
public int lastIndexOf(Object target)
```

Returns the index of the last element that is equal to `target`. Uses the method `equals` of the object `target` to test for equality. Returns `-1` if `target` is not found.

**Throws:**

`ClassCastException` if the type of `target` is incompatible with the calling object (optional).  
`NullPointerException` if `target` is `null` and the calling object does not support `null` elements (optional).

```
public List<T> subList(int fromIndex, int toIndex)
```

Returns a *view* of the elements at locations `fromIndex` to `toIndex` of the calling object; the object at `fromIndex` is included; the object, if any, at `toIndex` is not included. The *view* uses references into the calling object; so, changing the *view* can change the calling object. The returned object will be of type `List<T>` but need not be of the same type as the calling object. Returns an empty `List<T>` if `fromIndex` equals `toIndex`.

**Throws:**

`IndexOutOfBoundsException` if `fromIndex` and `toIndex` do not satisfy:  
`0 <= fromIndex <= toIndex <= size()`.

```
ListIterator<T> listIterator()
```

Returns a list iterator for the calling object. (Iterators are discussed in Section 16.3.)

```
ListIterator<T> listIterator(int index)
```

Returns a list iterator for the calling object starting at `index`. The first element to be returned by the iterator is the one at `index`. (Iterators are discussed in Section 16.3.)

**Throws:**

`IndexOutOfBoundsException` if `index` does not satisfy:  
`0 <= index <= size()`

```
public T get(int index)
```

Returns the object at position `index`.

Throws an `IndexOutOfBoundsException` if the `index` is not in the range:  
`0 <= index < size()`.

```
public T remove(int index) (Optional)
```

Removes the element at position `index` in the calling object. Shifts any subsequent elements to the left (subtracts one from their indices). Returns the element that was removed from the calling object.

**Throws:**

`UnsupportedOperationException` if the `remove` method is not supported by the calling object.

`IndexOutOfBoundsException` if `index` does not satisfy:

```
0 <= index < size()
```

```
public T set(int index, T newElement) (Optional)
```

Sets the element at the specified `index` to `newElement`. The element previously at that position is returned.

**Throws:**

`IndexOutOfBoundsException` if the `index` is not in the range:

```
0 <= index < size().
```

`UnsupportedOperationException` if the `set` method is not supported by the calling object.

`ClassCastException` if the class of `newElement` prevents it from being added to the calling object.

`NullPointerException` if `newElement` is null and the calling object does not support null elements.

`IllegalArgumentException` if some aspect of `newElement` prevents it from being added to the calling object.

## ListIterator<T> Interface

Package: `java.util`

Ancestor interfaces: `Iterator<T>`

The *cursor position* is explained in Chapter 16.

All the exception classes mentioned are the kind that are not required to be caught in a `catch` block or declared in a `throws` clause.

`NoSuchElementException` is in the `java.util` package, which requires an `import` statement if your code mentions the `NoSuchElementException` class. All the other exception classes mentioned are in the package `java.lang` and so do not require any `import` statements.

```
public void add(T newElement) (Optional)
```

Inserts `newElement` at the location of the iterator cursor (that is, before the value, if any, that would be returned by `next()` and after the value, if any, that would be returned by `previous()`). Cannot be used if there has been a call to `add` or `remove` since the last call to `next()` or `previous()`.

**Throws:**

`IllegalStateException` if neither `next()` nor `previous()` has been called, or the `add` or `remove` method has already been called after the last call to `next()` or `previous()`.

`UnsupportedOperationException` if the `remove` operation is not supported by this `Iterator`.

`ClassCastException` if the class of `newElement` prevents it from being added.

`IllegalArgumentException` if some property other than the class of `newElement` prevents it from being added.

```
public boolean hasNext()
```

Returns `true` if there is a suitable element for `next()` to return; returns `false` otherwise.

```
public boolean hasPrevious()
```

Returns `true` if there is a suitable element for `previous()` to return; returns `false` otherwise.

```
public int nextIndex()
```

Returns the index of the element that would be returned by a call to `next()`. Returns the list size if the cursor position is at the end of the list.

```
public T next()
```

Returns the next element of the list that produced the iterator. More specifically, returns the element immediately after the cursor position.

**Throws:**

`NoSuchElementException` if there is no next element.

```
public T previous()
```

Returns the previous element of the list that produced the iterator. More specifically, returns the element immediately before the cursor position.

**Throws:**

`NoSuchElementException` if there is no previous element.

```
public int previousIndex()
```

Returns the index that would be returned by a call to `previous()`. Returns `-1` if the cursor position is at the beginning of the list.

```
public void remove() (Optional)
```

Removes from the collection the last element returned by `next()` or `previous()`.

This method can be called only once per call to `next()` or `previous()`.

Cannot be used if there has been a call to `add` or `remove` since the last call to `next()` or `previous()`.

**Throws:**

`IllegalStateException` if neither `next()` nor `previous()` has been called, or the `add` or `remove` method has already been called after the last call to `next()` or `previous()`.

`UnsupportedOperationException` if the `remove` operation is not supported by this `Iterator`.

```
public void set(T newElement) (Optional)
```

Replaces the last element returned by `next()` or `previous()` with `newElement`. Cannot be used if there has been a call to `add` or `remove` since the last call to `next()` or `previous()`.

#### Throws:

`UnsupportedOperationException` if the set operation is not supported by this `Iterator`.  
`IllegalStateException` if neither `next()` nor `previous()` has been called, or the `add` or `remove` method has been called since the last call to `next()` or `previous()`.  
`ClassCastException` if the class of `newElement` prevents it from being added.  
`IllegalArgumentException` if some property other than the class of `newElement` prevents it from being added.

## Long

This is a wrapper class for `long`. See Section 5.1 in Chapter 5.

## Math

Package: `java.lang`

```
Object
|
+--Math
```

The `Math` class is marked `final`, which means it cannot be used as a base class to derive other classes.

```
public static double abs(double argument)
public static float abs(float argument)
public static long abs(long argument)
public static int abs(int argument)
```

Returns the absolute value of the argument. (The method name `abs` is overloaded to produce four similar methods.)

#### EXAMPLES

`Math.abs(-6)` and `Math.abs(6)` both return 6. `Math.abs(-5.5)` and `Math.abs(5.5)` both return 5.5.

```
public static double ceil(double argument)
```

Returns the smallest whole number greater than or equal to the argument.

#### EXAMPLE

`Math.ceil(3.2)` and `Math.ceil(3.9)` both return 4.0.

```
public static double floor(double argument)
```

Returns the largest whole number less than or equal to the argument.

#### EXAMPLE

`Math.floor(3.2)` and `Math.floor(3.9)` both return 3.0.

```
public static double max(double n1, double n2)
public static float max(float n1, float n2)
public static long max(long n1, long n2)
public static int max(int n1, int n2)
```

Returns the maximum of the arguments n1 and n2. (The method name max is overloaded to produce four similar methods.)

**EXAMPLE**

Math.max(3, 2) returns 3.

```
public static double min(double n1, double n2)
public static float min(float n1, float n2)
public static long min(long n1, long n2)
public static int min(int n1, int n2)
```

Returns the minimum of the arguments n1 and n2. (The method name min is overloaded to produce four similar methods.)

**EXAMPLE**

Math.min(3, 2) returns 2.

```
public static double pow(double base, double exponent)
```

Returns base to the power exponent.

**EXAMPLE**

Math.pow(2.0, 3.0) returns 8.0.

```
public static double random()
```

Returns a random number greater than or equal to 0.0 and less than 1.0.

**EXAMPLE**

Math.random() returns 0.5505562535943004 (example value only; will return a pseudo-random number that is less than 1 and greater than or equal to 0 the next time the statement is executed).

```
public static long round(double argument)
public static int round(float argument)
```

Rounds its argument.

**EXAMPLES**

Math.round(3.2) returns 3. Math.round(3.6) returns 4.

```
public static double sqrt(double argument)
```

Returns the square root of its argument.

**EXAMPLE**

Math.sqrt(4) returns 2.0.

## Map<K, V> Interface

Package: `java.util`

Ancestor interfaces: none

All the exception classes mentioned are unchecked exceptions, which means they are not required to be caught in a `catch` block or declared in a `throws` clause. No `import` statement is required because these exception classes are in the package `java.lang`.

### CONSTRUCTORS

Although not officially required by the interface, any class that implements the `Map<K, V>` interface should have at least two constructors: a no-argument constructor that creates an empty `Map<K, V>` object, and a constructor with one `Map<K, V>` parameter that creates a `Map<K, V>` object with the same elements as the constructor argument. The interface does not specify whether the copy produced by the one-argument constructor is a shallow copy or a deep copy of its argument.

### METHODS

```
public boolean containsKey(Object key)
```

Returns `true` if the calling object contains `key` as one of its keys.

#### Throws:

`ClassCastException` if the type of `key` is incompatible with the type for this map (optional).

`NullPointerException` if the key is `null` and this map does not permit `null` keys (optional).

```
public boolean containsValue(Object value)
```

Returns `true` if the calling object contains one or more keys that map to an instance of `value`.

#### Throws:

`ClassCastException` if the type of `value` is incompatible with the type for this map (optional).

`NullPointerException` if the value is `null` and this map does not permit `null` values (optional).

```
public Set<Map.Entry<K, V>> entrySet()
```

Returns a set view consisting of (key, value) mappings for all entries in the map. Changes to the map are reflected in the set and vice-versa.

```
public boolean equals(Object other)
```

This is the `equals` of the map, not the `equals` of the elements in the map. Overrides the inherited method `equals`.

```
public V get(Object key)
```

Returns the value onto which the calling object maps `key`. If `key` is not in the map, then `null` is returned. Note that this does not always mean that the key is not in the map, because it is possible to map a key to `null`. The `containsKey` method can be used to distinguish the two cases.

#### Throws:

`ClassCastException` if the type of `key` is incompatible with the type for this map (optional).

`NullPointerException` if the key is `null` and this map does not permit `null` keys (optional).

```
public int hashCode()
```

Returns the hash code value for the calling object. The hash code of a map is defined to be the sum of the `hashCode`s of each entry in the map's `entrySet` view.

```
public boolean isEmpty()
```

Returns `true` if the calling object is empty; otherwise returns `false`.

```
public int size()
```

Returns the number of (key, value) mappings in the calling object.

```
public Collection<V> values()
```

Returns a collection view consisting of all values in the map. Changes to the map are reflected in the collection and vice-versa.

#### OPTIONAL METHODS

The following methods are optional, which means they still must be implemented, but the implementation can simply throw an `UnsupportedOperationException` if, for some reason, you do not want to give them a “real” implementation. An `UnsupportedOperationException` is a `RuntimeException` and so is not required to be caught or declared in a `throws` clause.

```
public V put(K key, V value) (Optional)
```

Associates `key` to `value` in the map. If `key` was associated with an existing value, then the old value is overwritten and returned. Otherwise `null` is returned.

##### Throws:

`ClassCastException` if the type of `key` or `value` is incompatible with the type for this map (optional).

`NullPointerException` if the `key` or `value` is `null` and this map does not permit `null` keys or values (optional).

`IllegalArgumentException` if some aspect of the `key` or `value` prevents it from being stored in this map (optional).

`UnsupportedOperationException` if the `put` operation is not supported by this map (optional).

```
public void putAll(Map<? extends K, ? extends V> mapToAdd) (Optional)
```

Adds all mappings of `mapToAdd` into the calling object's map.

##### Throws:

`ClassCastException` if any type of `key` or `value` of `mapToAdd` is incompatible with the type for this map (optional).

`NullPointerException` if `mapToAdd` is `null` or any `key` or `value` of `mapToAdd` is `null` and this map does not permit `null` keys or values (optional).

`IllegalArgumentException` if some aspect of the `key` or `value` from `mapToAdd` prevents it from being stored in this map (optional).

`UnsupportedOperationException` if the `putAll` operation is not supported by this map (optional).



```
public V remove(Object key) (Optional)
```

Removes the mapping for the specified key. If the key is not found in the map, then `null` is returned, otherwise the previous value for the key is returned.

**Throws:**

`ClassCastException` if the type of key is incompatible with the type for this map (optional).  
`NullPointerException` if the key is `null` and this map does not permit `null` keys (optional).  
`UnsupportedOperationException` if the remove operation is not supported by this map (optional).

## ObjectInputStream

Package: `java.io`

The `FileInputStream` class is also in this package.

Ancestor classes:

```

Object
|
+--InputStream
|
+--ObjectInputStream

```

```
public ObjectInputStream(InputStream streamObject)
```

There is no constructor that takes a file name as an argument. If you want to create a stream using a file name, use

```
new ObjectInputStream(new FileInputStream(File_Name))
```

Alternatively, you can use an object of the class `File` in place of the `File_Name`, as follows:

```
new ObjectInputStream(new FileInputStream(File_Object))
```

The constructor for `FileInputStream` may throw a `FileNotFoundException`, which is a kind of `IOException`. If the `FileInputStream` constructor succeeds, then the constructor for `ObjectInputStream` may throw a different `IOException`.

```
public void close() throws IOException
```

Closes the stream's connection to a file.

```
public boolean readBoolean() throws IOException
```

Reads a `boolean` value from the input stream and returns that `boolean` value. If `readBoolean` tries to read a value from the file and that value was not written using the method `writeBoolean` of the class `ObjectOutputStream` (or written in some equivalent way), then problems will occur.

If an attempt is made to read beyond the end of the file, an `EOFException` is thrown.

```
public char readChar() throws IOException
```

Reads a char value from the input stream and returns that char value. If readChar tries to read a value from the file and that value was not written using the method writeChar of the class ObjectOutputStream (or written in some equivalent way), then problems will occur.

If an attempt is made to read beyond the end of the file, an EOFException is thrown.

```
public double readDouble() throws IOException
```

Reads a double value from the input stream and returns that double value. If readDouble tries to read a value from the file and that value was not written using the method writeDouble of the class ObjectOutputStream (or written in some equivalent way), then problems will occur.

If an attempt is made to read beyond the end of the file, an EOFException is thrown.

```
public float readFloat() throws IOException
```

Reads a float value from the input stream and returns that float value. If readFloat tries to read a value from the file and that value was not written using the method writeFloat of the class ObjectOutputStream (or written in some equivalent way), then problems will occur.

If an attempt is made to read beyond the end of the file, an EOFException is thrown.

```
public int readInt() throws IOException
```

Reads an int value from the input stream and returns that int value. If readInt tries to read a value from the file and that value was not written using the method writeInt of the class ObjectOutputStream (or written in some equivalent way), then problems will occur.

If an attempt is made to read beyond the end of the file, an EOFException is thrown.

```
public long readLong() throws IOException
```

Reads a long value from the input stream and returns that long value. If readLong tries to read a value from the file and that value was not written using the method writeLong of the class ObjectOutputStream (or written in some equivalent way), then problems will occur.

If an attempt is made to read beyond the end of the file, an EOFException is thrown.

```
Object readObject() throws ClassNotFoundException, IOException
```

Reads an object from the input stream. The object read should have been written using writeObject of the class ObjectOutputStream.

**Throws:**

ClassNotFoundException if the class of a serialized object cannot be found.

If an attempt is made to read beyond the end of the file, an EOFException is thrown.

May throw various other IOExceptions.

```
public int readShort() throws IOException
```

Reads a short value from the input stream and returns that short value. If `readInt` tries to read a value from the file and that value was not written using the method `writeShort` of the class `ObjectOutputStream` (or written in some equivalent way), then problems will occur.

If an attempt is made to read beyond the end of the file, an `EOFException` is thrown.

```
public String readUTF() throws IOException
```

Reads a `String` value from the input stream and returns that `String` value. If `readUTF` tries to read a value from the file and that value was not written using the method `writeUTF` of the class `ObjectOutputStream` (or written in some equivalent way), then problems will occur.

If an attempt is made to read beyond the end of the file, an `EOFException` is thrown.

```
public int skipBytes(int n) throws IOException
```

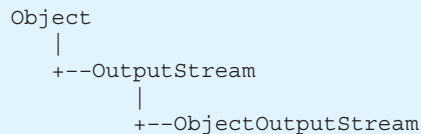
Skips `n` bytes.

## ObjectOutputStream

Package: `java.io`

The `FileOutputStream` class is also in this package.

Ancestor classes:



```
public ObjectOutputStream(OutputStream streamObject)
```

There is no constructor that takes a file name as an argument. If you want to create a stream using a file name, use

```
new ObjectOutputStream(new FileOutputStream(File_Name))
```

This creates a blank file. If there already is a file named *File\_Name*, then the old contents of the file are lost.

If you want to create a stream using an object of the class `File`, use

```
new ObjectOutputStream(new FileOutputStream(File_Object))
```

The constructor for `FileOutputStream` may throw a `FileNotFoundException`, which is a kind of `IOException`. If the `FileOutputStream` constructor succeeds, then the constructor for `ObjectOutputStream` may throw a different `IOException`.

```
public void close() throws IOException
```

Closes the stream's connection to a file. This method calls `flush` before closing the file.

```
public void flush() throws IOException
```

Flushes the output stream. This forces an actual physical write to the file of any data that has been buffered and not yet physically written to the file. Normally, you should not need to invoke `flush`.

```
public void writeBoolean(boolean b) throws IOException
```

Writes the boolean value `b` to the output stream.

```
public void writeChar(int n) throws IOException
```

Writes the char value `n` to the output stream. Note that it expects its argument to be an `int` value. However, if you simply use the char value, then Java will automatically type cast it to an `int` value. The following are equivalent:

```
outputStream.writeChar((int) 'A');
```

and

```
outputStream.writeChar('A');
```

```
public void writeDouble(double x) throws IOException
```

Writes the double value `x` to the output stream.

```
public void writeFloat(float x) throws IOException
```

Writes the float value `x` to the output stream.

```
public void writeInt(int n) throws IOException
```

Writes the int value `n` to the output stream.

```
public void writeLong(long n) throws IOException
```

Writes the long value `n` to the output stream.

```
public void writeObject(Object anObject) throws IOException
```

Writes its argument to the output stream. The object argument should be an object of a serializable class, a concept discussed in Chapter 10.

**Throws:**

Various `IOExceptions`.

```
public void writeShort(short n) throws IOException
```

Writes the short value `n` to the output stream.

```
public void writeUTF(String aString) throws IOException
```

Writes the `String` value `aString` to the output stream. UTF refers to a particular method of encoding the string. To read the string back from the file, you should use the method `readUTF` of the class `ObjectInputStream`.

## PrintWriter

Package: `java.io`

The `FileOutputStream` class is also in this package.

Ancestor classes:

```

Object
|
+--Writer
 |
 +--PrintWriter

```

```
public PrintWriter(OutputStream streamObject)
```

This is the only constructor you are likely to need. There is no constructor that accepts a file name as an argument. If you want to create a stream using a file name, use

```
new PrintWriter (new FileOutputStream (File_Name))
```

When the constructor is used in this way, a blank file is created. If there already was a file named *File\_Name*, then the old contents of the file are lost. If you want instead to append new text to the end of the old file contents, use

```
new PrintWriter(new FileOutputStream (File_Name, true))
```

(For an explanation of the argument `true`, see Chapter 10.)

When used in either of these ways, the `FileOutputStream` constructor, and so the `PrintWriter` constructor invocation, can throw a `FileNotFoundException`, which is a kind of `IOException`. If you want to create a stream using an object of the class `File`, you can use a `File` object in place of the *File\_Name*.

```
public void close()
```

Closes the stream's connection to a file. This method calls `flush` before closing the file.

```
public void flush()
```

Flushes the output stream. This forces an actual physical write to the file of any data that has been buffered and not yet physically written to the file. Normally, you should not need to invoke `flush`.

```
public final void print (Argument)
```

Same as `println`, except that this method does not end the line, and so the next output will be on the same line.

```
public final void println (Argument)
```

The *Argument* can be a string, character, integer, floating-point number, boolean value, or any combination of these, connected with `+` signs. The *Argument* can also be any object, although it will not work as desired unless the object has a properly defined `toString()` method. The *Argument* is output to the file connected to the stream. After the *Argument* has been output, the line ends, and so the next output is sent to the next line.

## Random

Package: `java.util`

Ancestor classes:

```
Object
|
+--Random
```

```
public Random()
```

Creates a new random number generator.

```
public Random(long seed)
```

Creates a new random number generator with the specified seed value.

```
public int nextInt(int n)
```

Returns a pseudorandom, uniformly distributed `int` value between 0 (inclusive) and the value `n` (exclusive).

```
public double nextDouble(double n)
```

Returns a pseudorandom, uniformly distributed `double` value between 0 (inclusive) and 1 (exclusive).

## RandomAccessFile

Package: `java.io`

```
Object
|
+--RandomAccessFile
```

```
public RandomAccessFile(String fileName, String mode)
```

```
public RandomAccessFile(File fileObject, String mode)
```

Opens the file, does not delete data already in the file, but does position the file pointer at the first (zeroth) location.

The mode must be one of the following:

"r" Open for reading only.

"rw" Open for reading and writing.

"rws" Same as "rw", and also requires that every update to the file's content or metadata be written synchronously to the underlying storage device.

"rwd" Same as "rw", and also requires that every update to the file's content be written synchronously to the underlying storage device.

"rws" and "rwd" are not covered in this book.

```
public void close() throws IOException
```

Closes the stream's connection to a file.

```
public void setLength(long newLength) throws IOException
```

Sets the length of this file.

If the present length of the file as returned by the `length` method is greater than the `newLength` argument, then the file will be truncated. In this case, if the file pointer location as returned by the `getFilePointer` method is greater than `newLength`, then after this method returns, the file pointer location will be equal to `newLength`.

If the present length of the file as returned by the `length` method is smaller than `newLength`, then the file will be extended. In this case, the contents of the extended portion of the file are not defined.

```
public long getFilePointer() throws IOException
```

Returns the current location of the file pointer. Locations are numbered starting with 0.

```
public long length() throws IOException
```

Returns the length of the file.

```
public int read() throws IOException
```

Reads a byte of data from the file and returns it as an integer in the range 0 to 255.

```
public int read(byte[] a) throws IOException
```

Reads up to `a.length` bytes of data from the file into the array of bytes. Returns the total number of bytes read or -1 if the end of the file is reached.

```
public final boolean readBoolean() throws IOException
```

Reads a boolean value from the file and returns that value.

If an attempt is made to read beyond the end of the file, an `EOFException` is thrown.

```
public final byte readByte() throws IOException
```

Reads a byte value from the file and returns that value.

If an attempt is made to read beyond the end of the file, an `EOFException` is thrown.

```
public final char readChar() throws IOException
```

Reads a char value from the file and returns that value.

If an attempt is made to read beyond the end of the file, an `EOFException` is thrown.

```
public final double readDouble() throws IOException
```

Reads a double value from the file and returns that value.

If an attempt is made to read beyond the end of the file, an `EOFException` is thrown.

```
public final float readFloat() throws IOException
```

Reads a float value from the file and returns that value.

If an attempt is made to read beyond the end of the file, an `EOFException` is thrown.

```
public final int readInt() throws IOException
```

Reads an int value from the file and returns that value.

If an attempt is made to read beyond the end of the file, an EOFException is thrown.

```
public final long readLong() throws IOException
```

Reads a long value from the file and returns that value.

If an attempt is made to read beyond the end of the file, an EOFException is thrown.

```
public final short readShort() throws IOException
```

Reads a short value from the file and returns that value.

If an attempt is made to read beyond the end of the file, an EOFException is thrown.

```
public final String readUTF() throws IOException
```

Reads a String value from the file and returns that value.

If an attempt is made to read beyond the end of the file, an EOFException is thrown.

```
public void seek(long location) throws IOException
```

Moves the file pointer to the specified location.

```
public void write(byte[] a) throws IOException
```

Writes a.length bytes from the specified byte array to the file.

```
public void write(int b) throws IOException
```

Writes the specified byte to the file.

```
public final void writeBoolean(boolean b) throws IOException
```

Writes the boolean b to the file.

```
public final void writeByte(byte b) throws IOException
```

Writes the byte b to the file.

```
public final void writeChar(char c) throws IOException
```

Writes the char c to the file.

```
public final void writeDouble(double d) throws IOException
```

Writes the double d to the file.

```
public final void writeFloat(float f) throws IOException
```

Writes the float f to the file.

```
public final void writeInt(int n) throws IOException
```

Writes the int n to the file.



```
public final void writeLong(long n) throws IOException
```

Writes the long `n` to the file.

```
public final void writeShort(short n) throws IOException
```

Writes the short `n` to the file.

```
public final void writeUTF(String s) throws IOException
```

Writes the String `s` to the file.

## Scanner

Package: `java.util`

Ancestor classes:

```
Object
|
+--Scanner
```

The `Scanner` class can be used to obtain input from files as well as from the keyboard. Values to be read should be separated by whitespace characters, such as blanks and/or new lines. When reading values, these whitespace characters are skipped. (It is possible to change the separators from whitespace to something else, but whitespace is the default.)

```
public Scanner(InputStream streamObject)
```

There is no constructor that accepts a file name as an argument. If you want to create a stream using a file name, you can use

```
new Scanner(new FileInputStream(File_Name))
```

When used in this way, the `FileInputStream` constructor, and thus the `Scanner` constructor invocation, can throw a `FileNotFoundException`, which is a kind of `IOException`. To create a stream connected to the keyboard, use

```
new Scanner(System.in)
```

```
public Scanner(File fileObject)
```

If you want to create a stream using a file name, you can use

```
new Scanner(new File(File_Name))
```

```
public int nextInt()
```

Returns the next token as an `int`, provided the next token is a well-formed string representation of an `int`.

### Throws:

`NoSuchElementException` if there are no more tokens.

`InputMismatchException` if the next token is not a well-formed string representation of an `int`.

`IllegalStateException` if the `Scanner` stream is closed.

```
public boolean hasNextInt()
```

Returns `true` if the next token is a well-formed string representation of an `int`; otherwise returns `false`.

**Throws:**

`IllegalStateException` if the `Scanner` stream is closed.

```
public long nextLong()
```

Returns the next token as a `long`, provided the next token is a well-formed string representation of a `long`.

**Throws:**

`NoSuchElementException` if there are no more tokens.

`InputMismatchException` if the next token is not a well-formed string representation of a `long`.

`IllegalStateException` if the `Scanner` stream is closed.

```
public boolean hasNextLong()
```

Returns `true` if the next token is a well-formed string representation of a `long`; otherwise returns `false`.

**Throws:**

`IllegalStateException` if the `Scanner` stream is closed.

```
public byte nextByte()
```

Returns the next token as a `byte`, provided the next token is a well-formed string representation of a `byte`.

**Throws:**

`NoSuchElementException` if there are no more tokens.

`InputMismatchException` if the next token is not a well-formed string representation of a `byte`.

`IllegalStateException` if the `Scanner` stream is closed.

```
public boolean hasNextByte()
```

Returns `true` if the next token is a well-formed string representation of a `byte`; otherwise returns `false`.

**Throws:**

`IllegalStateException` if the `Scanner` stream is closed.

```
public short nextShort()
```

Returns the next token as a `short`, provided the next token is a well-formed string representation of a `short`.

**Throws:**

`NoSuchElementException` if there are no more tokens.

`InputMismatchException` if the next token is not a well-formed string representation of a `short`.

`IllegalStateException` if the `Scanner` stream is closed.

```
public boolean hasNextShort ()
```

Returns true if the next token is a well-formed string representation of a short; otherwise returns false.

**Throws:**

IllegalStateException if the Scanner stream is closed.

```
public double nextDouble ()
```

Returns the next token as a double, provided the next token is a well-formed string representation of a double.

**Throws:**

NoSuchElementException if there are no more tokens.

InputMismatchException if the next token is not a well-formed string representation of a double.

IllegalStateException if the Scanner stream is closed.

```
public boolean hasNextDouble ()
```

Returns true if the next token is a well-formed string representation of a double; otherwise returns false.

**Throws:**

IllegalStateException if the Scanner stream is closed.

```
public float nextFloat ()
```

Returns the next token as a float, provided the next token is a well-formed string representation of a float.

**Throws:**

NoSuchElementException if there are no more tokens.

InputMismatchException if the next token is not a well-formed string representation of a float.

IllegalStateException if the Scanner stream is closed.

```
public boolean hasNextFloat ()
```

Returns true if the next token is a well-formed string representation of a float; otherwise returns false.

**Throws:**

IllegalStateException if the Scanner stream is closed.

```
public String next ()
```

Returns the next token.

**Throws:**

NoSuchElementException if there are no more tokens.

IllegalStateException if the Scanner stream is closed.

```
public boolean hasNext()
```

Returns `true` if there is another token. May wait for a next token to enter the stream.

**Throws:**

`IllegalStateException` if the `Scanner` stream is closed.

```
public boolean nextBoolean()
```

Returns the next token as a `boolean` value, provided the next token is a well-formed string representation of a `boolean`.

**Throws:**

`NoSuchElementException` if there are no more tokens.

`InputMismatchException` if the next token is not a well-formed string representation of a `boolean` value.

`IllegalStateException` if the `Scanner` stream is closed.

```
public boolean hasNextBoolean()
```

Returns `true` if the next token is a well-formed string representation of a `boolean` value; otherwise returns `false`.

**Throws:**

`IllegalStateException` if the `Scanner` stream is closed.

```
public String nextLine()
```

Returns the rest of the current input line. Note that the line terminator '`\n`' is read and discarded; it is not included in the string returned.

**Throws:**

`NoSuchElementException` if there are no more lines.

`IllegalStateException` if the `Scanner` stream is closed.

```
public boolean hasNextLine()
```

Returns `true` if there is a next line. May wait for a next line to enter the stream.

**Throws:**

`IllegalStateException` if the `Scanner` stream is closed.

```
public Scanner useDelimiter(String newDelimiter);
```

Changes the delimiter for input so that `newDelimiter` will be the only delimiter that separates words or numbers. See the subsection "Other Input Delimiters" in Chapter 2 for the details. (You can use this method to set the delimiters to a more complex pattern than just a single string, but we are not covering that.)

Returns the calling object, but we have always used it as a `void` method.

## Serializable Interface

See Section 10.4 in Chapter 10.

## Set<T> Interface

Package: `java.util`

Ancestor interfaces: `Collection <T>`

All the exception classes mentioned are the kind that are not required to be caught in a `catch` block or declared in a `throws` clause.

All the exception classes mentioned are in the package `java.lang` and so do not require any `import` statement.

### CONSTRUCTORS

```
public boolean contains(Object target)
```

Returns `true` if the calling object contains at least one instance of `target`. Uses `target.equals` to determine if `target` is in the calling object.

#### Throws:

`ClassCastException` if the type of `target` is incompatible with the calling object (optional).

`NullPointerException` if `target` is `null` and the calling object does not support `null` elements (optional).

```
public boolean containsAll(Collection<?> collectionOfTargets)
```

Returns `true` if the calling object contains all of the elements in `collectionOfTargets`. For `element` in `collectionOfTargets`, this method uses `element.equals` to determine if `element` is in the calling object. If `collectionOfTargets` is itself a `Set <T>`, this is a test to see if `collectionOfTargets` is a subset of the calling object.

#### Throws:

`ClassCastException` if the types of one or more elements in `collectionOfTargets` are incompatible with the calling object (optional).

`NullPointerException` if `collectionOfTargets` contains one or more `null` elements and the calling object does not support `null` elements (optional).

`NullPointerException` if `collectionOfTargets` is `null`.

```
public boolean equals(Object other)
```

If the argument is a `Set <T>`, returns `true` if the calling object and the argument contain exactly the same elements; otherwise returns `false`. If the argument is not a `Set <T>`, `false` is returned.

```
public int hashCode()
```

Returns the hash code value for the calling object. Neither hash codes nor this method are discussed in this book. This entry is only here to make the definition of the `Set <T>` interface complete. You can safely ignore this entry until you go on to study hash codes in a more advanced book. In the meantime, if you need to implement this method, have it throw an `UnsupportedOperationException`.

```
public boolean isEmpty()
```

Returns `true` if the calling object is empty; otherwise returns `false`.

```
Iterator <T> iterator()
```

Returns an iterator for the calling object. (Iterators are discussed in Section 16.3.)

```
public Object [] toArray()
```

Returns an array containing all of the elements in the calling object. A new array should be returned so that the calling object has no references to the returned array.

```
public <E> E[] toArray(E[] a)
```

Note that the type parameter *E* is not the same as *T*. So, *E* can be any reference type; it need not be the type *T* in `Collection <T>`. For example, *E* might be an ancestor type of *T*. Returns an array containing all of the elements in the calling object. The argument *a* is used primarily to specify the type of the array returned. The exact details are described in the table for the `Collection <T>` interface.

#### Throws:

`ArrayStoreException` if the type of *a* is not an ancestor type of the type of every element in the calling object.

`NullPointerException` if *a* is `null`.

```
public int size()
```

Returns the number of elements in the calling object. If the calling object contains more than `Integer.MAX_VALUE` elements, returns `Integer.MAX_VALUE`.

### ADDING AND REMOVING ELEMENTS

Although many are optional, the following methods are almost always implemented for classes that implement the `Set <T>` interface.

```
public boolean add(T element) (Optional)
```

If *element* is not already in the calling object, *element* is added to the calling object and `true` is returned. If *element* is in the calling object, the calling object is unchanged and `false` is returned.

#### Throws:

`UnsupportedOperationException` if the `add` method is not supported by the set.

`ClassCastException` if the class of *element* prevents it from being added to the set.

`NullPointerException` if *element* is `null` and the set does not support `null` elements.

`IllegalArgumentException` if some other aspect of *element* prevents it from being added to this set.

```
public boolean addAll(Collection<? extends T> collectionToAdd) (Optional)
```

Ensures that the calling object contains all the elements in *collectionToAdd*. Returns `true` if the calling object changed as a result of the call; returns `false` otherwise. Thus, if *collectionToAdd* is a `Set <T>`, then the calling object is changed to the union of itself with *collectionToAdd*.

#### Throws:

`UnsupportedOperationException` if the `addAll` method is not supported by the set.

`ClassCastException` if the class of some element of *collectionToAdd* prevents it from being added to the calling object.

`NullPointerException` if *collectionToAdd* contains one or more `null` elements and the calling object does not support `null` elements, or if *collectionToAdd* is `null`.

`IllegalArgumentException` if some aspect of some element of *collectionToAdd* prevents it from being added to the calling object.

```
public void clear()
```

Removes all the elements from the calling object.

**Throws:**

`UnsupportedOperationException` if the `clear` method is not supported by the calling object.

```
public boolean remove(Object element)
```

Removes the `element` from the calling object, if it is present. Returns `true` if the calling object contained the `element`; returns `false` otherwise.

**Throws:**

`ClassCastException` if the type of `element` is incompatible with the calling object (optional).  
`NullPointerException` if `element` is `null` and the calling object does not support `null` elements (optional).

`UnsupportedOperationException` if the `remove` method is not supported by the calling object.

```
public boolean removeAll(Collection<?> collectionToRemove) (Optional)
```

Removes all the calling object's elements that are also contained in `collectionToRemove`. Returns `true` if the calling object was changed; otherwise returns `false`.

**Throws:**

`UnsupportedOperationException` if the `removeAll` method is not supported by the calling object.

`ClassCastException` if the types of one or more elements in `collectionToRemove` are incompatible with the calling object (optional).

`NullPointerException` if the calling object contains a `null` element and `collectionToRemove` does not support `null` elements (optional).

`NullPointerException` if `collectionToRemove` is `null`.

```
public boolean retainAll(Collection<?> saveElements) (Optional)
```

Retains only the elements in the calling object that are also contained in the collection `saveElements`. In other words, removes from the calling object all of its elements that are not contained in the collection `saveElements`. Returns `true` if the calling object was changed; otherwise returns `false`. If the argument is itself a `Set <T>`, this changes the calling object to the intersection of itself with the argument.

**Throws:**

`UnsupportedOperationException` if the `retainAll` method is not supported by the calling object.

`ClassCastException` if the types of one or more elements in the calling object are incompatible with `saveElements` (optional).

`NullPointerException` if `saveElements` contains a `null` element and the calling object does not support `null` elements (optional).

`NullPointerException` if `saveElements` is `null`.

## Short

Wrapper class for `short`. See Section 5.1 in Chapter 5.

## String

Package: `java.lang`

`String` is marked `final` and so you cannot use it as a base class to derive another class.

Implements interfaces: `CharSequence`, `Comparable`, `Serializable`

Ancestor classes:

```
Object
|
+---String
```

### CONSTRUCTORS

```
public String()
```

Creates a `String` object that represents an empty character sequence. Note that this is a pretty useless constructor because `String` objects are immutable.

```
public String(BufferedString buffer)
```

Creates a new `String` object that contains the same sequence of characters that is currently contained in the `BufferedString` argument. This is a deep copy; subsequent modification of the `BufferedString` object does not affect the newly created string.

#### Throws:

`NullPointerException` if `buffer` is `null`.

```
public String(char[] value, int offset, int count)
```

Creates a new `String` that contains characters from a subarray of the character array argument. The `offset` argument is the index of the first character of the subarray, and the `count` argument specifies the length of the subarray. The contents of the subarray are copied. This is a deep copy; subsequent modifications of the character array do not affect the newly created string.

#### Throws:

`IndexOutOfBoundsException` if the elements specified by `offset` and `count` are not all within the bounds of the value array.

`NullPointerException` if `value` is `null`.

```
public String(String original)
```

Creates a new `String` object so that it represents the same sequence of characters as the argument. Unless an explicit copy of `original` is needed, use of this constructor is unnecessary because `String` objects are immutable.

#### Throws:

`NullPointerException` if `original` is `null`.



## METHODS

```
public char charAt(int position)
```

Returns the character in the calling object string at the `position`. Positions are counted 0, 1, 2, etc.

**EXAMPLE**

After program executes `String greeting = "Hello!"`;  
`greeting.charAt(0)` returns 'H', and  
`greeting.charAt(1)` returns 'e'.

**Throws:**

`IndexOutOfBoundsException` if `position` is negative or not less than the length of the calling object string.

```
public int compareTo(String aString)
```

Compares the calling object string and the string argument to see which comes first in the lexicographic ordering. Lexicographic order is the same as alphabetical order but with the characters ordered as in Appendix 3. Note that in Appendix 3 all the uppercase letters are in regular alphabetical order and all the lowercase letters are in alphabetical order, but all the uppercase letters precede all the lowercase letters. So, lexicographic ordering is the same as alphabetical ordering when either both strings are all uppercase letters or both strings are all lowercase letters. If the calling string is first, it returns a negative value. If the two strings are equal, it returns zero. If the argument is first, it returns a positive number.

**EXAMPLE**

After program executes `String entry = "adventure"`;  
`entry.compareTo("zoo")` returns a negative number,  
`entry.compareTo("adventure")` returns 0, and  
`entry.compareTo("above")` returns a positive number.

**Throws:**

`NullPointerException` if `aString` is null.

```
public int compareToIgnoreCase(String aString)
```

Compares the calling object string and the string argument to see which comes first in the lexicographic ordering, treating upper- and lowercase letters as being the same. (To be precise, all uppercase letters are treated as if they were their lowercase versions in doing the comparison.) Thus, if both strings consist entirely of letters, the comparison is for ordinary alphabetical order. If the calling string is first, it returns a negative value. If the two strings are equal, ignoring cases, it returns zero. If the argument is first, it returns a positive number.

**EXAMPLE**

After program executes `String entry = "adventure"`;  
`entry.compareToIgnoreCase("Zoo")` returns a negative number,  
`entry.compareToIgnoreCase("Adventure")` returns 0, and  
`"Zoo".compareToIgnoreCase(entry)` returns a positive number.

**Throws:**

`NullPointerException` if `aString` is null.

```
public boolean contentEquals(StringBuffer stringBufferObject)
```

Returns true if and only if this String represents the same sequence of characters as the StringBuffer argument.

**Throws:**

NullPointerException if stringBufferObject is null.

```
public boolean equals(String otherString)
```

Returns true if the calling object string and the otherString are equal. Otherwise returns false.

**EXAMPLE**

After program executes `String greeting = "Hello";`  
`greeting.equals("Hello")` returns `true`  
`greeting.equals("Good-Bye")` returns `false`  
`greeting.equals("hello")` returns `false`

Note that case matters: "Hello" and "hello" are not equal because one starts with an uppercase letter and the other starts with a lowercase letter.

```
public boolean equalsIgnoreCase(String otherString)
```

Returns true if the calling object string and the otherString are equal, considering upper- and lowercase versions of a letter to be the same. Otherwise returns false.

**EXAMPLE**

After program executes `String name = "mary";`  
`name.equalsIgnoreCase("Mary")` returns `true`.

```
public int indexOf(String aString)
```

Returns the index (position) of the first occurrence of the string aString in the calling object string. Positions are counted 0, 1, 2, etc. Returns -1 if aString is not found.

**EXAMPLE**

After program executes `String greeting = "Hi Mary!";`  
`greeting.indexOf("Mary")` returns 3, and  
`greeting.indexOf("Sally")` returns -1.

**Throws:**

NullPointerException if aString is null.

```
public int indexOf(String aString, int start)
```

Returns the index (position) of the first occurrence of the string `aString` in the calling object string that occurs at or after position `start`. Positions are counted 0, 1, 2, etc. Returns `-1` if `aString` is not found.

**EXAMPLE**

After program executes `String name = "Mary, Mary quite contrary";`  
`name.indexOf("Mary", 1)` returns 6.  
The same value is returned if 1 is replaced by any number up to and including 6.  
`name.indexOf("Mary", 0)` returns 0.  
`name.indexOf("Mary", 8)` returns `-1`.

**Throws:**

`NullPointerException` if `aString` is null.

```
public int lastIndexOf(String aString)
```

Returns the index (position) of the last occurrence of the string `aString` in the calling object string. Positions are counted 0, 1, 2, etc. Returns `-1` if `aString` is not found.

**EXAMPLE**

After program executes `String name = "Mary, Mary, Mary quite so";`  
`greeting.indexOf("Mary")` returns 0, and  
`name.lastIndexOf("Mary")` returns 12.

**Throws:**

`NullPointerException` if `aString` is null.

```
public int length()
```

Returns the length of the calling object (which is a string) as a value of type `int`.

**EXAMPLE**

After program executes `String greeting = "Hello!";`  
`greeting.length()` returns 6.

```
public String substring(int start)
```

Returns the substring of the calling object string starting from `start` through to the end of the calling object. Positions are counted 0, 1, 2, etc. Be sure to notice that the character at position `start` is included in the value returned.

**EXAMPLE**

After program executes `String sample = "AbcdefG";`  
`sample.substring(2)` returns `"cdefG"`.

**Throws:**

`IndexOutOfBoundsException` if `start` is negative or larger than the length of the calling object.

```
public String substring(int start, int end)
```

Returns the substring of the calling object string starting from position `start` through, but not including, position `end` of the calling object. Positions are counted 0, 1, 2, etc. Be sure to notice that the character at position `start` is included in the value returned, but the character at position `end` is not included.

#### EXAMPLE

After program executes `String sample = "AbcdefG";`  
`sample.substring(2, 5)` returns "cde".

#### Throws:

`IndexOutOfBoundsException` if the `start` is negative, or `end` is larger than the length of this `String` object, or `start` is larger than `end`.

```
public String toLowerCase()
```

Returns a string with the same characters as the calling object string, but with all letter characters converted to lowercase.

#### EXAMPLE

After program executes `String greeting = "Hi Mary!";`  
`greeting.toLowerCase()` returns "hi mary!".

```
public String toUpperCase()
```

Returns a string with the same characters as the calling object string, but with all letter characters converted to uppercase.

#### EXAMPLE

After program executes `String greeting = "Hi Mary!";`  
`greeting.toUpperCase()` returns "HI MARY!".

```
public String trim()
```

Returns a string with the same characters as the calling object string, but with leading and trailing whitespace removed. Whitespace characters are the characters that print as whitespace on paper, such as the blank (space) character, the tab character, and the new-line character `'\n'`.

#### EXAMPLE

After program executes `String pause = " Hmm ";`  
`pause.trim()` returns "Hmm".

## StringBuffer

Package: `java.lang`

`StringBuffer` is marked `final` and so you cannot use it as a base class to derive another class.

Ancestor classes:

```

Object
|
+--StringBuffer

```

## CONSTRUCTORS

```
public StringBuffer()
```

Creates a `StringBuffer` object with no characters in it and an initial capacity of 16 characters.

```
public StringBuffer(int capacity)
```

Constructs a `StringBuffer` object with no characters in it and an initial capacity specified by the argument.

**Throws:**

`NegativeArraySizeException` if `length` is less than 0. `NegativeArraySizeException` is a derived class of `RuntimeException`, and so is an unchecked exception, which means it is not required to be caught or declared in a `throws` clause.

```
public StringBuffer(String ordinaryString)
```

Constructs a string buffer so that it represents the same sequence of characters as the `ordinaryString` argument; in other words, the initial content of the string buffer is a copy of `ordinaryString`. The initial capacity of the string buffer is 16 plus the length of `ordinaryString`.

**Throws:**

`NullPointerException` if `ordinaryString` is `null`.

## METHODS

```
public StringBuffer append(char[] charArray, int offset, int length)
```

Appends the string representation of the characters in `charArray` starting at `charArray[offset]` and extending for a total of `length` characters. Note that the calling object is changed and a reference to the changed calling object is returned.

**Throws:**

`ArrayIndexOutOfBoundsException` if `offset` and `length` are not consistent with the range of `charArray`.

```
public StringBuffer append(char c)
```

Appends the character argument to the `StringBuffer` calling object and returns this longer string.

```
public StringBuffer append(char[] charArray)
```

Appends the string representation of the `char` array argument to this string buffer. Note that the calling object is changed and a reference to the changed calling object is returned.

```
public StringBuffer append(double d)
```

Appends the string representation of the `double` argument to the `StringBuffer` calling object and returns this longer string.

```
public StringBuffer append(float d)
```

Appends the string representation of the `float` argument to the `StringBuffer` calling object and returns this longer string.

```
public StringBuffer append(int n)
```

Appends the string representation of the `int` argument to the `StringBuffer` calling object and returns this longer string.

```
public StringBuffer append(long n)
```

Appends the string representation of the `long` argument to the `StringBuffer` calling object and returns this longer string.

```
public StringBuffer append(String ordinaryString)
```

Appends the `String` argument to the `StringBuffer` calling object and returns this longer string. If `ordinaryString` is `null`, then the four characters "null" are appended to this string buffer. Note that the calling object is changed and a reference to the changed calling object is returned.

```
public StringBuffer append(StringBuffer bufferedString)
```

Appends the `StringBuffer` argument to the `StringBuffer` calling object and returns this longer string. If `bufferedString` is `null`, then the four characters "null" are appended to this string buffer. Note that the calling object is changed and a reference to the changed calling object is returned.

```
public int capacity()
```

Returns the current capacity of the calling object. The capacity is the amount of storage currently available for characters. The capacity will automatically be increased if necessary.

```
public char charAt(int position)
```

Returns the character in the calling object string at `position`. Positions are counted 0, 1, 2, etc.

**Throws:**

`IndexOutOfBoundsException` if `position` is negative or not less than the length of the calling object.

```
contentEquals
```

There is no such method for the class `StringBuffer`, but see the method `contentEquals` for the class `String`.

```
public StringBuffer delete(int start, int end)
```

Removes the characters in a substring of the calling object. The substring to remove begins at the specified start and extends to the character at index `end - 1` or to the end of the calling object if no such character exists. If `start` is equal to `end`, no changes are made. Note that the calling object is changed and a reference to the changed calling object is returned.

**Throws:**

`StringIndexOutOfBoundsException` if `start` is negative, greater than `length()`, or greater than `end`. `StringIndexOutOfBoundsException` is a derived class of `RuntimeException`, and so is an unchecked exception, which means it is not required to be caught or declared in a `throws` clause.

```
public void ensureCapacity(int minimumCapacity)
```

Ensures that the capacity of the calling object is at least equal to `minimumCapacity`. If the current capacity of the calling object is less than `minimumCapacity`, then the capacity is increased. The new capacity is the larger of: `minimumCapacity` and twice the old capacity, plus 2. If the `minimumCapacity` is nonpositive, this method takes no action and simply returns.

```
Start public boolean equals(Object otherObject)
```

Warning: This is the method inherited from `Object`. It is not overridden for the class `StringBuffer` and does not work as you might expect. Normally, it should not be used.

```
public int indexOf(String aString)
```

Returns the index (position) of the first occurrence of the string `aString` in the calling object. Positions are counted 0, 1, 2, etc. Returns `-1` if `aString` is not found. Note that the argument is of type `String`, not `StringBuffer`.

**Throws:**

`NullPointerException` if `aString` is null.

```
public int indexOf(String aString, int start)
```

Returns the index (position) of the first occurrence of the string `aString` in the calling object that occurs at or after position `start`. Positions are counted 0, 1, 2, etc. Returns `-1` if `aString` is not found. Note that the argument is of type `String`, not `StringBuffer`.

**Throws:**

`NullPointerException` if `aString` is null.

```
public int lastIndexOf(String aString)
```

Returns the index (position) of the last occurrence of the string `aString` in the calling object string. Positions are counted 0, 1, 2, etc. Returns `-1` if `aString` is not found. Note that the argument is of type `String`, not `StringBuffer`.

**Throws:**

`NullPointerException` if `aString` is null.

```
public int length()
```

Returns the length of the calling object as a value of type `int`.

```
public StringBuffer replace(int start, int end, String ordinaryString)
```

Replaces the characters in a substring of the calling object with characters in the `ordinaryString`. The substring begins at the specified `start` and extends to the character at index `end - 1` or to the end of the calling object if no such character exists. First the characters in the substring are removed and then the specified `ordinaryString` is inserted at `start`. (The calling object will be lengthened to accommodate the `ordinaryString` if necessary.) Note that the calling object is changed and a reference to the changed calling object is returned.

**Throws:**

`StringIndexOutOfBoundsException` if `start` is negative, greater than `length()`, or greater than `end`. `StringIndexOutOfBoundsException` is a derived class of `RuntimeException`, and so is an unchecked exception, which means it is not required to be caught or declared in a `throws` clause.

```
public void setLength(int newLength)
```

Sets the length of the calling object. The calling object is altered to represent a new character sequence whose length is specified by the argument. For every nonnegative index  $k$  less than `newLength`, the character at index  $k$  in the new character sequence is the same as the character at index  $k$  in the old sequence. If the `newLength` argument is less than the current length of the string buffer, the string buffer is truncated to contain exactly the number of characters given by the `newLength` argument.

If the `newLength` argument is greater than the current length, sufficient null characters (`'\u0000'`) are appended to the string buffer so that length becomes the `newLength` argument.

**Throws:**

`IndexOutOfBoundsException` if `newLength` is negative.

```
public String substring(int start)
```

Returns the substring of the calling object starting from `start` through to the end of the calling object. Positions are counted 0, 1, 2, etc. Be sure to notice that the character at position `start` is included in the value returned. Note that the substring is returned as a value of type `String`, not `StringBuffer`.

**Throws:**

`StringIndexOutOfBoundsException` if `start` is negative or larger than the length of the calling object. `StringIndexOutOfBoundsException` is a derived class of `RuntimeException`, and so is an unchecked exception, which means it is not required to be caught or declared in a `throws` clause.

```
public String substring(int start, int end)
```

Returns the substring of the calling object starting from position `start` through, but not including, position `end` of the calling object. Positions are counted 0, 1, 2, etc. Be sure to notice that the character at position `start` is included in the value returned, but the character at position `end` is not included. Also note that the substring is returned as a value of type `String`, not `StringBuffer`.

**Throws:**

`StringIndexOutOfBoundsException` if `start` is negative, or `end` is larger than the length of this calling object, or `start` is larger than `end`. `StringIndexOutOfBoundsException` is a derived class of `RuntimeException`, and so is an unchecked exception, which means it is not required to be caught or declared in a `throws` clause.

```
public String toString()
```

Creates a new `String` object that contains the same character sequence calling object and returns that `String` object. Subsequent changes to the calling object do not affect the contents of the `String` returned.

## StringTokenizer

`StringTokenizer` is a legacy class that is retained for compatibility purposes.

Package: `java.util`.

Ancestor classes:

```

Object
|
+--StringTokenizer

```



```
public StringTokenizer(String theString)
```

Constructor for a tokenizer that will use whitespace characters as separators when finding tokens in `theString`.

```
public StringTokenizer(String theString, String delimiters)
```

Constructor for a tokenizer that will use the characters in the string `delimiters` as separators when finding tokens in `theString`.

```
public StringTokenizer(String theString, String delimiters,
 boolean returnDelimiters)
```

Creates a tokenizer similar to `StringTokenizer(String theString, String delimiters)`, but with the following differences: If `returnDelimiters` is `true`, the delimiters are also returned by `nextToken`; each delimiter is returned as a one-character `String`. If `returnDelimiters` is `false`, the delimiters are not returned by `nextToken`. Thus, if `returnDelimiters` is `false`, the tokenizer created is the same as with `StringTokenizer(String theString, String delimiters)`.

```
public int countTokens()
```

Returns the number of tokens remaining to be returned by `nextToken`.

```
public boolean hasMoreElements()
```

Same as `hasMoreTokens`.

```
public boolean hasMoreTokens()
```

Tests whether there are more tokens available from this tokenizer's string. When used in conjunction with `nextToken`, it returns `true` as long as `nextToken` has not yet returned all the tokens in the string; returns `false` otherwise.

```
public String nextToken()
```

Returns the next token from this tokenizer's string.

#### Throws:

`NoSuchElementException` if there are no more tokens to return. `NoSuchElementException` is one of the exceptions that need not be declared in a `throws` clause or caught in a `catch` block.

## Vector<T>

This class was retrofitted in Java v1.2 to implement the `List` interface. Unlike the new collection implementations, `Vector` is synchronized. If a thread-safe implementation is not needed then it is recommended to use `ArrayList` instead of `Vector`.

Package: `java.util`

Ancestor classes:

```

Object
|
+--AbstractCollection<T>
 |
 +--AbstractList<T>
 |
 +--Vector<T>

```

**CONSTRUCTORS**

```
public Vector()
```

Creates an empty vector with an initial capacity of 10. When the vector needs to increase its capacity, the capacity doubles.

```
public Vector(Collection<? extends T> c)
```

Creates a vector that contains all the elements of the collection `c` in the same order as they have in `c`. If `c` is a vector, the capacity of the created vector will be `c.size()`, not `c.capacity`.

**Throws:**

`NullPointerException` if `c` is `null`.

```
public Vector(int initialCapacity)
```

Creates an empty vector with the specified initial capacity. When the vector needs to increase its capacity, the capacity doubles.

```
public Vector(int initialCapacity, int capacityIncrement)
```

Constructs an empty vector with the specified initial capacity and capacity increment. When the vector needs to grow, it will add room for `capacityIncrement` more items.

**ARRAYLIKE METHODS**

```
public T get(int index)
```

Returns the element at the specified index. This is analogous to returning `a[index]` for an array `a`.

**Throws:**

`ArrayIndexOutOfBoundsException` if the `index` is not greater than or equal to 0 and less than the current size of the vector.

```
public T set(int index, T newElement)
```

Sets the element at the specified `index` to `newElement`. The element previously at that position is returned. If you draw an analogy between the vector and an array `a`, this is analogous to setting `a[index]` to the value `newElement`.

**Throws:**

`ArrayIndexOutOfBoundsException` if the `index` is not greater than or equal to 0 and strictly less than the current size of the vector.

**METHODS TO ADD ELEMENTS**

```
public void add(int index, T newElement)
```

Inserts `newElement` as an element in the calling vector at the specified index. Each element in the vector with an index greater or equal to `index` is shifted upward to have an index that is one greater than the value it had previously.

Note that you can use this method to add an element after the last current element. The capacity of the vector is increased if this is required.

**Throws:**

`ArrayIndexOutOfBoundsException` if the `index` is not greater than or equal to 0 and less than *or equal to* the current size of the vector.

```
public boolean add(T newElement)
```

Adds `newElement` to the end of the calling vector and increases its size by one. The capacity of the vector is increased if this is required. Returns `true` if successful. Normally used as a `void` method.

#### METHODS TO REMOVE ELEMENTS

```
public void clear()
```

Removes all elements from the calling vector and sets its size to zero.

```
public T remove(int index)
```

Deletes the element at the specified index and returns the element deleted. Each element in the vector with an index greater than or equal to `index` is decreased to have an index that is one less than the value it had previously.

#### Throws:

`ArrayIndexOutOfBoundsException` if the `index` is not greater than or equal to 0 and less than the current size of the vector.

```
public boolean remove(Object theElement)
```

Removes the first occurrence of `theElement` from the calling vector. If `theElement` is found in the vector, then each element in the vector with an index greater than or equal to `theElement`'s index is decreased to have an index that is 1 less than the value it had previously. Returns `true` if `theElement` was found (and removed). Returns `false` if `theElement` was not found in the calling vector.

#### SEARCH METHODS

```
public boolean contains(Object target)
```

Returns `true` if `target` is an element of the calling vector; otherwise returns `false`.

```
public int indexOf(Object target)
```

Returns the index of the first element that is equal to `target`. Uses the method `equals` of the object `target` to test for equality. Returns `-1` if `target` is not found.

```
public int indexOf(Object target, int startIndex)
```

Returns the index of the first element that is equal to `target`, but considers only indices that are greater than or equal to `startIndex`. Uses the method `equals` of the object `target` to test for equality. Returns `-1` if `target` is not found.

```
public boolean isEmpty()
```

Returns `true` if the calling vector is empty (that is, has size 0); otherwise returns `false`.

```
public int lastIndexOf(Object target)
```

Returns the index of the last element that is equal to `target`. Uses the method `equals` of the object `target` to test for equality. Returns `-1` if `target` is not found.

```
public T firstElement()
```

Returns the first element of the calling vector.

**Throws:**

`NoSuchElementException` if the vector is empty.

```
public T lastElement()
```

Returns the last element of the calling vector.

**Throws:**

`NoSuchElementException` if the vector is empty.

## ITERATORS

```
public Iterator <T> iterator()
```

Returns an iterator for the calling vector. (Iterators are discussed in Chapter 16.)

```
ListIterator <T> listIterator()
```

Returns a list iterator for the calling vector. (Iterators are discussed in Chapter 16.)

```
ListIterator <T> listIterator(int index)
```

Returns a list iterator for the calling vector starting at `index`. The first element to be returned by the iterator is the one at `index`. (Iterators are discussed in Chapter 16.)

## CONVERTING TO AN ARRAY

```
public Object[] toArray()
```

Returns an array containing all of the elements in the vector. The elements of the array are indexed the same as in the vector.

```
public <E> E[] toArray(E[] a)
```

Note that the type parameter `E` is not the same as `T`. So, `E` can be any reference type; it need not be the type `T` in `Collection <T>`. For example, `E` might be an ancestor type of `T`. Returns an array containing all of the elements in the calling object. The elements in the returned array are in the same order as in the calling object. The argument `a` is used primarily to specify the type of the array returned. The exact details are described in the table for the `Collection <T>`.

**Throws:**

`ArrayStoreException` if the base type of `a` is not an ancestor class of all the elements in the vector.

`NullPointerException` if `a` is `null`.

## MEMORY MANAGEMENT

```
public int capacity()
```

Returns the current capacity of the calling vector.

```
public void ensureCapacity(int newCapacity)
```

Increases the capacity of the calling vector to ensure that it can hold at least `newCapacity` elements. Using `ensureCapacity` can sometimes increase efficiency, but its use is not needed for any other reason.

```
public void setSize(int newSize)
```

Sets the size of the calling vector to `newSize`. If `newSize` is greater than the current size, the new elements receive the value `null`. If `newSize` is less than the current size, all elements at index `newSize` and greater are discarded.

**Throws:**

`ArrayIndexOutOfBoundsException` if `newSize` is negative.

```
public int size()
```

Returns the number of elements in the calling vector.

```
public void trimToSize()
```

Trims the capacity of the calling vector to be the vector's current size. This is used to save storage.

MAKE A COPY

```
public Object clone()
```

Returns a clone of the calling vector. The clone is an identical copy of the calling vector.

OLDER METHODS

These are methods that are not part of the newer collection framework, but are retained for backward compatibility. You should use the previously described newer methods instead. But, you may find these used in older code.

```
public void addElement(T newElement)
```

Same as `add`.

```
public void insertElementAt(T newElement, int index)
```

Same as `add`.

```
public T elementAt(int index)
```

Same as `get`.

```
public void removeAllElements()
```

Same as `clear`.

```
public boolean removeElement(Object theElement)
```

Same as `remove`.

```
public void removeElementAt (int index)
```

Same as `remove` but does not return the element removed.

```
public void setElementAt (T newElement, int index)
```

Same as `set` with the arguments reversed but does not return the element replaced.

## WindowListener Interface

Package: `java.awt.event`

The `WindowEvent` class is also in this package.

Extends the `EventListener` interface.

```
public void windowActivated (WindowEvent e)
```

Invoked when a window is activated. When you click in a window, it becomes the activated window. Other actions can also activate a window.

```
public void windowClosed (WindowEvent e)
```

Invoked when a window has been closed.

```
public void windowClosing (WindowEvent e)
```

Invoked when a window is in the process of being closed. Clicking the close-window button causes an invocation of this method.

```
public void windowDeactivated (WindowEvent e)
```

Invoked when a window is deactivated. When a window is activated, all other windows are deactivated. Other actions can also deactivate a window.

```
public void windowDeiconified (WindowEvent e)
```

Invoked when a window is deiconified. When you activate a minimized window, it is deiconified.

```
public void windowIconified (WindowEvent e)
```

Invoked when a window is iconified. When you click the minimize button in a `JFrame`, it is iconified.

```
public void windowOpened (WindowEvent e)
```

Invoked when a window has been opened.

This page intentionally left blank

Note: key terms are in **bold**.

## Symbols

# (sharp), 727  
 % (modulo, remainder) operator, 55, 58–59  
 %n (line break), 97  
 && (and) operator, 151, 156  
 ' (quote), 55  
 () (parentheses), 56–58, 133, 157–163  
 -- (**decrement operator**), 62–64  
 \$ (dollar sign), 45, 46  
 \* (asterisk), 49  
 \* (conditional) operator, 144–145  
 \* (multiplication), 55  
 /\* symbol, 81–82  
 \*/ symbol, 81–82  
 ... (ellipsis), 414, 727  
 @ **tags**, 363–364  
 [] (square brackets), 380, 382  
 {} (braces), 37, 134, 135, 219  
 || (or) operator, 151–152, 156  
 ~ (tilde), 727  
 + (concatenation) operator, 66–67  
 + (plus) operator, 55, 237  
 ++ (**increment operator**), 62–64  
 = (assignment operator), 39, 48–50, 146, 159  
 with arrays, 392–394  
 with class type variables, 323  
 = (equal sign), 39, 48  
 == (equality) operator, 146–148, 182, 236  
 with arrays, 392–394  
 with enumerated types, 427  
 with variables of class type, 327–328  
 / (division) operator, 55, 58, 60  
 . (dot), 39  
 / (forward slash), 642  
 ; (semicolons), in `for` statements, 175–176  
 - (subtraction), 55  
 , (commas), in `for` statements, 173–174

## A

abbreviations, 81  
 abs method, 305  
 AbstractButton class, 1026, 1028, 1207–1208  
 abstract classes, 516, 541–548  
   interfaces, implementing, 751, 752  
 AbstractCollection<T> class, 947  
**abstraction**, 239. *See also*  
   information hiding  
 AbstractList<T> class, 946  
 AbstractMap<K,V> class, 957–958, 960  
**abstract methods**, 542, 544–545  
**abstract names**, 645  
 abstract path names, 647  
 AbstractSequential-List<T> class, 951  
 AbstractSet<T> class, 946  
**Abstract Window Toolkit (AWT)**, 982, 1017–1018  
 accept method, 1138  
 access  
   ArrayList class, 799–800  
   arrays, 379–382  
   default, 488, 489  
   friendly, 488, 489  
   inner/outer classes, 771  
   packages, 487–492  
   permissions, overriding, 472–473  
   private members, 247–248  
   random access to binary files, 668–674  
   to redefined base method, 493–495  
   static variables, 300  
 access modifiers, 487–492  
**accessor methods**, 242–248, 347–349, 408  
   for array instance variable, 416–419  
   with exception classes, 578–580  
**action command**, 1029–1030

action drawings, 1092, 1094–1098  
**action events**, 992–998  
 ActionListener interface, 993, 994, 1030–1032  
**action listeners**, 992–998, 1025  
 actionPerformed method, 993–994, 1013, 1019, 1036, 1127  
**activation records**, 695–696  
**actual parameters**, 227. *See also*  
   arguments  
**Adaptor (Adapter) pattern**, 731–732  
 addActionListener  
   button., 1175  
   method, 992, 1028  
 addAll method, 941, 942, 943, 953  
 adding  
   to ArrayList class, 800–802  
   colors, 1016, 1099  
   menus, to JFrame, 1026–1027  
   nodes, 848–849, 879–883, 885–888  
 add method, 800–801, 804, 911, 941, 942, 943, 952, 969, 1002  
**addresses**, 317  
   memory, 323, 389, 392–393  
 addToStart method, 848  
 ADT (abstract data type), 239  
**algorithms**, 166–169  
   binary search, 704–715  
   selection sort, 420–423  
   sorting, 733–740  
 Allman style, 135  
 alphabetical order, 148–149  
 ancestor classes, 470  
 and (&&) operator, 151, 156  
**anonymous classes**, 782–784  
**anonymous objects**, 330–331  
**Apache Derby**, 1145–1147, 1151–1152, 1167–1169  
 Apache Tomcat, 1160  
 API (application programming interface), 239, 362



- appending**, to a file, 623–625
- applets**, 36–37, 1158–1159
- applet viewer, 36–37
- applications**, 36. *See also*
  - programs
  - sample, 37–40
- arcs, drawing, 1087–1091
- areas (text), 1033–1046
- scroll bars, 1039, 1072–1077
- view port, 1072, 1073
- arg array, 413
- args identifier, 397–398
- arguments**, 39, 68, 69, 221, 223–225, 227
  - array, 390–392, 395–396
  - automatic type conversion, 254–256
  - constructor, 264
  - copy constructors, 345–351
  - correspondence, 223
  - indexed variables as, 390
  - length of arrays, 379, 385
  - no-argument constructors, 267–268, 270, 315, 336
  - null as, 341
  - wrapper classes, 315
- arithmetic expressions, 55–56
  - evaluating, 153
- arithmetic if, 144–145
- arithmetic if operators**. *See*
  - conditional operators
- arithmetic operators, 55–56
- ArrayIndexOutOfBoundsException
  - BoundsException, 384, 604
- ArrayList, 796–814, 1175–1176
- ArrayList<T> class, 939, 950–956, 1208–1213
- ArrayList class, 796–814
  - accessing, 799–800
  - adding to, 800–802
  - base type, 798
  - capacity, 798, 812
  - creating and naming objects, 799
  - example, 809–812
  - for-each loop, 806–809
  - methods, 803–806, 812–813
  - nonparameterized, 814
  - using, 798–803
- arrays**
  - accessing, 379–382
  - arguments, 390–392, 395–396
  - base type, 379, 390
  - in binary files, 666–668
  - creating, 379–382, 432
  - declaring, 380, 431, 432
  - enumerated types, 424–431
  - equality testing, 394, 396–397
  - for-each loops with, 408–412
  - indexed variables, 432
  - indices, 384, 432
  - initializing, 385
  - instance variables, 416–419, 434
  - introduction to, 378–387
  - length (size), 379, 385
  - length instance variable, 382–383
  - for loops with, 384, 434
  - for main method, 397–398
  - methods that return, 399–400, 435–441
  - multidimensional, 431–442
  - as objects, 388–389
  - out of bounds, 384
  - parameters, 390–392, 395, 398
  - partially filled, 401–410
  - privacy leaks with, 416–418
  - programming with, 400–431
  - ragged, 435
  - references and, 388–400
  - sorting, 420–423
  - vs. strings, 387
  - two-dimensional, 432–434
  - use of = and == with, 392–394
- array type, 389
- indexed variables, 392
- names, 400
- arrows, in inheritance diagrams, 728–729
- ASCII**
  - character set, 53, 75–76, 654, 905–906, 1203
  - files, 615. *See also* text files
- assertion checks, 189–191
- AssertionError class, 603–604
- assertions**, 189
- assert statement, 189
- assignment operator (=), 39, 48–50, 146, 159
  - with arrays, 392–394
  - with class type variables, 323
- assignment statements**, 48–52
  - Boolean expressions, 154
  - compatibility, 52–54
  - with primitive types, 49
- associativity rules**, 57, 157–163, 1201
- @author, 363
- author option, 365
- automatic boxing, 310–312, 822–823
- automatic garbage collection**, 849
- automatic type conversion, 254–256
- automatic unboxing, 311
- AWT (Abstract Window Toolkit)**, 982, 1017
  - classes, 1018
- B**
- back**, of list, 895
- background color, 999
- backslash (\), 74–75, 642
- BadNumberException, 578–580
- BankAccount class, 771–774
- bars
  - menu, 1025, 1027
  - scroll, 1039, 1072–1077
- base cases, 692, 698
- base classes**, 460, 464, 467, 470, 480
  - instance variables, 468, 476, 485–486
  - private methods, 486–487
  - super constructor, 474–476
- base types**, 379, 798, 806
- Big- O notation, 901–903
- binary files**, 615–616, 649–674
  - array objects in, 666–668
  - binary I/O of objects, 662–666
  - checking for end of, 660–662
  - closing, 654, 658
  - opening, 652–653, 656
  - random access to, 668–674
  - reading from, 655–659
  - UTF and, 654–655
  - writing to, 650–654
- binary operators, 159

- binary search**, 704–715
  - binary search tree**, 919–924
  - Binary Search Tree Storage Rule, 918–919
  - binary trees**, 916–917
  - binding**, 160, 517–518
  - bits, 318
  - block comments**, 81–82
  - blocks**, 219–220, 557, 559, 562–563, 569–570, 601–602, 622, 1142
    - catch, 557–559, 569–570
    - multiple catch, 583–587
    - nested, 601
    - try-catch, 65, 557–559, 601
  - body**
    - of loops, 164
    - of method, 213
  - Boolean class, 313, 1213–1214
  - Boolean expressions**, 132, 145–164
  - associatively rules, 157–163
    - building, 151–152
    - evaluating, 152–164
    - if-else statements, 132–139
    - lexicographic and alphabetical order, 148–149
    - loops and, 164–166
    - precedence rules, 157–163
    - simple, 145–148
  - boolean types, 48, 53, 55, 132, 154–155, 231–234
    - methods that return, 231–234
    - returned by mutator methods, 248
  - boolean variables, 154–155
  - BorderLayout class, 1003–1006, 1010, 1012
  - border layout managers, 1003–1006
  - bottom-up testing**, 237
  - bounding boxes**, 1081
  - bounds**
    - multiple, 827
    - type parameter, 825–828, 866–867
  - boxing**, 309, 310–312, 822–823
  - braces {}, 37, 134, 135, 219
  - branching mechanisms, 132–145
    - compound statements, 134–135
    - conditional operator, 144–145
    - if-else statements, 132–136
    - if statements, 133, 134, 136
    - multiway if-else statements, 136–139
    - nested statements, 136
    - switch statement, 139–144
  - break statement, 140, 143, 180–181
  - brighter method, 1101
  - BufferedReader class, 625, 1138–1139
    - methods, 637–638
    - reading numbers with, 639
    - reading text file using, 635–639
    - testing for end of file with, 639–641
  - buffers**, 621
  - bug, 44
  - buttons, 991–992
    - AbstractButton class, 1026, 1028
    - addActionListener, 1175
    - close-window, 990, 992
    - with icons, 1068
    - setActionCommand method, 1029–1030
  - Byte class, 1214–1215
  - byte-code**, 35, 41–42
  - bytes**, 317, 318
  - byte type, 48, 53, 56
- ## C
- calculator, Swing, 1041–1046
  - call-by-reference parameter mechanism, 324
  - call-by-value parameter mechanism, 224, 225
  - calling**, 39
    - methods, 39, 68
    - objects, 68
    - recursive methods, 688–691
  - canRead method, 648
  - canWrite method, 648
  - capacity**, 798, 812
  - capacity method, 954
  - case labels**, 140
  - case reserved word, 140
  - case-sensitivity, 46
  - Catch\_Block\_Parameter, 570
  - catch blocks, 557–559, 569–570
    - multiple, 583–587
    - nested, 601
    - parameters, 558–559
  - Catch or Declare Rule, 591, 592, 594
    - exception to, 593–594
  - ceil method, 306, 307
  - censor method, 415–416
  - c format specifier, 95, 96
  - chaining, hash tables with, 904–916
  - Character class, 313–316, 334
  - character sets
    - ASCII, 75–76
    - Unicode, 75–77
  - characters per line, 1039
  - charAt method, 71
  - char type, 48, 53, 54
  - checked exceptions**, 594
  - child classes**, 470. *See also* derived classes
  - class diagrams, 727–728
  - classes**, 35–36, 42, 67–69
    - abstract, 516, 541–548, 751
    - AbstractButton, 1026, 1028, 1207–1208
    - AbstractCollection<T>, 947
    - AbstractList<T>, 946
    - AbstractMap<K, V>, 957–958, 960
    - AbstractSequentialList<T>, 951
    - AbstractSet<T>, 946
    - accessor methods, 242–248
    - ancestor, 470
    - anonymous, 782–784
    - ArrayList, 796–814
    - ArrayList<T>, 939, 950–956, 1208–1213
    - AssertionError, 603–604
    - BankAccount, 771–774
    - base, 460, 464, 467, 470, 480
    - Boolean, 313, 1213–1214
    - BorderLayout, 1003–1006, 1010, 1012
    - BufferedReader, 625
    - Byte, 1214–1215
    - Character, 313, 334
    - child, 470

**classes** (*continued*)

- collection, 408–411, 806, 936–957
- Color, 1100–1101, 1219–1220
- commenting, 362–364
- compiling, 42–43, 360–361
- concrete, 545, 946–955, 960–963
- constructors, 258–279, 474–477
- container, 1002, 1017
- Container, 1016–1017
- Counter, 1134
- creating, 1002
- DataOutputStream, 650
- Date, 342–343
- DateClient, 1141–1142
- DateServer, 1140–1141
- definitions, 206–238, 270
- derived, 460–473, 477–480, 594–595
- descendent, 470
- Dimension, 1029
- DivisionByZero Exception, 572–576, 591
- Double, 1221
- encapsulation, 239–250
- EndingListener, 993
- Error, 593
- errors, 594
- Exception, 564–565, 570–571, 824–825
- exceptions, 569–583
- File, 631, 645–649, 1221–1222
- FileInputStream, 121
- FileNotFoundException, 121
- FileOutputStream, 617
- FileReader, 637
- final modifier, 519
- Float, 1223
- FlowLayout, 1006–1007, 1010
- Font, 1108, 1109, 1110, 1223
- generic, 814–832. *See also* generics
- Graphics, 1081–1098, 1224–1225
- GridLayout, 1007–1010
- HashMap<K, V>, 957–958, 960–963, 1226
- HashSet<T>, 947–950, 1227
- HashTable, 906–907
- helping, 770–775
- hierarchy, 461
- ImageIcon, 1066, 1069
- immutable, 351–354
- implementation, 362
- importing, 102–104
- information hiding, 239–250
- inner, 748
  - anonymous, 782–784
  - .class file, 775
  - inheritance and, 781
  - listeners ad, 1030–1032
  - nested, 781
  - node, 852–855
  - public, 777–781
  - static, 776–777
  - uses of, 770–776
  - window listener, 1060
- Insets, 1071
- instance of, 206, 207
- instance variables, 209–212
- Integer, 309, 310, 1227
- interactions, 728
- interface, 362
- IOException, 619
- iterators, 874–879. *See also* iterators
  - JButton, 991–992, 1069–1070
  - JColorChooser, 1102–1105
  - JFrame, 986, 988–990, 995–998, 1016–1017, 1228–1230
  - JLabel, 998, 1005, 1069–1070
  - JMenu, 1021, 1025
  - JMenuBar, 1021, 1025
  - JMenuItem, 1021, 1025, 1069–1070, 1230
  - JPanel, 1011–1016, 1017, 1092
  - JScrollPane, 1072–1077, 1231
  - JTextArea, 1037–1039
  - JTextComponent, 1038, 1232
  - JTextField, 1034–1037, 1038–1039
  - LinkedHashMap<K, V>, 960
  - LinkedList, 840
  - LinkedList<T>, 951, 955
  - Locale, 102, 103
  - Long, 1240
  - Math, 305–309, 356, 1240–1241
  - MergeSort, 737
  - methods. *See* methods
  - mutable, 351–354
  - mutator methods, 242–248
  - name clashes, 361
  - naming conventions, 46, 68
  - NumberFormat, 99–102, 103
  - Object, 496–497
  - ObjectInputStream, 649–650, 655–659, 1244–1246
  - ObjectOutputStream, 649–654, 1246–1247
  - objects, 68, 206–209
  - outer, 770, 771, 779–780
  - overloading, 250–258
  - packages, 354–366
  - Packer, 1127, 1129
  - parameterized, 814
  - parameters. *See* parameters
  - parent, 470
  - Person, 336–341, 344, 345–347
  - vs. primitive types, 68
  - PrintWriter, 615–623, 1248
  - Queue, 896–898
  - RaceConditionTest, 1134–1135
  - Random, 192–193, 1249
  - RandomAccessFile, 668–674, 1249–1252
  - RuntimeException, 593
  - Scanner, 108–120, 625–630, 1252–1255
  - SelectionSort, 422–423
  - serializable, 663–666
  - ServerSocket, 1138–1139
  - Set<T>, 950
  - Short, 1259
  - Stack, 893–895
  - String, 65–77, 356, 1259–1263
  - StringBuffer, 75, 1263–1267
  - StringTokenizer, 274–279, 331–332, 481–484, 1267–1268
  - subclasses, 464, 467, 480

- superclasses, 464, 467, 480
- System, 181
- Thread, 1127–1130
- Throwable, 593
- TreeSet<T>, 955
- UML, 727–730
- Vector, 813–814
- Vector<T>, 950–956, 1268–1273
- WindowAdapter, 1064–1065
- WindowEvent, 1058
- wrapper, 309–316, 356
- classes, loaders, 42
- .class files, 43
- for inner class, 775
- class instance variables, 665
- class invariant, 336–341
- ClassNotFoundException, 1149
- class path
  - not including current directory in, 360
  - specifying, when compiling, 360–361
- classpath option, 365
- CLASSPATH** variable, 356–361
- class types
  - arrays with, 390
  - mixing, in same file, 666
  - parameters of, 225, 229, 323–329
  - variables of, 316–317, 318–323, 327–328
- clause (throws), 589–592, 594–595
- clear method, 804, 941, 953
- clients, 1138
- client/server model, 1138–1139, 1158
- Cloneable interface, 765–769, 870, 873
- clone method, 347, 350, 497, 536–541, 765–768, 805, 954
- copy constructors, 538–541, 862–873
  - HashMap<K, V> class, 961
  - shallow copies, 812–813
- close method, 621–622, 652, 669
- close-window button, 990, 992
- closing
  - binary files, 654, 658
  - text files, 621–622
- code**, 35, 42
  - incremental development, 188–189
  - legacy, 98
  - review, 188–189
- collecting, 1176–1178
- Collection<T> interface, 936–941, 947, 1216–1219
- collections**, 408–411, 806, 936–957
  - concrete classes, 946–955
  - exceptions, 945–946
  - for-each loops, 944
  - framework, 938–946
  - interfaces, 944
  - iterators and, 937
  - nonparameterized, 956
  - optional operations, 944–945
  - overview of, 936–937
  - wildcards, 938
- collisions, 904
- Color class, 1100–1101, 1219–1220
- color constructors, 1100
- colors**
  - adding, 1016, 1099
  - constants, 1000
  - defining, 1100–1102
  - JColorChooser dialog window, 1102–1105
  - RGB color system, 1100
  - setting, 999–1002
  - specifying, 1099
  - Swing, 1098–1105
- commands
  - action, 1029–1030
  - run, 42
  - SQL, 1146–1157
- commas, in for statements, 173–174
- comments**, 79, 81–82
  - block, 81–82
  - for javadoc, 362–364
  - line, 81, 82
  - when to add, 82
- Common Gateway Interface (CGI)**, 1161–1162
- Comparable interface, 739, 755–761, 826, 1220
- compare method, 739
- compareToIgnoreCase method, 73, 149
- compareTo method, 73, 148, 427, 739, 755, 826
- comparing, strings, 150, 193
- comparison operators, 146
- compiler**, 35, 40
  - Just-In-Time (JIT), 41
- compiler errors, 146, 240, 957
- compiling**, 40–42
  - classes, 42–43, 360–361
  - Xlint option, 817
- complete evaluation**, 156
- component model, 1143–1144
- components**, 1017
  - with changing visibility, 1077–1080
- composition, 493
- compound statements**, 134–135, 219–220
- concatenation (+) operator, 66–67
- concatenation**, of strings, 66–67
- concrete classes**, 545
  - collections, 946–955
  - maps, 960–963
- conditional (\*) operator**, 144–145
- connections
  - databases, 1145–1157
  - networks. *See* networking
- connection strings**, 1149
- console I/O**, 90
  - file input, 121–124
  - screen output, 90–108
  - using Scanner class, 108–120
- constants**, 53–55
  - color, 1000
  - defined, in interfaces, 761–765
  - inconsistent, 762–763
  - location, 102
  - Math class, 307
  - naming, 78–80
  - null, 329–330, 341
  - static import of, 335
  - String, 65–66

- constructors**, 258–279
  - arguments, 264
  - color, 1100
  - copy, 345–351, 862–873
  - default, 268
  - definitions, 258–266
  - exception classes, 572–573
  - nested, 642–643
  - no-argument, 267–268, 270, 315, 336
  - `StringTokenizer` class, 274–279
  - super, 474–476, 495, 577
  - this, 266–267, 476–477
  - use of, 265–266
  - using, 273–274
  - variable initialization, 269
  - wrapper classes, 315
- Container class, 1016–1017
- container classes**, 1002, 1017
- Container-Iterator** pattern, 731
- containers, 731
- `containsAll` method, 939
- `containsKey` method, 959
- `contains` method, 804, 911, 915–916, 939, 953
- `containsValue` method, 959
- content pane, 999
- continue statement, 180–181
- controlling expressions**, 140
- conversion characters**, 95
- coordinate system, for graphics objects, 1081
- copy constructors**, 345–351, 536, 538–541
  - `clone` method, 862–873
- `copyOf` method, 862, 865
- `Counter` class, 1134
- `countTokens` method, 277
- covariant return types**, 471–472
- `createNewFile` method, 648
- CREATE TABLE statements, 1149, 1150
- critical region, 1135
- currency format, 99–102
- current directory**, 360, 361
- cursor, 970
- D**
- database management system (DBMS)**, 1145
- databases**
  - Apache Derby, 1145–1146, 1151–1152, 1167–1169
  - connections, 1145–1157
  - Java DB, 1145–1146
  - JDBC, 1146
  - relational, 1145, 1148
  - tables, 1145, 1148
  - updating, 1169–1163
- `DataClass`, 767–768
- `DataOutputStream` class, 650, 1138
- data structures. *See* linked data structures
- data types, 317
- `Date` class, 342–343
- date classes, 232–234, 241, 243–246
- `DateClient` class, 1141–1142
- `DateServer` class, 1140–1141
- debugging**, 44, 182–191
  - assertion checks, 189–191
  - collections, 945–946
  - examples, 184–188
  - general techniques, 183–184
  - loop bugs, 182
  - preventive coding, 188–189
  - tracing variables, 182–183
- `DecimalFormat` class, 104–108
- decimal points, 54
- decimals, formatting, 104–108
- declarations**, 1163
- declaring**, 47, 590–592
  - arrays, 380, 431, 432
  - variables, 47, 220
- decrement operator (--), 62–64
- deep copy**, 353, 416, 536
  - `clone` method, 867–872
- default access**, 488, 489
- default constructors**, 268
- default package**, 359–360, 490
- default section, 140
- defined constants, in interfaces, 761–765
- defining
  - classes, 206–238, 270
  - `equals` method, 497–503
  - exception classes, 572–578
  - iterators, 874–879, 972
  - `JFrame` class, 995–998
  - methods, 210, 212–218
- `deleteHeadNode` method, 848–849
- `delete` method, 648
- deletion, of nodes, 879–883, 885–888
- delimiters
  - choosing, 276
  - input, 119–120
- @deprecated, 363, 364
- deprecated methods**, 364
- derived classes**, 460–470, 480
  - constructors, 474–476
  - generic, 830–832
  - instance variables, 476, 485–486
  - objects, 477–479
  - overriding methods, 471–473
  - package access, 487–492
  - protected access, 487–492
  - redefined methods, 493–494
  - throws clause in, 594–595
- derived interfaces, 751–752
- descendent classes**, 470
- `d` format specifier, 95
- diagrams
  - class, UML, 727–728
  - inheritance, 728–730
- `Dimension` class, 1029
- directives, 1163, 1167
- directories, 43
- current, 360, 361
- package, 356–359
- `dispose` method, 989, 1063
- division
  - floating-point numbers, 59
  - integer, 58–60
  - whole numbers, 60
- `DivisionByZeroException` class, 572–576, 591
- `doNothing` method, 1121
- d option, 365
- dot (.), 39
- `Double` class, 1221
- `Double.parseDouble` method, 331–333
- double quotes, 55

double type, 48, 54, 55, 56  
**doubly linked lists**, 884–893  
do-while statements, 164–166, 167  
**downcasting**, 529–535  
draw3DRect method, 1086  
drawArc method, 1086, 1087–1091  
drawing  
    action, 1092, 1094–1098  
    arcs, 1087–1091  
    in JPanel, 1092  
    objects, 1081–1098  
    ovals, 1087  
    paint method, 1082–1085  
drawLine method, 1084–1085  
drawOval method, 1084, 1086, 1087  
drawRect method, 1085  
drawRoundRect method, 1086  
drawString method, 1105–1107, 1109  
**driver programs**, 237  
**dynamic binding**. *See also* late binding  
dynamic binding, 517–518

## E

**early binding**, 517  
echo input, 116  
E constant, 307  
e format specifier, 95  
elements, of array, 379  
**ellipsis (...)**, 414, 727  
empty lists, 846, 847  
**empty statements**, 175–176  
**empty strings**, 116  
empty trees, 918  
**em spaces**, 1039  
**encapsulation**, 239–250, 362, 484–492  
endButton, 992–993  
EndingListener class, 993  
end-of-line character, 112, 115–116  
**enhanced for loops**, 411. *See also* for-each loops  
EnhancedStringTokenizer class, 481–484  
e notation, 54, 106

ensureCapacity method, 805, 954  
**Enterprise JavaBeans**, 1144  
entrySet method, 959  
**enumerated types**, 424–431  
    methods, 426–429  
    in switch statements, 429–431  
environment variable, 356–357  
EOFException, 660–662  
equalArrays method, 394  
equality (==) operator, 146–148, 182, 236  
    with arrays, 392–394  
    with class type variables, 327–328  
    with enumerated types, 427  
equality testing, 182, 327–328  
    arrays, 394, 396–397  
equal sign (=), 39, 48  
equalsIgnoreCase method, 70, 146, 147  
equals method, 70, 146–148, 233–237, 247, 328, 341, 426–427, 496, 805, 959  
    Collection<T> interface, 940  
    Color class, 1101  
    defining, 497–503  
    linked lists, 860–861  
    overriding, 948  
error classes, 593, 594  
error messages, redirecting, 644–645  
errors  
    compiler, 146, 240, 957  
    debugging, 182–191  
    logic, 44  
    loop, 182  
    null pointer exception, 330, 390  
    off-by-one, 182  
    out of bounds, 384  
    round-off, 59  
    run-time, 44  
    stack overflow, 696  
    syntax, 44  
escape character, 74–75, 642  
**escape sequences**, 74–75, 112  
evaluating  
    arithmetic expressions, 153  
    Boolean expressions, 152–164

evaluation  
    complete, 156  
    short-circuit, 156  
**event-driven programming**, 599, 982–984  
**event handlers**, 983  
event handling model, 1144  
**events**, 983–984  
    action, 992–998  
    firing, 599, 983, 984, 992  
    window, 1058  
Exception class, 564–565, 570–571, 824–825  
exception classes, 569–583  
    accessor methods, 578–580  
    constructors, 572–573  
    defining, 572–578  
    from standard packages, 570–572  
exception handlers, 558  
exception handling  
    basics of, 556–587  
    as event-driven programming, 599  
    example, 564–569  
    exception classes, 569–583  
    loops, 560–561  
    in methods, 588–600  
    programming techniques for, 601–604  
    with Scanner class, 559–560  
    try-catch mechanism, 557–559  
exception objects, 577  
**exceptions**  
    ArrayIndexOutOfBoundsException, 604  
    BadNumberException, 578–580  
    checked, 594  
    ClassNotFoundException, 1149  
    collections, 945–946  
    copy constructors and, 863  
    declaring, 590–592  
    EOFException, 660–662  
    as events, 983  
    FileNotFoundException, 617, 619, 626, 638  
    IllegalAccessException, 1149

**exceptions** (*continued*)

- `IllegalArgumentException`, 429, 989
- `IllegalStateException`, 965, 969
- `InstantiationException`, 1149
- `InterruptedException`, 1124
- `IOException`, 619, 638
- in methods, 588–600
- multiple, 583–587
- `NegativeNumberException`, 586
- `NoSuchElementException`, 946, 968
- `NullPointerException`, 863
- predefined, 570–572
- rethrowing, 603
- `SecurityException`, 989
- `SQLException`, 1149
- throwing, 556, 557, 562–565
- uncaught, 594, 1041
- unchecked, 594, 634
- `UnsupportedOperationException`
  - Exception class, 945, 965
  - when to use, 595–599
- execute method, 1149
- executeQuery method, 1149, 1152
- exists method, 647
- exit statement, 181
- explicit memory management**, 880
- exponents, 54
- expressions**
  - arithmetic, 55–58
  - Boolean, 132, 145–164
  - JSP, 1163
  - precedence rules with, 56–58
- extends phrase, 464, 938

**F**

- false value, 48, 53, 55, 132, 152–155
- `f` format specifier, 95
- fields**, 207
- field width, 94
- `File` class, 631, 645–649, 1221–1222
  - methods, 647–649
  - programming with, 645–647

- `FileInputStream` class, 121
- file I/O
  - of array object, 666–668
  - binary, of objects, 662–666
  - binary files, 649–674
  - `File` class, 645–649
  - introduction to, 121–124, 614–616
  - nested constructors, 642–643
  - random access to binary files, 668–674
  - text files, 121–124, 615–645
- `FileNotFoundException` class, 121, 617, 619, 626, 638
- `FileOutputStream` class, 617
- file pointer, 668
- `FileReader` class, 637
- files**
  - ASCII, 615
  - binary, 615–616, 649–674
    - array objects in, 666–668
    - binary I/O of objects, 662–666
  - checking for end of, 660–662
  - closing, 654, 658
  - opening, 652–653, 656
  - random access to, 668–674
  - reading from, 655–659
  - UTF and, 654–655
  - writing to, 650–654
- locations, 211
- names, 211, 617, 619, 641–642
- opening, 616–617
- output, 622–623
- text, 615–616
  - appending, 623–625
  - closing, 621–622
  - opening, 618
  - path names, 641–642
  - reading, 625–628, 635–639
  - testing for end of, 628–630, 639–641
  - writing to, 615–623
- `fill3DRect` method, 1086
- `fillArc` method, 1087
- `fillOval` method, 1086
- `fillRect` method, 1085
- `fillRoundRect` method, 1086
- finally block, 601–602

- final modifier, 300, 471, 519
- firing the event**, 599, 983, 984, 992
- `Float` class, 1223
- floating-point notation**, 54
- floating-point numbers**, 47, 48, 54, 55, 56
  - comparing, 182
  - division of, 59
  - round-off errors in, 59
- float type, 48, 55
- floor method, 306, 307
- `FlowLayout` manager, 1006–1007, 1010
- flow of control
  - Boolean expressions, 145–164
  - branching mechanism, 132–145
  - debugging, 182–191
  - loops, 164–181
  - random number generation, 191–194
- flush method, 621
- folders, 43
- follows method, 750, 754
- `Font` class, 1108, 1109, 1110, 1223
- fonts
  - point size, 1109
  - serifs, 1109
  - Swing, 1105–1110
- for-each loops, 408–412, 428, 806–809, 944
  - as iterators, 967–968
- formal parameters**. *See* parameters
- format method, 100–101, 104–106
- format specifiers**, 94–95
- format strings**, 96
- formatting
  - decimals, 104–108
  - money, 97, 99–102, 103
  - output, 93–99
- forms, HTML, 1161–1162, 1166
- for statements, 170–176
  - with arrays, 384, 434
  - comma in, 173–174
  - declaring variables in, 220
  - extra semicolon in, 175–176
- friendly access**, 488, 489



**front**, of list, 895  
**full path names**, 641–642  
**fully qualified class names**, 361  
functional programming, Java8,  
1172–1178  
**functions**, 899  
hash, 904

## G

**garbage collection**, 880  
automatic, 849  
**generics**, 796, 814  
basics of, 811–121  
bounds for type parameters,  
825–828  
class definition, 823  
Exception class, 824–825  
inheritance with, 830–832  
instantiation, 815, 822  
interfaces, 828  
linked lists, 855–860,  
867–872  
methods, 828–829  
ordered pairs, 817–820,  
822–823  
getActionCommand method, 1028,  
1030  
getBlue method, 1101  
getClass method, 499–502  
getContentPane method, 999  
getFilePointer method, 669  
getGraphics method, 1125–1126  
getInsideArray method, 418  
getMessage method, 563,  
576–578  
get method, 804, 952, 959  
getName method, 648  
getPath method, 648  
getRed method, 1101  
getText method, 1028, 1034,  
1036–1037, 1038  
g format specifier, 95  
GlassFish Enterprise Server, 1160  
global variables, 219  
Gosling, James, 34  
**graphical user interfaces (GUIs)**,  
982  
adding and arranging  
components in, 1002–1019  
calculator, 1041–1046

components with changing  
visibility, 1077–1080  
designing, 1019  
ending, 994  
with labels and color, 1000–1002  
with menu, 1021–1025  
nonresponsive, 1121–1124,  
1126  
panels, 1011–1016  
programs, 594, 732  
updating, 1098  
Graphics class, 1081–1098,  
1224–1225  
arcs, drawing, 1087–1091  
coordinate system, 1081  
methods, 1082–1087  
ovals, drawing, 1087  
repaint method, 1094–1098  
rounded rectangles, 1091–1092  
Graphics object, 1084  
greater than (>), 146  
greater than or equal to, 146  
GridLayout managers, 1007–1010

## H

handlers  
event, 983  
exception, 558  
**handling**  
events, 1144  
**exceptions**. *See* exception  
handling  
“has a” relationship, 493, 1002  
hashCode method, 940, 948, 959  
hash functions, 904  
for strings, 905–908  
HashMap<K, V> class, 957–958,  
960–963, 1226  
hash maps. *See* hash tables  
HashSet<T> class, 947–950, 1227  
HashSet method, 947  
HashTable class, 906–907  
hash tables  
constructing, 904–905  
efficiency of, 908–909  
HashMap<K, V> class, 960–961  
linked data structures, 904–916  
hasMoreTokens method, 276, 277  
hasNextBoolean method, 633  
hasNextByte method, 632

hasNextDouble method, 633  
hasNextFloat method, 633  
hasNextInt method, 628, 630,  
631  
hasNextLine method, 628–630,  
634  
hasNextLong method, 632  
hasNext method, 633, 879, 965,  
969  
hasNextShort method, 632  
hasPrevious method, 969  
heading levels, 1164–1165  
**headings**, 210  
**head nodes**, 840  
head variable, 911  
helping classes, 770–775  
**higher precedence**, 57, 157  
**high-level languages**, 40  
HotJava, 35  
HTML, 362, 1158  
HTML forms, 1161–1162, 1166

## I

**icons**  
button, 1068  
changing visibility, 1077–1080  
Swing, 1066–1072  
using, 1067–1068  
**identifiers**, 45–46  
naming conventions, 80–81  
if-else statements, 132–133  
compound, 134–135  
multiple alternatives, 135  
multiway, 136–139  
nested, 136  
placement of braces in, 135  
if statements, 133, 134, 136  
IllegalAccessExcepton, 1149  
IllegalArgumentException, 429, 989  
IllegalStateException, 965,  
969  
ImageIcon class, 1066, 1069  
immutable classes, 351–354  
**immutable objects**, 75, 352  
**implementation**, 362  
importing  
classes, 102–104  
packages, 102–104  
import method, 104, 108



- import statements, 90, 103–104, 798
  - packages and, 355–356
  - static, 333–335
- inconsistent interfaces, 762–763
- incremental development, 188–189
- increment method, 1133, 1136
- increment operator (++), 62–64
- indentation, 82–83
- nested statements, 136
- indexed variables**, 379, 380, 389, 390, 432
  - as arguments, 390–391
- indexes**, 73–74
  - array, 384, 432
- indexOf method, 72, 805, 943, 953
- inequalities, strings of, 152
- i.next method, 970–971
- infinite loops**, 176–177, 182, 660
- infinite recursion**, 693
- information hiding**, 239–250, 362
- inheritance**, 1002
  - derived classes, 461–470
  - encapsulation, 484–492
  - with generic classes, 830–832
  - inner classes and, 781
  - instance variables, 485–486
  - overriding method definition, 471
  - overview of, 460–484
  - private methods, 486–487
  - programming with, 493–503
  - static variables, 493
  - in Swing, 984
- inheritance diagrams**, 728–730
  - initial capacity, 960
  - initialization
    - arrays, 385
    - automatic, 385
    - default, 300
    - instance variables, 258, 264, 336
    - variables, 50–51, 269–274, 300
- inner classes**, 748
  - access privileges, 771
- anonymous, 782–784
  - .class file, 775
  - helping classes, 770–775
  - inheritance and, 781
  - listeners as, 1030–1032
  - nested, 781
  - node, 852–855
  - public, 777–781
  - reasons to use, 784
  - static, 776–777
  - uses of, 770–776
  - window listener, 1060
- inorder processing**, 918
- input**. *See also* file I/O
  - delimiters, 119–120
  - echo, 116
  - file, 121–124
  - numbers, 1040
  - prompt, 116
  - text file, 121–124
  - using Scanner class, 108–120
  - using StringTokenizer class, 274–279, 331–332
- InputMismatchException, 559–561
- input streams**, 614–615
- INSERT statements, 1150
- Insets class, 1071
- instanceof operator, 499–501, 533
- instances**, 206, 207
- instance variables**, 50, 207, 209–212, 219, 269–274
  - accessor methods for, 416–419
  - base class, 468, 476, 485–486
  - class, 665
  - initializing, 258, 264, 336
  - interfaces and, 762
  - length, 382–383, 389, 434
  - privacy leaks with, 416–418
  - private, 240, 416–418, 485–486
  - protected and package access, 487–492
  - resetting values, 265
- instantiation, generic classes, 815, 822
- InstantiationException, 1149
- int, 39
- Integer class, 309, 310, 1227
- integers**, 53
  - binary search tree for, 921–922
  - division of, 58–60
- Integrated Development Environment (IDE), 42, 182
- interfaces**, 362
  - abstract classes, 751, 752
  - ActionListener, 993, 994, 1030–1032
  - Cloneable, 765–769
  - Collection<T>, 936–937, 947, 1216–1219
  - Comparable, 755–761, 826, 1220
  - defined constants in, 761–765
  - derived, 751–752
  - extending, 751–752
  - generic, 828
  - implementing, 749–751
  - inconsistent, 762–763
  - introduction to, 748–751
  - Iterator, 879
  - Iterator<T>, 964–967, 1228
  - List<T>, 941–944, 950, 1233–1238
  - ListIterator<T>, 968–970, 1238–1240
  - Map<K,V>, 1242–1244
  - NumberCarrier, 783–784
  - Ordered, 749, 754
  - Runnable, 1130–1132
  - semantics, 753–754
  - Serializable, 765, 1256
  - Set<T>, 941–942, 944, 947, 950, 1256–1258
  - SortedSet<T>, 955
  - WindowListener, 1273
- intermediate language**, 35
- interpreters**, 41
- InterruptedException, 1124
- intersection method, 911
- introspection**, 1144
- int type, 48, 53, 54, 55–56
- intValue method, 310
- invocation**, 39
- constructors, 264
- methods, 68, 69, 70, 210–211, 213, 266, 295–296
- IOException class, 619, 638

**“is a” relationship**, 478, 493, 1002

isDigit method, 314

isDirectory method, 648

isEditable method, 1038

isEmpty method, 939, 953, 959

isFile method, 648

isLetter method, 314

isLetterOrDigit method, 314

isLowerCase method, 314

isUpperCase method, 314

isWhitespace method, 314

iteration, 164

vs. recursion, 696–697

iterative version, 696–697,

710–711

Iterator<T> interface, 964–967, 1228

Iterator interface, 879

iterator method, 940, 954

**iterators**, 410, 731, 873–883, 936

collections and, 937

defining, 874–879, 972

doubly linked list with, 889–892

for-each loops as, 967–968

Iterator<T> interface, 964–967

list, 968–970

nodes, adding and deleting, 879–883

overview of, 964

references, returning, 970–972

## J

Java

introduction to, 34–45

naming of, 36

origins of, 34–35

Java applets, 1158–1159

Java application programs

compiling, 42–43

running, 43

sample, 37–40

styles, 78–83

java.awt class, 1084

**JavaBeans**, 1143–1145

Java byte-code, 35

javac command, 42–43

java command, 43

Java Database Connectivity (JDBC), 1146, 1147

**Java DB**, 1145–1146

javadoc program, 81–82, 354

commenting classes for, 362–364

introduction to, 362

options, 365

running, 364–366

Java8, 1172–1178

.java files, 42

JavaFX, 1180–1193

java.io package, 616

java.lang package, 104, 334, 356

**Java Server Pages (JSP)**,

1158–1164

applets, 1158–1159

declarations, 1163

directives, 1163, 1167

expressions, 1163

HTML forms, 1161–1162, 1166

Oracle GlassFish Enterprise Server, 1160

scriptlets, 1163–1164

servlets, 1158–1159

Java servlets, 1158–1159

Java Software Development Kit (SDK), 1145–1146

java.text, 102–103

java.util package, 102, 103, 108–109, 277, 941

**Java Virtual Machine (JVM)**, 41–42

JButton class, 991–992, 1069–1070

JColorChooser class, 1102–1105

JDBC (Java Database Connectivity), 1146, 1147

JDK, 42

JFrame class, 986, 1016–1017, 1228–1230

color, adding, 1016

color, setting, 999–1002

content pane, 999

defining, 995–998

dispose method, 1063

menus, adding, 1026–1027

methods, 988–990

title, setting, 1000

JLabel class, 998, 1005, 1069–1070

JMenuBar class, 1021, 1025

JMenu class, 1021, 1025

JMenuItem class, 1021, 1025, 1069–1070, 1230

join method, 733, 1134

JPanel class, 1011–1016, 1017, 1092

JScrollPane class, 1072–1077, 1231

JTextArea class, 1037–1039

JTextComponent class, 1038, 1232

JTextField class, 1034–1037, 1038–1039

Just-In-Time (JIT), 41

**JVM (Java Virtual Machine)**, 41–42

## K

Kernighan & Ritchie (K&R) style, 135

keyboard input. *See also* file I/O; input

using Scanner class, 108–120

using StringTokenizer class, 274–279, 331–332

keys, hash table, 904

**keywords**, 46, 1199

## L

**labels**

with changing visibility, 1077–1080

icon, 1066

statement, 180–181

Swing, 998, 1000–1002, 1005

text field, 1040

lamda expressions, 1172–1173

**largest and smallest values**, 312

lastIndexOf method, 72, 805, 944, 953

**last-in/first-out (LIFO)**, 695, 895

**late binding**, 517–518, 531

example, 520–527

final modifier and, 519

static methods and, 528–529, 530

with toString, 527–528

- layout managers**, 1002–1011
    - border, 1003–1006, 1012
    - flow, 1006–1007
    - grid, 1007–1010
  - lazy evaluation**, 156
  - leaf nodes**, 918
  - leaks. *See* privacy leaks
  - left justified output, 96
  - legacy code, 98
  - length**, of array, 379, 385
  - length instance variable, 382–383, 389, 434
  - length method, 70, 669
  - less than (<), 146
  - less than or equal to, 146
  - lexicographic ordering, 148–149
  - LIFO. *See* last-in/first-out (LIFO)
  - line, characters per, 1039
  - linear running time, 903
  - line breaks, 96–97
  - line comments**, 81, 82
  - line terminator ‘BACKSLASHn’, 112, 115–116
  - linked data structures**
    - Big- *O* notation, 901–903
    - copy constructors and clone method, 862–873
    - doubly linked lists, 884–893
    - hash tables, 904–916
    - introduction to, 840–841
    - iterators, 873–883
    - linked lists, 840, 842–861, 903–904
    - queues, 895–898
    - running times, 898–903
    - sets, 909–916
    - stacks, 893–895
    - trees, 916–924
  - LinkedHashMap<K, V> class, 960
  - LinkedList<T> class, 951, 955
  - LinkedList class, 840
  - linked lists**, 784, 840–861
    - copy constructor and clone method for, 863–865
    - with deep copy clone method, 867–872
    - doubly, 884–893
    - efficiency of, 903–904
    - empty, 846, 847
    - end of, 847
    - equals method, 860–861
    - example, 842, 850
    - generic, 855–860
    - inner classes, 852–855
    - with iterator, 874–879
    - nodes, adding and deleting, 879–883, 885–888
    - nodes, adding and removing, 848–849
    - privacy leaks, 851
    - sets using, 910–911, 915–916
    - traversing, 846–848
    - variations on, 884–904
    - working with, 846–851
  - link option, 365
  - links**, 840, 841
  - List<T> interface, 941–944, 950, 1233–1238
  - listeners**, 983–984
    - action, 992–998, 1025
    - as inner classes, 1030–1032
    - menu item, 1026–1027
    - registering, 992
    - window, 1058–1065
  - ListIterator<T> interface, 968–970, 1238–1240
  - listIterator method, 944, 954
  - lists
    - empty, 846, 847
    - iterators, 968–970
    - linked. *See* linked lists
  - literals**, 53–55. *See also* constants
  - loaders, classes, 42
  - load factor, 960
  - Locale class, 102, 103
  - localhost method, 1139
  - local variables**, 50, 218–219
    - initializing, 269
    - parameters as, 225–227
  - location constants, 102
  - logic errors**, 44
  - Long class, 1240
  - long type, 48
  - loops**, 164–181
    - with arrays, 384
    - body of, 164
    - break statement, 180–181
    - continue statement, 180–181
    - debugging, 182
    - do-while statement, 164–166, 167
    - for exception handling, 560–561
    - exit statement, 181
    - for-each, 408–412, 428, 944, 967–968
    - infinite, 176–177, 182, 660
    - iteration, 164
    - nested, 177
    - repeating, 175
    - sentinel values, 169
    - for statement, 170–176, 220, 384
    - while statement, 164–166, 167, 176
  - low-level languages**, 40
- ## M
- machine language**, 40
  - main memory**, 317
  - main method, 36, 37, 43, 218–219, 225, 296–299
    - arguments for, 397–398
    - GUI programs, 1009
  - mantissa**, 106
  - Map<K, V> interface, 957–960, 1242–1244
  - maps**, 957–963
    - concrete classes, 960–963
  - masks**, 230
  - Math class, 305–309, 356, 1240–1241
  - Math.random method, 193
  - max method, 305, 412–413
  - members**, 207
  - memory**
    - address, 323, 389, 392–393
    - explicit management, 880
    - location, 317
    - main, 317
    - secondary, 317
    - variables and, 317–320
  - menu bars**, 1025, 1027
  - menu items**, 1021, 1025–1027
  - menus**
    - adding, to JFrame, 1026–1027
    - menu bars, 1025, 1027
    - menu items, 1021, 1025–1027
    - nested, 1026

- setActionCommand method, 1029–1030
- Swing, 1021–1033
- merge sort**, 734, 735–736, 737
- MergeSort class, 737
- messages, sending, 39, 68
- method calls, 68
- recursive, 688–691
- method headings, inconsistent, 763
- methods**, 35–36, 39, 69, 206–207, 529
  - abs, 305
  - abstract, 542, 544–545
  - accept, 1138
  - accessor, 242–248, 347–349, 408
  - actionPerformed, 993–994, 1013, 1019, 1036, 1127
  - add, 800–801, 804, 911, 941, 942, 943, 952, 969, 1002
  - addActionListener, 1028
  - addAll, 941, 942, 943, 953
  - addToStart, 848
  - arguments, 341
  - ArrayList class, 803–806
  - body, 210, 213
  - brighter, 1101
  - canRead, 648
  - canWrite, 648
  - capacity, 954
  - ceil, 306, 307
  - censor, 415–416
  - charAt, 71
  - clear, 804, 941, 953
  - clone, 350, 497, 536–541, 805, 812–813, 961
  - close, 621–622, 652, 669
  - Collection<T> interface, 939–941
  - compare, 739
  - compareTo, 73, 148, 427, 739, 755, 826
  - compareToIgnoreCase, 73, 149
  - constructors, 258–279
  - contains, 804, 911, 915–916, 939, 953
  - containsAll, 939
  - containsKey, 959
  - containsValue, 959
  - copyOf, 862, 865
  - countTokens, 277
  - createNewFile, 648
  - defining, 210, 212–218
  - delete, 648
  - deleteHeadNode, 848–849
  - deprecated, 364
  - dispose, 989, 1063
  - doNothing, 1121
  - Double.parseDouble, 331–333
  - d raw3DRect, 1086
  - drawArc, 1086, 1087–1091
  - drawLine, 1084–1085
  - drawOval, 1084, 1086, 1087
  - drawRect, 1085
  - drawRoundRect, 1086
  - drawString, 1105–1107, 1109
  - ensureCapacity, 805, 954
  - entrySet, 959
  - with enumerated types, 426–429
  - equalArrays, 394
  - equals, 70, 146–148, 233–237, 247, 328, 341, 426, 427, 496–503, 805, 940, 948, 959, 1101
  - equalsIgnoreCase, 70, 146, 147
  - execute, 1149
  - executeQuery, 1149, 1152
  - exists, 647
  - File class, 647–649
  - fill3DRect, 1086
  - fillArc, 1087
  - fillOval, 1086
  - fillRect, 1085
  - fillRoundRect, 1086
  - floor, 306, 307
  - flush, 621
  - follows, 754
  - format, 100–101, 104–106
  - generic, 828–829
  - get, 804, 952, 959
  - getActionCommand, 1028, 1030
  - getBlue, 1101
  - getClass, 499–502
  - getContentPane, 999
  - getFilePointer, 669
  - getGraphics, 1125–1126
  - getInsideArray, 418
  - getMessage, 563, 576–577, 578
  - getName, 648
  - getPath, 648
  - getRed, 1101
  - getText, 1028, 1034, 1036–1037, 1038
  - hashCode, 940, 948, 959
  - HashSet, 947
  - hasMoreTokens, 276, 277
  - hasNext, 633, 879, 965, 969
  - hasNextBoolean, 633
  - hasNextByte, 632
  - hasNextDouble, 633
  - hasNextFloat, 633
  - hasNextInt, 631
  - hasNextLine, 634
  - hasNextLong, 632
  - hasNextShort, 632
  - hasPrevious, 969
  - import, 104, 108
  - increment, 1133, 1136
  - indexOf, 72, 805, 943, 953
  - inherited, 468, 471
  - intersection, 911
  - intValue, 310
  - invoking, 39, 68, 69, 70, 210–211, 213, 266, 295–296
  - isDigit, 314
  - isDirectory, 648
  - isEditable, 1038
  - isEmpty, 939, 953, 959
  - isFile, 648
  - isLetter, 314
  - isLetterOrDigit, 314
  - isLowerCase, 314
  - isUpperCase, 314
  - isWhitespace, 314
  - iterator, 954
  - join, 733, 1134
  - lastIndexOf, 72, 805, 944, 953
  - late binding, 517–518
  - length, 70, 669
  - listIterator, 944, 954
  - local variables, 218–219
  - main, 218–219, 225, 296–299
  - Math, 305–309
  - Math.random, 193

**methods** (*continued*)

- max, 305, 412–413
- min, 305
- mkdir, 648
- mkdirs, 649
- multidimensional arrays, 435–441
- mutator, 242–248, 349–350, 352
- next, 112, 113, 633, 965, 968, 970, 971
- nextBoolean, 114, 633
- nextByte, 113, 632
- nextDouble, 110–112, 113, 632
- nextFloat, 113, 633
- nextIndex, 969
- nextInt, 110, 113, 212, 631
- nextLine, 112, 114, 115–116, 634
- nextLong, 113
- nextShort, 113, 632
- nextToken, 276, 277
- optional, 944–945
- ordinal, 427
- out.println, 1163
- overloading, 250–258
- overriding, 471
  - changing access permission, 472–473
  - changing return type, 471–472
  - invoking old version, 494
  - vs. overloading, 473–474
- pack, 1098
- paint, 1082–1085, 1098
- paintComponent, 1092, 1093–1094
- parameters, 220–228
- parseDouble, 312–313
- parseInt, 313
- postconditions, 249–250
- pow, 305
- power, 698–701, 703
- precedes, 248, 751, 754
- preconditions, 249–250
- previous, 969, 970, 971
- previousIndex, 969
- print, 91–92, 616, 619–620
- printf, 93–99, 620, 1205
- println, 90–92, 212, 236, 425, 527–528, 616, 619
- private, 486–487, 544
- protected, 487
- public, 240
- put, 959
- putAll, 960
- random, 306
- read, 638, 670
- readBoolean, 657, 671
- readByte, 670
- readChar, 657, 671
- readDouble, 657, 671
- readFloat, 657, 671
- readInt, 657, 671
- readLine, 635, 637, 639
- readLong, 657, 671
- readObject, 658, 666
- readShort, 657, 670
- readUTF, 658, 671
- recursive, 237, 684, 697–702
- redefined, 493–494
- remove, 804, 941, 943, 953, 960, 965, 969
- removeActionListener, 1028
- removeAll, 941
- removeRange, 804, 953
- renameTo, 648
- repaint, 1092, 1094–1098
- retainAll, 941
- returned values, 69
- round, 306
- run, 1127, 1129
- Scanner class, 113–114, 631–634
- search, 708–710
- set, 270, 273–274, 803, 952, 969
- setActionCommand, 1028, 1029–1030, 1068
- setBackground, 1038
- setColor, 1099
- setDate, 265
- setDefaultCloseOperation, 989, 1064
- setEditable, 1037–1038
- setFont, 1108, 1109, 1110
- setHorizontal
  - ScrollBarPolicy, 1073
- setIcon, 1066, 1070, 1071
- setJMenuBar, 989
- setLayout, 989, 1003, 1006
- setLength, 669, 671
- setLineWrap, 1037
- setMargin, 1070
- setMaximumSize, 1028
- setMinimumSize, 1028
- setPreferredSize, 1028
- setSize, 989
- setText, 1028, 1034, 1036–1037, 1066, 1068, 1070, 1071
- setTitle, 989
- setVertical ScrollBarPolicy, 1073–1074
- setVertical TextPosition, 1070
- setVisible, 990, 1078
- setWindowListener, 1058
- size, 801–802, 805, 940, 954, 959
- skipBytes, 658
- sort, 420–421
- split, 733, 739–740
- sqrt, 306
- start, 1129
- startThread, 1130
- static, 293–299, 312–316, 528–529, 530
- String class, 68–74
- stringToDouble, 1041
- sublist, 944
- substring, 71–72
- System.out.println, 1163–1164
- testing, 237–238
- that return arrays, 399–400, 435–441
- that return a value, 212–215
- that return Boolean values, 231–234
- Thread.sleep, 1121–1122, 1124–1125
- throwing exceptions in, 588–600
- toArray, 805, 940, 954
- toLowerCase, 71, 313
- toString, 233–237, 330, 426, 493, 496, 527–528, 531
- toUpperCase, 71, 313
- trim, 71, 313
- trimToSize, 805, 954
- useDelimiter, 114, 634

validate, 1098  
 valueOf, 427, 429  
 values, 427, 428–429, 959  
 with variable number of  
     parameters, 412–418  
 void, 212–216, 225, 685–697,  
     704  
 windowActivated, 1059  
 windowClosed, 1059  
 windowClosing, 1059  
 windowDeiconified, 1059  
 windowIconified, 1059  
 windowOpened, 1059  
 write, 669  
 writeBoolean, 670  
 writeByte, 670  
 writeChar, 654, 670  
 writeDouble, 651, 670  
 writeFloat, 652, 670  
 writeInt, 651, 653, 670  
 writeLong, 651, 670  
 writeObject, 652, 666  
 writeOutput, 229, 236  
 writeShort, 651, 670  
 writeUTF, 652, 654–655, 670  
 method signature, 250, 253, 256  
 min method, 305  
 mixing types, 55–56  
 mkdir method, 648  
 mkdirs method, 649  
**Model-View-Controller pattern,**  
     732–733, 1020  
 modifiers  
     final, 519  
     protected, 488  
     public/private, 240–241, 418,  
         488  
 modifying, access permissions,  
     472–473  
 modulo (%) operator, 55, 58–59  
 money  
     formatting, 97, 99–102  
     outputting, 103  
 moneyFormatter object, 99–102  
 multidimensional arrays, 431–442  
     parameters and returned values,  
         435–441  
 multithreading, 1120–1137  
     getGraphics method,  
         1125–1126

race conditions, 1133–1137  
 Runnable interface,  
     1130–1132  
 Thread class, 1127–1130  
 Thread.sleep method,  
     1121–1122, 1124–1125  
 thread synchronization,  
     1133–1137  
 multiway if-else statements,  
     136–139  
**mutable classes,** 351–354  
**mutable objects,** 352, 353  
**mutator methods,** 242–248,  
     349–350, 352  
  
**N**  
**names.** *See also* identifiers  
     abstract, 645  
     array type, 400  
     **clashes,** 361  
     package, 356–359  
     path, 641–642  
     naming conventions, 46, 68  
     constants, 78–80  
     files, 211, 619  
     spelling conventions, 80–81  
 Naughton, Patrick, 35  
 NegativeNumberException, 586  
 nested constructors, 642–643  
 nested inner classes, 781  
 nested loops, 177  
 nested menus, 1026  
 nested statements, 136  
 Netscape, 35  
 networking, with stream sockets,  
     1138–1143  
 \n (new line) character, 74  
 new operator, 207–209, 264–265,  
     330–331  
 nextBoolean method, 114, 633  
 nextByte method, 113, 632  
 nextDouble method, 110–112,  
     113, 632  
 nextFloat method, 113, 633  
 nextIndex method, 969  
 nextInt method, 110, 113, 212,  
     631  
 nextLine method, 112, 114,  
     115–116, 634  
 nextLong method, 113

next method, 112, 113, 633, 965,  
     968, 970, 971  
 nextShort method, 113, 632  
 nextToken method, 276, 277  
**no-argument (no-arg)**  
     **constructor,** 267–268,  
         270, 315, 336, 577, 578  
 Node<T> class, 860  
 Node class, 843, 860  
**nodes,** 840, 841  
     adding and deleting, 848–849,  
         879–883, 885–888  
     head node, 840  
     inner classes, 852–855  
     leaf, 918  
     root node, 917  
     tree, 917–918  
 nonstatic methods, invoking, with  
     static method, 295–296  
 NoSuchElementException, 946,  
     968  
 notation  
     Big -O, 901–903  
     e, 54, 106  
 not equal to, 146  
 null constant, 329–330, 341, 847  
 NullPointerException, 330,  
     390, 863  
 null statements, 175–176  
 NumberCarrier interface,  
     783–784  
 NumberFormat class, 99–102, 103  
 numbers. *See also* integers  
     E-notation, 106  
     floating-point. *See* floatingpoint  
         numbers  
     inputting and outputting, 1040  
     naming, 78–80  
     percentages, 106  
     pseudorandom, 191  
     random, 191–194, 309  
     reading, 639  
     vertical, 685–688  
     whole, 60  
 numerical analysis, 59

## O

Object class, 496–497  
 clone method, 536–541, 865–866  
 object code, 42

- ObjectInputStream class, 649–650, 655–659, 1244–1246
- object-oriented programming (OOP), 35–36, 206, 460, 516, 726
- ObjectOutputStream class, 649–650, 651–652, 653–654, 1246–1247
- object program, 42
- objects**, 35–36, 39, 68, 69, 206–207
  - anonymous, 330–331
  - ArrayList, 799
  - binary I/O of, 662–666
  - calling, 68
  - copying, 345–351, 353
  - creating, 207–209, 264–265
  - derived classes, 477–479
  - drawing, 1081–1098
  - exception, 577
  - graphics, 1081–1098
  - Graphics, 1084
  - immutable, 75, 352
  - members of, 207
  - method definitions and, 531
  - mutable, 352, 353
  - private members of, 247–248
  - resetting values, 265
  - sending messages to, 39
  - String, 65–66
- objects, arrays as, 388–389
- off-by-one errors, 182
- OOP. *See* object-oriented programming (OOP)
- opening files, 618, 652–653, 656
- operations, 900
  - optional, 944–945
  - sets, 910–911
- operators
  - and, 151, 156
  - arithmetic, 55–56
  - assignment, 39, 48–50, 323, 392–394
  - binary, 159
  - binding, 160
  - comparison, 146
  - concatenation, 66–67
  - conditional, 144–145
  - decrement, 62–64
  - equality, 146–148, 182, 236, 392–394, 427
  - increment, 62–64
  - instanceof, 499–501, 533
  - new, 207–209, 264–265, 330–331
  - or, 151–152, 156
  - overloading, 257
  - plus, 237
  - precedence, 157–163
  - remainder, 58–59
- options
  - javadoc, 365
  - Xlint, 817
- or (||) operator, 151–152, 156
- Oracle Corporation, 35
- Oracle GlassFish Enterprise Server, 1160
- Ordered interface, 749, 754
- ordered pairs, 817–820, 822–823
- ordinal method, 427
- origin**, 1081
- outer classes**, 770, 771, 779–780
- out of bounds**, 384
- out.println method, 1163
- output**. *See also* file I/O
  - buffered, 621–622
  - formatting, 93–99
  - money formats, 99–102, 103
  - numbers, 1040
  - redirecting, 643–645
  - screen, 90–108
- output files, overwriting, 622–623
- outputList method, 848
- output streams, 614–615, 643–645
- outputStream variable, 616–617
- ovals, drawing, 1087
- overloading**, 250–258, 412
  - automatic type conversion and, 254–256
  - constructors, 264
  - operators, 257
  - vs. overriding, 473–474
  - rules for, 250–253
- overriding**
  - access permission, 472–473
  - equals method, 948
  - invoking old version, 494
  - method definition, 471
  - vs. overloading, 473–474
  - return type, 471–472
- P**
- packages**, 90, 102, 108, 354–366
  - access, 487–492
  - default, 359–360, 490
  - directories, 356–359
  - importing, 102–104
  - import statements, 355–356
  - java.lang, 356
  - name clashes, 361
  - names, 356–359
- Packer class, 1127, 1129
- pack method, 1098
- page import directive, 1167
- paintComponent method, 1092, 1093–1094
- paint method, 1082–1085, 1098
- pair programming, 189
- panels**, 1011–1016
  - paintComponent method, 1092
- @param, 363, 364
- parameterized classes**, 814. *See also* generics
- parameter lists, 213
- parameters**, 220–231
  - arguments, 221, 223–225, 227
  - array, 390–392, 395–396, 398
  - catch block, 558–559
  - of class type, 225, 229, 323–329
  - as local variables, 225–227
  - methods with variable number of, 412–416, 417–418
  - multidimensional arrays, 435–441
  - of a primitive type, 220–228, 325, 326–327
  - terminology, 227
  - this, 229–231, 266–267
  - type, 815–816, 820–821, 823–824
- parent classes**, 470. *See also* base classes
- parentheses (), 56–58, 133
  - precedence rules and, 157–163
- parseDouble method, 312–313
- parseInt method, 313
- partially filled arrays**, 401–410
- path names**, 356–357, 641–642
- PATH variable, 292, 354–355, 357



**patterns**, 104–106

- Adaptor (Adapter), 731–732
- Container-Iterator, 731

- formalism, 740

- introduction to, 726, 731

- Model-View-Controller, 732–733, 1020

- sorting, 733–740

- Payne, Jonathan, 35

- percentages, 106

- persistence, 1144

- Person class, 336–341, 344, 345–347

- PI constant, 307–308

- pixels**, 988

- point, 1109

- pointers, 320

- point size**, 1109

**polymorphism**

- clone method, 536–541
- downcasting and upcasting, 529–535

- final modifier, 519

- introduction to, 516

- late binding, 517–518, 520–528

- pop, 893

- ports, 1138

- positions, 73

- position variable, 847–848, 879, 880

- postconditions**, 249–250

- postorder processing**, 918

- power method, 698–701, 703

- pow method, 305

- precedence rules, 1201

- arithmetic expressions, 56–58

- Boolean expressions, 157–163

- precedes method, 248, 750, 751, 754

- preconditions**, 249–250

- predefined words, 46

- preorder processing**, 918

- preventive coding, 188–189

- previousIndex method, 969

- previous method, 969, 970, 971

- previous variable, 879

- primitive types**, 48, 68

- assignment compatibility, 52–53, 54

- assignment statements with, 49

- largest and smallest values of, 312

- mixing, 55–56

- parameters of, 220–228, 325, 326–327

- type parameters and, 821

- variables of, 316–317, 318, 320

- printf method, 93–99, 620, 1205

- println method, 38, 39, 90, 212, 236, 387, 425, 527–528, 616, 619

- vs. print, 91–92

- print method, 91–92, 616, 619–620

- PrintWriter class, 615–623, 1248

- privacy leaks**, 347–350, 354, 765–766

- arrays, 416–418

- linked lists, 851

- private instance variables, 416–418

- private members, access to, 247–248

- private methods, 486–487, 544

- private modifier, 239, 240–241, 418, 488

- private option, 365

- programming

- with arrays, 400–431

- event-driven, 599, 982, 983–984

- with File class, 645–647

- functional, 1172–1178

- with inheritance, 493–503

- object-oriented, 35–36, 206, 460, 516, 726

- pair, 189

- web, 1158–1164

- programming languages

- high-level languages, 40

- intermediate language, 35

- low-level languages, 40

- machine language, 40

- object-oriented, 35–36

- semantics, 44

- syntax, 44

**programs**

- comments, 81–82

- compiling, 42–43

- driver, 237

- indenting, 82–83

- javadoc, 81–82, 354, 362–366

- running, 43

- running times, 898–903

- styles, 78–83

- Swing. *See* Swing

- protected access, 487–492

- protected method, 487

- protected modifier, 488

- protocols, 1138

- pseudocode**, 166–169, 726–727

- pseudorandom numbers, 191

- public, 207, 212

- public inner classes, 777–781

- PubliclyCloneable interface, 866, 870–871

- public modifier, 240–241, 488

- public static final, 80

- push, 893

- putAll method, 960

- put method, 959

**Q**

- quadratic running time, 903

- Queue class, 896–898

- queues**, 884, 895–898

- quick sorts**, 734, 736–739

- quoted strings, 65

- quotes, 55

**R**

- race conditions, 1133–1137

- RaceConditionTest class, 1134–1135

- ragged arrays**, 435

- random access, to binary files, 668–674

- RandomAccessFile class, 668–674, 1249–1252

- Random class, 192–193, 1249

- random method, 306

- random numbers, generating, 191–194, 309

- Random object, 192–193

- readBoolean method, 657, 671

- readByte method, 670



- readChar method, 657, 671
- readDouble method, 657, 671
- readFloat method, 657, 671
- reading
  - files, 625–628, 635–639, 655–659
  - numbers, 639
  - text files, 121–124
- readInt method, 657, 671
- readLine method, 635, 637, 639
- readLong method, 657, 671
- read method, 638, 670
- readObject.readShort method, 657, 670
- readUTF method, 658, 671
- rectangles, rounded, 1091–1092
- recursion**
  - binary search, 704–715
  - calls, tracing, 688–691
  - design techniques, 703–715
  - infinite, 693
  - introduction to, 684–685
  - vs. iteration, 696–697
  - methods, returning values, 697–702
  - stacks, 694–696
  - tail, 712
  - void method, 685–697, 704
- recursive methods, 237, 684
- redefined methods, base, access to, 493–495
- references**, 318–323
  - arrays and, 388–400
  - holding, 320
  - iterators returning, 970–972
  - to mutable objects, 353
  - use of, 335–354
- reference types, 320
- registering**, listeners, 992
- rehashing, 960
- relational databases**, 1145, 1148
- relative path names**, 641–642
- removeActionListener method, 1028
- removeAll method, 941
- remove method, 804, 941, 943, 953, 960, 965, 969
- removeRange method, 804, 953
- renameTo method, 648
- repaint manager**, 1098
- repaint method, 1092, 1094–1098
- reserved words**, 46
- resolution, 988
- retainAll method, 941
- rethrowing exceptions, 603
- retrieving a high score, 596–599
- @return, 363
- returned values**, 69
- multidimensional arrays, 435–441
- return statements**, 213–214, 215, 234
- return type
  - changing, of overridden method, 471–472
  - covariant, 471–472
- RGB (red, green, blue) color system**, 1100
- right justified output, 96
- root nodes**, 917
- rounded rectangles**, 1091–1092
- round method, 306
- rules
  - associatively, 57, 157–163, 173
  - Boolean expressions, 157–163
  - Catch or Declare, 591, 592, 594
  - expressions, 56–58
  - overloading, 250–253
  - precedence, 56–58, 157–163
  - spelling conventions, 80–81
- run command**, 42
- run method, 1127, 1129
- Runnable interface, 1130–1132
- running, Java programs, 43
- running times, 898–903
- run-time errors**, 44
- RuntimeException class, 593
- S**
- Scanner class, 559–560, 1252–1255
  - console input, 108–120
  - keyboard input, 114–115
  - methods, 113–114, 631–634
  - reading text file using, 625–628
  - testing for end of file with, 628–630
  - text file input, 121–124
- Scene Builder, 1187–1192
- scene graph**, 1180
- scientific notation**, 54
- screen coordinate system, 1081
- screen output, 90–108
- scriptlets**, 1163–1164
- scroll bars, 1039, 1072–1077
- SDK, 42
- search method, 708–710
- secondary memory**, 317
- SecurityException, 989
- @see, 363
- SelectionSort class, 422–423
- selection sorts**, 420–423, 740
- SELECT statements, 1150, 1152–1155
- self-documenting**, 82
- semantics**, 44
  - interface, 753–754
- semicolons (;), in for statements, 175–176
- sending a message**, 39, 68
- sentinel values**, 169
- separator characters, 276–277
- Serializable interface, 662–666, 765, 1256
- serifs, 1109
- server, 1138
- ServerSocket class, 1138–1139
- servlets**, 1158–1159
- Set<T> interface, 911–907, 915–916, 941–942, 944, 947, 950, 1256–1258
- setActionCommand method, 1028, 1029–1030, 1068
- setBackground method, 1038
- setColor method, 1099
- setDate method, 265
- setDefaultCloseOperation method, 985, 989, 986, 1064
- setEditable method, 1037–1038
- setFont method, 1108, 1109, 1110
- setHorizontalScrollBarPolicy method, 1073
- setIcon method, 1066, 1070, 1071
- setJMenuBar method, 989
- setLayout method, 989, 1003, 1006
- setLength method, 669, 671
- setLineWrap method, 1037
- setMargin method, 1070

- setMaximumSize method, 1028
- set method, 270, 273–274, 803, 952, 969
- setMinimumSize method, 1028
- setPreferredSize method, 1028
- sets, 909–916
  - efficiency of, 915–916
  - fundamental operations, 910–911
  - linked lists, 910–911
- setSize method, 989
- setText method, 1028, 1034, 1036–1037, 1066, 1068, 1070, 1071
- setTitle method, 989
- setVertical ScrollBarPolicy method, 1073–1074
- setVerticalTextPosition method, 1070
- setVisible method, 990, 1078
- setWindowListener method, 1058
- s format specifier, 95, 96
- shallow copy, 353, 812–813
- short-circuit evaluation**, 156
- Short class, 1259
- short type, 48, 53, 56
- side effects**, 161–163
- signature, method, 250, 253, 256
- single quotes, 55
- size**
  - of array, 379, 385
  - resolution and, 988
- size method, 801–802, 805, 940, 954, 959
- skipBytes method, 658
- Socket class, 1139
- sockets, 1138–1143
- threading and, 1142
- SortedSet<T> interface, 955
- sorting
  - arrays, 420–423
  - quick sort, 734, 736–739
- sorting pattern, 733–740
  - efficiency of, 739–740
  - restrictions on, 739
- sort method, 420–421
- source code**, 42
- source program, 42
- spelling conventions, 80–81
- split method, 733, 739–740
- splitting value**, 734
- splitting value, 736
- SQL (Structured Query Language)**, 1146–1157
  - SQLException, 1149
  - sqrt method, 306
  - square brackets [], 380, 382
  - Stack class, 893–895
  - stack frames**, 695–696
  - stack overflow, 696
  - stacks**, 694–696, 884, 893–895
  - start method, 1129
  - startThread method, 1130
  - statements**
    - assignment, 48–54, 154
    - break, 140, 143, 180–181
    - compound, 134–135, 219–220
    - continue, 180–181
    - CREATE TABLE, 1149, 1150
    - do-while, 164–167
    - empty, 175–176
    - exit, 181
    - for, 170–176, 220, 384, 434
    - if, 133, 134, 136
    - if-else, 132–139
    - import, 90, 103–104, 333–335, 355–366, 798
    - INSERT, 1150
    - multiway, 136–139
    - nested, 136
    - null, 175–176
    - return, 213–215, 234
    - SELECT, 1150, 1152–1155
    - SQL, 1150
    - static import, 333–335
    - switch, 139–144, 180, 429–431
    - throw, 562–563, 569–570, 572
    - UPDATE, 1150, 1153, 1155–1157
    - while, 164–167, 176
  - static import statements**, 333–335
  - static inner classes**, 776–777
  - static keyword, 293
  - static methods**, 293–299
    - late binding and, 528–529, 530
    - in wrapper classes, 312–316
- static variables**, 50, 219, 300–304, 493
- stopping cases**, 692, 698, 705–706
- streams**, 614–615, 625
  - sockets, networking with, 1138–1143
- StringBuffer class, 75, 1263–1267
- String class, 48, 65–77, 356, 1259–1263
  - classes, 67–69
  - concatenation, 66–67
  - constants and variables, 65–66
  - escape sequences, 74–75
  - indexes, 73–74
  - methods, 68–74, 313
  - processing, 75, 76
  - Unicode character set, 75–77
- strings**
  - vs. arrays, 387
  - comparing, 150, 193
  - decomposing, 274–279
  - empty, 116
  - format, 96
  - hash functions for, 905–908
  - patterns, 104–106
  - processing, 312–316, 415–416, 417–418
  - using == with, 147–148, 236
- strings, of inequalities, 152
- stringToDouble method, 1041
- StringTokenizer class, 274–279, 331–332, 481–484, 1267–1268
- String variables, 426, 617
- Structured Query Language. *See* SQL (Structured Query Language)
- stubs**, 238, 1019
- subclasses**, 464, 467, 480. *See also* derived classes
- subdirectories, 359
- sublist method, 944
- subscripted variables**, 379
- substring method, 71–72
- Sun Microsystems, 34, 35
- superclasses**, 464, 467, 480
- super constructor, 474–476, 495, 577, 938

- super.paint, 1084
- super relationships, 493–494
- Swing, 982
  - action events, 992–998
  - action listeners, 992–998, 1025
  - buttons, 991–992, 1026
  - calculator, 1041–1046
  - class hierarchy, 1018
  - close-window button, 990, 992
  - colors, 999–1002, 1016, 1098–1105
  - Container class, 1016–1017
  - drawString method, 1105–1107, 1109
  - ending, 994
  - event-driven programming, 983–984
  - fonts, 1105–1110
  - Graphics class, 1081–1098
  - icons, 1066–1072
  - labels, 998, 1000–1002, 1005
  - layout managers, 1002–1011
  - menus, 1021–1033
  - Model-View-Controller pattern, 1020
  - numbers, 1040
  - overview of, 984–1002
  - panels, 1011–1016
  - scroll bars, 1072–1077
  - text fields and areas, 1033–1046
  - updating, 1098
  - window listeners, 1058–1065
  - windows, 985–986
- switch statements**, 139–144, 180
  - enumerated types in, 429–431
- // symbol, 81
- synchronized keyword, 1135–1136
- syntactic variables**, 47
- syntax**, 44
- syntax errors, 44
- System class, 181
- System.err, 643–644
- System.exit, 994
- System.in, 615, 643–644
- System.out, 90, 615, 643–644
- System.out.print, 92, 616
- System.out.printf, 93–99, 414
- System.out.println, 38–39, 236, 616, 1163–1164
- screen output, 90–93
- T**
- <T>, omitting, 957
- tables, 1145, 1148
  - hash. *See* hash tables
  - truth, 153–154
- tail recursion, 712
- tails**, 895
- ternary operator, 144–145
- testing
  - bottom-up, 237
  - methods, 237–238
- text**
  - areas**, 1033–1046
    - scroll bars, 1039, 1072–1077
    - view port, 1072, 1073
  - fields**, 1040
  - files**, 615–616, 1033–1046
    - appending, 623–625
    - vs. binary files, 615–616
    - closing, 621–622
    - input, 121–124
    - opening, 618, 626
    - path names, 641–642
    - reading, 121–124, 625–628, 635–639
    - testing for end of, 628–630, 639–641
    - writing to, 615–623
  - this constructor, 476–477
  - this parameter, 229–231, 266–267, 1025
  - Thread class, 1127–1130
  - threads**, 1120–1121, 1126, 1130. *See also* multithreading
    - sockets and, 1142
  - Thread.sleep method, 1121–1122, 1124–1125
  - thread synchronization, 1133–1137
  - Throwable class, 593
  - throwing an exception**, 556, 596
    - @throws, 363
    - throws clause, 589–592, 594
    - in derived classes, 594–595
  - throw statements, 562–563, 569–570, 572
- tilde (~), 727
- time-space tradeoff, 909
- toArray method, 805, 940, 954
- tokens**, 275–277
- toLowerCase method, 71, 313
- toString method, 233–237, 330, 426, 493, 496, 529, 531
  - late binding with, 527–528
  - text file output and, 624
- toUpperCase method, 71, 313
- tracing variables, 182–183
- Transmission Control Protocol (TCP), 1138
- TreeMap<K, V> class, 960
- trees, 916–924
  - binary, 916–917
  - binary search, 919–924
  - empty, 918
  - properties, 916–919
- TreeSet<T> class, 955
- trim method, 71, 313
- trimToSize method, 805, 812, 954
- true, 48, 53, 55, 132, 152–155
- truth tables, 153–154
- try blocks**, 557, 559, 562–563, 569–570, 622
- try-catch-finally mechanism, 601–602
- try-catch mechanism, 557–559
  - nested, 601
- try-throw-catch mechanism, 569–570
- two-dimensional arrays, 432–434
- type casting, 53, 61–62, 223
- type coercion**, 62
- type inference, 798, 816
- type mismatch, 52–53
- type parameters**, 815–816, 820–821, 823–824
  - bound, 825–828, 866–867
  - bounds for, 825–828
  - collections, 956
  - multiple, 823–824
- U**
- UML. *See* Unified Modeling Language (UML)
- unboxing**, 310–312
- uncaught exceptions, 594, 1041

**unchecked exceptions**, 594, 634

**Unicode**, 53, 75–77, 654

**Unified Modeling Language (UML)**

class diagrams, 727–728

class hierarchy, 730

class interactions, 728

history of, 727

inheritance diagrams, 728–730

introduction to, 726–727

**uninitialized variables**, 50

UNIX, 358, 642

UnorderedPair class, 830–831

Unsupported Operation

Exception class, 945, 965

**upcasting**, 529–535

UPDATE statements, 1150, 1153, 1155–1157

useDelimiter method, 114, 634

User Datagram Protocol (UDP), 1138

UTF coding, 654–655

## V

v++ versus ++v, 63

validate method, 1098

valueOf method, 427, 429

values method, 427, 428–429, 959

**vararg specification**, 414, 415

**variables**, 47–48

of array type, 388

assignment compatibility, 52–53, 54

within blocks, 219–220

boolean, 154–155

CLASSPATH, 356–359, 360–361

of class type, 316–323, 327–328

declaring, 47, 220

environment, 356–357

global, 219

indexed, 379, 380, 389, 390, 432

initializing, 50–51, 269–274, 300

instance. *See* instance variables

int, 39

local, 50, 218–219, 225–227, 269

masks, 230

memory and, 317–320

naming conventions, 46

PATH, 354–355, 357

primitive types, 48, 316–318, 320

static, 50, 219, 300–304, 493

String, 65–66, 426

subscripted, 379

syntactic, 47

tracing, 182–183

uninitialized, 50

Vector<T> class, 950–956, 1268–1273

Vector class, 813–814

@version, 363

-version option, 365

vertical numbers, 685–688

**view ports**, 1072, 1073

visibility, components with changing, 1077–1080

void method, 212, 213, 214, 215, 216, 225, 704

recursive, 685–697

## W

web programming, with Java

Server Pages, 1158–1164

**while statements**, 164–167, 176

whitespace, 110, 113, 119, 276

whole numbers, 60

**wildcards**, 938

windowActivated method, 1059

WindowAdapter class, 1064–1065

windowClosed method, 1059

windowClosing method, 1059

windowDeiconified method, 1059

WindowEvent class, 1058

window events, 1058

windowIconified method, 1059

Windowing interface, 36–37

WindowListener interface, 1058–1059, 1273

**window listeners**, 1058–1065

dispose method, 1063

inner class, 1060

WindowAdapter class, 1064–1065

windowOpened method, 1059

**windows**

close-window button, 990, 992

disposed, 989

Swing, 985–986

words, 112

worst-case running time, 899–901

**wrapper classes**, 309–312, 356

no-argument constructors and, 315

static methods in, 312–316

writeBoolean method, 670

writeByte method, 670

writeChar method, 654, 670

writeDouble method, 651, 670

writeFloat method, 652, 670

writeInt method, 651, 653, 670

writeLong method, 651, 670

write method, 669

writeObject method, 652, 666

writeOutput method, 229, 236

writeShort method, 651, 670

writeUTF method, 652, 654–655, 670

## X

x-coordinate, 1081

-Xlint option, 817

## Y

y-coordinate, 1081